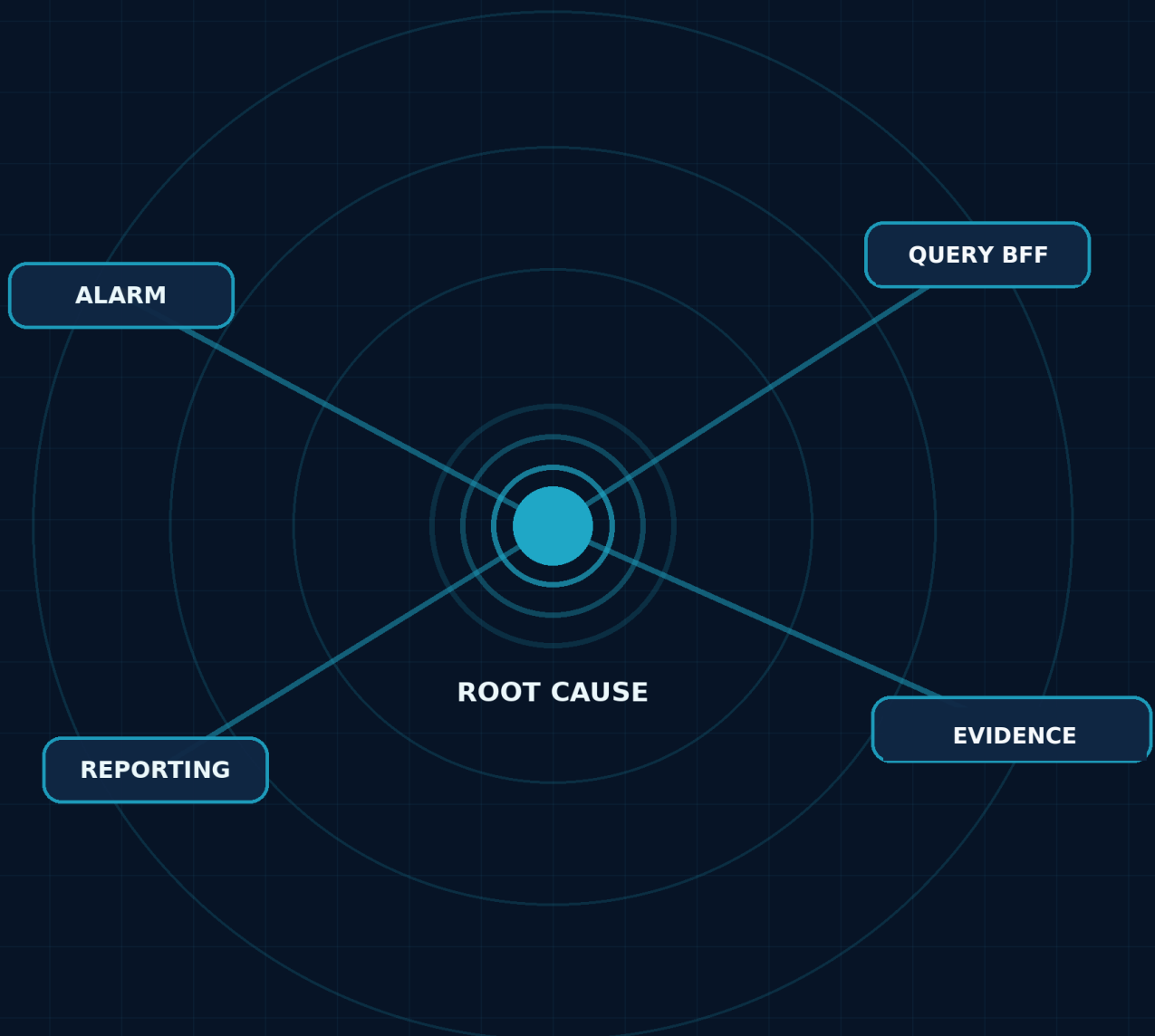


FROM SERVICES TO RUNTIME

An Architecture and Implementation Blueprint for NEXUS-1 in .NET



GRIGORIOS KYRIAKOS AGATHANGELIDIS

Educational Phase-0 distributed demonstrator

DEDICATION

*To the woman who devoted her life to me, I can only dedicate this
book.
To my wife, Vladimirka.*

GRIGORIOS AGATHANGELIDIS

FRONT MATTER

Standing Boundary and Copyright Note

STANDING BOUNDARY

This volume is a software architecture and learning companion. It is not safety-class software, not certified operational guidance, and not connected to any real facility.

From Services to Runtime - Definitive Final Edition. An Architecture and Implementation Blueprint for the NEXUS-1 Microservice Runtime in .NET. A NEXUS-1 Companion Implementation Blueprint. Copyright (c) 2026 Grigorios Kyriakos Agathangelidis. All rights reserved.

NEXUS-1 remains a Phase-0 educational demonstrator. The reference runtime is designed to teach boundaries, service-owned truth, public contracts, messaging, delayed consistency, security boundaries, observability, migration discipline, and failure recovery. It is never presented as software for operating a plant.

Every service boundary in this book is an implementation hypothesis that must earn evidence. Every benefit is paired with a runtime cost. Every distributed decision must explain what becomes slower, less certain, harder to coordinate, or harder to operate.

STANDING RULE

Nothing claims to exist that does not. A diagram is valid only when it matches the repository, contracts, data ownership, tests, and observable runtime behaviour.

The artifacts associated with this volume are intended to be published with the wider NEXUS-1 work at <https://github.com/gregory82gr/Nexus-1-phase-0>. The final edition must identify the exact artifact path, package versions, build identity, and reproduction command used by the manuscript.

SCOPE OF THIS DEFINITIVE FINAL EDITION

- Full-bleed cover and completed dedication page.
- Standing boundary, sixty-chapter contents, preface, reading guide, series position, declared reference slice and evidence rule.
- Parts I-X, Chapters 1-60, Epilogue, Appendices A-H, Glossary, Notation, Sources, Working Index, Final Consistency Audit, Final Reproduction Readiness, Final Publication Artifact Record and Final Page-Aware Index are present.
- The publication set includes the final PDF, authentic code-based editable source, 2100 x 3000 full-bleed cover PNG and external SHA-256 artifact manifest.
- Definitive book checkpoint: 2,075 pages. The reference application is NOT IMPLEMENTED; restore, build, tests, migrations and runtime reproduction are NOT EXECUTED; production readiness is NOT CLAIMED.

CURRENT MANUSCRIPT MAP

Contents

Sixty chapters across ten Parts - Chapters 1-48 present - Chapters 49-60 planned

PART I - CROSSING INTO DISTRIBUTION

01	The Handoff from From Flow to Services	p. 16
02	What Must Remain True After Extraction	p. 33
03	Choosing the First Distributed Slice	p. 49
04	Repository and Solution Topology	p. 65
05	Shared Nothing, Shared Little	p. 87
06	Definition of Done for a NEXUS-1 Service	p. 108

PART II - THE SERVICE FOUNDATION

07	The Golden-Path Service Template	p. 128
08	Host Composition and Dependency Direction	p. 152
09	Domain, Application, Infrastructure, and Edge Inside a Service	p. 175
10	Configuration and Environment Boundaries	p. 199
11	Liveness, Readiness, and Startup Dependencies	p. 221
12	The Testing Pyramid for a Distributed Service	p. 243

CURRENT MANUSCRIPT MAP

Contents continued

PART III - DATA OWNERSHIP IN EXECUTABLE FORM

13	From Seventeen Schemas to Service-Owned Databases	p. 266
14	Building AlarmManagementDb	p. 295
15	Building RootCauseDb	p. 318
16	Independent Migrations and Database Lifecycle	p. 341
17	Cross-Service Identity Without Cross-Database Foreign Keys	p. 363
18	Local Transactions, Concurrency, and Commit Boundaries	p. 387

PART IV - PUBLIC CONTRACTS AND THE SYNCHRONOUS EDGE

19	Domain Events, Integration Events, and API Contracts	p. 416
20	The Versioned Message Envelope	p. 442
21	Commands and Queries at a Service Boundary	p. 464
22	The Operator Console BFF	p. 486
23	Versioning, Problem Details, and Idempotency Keys	p. 509
24	Contract Compatibility Tests	p. 532

CURRENT MANUSCRIPT MAP

Contents continued**PART V - THE MESSAGE BACKBONE**

25	RabbitMQ Topology for NEXUS-1	p. 555
26	The Transactional Outbox in Each Producer	p. 576
27	Dispatch, Publisher Confirms, and Recovery	p. 594
28	Inbox and Idempotent Consumers	p. 619
29	Retry, Backoff, Dead Letters, and Poison Messages	p. 645
30	Ordering, Duplication, and Contract Evolution	p. 680

PART VI - THE FIRST END-TO-END DISTRIBUTED SLICE

31	AlarmManagement Enters the Runtime	p. 712
32	Publishing AlarmFloodDetected.v1	p. 744
33	RootCause Consumes and Opens a Case	p. 778
34	A Verdict Fans Out to Audit and Compliance	p. 814
35	Reporting Builds Delayed Truth	p. 850
36	The Complete Slice and the Optional RL Advisory Branch	p. 894

CURRENT MANUSCRIPT MAP

Contents continued

PART VII - WORKFLOW, CONSISTENCY, AND REPAIR

37	The NEXUS-1 Distributed Process Manager	p. 924
38	Choreography, Orchestration, and the Decision Record	p. 956
39	Timeouts, Escalations, and Human Review	p. 985
40	Compensation Without Pretending to Undo History	p. 1016
41	Eventual Consistency in the Operator Experience	p. 1047
42	Reconciliation and Repair Jobs	p. 1082

PART VIII - SECURITY AND TRUST ACROSS SERVICES

43	Threat and Trust Boundaries Before Mechanisms	p. 1132
44	Edge Authentication and Operator Authorization	p. 1182
45	Service-to-Service Identity	p. 1231
46	Claims Minimization and Policy Mapping	p. 1280
47	Secrets, Certificates, and Configuration Handling	p. 1334
48	Security Tests and Audit Evidence	p. 1375

CURRENT MANUSCRIPT MAP**Contents continued****PART IX - OBSERVABILITY, RESILIENCE, AND FAILURE EVIDENCE**

49	Correlation and Causation Across Every Boundary	p. 1417
50	Structured Logs That Preserve Domain Meaning	p. 1457
51	Distributed Tracing with OpenTelemetry	p. 1489
52	Metrics, Service Objectives, and Dashboards	p. 1518
53	Timeout, Retry, Circuit Breaker, and Bulkhead	p. 1549
54	Failure Injection and Recovery Evidence	p. 1578

PART X - MIGRATION, DELIVERY, AND FINAL ARCHITECTURE

55	The Reproducible Docker Compose Runtime	p. 1608
56	Extracting Without a Big-Bang Rewrite	p. 1637
57	CI Quality Gates and Container Images	p. 1669
58	Rollout, Database Change, and Rollback	p. 1698
59	Translating Proven Needs to Kubernetes	p. 1725
60	The Final NEXUS-1 Distributed Demonstrator	p. 1751

BACK MATTER - COMPLETE

Epilogue - What the Runtime Has Actually Proved	p. 1774
Appendix A - Solution and Project Map	p. 1786
Appendix B - Service Catalogue	p. 1806
Appendix C - API and Event Contract Catalogue	p. 1832
Appendix D - Configuration Reference	p. 1860
Appendix E - Failure Scenario and Recovery Catalogue	p. 1888
Appendix F - Security and Trust Boundary Notes	p. 1914
Appendix G - Commands, Scripts, and Reproduction Guide	p. 1942
Glossary - Microservice and Distributed-Systems Terms	p. 1968
Mathematical and Timing Notation Used in the Book	p. 1986
Sources and Reference Notes	p. 1999
Working Index	p. 2010
Final Consistency Audit	p. 2018
Final Reproduction Readiness and Deferred Evidence Checkpoint	p. 2037
Final Publication Artifact and Build Identity	p. 2045
Final Page-Aware Index	p. 2054
	p. 2057

PREFACE

Architecture Has Reached the Runtime

The previous NEXUS-1 books made the system progressively more explicit. This volume asks whether that clarity survives when certainty is no longer local.

A modular backend can call a method and remain inside one process. It can update several tables inside one transaction. It can step through a failure with one debugger and read one ordered log. Those advantages are not signs of immaturity. They are forms of local certainty.

A microservice boundary spends some of that certainty. A method call becomes a remote interaction or a message. One transaction becomes several local commits. One current screen becomes a projection that may be delayed. One test suite becomes a family of unit, integration, contract, end-to-end, and recovery tests. One log becomes a trace whose context must survive every boundary.

From Flow to Services treated that cost seriously. It did not promise that every named NEXUS-1 sector should become a service. It classified RootCause, AlarmManagement, Compliance, Reporting, and Audit as the strongest candidates, while keeping DigitalTwin, Instrumentation, ReactorFleet, and ReinforcementLearning Advisory conditional. It ended before implementation because architecture should not pretend to have produced runtime evidence.

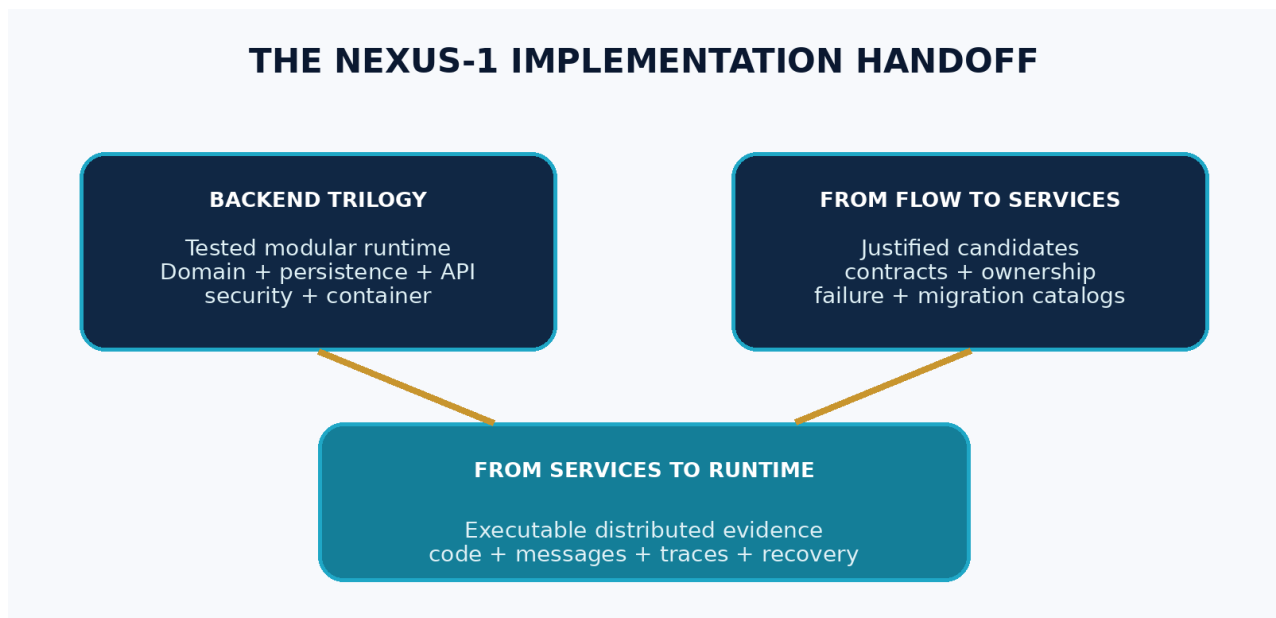


Figure 0.1 - The two inherited bodies of evidence converge in this implementation volume.

From Services to Runtime begins where that honesty left the system. The target is not a service explosion. The target is one small, coherent distributed slice that can be built, observed, interrupted, recovered, and explained. AlarmManagement detects a flood. RootCause opens and closes a case. Audit preserves evidence. Compliance owns review. Reporting builds delayed truth. Each service owns only the truth it is allowed to decide.

POSITIONING SENTENCE

This book converts architectural readiness into executable evidence, one boundary and one failure mode at a time.

The reader should therefore expect code, but not code without argument. Tools appear after the responsibility they support is clear. RabbitMQ does not choose the event. Docker does not choose the service boundary. OpenTelemetry does not create causality; it preserves evidence of causality already designed into the flow. Kubernetes does not make a service independently operable; it exposes whether that operational ownership w

The result remains educational Phase-0 software. Its value lies in disciplined demonstration: a controlled environment where a developer can see exactly what distribution changes, what patterns protect the system, what failure still remains possible, and which candidates should continue to live as modules.

CURRENT MANUSCRIPT STATUS

Chapters 1-48	Manuscript content present and structurally reviewed.
Runtime evidence	PENDING unless the chapter names exact retained execution artifacts.
Chapters 49-60	Planned contents only; no prose or implementation is claimed yet.
Reference code	Compile-oriented; it is not repository execution evidence.

READING GUIDE

How to Read This Book

Read the volume as a proof sequence. Each Part must leave behind evidence that the next Part is allowed to trust.

Reader	Recommended path	What to watch
C#/.NET developer	Read in order and run the reference build at each Part checkpoint.	Dependency direction, database ownership, message semantics, and tests.
Software architect	Focus on Parts I, III, IV, V, VII, IX, and X.	Whether diagrams and catalogues remain true after execution and failure.
Engineer new to messaging	Use the practical doorway at the start of each new mechanism.	Acknowledgements, duplicates, ordering, delayed truth, and partial progress.
Reader of the full series	Treat earlier volumes as inherited ledgers, not material to relearn.	Exactly where the network changes a previously local guarantee.

STABLE CHAPTER RHYTHM

1. The ordinary developer problem.
2. The naive solution and why it fails.
3. A plain-language principle.
4. The NEXUS-1 application.
5. A compile-checked implementation shape where it helps.
6. A failure or boundary test.
7. What has now been proved - and what has not.
8. Checkpoint questions before the reader continues.

WRITING RULE

Never treat a tool as the architecture. A tool enters only after the service responsibility, contract, failure behaviour, and evidence need are named.

SERIES POSITION

What This Volume Inherits

The book does not begin from a greenfield example. It inherits a domain, a data backbone, a tested backend, and an architectural decision record.

Earlier volume	Inheritance carried into this book
From Domain to Twin	Entities, value objects, aggregates, invariants, bounded contexts, and language ownership.
From Schema to System	The seventeen-sector data backbone and the requirement that ownership be visible.
From Entity to Context	One mapping discipline per context and explicit EF Core boundaries.
From Context to Flow	Published language, integration events, outbox/inbox, idempotency, process managers, compensation, and eventual consistency.
Backend Trilogy	Invariant-first domain/application code, persistence, public API, Phase-0 security, tests, configuration, observability, and a containerized modular runtime.
From Flow to Services	Candidate classification, seven readiness gates, data and contract catalogues, failure modes, migration roadmap, and final pre-implementation checklist.

BOUNDARY OF INHERITANCE

Earlier books can prove that a boundary is meaningful. Only this implementation volume can prove how that boundary behaves when the network, independent persistence, and partial failure are real.

REFERENCE SCENARIO

The Executable Slice

The book follows one recurring path so that every new mechanism can be evaluated against the same business meaning.

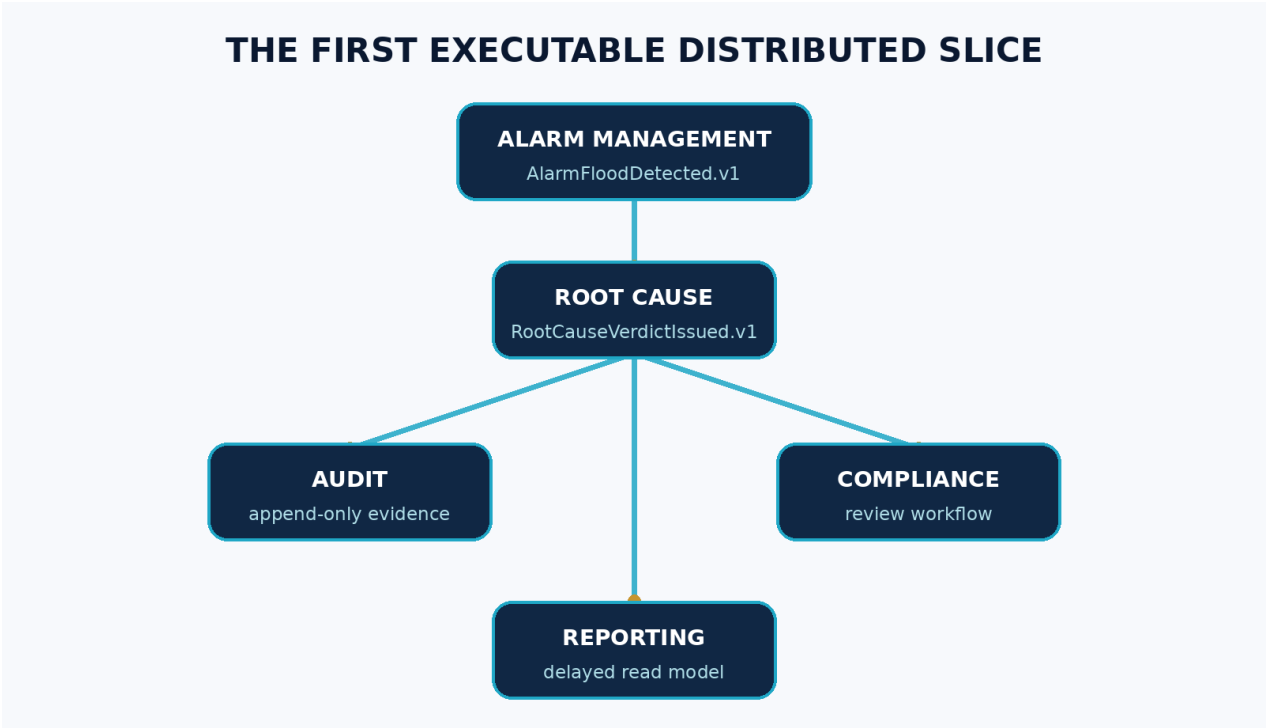


Figure 0.2 - The first distributed slice keeps five strong candidates and one visible direction of truth.

AlarmManagement owns alarms and flood detection. It publishes AlarmFloodDetected.v1. RootCause owns the case, evidence rules, hypotheses, and verdict. It publishes RootCauseVerdictIssued.v1. Audit appends evidence without becoming a workflow engine. Compliance owns the review decision. Reporting owns projections and freshness metadata, never the source truth.

The Optional ReinforcementLearning Advisory branch may observe a verdict and publish a recommendation. It does not issue commands and it does not become mandatory for the first slice. Its conditional status is preserved until separate deployment and failure ownership are justified.

RUNTIME RULE

A service may be temporarily unavailable without erasing a truth already committed by another service. Recovery must converge through explicit contracts and durable evidence.

METHOD

The Evidence Rule

A chapter may explain a mechanism before the code exists. It may not present the mechanism as completed until the corresponding evidence has been produced.

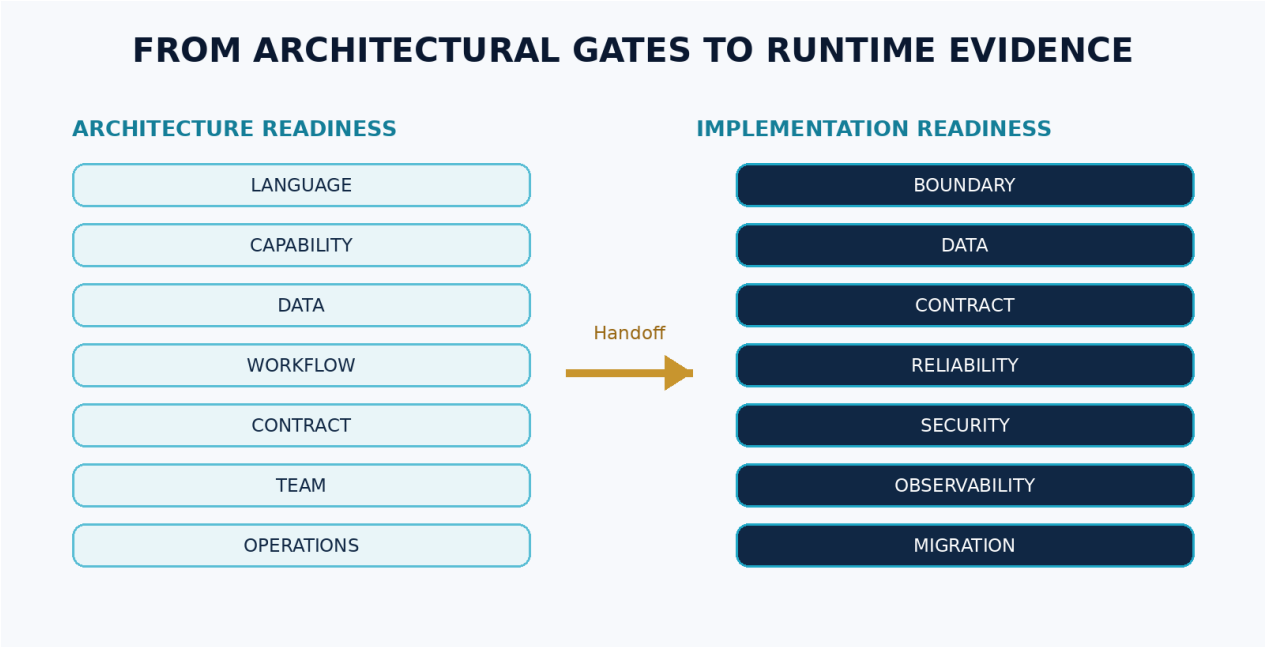


Figure 0.3 - Architecture asks whether the boundary deserves extraction; implementation asks whether the extraction is responsible.

Mark	Meaning	Finalization requirement
BASE	Primarily inherits an established NEXUS-1 decision.	Verify vocabulary and handoff; do not duplicate earlier prose.
EXTEND	An existing artifact must change for distribution.	Show the before/after boundary and keep the old guarantee visible.
NEW	New code, schema, contract, diagram, or runbook is required.	Compile or structurally validate the artifact.
VERIFY	Runtime experiment, failure injection, measurement, or official-source check is required.	Record expected state, observed state, and conclusion.

NO-CLAIM RULE

When evidence is missing, narrow the claim. Do not fill a proof gap with confident prose.

PART I

PART I

Crossing into Distribution

*The boundary is not crossed when a new project is created.
It is crossed when ownership can survive execution.*

Part I turns the final catalogues of From Flow to Services into an implementation contract. It establishes the handoff, identifies what must remain true, chooses the first slice, creates the repository topology, limits shared code, and defines the evidence required before a NEXUS-1 service is called complete.

WHAT PART I GIVES YOU

A preservation ledger, a first-slice decision, a buildable solution shape, explicit sharing rules, and a service definition of done that later Parts can test.

CHAPTERS 1-6

- The Handoff from From Flow to Services
- What Must Remain True After Extraction
- Choosing the First Distributed Slice
- Repository and Solution Topology
- Shared Nothing, Shared Little
- Definition of Done for a NEXUS-1 Service

PART I - CROSSING INTO DISTRIBUTION

1. The Handoff from From Flow to Services

The previous book ended with a gate. This chapter does not open it by creating five Web APIs. It first records exactly what the architecture has decided - and exactly what the runtime still has to prove.

CHAPTER ARGUMENT

Architecture becomes implementation only when inherited decisions are converted into observable proof obligations.

From Flow to Services closed honestly. It named the strongest candidates, public contracts, ownership boundaries, expected failures, security needs, observability needs, and migration conditions. It did not claim that the services were built. That distinction is not editorial modesty. It is the first reliability rule of this volume.

A system can be architecturally ready to discuss a service and still be technically unready to deploy one. RootCause can own a clear language and a meaningful verdict while its public event schema remains untested. AlarmManagement can own flood detection while its data still sits inside a database that every module can write. Audit can be a distinct evidence responsibility while no operator can yet see a poison message or replay a failed delivery.

The handoff therefore has two sides. The left side contains decisions. The right side must contain evidence. The job of this book is to make the crossing without silently changing the language that made the boundary credible in the first place.

The previous volume closed before implementation

Chapter 72 of From Flow to Services organized the final decision around seven implementation-readiness gates: boundary, data, contract, reliability, security, observability, and migration. A weak gate was not hidden as future work. It either stopped extraction or forced the migration to remain supervised and reversible.

Final gate	Question inherited by this volume	Evidence this book must produce
Boundary	Does the candidate own a real responsibility and language?	Preservation ledger, capability statement, decision owner, and first-slice ADR.
Data	Can the service own truth without shared-table escape routes?	Service-owned database, migrations, permissions, identifiers, and no cross-service writes.
Contract	Can others use the service without knowing its internals?	Versioned APIs/events, compatibility tests, and producer-owned schemas.
Reliability	What happens under duplication, delay, outage, or poison data?	Outbox/inbox, retry/DLQ rules, failure tests, and recovery evidence.
Security	Which user and service identities are trusted?	Local policies, least privilege, secret handling, and negative tests.
Observability	Can a failed case be explained and operated?	Logs, metrics, traces, health checks, dashboards, correlation, and runbooks.
Migration	Can extraction be introduced and reversed safely?	Strangler route, coexistence, contract gates, rollback, and release evidence.

FINAL RULE INHERITED FROM CHAPTER 72

Do not distribute a boundary until ownership. truth. failure. security. operation. and

migration are visible.

Two sets of seven gates - two different jobs

Earlier in From Flow to Services, the service-boundary checklist used Language, Capability, Data, Workflow, Contract, Team, and Operations. Those gates asked whether the boundary deserved extraction. The final checklist used Boundary, Data, Contract, Reliability, Security, Observability, and Migration. Those gates asked whether extraction could be performed responsibly.

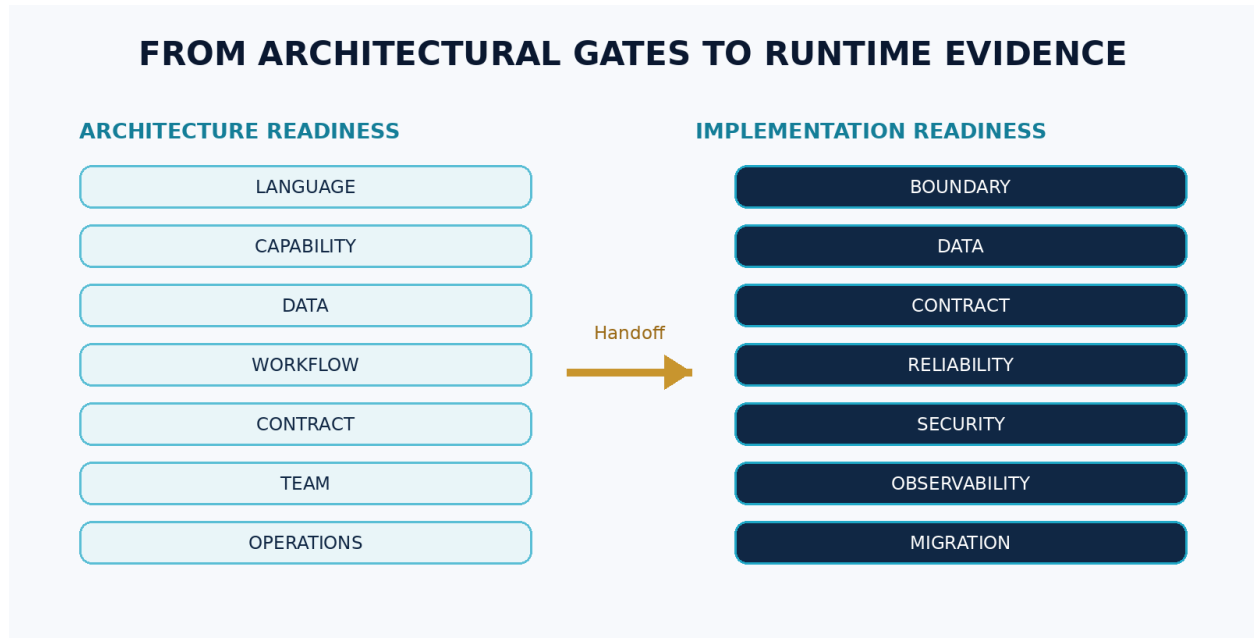


Figure 1.1 - The architectural gates are not replaced. They are carried into a stricter implementation gate.

The two lists should not be collapsed carelessly. Language and capability do not disappear; they become the heart of boundary readiness. Workflow does not disappear; it becomes contract and reliability evidence. Team and operations do not disappear; they become observability, migration, deployment, runbook, and incident ownership.

Architecture gate	Implementation destination	Reason
Language + Capability	Boundary readiness	A runtime boundary is justified by owned meaning and decisions, not a folder or controller.
Data	Data readiness	The source of truth must become enforceably local.
Workflow	Contract + Reliability readiness	Named flow becomes public messages, timeouts, partial progress, and recovery.
Contract	Contract readiness	Public language must be versioned and tested outside private implementation.
Team + Operations	Security + Observability + Migration readiness	Independent deployment creates independent operational responsibility.

The six inherited ledgers

The handoff is not one diagram. It is a small set of ledgers that constrain every later implementation choice. If the code contradicts a ledger, the implementation must either change or record a new architectural decision. Silence is not a valid migration strategy.

Inherited ledger	What it already says	What remains unproved
Candidate map	Which contexts are strong, conditional, special, read-side, or protected.	Whether the strongest candidates can run and fail independently.
Service catalogue	Responsibility, language, owner, public surface, and dependencies.	Whether each runtime can keep those responsibilities local.
API/event catalogue	Canonical public names and intended consumers.	Serialization, compatibility, delivery, replay, and version evolution.
Data-ownership catalogue	Which context owns source truth and which data is projection or reference.	Physical separation, permissions, migrations, concurrency, and forbidden writes.
Failure catalogue	Expected timeouts, duplicates, delays, poison messages, and incomplete workflows.	Whether the system exposes, contains, and recovers from those failures.
Pre-implementation checklist	The seven final gates and required evidence categories.	The evidence itself.

HANDOFF DISCIPLINE

A catalogue is a constraint on implementation, not a decorative appendix. Every later Part must update the ledgers with evidence or a reasoned exception.

What is true at the start of this book

NEXUS-1 does not enter this volume as an unstructured application. It already has domain language, bounded contexts, schema discipline, EF Core mappings, application flow, APIs, tests, configuration, logging, and a containerized modular backend. The service discussion is therefore grounded in an existing engineering narrative.

- RootCause owns cases, evidence, hypotheses, verdicts, and the meaning of a root-cause decision.
- AlarmManagement owns alarms and flood detection; it does not lend private alarm tables to RootCause.
- Audit owns append-only evidence and must not become a hidden workflow engine.
- Compliance owns review and compliance decisions; Reporting may display them but may not decide them.
- Reporting owns projections and freshness metadata, never operational source truth.
- The operator console needs composition, but the BFF must not become a new domain owner.
- DigitalTwin, Instrumentation, ReactorFleet, and RL Advisory remain conditional or protected until evidence improves.

What is not yet true

The architecture book deliberately stopped before runtime. At this point in the implementation story, the reader must assume that the following claims are still false or unproved:

- There are not yet five independently deployable services.
- There is not yet a broker topology whose delivery and recovery behaviour has been demonstrated.
- Service-owned databases and independent migration histories have not yet been proved.
- Outbox, inbox, duplicate handling, ordering rules, and poison-message recovery have not yet been exercised across real process boundaries.
- User identity, service identity, least privilege, and secret rotation have not yet been demonstrated across the distributed runtime.
- A complete alarm-to-reporting case cannot yet be followed through logs, metrics, traces, and durable state.

- The strangler route, coexistence plan, release gates, and rollback behaviour remain architectural commitments, not runtime evidence.

HONEST STARTING STATE

The service map is ready to be tested. The services themselves are not yet claimed as built.

The first implementation posture

The final candidate table of From Flow to Services gives this volume a conservative starting posture. RootCause is the strongest candidate because it owns a decision-rich language, a clear source of truth, and a meaningful public fact. AlarmManagement is strong but dependency-heavy. Compliance is a good workflow candidate. Reporting is a read-side candidate. Audit is a special append-only candidate. The BFF is a useful edge service, not a domain service.

Candidate	Inherited posture	Implementation obligation
RootCause	Strong	Preserve case/verdict invariants; own data; publish stable verdict facts; tolerate subscriber outage.
AlarmManagement	Strong, dependency-heavy	Protect alarm truth; publish flood facts; avoid leaking instrumentation or alarm tables.
Audit	Special	Append evidence idempotently; never command the business workflow.
Compliance	Good	Own review state and human escalation; consume evidence without rewriting it.
Reporting	Read-side	Own projections, watermarks, and freshness; never present delayed truth as current truth.
RL Advisory	Conditional	Remain advisory; recommendations are not commands; join only as an optional branch.

The recurring flow becomes the test surface

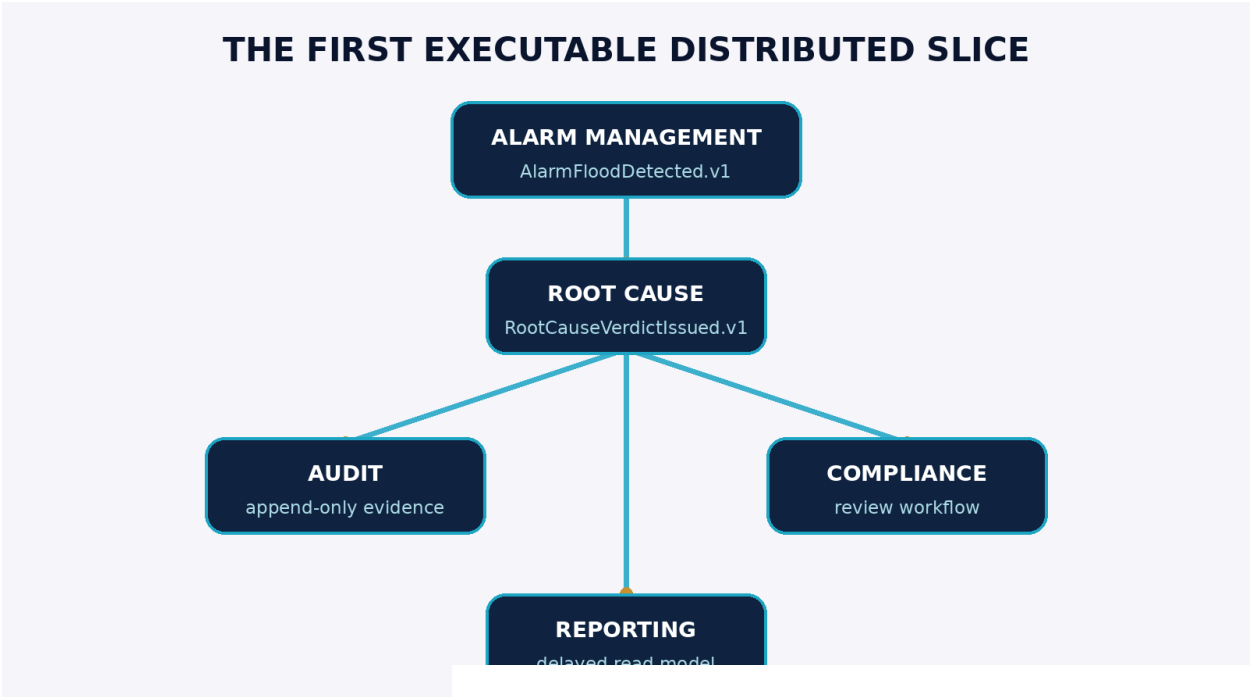


Figure 1.2 - The same flow will be used to test ownership, delivery, delayed truth, and recovery.

The scenario is intentionally narrow. A narrow slice makes it possible to see the consequences of each boundary. AlarmManagement commits a local truth and records an event in its outbox. RootCause consumes that fact, opens a local case, and later publishes a verdict. Audit and Compliance may receive the verdict at different times. Reporting may remain behind both. None of those delays is allowed to rewrite RootCause history.

This flow also prevents the book from becoming a set of unrelated samples. The message envelope, outbox, inbox, process manager, security context, trace propagation, metrics, failure experiments, migration path, and rollback rules will all be tested against the same case. A pattern is admitted because it protects this system, not because it is fashionable.

The handoff ledger

Before Chapter 2 asks what must remain true after extraction, Chapter 1 closes with a ledger that later Parts can update. The status values are intentionally strict: Inherited means the architectural decision exists; Unproved means runtime evidence does not.

Obligation	Start status	Evidence owner
Boundary and language preserved	Inherited / unproved in separate runtime	Part I
Independent source-of-truth data	Mapped / unproved	Part III
Stable public HTTP and event contracts	Catalogued / unproved	Part IV
Durable messaging and idempotent consumption	Designed / unproved	Part V
End-to-end distributed business flow	Modelled / unproved	Parts VI-VII
User and service trust	Required / unproved	Part VIII
Logs, metrics, traces, health, and failure recovery	Required / unproved	Part IX
Reproducible deployment, migration, and rollback	Planned / unproved	Part X

The handoff becomes an executable contract

The ledgers above should not remain six disconnected documents that reviewers interpret differently. A small typed model can preserve the distinction between an inherited architectural decision, a runtime proof obligation, and the evidence that eventually satisfies it. This is not the service implementation. It is the executable shape of the handoff.

CODE-READING RULE

Inherited is not Proved. A decision may exist without runtime evidence; an obligation may be specified without being satisfied; evidence must name an immutable artifact and the environment in which it was produced.

IMPLEMENTATION SHAPE 1-A — HANDOFF TYPES

```
public enum ProofStatus
{
    Inherited,
    Specified,
    Demonstrated,
    Rejected
}

public sealed record ArchitectureDecision(
    string DecisionId,
    string Ledger,
    string Statement,
    string Owner);

public sealed record ProofObligation(
    string ObligationId,
    string DecisionId,
    string Claim,
    ProofStatus Status,
    IReadOnlyList<EvidenceRef> Evidence);

public sealed record EvidenceRef(
    string EvidenceId,
    Uri Artifact,
    string Sha256,
    string EnvironmentId,
    DateTimeOffset ObservedAtUtc);
```

The model avoids a Boolean `IsDone` because the useful states are not binary. `Inherited` means the architecture already made the decision. `Specified` means the runtime experiment has a defined shape. `Demonstrated` means retained evidence supports the claim in a named environment. `Rejected` means the experiment contradicted the claim or the evidence was invalid.

Representing the first slice explicitly

The recurring `AlarmManagement` to `RootCause` flow can now be represented as data consumed by validation, documentation, release gates, and later test fixtures. Notice that the descriptor names ownership and expected evidence; it does not pretend that a broker or database exists.

IMPLEMENTATION SHAPE 1-B — FIRST-SLICE DESCRIPTOR

```
public sealed record ServiceCandidate(
    string Name,
    string Owns,
    string PublicFact,
    string SourceOfTruth,
    string[] MustNotAccess);

public static class FirstDistributedSlice
{
    public static readonly ServiceCandidate AlarmManagement = new(
        Name: "AlarmManagement",
        Owns: "alarm lifecycle and flood detection",
        PublicFact: "AlarmFloodDetectedV1",
        SourceOfTruth: "AlarmManagementDb",
        MustNotAccess: ["RootCauseDb"]);

    public static readonly ServiceCandidate RootCause = new(
        Name: "RootCause",
        Owns: "cases, evidence, hypotheses and verdicts",
        PublicFact: "RootCauseVerdictIssuedV1",
        SourceOfTruth: "RootCauseDb",
        MustNotAccess: ["AlarmManagementDb"]);

    public static readonly ProofObligation[] Obligations =
    [
        new(
            "DATA-001",
            "OWN-ALARM-001",
            "AlarmManagement commits alarm truth without writing RootCauseDb.",
            ProofStatus.Specified,
            []),
        new(
            "MSG-001",
            "CONTRACT-FLOOD-001",
            "A committed AlarmFloodDetectedV1 survives broker unavailability.",
            ProofStatus.Specified,
            []),
        new(
            "IDEMP-001",
            "CONSUMER-RC-001",
            "Repeated delivery creates one RootCause business effect.",
            ProofStatus.Specified,
            [])
    ];
}
```

The gate must reject evidence-free success

The simplest useful automation is a validator that refuses an unsupported success claim. This small rule prevents dashboards, generated documentation, or release scripts from turning “specified” into “proved” merely because an enum value changed.

IMPLEMENTATION SHAPE 1-C — PROOF-STATUS VALIDATION

```
public static class HandoffGate
{
    public static void Validate(ProofObligation obligation)
    {
        if (obligation.Status is ProofStatus.Demonstrated &&
            obligation.Evidence.Count == 0)
        {
            throw new InvalidOperationException(
                $"{obligation.ObligationId} is Demonstrated " +
                "but contains no retained evidence.");
        }

        foreach (var evidence in obligation.Evidence)
        {
            if (string.IsNullOrEmpty(evidence.Sha256) ||
                string.IsNullOrEmpty(evidence.EnvironmentId))
            {
                throw new InvalidOperationException(
                    $"{evidence.EvidenceId} is not reproducibly identified.");
            }
        }
    }
}
```

CHAPTER 1 - CODE RE-AUDIT

The handoff becomes an identified input, not inherited prose

Chapter 1 already names the candidate services and proof states. This audit adds the missing identity boundary: every inherited statement must point to a versioned source, owner, requirement set and immutable digest.

EVIDENCE STATUS

No repository, SDK, package graph, host, database, broker or deployment run has executed here. The following assets are compile-oriented; all HND evidence remains PENDING.

EXECUTABLE ASSET 1-E - IDENTIFIED HANDOFF MANIFEST

```
public enum HandoffClaimStatus
{
    Inherited,
    Specified,
    Evidenced,
    Rejected
}

public sealed record HandoffSource(
    string SourceId,
    string ArtifactName,
    string ArtifactVersion,
    string Sha256);

public sealed record HandoffRequirement(
    string RequirementId,
    string Owner,
    string EvidenceType,
    bool Blocking);

public sealed record HandoffClaim(
    string ClaimId,
    string Owner,
    HandoffClaimStatus Status,
    IReadOnlyList<string> SourceIds,
    IReadOnlyList<string> RequirementIds);

public sealed record HandoffManifest(
    string ManifestProfile,
    IReadOnlyList<HandoffSource> Sources,
    IReadOnlyList<HandoffRequirement> Requirements,
    IReadOnlyList<HandoffClaim> Claims,
    string ManifestSha256);
```

CANONICAL HANDOFF IDENTITY

Formatting cannot change the architectural baseline

The identity is computed from typed, ordinally sorted fields. JSON property order, indentation and operating-system line endings never define architectural equivalence.

EXECUTABLE ASSET 1-F - TYPED CANONICAL DIGEST

```
public static class HandoffManifestIdentity
{
    public const string Profile =
        "nexus.handoff.v1/len32-utf8+sha256";

    public static string Compute(HandoffManifest manifest)
    {
        using var hash = IncrementalHash.CreateHash(
            HashAlgorithmName.SHA256);
        using var writer = new CanonicalHashWriter(hash);

        writer.WriteUtf8(Profile);
        foreach (var source in manifest.Sources
            .OrderBy(x => x.SourceId, StringComparer.Ordinal))
        {
            writer.WriteUtf8(source.SourceId);
            writer.WriteUtf8(source.ArtifactName);
            writer.WriteUtf8(source.ArtifactVersion);
            writer.WriteHex256(source.Sha256);
        }

        foreach (var requirement in manifest.Requirements
            .OrderBy(x => x.RequirementId,
                StringComparer.Ordinal))
        {
            writer.WriteUtf8(requirement.RequirementId);
            writer.WriteUtf8(requirement.Owner);
            writer.WriteUtf8(requirement.EvidenceType);
            writer.WriteBoolean(requirement.Blocking);
        }

        foreach (var claim in manifest.Claims
            .OrderBy(x => x.ClaimId, StringComparer.Ordinal))
            WriteClaim(writer, claim);

        return Convert.ToHexString(hash.GetHashAndReset())
            .ToLowerInvariant();
    }
}
```

EXECUTABLE ASSET 1-G - DIGEST DETERMINISM TEST

```
public sealed class HandoffIdentityTests
{
    [Fact]
    public void HND_ID_001_order_and_format_do_not_change_identity()
    {
        var first = HandoffFixture.Manifest();
        var reordered = first with
        {
            Sources = first.Sources.Reverse().ToArray(),
            Claims = first.Claims.Reverse().ToArray()
        };

        HandoffManifestIdentity.Compute(reordered)
            .Should().Be(
                HandoffManifestIdentity.Compute(first));
    }
}
```

CLOSED CLAIM CATALOGUE

Every inherited statement has one owner and one blocking proof

The catalogue is deliberately small. It freezes the first distributed slice without pretending that the slice has already run.

EXECUTABLE ASSET 1-H - REQUIRED HANDOFF CLAIMS

```
public static class RequiredHandoffClaims
{
    public static readonly IReadOnlyDictionary<string, string>
        OwnerByClaim =
            new Dictionary<string, string>(StringComparer.Ordinal)
            {
                ["OWN-ALARM-001"] = "AlarmManagement",
                ["CONTRACT-FLOOD-001"] = "AlarmManagement",
                ["CONSUMER-RC-001"] = "RootCause",
                ["HND-REV-001"] = "MigrationAuthority",
                ["HND-UX-001"] = "OperatorConsole"
            };

    public static void Validate(HandoffManifest manifest)
    {
        var byId = manifest.Claims.ToDictionary(
            x => x.ClaimId, StringComparer.Ordinal);
        byId.Keys.Should().BeEquivalentTo(
            OwnerByClaim.Keys);

        foreach (var expected in OwnerByClaim)
        {
            var claim = byId[expected.Key];
            claim.Owner.Should().Be(expected.Value);
            claim.SourceIds.Should().NotBeEmpty();
            claim.RequirementIds.Should().NotBeEmpty();
        }
    }
}
```

EXECUTABLE ASSET 1-I - NO ORPHANED REQUIREMENT

```
public sealed class HandoffCatalogueTests
{
    [Fact]
    public void HND_CAT_001_every_blocker_is_referenced()
    {
        var manifest = HandoffFixture.Manifest();
        var used = manifest.Claims
            .SelectMany(x => x.RequirementIds)
            .ToHashSet(StringComparer.Ordinal);

        manifest.Requirements
            .Where(x => x.Blocking)
            .Should().OnlyContain(
                x => used.Contains(x.RequirementId));
    }

    [Fact]
    public void HND_CAT_002_unknown_source_is_rejected()
    {
        var manifest = HandoffFixture
            .WithUnknownSource("missing-book");
        var act = () => HandoffValidator.Validate(manifest);
        act.Should().Throw<UnknownHandoffSource>();
    }
}
```

EVIDENCE-BOUND TRANSITIONS

A claim cannot promote itself from prose to runtime fact

Only a retained execution record may move a claim from Specified to Evidenced. The record must identify the suite, environment and exact manifest it tested.

EXECUTABLE ASSET 1-J - PROMOTION STATE MACHINE

```
public enum HandoffEvidenceStatus
{
    Pending,
    Passed,
    Failed
}

public sealed record HandoffEvidence(
    string EvidenceId,
    string RequirementId,
    HandoffEvidenceStatus Status,
    int Discovered,
    int Executed,
    int Skipped,
    string ManifestSha256,
    string EnvironmentSha256,
    string ResultSha256);

public static class HandoffPromotion
{
    public static HandoffClaim Promote(
        HandoffClaim claim,
        IReadOnlyCollection<HandoffEvidence> evidence,
        string manifestSha256,
        string environmentSha256)
    {
        var required = claim.RequirementIds
            .Select(id => evidence.SingleOrDefault(
                x => x.RequirementId == id)
                ?? throw new MissingHandoffEvidence(id))
            .ToArray();

        if (required.Any(x =>
            x.Status != HandoffEvidenceStatus.Passed
            || x.Discovered <= 0
            || x.Executed != x.Discovered
            || x.Skipped != 0
            || x.ManifestSha256 != manifestSha256
            || x.EnvironmentSha256 != environmentSha256))
            throw new UnacceptedHandoffEvidence();

        return claim with
        {
            Status = HandoffClaimStatus.Evidenced
        };
    }
}
```

NO SELF-ATTESTATION

A document edit may add requirements or change Specified meaning. It cannot set Passed, invent discovered tests or reuse evidence from another manifest digest.

FIRST-SLICE AUTHORITY GRAPH

Message flow does not transfer decision authority

The graph freezes who emits, consumes and decides. Audit and Reporting receive facts; neither gains authority to issue or correct a RootCause verdict.

EXECUTABLE ASSET 1-K - TYPED FIRST-SLICE GRAPH

```
public enum FlowCapability { Emit, Consume, Decide, Project }

public sealed record ServiceCapability(
    string Service,
    FlowCapability Capability,
    string FactOrDecision);

public static class FirstSliceAuthority
{
    public static readonly ServiceCapability[] Expected =
    [
        new("AlarmManagement", FlowCapability.Emit,
            "AlarmFloodDetected.v1"),
        new("RootCause", FlowCapability.Consume,
            "AlarmFloodDetected.v1"),
        new("RootCause", FlowCapability.Decide,
            "RootCauseVerdict"),
        new("RootCause", FlowCapability.Emit,
            "RootCauseVerdictIssued.v1"),
        new("Audit", FlowCapability.Consume,
            "RootCauseVerdictIssued.v1"),
        new("Compliance", FlowCapability.Consume,
            "RootCauseVerdictIssued.v1"),
        new("Reporting", FlowCapability.Project,
            "RootCauseVerdictIssued.v1")
    ];

    public static void Validate(
        IReadOnlyCollection<ServiceCapability> actual)
    {
        actual.Should().BeEquivalentTo(Expected);
        actual.Where(x => x.Capability == FlowCapability.Decide)
            .Should().ContainSingle()
                .Which.Service.Should().Be("RootCause");
    }
}
```

EXECUTABLE ASSET 1-L - AUTHORITY REGRESSION TEST

```
public sealed class FirstSliceAuthorityTests
{
    [Fact]
    public void HND_AUTH_001_subscribers_receive_no_verdict_authority()
    {
        var actual = FirstSliceFixture.Capabilities();
        FirstSliceAuthority.Validate(actual);

        actual.Where(x => x.Service is "Audit"
            or "Compliance" or "Reporting")
            .Should().NotContain(
                x => x.Capability == FlowCapability.Decide);
    }
}
```

REFERENCE-BEHAVIOUR ORACLE

Distribution must match the frozen modular meaning

The modular and distributed paths may differ in timing and transport. They must converge on the same owner-issued decision, public payload and append-only audit meaning for the same canonical input.

EXECUTABLE ASSET 1-M - BEHAVIOURAL COMPARISON CONTRACT

```
public sealed record SliceObservation(
    string CaseId,
    string OwnerDecisionSha256,
    string PublicFactSha256,
    string AuditChainSha256,
    bool ProjectionIsExplicitlyStale);

public sealed record SliceEquivalence(
    bool Accepted,
    IReadOnlyList<string> Mismatches);

public static class HandoffOracle
{
    public static SliceEquivalence Compare(
        SliceObservation modular,
        SliceObservation distributed)
    {
        var mismatch = new List<string>();
        AddIfDifferent("case", modular.CaseId,
            distributed.CaseId, mismatch);
        AddIfDifferent("decision",
            modular.OwnerDecisionSha256,
            distributed.OwnerDecisionSha256, mismatch);
        AddIfDifferent("public-fact",
            modular.PublicFactSha256,
            distributed.PublicFactSha256, mismatch);
        AddIfDifferent("audit-chain",
            modular.AuditChainSha256,
            distributed.AuditChainSha256, mismatch);

        if (!distributed.ProjectionIsExplicitlyStale)
            mismatch.Add("projection-freshness-not-explicit");
        return new(mismatch.Count == 0, mismatch);
    }
}
```

EXECUTABLE ASSET 1-N - FALSE EQUIVALENCE IS REJECTED

```
public sealed class HandoffOracleTests
{
    [Fact]
    public void HND_ORACLE_001_changed_public_fact_blocks_handoff()
    {
        var baseline = HandoffFixture.ModularObservation();
        var distributed = baseline with
        {
            PublicFactSha256 = Hashes.Other
        };

        var result = HandoffOracle.Compare(
            baseline, distributed);
        result.Accepted.Should().BeFalse();
        result.Mismatches.Should()
            .Contain("public-fact");
    }
}
```


IDENTIFIED EXECUTION

A retained test run names the toolchain and complete result

Exit code alone is insufficient. The gate binds source, SDK, lock file, test platform, manifest and result identities, then rejects zero discovery, skips, partial execution and stale evidence.

EXECUTABLE ASSET 1-O - RUN IDENTITY AND GATE

```
public sealed record HandoffRun(
    string CommitSha,
    string SdkVersion,
    string PackageLockSha256,
    string TestPlatform,
    string ManifestSha256,
    string EnvironmentSha256,
    string ResultSha256,
    DateTimeOffset CompletedAtUtc,
    int Discovered,
    int Executed,
    int Passed,
    int Failed,
    int Skipped,
    int ExitCode);

public static class HandoffRunGate
{
    public static void Accept(
        HandoffRun run,
        DateTimeOffset now,
        TimeSpan maximumAge)
    {
        if (new[]
        {
            run.CommitSha, run.SdkVersion,
            run.PackageLockSha256, run.TestPlatform,
            run.ManifestSha256, run.EnvironmentSha256,
            run.ResultSha256
        }.Any(String.IsNullOrEmpty))
            throw new UnidentifiedHandoffRun();
        if (run.Discovered <= 0)
            throw new ZeroDiscoveryHandoffRun();
        if (run.Executed != run.Discovered
            || run.Passed != run.Executed
            || run.Failed != 0 || run.Skipped != 0
            || run.ExitCode != 0)
            throw new IncompleteHandoffRun();
        if (now - run.CompletedAtUtc > maximumAge)
            throw new StaleHandoffRun();
    }
}
```

EXECUTABLE ASSET 1-P - ZERO DISCOVERY REMAINS RED

```
public sealed class HandoffRunGateTests
{
    [Fact]
    public void HND_DISC_001_zero_tests_cannot_accept_handoff()
    {
        var run = HandoffFixture.Run() with
        {
            Discovered = 0,
            Executed = 0,
            Passed = 0
        };

        var act = () => HandoffRunGate.Accept(
            run, HandoffFixture.Now, TimeSpan.FromHours(24));
        act.Should().Throw<ZeroDiscoveryHandoffRun>();
    }
}
```

CHAPTER 1 EVIDENCE

The handoff now closes with an explicit non-claim

EXECUTABLE ASSET 1-Q - HANDOFF EVIDENCE RECORD

```
{
  "evidenceId": "HND-REVIEW-001",
  "chapter": 1,
  "manifestProfile": {
    "nexus.handoff.v1/len32-utf8+sha256",
    "commitSha": null,
    "sdkVersion": null,
    "packageLockSha256": null,
    "testPlatform": null,
    "manifestSha256": null,
    "environmentSha256": null,
    "resultSha256": null,
    "tests": {
      "HND-ID-001": "PENDING",
      "HND-CAT-001": "PENDING",
      "HND-CAT-002": "PENDING",
      "HND-AUTH-001": "PENDING",
      "HND-ORACLE-001": "PENDING",
      "HND-DISC-001": "PENDING"
    }
  },
  "discovered": null,
  "executed": null,
  "passed": null,
  "failed": null,
  "skipped": null,
  "accepted": false
}
```

Chapter 1 re-audit result

The chapter now has the same closure discipline as the later audits: identified sources, a closed claim catalogue, canonical identity, owner-preserving authority graph, behavioural oracle, retained-run gate and an explicit evidence record.

CURRENT VERDICT

The architectural handoff is implementable and falsifiable, but it is not runtime-verified here. Every HND result remains PENDING until the pinned repository executes the printed assets.

The rule is visible in tests

TEST SHAPE 1-D — HONEST CLAIM TRANSITIONS

```
public sealed class HandoffGateTests
{
    [Fact]
    public void Specified_obligation_may_wait_for_runtime_evidence()
    {
        var obligation = new ProofObligation(
            "MSG-001",
            "CONTRACT-FLOOD-001",
            "Committed flood event survives broker outage.",
            ProofStatus.Specified,
            []);

        var act = () => HandoffGate.Validate(obligation);

        act.Should().NotThrow();
    }

    [Fact]
    public void Demonstrated_claim_without_evidence_is_rejected()
    {
        var obligation = new ProofObligation(
            "MSG-001",
            "CONTRACT-FLOOD-001",
            "Committed flood event survives broker outage.",
            ProofStatus.Demonstrated,
            []);

        var act = () => HandoffGate.Validate(obligation);

        act.Should().Throw<InvalidOperationException>()
            .WithMessage("*contains no retained evidence*");
    }
}
```

These snippets are deliberately dependency-light and compile-oriented, but this [BASE] chapter does not claim a compiled repository artifact. Chapter 4 owns the pinned solution and build evidence. Later chapters will replace the empty evidence lists with database, broker, contract, security, observability, migration, and rollback artifacts.

Code asset	Ambiguity removed	Later proof owner
1-A Handoff types	Separates decisions, obligations, states, and retained evidence.	All Parts
1-B Slice descriptor	Makes ownership, public facts, forbidden data access, and first claims explicit.	Parts III–VI
1-C Gate	Prevents evidence-free Demonstrated status.	Parts IV–X
1-D Tests	Shows the allowed pending state and the rejected false-success state.	Chapter 4 test project

What this chapter has proved

This chapter has not proved a microservice. It has proved that the implementation can begin without inventing a new architecture. The candidate posture, public language, ownership decisions, failure expectations, and final readiness gates are explicit enough to become constraints on the build.

It has also proved that two different readiness questions must remain separate. First: does the boundary deserve extraction? Second: can the extraction be performed responsibly? A positive answer to the first never excuses a weak answer to the second.

CHAPTER CLOSING

The handoff is complete when every inherited decision has a named runtime proof obligation - not when the first service project appears in the solution.

Chapter checkpoint

You are ready to continue when the expected answers below can be stated without turning an architectural decision into a runtime claim.

Question	Expected answer
Why did From Flow to Services stop before implementation?	Because architecture can justify a boundary but cannot claim executable evidence that has not been built and observed.
Why are there two sets of seven gates?	The first tests whether a boundary deserves extraction; the final set tests whether extraction can be performed responsibly.
Which six ledgers constrain the implementation?	Candidate map, service catalogue, API/event catalogue, data-ownership catalogue, failure catalogue, and pre-implementation checklist.
Why is RootCause stronger than a generic Evidence service?	RootCause owns a decision-rich language, invariants, case lifecycle, verdict, and meaningful public fact; Evidence alone is only a noun.
Why are Audit and Reporting special candidates?	Audit preserves append-only evidence without directing workflow; Reporting owns delayed projections rather than operational truth.
What remains unproved?	Independent runtimes and data, delivery and recovery, distributed trust, complete tracing, and reversible migration.
Why reuse one recurring flow?	It makes every pattern answer to the same business meaning and exposes contradictions between samples.
When does a decision become runtime evidence?	When a named artifact or experiment demonstrates it and the result remains consistent with the inherited ledgers.

NEXT CHAPTER

Chapter 2 creates the preservation ledger: the exact invariants, language, authorization, auditability, and deterministic decisions that must remain true after a boundary becomes remote.

PART I - CROSSING INTO DISTRIBUTION

2. What Must Remain True After Extraction

A service extraction succeeds only when the system changes where code runs without changing who owns truth, language, decisions, evidence, or deterministic behaviour.

CHAPTER ARGUMENT

Distribution is allowed to change latency, availability, and coordination. It is not allowed to change the meaning of the domain.

The easiest extraction to approve is one that moves classes into a new project, gives them a Web API, and leaves the existing business language untouched on the diagram. The most dangerous extraction is one that looks exactly like that while quietly moving decisions into callers, retries, integration handlers, reporting projections, or workflow scripts.

A correct modular model can become an incorrect distributed model without any single line of code looking obviously reckless. A duplicate message may attach the same evidence twice. A Reporting projection may begin to expose a derived status as if it were source truth. An Audit consumer may decide whether Compliance should proceed. An RL recommendation may be treated as a command because the network contract did not preserve its advisory meaning.

This chapter creates the preservation ledger that prevents those changes from becoming accidental architecture. The ledger is not a list of classes to copy. It is a list of truths that must remain demonstrable after the boundary becomes remote.

Extraction can preserve structure and still break meaning

Inside one process, a domain method, application handler, transaction, and caller often sit close enough that ownership is visible in code navigation. Across services, the same responsibility is distributed among a producer, a public contract, a broker, one or more consumers, local transactions, retries, projections, and operator tools. Every additional surface can accidentally acquire authority.

Apparently harmless change	Meaning that can be lost	Preservation response
Expose RootCauseCase through a public DTO	Private aggregate structure becomes external law.	Publish intention-based commands and stable summary contracts.
Retry AttachEvidence automatically	One business action can create repeated evidence.	Use message/request identity and enforce idempotent local effects.
Let Reporting calculate case state	A projection becomes decision owner.	Project the owner-issued fact and expose freshness, never invent state.
Let Audit trigger Compliance	Evidence storage becomes workflow authority.	Audit records facts; a process manager or owning context coordinates.
Send RecommendationGenerated as a command	Advisory intelligence becomes control logic.	Preserve explicit advisory vocabulary and forbid command-side use.
Share RootCause tables during migration	Temporary access becomes permanent hidden ownership.	Time-box the seam, log access, and define its removal gate.

PLAIN EXPLANATION

Preservation asks a simple question: after distribution, can every important fact still be traced to the context that is allowed to decide it?

The eight preservation dimensions

The ledger uses eight dimensions. The first seven protect domain and architectural truth; the eighth protects how delayed truth is communicated to users. Each dimension must name both a rule and the runtime proof that will eventually support it.

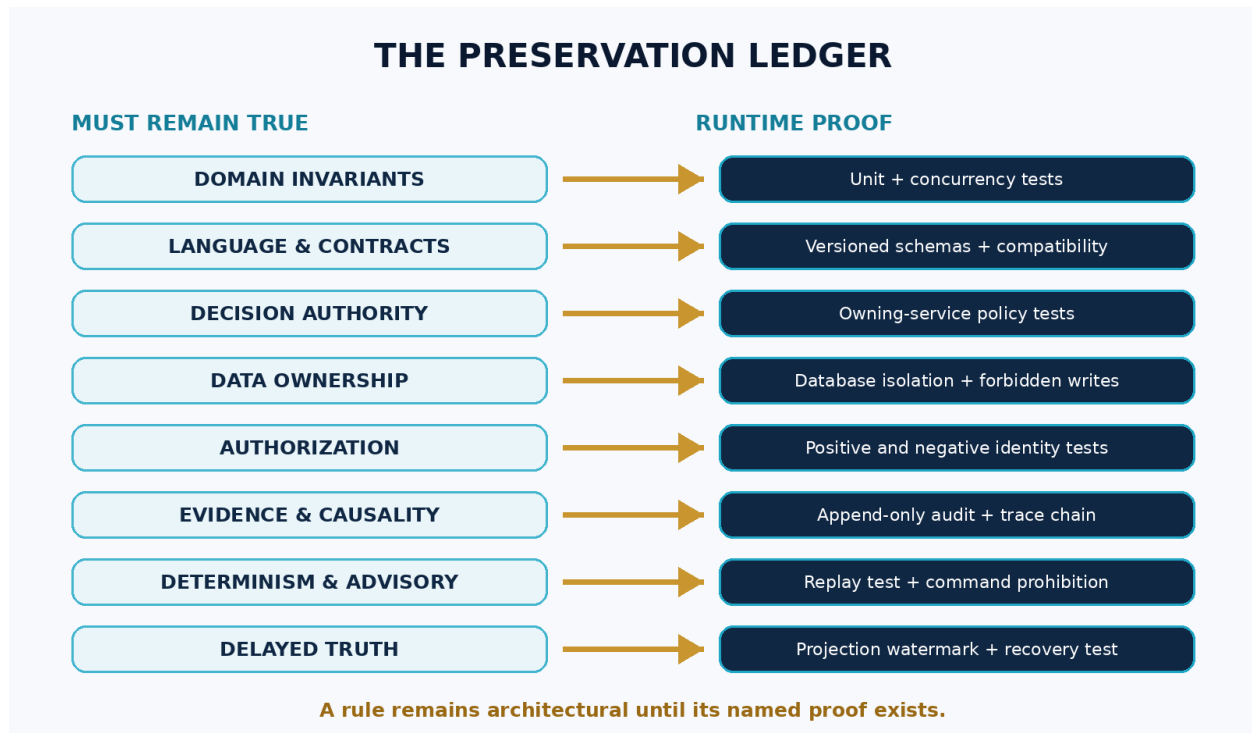


Figure 2.1 - The preservation ledger connects each protected truth to a concrete runtime proof.

Dimension	What is protected	Typical proof
Domain invariants	Rules that define valid aggregate state and transitions.	Unit, concurrency, and state-transition tests.
Language and contracts	Stable meaning of commands, queries, domain events, and public facts.	Schema review, versioning rules, and compatibility tests.
Decision authority	Which context may decide, correct, reject, or compensate.	Owning-service policies and forbidden-action tests.
Data ownership	Which service owns source truth and which data is only reference or projection.	Database isolation, permissions, and forbidden-write tests.
Authorization	Which user or service identity may request each capability.	Positive and negative policy tests with auditable identity.
Evidence and causality	Append-only history and the chain from cause to consequence.	Audit records, correlation/causation propagation, and trace review.
Determinism and advisory boundary	Repeatable engine decisions and the difference between advice and command.	Replay tests, configuration identity, and command prohibition.
Delayed truth	The fact that read models can be correct but temporarily behind.	Projection watermark, pending state, lag metric, and recovery test.

The first preservation ledger

The rule identifiers below are intentionally stable. Later chapters may add evidence references, implementation locations, and test identities, but they should not casually rename the rule. A stable identifier makes it possible to connect prose, code, tests, runbooks, and final evidence catalogues.

Rule ID	Owner	Preservation statement
RC-INV-001	RootCause	A closed RootCauseCase cannot accept new evidence, hypotheses, or ordinary state changes.
RC-INV-002	RootCause	A case cannot close with a verdict unless evidence exists.
RC-OWN-001	RootCause	Only RootCause may issue or correct the root-cause verdict.
ALM-DET-001	AlarmManagement	Alarm-flood detection remains deterministic for the same ordered inputs and configuration identity.
AUD-INV-001	Audit	Audit evidence is append-only; correction adds evidence and never rewrites history.
AUD-OWN-001	Audit	Audit records what occurred and never becomes a workflow command source.
CMP-OWN-001	Compliance	Only Compliance decides review state, escalation, and completion.
RPT-OWN-001	Reporting	Reporting owns projections and freshness metadata, not operational source truth.
RL-ADV-001	RL Advisory	A recommendation remains advice and is never interpreted as an operational command.
DATA-OWN-001	All services	No service writes another service's source-of-truth database.
FLOW-IDEM-001	All consumers	Repeated delivery of one message creates at most one business effect per consumer.
FLOW-CAUS-001	All boundaries	Correlation and causation identities survive HTTP, message, worker, and audit boundaries.
SEC-AUTH-001	Owning service	A privileged action requires authenticated identity and local authorization policy.
CONTRACT-001	Producer	Public event names and versions remain stable; private domain events are not leaked.
UX-DELAY-001	BFF/Reporting	Delayed or incomplete projection truth is visible rather than presented as current certainty.

LEDGER RULE

If an extraction cannot preserve a blocking rule, the candidate remains a module, the boundary is narrowed, or a transitional seam is introduced with an explicit removal condition.

RootCause invariants do not move to the network

The RootCause aggregate already owns rules that are local, precise, and testable. Distribution must not relocate those rules into API controllers, message consumers, BFF validation, or Reporting logic. Remote callers may request an action. Only the aggregate and its owning application boundary may decide whether the action is valid.

INHERITED INVARIANT SHAPE

```
public sealed class RootCauseCase
{
    private readonly List<Evidence> _evidence = new();

    public string? FinalVerdict { get; private set; }

    public void AddEvidence(Evidence evidence)
    {
        if (FinalVerdict is not null)
            throw new InvalidOperationException(
                "A closed case cannot change.");

        _evidence.Add(evidence);
    }

    public void CloseWithVerdict(string verdict)
    {
        if (_evidence.Count == 0)
            throw new InvalidOperationException(
                "A case cannot close without evidence.");

        FinalVerdict = verdict;
    }
}
```

The network can change when the request arrives and how the result is published. It cannot change the owner of the decision. If the same AttachEvidence request arrives twice, idempotency must protect the aggregate from a repeated business effect. If a late evidence message arrives after closure, the owning rule must reject it or route it through an explicit correction process. The consumer is not allowed to silently reopen the case.

Invariant	New distributed risk	Required evidence
Closed case cannot change	Late or redelivered message arrives after verdict.	State-transition test plus duplicate/late-message integration test.
Cannot close without evidence	Remote orchestration requests closure before evidence has converged.	Aggregate unit test plus incomplete-workflow test.
Only RootCause issues verdict	BFF, Reporting, or process manager derives final state.	Contract review and forbidden-write/forbidden-command tests.
Correction does not erase history	A remote update overwrites the original verdict.	Versioned correction event and append-only audit evidence.

Language must survive serialization

A public event is not simply a domain event encoded as JSON. It is a producer-owned promise whose words may outlive the current aggregate shape. The preservation ledger therefore protects names as carefully as state.

Internal language	Public contract	Preservation boundary
VerdictIssued	RootCauseVerdictIssued.v1	Internal model may evolve; public meaning and version remain stable.
AlarmFloodDetected	AlarmFloodDetected.v1	Payload communicates the fact without exposing alarm tables or detector internals.

Internal language	Public contract	Preservation boundary
VerdictCorrected	RootCauseVerdictCorrected.v1	Correction is a new public fact, not mutation of the original event.
RecommendationGenerated	RecommendationGenerated.v1	The contract preserves advisory meaning and contains no command semantics.

TERMINOLOGY DISCIPLINE

RootCauseVerdictIssued.v1 remains the public fact. Do not replace it with a convenient synonym unless a new contract version and migration decision explicitly justify the change.

Decision authority is stricter than data access

Preventing cross-service database writes is necessary but insufficient. A service can avoid another database and still steal authority by calculating another context's decision locally. Decision ownership must therefore be tested at the contract and workflow level, not only at the SQL permission level.

Context	May decide	May observe but not decide
AlarmManagement	Alarm state and alarm-flood detection.	RootCause case status, compliance outcome, reporting freshness.
RootCause	Case lifecycle, evidence acceptance, hypothesis state, verdict and correction.	Compliance completion and reporting projection state.
Audit	Whether an audit record is valid and appendable.	Whether workflow should continue, compensate, or close.
Compliance	Review state, escalation, human decision, and completion.	Root-cause verdict content and audit history.
Reporting	Projection shape, rebuild, watermark, and freshness status.	Operational alarm, verdict, audit, or compliance truth.
RL Advisory	Recommendation calculation and confidence metadata.	Any operational command or mandatory action.

Authorization follows decision ownership

Authentication tells a service who is making a request. Authorization asks whether that identity may request a capability owned by this service. The policy must remain local to the owner of the rule. The BFF may reject an obviously unauthorized request early, but its decision cannot replace authorization inside RootCause or Compliance.

- Opening or viewing a case and issuing a final verdict are different capabilities.
- A service identity proves which runtime made a call; it does not automatically inherit the user's permissions.
- Forwarded user context is minimized and auditable; it is not treated as unrestricted delegation.
- Administrative repair actions require separate policy, reason, correlation, and audit evidence.
- Reporting access does not grant authority to modify source truth.

AUTHORIZATION INVARIANT

A boundary is not preserved when the correct service receives the request but the wrong identity is allowed to make the decision.

Auditability preserves history, not control

Audit must be able to prove what occurred across service boundaries without becoming the central brain of the workflow. That requires append-only evidence, stable identifiers, actor and service identity, event type and version, occurred time, recorded time, correlation ID, causation ID, and a reason for privileged or corrective actions.

Evidence field	Why it must survive distribution
Message / request ID	Distinguishes replay from a new business action and supports idempotency evidence.
Correlation ID	Connects the complete alarm-to-compliance case.
Causation ID	Explains which fact or request directly caused the current action.
User identity	Shows the human authority behind an action where relevant.
Service identity	Shows which runtime performed or recorded the action.
Occurred / recorded time	Separates domain occurrence from delayed observation.
Contract type and version	Makes historical evidence interpretable after contracts evolve.
Correction reason	Preserves why a new fact superseded an earlier conclusion without erasing it.

Determinism and the advisory boundary

NEXUS-1 protects deterministic decision paths. Distribution must not let timing, retry count, consumer order, or an LLM explanation change the underlying result. The same ordered inputs, model/configuration identity, and deterministic engine version must produce the same decision. If the result changes, the evidence must identify which input or version changed.

The same discipline protects Reinforcement Learning Advisory. A recommendation may depend on a learned policy and may include uncertainty, but it remains advice. It cannot silently cross the boundary into a command because a consumer finds that convenient. Human or policy-owned authorization remains explicit.

Preserved property	Forbidden distributed shortcut	Proof direction
Deterministic causal verdict	Use arrival timing or retry count as hidden decision input.	Replay identical ordered inputs and configuration identity.
LLM is explanatory	Allow generated text to change deterministic case state.	Assert read-only explanation path and compare domain state before/after.
Recommendation is advisory	Translate recommendation directly into operational command.	Contract review plus command-side rejection test.
Abstention remains valid	Force an answer because a remote workflow expects completion.	Test insufficient-evidence path and visible incomplete state.

Delayed truth is a preservation concern

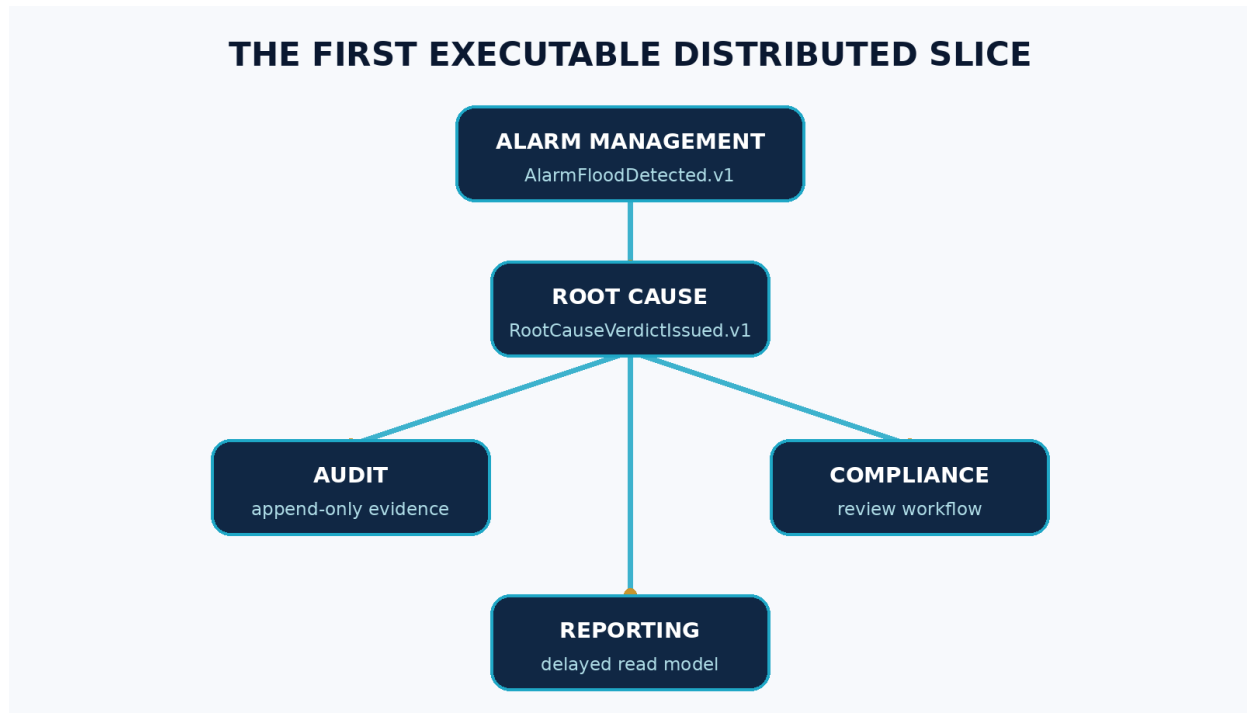


Figure 2.2 - Different services may observe the same case at different times without acquiring each other's decision authority.

Eventual consistency does not permit the interface to lie. Reporting may be behind RootCause. Compliance may have received a verdict while the read model still shows the case as pending. The preservation rule is not that every screen must be current. It is that the screen must distinguish source truth, projected truth, and projection age.

- A projection carries a last-applied event position or equivalent watermark.
- The BFF can expose pending, delayed, completed, and failed projection states.
- A refresh does not create a new business decision; it only asks for a newer observation.
- A repair or replay is auditable and idempotent.
- The user is never told that a case is complete merely because the projection has no pending row.

A reference model for the ledger

The ledger needs a representation that can later live beside the repository and evidence catalogues. Chapter 4 will place the model inside the buildable solution. At this stage the type contract is defined so that the architecture cannot postpone naming owner, rule, proof, and blocking status.

REFERENCE TYPE CONTRACT - COMPILE GATE OWNED BY CHAPTER 4

```
public enum PreservationKind
{
    DomainInvariant,
    Language,
    DecisionAuthority,
    DataOwnership,
    Authorization,
    EvidenceAndCausality,
    DeterminismAndAdvisory,
    DelayedTruth
}

public sealed record PreservationRule(
    string Id,
    string Owner,
    PreservationKind Kind,
    string Statement,
    string RequiredEvidence,
    bool BlocksExtraction);

public sealed class PreservationLedger
{
    private readonly List<PreservationRule> _rules = new();

    public IReadOnlyCollection<PreservationRule> Rules => _rules;

    public void Add(PreservationRule rule)
    {
        if (_rules.Any(x => x.Id == rule.Id))
            throw new InvalidOperationException(
                $"Duplicate preservation rule '{rule.Id}'.");

        _rules.Add(rule);
    }
}
```

The code deliberately does not contain a `CanExtract` boolean. A rule is not satisfied because a developer toggles a value. Later evidence records must point to a test, contract, migration artifact, trace, runbook, or reviewed decision. Extraction status is computed from those records, not declared by optimism.

From rule to test

Rule	Primary test level	Failure experiment or review
RC-INV-001 / 002	Aggregate unit and concurrency tests	Late and duplicate evidence after closure.
RC-OWN-001	Contract and authorization tests	Attempt verdict write through BFF, Reporting, and workflow coordinator.
ALM-DET-001	Deterministic replay test	Reorder transport delivery while preserving ordered domain input.
AUD-INV-001	Persistence integration test	Attempt update/delete and repeated append.
DATA-OWN-001	Database permission/integration test	Attempt direct cross-service write.
FLOW-IDEM-001	Consumer component test	Deliver the same message repeatedly and restart mid-processing.
FLOW-CAUS-001	End-to-end trace assertion	Break one propagation header and locate the evidence gap.

CHAPTER 2 — CODE AUDIT

The preservation ledger becomes enforceable

A rule that cannot be observed in code or a test remains vulnerable to reinterpretation. The following implementation shapes connect the stable rule IDs to concrete rejection paths.

SCOPE OF THIS REVISION

These assets are compile-oriented reference code. Chapter 4 still owns the pinned solution, package versions, clean build, and retained execution evidence. No test is reported as run here.

RC-INV-001 and RC-INV-002 belong to the aggregate

Remote handlers may deliver commands late or more than once, but they must never recreate RootCause policy in transport code. The aggregate rejects the invalid state transition; tests name the preservation rule so a later refactor cannot weaken it invisibly.

TEST SHAPE 2-A — ROOTCAUSE INVARIANT PRESERVATION

```
public sealed class RootCauseCaseTests
{
    [Fact]
    public void RC_INV_001_closed_case_rejects_late_evidence()
    {
        var sut = RootCauseCase.Open(CaseId.New());
        sut.AddEvidence(Evidence.Observed("pressure-drop"));
        sut.CloseWithVerdict("feedwater-loss");

        var act = () =>
            sut.AddEvidence(Evidence.Observed("late-sample"));

        act.Should().Throw<DomainRuleViolation>()
            .Which.RuleId.Should().Be("RC-INV-001");
    }

    [Fact]
    public void RC_INV_002_case_without_evidence_cannot_close()
    {
        var sut = RootCauseCase.Open(CaseId.New());

        var act = () => sut.CloseWithVerdict("unknown");

        act.Should().Throw<DomainRuleViolation>()
            .Which.RuleId.Should().Be("RC-INV-002");
    }
}
```

The public exception exposes a stable rule identifier, not an internal stack trace. The application boundary maps it to a versioned rejection contract; it does not convert rejection into a successful no-op.

PRESERVATION DIMENSION — DOMAIN INVARIANTS

Transport duplication must not duplicate business effects

Invariant enforcement and idempotency solve different problems. The aggregate protects legal state; the application boundary prevents the same request identity from applying the same legal transition twice. The owner transaction must contain both the business effect and completion of the request claim.

IMPLEMENTATION SHAPE 2-B — OWNER-LOCAL COMMAND BOUNDARY

```
public sealed class AttachEvidenceHandler
{
    private readonly IRootCauseUnitOfWork _unitOfWork;
    private readonly IProcessedRequestStore _processed;

    public async Task Handle(
        AttachEvidence command,
        CancellationToken cancellationToken)
    {
        await _unitOfWork.ExecuteAsync(async () =>
        {
            if (await _processed.ContainsAsync(
                command.RequestId, cancellationToken))
                return;

            var rootCauseCase = await _unitOfWork.Cases.GetRequiredAsync(
                command.CaseId, cancellationToken);

            rootCauseCase.AddEvidence(command.Evidence);

            await _processed.AddAsync(
                command.RequestId, cancellationToken);
        }, cancellationToken);
    }
}
```

TEST SHAPE 2-C — DUPLICATE REQUEST, ONE EFFECT

```
[Fact]
public async Task FLOW_IDEM_001_duplicate_request_has_one_effect()
{
    var request = AttachEvidenceMother.Valid(
        requestId: RequestId.Parse("req-884"));

    await handler.Handle(request, CancellationToken.None);
    await handler.Handle(request, CancellationToken.None);

    var saved = await fixture.LoadCase(request.CaseId);
    saved.Evidence.Should().ContainSingle();
    fixture.ProcessedRequests.Should().ContainSingle(
        x => x.RequestId == request.RequestId);
}
```

BOUNDARY WARNING

This shape requires a real database transaction and a uniqueness constraint on RequestId. An in-memory HashSet or a check followed by a separate commit does not prove FLOW-IDEM-001.

PRESERVATION DIMENSION — AUTHORIZATION

Authorization follows decision ownership

The BFF may improve the user experience by hiding an action, but RootCause must enforce the policy that protects verdict authority. Service identity and delegated user authority are separate inputs; neither implies the other.

IMPLEMENTATION SHAPE 2-D — LOCAL CAPABILITY POLICY

```
public static class RootCausePolicies
{
    public const string IssueVerdict = "rootcause.issue-verdict";

    public static AuthorizationPolicy IssueVerdictPolicy() =>
        new AuthorizationPolicyBuilder()
            .RequireAuthenticatedUser()
            .RequireClaim("scope", "rootcause.write")
            .RequireClaim("nexus.site")
            .AddRequirements(new HumanDecisionRequirement())
            .Build();
}

public sealed class HumanDecisionHandler
    : AuthorizationHandler<HumanDecisionRequirement>
{
    protected override Task HandleRequirementAsync(
        AuthorizationHandlerContext context,
        HumanDecisionRequirement requirement)
    {
        var actorKind = context.User.FindFirst("actor_kind")?.Value;
        if (actorKind == "human")
            context.Succeed(requirement);

        return Task.CompletedTask;
    }
}
```

TEST SHAPE 2-E — SERVICE IDENTITY CANNOT ISSUE VERDICT

```
[Fact]
public async Task SEC_AUTH_001_service_token_is_not_human_authority()
{
    var principal = PrincipalMother.Service(
        service: "reporting",
        scopes: ["rootcause.write"],
        site: "site-17");

    var result = await authorization.AuthorizeAsync(
        principal,
        resource: null,
        RootCausePolicies.IssueVerdict);

    result.Succeeded.Should().BeFalse();
}
```

The test is intentionally negative. It proves that possession of a technically valid service token and a broad scope does not grant the human decision authority protected by RC-OWN-001.

PRESERVATION DIMENSION — EVIDENCE AND CAUSALITY

Audit is append-only evidence, not a workflow engine

Audit accepts facts with stable identity and causal metadata. It exposes no method that changes an earlier record and no method that commands Compliance. Correction is represented by another record that points to the superseded evidence.

IMPLEMENTATION SHAPE 2-F — IMMUTABLE AUDIT RECORD

```
public sealed class AuditRecord
{
    private AuditRecord() { }

    public AuditRecordId Id { get; private init; }
    public MessageId MessageId { get; private init; }
    public string EventType { get; private init; } = null!;
    public int SchemaVersion { get; private init; }
    public CorrelationId CorrelationId { get; private init; }
    public CausationId CausationId { get; private init; }
    public string ActorIdentity { get; private init; } = null!;
    public string ServiceIdentity { get; private init; } = null!;
    public DateTimeOffset OccurredAtUtc { get; private init; }
    public DateTimeOffset RecordedAtUtc { get; private init; }
    public AuditRecordId? Supersedes { get; private init; }
    public string? CorrectionReason { get; private init; }
}
```

IMPLEMENTATION SHAPE 2-G — PERSISTENCE REJECTS MUTATION

```
public sealed class AppendOnlyAuditInterceptor : SaveChangesInterceptor
{
    private static void RejectMutation(DbContextEventData eventData)
    {
        var changed = eventData.Context!.ChangeTracker
            .Entries<AuditRecord>()
            .Where(x => x.State is EntityState.Modified
                or EntityState.Deleted)
            .Select(x => x.Entity.Id)
            .ToArray();

        if (changed.Length > 0)
            throw new AuditMutationRejected(changed);
    }

    public override InterceptionResult<int> SavingChanges(
        DbContextEventData eventData,
        InterceptionResult<int> result)
    {
        RejectMutation(eventData);
        return result;
    }
}
```

EVIDENCE BOUNDARY

An EF Core interceptor is defence in depth, not the only control. Database permissions, append-only storage policy, integration tests, and operator access rules must agree.

PRESERVATION DIMENSION — DETERMINISM

A deterministic result carries its replay identity

The engine result is interpretable only when the ordered input set, configuration, and engine version are immutable and identifiable. Arrival time, delivery attempt, and generated explanation remain outside the decision input.

IMPLEMENTATION SHAPE 2-H — DETERMINISTIC DECISION RECEIPT

```
public sealed record DecisionInput(
    IReadOnlyList<SignalSample> OrderedSamples,
    string ConfigurationSha256,
    string EngineVersion);

public sealed record DecisionReceipt(
    string InputSha256,
    string ConfigurationSha256,
    string EngineVersion,
    RootCauseVerdict Verdict);

public static DecisionReceipt Evaluate(DecisionInput input)
{
    var canonicalBytes = CanonicalSerializer.Serialize(
        input.OrderedSamples);

    var verdict = DeterministicCausalEngine.Evaluate(
        input.OrderedSamples,
        input.ConfigurationSha256);

    return new(
        Sha256.Hex(canonicalBytes),
        input.ConfigurationSha256,
        input.EngineVersion,
        verdict);
}
```

TEST SHAPE 2-I — REPLAY AND ADVISORY REJECTION

```
[Fact]
public void ALM_DET_001_same_identity_produces_same_verdict()
{
    var input = DecisionInputMother.Fixed();

    Evaluate(input).Should().Be(Evaluate(input));
}

[Fact]
public async Task RL_ADV_001_recommendation_cannot_enter_command_bus()
{
    var recommendation = RecommendationMother.Valid();

    var act = () => commandBus.Send(
        recommendation, CancellationToken.None);

    await act.Should().ThrowAsync<AdvisoryMessageRejected>();
}
```

The equality test is necessary but not sufficient: the retained evidence must also include canonical input bytes, configuration hash, engine version, and the exact executable identity.

PRESERVATION DIMENSION — DELAYED TRUTH

The BFF exposes observation state, not invented certainty

Reporting may be correct and still be behind. The response contract therefore carries the owner-issued status separately from projection freshness. A missing projection is not converted into a completed case or an empty-success response.

IMPLEMENTATION SHAPE 2-J — EXPLICIT PROJECTION FRESHNESS

```
public enum ProjectionState
{
    Current,
    Pending,
    Delayed,
    Failed
}

public sealed record RootCauseCaseView(
    CaseId CaseId,
    string? OwnerIssuedStatus,
    ProjectionState ProjectionState,
    long LastAppliedPosition,
    DateTimeOffset ObservedAtUtc,
    TimeSpan ProjectionAge);

public static ProjectionState Classify(
    long ownerPosition,
    long projectionPosition,
    DateTimeOffset observedAtUtc,
    DateTimeOffset nowUtc) =>
    projectionPosition >= ownerPosition ? ProjectionState.Current :
    nowUtc - observedAtUtc > TimeSpan.FromMinutes(2)
    ? ProjectionState.Delayed
    : ProjectionState.Pending;
```

TEST SHAPE 2-K — DELAYED PROJECTION STAYS VISIBLE

```
[Fact]
public void UX_DELAY_001_lag_is_not_reported_as_completion()
{
    var view = bff.Compose(
        ownerPosition: 884,
        projectionPosition: 879,
        projectionStatus: null,
        nowUtc: Clock.At("2026-07-25T10:05:00Z"));

    view.OwnerIssuedStatus.Should().BeNull();
    view.ProjectionState.Should().Be(ProjectionState.Delayed);
    view.ProjectionAge.Should().BeGreaterThan(TimeSpan.Zero);
}
```

Technical material still required

Rule family	Executable evidence still required
RC-INV / FLOW-IDEM	Pinned aggregate tests, real database uniqueness and atomicity, duplicate/late-delivery fixture.
SEC-AUTH	Real authentication schemes, policy registration, site/tenant isolation, positive and negative API tests.
AUD-INV	Database permissions, update/delete rejection, correction append, restore and tamper evidence.
ALM-DET / RL-ADV	Canonical serializer, versioned engine artifact, replay corpus, command-side rejection.
UX-DELAY	Reporting pause/recovery test, watermark evidence, BFF and UI rendering test.

REVISION RESULT

The preservation ledger now has visible code paths for rejection, idempotency, authorization, append-only evidence, deterministic replay, advisory isolation, and delayed truth. The chapter still states honestly that repository-level compile and runtime evidence belong to later gates.

Rule	Primary test level	Failure experiment or review
SEC-AUTH-001	Positive/negative policy tests	Expired, missing, insufficient, and wrong service identity.
UX-DELAY-001	BFF/UI component test	Pause Reporting and verify visible lag/pending state.

When preservation fails

A preservation failure is an architectural result, not an inconvenience to hide. The team has four responsible responses, ordered from safest to most ambitious:

1. Keep the candidate inside the modular runtime and strengthen the internal boundary.
2. Narrow the proposed service so that one owner, one truth, and one contract remain coherent.
3. Introduce an anti-corruption or transitional seam with an owner, telemetry, deadline, and removal gate.
4. Change the domain or public contract through an explicit versioned decision, migration plan, and evidence update.

STOP CONDITION

Do not accept a distributed implementation that passes technical tests by weakening the original invariant or moving the decision to a more convenient owner.

What this chapter has proved

Chapter 2 has produced the first preservation ledger and stable rule identifiers. It has named the domain invariants, language, authority, data ownership, authorization, evidence, determinism, advisory boundary, and delayed-truth rules that later implementation must prove.

It has not yet proved those rules in a distributed runtime. The repository topology, executable service template, databases, contracts, broker, identities, traces, and failure laboratories do not exist at this point in the narrative. The ledger prevents those later artifacts from quietly redefining success.

CHAPTER CLOSING

Distribution may weaken local certainty. It may not weaken the rules that tell NEXUS-1 which truth is valid and who is allowed to decide it.

Chapter checkpoint

You are ready to continue when the questions below can be answered from the ledger rather than from a future implementation promise.

Question	Expected answer
What is a preservation ledger?	A stable set of owned rules and named proof obligations that constrain service extraction.
Why is copying classes insufficient?	Because decision authority, data ownership, retries, projections, and workflow can change meaning without changing class names.
Which RootCause rules block extraction?	Closed cases cannot change, cases cannot close without evidence, and only RootCause may issue or correct verdicts.
Why is authorization part of preservation?	The correct owner still produces invalid truth if the wrong identity is allowed to request the decision.
Why must Audit remain append-only but non-directive?	It preserves evidence of decisions while leaving workflow and domain authority with their owners.
How is deterministic behaviour protected?	By replaying the same ordered inputs and configuration/engine identity and excluding timing, retries, or LLM text from the decision.
How is delayed truth communicated honestly?	With projection watermark, age, pending/failure state, and auditable recovery.
What happens if a blocking rule cannot be preserved?	The service waits, narrows, uses an explicit transitional seam, or changes through a versioned decision; the rule is not silently weakened.

NEXT CHAPTER

Chapter 3 applies the readiness and preservation ledgers to choose the first distributed slice. It will justify why AlarmManagement -> RootCause is the starting seam and why Audit, Compliance, and Reporting enter as distinct subscribers rather than as one distributed transaction.

PART I - CROSSING INTO DISTRIBUTION

3. Choosing the First Distributed Slice

The first extraction must be small enough to reverse, rich enough to expose distributed failure, and precise enough that every decision still has one owner.

CHAPTER ARGUMENT

Choose the seam that teaches the system how to distribute truth safely. For NEXUS-1, that seam is AlarmManagement -> RootCause, followed by independent subscribers to the verdict fact.

The strongest service candidate is not automatically the strongest first distributed slice. RootCause has cohesive language, invariants, lifecycle, data, and decision authority. Moving it into a separate runtime would therefore be defensible. But extracting one box and continuing to call it synchronously would prove little about the architecture that the rest of this book must build. It could demonstrate deployment. It would not necessarily demonstrate durable fact publication, redelivery, independent local commitments, delayed observation, or recovery after partial failure.

A slice crosses a meaningful seam. It begins with a producer-owned fact, transfers that fact without transferring decision authority, and ends with a consumer-owned decision or durable state change. It contains the failure states that make distribution real. The first slice must therefore be selected as an experiment in architecture, not as a race to increase the service count.

The selected path begins when AlarmManagement detects an alarm flood and publishes AlarmFloodDetected.v1. RootCause consumes that fact, opens or advances its own case under its own invariants, and later publishes RootCauseVerdictIssued.v1. Audit, Compliance, and Reporting observe that verdict independently. No subscriber receives permission to decide what RootCause decided, and no distributed transaction attempts to make all four reactions succeed at once.

Selection is a gate, not a score contest

Candidate comparison is useful only when it exposes trade-offs. A numerical score would create false precision: a boundary with unclear ownership must not win because its deployment cost happens to be low. The selection therefore uses qualitative gates. A candidate must pass the blocking gates; strengths on the remaining dimensions determine its teaching value and order.

Selection gate	Question	Blocking condition
Owned trigger	Can the slice start from a fact whose producer is unambiguous?	The fact is assembled from shared tables or negotiated by several contexts.
Owned outcome	Does the consumer make a decision or durable change in its own language?	The consumer merely mirrors the producer or steals its authority.
Data separation	Can each side commit to data it owns?	Correctness requires a cross-service write or shared transaction.
Contract stability	Can the first public fact be small, meaningful, and versioned?	The payload exposes private object graphs or unstable internal events.
Failure visibility	Will delay, duplication, ordering, and outage produce observable states?	Failure is hidden behind a synchronous facade or cannot be diagnosed.
Safety boundary	Can the slice fail without issuing an unsafe control action?	A delayed or repeated message could directly control protected equipment.
Reversibility	Can traffic return to the modular path without corrupting truth?	Publication creates irreversible dual ownership or incompatible history.

SELECTION RULE

A candidate that fails ownership, safety, or reversibility remains modular regardless of how attractive its technology demonstration appears.

Comparing the candidate seams

The comparison below applies the readiness catalogue from Chapter 1 and the preservation rules from Chapter 2. The terms strong, mixed, and weak describe current evidence, not permanent worth. A deferred seam can become the correct choice after its language, contracts, or failure boundaries mature.

Candidate seam	Boundary truth	Failure learning	Migration safety	Decision
AlarmManagement -> RootCause	Strong: alarm/flood fact becomes case/evidence/verdict language.	High: duplicate, delay, outage, replay, and local commit are visible.	Strong with a strangler seam and one active case writer.	SELECT FIRST
RootCause -> Audit / Compliance	Strong subscriber ownership, but it begins after the central case decision.	High fan-out and lag learning; weak as the only opening seam.	Strong when verdict contract is stable.	PHASE 2 OF SLICE
Instrumentation -> DigitalTwin	Mixed: high-rate measurements and simulation state remain tightly coupled.	High volume learning, but failure meaning is costly and noisy.	Weak until state, cadence, and replay rules are explicit.	WAIT
RL Advisory -> operator experience	Clear advisory vocabulary, optional to the core flow.	Useful trust and timeout learning; limited ownership proof.	Strong if advisory remains optional.	LATER / OPTIONAL
Reporting-only extraction	Clear projection ownership but no authoritative domain decision.	Good lag/rebuild learning; insufficient end-to-end decision evidence.	Strong and low risk.	SUPPORTING SLICE

Why not extract RootCause alone?

RootCause alone is a service candidate; it is not yet a distributed lesson. A new RootCause process with an HTTP endpoint and a database can still preserve the old process assumptions: one caller waits for one answer, retry behaviour is hidden inside a client, and downstream work begins only after the request returns. The solution has more deployments but has not confronted the central uncertainty of an event-driven system: what happens when one owner has committed and the next owner has not yet observed the fact?

Extraction shape	What it can prove	What it leaves unproved
RootCause as a separate synchronous API	Independent hosting, HTTP identity, local authorization, database ownership.	Durable publication, redelivery, fan-out, projection lag, broker recovery, consumer idempotency.
AlarmManagement -> RootCause event seam	Producer contract, asynchronous acceptance, consumer-owned transaction, inbox/idempotency, recovery.	Downstream fan-out and delayed read-side truth until subscribers are added.
Verdict subscriber fan-out	Independent Audit, Compliance, and Reporting reactions; partial success; replay and lag.	Production scale and operational maturity until later parts add experiments and evidence.

BOUNDARY TEST

If removing the network would leave every transaction and every failure state unchanged, the first slice is probably demonstrating packaging rather than distribution.

The selected slice

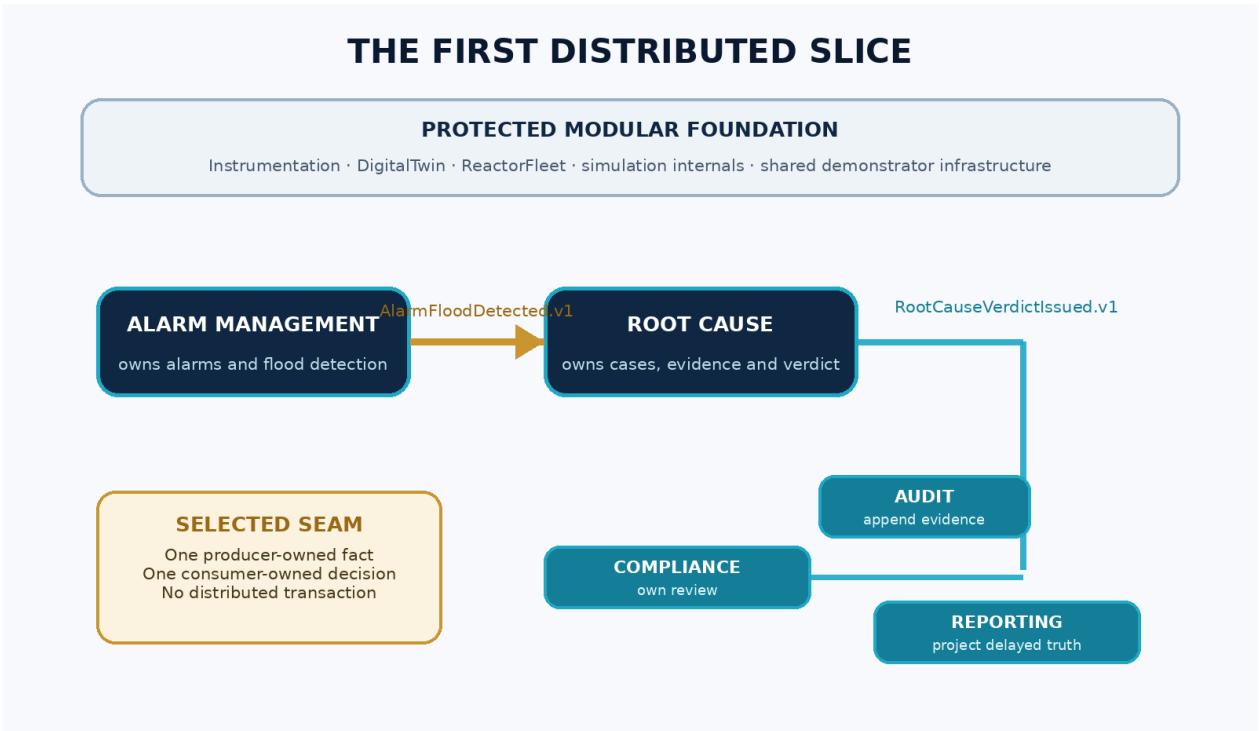


Figure 3.1 - The first distributed slice crosses one producer-consumer seam, then expands through independent verdict subscribers.

AlarmManagement and RootCause use different bounded-context languages. AlarmManagement understands alarm identity, activation, suppression, density, windows, and flood detection. RootCause understands a case, evidence, hypothesis, confidence, abstention, verdict, correction, and closure. AlarmFloodDetected.v1 crosses between those languages without pretending that the producer has opened a case or decided a cause.

That transformation is precisely why the seam is valuable. The producer states what it knows. The consumer decides what that fact means inside its own model. Neither side needs the other's tables, aggregate shape, or transaction. A failure can delay the transition from detected flood to active case, but it cannot make AlarmManagement the owner of a RootCause verdict.

Boundary element	AlarmManagement owns	RootCause owns	Must not cross
Language	Alarm, activation, window, flood.	Case, evidence, hypothesis, verdict.	Private aggregates and internal domain events.
Source data	Alarm observations and detector state.	Case history, evidence references, verdict and correction.	Table access or shared ORM entities.
Decision	Whether an alarm flood was detected.	Whether a case opens, accepts evidence, abstains, closes, or corrects.	Remote mutation of the other context's state.
Public fact	AlarmFloodDetected.v1.	RootCauseVerdictIssued.v1 and later correction facts.	Commands disguised as past-tense events.
Local commit	Detector state plus durable outgoing fact.	Inbox receipt plus case change and durable outgoing fact.	A transaction spanning both databases or the broker.

The first event carries evidence, not authority

AlarmFloodDetected.v1 must contain enough stable evidence for RootCause to identify the occurrence, reject a duplicate business effect, connect the case to the wider flow, and retrieve additional information through an explicit contract if needed. It must not serialize the AlarmManagement aggregate or instruct RootCause to open, close, or classify a case.

REFERENCE PUBLIC-CONTRACT SHAPE - [EXTEND][NEW]

```
public sealed record AlarmFloodDetectedV1(  
    Guid EventId,  
    string AlarmGroupId,  
    DateTimeOffset WindowStartedAt,  
    DateTimeOffset WindowEndedAt,  
    int ActivationCount,  
    string DetectorVersion,  
    string CorrelationId,  
    string CausationId,  
    DateTimeOffset OccurredAt);  
  
public sealed record RootCauseVerdictIssuedV1(  
    Guid EventId,  
    Guid CaseId,  
    string VerdictCode,  
    decimal Confidence,  
    string EvidenceRevision,  
    string CorrelationId,  
    string CausationId,  
    DateTimeOffset OccurredAt);
```

These records are architectural reference contracts, not compile-checked production types. Chapter 4 owns the first repository compile gate. Chapters 19 and 20 will extend envelope metadata, serialization, schema compatibility, and contract-test details. The names and responsibilities are binding now; transport-specific representation remains deliberately open.

TECHNICAL MATERIAL STATUS - [EXTEND][NEW]

This chapter introduces the first public event shapes and ADR. Repository placement, nullable rules, serialization configuration, schema fixtures, and automated compatibility checks require additional technical material in Chapters 4, 19, and 20.

Fan-out means independent commitments

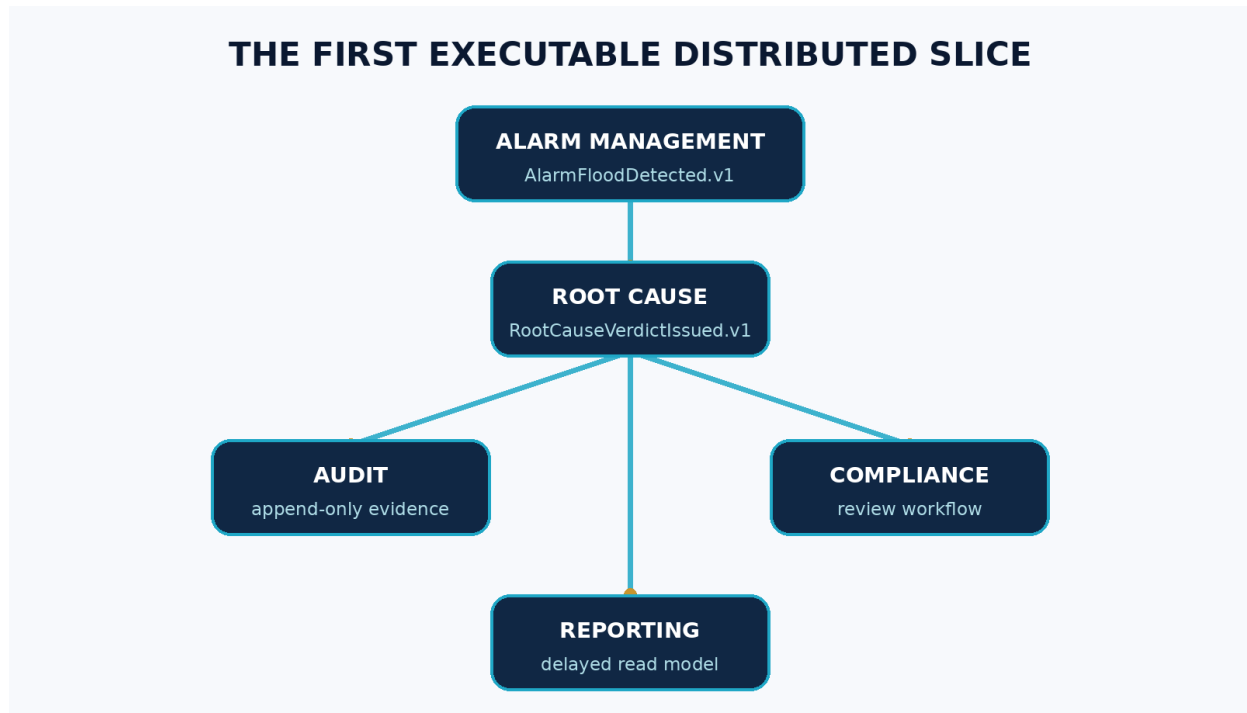


Figure 3.2 - Audit, Compliance, and Reporting can observe the same verdict at different times while preserving separate authority.

After RootCause commits a verdict, it publishes RootCauseVerdictIssued.v1 through a durable local publication mechanism. The event may then reach Audit, Compliance, and Reporting in any order. One subscriber may be current, another delayed, and another temporarily unavailable. That is not a corrupted distributed transaction. It is the expected state of independent services, provided the lag is visible, recovery is bounded, and each consumer's effect is idempotent.

Subscriber	Local commitment	Allowed delay	Forbidden authority
Audit	Append an immutable evidence record with identity, type/version, time, correlation, and causation.	May lag and replay; missing evidence must alert and recover.	Must not direct workflow, approve verdict, or close Compliance.
Compliance	Open or advance its own review under local policy and human-decision rules.	May remain pending while unavailable; backlog and age must be visible.	Must not rewrite the RootCause verdict or Audit history.
Reporting	Update a projection, watermark, freshness state, and case timeline.	Expected eventual consistency; UI exposes age and failure.	Must not present a projection as operational source truth.
RL Advisory - later	Optionally produce a recommendation with policy/model identity and confidence.	Timeout or absence cannot block the authoritative flow.	Must never translate advice directly into an operational command.

NO DISTRIBUTED TRANSACTION

RootCause does not roll back a valid verdict because Reporting is behind or Compliance is unavailable. Each owner commits locally; delivery, retry, reconciliation, and operator evidence connect the results.

The runtime state is a set of truths

A distributed case does not have one instantaneous global state. It has a set of owner-issued truths and observer positions. At a particular moment RootCause may truthfully report VerdictIssued, Audit may truthfully report EvidencePending, Compliance may truthfully report ReviewNotStarted, and Reporting

may truthfully expose a watermark behind the verdict event. The BFF can compose those statements. It cannot erase their timing differences by inventing a single status.

Observed condition	Correct interpretation	Required operator/user signal
RootCause committed; broker publish pending	Verdict is source truth; dissemination is incomplete.	Outbox age, retry state, alert threshold, correlation link.
Audit applied; Compliance unavailable	Evidence exists; review has not yet begun.	Subscriber backlog and explicit compliance-pending state.
Compliance applied; Reporting behind	Workflow state changed; read model is stale.	Projection watermark, age, and refresh/recovery status.
Duplicate verdict delivered	Transport repeated a fact; business outcome must remain singular per consumer.	Inbox/idempotency evidence and duplicate metric.
Events arrive out of order	Consumer observes a later fact before required predecessor.	Buffer/retry/dead-letter policy and causal diagnostic.

An incremental implementation sequence

The selected slice is implemented in stages so that every new failure mode arrives with an observable rollback point. The order is part of the architecture: it prevents a team from introducing broker, databases, identity, fan-out, and migration simultaneously and then being unable to explain which assumption failed.

1. Freeze the modular reference behaviour and capture preservation tests for AlarmFloodDetected and RootCause case decisions.
2. Create the buildable solution topology and service template without moving business ownership. Chapter 4 owns this compile gate.
3. Introduce the versioned AlarmFloodDetected.v1 contract and verify producer/consumer compatibility without activating remote traffic.
4. Give RootCause an isolated store, idempotent consumption boundary, and durable local publication path.
5. Shadow or selectively route eligible floods through the seam while comparing business outcomes with the modular reference path.
6. Make RootCause the single active case writer for the selected cohort; preserve a bounded rollback route that does not create dual writes.
7. Publish RootCauseVerdictIssued.v1 and add Audit, Compliance, and Reporting one subscriber at a time.
8. Run duplicate, delay, outage, restart, poison-message, replay, and projection-lag experiments before expanding traffic.

ADR-001 - Select AlarmManagement -> RootCause

The choice is recorded as an architecture decision because later chapters must be able to distinguish an intentional seam from an accidental example. If implementation evidence invalidates an assumption, ADR-001 is amended or superseded; the evidence is not rewritten to make the original choice appear inevitable.

ADR field	ADR-001 record
Title	Select AlarmManagement -> RootCause as the first distributed slice.
Status	Accepted for the reference implementation; production suitability remains evidence-gated.
Context	RootCause is the strongest decision-rich candidate, but an isolated extraction would not test durable asynchronous ownership. The first slice must be reversible and outside safety-like control.
Decision	Publish AlarmFloodDetected.v1 from AlarmManagement; let RootCause consume it under local invariants and publish RootCauseVerdictIssued.v1; add Audit, Compliance, and Reporting as independent subscribers.
Consequences	The design accepts eventual consistency, duplicate delivery, partial subscriber success, observable lag, local idempotency, contract versioning, outbox/inbox work, and operational recovery.
Rejected alternatives	RootCause-only synchronous extraction, Instrumentation -> DigitalTwin first, Reporting-only as the main slice, RL Advisory as the primary flow, and a transaction spanning RootCause plus all subscribers.
Reversal condition	Reverse or narrow the seam if one writer cannot be maintained, invariants diverge from the modular reference, contract meaning is unstable, recovery cannot be bounded, or safety/control authority leaks across it.
Evidence required	Build/contract tests, isolated ownership proof, duplicate/replay tests, failure experiments, trace continuity, projection freshness, runbook recovery, and business-outcome comparison during migration.

Risk register for the first slice

The register names design risks before infrastructure choices create momentum. Mitigation chapters are owners of future evidence, not a promise that the risk has already been removed.

Risk	Early control	Evidence owner
Shared-database escape	Separate credentials; prohibit cross-service writes; time-box any migration read seam.	Data ownership and migration chapters.
Unstable event contract	Small producer-owned fact, explicit v1, fixtures and compatibility review.	Chapters 19-20 [EXTEND].
Duplicate / redelivered fact	Message identity, consumer inbox/idempotency, singular local effect.	Messaging reliability chapters.
Out-of-order observation	Causation metadata, explicit predecessor policy, retry/dead-letter diagnostics.	Messaging and failure-lab chapters.
Broken correlation or causation	Create once, propagate across HTTP, message, worker, audit, and trace boundaries.	Observability chapters.
Subscriber steals a decision	Contract review, forbidden-action tests, local authorization and data permissions.	Service and security chapters.

Risk	Early control	Evidence owner
Projection freshness is hidden	Watermark, lag metric, pending/failure UX, rebuild runbook.	Reporting and BFF chapters.
Broker unavailable	Durable local outbox, bounded retry, alert, operator recovery without losing source truth.	Platform and reliability chapters.
Rollback after publication	Cohort routing, one active writer, event-history retention, explicit reconciliation plan.	Migration chapters.

RISK RULE

A mitigation is not complete when a pattern is named. It is complete when a failure experiment and an operator-visible artifact demonstrate the intended recovery.

The migration and reversal boundary

Reversibility is protected by keeping the modular implementation as the reference path until the remote slice has evidence. Traffic is routed by an explicit cohort or feature decision. For any alarm-flood occurrence, exactly one path is authoritative for RootCause case mutation. Shadow processing may compare decisions, but the shadow path cannot publish authoritative verdicts or update subscriber truth.

Phase	Authoritative writer	Allowed comparison	Rollback action
Baseline	Modular RootCause path.	Capture inputs, outcomes, and preservation tests.	No remote traffic to unwind.
Dark / shadow	Modular path only.	Remote consumer evaluates copied inputs without authoritative effects.	Stop shadow delivery; preserve diagnostics.
Cohort cutover	Exactly one path per routed cohort.	Compare normalized outcomes and timing across cohorts.	Route new work back; reconcile in-flight messages explicitly.
Remote authoritative	RootCause service for declared scope.	Continuously compare invariants, traces, lag, and failure evidence.	Invoke runbook; do not dual-write; retain event and audit history.

Rollback does not mean deleting the events that were already published. Published facts are history. Reversal changes the route for new work, drains or quarantines in-flight delivery, and reconciles owner state using explicit evidence. If the team cannot describe that sequence before cutover, the slice has not passed the reversibility gate.

CHAPTER 3 — CODE AUDIT

Selection gates are executable constraints

The first slice is not selected by enthusiasm or by a weighted technology score. Every blocking gate must pass; teaching value orders only the candidates that remain.

SCOPE OF THIS REVISION

The code makes ADR-001 computable and testable, but it does not claim that the broker, databases, contracts, or rollout environment have been exercised. Those remain later evidence gates.

IMPLEMENTATION SHAPE 3-A — QUALITATIVE EXTRACTION GATES

```
public enum SelectionGate
{
    OwnedTrigger,
    OwnedOutcome,
    DataSeparation,
    ContractStability,
    FailureVisibility,
    SafetyBoundary,
    Reversibility
}

public enum GateResult { Pass, Mixed, Fail }

public sealed record GateAssessment(
    SelectionGate Gate,
    GateResult Result,
    string Reason,
    bool Blocking);

public sealed record SliceCandidate(
    string Producer,
    string Consumer,
    string TriggerFact,
    string OwnedOutcome,
    IReadOnlyList<GateAssessment> Assessments);

public sealed record SelectionDecision(
    bool Selected,
    IReadOnlyList<GateAssessment> BlockingFailures);
```

IMPLEMENTATION SHAPE 3-B — NO SCORE CAN WAIVE A BLOCKER

```
public static SelectionDecision Evaluate(SliceCandidate candidate)
{
    var blockers = candidate.Assessments
        .Where(x => x.Blocking && x.Result == GateResult.Fail)
        .ToArray();

    return blockers.Length == 0
        ? new(true, [])
        : new(false, blockers);
}
```

There is deliberately no total score. A candidate cannot compensate for ambiguous ownership or unsafe authority by being cheap to deploy or technically fashionable.

ADR-001 — SELECTION EVIDENCE

The selected seam and a rejected seam use the same gate

TEST SHAPE 3-C — ALARMMANAGEMENT TO ROOTCAUSE PASSES

```
[Fact]
public void ADR_001_alarm_to_rootcause_passes_blocking_gates()
{
    var candidate = CandidateMother.AlarmToRootCause(
        ownedTrigger: GateResult.Pass,
        ownedOutcome: GateResult.Pass,
        dataSeparation: GateResult.Pass,
        safetyBoundary: GateResult.Pass,
        reversibility: GateResult.Pass);

    var decision = Evaluate(candidate);

    decision.Selected.Should().BeTrue();
    decision.BlockingFailures.Should().BeEmpty();
}
```

TEST SHAPE 3-D — UNSAFE OR AMBIGUOUS SEAM IS REJECTED

```
[Theory]
[InlineData(SelectionGate.OwnedTrigger)]
[InlineData(SelectionGate.OwnedOutcome)]
[InlineData(SelectionGate.SafetyBoundary)]
[InlineData(SelectionGate.Reversibility)]
public void failed_blocking_gate_cannot_be_outvoted(
    SelectionGate failedGate)
{
    var candidate = CandidateMother
        .InstrumentationToDigitalTwin()
        .WithFailure(failedGate);

    var decision = Evaluate(candidate);

    decision.Selected.Should().BeFalse();
    decision.BlockingFailures.Should().ContainSingle(
        x => x.Gate == failedGate);
}
```

The rejected test matters as much as the selected test.

Without it, a later scoring spreadsheet or generator could silently turn a blocking gate into a preference. The test preserves the meaning of the Chapter 3 selection rule.

EVIDENCE BOUNDARY

CandidateMother is a compile-oriented fixture. The actual assessment reasons, reviewers, source catalogue versions, and ADR hash must be retained when Chapter 4 creates the repository.

PRODUCER-OWNED PUBLIC FACT

The mapping exports evidence without exporting the aggregate

AlarmManagement decides that a flood occurred. The mapper translates that owner-local result into a stable public fact. It does not expose detector objects, ORM entities, active alarms, or an instruction for RootCause to open a case.

IMPLEMENTATION SHAPE 3-E — EXPLICIT CONTRACT MAPPING

```
internal static class AlarmFloodIntegrationMapper
{
    public static AlarmFloodDetectedV1 ToIntegrationEvent(
        AlarmFloodDetected domainEvent,
        CorrelationContext correlation)
    {
        ArgumentNullException.ThrowIfNull(domainEvent);

        return new(
            EventId: domainEvent.EventId.Value,
            AlarmGroupId: domainEvent.GroupId.Value,
            WindowStartedAt: domainEvent.Window.Start,
            WindowEndedAt: domainEvent.Window.End,
            ActivationCount: domainEvent.ActivationCount,
            DetectorVersion: domainEvent.DetectorVersion.Value,
            CorrelationId: correlation.CorrelationId,
            CausationId: correlation.CausationId,
            OccurredAt: domainEvent.OccurredAt);
    }
}
```

TEST SHAPE 3-F — PRIVATE STATE DOES NOT CROSS

```
[Fact]
public void CONTRACT_001_mapping_contains_only_public_evidence()
{
    var domainEvent = AlarmFloodMother.Detected();

    var message = AlarmFloodIntegrationMapper.ToIntegrationEvent(
        domainEvent,
        CorrelationMother.Fixed());

    message.ActivationCount.Should().Be(domainEvent.ActivationCount);

    typeof(AlarmFloodDetectedV1).GetProperties()
        .Select(x => x.Name)
        .Should().NotContain([
            "ActiveAlarms",
            "DetectorState",
            "SuppressionPolicy",
            "OpenRootCauseCase"
        ]);
}
```

CONTRACT WARNING

Reflection guards are a useful local tripwire, not a substitute for schema fixtures and producer/consumer compatibility tests. Chapters 19, 20, and 24 own those stronger proofs.

INDEPENDENT FAN-OUT

Subscriptions describe observation, never transferred authority

Audit, Compliance, and Reporting receive the same verdict through separate subscriptions. Each handler commits only its own local effect. The plan contains no global completion flag and no rollback callback into RootCause.

IMPLEMENTATION SHAPE 3-G — SUBSCRIBER CAPABILITY PLAN

```
public enum SubscriberCapability
{
    AppendEvidence,
    OpenComplianceReview,
    UpdateProjection
}

public sealed record VerdictSubscription(
    string Subscriber,
    string Queue,
    SubscriberCapability Capability,
    bool MayDelayAuthoritativeFlow);

public static readonly VerdictSubscription[] VerdictFanOut =
[
    new("Audit", "audit.verdicts.v1",
        SubscriberCapability.AppendEvidence, true),
    new("Compliance", "compliance.verdicts.v1",
        SubscriberCapability.OpenComplianceReview, true),
    new("Reporting", "reporting.verdicts.v1",
        SubscriberCapability.UpdateProjection, true)
];
```

TEST SHAPE 3-H — NO SUBSCRIBER OWNS ROOTCAUSE VERDICT

```
[Fact]
public void RC_0WN_001_fanout_contains_no_verdict_authority()
{
    VerdictFanOut.Should().OnlyContain(x =>
        x.Capability is SubscriberCapability.AppendEvidence
        or SubscriberCapability.OpenComplianceReview
        or SubscriberCapability.UpdateProjection);

    VerdictFanOut.Should().OnlyContain(
        x => x.MayDelayAuthoritativeFlow);
}

[Fact]
public void subscribers_use_independent_queues()
{
    VerdictFanOut.Select(x => x.Queue)
        .Should().OnlyHaveUniqueItems();
}
```

The second test is architectural: a shared competing-consumer queue would allow one capability to steal another capability's delivery. Independent queues preserve independent progress.

MIGRATION AND REVERSAL

One occurrence has exactly one authoritative writer

Cohort routing is evaluated before case mutation. Shadow evaluation may compare outcomes, but only the selected authoritative path receives permission to persist or publish a verdict.

IMPLEMENTATION SHAPE 3-I — COHORT ROUTE DECISION

```
public enum RootCausePath
{
    ModularAuthoritative,
    RemoteShadow,
    RemoteAuthoritative
}

public sealed record RouteDecision(
    AlarmOccurrenceId OccurrenceId,
    RootCausePath AuthoritativePath,
    bool RunRemoteShadow,
    string CohortVersion);

public sealed class SingleWriterGuard
{
    public void Validate(
        RouteDecision decision,
        IReadOnlyCollection<RootCausePath> writeRequests)
    {
        var authoritativeWrites = writeRequests
            .Count(x => x == decision.AuthoritativePath);

        if (authoritativeWrites != 1 || writeRequests.Count != 1)
            throw new DualWriterRejected(decision.OccurrenceId);
    }
}
```

TEST SHAPE 3-J — SHADOW CANNOT MUTATE

```
[Fact]
public void MIGRATION_one_writer_is_enforced_during_shadow()
{
    var route = new RouteDecision(
        OccurrenceId.Parse("occ-884"),
        RootCausePath.ModularAuthoritative,
        RunRemoteShadow: true,
        CohortVersion: "cohort-v3");

    var act = () => guard.Validate(route, [
        RootCausePath.ModularAuthoritative,
        RootCausePath.RemoteShadow
    ]);

    act.Should().Throw<DualWriterRejected>();
}
```

ROLLBACK MEANING

Rollback changes routing for new work and reconciles in-flight messages. It never deletes published facts, rewrites audit history, or enables both writers to make the transition appear easy.

CONTROLLED ROLLOUT

The slice advances through explicit states

IMPLEMENTATION SHAPE 3-K — ROLLOUT STATE MACHINE

```
public enum SlicePhase
{
    Baseline,
    ContractDark,
    RemoteShadow,
    CohortCutover,
    RemoteAuthoritative,
    ReconciliationRequired
}

public static readonly IReadOnlyDictionary<
    SlicePhase, SlicePhase[]> AllowedTransitions =
    new Dictionary<SlicePhase, SlicePhase[]>
    {
        [SlicePhase.Baseline] =
            [SlicePhase.ContractDark],
        [SlicePhase.ContractDark] =
            [SlicePhase.RemoteShadow, SlicePhase.Baseline],
        [SlicePhase.RemoteShadow] =
            [SlicePhase.CohortCutover, SlicePhase.Baseline],
        [SlicePhase.CohortCutover] =
            [SlicePhase.RemoteAuthoritative,
             SlicePhase.ReconciliationRequired],
        [SlicePhase.RemoteAuthoritative] =
            [SlicePhase.ReconciliationRequired],
        [SlicePhase.ReconciliationRequired] =
            [SlicePhase.Baseline, SlicePhase.RemoteAuthoritative]
    };
```

TEST SHAPE 3-L — REVERSAL CANNOT SKIP RECONCILIATION

```
[Fact]
public void published_remote_work_requires_reconciliation()
{
    AllowedTransitions[SlicePhase.RemoteAuthoritative]
        .Should().Equal(SlicePhase.ReconciliationRequired);
}

[Fact]
public void shadow_can_return_to_baseline_without_dual_write()
{
    AllowedTransitions[SlicePhase.RemoteShadow]
        .Should().Contain(SlicePhase.Baseline);
}
```

Technical material still required

Asset	Evidence still required
Selection gate	Versioned candidate catalogue, reviewer identity, ADR hash, rejected-gate evidence.
Contract mapping	Pinned contract project, serializer settings, JSON schema, fixtures and compatibility tests.
Fan-out plan	Declared queues/bindings, independent consumer tests, lag and outage evidence.
Single-writer guard	Real cohort store, concurrency protection, shadow side-effect prohibition and audit.
Rollout machine	Deployment integration, in-flight inventory, reconciliation runbook and rollback exercise.

REVISION RESULT

ADR-001 is now supported by executable selection, contract-mapping, fan-out, single-writer, and rollout shapes. The implementation remains evidence-gated exactly as the original chapter requires.

Expected failure states

The first slice is deliberately chosen because failure is informative without becoming a direct equipment-control action. The reference implementation must make the states below observable and recoverable. A green happy-path demonstration is necessary but not sufficient evidence.

Failure state	Truth that remains valid	Recovery obligation
AlarmManagement commits but cannot publish immediately	The flood was detected in AlarmManagement.	Outbox retains the fact; age and retries are visible; eventual publication preserves identity.
RootCause receives the same event repeatedly	One producer fact exists.	Consumer records the identity and creates at most one business effect.
RootCause commits but verdict publication is delayed	The verdict exists in RootCause.	Local durable publication resumes without recomputing or duplicating the verdict.
One verdict subscriber is unavailable	Other subscriber commits remain valid.	Backlog is isolated, visible, and replayable; no global rollback occurs.
Reporting rebuild is in progress	Operational owners retain source truth.	Projection exposes rebuilding/freshness state and resumes from a known position.
Poison message blocks a consumer	Producer fact remains immutable and other subscriptions may continue.	Quarantine preserves payload and metadata; operator repair and replay are audited.

What is deliberately not in the first slice

Scope discipline protects the value of the experiment. The following capabilities remain outside the first cut unless a later chapter introduces them with their own proof obligations:

- No extraction of Instrumentation, DigitalTwin, ReactorFleet, or simulation internals; their cadence, state, and coupling remain protected inside the modular foundation.
- No safety-like or equipment-control command crosses the selected event seam.
- No shared enterprise event model and no generic Evidence service are introduced to make unlike languages look uniform.
- No service mesh, multi-region platform, autoscaling claim, or production capacity claim is required to validate the first ownership boundary.
- No LLM-generated explanation changes case state, evidence acceptance, verdict, compliance decision, or operational command.
- No promise of exactly-once transport; the business effect is made idempotent at each owner instead.
- No UI screen is allowed to hide asynchronous lag behind a single invented global status.

PROTECTED FOUNDATION

Instrumentation, DigitalTwin, ReactorFleet, simulation internals, shared platform concerns, and safety-like control remain modular until their own language, data, failure, and reversal gates are satisfied.

What this chapter has proved

Chapter 3 has made the first binding implementation choice. AlarmManagement -> RootCause is the selected producer-consumer seam. RootCauseVerdictIssued.v1 expands the slice to Audit, Compliance, and Reporting as independent subscribers. The decision names ownership, local commitments, failure states, risks, migration stages, reversal conditions, rejected alternatives, and the additional technical material still required.

The chapter has not proved that the repository builds, that the contracts serialize compatibly, that the broker delivers, that local databases are isolated, or that failure recovery works. These claims begin

only when later chapters create executable artifacts and record their evidence. ADR-001 constrains that work; it does not substitute for it.

CHAPTER CLOSING

The first slice is selected because it makes partial success visible while keeping truth local, history durable, and reversal possible.

Chapter checkpoint

You are ready to continue when the selected seam can be defended without referring to service count, technology fashion, or a distributed transaction.

Question	Expected answer
Why is RootCause alone not the complete first slice?	It proves an independent runtime but not necessarily durable asynchronous transfer, redelivery, fan-out, partial success, or lag.
Why is AlarmManagement -> RootCause selected?	It crosses a clear language and decision boundary, supports isolated local commits, exposes recoverable distributed failure, avoids control authority, and can be reversed.
What does AlarmFloodDetected.v1 authorize?	Nothing by itself; it states a producer-owned fact that RootCause interprets under its own rules.
Why are Audit, Compliance, and Reporting independent subscribers?	Each owns a different local commitment and can be delayed or unavailable without invalidating the other owners' truth.
Why is there no distributed transaction?	Each service commits only its own data; delivery and reconciliation connect results without pretending simultaneous global state.
What makes the migration reversible?	A modular reference path, cohort routing, one active writer per case, durable history, and an explicit in-flight reconciliation runbook.
Which candidates remain protected?	Instrumentation, DigitalTwin, ReactorFleet, simulation internals, safety-like control, and other seams whose gates are not yet satisfied.
What technical material is still required?	Buildable topology, contract schemas and fixtures, serialization rules, inbox/outbox implementation, security, telemetry, failure experiments, and runbooks.

NEXT CHAPTER

Chapter 4 turns ADR-001 into a buildable repository and solution topology. It establishes the first compile gate, project boundaries, dependency rules, shared-building-block limits, contract placement, and test architecture without allowing the solution structure to become a second domain model.

PART I - CROSSING INTO DISTRIBUTION

4. Repository and Solution Topology

The repository becomes the first executable architecture boundary when ownership, dependency direction, public contracts, hosts, tests, and evidence all have an unambiguous home.

CHAPTER ARGUMENT

Organize source by bounded context, compose deployment through separate hosts, and make forbidden dependencies fail the build before they become runtime coupling.

ADR-001 selected AlarmManagement -> RootCause as the first distributed seam. That decision now needs a physical structure. If the repository is organized only around Web APIs, workers, databases, and shared libraries, the solution will teach developers that technology owns the architecture. If it is organized only around a beautiful domain diagram, it will hide the hosts, contracts, tests, and operational artifacts that make the boundary executable.

The topology in this chapter uses bounded contexts as the primary ownership axis and hosts as a separate deployment axis. AlarmManagement can remain composed into the modular runtime while it produces AlarmFloodDetected.v1. RootCause can be composed into an independent host that consumes the fact under its own invariants. The same context projects can later be hosted differently without copying domain logic or changing public meaning.

This distinction is the repository equivalent of the rule established in Chapter 3: a project boundary is not automatically a service boundary. Projects express compile-time ownership. Hosts express runtime composition. Public contracts express promises between owners. Tests and evidence express what has actually been proved.

A repository is executable policy

A directory tree does not create architecture by itself. It becomes policy when a developer can answer four questions from it and the build can reject at least the most dangerous wrong answers.

Repository question	Required answer	Enforcement surface
Who owns this business rule?	Exactly one bounded context and its Domain/Application projects.	Project placement, namespaces, architecture tests, code review.
Who may reference it?	Only the permitted inward dependency graph; never another context's internals.	ProjectReference audit and architecture tests.
Where does it run?	In one or more explicit host composition roots.	Host projects, deployment manifests, startup tests.
How is it observed externally?	Through producer-owned contracts, authorized APIs, traces, tests, and evidence records.	Contract projects, test suites, CI artifacts, evidence catalogue.

REPOSITORY RULE

If ownership can be changed by moving a registration line in a host, ownership was never encoded strongly enough.

Do not collapse four different boundaries

Boundary	What it controls	NEXUS-1 example
Namespace	Naming and discoverability inside an assembly.	Nexus1.RootCause.Domain.Cases
Project / assembly	Compile-time dependency and independently testable code surface.	Nexus1.RootCause.Domain
Host / process	Runtime composition, configuration, identity, health, and deployment unit.	Nexus1.RootCause.Host
Service boundary	Owned language, decisions, source data, contracts, failure, and operational responsibility.	RootCause as the verdict owner

A context can contain several projects. A host can compose one context or several modules. A service may use more than one process for API and background work when operational evidence justifies it. None of those structural choices changes the owner of a verdict. Keeping the terms separate prevents solution folders from becoming a second, contradictory domain model.

The context-first repository

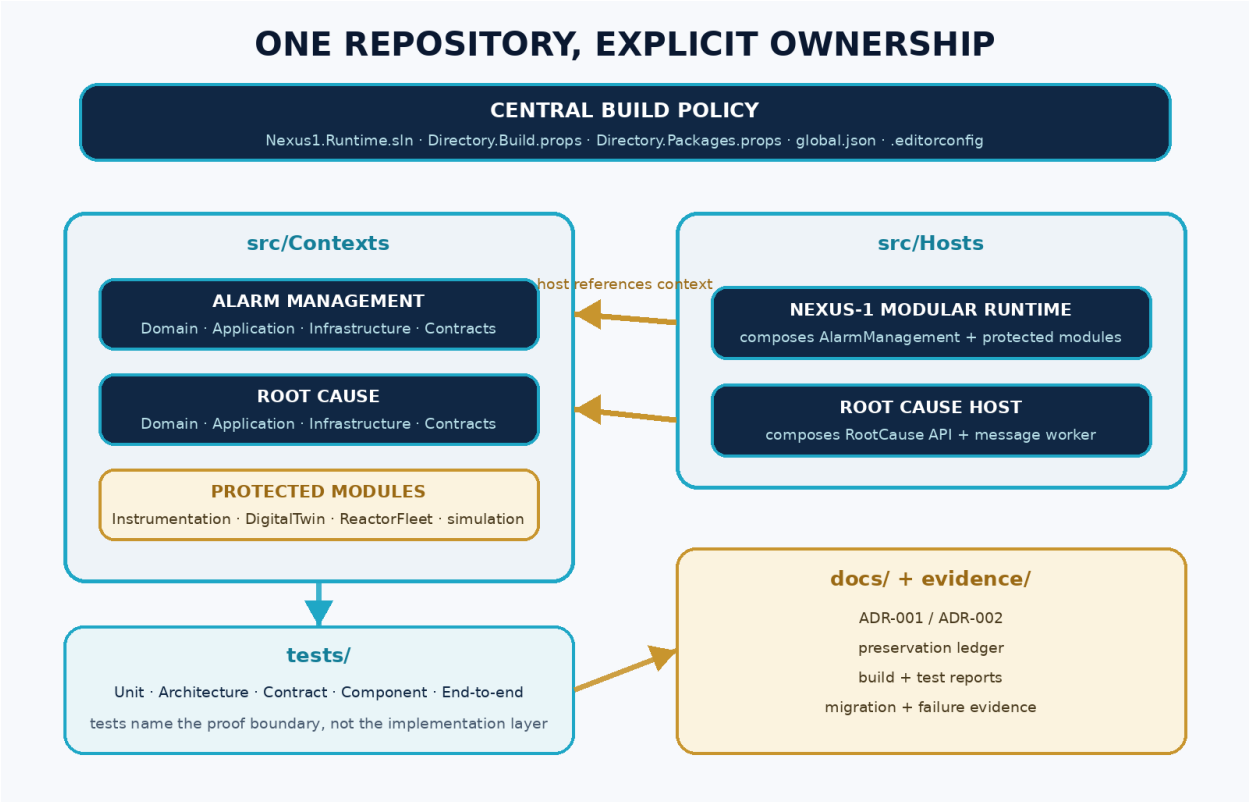


Figure 4.1 - Context projects encode ownership; hosts encode deployment composition; tests and evidence mirror the same boundaries.

The first repository contains only projects required by the modular reference path, the selected distributed slice, and the proof system around it. Audit, Compliance, Reporting, and RL Advisory receive projects when their first owned behaviour is implemented. Empty future-service projects create the appearance of progress while inviting shared abstractions to fill the vacuum.

REFERENCE REPOSITORY TREE - [NEW]

```
Nexus1.Runtime.sln
Directory.Build.props
Directory.Packages.props
global.json
.editorconfig

src/
  BuildingBlocks/
    Nexus1.BuildingBlocks.Domain/
    Nexus1.ServiceDefaults/
  Contexts/
    AlarmManagement/
      Nexus1.AlarmManagement.Domain/
      Nexus1.AlarmManagement.Application/
      Nexus1.AlarmManagement.Infrastructure/
      Nexus1.Contracts.AlarmManagement/
    RootCause/
      Nexus1.RootCause.Domain/
      Nexus1.RootCause.Application/
      Nexus1.RootCause.Infrastructure/
      Nexus1.Contracts.RootCause/
  Hosts/
    Nexus1.ModularRuntime/
    Nexus1.RootCause.Host/

tests/
  Nexus1.ArchitectureTests/
  Nexus1.AlarmManagement.UnitTests/
  Nexus1.RootCause.UnitTests/
  Nexus1.RootCause.ComponentTests/
  Nexus1.Contracts.ContractTests/
  Nexus1.DistributedSlice.EndToEndTests/

docs/
  adr/
  ledgers/
  runbooks/
  artifacts/evidence/
```

The tree deliberately places public contract assemblies beside their producer context while giving them names that do not imply access to producer internals. Consumers reference `Nexus1.Contracts.AlarmManagement`, never `Nexus1.AlarmManagement.Domain`. `RootCause` publishes through `Nexus1.Contracts.RootCause`. The physical folder communicates ownership; the assembly name communicates the permitted public surface.

Why context-first beats host-first

Concern	Context-first result	Host-first failure mode
Domain ownership	RootCause rules remain together regardless of hosting shape.	Rules drift between API, worker, and shared projects.
Migration	A context can move from modular host to remote host through composition.	Code is copied or rewritten merely to change deployment.
Testing	Unit and component suites align with one owner and one language.	Tests align with executables and miss cross-project decision leakage.
Contracts	Producer-owned public types are visible but isolated from internals.	DTOs are placed in a generic shared project and become ownerless.
Protected modules	Instrumentation and simulation remain coherent inside the modular runtime.	Every solution folder is mistaken for the next service to extract.

Project responsibilities

Project kind	Owns	Must not own
Domain	Aggregates, value objects, invariants, domain services, private domain events.	Transport, persistence provider, configuration, public integration schema.
Application	Use cases, ports, authorization intent, transaction intent, orchestration inside one context.	Database implementation, broker client, cross-context business decisions.
Infrastructure	Persistence, broker/HTTP adapters, outbox/inbox implementation, external integrations.	New domain rules or reinterpretation of another context's fact.
Contracts	Versioned public message and API types owned by the producer.	Aggregates, internal domain events, handlers, provider types, shared business logic.
Host	Composition root, process configuration, endpoints, health, telemetry, lifecycle.	Case invariants, verdict calculation, reusable business workflow.
Tests	Executable proof scoped to rule, boundary, contract, component, or complete slice.	Production behaviour that exists only to make a test convenient.

Dependency direction is a build rule

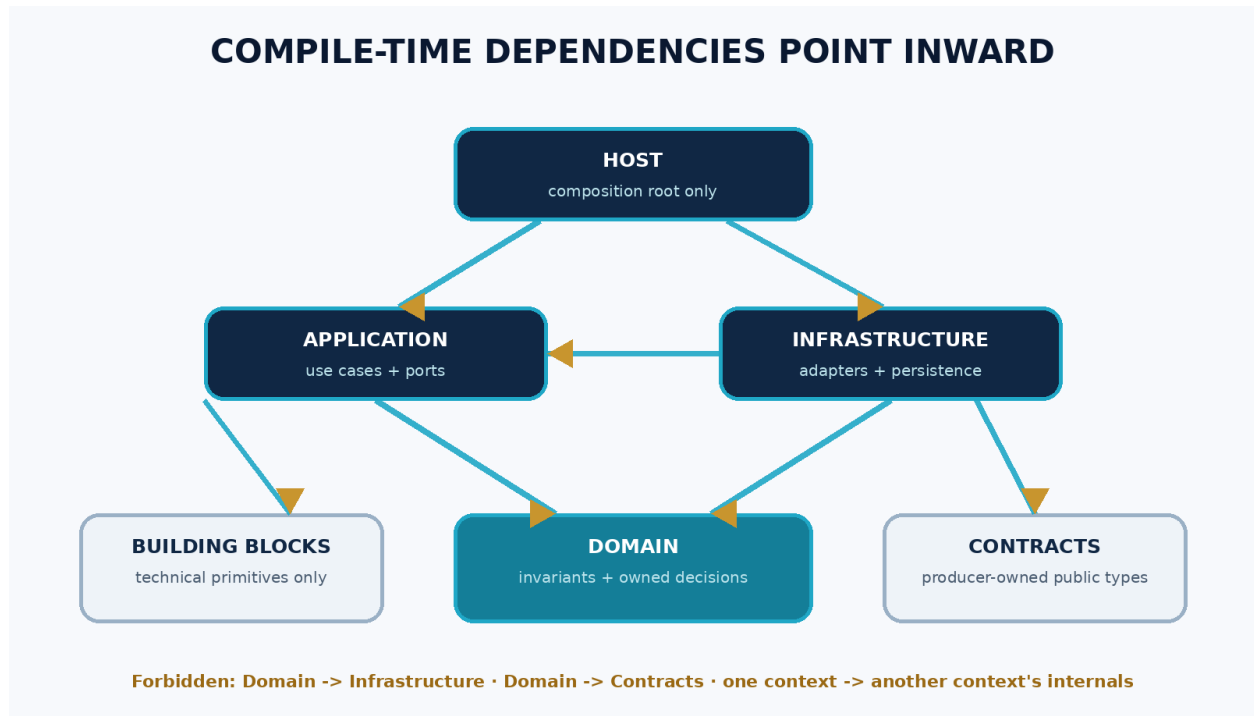


Figure 4.2 - Compile-time references point toward owned policy or explicit public contracts; runtime composition remains at the edge.

The arrow in Figure 4.2 means may contain a `ProjectReference` to. It does not mean data flow. At runtime a `Domain` object can produce an internal event that `Infrastructure` later maps to a public contract, even though `Domain` never references the `Contracts` assembly. This distinction prevents serialization concerns from defining the aggregate.

From	May reference	Forbidden reference
Domain	Minimal technical domain primitives, preferably the base class library only.	Application, Infrastructure, Host, public Contracts, another context.
Application	Its own Domain and narrow technical abstractions.	Concrete database/broker provider and another context's internals.
Infrastructure	Its own Application and Domain; required producer/consumer Contract assemblies.	Another context's Domain/Application/Infrastructure.
Contracts	Base class library and deliberately tiny serialization-neutral annotations if unavoidable.	Any Domain, Application, Infrastructure, Host, or provider package.
Host	Its context Application/Infrastructure, required Contracts, <code>ServiceDefaults</code> .	Business rules implemented in endpoints, startup, or hosted-service loops.
Test project	Only the production surface required by its proof level.	Broad solution references that erase which boundary is under test.

BLOCKING RULE

A context may consume another context's contract. It may never reference that context's Domain, Application, Infrastructure, or Host assembly.

Public contracts are producer-owned and dependency-poor

`AlarmFloodDetected.v1` belongs to `AlarmManagement` because `AlarmManagement` is the only context allowed to state that a flood was detected. `RootCause` consumes the contract; it does not co-own the type. `RootCauseVerdictIssued.v1` belongs to `RootCause` for the same reason. Keeping each contract

assembly dependency-poor allows consumers and contract tests to reference it without importing the producer's implementation model.

CONTRACT PROJECT BOUNDARY - [NEW]

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <AssemblyName>Nexus1.Contracts.AlarmManagement</AssemblyName>
    <RootNamespace>Nexus1.Contracts.AlarmManagement</RootNamespace>
  </PropertyGroup>

  <!-- No ProjectReference to Domain, Application,
        Infrastructure, Host, or another context. -->
</Project>
```

The contract project can contain the reference AlarmFloodDetectedV1 record introduced in Chapter 3, but not the Alarm aggregate, detector configuration object, persistence entity, message handler, or broker envelope. Chapters 19 and 20 will decide serialization and compatibility details. Chapter 4 fixes only placement and dependency law.

Map internal events at the edge of the owner

The internal domain event AlarmFloodDetected expresses a domain occurrence in the current model. The public AlarmFloodDetectedV1 contract expresses a long-lived promise to external consumers. A mapping component inside AlarmManagement Application or Infrastructure translates between them after the owning transaction determines what may be published. The Domain project remains unaware that RabbitMQ, JSON, or even a remote RootCause host exists.

Artifact	Lifetime	Allowed change	Consumer
AlarmFloodDetected domain event	Inside the owner and current model.	May evolve with the aggregate and internal handlers.	AlarmManagement only.
AlarmFloodDetectedV1 public contract	Across deployments and consumer upgrade windows.	Compatible extension or explicit new version.	RootCause and approved observers.
Broker envelope	Transport and operational boundary.	Versioned metadata policy; payload meaning unchanged.	Messaging infrastructure and all consumers.

Shared projects start nearly empty

The repository includes BuildingBlocks and ServiceDefaults because some technical repetition should be consistent. Their presence is not permission to create a shared domain. Chapter 5 examines this boundary in depth; Chapter 4 establishes the safe starting rule: a shared project may reduce mechanical duplication but may not own vocabulary, workflow, source data, or a cross-context decision.

Shared surface	Initially allowed	Rejected content
Nexus1.BuildingBlocks.Domain	Tiny identity/event primitives proven stable across existing modules.	Case, alarm, evidence, verdict, review, equipment, or simulation abstractions.
Nexus1.ServiceDefaults	Common host registration for health, telemetry, configuration validation, and resilient defaults.	Business authorization, domain workflow, service-specific endpoints or queues.
Test utilities	Fixture plumbing, deterministic clocks, container lifecycle helpers, assertion adapters.	Shared business fixtures that force contexts into one canonical model.
Contract assemblies	One producer's public types with minimal dependencies.	An enterprise-wide DTO catalogue or generic Events project owned by nobody.

EXTRACTION PRESSURE

When a shared abstraction needs a business word, move the word back to its owner or prove that the shared concept has a real lifecycle and authority of its own.

Hosts are composition roots, not application layers

Nexus1.ModularRuntime composes AlarmManagement and the protected modules that remain in-process. Nexus1.RootCause.Host composes the RootCause API, message consumer, persistence adapter, publication worker, security, health, and telemetry. The host knows which adapters are active. It does not decide whether a case can close.

ROOTCAUSE HOST COMPOSITION - [NEW]

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddNexusServiceDefaults(
    serviceName: "root-cause");
builder.Services.AddRootCauseApplication();
builder.Services.AddRootCauseInfrastructure(
    builder.Configuration);

var app = builder.Build();

app.MapNexusHealthEndpoints();
app.MapRootCauseEndpoints();

await app.RunAsync();
```

The extension methods are reference composition contracts, not compile-checked implementations. AddRootCauseApplication registers use-case handlers and policies. AddRootCauseInfrastructure selects database and message adapters. AddNexusServiceDefaults adds technical host behaviour. None is allowed to smuggle a RootCause verdict rule into a generic registration method.

One host first; split only for operational evidence

The reference topology begins with one RootCause host containing HTTP endpoints and background consumers. Separate API and Worker processes are introduced only when scaling, fault isolation, security, or deployment evidence requires them. Premature splitting adds configuration, identity, rollout, health, and observability surfaces without changing domain ownership.

Hosting choice	Use when	Do not claim
One RootCause host	The first slice needs one deployable owner with API and message processing.	That one process will always satisfy production scale or isolation.
Separate API and Worker hosts	Measured load, failure isolation, credentials, or rollout cadence differ materially.	That more executables create a better service boundary.
AlarmManagement in modular host	The strangler migration keeps the producer stable while RootCause becomes remote.	That AlarmManagement is not a bounded context or can share RootCause data.

Central build policy

Repository-wide build settings remove accidental variation. They do not remove context ownership. Directory.Build.props sets language and quality defaults; Directory.Packages.props centralizes package versions; global.json pins the SDK patch selected for the series baseline; .editorconfig defines formatting and analyzers. Service-specific behaviour remains in its project or host.

DIRECTORY.BUILD.PROPS BASELINE - [EXTEND][NEW]

```
<Project>
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
    <EnforceCodeStyleInBuild>true</EnforceCodeStyleInBuild>
    <Deterministic>true</Deterministic>
    <ContinuousIntegrationBuild
      Condition="'$(CI)' == 'true'">true</ContinuousIntegrationBuild>
  </PropertyGroup>
</Project>
```

The reference target remains net8.0 to preserve the series baseline. The exact SDK patch and every package version must be pinned and recorded by the executable repository; this manuscript does not invent a successful toolchain result. A future framework upgrade is a reviewed repository decision with build, test, compatibility, container, and operational evidence—not a silent edit to a global property.

TECHNICAL MATERIAL STATUS - [COMPILE REQUIRED]

The topology and snippets are normative reference material, but this environment does not contain the .NET SDK. The generated solution, exact SDK/package pins, restore result, compiler output, architecture-test runner, and CI logs remain blocking technical additions for the Chapter 4 evidence record.

The first compile gate

The gate is intentionally small. It proves that the repository is internally coherent enough to begin implementation. It does not prove messaging reliability, database ownership, security, deployment readiness, or production suitability.

CHAPTER 4 BUILD GATE - TARGET COMMAND SEQUENCE

```
dotnet --info
dotnet restore Nexus1.Runtime.sln
dotnet build Nexus1.Runtime.sln \
  --configuration Release \
  --no-restore
dotnet test tests/Nexus1.ArchitectureTests/Nexus1.ArchitectureTests.csproj \
  --configuration Release \
  --no-build
dotnet test Nexus1.Runtime.sln \
  --configuration Release \
  --no-build
dotnet format Nexus1.Runtime.sln \
  --verify-no-changes \
  --no-restore
```

Gate step	What success proves	What it does not prove
SDK information	The exact toolchain and runtime environment are captured.	That the chosen version is supported in every deployment target.
Restore	Declared packages resolve under the repository policy.	Supply-chain trust, runtime compatibility, or offline recovery.
Release build	Projects, references, generators, analyzers, and compiler rules succeed.	Correct domain behaviour or remote failure recovery.
Architecture tests	Selected forbidden dependency rules are executable and currently pass.	That every semantic ownership violation is mechanically detectable.
Test suite	Implemented tests pass in the recorded environment.	That absent tests or unmodelled failures are safe.
Format verification	The committed source matches repository formatting policy.	Code quality, design quality, or business correctness.

Compile evidence is a named artifact

A developer saying it builds is not the evidence. The evidence record includes commit identity, SDK and OS, command, start and finish time, exit code, warnings, test counts, architecture-rule version, and links to machine-readable reports. Local output is useful during development; the accepted record comes from a repeatable CI execution or an explicitly documented equivalent.

Evidence ID	Artifact	Acceptance rule
BUILD-001	Release restore/build log plus binary manifest.	Exit code zero, no compiler warnings, commit and SDK identified.
ARCH-001	Architecture-test result and permitted dependency graph.	All blocking reference rules pass; no ignored violation without ADR.
TEST-001	Unit/contract/component test reports available at this stage.	All discovered tests pass; skipped tests have an owner and reason.
FORMAT-001	Formatting/analyzer verification output.	No uncommitted mechanical changes required.

Architecture tests protect dependency law

Architecture tests inspect assemblies, project files, or both. Assembly inspection can detect namespace and type dependencies. Project-graph inspection detects references even when current code has not yet used them. The reference repository needs both views because a forbidden ProjectReference is latent coupling even before a type is imported.

Rule ID	Executable rule	Why it blocks
ARCH-DEP-001	Every Domain assembly references only approved base/domain-primitives assemblies.	Infrastructure or contract concerns would enter the invariant core.
ARCH-DEP-002	No context references another context's Domain, Application, Infrastructure, or Host.	Consumers would bypass public contracts and steal private meaning.
ARCH-DEP-003	Every Contracts assembly has no production ProjectReference.	Public schema would inherit implementation coupling.
ARCH-DEP-004	Host endpoints and workers delegate to Application; prohibited domain types do not escape publicly.	Composition code would become a second application layer.
ARCH-DEP-005	BuildingBlocks and ServiceDefaults contain no banned context vocabulary or context reference.	A shared project would become a hidden business owner.
ARCH-DEP-006	Test projects reference only the surfaces declared for their proof level.	Broad test access would hide accidental public APIs and ownership leakage.

TEST LIMITATION

Architecture tests can prohibit a reference. They cannot decide whether a conveniently generic type has stolen business meaning. Vocabulary and authority still require review against the preservation ledger.

The test topology mirrors proof boundaries

Test project	Primary proof	External dependencies
AlarmManagement.UnitTests	Flood detector determinism and owner rules.	None beyond deterministic fixtures.
RootCause.UnitTests	Case invariants, evidence acceptance, verdict and correction authority.	None beyond deterministic fixtures.
ArchitectureTests	Project/assembly dependency law and public-surface restrictions.	Repository metadata and built assemblies.
Contracts.ContractTests	Schema fixtures, version names, serialization and compatibility.	Serializer/schema tooling introduced in Chapters 19-20.
RootCause.ComponentTests	Host boundary with real persistence/messaging adapters under controlled conditions.	Disposable database/broker infrastructure added in later parts.
DistributedSlice.EndToEndTests	AlarmFloodDetected -> RootCause -> subscriber-observable outcome.	Complete local distributed environment and seeded identities.

The first Chapter 4 gate can run ArchitectureTests and any migrated unit tests. Contract, component, and end-to-end projects may initially contain explicit pending evidence records rather than empty green suites. A test project with zero discovered tests is not evidence and must fail or be excluded from the accepted gate with a documented reason.

Solution filters support feedback; the full solution owns truth

Nexus1.Runtime.sln is the authoritative build graph. Optional solution filters can shorten local feedback for AlarmManagement, RootCause, or architecture work, but CI always restores and builds the complete solution before accepting BUILD-001. A filter is a developer view, not a smaller architecture.

View	Contains	Use
Nexus1.Runtime.sln	All production and test projects in current scope.	Authoritative restore/build/test gate.
Nexus1.FastLoop.slnf	Domain, Application, unit, and architecture tests.	Rapid local policy and invariant feedback.
Nexus1.DistributedSlice.slnf	AlarmManagement/RootCause contexts, hosts, contracts, and slice tests.	Focused work on ADR-001 without protected modules.

CI stages preserve diagnostic clarity

1. Capture toolchain identity and validate repository cleanliness.
2. Restore once under central package policy.
3. Build the complete solution in Release with warnings treated as errors.
4. Run architecture and fast unit tests before provisioning external infrastructure.
5. Run contract and component suites when their required adapters exist.
6. Run the distributed-slice suite only after the local environment reports truthful readiness.
7. Publish test, coverage, dependency, and build evidence even when a later stage fails.

ADR-002 - Adopt a context-first solution topology

ADR field	ADR-002 record
Status	Accepted for the reference implementation; compile evidence remains pending.
Context	ADR-001 needs a repository that preserves modular ownership while allowing RootCause to run remotely and AlarmManagement to remain composed in the modular runtime during migration.
Decision	Organize production projects under bounded contexts; keep Domain, Application, Infrastructure, and producer Contracts separate; compose them through independent Host projects; mirror ownership in tests and evidence.
Dependency law	Dependencies point inward inside one context. Cross-context code references only producer-owned Contracts. Domain never references transport, persistence, Host, or public Contracts.
Consequences	More projects and explicit mapping; clearer ownership, replaceable hosts, isolated contracts, enforceable build rules, and safer migration.
Rejected alternatives	One project per service, host-first folder layout, layer-first global folders, generic Shared.Contracts, project per future service, and direct cross-context project references.
Reversal condition	Revise if the topology prevents a proved operational need, creates circular ownership, or architecture tests cannot express its blocking rules without widespread exceptions.
Evidence required	Generated solution/project files, exact toolchain pins, BUILD-001, ARCH-001, test discovery, dependency graph, and visual inspection of repository documentation.

Repository failure patterns

Pattern	Why it is dangerous	Correction
Shared.Contracts containing every DTO	No producer owns vocabulary or versioning; consumers couple transitively.	One dependency-poor contract assembly per producer context.
API/Worker/Database as top-level business folders	Technology becomes the ownership model and splits one decision across executables.	Place policy under Contexts and compose it from Hosts.
Domain references Infrastructure for convenience	Persistence or broker behaviour enters invariants and blocks deterministic tests.	Define an inward port or keep the decision entirely in Domain.
Host contains workflow decisions	Changing hosting changes business truth and makes alternate hosts inconsistent.	Delegate to Application and Domain; keep Host as composition root.
BuildingBlocks grows through copy-paste cleanup	Similarity is mistaken for shared meaning and creates coordinated releases.	Duplicate until stable semantics and ownership are proved.
Green empty test project	CI reports success without discovering evidence.	Fail on zero tests or record the suite explicitly as pending.
Solution filter becomes CI truth	Projects outside the filter silently stop building.	Build the authoritative complete solution in every acceptance pipeline.

STOP CONDITION

Do not implement the message broker or databases on top of a solution graph that still permits cross-context internals or cannot build as one authoritative repository.

CHAPTER 4 — CODE AUDIT

The repository gate becomes executable

A repository tree communicates intent. The assets below turn that intent into SDK selection, project references, failing architecture tests, repeatable commands, CI evidence, and startup proof.

VERIFICATION STATUS

The SDK and file shapes are current and executable in form. This environment has no dotnet executable, so BUILD-001, ARCH-001, TEST-001, FORMAT-001, and START-001 remain unproduced evidence.

Pin the toolchain before evaluating the graph

EXECUTABLE ASSET 4-A — GLOBAL.JSON

```
{
  "sdk": {
    "version": "8.0.423",
    "rollForward": "disable",
    "allowPrerelease": false
  }
}
```

EXECUTABLE ASSET 4-B — DIRECTORY.PACKAGES.PROPS

```
<Project>
  <PropertyGroup>
    <ManagePackageVersionsCentrally>true</ManagePackageVersionsCentrally>
    <CentralPackageTransitivePinningEnabled>
      true
    </CentralPackageTransitivePinningEnabled>
  </PropertyGroup>

  <ItemGroup>
    <!-- Every external package is declared here with an exact,
         reviewed version. Project files contain no Version. -->
    <PackageVersion Include="Microsoft.NET.Test.Sdk"
                   Version="$(MicrosoftNetTestSdkVersion)" />
    <PackageVersion Include="xunit"
                   Version="$(XunitVersion)" />
    <PackageVersion Include="xunit.runner.visualstudio"
                   Version="$(XunitRunnerVersion)" />
  </ItemGroup>
</Project>
```

The three version properties are supplied by the repository's reviewed version manifest, not environment variables or local machine state. The manuscript does not invent package versions that have not been restored together.

PINNING DECISION

rollForward: disable makes the Chapter 4 evidence specific to SDK 8.0.423. Updating the SDK creates a new reviewed evidence run; it does not silently reuse BUILD-001.

COMPILE-TIME OWNERSHIP

ProjectReference expresses the permitted graph

The project files below make the dependency law visible without depending on folder naming. Domain remains dependency-poor; Application points inward; Infrastructure implements owner-local ports and may consume producer-owned contracts; Host composes the outer graph.

EXECUTABLE ASSET 4-C — ROOTCAUSE PROJECT REFERENCES

```
<!-- Nexus1.RootCause.Application.csproj -->
<ItemGroup>
  <ProjectReference Include=
    "../Nexus1.RootCause.Domain/Nexus1.RootCause.Domain.csproj" />
</ItemGroup>

<!-- Nexus1.RootCause.Infrastructure.csproj -->
<ItemGroup>
  <ProjectReference Include=
    "../Nexus1.RootCause.Application/Nexus1.RootCause.Application.csproj" />
  <ProjectReference Include=
    "../Nexus1.RootCause.Domain/Nexus1.RootCause.Domain.csproj" />
  <ProjectReference Include=
    "../../AlarmManagement/Nexus1.Contracts.AlarmManagement/
    Nexus1.Contracts.AlarmManagement.csproj" />
</ItemGroup>

<!-- Nexus1.RootCause.Host.csproj -->
<ItemGroup>
  <ProjectReference Include=
    "../../Contexts/RootCause/Nexus1.RootCause.Application/
    Nexus1.RootCause.Application.csproj" />
  <ProjectReference Include=
    "../../Contexts/RootCause/Nexus1.RootCause.Infrastructure/
    Nexus1.RootCause.Infrastructure.csproj" />
  <ProjectReference Include=
    "../../BuildingBlocks/Nexus1.ServiceDefaults/
    Nexus1.ServiceDefaults.csproj" />
</ItemGroup>
```

EXECUTABLE ASSET 4-D — DEPENDENCY-POOR CONTRACT PROJECT

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <AssemblyName>Nexus1.Contracts.AlarmManagement</AssemblyName>
    <RootNamespace>Nexus1.Contracts.AlarmManagement</RootNamespace>
    <IsPackable>true</IsPackable>
    <GeneratePackageOnBuild>>false</GeneratePackageOnBuild>
  </PropertyGroup>

  <!-- Intentionally no ProjectReference and no provider package. -->
</Project>
```

GRAPH INVARIANT

RootCause Infrastructure may reference AlarmManagement Contracts. No RootCause project may reference AlarmManagement Domain, Application, Infrastructure, or Host.

ARCHITECTURE ENFORCEMENT

Inspect the project graph, not only compiled type usage

An unused forbidden ProjectReference is still coupling. The test reads every project file, normalizes references relative to repository root, classifies context ownership, and rejects cross-context internals before a type is imported.

EXECUTABLE ASSET 4-E — PROJECT GRAPH LOADER

```
public sealed record ProjectNode(
    string Name,
    string RelativePath,
    string Context,
    string Layer,
    IReadOnlyList<string> References);

public static IReadOnlyList<ProjectNode> LoadGraph(string repositoryRoot)
{
    return Directory.EnumerateFiles(
        repositoryRoot, "*.csproj", SearchOption.AllDirectories)
        .Select(path =>
        {
            var xml = XDocument.Load(path);
            var references = xml.Descendants("ProjectReference")
                .Select(x => Path.GetFullPath(
                    Path.Combine(
                        Path.GetDirectoryName(path)!,
                        x.Attribute("Include")!.Value)))
                .Select(x => Path.GetRelativePath(repositoryRoot, x))
                .ToArray();

            return ProjectClassifier.Classify(
                repositoryRoot, path, references);
        })
        .ToArray();
}
```

TEST SHAPE 4-F — CROSS-CONTEXT INTERNALS FAIL

```
[Fact]
public void ARCH_DEP_002_cross_context_internals_are_forbidden()
{
    var graph = LoadGraph(Repository.Root);
    var byPath = graph.ToDictionary(x => x.RelativePath);

    var violations =
        from source in graph
        from referencePath in source.References
        let target = byPath[referencePath]
        where source.Context != target.Context
        where target.Layer is
            "Domain" or "Application" or "Infrastructure" or "Host"
        select $"{source.Name} -> {target.Name}";

    violations.Should().BeEmpty(
        "cross-context code must use producer-owned Contracts");
}
```

A second test applies the same graph to assert that Contracts have no ProjectReference and Domain references only approved technical primitives. The permitted graph is stored as reviewed data so exceptions cannot hide in test code.

REPEATABLE LOCAL GATE

One script owns restore, build, tests, format, and evidence

EXECUTABLE ASSET 4-G — ENG/VERIFY.SH

```
#!/usr/bin/env bash
set -euo pipefail

repo_root="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
evidence_dir="${repo_root}/artifacts/evidence/chapter-04"
mkdir -p "${evidence_dir}"
cd "${repo_root}"

dotnet --info > "${evidence_dir}/dotnet-info.txt"
dotnet restore Nexus1.Runtime.sln \
  --locked-mode \
  2>&1 | tee "${evidence_dir}/restore.log"

dotnet build Nexus1.Runtime.sln \
  --configuration Release \
  --no-restore \
  2>&1 | tee "${evidence_dir}/build.log"

dotnet test Nexus1.Runtime.sln \
  --configuration Release \
  --no-build \
  --logger "trx;LogFileName=tests.trx" \
  --results-directory "${evidence_dir}/tests" \
  2>&1 | tee "${evidence_dir}/test.log"

dotnet format Nexus1.Runtime.sln \
  --verify-no-changes --no-restore \
  2>&1 | tee "${evidence_dir}/format.log"

python3 eng/assert_test_discovery.py \
  "${evidence_dir}/tests"
python3 eng/write_evidence_manifest.py \
  --source "${evidence_dir}" \
  --output "${evidence_dir}/manifest.json"
```

EXECUTABLE ASSET 4-H — ZERO-TEST GUARD

```
from pathlib import Path
import sys
import xml.etree.ElementTree as ET

files = list(Path(sys.argv[1]).rglob("*.trx"))
if not files:
    raise SystemExit("TEST-001 failed: no TRX results")

total = 0
for path in files:
    root = ET.parse(path).getroot()
    counters = next(
        node for node in root.iter()
        if node.tag.endswith("Counters"))
    total += int(counters.attrib["total"])

if total == 0:
    raise SystemExit("TEST-001 failed: zero tests discovered")

print(f"TEST-001 discovered {total} tests")
```

NO HIDDEN SUCCESS

set -euo pipefail and pipefail preserve command failures through tee. A green project with no TRX files or zero discovered tests is rejected explicitly.

CI ACCEPTANCE

The authoritative solution runs in a clean environment

EXECUTABLE ASSET 4-I — .GITHUB/WORKFLOWS/CHAPTER-04.YML

```
name: chapter-04-gate

on:
  pull_request:
  push:
    branches: [ main ]

permissions:
  contents: read

jobs:
  buildable-baseline:
    runs-on: ubuntu-latest
    timeout-minutes: 20
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-dotnet@v4
        with:
          global-json-file: global.json
          cache: true
          cache-dependency-path: "**/packages.lock.json"

      - name: Verify complete repository
        run: ./eng/verify.sh

      - name: Upload evidence on every outcome
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: chapter-04-evidence-${{ github.sha }}
          path: artifacts/evidence/chapter-04
          if-no-files-found: error
          retention-days: 30
```

CI uses the full solution and the repository's global.json. A solution filter may improve local feedback but cannot replace this gate. The evidence upload runs even after failure so diagnostics are retained instead of disappearing with the runner.

Supply-chain boundary

A production repository should pin action revisions according to organizational policy and use NuGet package-source mapping when public and private feeds coexist. Those controls are recorded separately from the architectural dependency graph.

ACCEPTANCE RULE

BUILD-001 belongs to one commit, one SDK, one lock-file set, and one CI environment. Re-running after any of those identities changes produces a new evidence record.

MACHINE-READABLE PROOF

Evidence is data, not a screenshot of a green check

EXECUTABLE ASSET 4-J — CHAPTER 4 EVIDENCE MANIFEST

```
{
  "evidenceId": "BUILD-001",
  "chapter": 4,
  "commitSha": "REQUIRED_AT_RUNTIME",
  "sdk": {
    "requested": "8.0.423",
    "resolved": "REQUIRED_AT_RUNTIME"
  },
  "environment": {
    "runnerImage": "REQUIRED_AT_RUNTIME",
    "osDescription": "REQUIRED_AT_RUNTIME"
  },
  "commands": [
    { "name": "restore", "exitCode": null, "logSha256": null },
    { "name": "build", "exitCode": null, "logSha256": null },
    { "name": "test", "exitCode": null, "logSha256": null },
    { "name": "format", "exitCode": null, "logSha256": null }
  ],
  "testSummary": {
    "total": null,
    "passed": null,
    "failed": null,
    "skipped": null
  },
  "accepted": false
}
```

TEST SHAPE 4-K — EVIDENCE CANNOT ACCEPT ITSELF

```
public static void Validate(BuildEvidence evidence)
{
    if (evidence.Commands.Any(x =>
        x.ExitCode is null || x.LogSha256 is null))
        throw new EvidenceIncomplete("BUILD-001");

    if (evidence.Commands.Any(x => x.ExitCode != 0))
        throw new EvidenceRejected("BUILD-001");

    if (evidence.TestSummary.Total <= 0)
        throw new EvidenceRejected("TEST-001");

    if (evidence.Sdk.Requested != evidence.Sdk.Resolved)
        throw new EvidenceRejected("SDK-001");
}
```

PLACEHOLDER DISCIPLINE

REQUIRED_AT_RUNTIME and **null** are deliberate pre-run values. The manifest is rejected until the gate replaces them with observed identities and hashes; prose cannot flip accepted to true.

START ONE SERVICE

Compilation is followed by an honest readiness smoke test

EXECUTABLE ASSET 4-L — STARTUP/READINESS SMOKE TEST

```
public sealed class RootCauseStartupTests
    : IClassFixture<WebApplicationFactory<Program>>
{
    private readonly HttpClient _client;

    public RootCauseStartupTests(
        WebApplicationFactory<Program> factory)
    {
        _client = factory.WithWebHostBuilder(builder =>
            builder.UseEnvironment("Chapter4Test"))
            .CreateClient();
    }

    [Fact]
    public async Task START_001_host_starts_and_reports_readiness()
    {
        using var response = await _client.GetAsync(
            "/health/ready");

        response.StatusCode.Should().Be(HttpStatusCode.OK);

        var body = await response.Content
            .ReadFromJsonAsync<HealthResponse>();
        body!.Status.Should().Be("Ready");
        body!.Service.Should().Be("root-cause");
    }
}
```

The Chapter4Test environment may substitute only technical adapters whose absence belongs to later Parts. It may not bypass application registration, authorization wiring, endpoint mapping, configuration validation, or the architecture graph. Chapter 11 later strengthens readiness with real dependency semantics.

Official-source validation

Checked 26 July 2026: the official .NET 8 download page lists SDK 8.0.423 with runtime 8.0.29. Microsoft documents `global.json` as the SDK-selection mechanism and Central Package Management through `Directory.Packages.props` with `ManagePackageVersionsCentrally` enabled.

Sources: <https://dotnet.microsoft.com/download/dotnet/8.0> ; <https://learn.microsoft.com/dotnet/core/tools/global-json> ; <https://learn.microsoft.com/nuget/consume-packages/central-package-management>

REVISION RESULT

Chapter 4 now supplies the missing executable toolchain, graph, architecture-test, gate-script, CI, evidence-manifest, and startup shapes. Because dotnet is unavailable here, every runtime result remains explicitly pending rather than represented as passed.

Implementation sequence for the repository

1. Create the repository root policy files and pin the series toolchain without adding service technology.
2. Add the authoritative solution and the two context project groups required by ADR-001.
3. Move or reference the modular AlarmManagement behaviour without changing its domain result.
4. Place RootCause invariants and application use cases behind their own project boundaries.
5. Create dependency-poor AlarmManagement and RootCause contract assemblies and map internal events at the owner edge.
6. Add ModularRuntime and RootCause Host composition roots with no business logic.
7. Add architecture tests for project graph, assembly references, namespaces, and shared-project vocabulary.
8. Migrate unit tests and prove that the full solution discovers them.
9. Run the Chapter 4 gate in a pinned .NET environment and publish BUILD-001 / ARCH-001 / TEST-001 / FORMAT-001.
10. Only then begin the reusable golden-path service template in Chapter 7 and the infrastructure expansions in later parts.

What this chapter has proved

Chapter 4 has defined where ownership, composition, public contracts, shared technical policy, tests, and evidence belong. It has established ADR-002, the permitted dependency graph, the reference repository tree, the target build sequence, the initial architecture rules, and the distinction between modular hosting and service ownership.

It has not proved that a generated .NET solution compiles in this environment. It has not selected or exercised database and broker adapters, created production containers, validated identity, or demonstrated failure recovery. Those remain explicit evidence obligations. The chapter is complete as an architectural draft and incomplete as runtime evidence until the named compile artifacts exist.

CHAPTER CLOSING

The repository is trustworthy when moving a host cannot move a decision, referencing a contract cannot expose an aggregate, and violating ownership produces a failed build or a visible review exception.

Chapter checkpoint

You are ready to continue when every project has one responsibility, every allowed reference can be defended, and the missing compile evidence is named rather than implied.

Question	Expected answer
Why is the repository context-first?	Because bounded contexts own language and decisions; hosts are replaceable runtime composition and must not become ownership boundaries.
Where does AlarmManagement run first?	Its context projects remain composed in the modular runtime while publishing the selected fact through an owner-controlled seam.
Where does RootCause run first?	Its context projects are composed by an independent RootCause host containing API and background processing.
How may RootCause consume AlarmManagement?	By referencing Nexus1.Contracts.AlarmManagement and never AlarmManagement Domain, Application, Infrastructure, or Host.
Why does Domain not reference public Contracts?	The aggregate owns internal meaning; mapping to a long-lived serialized promise occurs at the owner edge.

Question	Expected answer
What may be shared?	Small stable technical primitives, service defaults, and test plumbing—never ownerless business vocabulary or workflow.
What does the compile gate prove?	Toolchain, restore, project graph, analyzers, compilation, discovered tests, and selected architecture rules in a recorded environment.
What remains unproved?	The actual build result here, runtime adapters, messaging/data reliability, security, deployment, scale, and failure recovery.

NEXT CHAPTER

Chapter 5 narrows the shared surface. It classifies what can safely live in BuildingBlocks and ServiceDefaults, shows how shared packages create release coupling, and defines the tests and ownership questions that keep 'shared little' from becoming a distributed monolith.

PART I - CROSSING INTO DISTRIBUTION

5. Shared Nothing, Shared Little

Independent services share standards and explicit promises; they do not share ownerless business truth, correctness mechanisms, or a release train disguised as a library.

CHAPTER ARGUMENT

Share technical policy when it reduces mechanical variation. Share business meaning only through an owner-controlled contract. Keep source truth, decisions, failure, and upgrade authority local.

Chapter 4 created `Nexus1.BuildingBlocks.Domain`, `Nexus1.ServiceDefaults`, producer-owned `Contracts` projects, central build policy, and a context-first solution topology. Those names are useful and dangerous. Without a stricter rule, every repeated class can migrate into `BuildingBlocks`, every public message can migrate into `Shared.Contracts`, and every host can depend on the same convenience package until the repository builds only as one coordinated unit.

Shared nothing is not a demand that every service run on separate physical hardware or reimplement the base class library. It is an ownership rule. No service relies on another service's mutable tables, transaction, cache entry, filesystem, in-process singleton, deployment order, or private object model to preserve its own correctness. Shared little is the complementary engineering rule: common technical policy is allowed only when its semantics are narrow, its owner is named, its version can change safely, and consumers retain upgrade control.

The result is deliberately asymmetric. `AlarmManagement` owns `AlarmFloodDetected.v1` and `RootCause` owns `RootCauseVerdictIssued.v1`, even though several consumers reference those contracts. Platform engineering can own `ServiceDefaults`, but `ServiceDefaults` cannot decide whether a `RootCause` case closes. A database server or broker cluster may be physically shared in the Phase-0 environment, but credentials, logical data ownership, queues, quotas, health, and failure evidence remain separated.

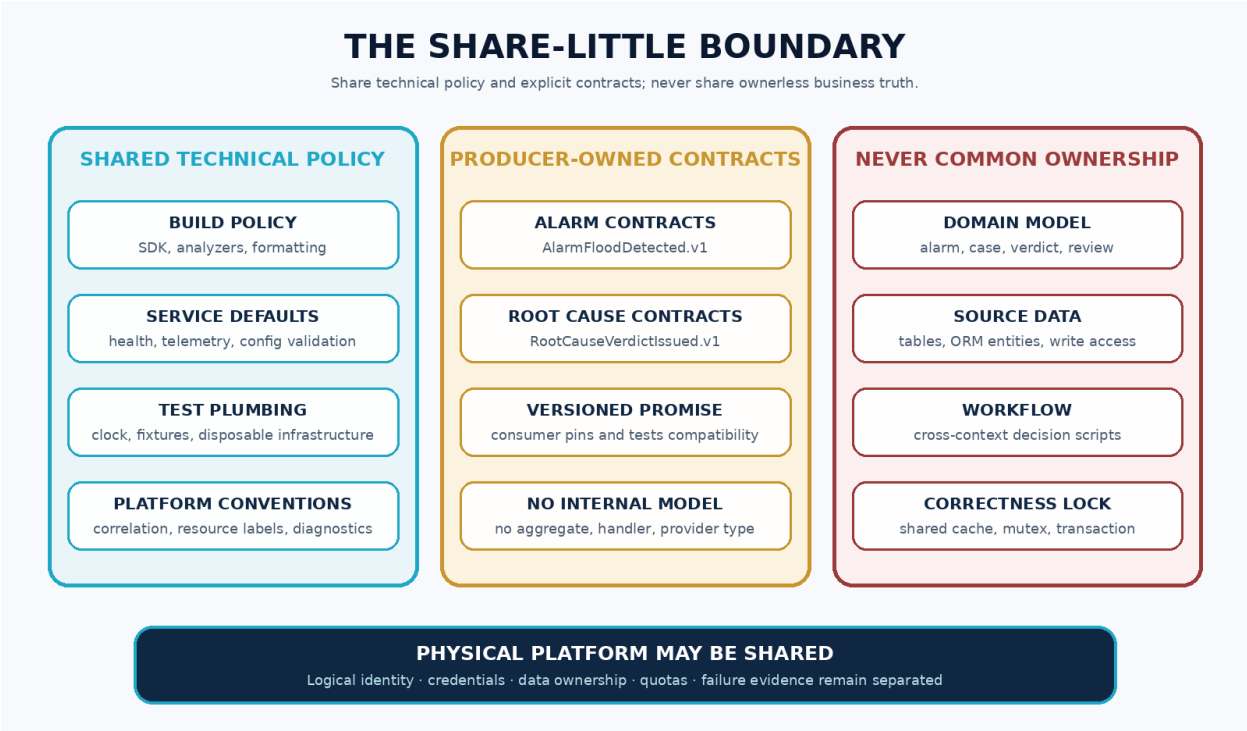
Shared nothing protects five forms of independence

Independence	Protected rule	Hidden violation
Semantic	Each context owns its vocabulary and decision meaning.	A common model defines Alarm, Case, Evidence, and Review for everyone.
Data	Each owner writes its own source truth and exposes contracts for observation.	Direct reads or writes to another service's tables become required for correctness.
Transactional	A local commit succeeds without a transaction spanning another owner.	A shared unit of work or distributed lock decides whether several services may commit.
Failure	One service can be delayed or unavailable without corrupting another owner's committed truth.	A shared cache, filesystem, library service, or deployment precondition becomes a single correctness dependency.
Release	A consumer can choose when to adopt a compatible contract or technical package.	One package update forces all services to rebuild, retest, and deploy together.

PLAIN EXPLANATION

Sharing is unsafe when a local decision stops being valid merely because another team, package, store, or deployment did not change at the same time.

The allowed shared surface



Physical capability	May be shared	Must remain isolated
RabbitMQ cluster	Nodes, operator tooling, base monitoring, certificate authority.	Virtual host or namespace, credentials, exchanges/queues, quotas, consumer lag, dead-letter evidence.
Telemetry backend	Collector and analysis platform.	Service identity, resource attributes, sampling accountability, alert ownership, sensitive-data policy.
Identity provider	Authentication authority, keys, token format, federation.	Service authorization policy, user capability checks, privileged-action evidence.
Configuration/secrets platform	Delivery mechanism, encryption, rotation tooling.	Configuration ownership, validation, least privilege, rollout and rollback decision.
CI/CD platform	Runners, artifact store, policy engine, deployment mechanism.	Build evidence, release approval, service rollout cadence, rollback authority.

A Phase-0 demonstrator may use one container host, one broker cluster, and one relational server to control cost. The architecture remains distributed only if service identities and logical ownership are enforceable and the failure catalogue does not pretend that physical co-location has removed dependency risk.

Shared code creates more than compile-time coupling

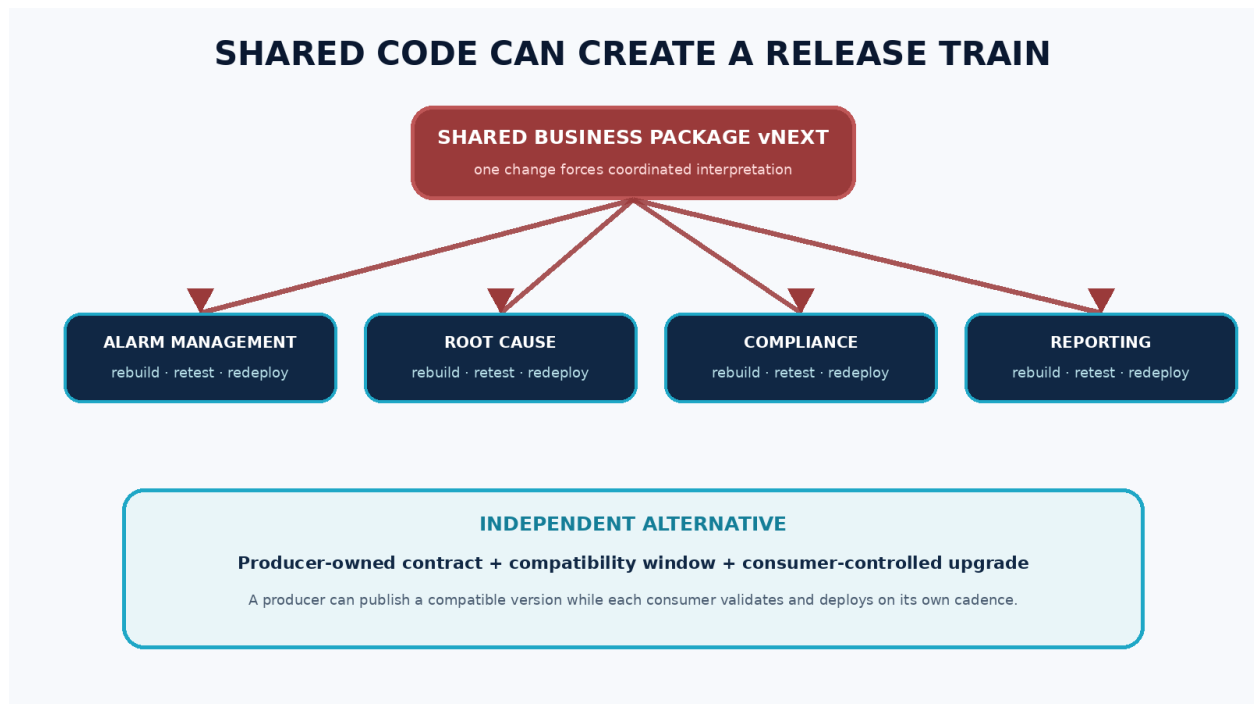


Figure 5.2 - A shared business package can turn independently deployable contexts into one coordinated release train.

A library can be small in lines of code and large in coordination cost. The cost appears when a package changes interpretation, requires a framework update, changes serialization, alters failure behaviour, or carries a transitive dependency. If every service must adopt the package before any service can deploy, the repository contains a distributed monolith even if each process has its own database.

Coupling introduced	Observable symptom	Required control
Semantic coupling	One type or helper gives the same word a forced meaning in several contexts.	Keep business vocabulary with its owner; integrate through translation.
Build coupling	A package update breaks several service compilations at once.	Stable API, automated compatibility, pinned versions, small dependency graph.
Release coupling	A change is safe only when producers and consumers deploy together.	Backward/forward compatibility window and consumer-controlled adoption.
Runtime coupling	A common remote helper or cache must be available for local correctness.	Local fallback or local ownership; shared runtime service cannot hide inside a library API.
Data coupling	A shared repository or ORM model becomes the path to source truth.	Owner API/event, consumer-owned projection, separate credentials.
Operational coupling	One global configuration or retry default causes correlated failure.	Service-level overrides, bounded defaults, blast-radius and failure experiments.

DRY BOUNDARY

Do not remove duplication across bounded contexts until the duplicated code has the same meaning, owner, lifecycle, failure mode, and release cadence—not merely the same shape.

Duplicate first, extract after evidence

Two contexts may each contain a Result type, validation helper, identifier wrapper, or retry classification. That duplication is cheaper than a premature shared abstraction because each owner can change meaning locally. Extraction becomes justified only after repeated evolution shows that the semantics remain aligned and the resulting package can be owned without making consumer releases synchronous.

Signal	Keep local	Candidate for technical sharing
Same syntax	Yes. Similar code alone proves nothing about shared meaning.	No.
Same semantics today	Usually; observe how each copy evolves under real use.	Only after owner and compatibility policy are named.
Same semantics across several changes	Re-evaluate duplication cost and consumer needs.	Possible if failure and upgrade remain independent.
Contains domain words or decisions	Yes, with the bounded-context owner.	No technical shared package; use a producer-owned contract if external.
Mechanical host/test plumbing	Local copy is acceptable while policy is immature.	Strong candidate after two implementations prove the stable seam.

The share decision gate

A proposed shared surface must answer every blocking question. The burden of proof belongs to the extraction, not to the teams that prefer local code.

Gate	Pass condition	Fail response
Named owner	One team or role owns API, versions, security, support, and retirement.	Keep local; an ownerless library is not shared infrastructure.
Semantic neutrality	The surface contains technical policy and no bounded-context decision.	Return business vocabulary to its context or expose a producer contract.
Independent adoption	Consumers can remain on a supported prior version while another upgrades.	Redesign compatibility or accept explicit coordinated-release status.
Local failure	A defect or outage can be contained and diagnosed per consumer.	Remove runtime dependency or define a platform SLO and fallback that preserves correctness.
Dependency budget	Public API and transitive packages are small, reviewed, and justified.	Split, simplify, or keep the helper local.
Replaceability	A consumer can wrap, replace, or stop using the surface without migrating business truth.	The surface has acquired hidden ownership; redesign the boundary.
Evidence	Pack/consume, compatibility, architecture, failure, and upgrade tests have named records.	The proposal remains experimental and cannot become a default dependency.

The initial BuildingBlocks inventory

Nexus1.BuildingBlocks.Domain starts smaller than its name suggests. It exists only for primitives already proven stable across the modular baseline. If no such primitive survives the gate, the correct initial implementation is an empty project removed from the solution—not a catalogue waiting for business concepts.

Candidate	Initial decision	Reason
IDomainEvent marker	CONDITIONAL ALLOW	Can support uniform in-process dispatch if it carries no context vocabulary or transport rule.
Occurred-at value / clock port	CONDITIONAL ALLOW	Technical time semantics can be shared if deterministic tests and clock ownership remain explicit.
Entity / AggregateRoot base class	KEEP LOCAL	Lifecycle, equality, event collection, and persistence assumptions often differ by context.
Result<T> and error hierarchy	KEEP LOCAL	Generic result types quickly standardize business failure meaning and public error mapping.
Strongly typed generic identifier	KEEP LOCAL FIRST	Serialization, persistence, generation, and equality policy require evidence before extraction.
Money, Evidence, Alarm, Case, Verdict, Review	PROHIBITED	These are business concepts with a bounded-context owner, not technical primitives.

MAXIMUM INITIAL DOMAIN PRIMITIVE - [EXTEND][NEW]

```
namespace Nexus1.BuildingBlocks.Domain;

public interface IDomainEvent
{
    DateTimeOffset OccurredAt { get; }
}
```

Even this marker is accepted only if the existing modular code already demonstrates common semantics. It must not contain correlation, causation, broker headers, serialization attributes, retry policy, or public event version. Those belong to different boundaries. Contexts remain free to define richer private domain-event interfaces locally.

ServiceDefaults standardizes host mechanics

Nexus1.ServiceDefaults is a technical composition package. It can register health infrastructure, telemetry resources, correlation propagation, configuration validation, secure defaults, and common diagnostic endpoints. It does not own service-specific readiness, authorization, queue names, retry budgets, or business degradation policy.

SERVICEDEFAULTS PUBLIC SURFACE - [EXTEND][NEW]

```
using Microsoft.Extensions.DependencyInjection;

public sealed record NexusServiceDefaultsOptions(
    string ServiceName,
    bool EnableTracing = true,
    bool EnableMetrics = true,
    bool ValidateConfigurationOnStart = true);

public static class NexusServiceDefaultsExtensions
{
    public static IServiceCollection AddNexusServiceDefaults(
        this IServiceCollection services,
        NexusServiceDefaultsOptions options)
    {
        ArgumentNullException.ThrowIfNull(services);
        ArgumentNullException.ThrowIfNull(options);

        // Register technical host mechanisms only.
        return services;
    }
}
```

Concern	Shared default may provide	Service must still decide
Health	Endpoint plumbing, response format, registration mechanism.	Which owned dependencies make this instance ready or degraded.
Telemetry	Resource naming, propagator, exporter wiring, baseline meters.	Business spans, cardinality, sensitive fields, alerts, sampling exceptions.
Configuration	Binding and validate-on-start mechanism.	Required settings, value constraints, rollout and fallback policy.
Resilience	Safe technical hooks and telemetry for retries/timeouts.	Per-operation timeout, idempotency, retry eligibility, degradation behaviour.
Security	Authentication plumbing and secure transport defaults.	Local authorization, service/user capability, privileged repair policy.

DEFAULT RULE

A shared default can supply a mechanism. The owning service must supply the meaning and the risk decision.

Test plumbing is shared; business fixtures are not

A deterministic clock, disposable infrastructure lifecycle, log capture adapter, and eventual-assertion helper can reduce mechanical test duplication. A RootCauseCaseBuilder, ValidAlarmFlood factory, ComplianceScenario, or enterprise-wide ExpectedVerdict fixture embeds business meaning and stays with its context or end-to-end scenario owner.

Contract packages are not a shared domain model

Nexus1.Contracts.AlarmManagement can be consumed by RootCause, tests, and approved observers. That wide use does not make the contract co-owned. AlarmManagement controls the meaning, version, deprecation, and compatibility window. Consumers control when they adopt a supported version and how they translate the public fact into local language.

Distribution form	Strength	Risk / required evidence
Binary package	Simple typed consumption and build-time discovery.	Can force framework/package coupling; prove dependency poverty and multi-version compatibility.
Schema artifact + generated type	Language-neutral public description and consumer-local generation.	Generator/runtime differences; preserve schema fixtures and generated-code review.
Schema only + local type	Maximum consumer independence and explicit translation.	More duplication; compatibility and semantic conformance need strong tests.
Copied producer DTO source	Fast initial experiment.	No version provenance or upgrade signal; rejected for accepted contracts.
Generic Shared.Contracts package	Convenient single reference.	Ownerless vocabulary, transitive coupling, coordinated release; prohibited.

Chapter 4 chose dependency-poor producer assemblies as the reference shape. Chapters 19 and 20 will compare distribution forms, envelopes, serialization, compatibility, and version retirement. Chapter 5 adds a blocking requirement: whichever form is chosen, a consumer must not import the producer's implementation dependencies or require a coordinated deployment for a compatible change.

Compatibility is stronger than matching versions

Producer action	Compatible consumer expectation	Forbidden assumption
Publish optional additive field	Older consumers continue; newer consumers can use it after independent upgrade.	Every consumer deploys before the producer emits the field.
Deprecate a field	Usage is measured; support window and removal version are explicit.	Silence means no consumer depends on it.
Change meaning	Publish a new version or a new fact with migration evidence.	Same field name permits semantic reinterpretation.
Fix producer bug	Corrective fact/process preserves history and contract meaning.	Consumers silently recalculate producer truth from shared code.
Retire a version	All known consumers prove migration or explicit retirement.	Monorepo compilation proves deployed consumers have upgraded.

Central package management is not lockstep deployment

Directory.Packages.props can pin versions for a reproducible repository build. It must not erase compatibility windows. Contract tests retain fixtures for supported previous versions, and service deployment manifests record the version actually deployed. The authoritative question is not whether the monorepo currently compiles against one version; it is whether supported producer and consumer versions can coexist at runtime.

MONOREPO WARNING

One commit graph improves visibility. It does not prove independent deployability unless compatibility tests and deployment records exercise version skew.

Never share source truth for query convenience

Reporting, the BFF, and operational tools will often need data from several contexts. The safe response is an owner API, public fact, or consumer-owned projection with freshness evidence. A read-only cross-service SQL query still creates schema, availability, security, and migration coupling. Read-only permission prevents writes; it does not prevent ownership leakage.

Need	Safe path	Rejected shortcut
Reporting timeline	Project owner-issued facts into Reporting-owned storage with a watermark.	Join AlarmManagement and RootCause tables.
BFF case view	Compose APIs/projections and expose partial or stale state.	Use shared cache as the global current status.
RootCause reference data	Replicate a versioned owner-issued reference set or request it explicitly.	Foreign key into the producer database.
Audit evidence	Append public facts and privileged-action records under Audit ownership.	Database trigger across service schemas.
Migration comparison	Time-boxed, logged, read-only seam with removal gate and evidence.	Permanent compatibility view owned by nobody.

The Shared Surface Ledger

Every accepted shared surface receives a stable ledger entry. The ledger distinguishes technical packages from producer contracts and platform capabilities, names the owner, defines consumers and compatibility, and records how the surface can be retired. A dependency that is not in the ledger remains local or experimental.

Field	Required content
Surface ID and category	Stable identifier; standard, technical package, producer contract, platform, or replicated data.
Owner	Team/role responsible for meaning, API, security, releases, incidents, and retirement.
Allowed consumers	Named scope or explicit eligibility rule; not 'all services' by default.
Semantic boundary	What the surface means and which business words/decisions it must never acquire.
Version and compatibility	Supported versions, skew window, deprecation signal, migration and retirement rules.
Failure boundary	Expected failure modes, fallback, observability, blast radius, and service-level responsibility.
Evidence	Architecture, pack/consume, compatibility, security, failure, and independent-upgrade records.
Exit condition	How a consumer can replace/remove the surface and who approves breaking retirement.

Surface ID	Category / owner	Initial status
SHR-STD-001	Repository build policy / Architecture maintainers	ACCEPT: common standard; no runtime dependency.
SHR-PKG-001	Nexus1.ServiceDefaults / Platform	CONDITIONAL: public surface and pack/consume evidence pending.
SHR-PKG-002	Nexus1.BuildingBlocks.Domain / Architecture maintainers	RESTRICT: IDomainEvent only if modular baseline proves stable semantics.
SHR-CON-ALM-001	AlarmFloodDetected.v1 / AlarmManagement	ACCEPT OWNER; compatibility and distribution evidence pending.
SHR-CON-RC-001	RootCauseVerdictIssued.v1 / RootCause	ACCEPT OWNER; compatibility and distribution evidence pending.
SHR-PLT-001	Phase-0 broker/database/telemetry platform / Platform	ACCEPT PHYSICAL SHARING; logical isolation evidence pending.
SHR-DATA-001	Cross-service source database / none	PROHIBITED.
SHR-DOM-001	Generic business shared kernel / none	PROHIBITED until a genuine shared owner and lifecycle are proved.

Stable shared-surface rules

Rule ID	Blocking rule
SHR-OWN-001	Every shared surface has one named owner for meaning, version, failure, and retirement.
SHR-SEM-001	A technical shared package contains no bounded-context vocabulary, decision, or workflow.
SHR-REL-001	A consumer is compatible to share data with another consumer if and only if

Rule ID	Blocking rule
SHR-DATA-001	No service depends on another service's mutable source data for correctness.
SHR-FAIL-001	Shared platform/package failure has a bounded, observable effect and does not silently change business meaning.
SHR-AUTH-001	Shared authentication plumbing never replaces authorization by the service that owns the action.
SHR-DEP-001	Shared packages have an explicit public and transitive dependency budget with no provider leakage.
SHR-EVID-001	Acceptance requires compatibility and independent-upgrade evidence, not only successful compilation.

Architecture tests for the shared surface

Test	Assertion
Shared-project references	BuildingBlocks and ServiceDefaults reference no context, contract, host, persistence, or broker provider project.
Vocabulary boundary	Banned context namespaces and reviewed business terms do not appear in technical shared public APIs.
Contracts dependency graph	Each producer Contracts project references no production project and exposes no internal aggregate/provider type.
Provider leakage	Public signatures in shared packages contain no EF, broker-client, HTTP-client, telemetry-exporter, or container type.
Consumer pinning	Services declare explicit supported package/contract versions; unbounded floating versions fail policy.
Zero-test / compatibility gate	Accepted shared packages have discovered pack/consume and compatibility tests; empty green suites fail.

ADR-003 - Adopt a share-little policy

ADR field	ADR-003 record
Status	Accepted for the reference architecture; package and compatibility evidence remains pending.
Context	The context-first repository needs limited technical reuse without allowing BuildingBlocks, ServiceDefaults, contracts, or physical platform resources to become hidden business owners or coordinated-release mechanisms.
Decision	Share central standards, narrow technical packages, producer-owned contracts, and physical platform capabilities under a Shared Surface Ledger. Prohibit shared mutable business state and an ownerless business shared kernel.
Consequences	Some code remains duplicated; mappings are explicit; packages need ownership/versioning/evidence; consumers retain local models and upgrade authority.
Rejected alternatives	Global Shared.Contracts, one enterprise domain model, shared ORM entities/repositories, one cross-service UnitOfWork, copy every common helper into a package, and forbid all physical platform sharing.
Exception	A business shared kernel requires a real joint owner, one lifecycle, explicit release coordination status, small public surface, and evidence that translation would be more dangerous than coupling.
Reversal condition	Revise when repeated evolution proves stable shared semantics and independent adoption, or when a shared surface creates recurring incidents, security exposure, or release coupling.
Evidence required	Shared Surface Ledger, dependency graph, package API manifest, pack/consume tests, supported-version matrix, version-skew test, failure experiment, and independent deployment record.

Risk register for sharing

Risk	Early control	Evidence owner
BuildingBlocks becomes a shared domain	Vocabulary ban, public API review, no context references, extraction gate.	Architecture tests + ADR review.
Contract package leaks implementation	Dependency-poor assembly and public-signature scan.	Contract chapters 19-20.
ServiceDefaults changes service meaning	Mechanism-only API; local options and readiness/authorization decisions.	Golden-path and service-foundation chapters.
Central package update creates lockstep	Pinned versions, compatibility window, independent-upgrade test, release fan-out metric.	CI/release evidence.
Shared platform hides correlated failure	Logical isolation, per-service health/lag/quota, platform failure experiment.	Platform and failure-lab chapters.
Read-only data access becomes permanent	Migration-only seam with access log, deadline, owner, and removal gate.	Data ownership/migration chapters.
Authentication library becomes authorization owner	Local policy tests and explicit service/user capability mapping.	Security chapters.
Duplicated technical code drifts dangerously	Compare observed changes; extract only after semantics and owner stabilize.	Shared Surface Ledger review.

RISK RULE

The cost of duplication is visible code. The cost of premature sharing is hidden coordination. Measure both before extracting a common package.

CHAPTER 5 - CODE AUDIT

The shared surface becomes a package, not a promise

The policy now receives a concrete package boundary. The package remains dependency-poor, versioned, and replaceable; it contains no NEXUS-1 business vocabulary.

EVIDENCE STATUS

These listings are compile-oriented reference assets. This environment has no dotnet SDK, so package, API-compatibility, and version-skew results remain PENDING. No green result is claimed.

EXECUTABLE ASSET 5-A - NEXUS1.SERVICEDEFAULTS.CSPROJ

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <PackageId>Nexus1.ServiceDefaults</PackageId>
    <Version>1.0.0</Version>
    <IsPackable>true</IsPackable>
    <GeneratePackageOnBuild>false</GeneratePackageOnBuild>
    <EnablePackageValidation>true</EnablePackageValidation>
    <PackageValidationBaselineVersion>0.9.0</PackageValidationBaselineVersion>
    <RestorePackagesWithLockFile>true</RestorePackagesWithLockFile>
  </PropertyGroup>

  <ItemGroup>
    <FrameworkReference Include="Microsoft.AspNetCore.App" />
    <PackageReference Include="Microsoft.CodeAnalysis.PublicApiAnalyzers"
      PrivateAssets="all" />
    <InternalsVisibleTo Include="Nexus1.ServiceDefaults.Tests" />
  </ItemGroup>
</Project>
```

The ASP.NET Core shared-framework reference supplies the narrow host abstraction without adding a provider package to the nuspec. The API analyzer is build-only and centrally versioned. If later mechanics need OpenTelemetry, RabbitMQ, EF Core, or an exporter, the provider adapter belongs in a separate package. The baseline version activates package validation against the last supported public API.

EXECUTABLE ASSET 5-B - MECHANISM-ONLY PUBLIC API

```
namespace Nexus1.ServiceDefaults;

public sealed record NexusServiceDefaultsOptions(
    string ServiceName,
    bool EnableTracing = true,
    bool EnableMetrics = true);

public interface INexusServiceModule
{
    void Register(IServiceCollection services);
}

public static class NexusServiceDefaultsExtensions
{
    public static IServiceCollection AddNexusServiceDefaults(
        this IServiceCollection services,
        NexusServiceDefaultsOptions options,
        params INexusServiceModule[] modules)
    {
        ArgumentException.ThrowIfNullOrWhiteSpace(options.ServiceName);
        foreach (var module in modules)
        {
            module.Register(services);
        }
        return services;
    }
}
```

SHARED SURFACE BOUNDARY

Public API and dependency poverty are reviewed as data

A package may compile while leaking provider types or acquiring a large transitive graph. The build therefore emits two reviewable artifacts: the public API text and the resolved package graph.

EXECUTABLE ASSET 5-C - PUBLIC API APPROVAL FILES

```
// PublicAPI.Shipped.txt
Nexus1.ServiceDefaults.INexusServiceModule
Nexus1.ServiceDefaults.INexusServiceModule.Register(
    Microsoft.Extensions.DependencyInjection.IServiceCollection! services) -> void
Nexus1.ServiceDefaults.NexusServiceDefaultsExtensions
Nexus1.ServiceDefaults.NexusServiceDefaultsOptions

// PublicAPI.Unshipped.txt
# New public members must be reviewed here before moving to Shipped.
```

EXECUTABLE ASSET 5-D - PACKAGE EVIDENCE SCRIPT

```
#!/usr/bin/env bash
set -euo pipefail

project="src/BuildingBlocks/Nexus1.ServiceDefaults/Nexus1.ServiceDefaults.csproj"
out="{1:-artifacts/share-surface}"
mkdir -p "$out/packages"

dotnet restore "$project" --locked-mode
dotnet build "$project" -c Release --no-restore
dotnet pack "$project" -c Release --no-build -o "$out/packages"

dotnet list "$project" package --include-transitive \
    > "$out/SHARE-API-001.dependencies.txt"

unzip -p "$out/packages/Nexus1.ServiceDefaults.1.0.0.nupkg" \
    Nexus1.ServiceDefaults.nuspec \
    > "$out/SHARE-API-001.nuspec.xml"

test -s "$out/SHARE-API-001.dependencies.txt"
test -s "$out/SHARE-API-001.nuspec.xml"
```

BLOCKING REVIEW

The evidence fails acceptance if the nuspec contains an unexplained dependency or if a provider type appears in the approved API. A package file alone is not SHARE-API-001.

ARCHITECTURE ENFORCEMENT

Provider leakage and business vocabulary fail before packaging

EXECUTABLE ASSET 5-E - REFLECTION-BASED PUBLIC SURFACE TEST

```
public sealed class SharedSurfaceApiTests
{
    private static readonly string[] ForbiddenNamespaces =
    [
        "Microsoft.EntityFrameworkCore",
        "RabbitMQ.Client",
        "OpenTelemetry.Exporter",
        "Nexus1.AlarmManagement",
        "Nexus1.RootCause"
    ];

    [Fact]
    public void SHR_DEP_001_public_api_exposes_no_provider_or_context_type()
    {
        var assembly = typeof(NexusServiceDefaultsOptions).Assembly;

        var exposed = assembly.ExportedTypes
            .SelectMany(type => type.GetMembers(BindingFlags.Public |
                BindingFlags.Instance |
                BindingFlags.Static))
            .SelectMany(PublicSignatureTypes)
            .Where(type => type.Namespace is not null)
            .Select(type => type.Namespace!)
            .Distinct();

        exposed.Should().NotContain(ns =>
            ForbiddenNamespaces.Any(prefix =>
                ns.StartsWith(prefix, StringComparison.Ordinal)));
    }
}
```

TEST SUPPORT - ENUMERATE TYPES USED BY A PUBLIC MEMBER

```
private static IEnumerable<Type> PublicSignatureTypes(MemberInfo member)
{
    return member switch
    {
        MethodInfo m => m.GetParameters().Select(p => p.ParameterType)
            .Append(m.ReturnType),
        PropertyInfo p => [p.PropertyType],
        FieldInfo f => [f.FieldType],
        EventInfo e when e.EventHandlerType is not null =>
            [e.EventHandlerType],
        Type t => [t],
        _ => []
    };
}
```

A second lexical rule scans exported type/member names for the reviewed business vocabulary list. Reflection is the safety net, not a substitute for API review: generic arguments, base types, implemented interfaces, and attributes must also be traversed in the repository implementation.

SCOPE LIMIT

Microsoft.Extensions.DependencyInjection.IServiceCollection is the intentionally accepted host seam. The allow-list is explicit; it must not grow merely to make a failing package pass.

CONSUMER BOUNDARY

Two clean consumers prove the package does not need repository source

EXECUTABLE ASSET 5-F - CLEAN CONSUMER PROJECT

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <RestorePackagesWithLockFile>true</RestorePackagesWithLockFile>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Nexus1.ServiceDefaults" Version="$(DefaultsVersion)" />
  </ItemGroup>
</Project>
```

EXECUTABLE ASSET 5-G - TWO-CONSUMER PACK/CONSUME PROOF

```
#!/usr/bin/env bash
set -euo pipefail

feed="$(pwd)/artifacts/share-surface/packages"
for consumer in AlarmConsumer RootCauseConsumer; do
  work="$(mktemp -d)"
  dotnet new web -n "$consumer" -o "$work/$consumer" --no-restore

  dotnet add "$work/$consumer/$consumer.csproj" package \
    Nexus1.ServiceDefaults --version 1.0.0 \
    --source "$feed" --no-restore

  dotnet restore "$work/$consumer/$consumer.csproj" \
    --source "$feed" --source https://api.nuget.org/v3/index.json
  dotnet build "$work/$consumer/$consumer.csproj" \
    -c Release --no-restore

  test ! -e "$work/src"
done
```

The consumers are created outside the repository tree and reference only the produced package. A hidden ProjectReference, imported repository props file, or source-relative path therefore becomes visible.

EVIDENCE RULE

SHARE-PACK-001 is accepted only with command, SDK identity, package hash, both consumer logs, and retained lock files. Re-running from a developer cache without those records is not proof.

INDEPENDENT ADOPTION

Version skew is the test that distinguishes reuse from lockstep

A monorepo build exercises one selected version. The compatibility matrix intentionally runs supported versions side by side so that one consumer can upgrade without moving the other.

EXECUTABLE ASSET 5-H - VERSION MATRIX

```
public static IEnumerable<object[]> SupportedSkew()
{
    yield return ["AlarmConsumer", "0.9.0"];
    yield return ["RootCauseConsumer", "1.0.0"];
}

[Theory]
[MemberData(nameof(SupportedSkew))]
public async Task SHARE_SKEW_001_supported_consumer_starts(
    string consumer, string defaultsVersion)
{
    var result = await ProcessRunner.DotNet(
        "run", "--project", $"tests/Consumers/{consumer}",
        $"-p:DefaultsVersion={defaultsVersion}",
        "--no-restore");

    result.ExitCode.Should().Be(0, result.StandardError);
    result.StandardOutput.Should().Contain("Application started");
}
```

CI ASSET - PACKAGE/VERSION-SKEW JOB

```
share-surface:
  strategy:
    matrix:
      consumer: [AlarmConsumer, RootCauseConsumer]
      defaults: ["0.9.0", "1.0.0"]
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-dotnet@v4
      with:
        global-json-file: global.json
    - run: dotnet restore tests/Consumers/${{ matrix.consumer }} --locked-mode
    - run: >
        dotnet test tests/SharedSurface.Tests -c Release --no-restore
        --filter SHARE_SKEW_001
        -p:Consumer=${{ matrix.consumer }}
        -p:DefaultsVersion=${{ matrix.defaults }}
        --logger "trx;LogFileName=skew.trx"
    - uses: actions/upload-artifact@v4
      if: always()
      with:
        name: skew-${{ matrix.consumer }}-${{ matrix.defaults }}
        path: "**/skew.trx"
```

The matrix is illustrative until both package versions exist in the retained feed. A breaking change either fails validation or requires a new major version and an explicit coordinated-release ADR.

SEMANTIC OWNERSHIP

A public fact is translated; it is never imported as the local model

EXECUTABLE ASSET 5-I - CONSUMER-OWNED TRANSLATION

```
// Producer-owned public fact
public sealed record AlarmFloodDetectedV1(
    Guid AlarmId,
    DateTimeOffset DetectedAt,
    int ActiveAlarmCount,
    string CorrelationId);

// RootCause-owned command and translation
internal sealed record OpenInvestigation(
    CaseId CaseId,
    EvidenceSeed Seed,
    CausationId CausationId);

internal static class AlarmFloodTranslation
{
    public static OpenInvestigation Translate(AlarmFloodDetectedV1 fact) =>
        new(
            CaseId.FromExternal("alarm", fact.AlarmId),
            EvidenceSeed.FromAlarmCount(
                fact.ActiveAlarmCount, fact.DetectedAt),
            CausationId.Parse(fact.CorrelationId));
}
```

RootCause does not pass the producer DTO into its aggregate. Translation is the explicit semantic boundary: AlarmManagement owns the fact; RootCause owns what that fact permits locally.

EXECUTABLE ASSET 5-J - TRANSLATION CONTRACT TEST

```
[Fact]
public void producer_fact_is_translated_without_importing_producer_policy()
{
    var fact = new AlarmFloodDetectedV1(
        Guid.Parse("8e87c88f-f49a-4f16-aad8-65c807486681"),
        DateTimeOffset.Parse("2026-07-26T10:00:00Z"),
        ActiveAlarmCount: 17,
        CorrelationId: "corr-0418");

    var command = AlarmFloodTranslation.Translate(fact);

    command.Seed.Count.Should().Be(17);
    command.CausationId.Value.Should().Be("corr-0418");
    typeof(OpenInvestigation).Assembly.Should()
        .NotBe(typeof(AlarmFloodDetectedV1).Assembly);
}
```

CHAPTER OWNERSHIP

This test demonstrates translation shape only. Serialization fixtures, envelope rules, additive fields, version retirement, and broker delivery remain the responsibility of Chapters 19-20.

MACHINE-READABLE GOVERNANCE

The Shared Surface Ledger can reject an ownerless dependency

EXECUTABLE ASSET 5-K - SHARED-SURFACES.YAML

```
surfaces:
- id: SHR-PKG-001
  category: technical-package
  name: Nexus1.ServiceDefaults
  owner: Platform
  allowedConsumers: [AlarmManagement, RootCause]
  semanticBoundary: "Host mechanics only; no business decisions"
  supportedVersions: ["0.9.0", "1.0.0"]
  failureBoundary: "Local startup/readiness diagnosis"
  exitCondition: "Consumer can replace registration modules"
  evidence:
    api: artifacts/share-surface/SHARE-API-001.json
    pack: artifacts/share-surface/SHARE-PACK-001.json
    skew: artifacts/share-surface/SHARE-SKEW-001.json
  accepted: false
```

EXECUTABLE ASSET 5-L - LEDGER ACCEPTANCE GUARD

```
public static void ValidateForAcceptance(SharedSurface surface)
{
    Require(surface.Id, nameof(surface.Id));
    Require(surface.Owner, nameof(surface.Owner));
    Require(surface.SemanticBoundary, nameof(surface.SemanticBoundary));
    Require(surface.FailureBoundary, nameof(surface.FailureBoundary));
    Require(surface.ExitCondition, nameof(surface.ExitCondition));

    if (surface.AllowedConsumers.Count == 0)
        throw new LedgerViolation(surface.Id, "No allowed consumer");

    foreach (var evidence in surface.Evidence.Values)
        if (!File.Exists(evidence) || new FileInfo(evidence).Length == 0)
            throw new LedgerViolation(surface.Id,
                $"Missing evidence: {evidence}");

    if (surface.Accepted && surface.SupportedVersions.Count < 2)
        throw new LedgerViolation(surface.Id,
            "Acceptance requires a supported skew window");
}
```

Official-source validation

Checked 26 July 2026. Microsoft documents dotnet pack as the package-production command, packages.lock.json plus locked restore for repeatability, and package validation against a baseline version. Those mechanisms support the evidence shape; they do not decide NEXUS-1 ownership.

Sources: <https://learn.microsoft.com/dotnet/core/tools/dotnet-pack> ;
<https://learn.microsoft.com/nuget/consume-packages/package-references-in-project-files> ;
<https://learn.microsoft.com/dotnet/fundamentals/apicompat/package-validation/overview>

REVISION RESULT

Chapter 5 now shows the package, API/dependency evidence, architecture rejection, clean consumers, version skew, semantic translation, and ledger guard. Runtime evidence remains explicitly pending.

Technical material still required

The chapter defines the policy and reference public surfaces. It does not yet prove that ServiceDefaults or BuildingBlocks can be packed, consumed by multiple hosts, upgraded independently, or rejected safely when incompatible. It also does not select the final contract distribution form.

TECHNICAL MATERIAL STATUS - [PACK][COMPATIBILITY REQUIRED]

Add an executable package project, API manifest, dependency report, pack/consume tests against at least two consumers, supported-version fixtures, version-skew CI job, vulnerability/license evidence, and an independent upgrade experiment. Contract serialization/distribution details remain owned by Chapters 19-20.

Evidence ID	Required artifact	Blocking result
SHARE-API-001	Public API and transitive dependency manifest for each accepted shared package.	Business/provider type or unexplained dependency appears.
SHARE-PACK-001	Deterministic package build and consume test from clean projects.	Package requires repository source layout or hidden project reference.
SHARE-COMP-001	Supported previous/current version compatibility matrix.	Compatible producer change breaks a supported consumer.
SHARE-SKEW-001	Two services run different supported package/contract versions in one environment.	Coordinated upgrade is required without explicit ADR status.
SHARE-FAIL-001	Failure/rollback experiment for a shared default or platform dependency.	Failure changes business meaning silently or lacks local diagnosis.

Implementation sequence

1. Inventory every current cross-project dependency and classify it as standard, technical package, producer contract, platform, replicated data, or prohibited state.
2. Create the Shared Surface Ledger and assign one owner, consumers, compatibility rule, failure boundary, and exit condition to each accepted surface.
3. Remove business vocabulary and provider types from BuildingBlocks and ServiceDefaults public APIs.
4. Keep domain helpers local until repeated evolution demonstrates stable shared semantics.
5. Add architecture tests for shared-project references, public signatures, banned vocabulary, provider leakage, and explicit version pinning.
6. Build ServiceDefaults and the conditional domain primitive as real packages in the Chapter 7 golden-path work.
7. Run pack/consume, version-skew, and independent-upgrade tests before a package becomes the default for additional services.
8. Give producer contracts their own compatibility evidence and never move them into a generic enterprise contract assembly.
9. Exercise physical platform failure while proving logical data, identity, health, quota, and recovery ownership per service.
10. Review release fan-out and shared-surface growth at every Part boundary; split or retire surfaces whose coordination cost increases.

What this chapter has proved

Chapter 5 has established the semantic boundary around sharing. It has classified standards, technical packages, producer contracts, platform capabilities, replicated data, and prohibited mutable state. It has defined the initial BuildingBlocks and ServiceDefaults limits, the Shared Surface Ledger, stable SHR rules, architecture tests, evidence IDs, risk controls, and ADR-003.

It has not proved package compilation, contract serialization, version skew, independent deployment, or platform isolation in a running environment. Those gaps are explicit and blocking. The architecture now knows what sharing must mean before the service template makes sharing easy.

CHAPTER CLOSING

Share the mechanism only when ownership, compatibility, failure, and exit remain explicit.
Share the fact through its producer. Never share the decision by accident.

Chapter checkpoint

You are ready to continue when every common dependency can be classified, owned, versioned, failed, replaced, and tested without appealing to convenience alone.

Question	Expected answer
What does shared nothing mean here?	No service depends on another owner's mutable state, transaction, private model, or coordinated release to preserve local correctness.
What may be shared little?	Central standards, narrow technical packages, producer-owned contracts, test plumbing, and physical platforms with logical isolation.
Why are producer contracts not a shared domain model?	One producer owns their meaning/version; consumers translate them into local language and control adoption.
When is duplication preferable?	When semantics, lifecycle, failure, ownership, or release cadence have not yet proved stable across contexts.
What belongs in BuildingBlocks initially?	At most tiny proven technical/domain markers; no aggregate base assumptions or business vocabulary.
What can ServiceDefaults decide?	Technical host mechanisms and safe defaults, never service readiness meaning, authorization, retry eligibility, or business degradation.
Can physical infrastructure be shared?	Yes, if logical identity, credentials, data, quotas, health, failure, and recovery evidence remain service-specific.
What evidence is still missing?	Package build/consume, API/dependency manifest, version compatibility/skew, independent upgrade, and shared-platform failure evidence.

NEXT CHAPTER

Chapter 6 defines when a NEXUS-1 boundary deserves to be called a completed service. It turns preservation, topology, sharing, data, contract, security, observability, reliability, operations, migration, and evidence obligations into one Definition of Done without confusing a deployed process with a production-ready service.

PART I - CROSSING INTO DISTRIBUTION

6. Definition of Done for a NEXUS-1 Service

A process becomes an evidenced service only when its ownership, behaviour, data, contracts, trust, failure, operations, migration, and residual risk can be demonstrated—not when its container first starts.

CHAPTER ARGUMENT

Done is a gate backed by evidence and acceptance. It is never a weighted score that allows strong documentation to cancel a broken invariant, shared database, missing authorization, or unrecoverable failure.

Part I has moved from architectural inheritance to an executable boundary plan. Chapter 1 separated decisions from runtime proof. Chapter 2 created preservation rules. Chapter 3 selected AlarmManagement -> RootCause. Chapter 4 defined the repository and compile boundary. Chapter 5 constrained sharing. The natural temptation is now to call RootCause a service because its box, contracts, projects, host, and responsibilities have names.

That claim would be premature. The generated repository has not been compiled in this environment. RootCauseDb does not yet exist. AlarmFloodDetected.v1 has not been serialized or delivered. No identity has been authorized, no duplicate has been absorbed, no outbox has recovered, no trace has crossed the seam, no operator has followed a runbook, and no migration cohort has been rolled back. The architecture is coherent enough to implement. The service is not done.

This chapter defines the evidence required before stronger language is permitted. The Definition of Done is intentionally layered. A boundary can be an accepted candidate, a buildable project graph, a locally executable component, an evidenced distributed slice, or an operable Phase-0 service. Production readiness is a separate review that requires operating history and organizational commitments the demonstrator cannot manufacture.

Do not collapse candidate, process, and service

Term	Minimum meaning	What the term does not imply
Service candidate	Architecture gates justify a possible independent owner and boundary.	Executable host, isolated data, delivery, security, observability, or operations.
Project / assembly	A compile-time unit with an explicit dependency surface.	Independent deployment, data, failure, or team ownership.
Host / process	A runnable composition root with configuration and lifecycle.	Correct service boundary, durability, authorization, recovery, or support.
Extracted service	One owner runs independently with isolated source truth and explicit contracts.	Operational fitness, production capacity, disaster recovery, or safe migration completion.
Operable Phase-0 service	The educational environment has evidence for the selected flow, failures, recovery, security, telemetry, and runbooks.	Production SLO history, scale, support rotation, compliance acceptance, or regional resilience.
Production-ready service	Organization-specific production review accepts measured SLO, security, capacity, DR, support, cost, and residual risk.	Permanent readiness; evidence can expire after change.

NAMING DISCIPLINE

Use the strongest term supported by the current evidence. Calling a candidate a service does not make the missing failure modes disappear.

The evidence maturity ladder

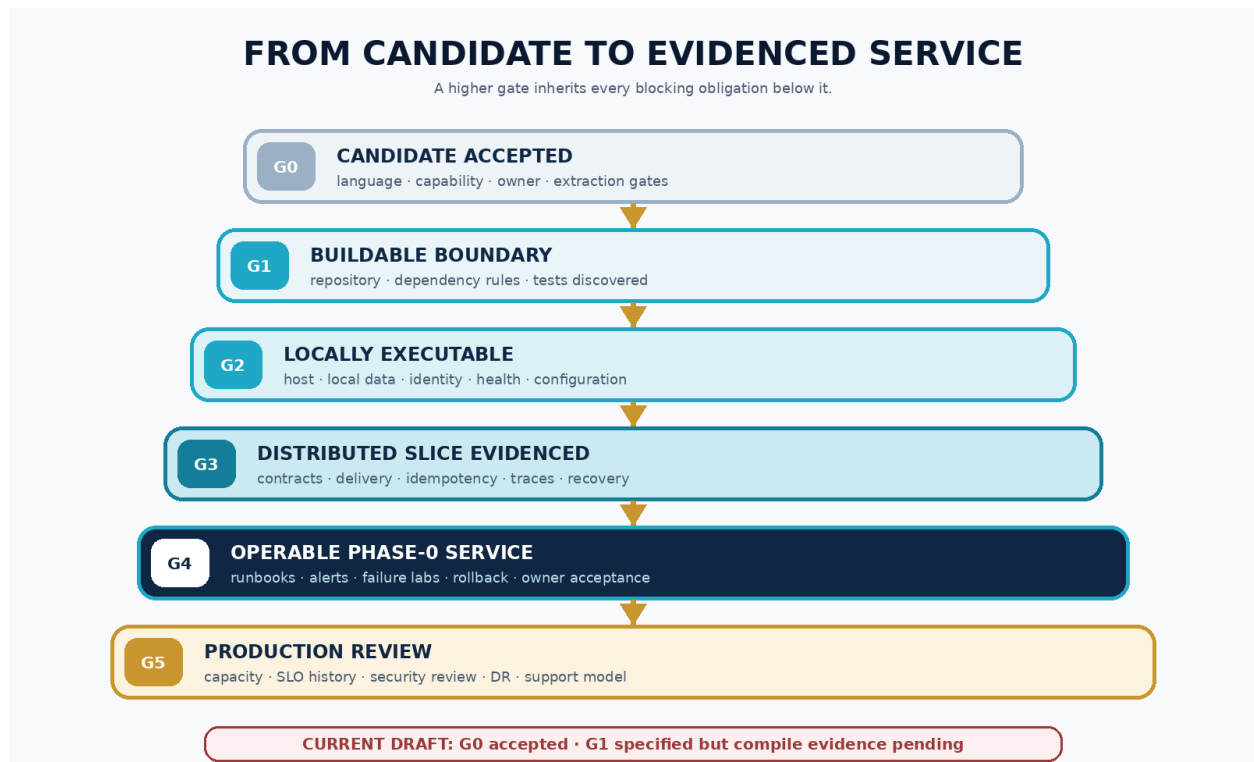


Figure 6.1 - Service maturity advances through inherited blocking gates; production review is not a synonym for completing the demonstrator.

Gate	Entry condition	Exit evidence
G0 - Candidate accepted	Language, capability, data, workflow, contract, team, and operations gates support extraction.	Candidate catalogue, owner, preservation rules, ADR-001.
G1 - Buildable boundary	Repository topology and permitted dependencies are defined.	Pinned toolchain, BUILD-001, ARCH-001, discovered tests, package/contract graph.
G2 - Locally executable	Host, local data, identity, configuration, and health exist without remote success assumptions.	Component tests, migration/restore evidence, authorization tests, truthful health.
G3 - Distributed slice evidenced	Producer and consumer communicate through versioned contracts and independent commits.	Delivery, idempotency, tracing, lag, outage, replay, and recovery evidence.
G4 - Operable Phase-0	The selected flow can be deployed, observed, failed, repaired, and reversed by documented operators.	Runbooks, alerts, failure labs, rollback, service passport, owner acceptance.
G5 - Production review	Real environment, traffic assumptions, support model, security/compliance, capacity, cost, and DR are available.	Organization-specific approval and measured history; outside the Phase-0 claim.

A higher gate inherits every blocking obligation below it. G3 does not excuse a broken G1 dependency rule because end-to-end tests happen to pass. G4 does not excuse an unauthorized verdict because the dashboard is excellent. G5 cannot be reached by adding more architecture prose to G0.

Statuses describe evidence, not optimism

Status	Meaning	Permitted claim
NOT STARTED	No accepted design or artifact exists.	Requirement identified.
DESIGNED	Decision, owner, rule, and target artifact are documented.	Implementation is planned, not present.

Status	Meaning	Permitted claim
IMPLEMENTED	Artifact exists in source or environment.	Code/configuration exists, not that it works.
EVIDENCED	Named execution demonstrates the required property for an identified version/environment.	The property held under the recorded conditions.
ACCEPTED	Named reviewer accepts evidence, scope, validity window, and residual risk.	The gate is satisfied for the declared scope.
BLOCKED	A blocking rule fails or evidence is missing/invalid.	No higher service-readiness claim is permitted.

NO WEIGHTED SCORE

A service with eleven accepted dimensions and one violated blocking invariant is blocked—not 92% done.

The twelve Definition-of-Done dimensions

The dimensions below form the minimum reference Definition of Done. Individual services can add stricter obligations. They cannot remove a blocking dimension merely because the current slice does not exercise it visibly.

Dimension	Blocking completion statement	Primary evidence
1. Boundary and owner	One context owns language, decisions, support, cost, and retirement; consumers are named.	Service catalogue, ADR, ownership and escalation record.
2. Domain preservation	Inherited invariants, authority, determinism, audit, and delayed-truth rules still hold.	Preservation ledger mapped to unit/concurrency/forbidden-action tests.
3. Build and dependency	Pinned toolchain builds the authoritative solution; forbidden references and shared-surface rules pass.	BUILD-001, ARCH-001, package/API/dependency manifests.
4. Data and transaction	The service owns source data, credentials, migrations, concurrency, backup/restore, and local commit boundary.	Isolation test, migration test, restore exercise, cross-write prohibition.
5. Public contracts	APIs/events have producer owner, version, compatibility window, identity, validation, and retirement plan.	Schema fixtures, compatibility tests, consumer inventory, deprecation evidence.
6. Reliability	Retries, idempotency, ordering, timeout, outbox/inbox, dead-letter, replay, and degradation preserve meaning.	Failure experiments, duplicate/replay tests, backlog and recovery records.
7. Security and trust	Human/service identity is authenticated and locally authorized; secrets and privileged repair are auditable.	Positive/negative policy tests, credential/rotation evidence, threat review.
8. Observability	Logs, metrics, traces, correlation/causation, lag, health, and business signals explain the selected flow and failures.	Trace assertions, dashboards, alert tests, telemetry field review.
9. Test evidence	Unit, architecture, contract, component, end-to-end, and failure suites discover and pass the required tests.	Machine-readable reports tied to commit/environment; zero-test protection.
10. Deployment and configuration	Artifact, configuration, schema, identity, startup, rollout, and rollback are deterministic and environment-specific.	Image/SBOM manifest, configuration validation, deployment/rollback record.
11. Operations and support	Named operators can diagnose, repair, replay, restore, and escalate with bounded permissions and runbooks.	Runbook drills, alert routing, audit record, ownership/on-call model.
12. Migration and reversibility	One active writer, cohort routing, reconciliation, history retention, cutover, and reversal are proved.	Migration manifest, comparison evidence, rollback rehearsal, cleanup gate.

Performance, capacity, cost, and compliance remain explicit

The twelve dimensions apply to every service. Performance, capacity, cost, retention, privacy, regulatory evidence, and disaster recovery add service-specific obligations according to risk and deployment target. The Phase-0 book can define experiments and record results; it cannot infer production capacity from a laptop or declare compliance without the relevant authority.

Concern	Phase-0 completion	Production review addition
Latency / throughput	Workload model, repeatable experiment, saturation signal, result limits.	Representative traffic, growth, headroom, regional/network effects, capacity owner.
Availability / recovery	Dependency outage and restart/recovery exercises with observed times.	Measured SLO history, redundancy, DR, support response, error-budget policy.
Cost	Resource inventory and obvious scaling drivers.	Forecast, unit cost, budget owner, capacity/cost trade-off.
Compliance / privacy	Data classification, retention intent, audit fields, prohibited sensitive telemetry.	Applicable control owner, legal/regulatory review, evidence retention and access.

The current RootCause status

The matrix records the strongest honest status at the end of Chapter 6. It is not a forecast and not a criticism. Making the gaps visible is the mechanism that prevents later chapters from quietly redefining completion.

Dimension	Current status	Available evidence	Blocking gap
Boundary / owner	ACCEPTED - G0	Candidate map, service catalogue, ADR-001, decision authority.	Runtime support and cost owner still need implementation assignment.
Domain preservation	DESIGNED	RC/ALM/AUD/CMP/RPT/RL preservation rules and reference tests.	Tests are not yet placed and executed in the repository.
Build / dependency	DESIGNED	ADR-002, project tree, dependency and architecture-test rules.	No .NET SDK/build/architecture-test execution in this environment.
Shared surface	DESIGNED	ADR-003, Shared Surface Ledger, SHR rules.	No package build, API manifest, version-skew, or independent-upgrade evidence.
Data / transaction	NOT STARTED	Ownership statement and no-shared-write rule.	RootCauseDb, credentials, migrations, concurrency, backup/restore not implemented.
Contracts	DESIGNED	AlarmFloodDetectedV1 and RootCauseVerdictIssuedV1 reference shapes.	Envelope, schema, serialization, compatibility, registry/distribution not evidenced.
Reliability	NOT STARTED	Named failure modes and target outbox/inbox/idempotency rules.	No broker delivery, duplicate, outage, replay, dead-letter, or recovery execution.
Security	NOT STARTED	Local-authorization preservation rule.	No identity configuration, policy tests, secret/credential or threat evidence.
Observability	NOT STARTED	Correlation/causation and delayed-truth requirements.	No runtime logs, metrics, traces, dashboards, alerts, or trace assertions.
Deployment / operations	NOT STARTED	Target host and evidence catalogue concepts.	No artifact, environment, health, deployment, rollback, runbook, or operator drill.
Migration / reversal	DESIGNED	Cohort, shadow, one-writer, reconciliation, and rollback boundary.	No migration data, routed traffic, comparison, reconciliation, or rollback rehearsal.

CURRENT VERDICT

RootCause is an accepted service candidate with a specified G1 boundary. It is not yet a buildable, extracted, distributed, operable, or production-ready service.

A Definition of Done can regress

Evidence is scoped to an artifact version, environment, configuration, contract set, and time. A framework upgrade can invalidate build and security evidence. A topology change can invalidate recovery and capacity evidence. A new consumer can invalidate a contract-retirement claim. A changed authorization policy can invalidate privileged-repair evidence. Acceptance therefore has a validity rule and a revalidation trigger.

Evidence type	Bound to	Typical revalidation trigger
Build / architecture	Commit, SDK, packages, analyzer/rule version.	Source, dependency, toolchain, generator, or rule change.
Contract compatibility	Producer/consumer versions, schema fixtures, serializer settings.	Schema/meaning, serializer, envelope, or supported-consumer change.
Security	Identity provider, policy, secrets, dependencies, threat assumptions.	Policy/role/key/dependency/topology or data-classification change.
Failure / recovery	Topology, configuration, data volume, runbook, operator role.	Infrastructure, retry, storage, deployment, queue, or runbook change.
Performance / capacity	Workload, hardware, network, build, dataset, measurement method.	Material workload, resource, topology, code, or dependency change.
Migration	Cohort, source/target versions, data state, route, reconciliation plan.	Cutover scope, data model, event history, writer, or rollback-path change.

The service evidence chain

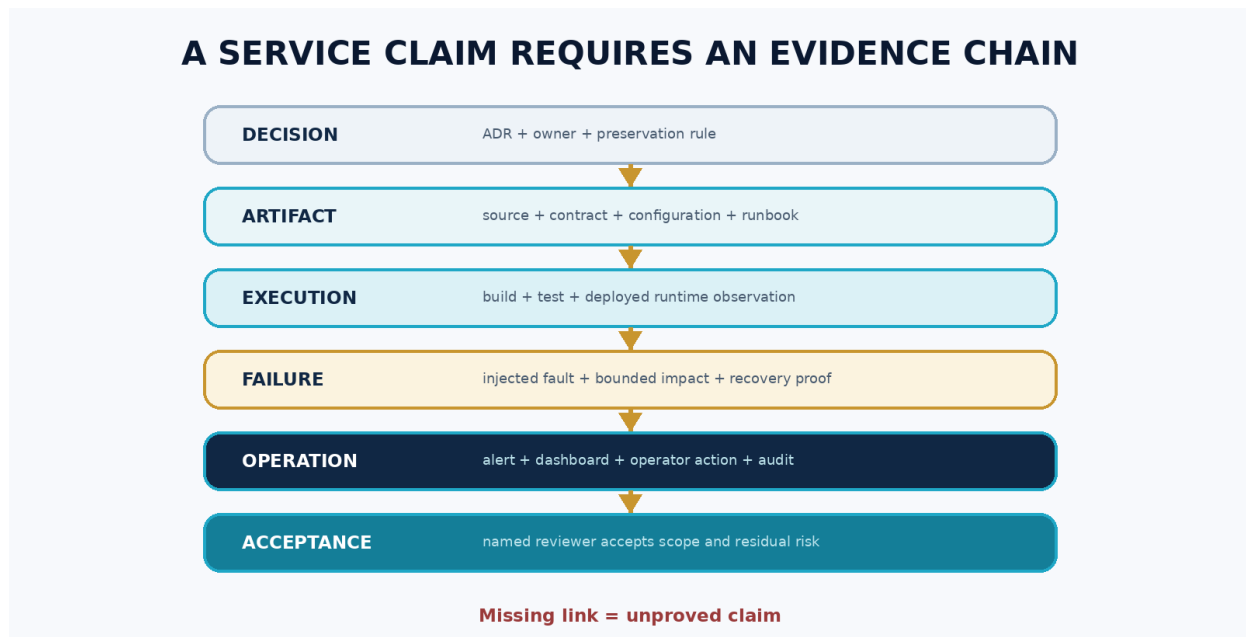


Figure 6.2 - A service-readiness claim remains unproved when any link from decision to acceptance is missing.

A decision explains why a property matters. An artifact implements or configures it. Execution demonstrates the intended path. Failure evidence demonstrates the boundary under stress. Operational evidence shows that a human or automation can detect and recover. Acceptance records who considers the evidence sufficient for a declared scope. A screenshot without a decision is trivia; a decision without execution is architecture; execution without recovery is a demo.

Evidence quality levels

Level	Evidence shape	Use
E0 - Assertion	A statement in prose or conversation with no artifact.	Planning only; never satisfies a gate.
E1 - Artifact	Source, contract, configuration, migration, dashboard, or runbook exists.	Shows implementation intent; does not prove execution.
E2 - Successful execution	Build/test/deployment or observed runtime path succeeds under recorded conditions.	Supports happy-path and static-property claims.
E3 - Failure and recovery	Injected/observed fault produces bounded impact, signal, repair, and preserved meaning.	Required for reliability, operations, and migration claims.
E4 - Independent repetition	CI or another operator repeats the result from documented inputs.	Supports reproducibility and removes author-only knowledge.
E5 - Accepted evidence	Named reviewer accepts scope, freshness, exceptions, and residual risk.	Satisfies the declared gate until a revalidation trigger.

EVIDENCE RULE

A pattern name, code sample, green test, dashboard, or runbook is not self-validating. The acceptance record must state what claim it supports and what remains outside scope.

The service passport

Each service carries a versioned passport that points to source evidence rather than copying every report. The passport is reviewable in the repository and is updated by automation where possible. Pending references remain visibly pending; they are not omitted.

SERVICE PASSPORT - REFERENCE MANIFEST [NEW]

```
service: root-cause
status: candidate
target_gate: G4-phase0
owner:
  business: pending-assignment
  engineering: pending-assignment
contracts:
  consumes: [AlarmFloodDetected.v1]
  produces: [RootCauseVerdictIssued.v1]
data:
  source_store: RootCauseDb
  cross_service_writes: forbidden
evidence:
  boundary: [ADR-001, RC-OWN-001]
  repository: [ADR-002, BUILD-001:pending, ARCH-001:pending]
  sharing: [ADR-003, SHARE-PACK-001:pending]
  runtime: [SLICE-E2E-001:pending]
  failure: [OUTAGE-001:pending, REPLAY-001:pending]
  operations: [RUNBOOK-001:pending]
acceptance:
  gate: G0
  accepted_by: architecture-review
  residual_risks: [compile, data, contract, runtime, security, operations]
```

The manifest syntax is illustrative until its schema and validation tool exist. The important property is referential integrity: every evidence identifier resolves to a versioned artifact or result, every pending item is visible, and acceptance cannot name a gate whose blocking evidence remains pending.

Blocking rules for every service

Rule ID	Blocking rule
DOD-OWN-001	The service has one named decision/data owner and one named operational escalation path.
DOD-PRES-001	Every blocking preservation rule maps to executable evidence without weakened semantics.
DOD-BUILD-001	The pinned authoritative solution builds and required tests are discovered; warnings/exceptions are explicit.
DOD-DATA-001	The service owns its source store, credentials, migrations, transactions, backup/restore, and prohibits cross-writes.
DOD-CON-001	Public contracts are producer-owned, versioned, compatible for the declared window, and inventoried by consumer.
DOD-REL-001	Duplicate, delay, outage, ordering, replay, poison, restart, and partial-success behaviour are bounded and evidenced.
DOD-SEC-001	Local authorization protects every privileged capability and repair path; identity and secrets are auditable.
DOD-OBS-001	Telemetry can reconstruct the selected flow and distinguish source truth, lag, failure, retry, and recovery.
DOD-OPS-001	An authorized operator can detect, diagnose, recover, replay/restore, escalate, and audit the declared failures.
DOD-MIG-001	Cutover preserves one writer, history, reconciliation, cleanup, and rehearsed reversal.
DOD-EVID-001	Every accepted claim identifies artifact, version, environment, execution, reviewer, validity, and residual risk.
DOD-SCOPE-001	The published service status names its gate and never implies production readiness from Phase-0 evidence.

False greens

Apparently green signal	Why it is insufficient	Required companion evidence
The container starts	Startup proves process creation, not truthful readiness, data, contracts, or recovery.	Configuration validation, readiness semantics, dependency failure and restart tests.
GET /health returns 200	A static endpoint can be green while required owned capabilities are unusable.	Liveness/readiness distinction and dependency-specific degradation evidence.
All unit tests pass	Unit tests do not prove serialization, data isolation, identity, delivery, telemetry, deployment, or operations.	Architecture, contract, component, end-to-end, failure, and runbook evidence.
The happy-path event arrived	One delivery says nothing about loss, duplicate, order, outage, poison, restart, or replay.	Failure matrix executed with local effects and operator signals asserted.
Retries are configured	Retries can amplify load or repeat business effects and do not guarantee recovery.	Eligibility, bounds, idempotency, backoff signal, exhaustion and recovery test.
A dashboard exists	A chart can show irrelevant or unowned metrics and still fail during the incident.	Alert test, trace/log link, business signal, runbook step, named responder.
Rollback YAML exists	A manifest is not proof that data, messages, schema, and one-writer state can reverse safely.	Rollback rehearsal with in-flight reconciliation and history preservation.
No incidents occurred	Absence of observed failure can mean absence of traffic, detection, or deliberate tests.	Workload and fault evidence with detection and recovery assertions.

REVIEW QUESTION

What would falsify this claim, and where would the evidence appear? If the answer is unknown, the criterion is not operationally defined.

Definition of Done belongs in CI and operations

Some criteria are evaluated on every commit, others on environment deployment, scheduled failure experiments, migration milestones, or review events. A single pipeline job cannot prove the whole service. It can enforce referential integrity, freshness rules, blocking statuses, and the presence of the required downstream evidence.

Evaluation point	Automated gates	Human/operational acceptance
Pull request	Build, unit, architecture, static security, contract compatibility, manifest/schema validation.	ADR or public-surface review when ownership/contract changes.
Artifact publish	Package/image identity, SBOM/provenance, scan, immutable configuration references.	Exception/risk approval for blocked findings.
Environment deploy	Configuration, migration, identity, startup/readiness, smoke and trace assertions.	Deployment owner confirms scope and rollback readiness.
Failure laboratory	Fault injection, impact, alert, recovery, state reconciliation, evidence capture.	Operator performs/observes runbook and accepts repair result.
Migration milestone	Cohort metrics, outcome comparison, one-writer, lag, in-flight inventory, cleanup checks.	Domain/data/migration owners accept cutover or reversal.
Gate review	Passport completeness, evidence freshness, unresolved blockers/exceptions.	Named cross-functional reviewers accept gate and residual risk.

Acceptance is risk-based, not committee theatre

The service owner is accountable for the passport and gate claim. Additional reviewers participate only where their risk is material. A producer-contract change needs consumer evidence. A privileged repair path needs security and operations. A schema cutover needs data and migration ownership. The review records dissent and exceptions instead of converting them into an average score.

Role	Accepts	Cannot delegate away
Domain/service owner	Language, invariant, decision authority, consumer promise, residual business risk.	Truth ownership and correction policy.
Engineering owner	Build, implementation, test, dependency, deployment, maintenance evidence.	Known technical blockers and evidence integrity.
Platform/operations	Runtime conventions, operability, alerts, recovery, support and platform residual risk.	Platform failure/restore evidence and escalation path.
Security reviewer	Identity, authorization, secret, threat, repair, dependency and audit evidence.	Unaccepted critical exposure or unauthorized capability.
Consumer representative	Compatibility, migration window, deprecation and observable delayed/partial truth.	Producer meaning; the consumer accepts only consumption risk.
Migration/data owner	One writer, data movement, reconciliation, rollback, retention and cleanup evidence.	Unreconciled source truth or irreversible cutover risk.

Exceptions are evidence with an expiry

An exception never changes a failed criterion into a pass. It records the failed rule, affected scope, reason, owner, compensating controls, detection, residual risk, expiry/review trigger, and removal plan. A higher gate can accept an exception only if that gate's authority explicitly allows it. Core invariant, source-data ownership, or privileged authorization violations are not normal schedule exceptions.

Exception field	Required statement
Rule and scope	Exact DOD/preservation rule, service/version/environment and affected traffic/data.
Reason and owner	Why correction is not immediate and who is accountable.
Compensating control	How impact is prevented, detected, bounded, and audited meanwhile.
Residual risk	What can still fail and who has accepted that risk.
Expiry / trigger	Date, release, traffic threshold, incident, or topology change that forces review.
Removal evidence	Artifact/test/migration/runbook that will close the exception.

ADR-004 - Adopt evidence-gated service completion

ADR field	ADR-004 record
Status	Accepted for the reference implementation.
Context	NEXUS-1 needs consistent service claims across chapters without allowing a runnable process, passing happy path, or checklist score to imply complete distributed ownership or production readiness.
Decision	Use cumulative gates G0-G5, twelve blocking DoD dimensions, evidence quality E0-E5, a versioned service passport, stable DOD rules, explicit exceptions, and named acceptance.

ADR field	ADR-004 record
Current claim	RootCause is G0 accepted. G1 is specified but blocked until BUILD-001/ARCH-001 and the generated repository evidence exist.
Consequences	Completion is slower to claim, missing evidence remains visible, reviews are scope-specific, evidence must be maintained, and service status can regress after material change.
Rejected alternatives	Binary done/not-done checklist, weighted maturity score, container/deployment as completion, test-count threshold, architecture approval as runtime proof, and Phase-0 evidence as production readiness.
Reversal condition	Revise when gates cannot distinguish materially different risk states or evidence maintenance costs exceed value; never remove blocking ownership, security, data, or recovery obligations for convenience.
Evidence required	Passport schema/validator, CI policy, evidence catalogue links, gate report, exception workflow, reviewer identity, and at least one complete G4 acceptance exercise.

Risk register for the Definition of Done

Risk	Control	Evidence owner
Checklist theatre	Every item names a falsifiable claim, evidence ID, scope, and blocker status.	Service owner + gate reviewer.
Evidence links rot	Stable catalogue IDs, automated referential-integrity check, retained artifacts.	CI/evidence maintainers.
Green pipeline hides runtime gaps	Separate commit, deploy, failure-lab, migration, and acceptance evidence.	Engineering + operations.
Production-ready overclaim	Mandatory gate/status label and DOD-SCOPE-001 in every passport/public report.	Service owner.
Exceptions become permanent	Owner, compensating control, expiry/trigger, alert, and removal evidence.	Risk acceptor + service owner.
Evidence becomes stale	Bind to version/environment and define automatic/manual revalidation triggers.	Evidence producer.
Review becomes centralized bottleneck	Risk-based reviewers and reusable automated policies; owner remains accountable.	Architecture/platform governance.
Metrics drive gaming	No weighted score; blocking rules and narrative residual risk remain visible.	Gate reviewer.
Operational knowledge stays with author	Independent operator repetition required for G4 recovery evidence.	Operations owner.

Technical material still required

The Definition of Done is structurally complete but not automated. The service-passport manifest, evidence catalogue, gate evaluator, exception schema, CI checks, evidence retention, reviewer identity, and report generation remain implementation work. Later chapters will create the underlying artifacts dimension by dimension.

TECHNICAL MATERIAL STATUS - [EVIDENCE AUTOMATION REQUIRED]

Add a machine-readable passport schema and validator; evidence-ID registry with artifact hashes; CI rules for blockers, zero tests, stale/missing evidence and status overclaim; signed/identified acceptance records.

CHAPTER 6 - CODE AUDIT

The service passport becomes a blocking contract

The following assets preserve the chapter's cumulative G0-G5 model. They do not award points: one unresolved inherited blocker prevents the stronger gate claim.

EVIDENCE STATUS

These are compile-oriented reference assets. The current environment has no dotnet SDK, therefore schema, validator, gate, and CI executions remain PENDING rather than reported green.

EXECUTABLE ASSET 6-A - ROOT-CAUSE.PASSPORT.JSON

```
{
  "schemaVersion": "1.0",
  "service": "root-cause",
  "artifact": {
    "commit": "PENDING",
    "environment": "phase0-local",
    "builtAtUtc": null
  },
  "claimedGate": "G0",
  "owner": {
    "domain": "architecture-review",
    "engineering": "PENDING",
    "operations": "PENDING"
  },
  "contracts": {
    "consumes": ["AlarmFloodDetected.v1"],
    "produces": ["RootCauseVerdictIssued.v1"]
  },
  "dimensions": {
    "boundary": ["ADR-001", "RC-OWN-001"],
    "build": ["BUILD-001:PENDING", "ARCH-001:PENDING"],
    "data": ["DOD-DATA-001:PENDING"],
    "reliability": ["OUTAGE-001:PENDING", "REPLAY-001:PENDING"],
    "operations": ["RUNBOOK-001:PENDING"]
  },
  "exceptions": [],
  "acceptance": null
}
```

Every known obligation is present. PENDING is a first-class state, not an absent property. The commit remains pending because this chapter has not produced a repository build.

MANIFEST CONTRACT

Schema validation rejects ambiguous progress

EXECUTABLE ASSET 6-B - CORE SCHEMA FRAGMENT

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "nexus1://schemas/service-passport/1.0",
  "type": "object",
  "additionalProperties": false,
  "required": [
    "schemaVersion", "service", "artifact", "claimedGate",
    "owner", "contracts", "dimensions", "exceptions", "acceptance"
  ],
  "properties": {
    "schemaVersion": { "const": "1.0" },
    "service": { "type": "string", "minLength": 1 },
    "claimedGate": {
      "enum": ["G0", "G1", "G2", "G3", "G4", "G5"]
    },
    "dimensions": {
      "type": "object",
      "required": [
        "boundary", "preservation", "build", "data", "contracts",
        "reliability", "security", "observability", "tests",
        "deployment", "operations", "migration"
      ],
      "additionalProperties": false
    },
    "exceptions": {
      "type": "array",
      "items": { "$ref": "exception-1.0.schema.json" }
    },
    "acceptance": {
      "oneOf": [
        { "type": "null" },
        { "$ref": "acceptance-1.0.schema.json" }
      ]
    }
  }
}
```

EXECUTABLE ASSET 6-C - DESERIALIZE AND STRUCTURAL GUARD

```
var options = new JsonSerializerOptions
{
  PropertyNameCaseInsensitive = false,
  UnmappedMemberHandling = JsonUnmappedMemberHandling.Disallow
};

await using var input = File.OpenRead(args[0]);
var passport = await JsonSerializer.DeserializeAsync(
  input, PassportJsonContext.Default.ServicePassport)
  ?? throw new PassportViolation("Passport is empty");

PassportStructure.Validate(passport);
```

TWO DISTINCT CHECKS

JSON Schema rejects structural ambiguity. The C# gate evaluator enforces NEXUS-1 semantics such as inherited blockers, evidence freshness, reviewer authority, and exception scope.

EVIDENCE INTEGRITY

An evidence ID resolves to immutable bytes and scope

EXECUTABLE ASSET 6-D - EVIDENCE-CATALOGUE.JSON

```
{
  "BUILD-001": {
    "status": "PENDING",
    "claim": "Authoritative solution builds under pinned SDK",
    "artifact": null,
    "sha256": null,
    "commit": null,
    "environment": null,
    "producedAtUtc": null,
    "validUntilUtc": null
  },
  "ADR-001": {
    "status": "EVIDENCED",
    "claim": "RootCause candidate boundary accepted",
    "artifact": "evidence/adr/ADR-001.pdf",
    "sha256": "REPLACE_WITH_64_HEX_DIGITS",
    "commit": "design-baseline",
    "environment": "repository",
    "producedAtUtc": "2026-07-01T09:00:00Z",
    "validUntilUtc": null
  }
}
```

EXECUTABLE ASSET 6-E - HASH AND SCOPE VERIFICATION

```
public static void Verify(EvidenceRecord evidence, PassportArtifact scope)
{
    if (evidence.Status is EvidenceStatus.Pending)
        return;

    Require(evidence.Artifact, evidence.Id, "artifact");
    Require(evidence.Sha256, evidence.Id, "sha256");
    Require(evidence.Commit, evidence.Id, "commit");
    Require(evidence.Environment, evidence.Id, "environment");

    if (evidence.Commit != scope.Commit ||
        evidence.Environment != scope.Environment)
        throw new EvidenceViolation(evidence.Id, "Scope mismatch");

    using var stream = File.OpenRead(evidence.Artifact!);
    var actual = Convert.ToHexString(SHA256.HashData(stream));
    if (!actual.Equals(evidence.Sha256,
        StringComparison.OrdinalIgnoreCase))
        throw new EvidenceViolation(evidence.Id, "Hash mismatch");
}
```

A hash proves byte identity, not truth. The claim, scope, execution producer, time, and acceptance still determine whether those bytes support the requested gate.

GATE EVALUATION

Cumulative blockers replace the false maturity score

EXECUTABLE ASSET 6-F - INHERITED GATE CATALOGUE

```
public static readonly IReadOnlyDictionary<Gate, GateRule[]> Rules =
    new Dictionary<Gate, GateRule[]>
    {
        [Gate.G0] =
        [
            Require("DOD-OWN-001"),
            Require("DOD-PRES-001")
        ],
        [Gate.G1] =
        [
            Require("DOD-BUILD-001"),
            Require("SHR-EVID-001")
        ],
        [Gate.G2] =
        [
            Require("DOD-DATA-001"),
            Require("DOD-SEC-001")
        ],
        [Gate.G3] =
        [
            Require("DOD-CON-001"),
            Require("DOD-REL-001"),
            Require("DOD-OBS-001")
        ],
        [Gate.G4] =
        [
            Require("DOD-OPS-001"),
            Require("DOD-MIG-001")
        ]
    };
```

EXECUTABLE ASSET 6-G - NO-WEIGHT GATE EVALUATOR

```
public GateReport Evaluate(ServicePassport passport, Gate requested)
{
    var required = Rules
        .Where(pair => pair.Key <= requested)
        .SelectMany(pair => pair.Value)
        .DistinctBy(rule => rule.RuleId);

    var results = required
        .Select(rule => rule.Evaluate(passport, _catalogue, _clock))
        .ToArray();

    var blockers = results
        .Where(result => result.Status is not RuleStatus.Accepted)
        .ToArray();

    return new GateReport(
        Requested: requested,
        Accepted: blockers.Length == 0,
        Results: results,
        Blockers: blockers);
}
```

DELIBERATE OMISSION

GateReport has no percentage field. A presentation layer may count items for navigation, but it must never convert an unresolved blocker into '92% done'.

EXCEPTION DISCIPLINE

Expiry never turns a failed rule into a pass

EXECUTABLE ASSET 6-H - EXCEPTION RECORD

```
public sealed record GateException(  
    string Id,  
    string RuleId,  
    string Service,  
    string ArtifactCommit,  
    string Environment,  
    string Owner,  
    string RiskAcceptor,  
    string Reason,  
    string CompensatingControl,  
    string ResidualRisk,  
    DateTimeOffset ExpiresAtUtc,  
    string RemovalEvidenceId);
```

EXECUTABLE ASSET 6-I - EXCEPTION VALIDATION

```
public RuleResult ApplyException(  
    RuleFailure failure,  
    GateException exception,  
    DateTimeOffset now)  
{  
    if (exception.RuleId != failure.RuleId)  
        throw new ExceptionViolation("Rule mismatch");  
    if (exception.ExpiresAtUtc <= now)  
        throw new ExceptionViolation("Exception expired");  
    if (!_authority.CanAccept(  
        exception.RiskAcceptor, failure.RuleId))  
        throw new ExceptionViolation("Acceptor lacks authority");  
    if (NonWaivableRules.Contains(failure.RuleId))  
        throw new ExceptionViolation("Rule is non-waivable");  
  
    return RuleResult.Excepted(  
        failure.RuleId,  
        exception.Id,  
        exception.ExpiresAtUtc,  
        failure.Message);  
}
```

TEST ASSET - EXPIRED EXCEPTION BLOCKS THE GATE

```
[Fact]  
public void expired_exception_does_not_hide_failed_authorization()  
{  
    var now = Instant("2026-07-29T10:00:00Z");  
    var exception = Fixture.Exception(  
        rule: "DOD-SEC-001",  
        expires: now.AddMinutes(-1));  
  
    var act = () => evaluator.ApplyException(  
        Fixture.Failure("DOD-SEC-001"), exception, now);  
  
    act.Should().Throw<ExceptionViolation>()  
        .WithMessage("*expired*");  
}
```

ACCEPTANCE

Reviewer identity is checked against the risk claimed

EXECUTABLE ASSET 6-J - ACCEPTANCE RECORD

```
{
  "service": "root-cause",
  "gate": "G1",
  "passportSha256": "PENDING",
  "reportSha256": "PENDING",
  "reviewer": {
    "subject": "architecture-review",
    "authority": ["G0", "G1", "build", "dependency"]
  },
  "acceptedAtUtc": null,
  "validUntilUtc": null,
  "exceptions": [],
  "residualRisks": [
    "G2-G4 runtime evidence remains outside this acceptance"
  ],
  "signature": null
}
```

EXECUTABLE ASSET 6-K - ACCEPTANCE GUARD

```
public void ValidateAcceptance(
    AcceptanceRecord acceptance,
    GateReport report,
    byte[] passportBytes)
{
    if (!report.Accepted)
        throw new AcceptanceViolation("Gate has blockers");
    if (acceptance.Gate != report.Requested)
        throw new AcceptanceViolation("Gate mismatch");
    if (!_authority.CanAccept(
        acceptance.Reviewer.Subject, acceptance.Gate))
        throw new AcceptanceViolation("Reviewer not authorized");

    var passportHash = Convert.ToHexString(
        SHA256.HashData(passportBytes));
    if (passportHash != acceptance.PassportSha256)
        throw new AcceptanceViolation("Passport changed");

    _signature.Verify(acceptance);
}
```

The signature mechanism is environment-owned and intentionally abstract here. A local HMAC embedded beside the file would not prove independent reviewer identity. Phase-0 may retain an identified CI/OIDC attestation; the final mechanism belongs to the deployment environment.

CURRENT HONEST RECORD

RootCause has no G1 acceptance record because BUILD-001 and ARCH-001 remain pending. The JSON above is a target shape, not an approval.

AUTOMATION BOUNDARY

CI validates the claim; later Parts produce the evidence

EXECUTABLE ASSET 6-L - COMMAND-LINE GATE

```
public static async Task<int> Main(string[] args)
{
    var passport = await LoadPassport(args[0]);
    var catalogue = await LoadCatalogue(args[1]);
    var requested = Gate.Parse(args[2]);

    var report = new GateEvaluator(catalogue, SystemClock.Instance)
        .Evaluate(passport, requested);

    await GateReportWriter.WriteJson(report, args[3]);
    await GateReportWriter.WriteMarkdown(report, args[4]);

    return report.Accepted ? 0 : 2;
}
```

CI ASSET - EVIDENCE GATE AND RETAINED REPORT

```
- name: Validate service passport
  run: >
    dotnet run --project tools/Nexus1.EvidenceGate --
    evidence/root-cause.passport.json
    evidence/catalogue.json
    G1
    artifacts/gate-report.json
    artifacts/gate-report.md

- name: Retain gate evidence
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: root-cause-g1-${{ github.sha }}
    path: artifacts/gate-report.*
    retention-days: 90
```

TEST ASSET - OVERCLAIM MUST FAIL

```
[Fact]
public void G1_claim_is_blocked_when_BUILD_001_is_pending()
{
    var passport = Fixture.Passport(claimedGate: Gate.G1);
    var catalogue = Fixture.Catalogue(
        ("DOD-OWN-001", Accepted),
        ("DOD-PRES-001", Accepted),
        ("DOD-BUILD-001", Pending));

    var report = Evaluate(passport, catalogue, Gate.G1);

    report.Accepted.Should().BeFalse();
    report.Blockers.Should().ContainSingle(
        x => x.RuleId == "DOD-BUILD-001");
}
```

Official-source validation

Checked 29 July 2026. The reference implementation uses the .NET runtime's System.Text.Json serialization APIs and SHA256.HashData for byte-identity checks. These platform mechanisms support reproducibility; the NEXUS-1 gate rules still define what the evidence means.

Sources: <https://learn.microsoft.com/dotnet/standard/serialization/system-text-json/overview> ;
<https://learn.microsoft.com/dotnet/api/system.security.cryptography.sha256.hashdata>

REVISION RESULT

Chapter 6 now supplies an executable passport, structural schema, evidence integrity, cumulative gate evaluation, exception expiry, acceptance guard, CI command, and overclaim test. Runtime and operational evidence remains explicitly pending.

Evidence ID	Required artifact	Blocking result
DOD-SCHEMA-001	Versioned service-passport and exception schemas with validation tests.	Invalid, ambiguous, unknown, or silently omitted blocking fields.
DOD-REF-001	Evidence catalogue resolves every non-pending ID to versioned artifact/result.	Broken link, mismatched commit/environment, or missing retention.
DOD-GATE-001	Generated gate report evaluates all inherited blockers and exceptions.	Weighted score, unresolved blocker, invalid exception, or higher unsupported status.
DOD-ACCEPT-001	Named acceptance record includes gate, scope, evidence set, exceptions and residual risk.	Anonymous approval or acceptance without reviewer authority.
DOD-REPEAT-001	Independent operator repeats the selected G4 failure/recovery path.	Author-only steps, missing signal, unreconciled state, or undocumented intervention.

Implementation sequence

1. Commit ADR-004, the DOD rule catalogue, gate definitions, statuses, and evidence-quality levels.
2. Create one RootCause service passport with every known item present and pending rather than omitted.
3. Define the passport/evidence/exception schemas and validate referential integrity in CI.
4. Map every preservation and shared-surface rule to a DoD dimension and planned evidence ID.
5. Implement G1 build and architecture evidence in Part II before permitting a buildable-service claim.
6. Add G2 local data, identity, configuration, health, component, migration, and restore evidence in Parts II-III/VI.
7. Add G3 contract, broker, idempotency, tracing, lag, partial-success, and recovery evidence in Parts IV-V/VII.
8. Add G4 deployment, runbook, alert, failure-lab, rollback, and independent-operator acceptance in Parts VIII-X.
9. Record exceptions without changing blocker status and automate their review triggers.
10. Publish the final evidence catalogue and Phase-0 limitations; route any production claim to a separate G5 review.

What this chapter has proved

Chapter 6 has defined the language, gates, dimensions, evidence chain, status model, passport, stable rules, acceptance, exception handling, risks, and automation requirements for calling a NEXUS-1 boundary done. It has also recorded the current RootCause claim precisely: G0 accepted, G1 specified, all stronger runtime and operational claims pending evidence.

It has not made RootCause a service by definition. It has created the contract that later implementation must satisfy. The value of the Definition of Done is that every later chapter can strengthen the same claim without silently changing what completion means.

CHAPTER CLOSING

A service is done for a declared gate when every blocking truth has current evidence, every exception has an accountable expiry, and an authorized reviewer accepts the remaining risk.

PART I CLOSING

Crossing into distribution is now constrained: the inherited architecture, preservation ledger, selected slice, repository topology, shared surface, and completion gates agree on one meaning of progress. Part II can begin implementation without pretending that scaffolding is evidence.

Chapter checkpoint

You are ready to enter Part II when the current claim and every missing stronger claim can be stated without ambiguity.

Question	Expected answer
What is RootCause at this point?	An accepted service candidate with a specified G1 boundary; it is not yet buildable or evidenced in this environment.
Why is there no weighted DoD score?	A blocking failure cannot be cancelled by success in unrelated dimensions.
What distinguishes implemented from evidenced?	An artifact exists when implemented; a named execution demonstrates the claim under recorded conditions when evidenced.
What does G4 mean?	The selected Phase-0 service can be deployed, observed, failed, repaired, reversed, and accepted with runbooks and evidence.
Why does G4 not mean production-ready?	Production needs representative capacity/SLO history, security/compliance, DR, cost and organizational support evidence outside the demonstrator.
What is the service passport?	A versioned manifest of owner, contracts, data, gate status, evidence links, pending items, exceptions and residual risks.
Can an exception make a blocker pass?	No. It records scoped risk, control, owner and expiry; the rule remains failed/excepted and higher acceptance needs authority.
When does evidence expire?	When its bound artifact/environment changes, a revalidation trigger occurs, or its declared validity is exceeded.
What must Part II prove first?	The generated repository builds under a pinned toolchain, architecture rules pass, tests are discovered, and the golden-path template carries truthful technical defaults.

NEXT CHAPTER - PART II

Chapter 7 builds the Golden-Path Service Template. It turns the G1 repository policy into reusable .NET projects, composition, configuration, health, telemetry, container, test, and evidence hooks while keeping every service-specific meaning outside the template.

PART II

PART II

The Service Foundation

*Predictable mechanics make meaningful differences visible.
The foundation removes accidental variation without owning domain truth.*

Part II converts the Part I repository and evidence policy into a reusable service foundation. It defines the golden path, composition direction, internal layers, environment boundaries, truthful health semantics, and the test structure that every later data, contract, messaging, and operations chapter can extend.

WHAT PART II GIVES YOU

A generated service shape with constrained dependencies, explicit runtime hooks, configuration and health contracts, and a test/evidence skeleton that remains subordinate to service-owned meaning.

CHAPTERS 7-12

- The Golden-Path Service Template
- Host Composition and Dependency Direction
- Domain, Application, Infrastructure, and Edge Inside a Service
- Configuration and Environment Boundaries
- Liveness, Readiness, and Startup Dependencies
- The Testing Pyramid for a Distributed Service

PART II - THE SERVICE FOUNDATION

7. The Golden-Path Service Template

A golden path is executable repository policy: it gives every NEXUS-1 service the same truthful technical starting point while refusing to decide the service's language, data meaning, contracts, authorization, or failure policy.

CHAPTER ARGUMENT

Standardize the mechanics that must be repeated and verified. Keep every business decision, source-of-truth rule, and public meaning inside its owning service.

Part I established a candidate, a preservation ledger, the AlarmManagement -> RootCause seam, a context-first repository, a narrow shared surface, and evidence-gated completion. Part II now begins implementation at the smallest reusable level. The first task is not to add RabbitMQ, Entity Framework, Kubernetes, or a generic command bus. It is to create a service shape whose dependency direction, startup hooks, test locations, and evidence obligations are difficult to evade.

The word template is dangerous because it can describe anything from a copied folder to a private framework. The NEXUS-1 golden path is narrower. It is a versioned multi-project source template plus a deterministic repository orchestrator and validator. The source template creates ordinary files that remain owned by the generated service. The orchestrator checks names and collisions, stages the output, integrates an explicit project list, and seeds the service passport. The validator proves that generation did not weaken the architecture.

The chapter can specify that mechanism without pretending it has run. This environment does not contain the pinned .NET toolchain or the generated NEXUS-1 repository. Therefore no template package has been packed, installed, invoked, compiled, or repeated here. The design is intended to become the first executable G1 mechanism; its evidence remains pending.

From repository rule to golden path

Term	Meaning in NEXUS-1	Ownership boundary
Golden path	The approved, evidence-producing route from service declaration to integrated repository fragment.	Architecture/platform owns the route; the service owner accepts the generated instance.
Source template	Ordinary source files plus .template.config/template.json used by the .NET template engine.	Template maintainers own the source; generated files transfer to the service.
Repository orchestrator	A deterministic wrapper that validates input, stages generation, updates the solution, and runs structural checks.	Repository engineering owns it; it may not infer business capabilities.
Nexus1.ServiceDefaults	A dependency-poor runtime library exposing host conventions and extension points.	Platform owns technical behaviour only; Chapter 5 sharing rules still apply.
Generated service	Context projects, host, tests, operations metadata, and evidence placeholders for one named owner.	The service team owns and may evolve the instance under architecture tests.
Starter example	Documentation showing one completed specialization such as RootCause.	Educational evidence; never an upstream dependency of generated services.

TRANSFER OF OWNERSHIP

Generated code is not vendor code and is not regenerated over local changes. Once accepted, it belongs to the service; future template releases are compared and adopted deliberately.

The golden path has six non-negotiable properties

Property	Required behaviour	Blocking failure
Minimal	Generate only the stable ownership, project, host, test, and evidence skeleton.	Speculative adapters, workflows, entities, or abstractions appear by default.
Truthful	Placeholders remain visibly pending; health and evidence cannot claim unavailable capabilities.	A new service starts green while required configuration or adapters are absent.
Deterministic	The same template version and declared inputs produce an equivalent repository fragment.	Time, machine identity, network data, or hidden prompts change output.
Replaceable	Generated projects depend on standard .NET and small NEXUS-1 technical surfaces.	The template becomes a mandatory framework that owns the service lifecycle.
Enforceable	Architecture tests and repository validation express every blocking dependency rule.	Correctness depends on reviewers remembering an unwritten convention.
Evidence-producing	Generation records template version, inputs, file manifest, structural checks, and pending runtime gates.	Scaffolding is mistaken for BUILD-001, operability, or production readiness.

The template responsibility boundary

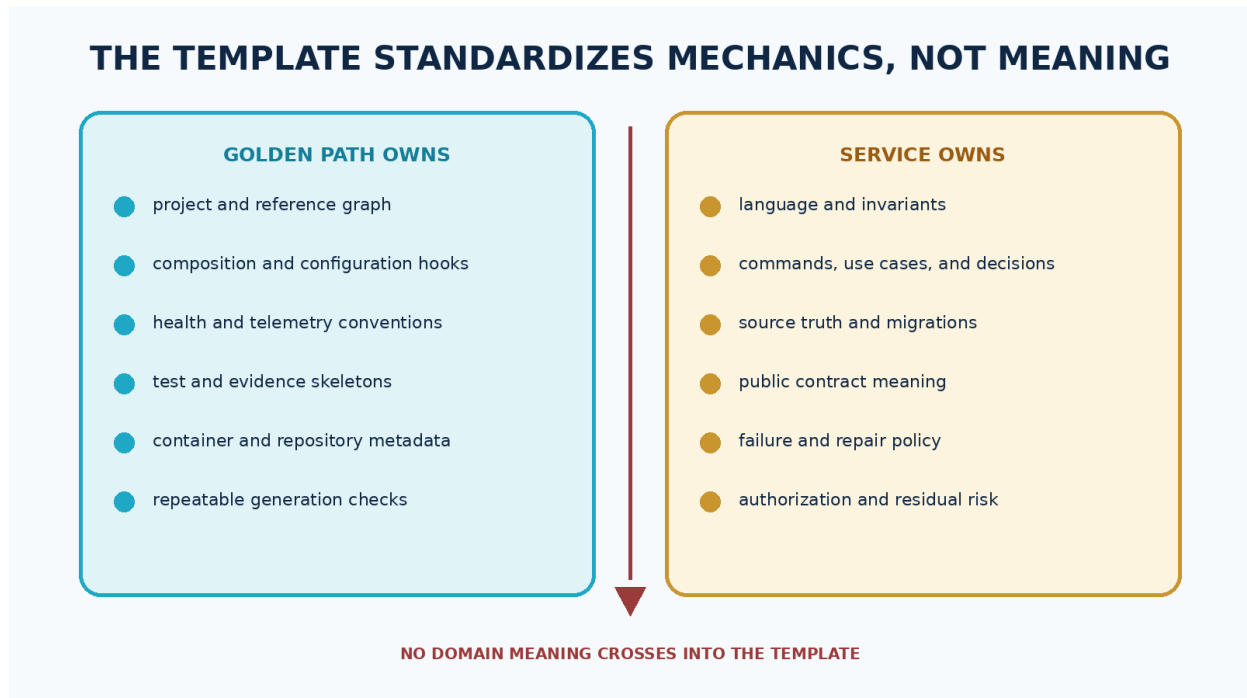


Figure 7.1 - The golden path owns repeatable mechanics; the generated service owns every statement of business meaning.

The left side is intentionally boring. A project graph, health endpoint convention, telemetry resource name, test folder, Docker build context, or service-passport location should not be rediscovered by every team. The right side must remain diverse. RootCause closes a case only under RootCause invariants. AlarmManagement detects a flood under AlarmManagement rules. Compliance decides review. A template that creates a generic Verdict, Case, Alarm, Approval, or Entity abstraction has crossed the line.

What may and may not be generated

Surface	Golden-path output	Forbidden template ownership
Domain	Empty project, assembly metadata, namespace convention, unit-test project.	Base aggregate, generic entity, verdict rules, domain event catalogue, shared vocabulary.
Application	Registration hook, command/query folders, validation and authorization extension points.	Generic use cases, cross-service workflow, policy decisions, consumer-specific orchestration.
Infrastructure	Adapter registration hook, configuration sections, migration and messaging locations.	Chosen schema, credentials, broker topology, retry eligibility, repair meaning.
Contracts	Producer-owned assembly, schema/test folders, versioning metadata placeholders.	A universal event envelope that erases producer ownership or copies domain models.
Host	Composition root, endpoint groups, background-work hooks, shutdown and diagnostics wiring.	Business branching in Program.cs, handlers, hosted-service loops, or generic middleware.
Tests	Unit, architecture, component, contract, smoke, and evidence-project locations.	Passing examples that claim service-specific invariants or external dependencies exist.
Operations	Container, service passport, runbook, alert, and dashboard placeholders with pending status.	Production capacity, SLO, compliance, support, or recovery claims without execution.

Capabilities are declared; obligations are never optional

The template creates one stable service skeleton. A separate service declaration records which capabilities the first instance intends to implement. This avoids a combinatorial template whose dozens of booleans produce untested project graphs. Capability declarations control validation and evidence expectations; they do not excuse core ownership, build, security, observability, or migration obligations.

Capability	RootCause declaration	Generated or seeded mechanics	Meaning left pending
http	required	Endpoint group location, authorization hook, API test fixture.	Routes, commands, response meaning, identity policies.
messaging-consumer	required	Consumer adapter location, inbox/idempotency evidence placeholders.	AlarmFloodDetected.v1 handling, retry eligibility, poison repair.
messaging-producer	required	Publisher/outbox locations and contract-test hook.	RootCauseVerdictIssued.v1 mapping, ordering, publication policy.
persistence	required	Database adapter location, migration project/folder, restore evidence hook.	RootCauseDb schema, credentials, concurrency, backup/restore.
scheduled-work	not declared	No scheduler-specific adapter or operational claim.	Any future scheduling semantics require a reviewed declaration.

VARIATION BUDGET

A capability declaration may add an adapter and new evidence obligations. It may not remove the Domain/Application boundary, local authorization, truthful health, observability, tests, ownership, or service passport.

ROOTCAUSE SERVICE DECLARATION - [NEW]

```
service: root-cause
template:
  identity: Nexus1.Templates.Service
  version: pending-package
owner:
  domain: pending-assignment
  engineering: pending-assignment
capabilities:
  http: required
  messaging_consumer: required
  messaging_producer: required
  persistence: required
  scheduled_work: not-declared
target_gate: G1
evidence:
  generation: pending
  build: pending
  architecture: pending
  repeatability: pending
```

The reference template package

The template is stored as repository source and packaged independently from the generated services. Its content folder represents a repository fragment, not a single Web project. This lets one invocation create the context projects, host, tests, and operations metadata that Chapter 4 already requires.

TEMPLATE-PACK SOURCE TREE - [NEW]

```
templates/
  Nexus1.ServiceTemplate.Pack/
    Nexus1.ServiceTemplate.Pack.csproj
    templates/
      nexus-service/
        .template.config/
          template.json
        src/
          Contexts/TemplateService/
            Nexus1.TemplateService.Domain/
            Nexus1.TemplateService.Application/
            Nexus1.TemplateService.Infrastructure/
            Nexus1.Contracts.TemplateService/
          Hosts/
            Nexus1.TemplateService.Host/
        tests/
          Nexus1.TemplateService.UnitTests/
          Nexus1.TemplateService.ComponentTests/
        ops/
          TemplateService/service-passport.yaml
          TemplateService/runbooks/README.md
          TemplateService/evidence/README.md
        container/
          TemplateService/Dockerfile
```

REPOSITORY FRAGMENT

The template does not create a second `global.json`, `Directory.Build.props`, `Directory.Packages.props`, `.editorconfig`, or authoritative solution. Those remain repository-root policy and are integrated by the orchestrator.

template.json names the source; it does not bless the output

The .NET template engine recognizes a template through `.template.config/template.json`. The `sourceName` value is replaced in file names and contents by the name supplied to `dotnet new`. That mechanism is useful but purely textual. It cannot decide whether the resulting namespace expresses a real owner, whether the project references are legal, or whether the service passport is honest. NEXUS-1 therefore keeps `template.json` small and moves architectural acceptance into repository validation.

.TEMPLATE.CONFIG/TEMPLATE.JSON - REFERENCE [NEW]

```
{
  "$schema": "https://json.schemastore.org/template",
  "author": "NEXUS-1",
  "classifications": ["NEXUS-1", "Service", "Distributed"],
  "identity": "Nexus1.Templates.Service",
  "name": "NEXUS-1 golden-path service",
  "shortName": "nexus-service",
  "sourceName": "TemplateService",
  "preferNameDirectory": false,
  "tags": {
    "language": "C#",
    "type": "project"
  }
}
```

The configuration is a reference artifact, not a validated package. Its schema, replacement boundaries, generated filenames, and interaction with the repository orchestrator must be tested against the pinned SDK. The template uses one replacement token on purpose. Service capability is recorded in the generated declaration rather than encoded as an unbounded matrix of template options.

Package the template; do not publish it as a runtime dependency

A template pack is distributed as a package so teams can install a versioned generator. Generated services do not reference that package. The package contains content, not reusable domain binaries. This preserves a clean upgrade decision: a team may compare a newer template instance or migration guide without changing a runtime dependency merely because the generator changed.

TEMPLATE-PACK PROJECT - REFERENCE [NEW]

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>netstandard2.0</TargetFramework>
    <PackageType>Template</PackageType>
    <PackageId>Nexus1.ServiceTemplate</PackageId>
    <PackageVersion>0.1.0-alpha</PackageVersion>
    <Title>NEXUS-1 Service Template</Title>
    <Authors>NEXUS-1</Authors>
    <Description>Golden-path repository fragment for a NEXUS-1 service.</Description>
    <PackageTags>dotnet-new;templates;nexus-1;microservices</PackageTags>
    <IncludeContentInPack>true</IncludeContentInPack>
    <IncludeBuildOutput>false</IncludeBuildOutput>
    <ContentTargetFolders>content</ContentTargetFolders>
  </PropertyGroup>
  <ItemGroup>
    <Content Include="templates/**/*"
             Exclude="templates/**/bin/**;templates/**/obj/**" />
    <Compile Remove="**/*" />
  </ItemGroup>
</Project>
```

TRUST BOUNDARY

Templates can create executable source and may contain post-actions. NEXUS-1 installs only pinned, reviewed template packages and forbids hidden network access, secret discovery, and opaque post-generation execution.

Generation happens in staging

Writing directly into the repository makes partial failure difficult to reverse. The orchestrator generates into a clean staging directory, validates the complete fragment, computes a manifest, and only then promotes an explicit file set. A collision, unresolved token, forbidden reference, or invalid declaration aborts before the authoritative solution changes.

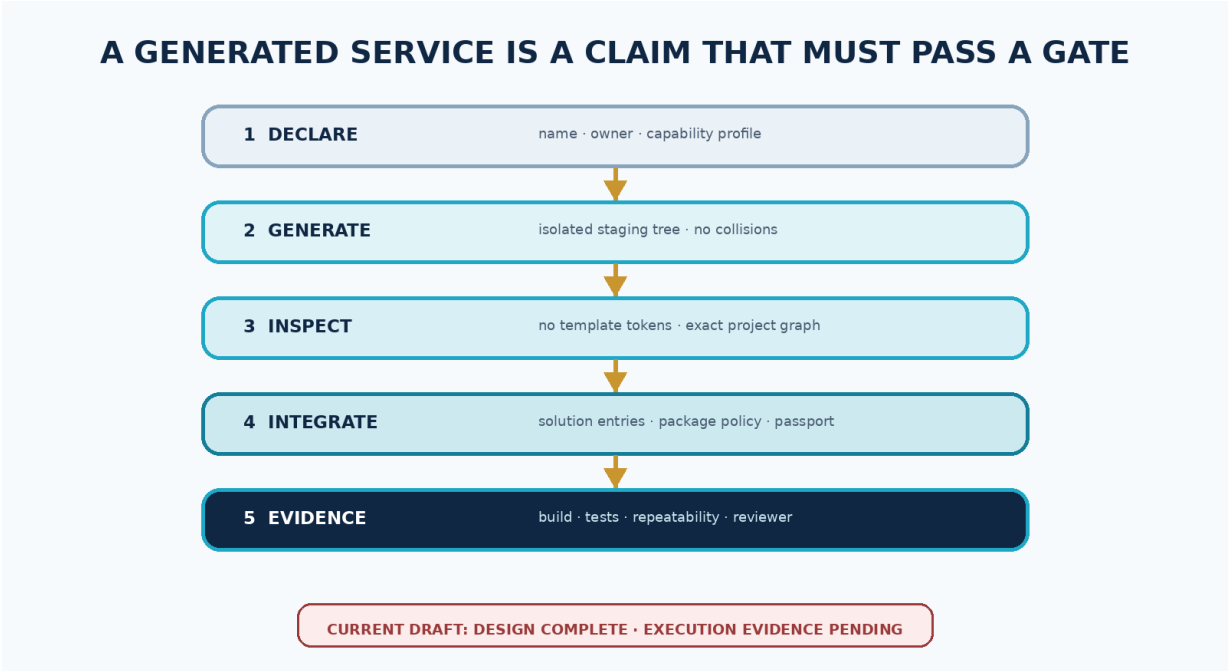


Figure 7.2 - Generation becomes acceptable only after declaration · isolation · structural inspection · integration · and evidence

Stage	Required action	Failure leaves
Declare	Validate service name, context owner, capability profile, template version, and target repository root.	No generated files.
Generate	Invoke the pinned template into a new staging directory with no repository mutation.	A disposable staging tree only.
Inspect	Reject unresolved tokens, unexpected files, symlinks, secrets, binaries, forbidden package sources, or project references.	Diagnostics and the rejected staging tree.
Integrate	Promote the approved file manifest; add an exact project list to Nexus1.Runtime.sln; update catalogue/passport metadata.	One atomic repository change set suitable for review.
Evidence	Restore/build/test the authoritative solution, repeat generation, compare normalized manifests, and record acceptance.	Pending G1 status when any proof is absent.

GOLDEN-PATH ORCHESTRATION - [COMPILE REQUIRED]

```
# Reference workflow; commands are not execution evidence.
dotnet new install ./artifacts/Nexus1.ServiceTemplate.0.1.0-alpha.nupkg

./eng/new-nexus-service --name RootCause --declaration ./services/root-cause.yaml --template-version 0.1.0-alpha --stage ./artifacts/generated/root-cause

./eng/validate-generated-service ./artifacts/generated/root-cause
./eng/promote-generated-service ./artifacts/generated/root-cause

dotnet build Nexus1.Runtime.sln -c Release --no-restore
dotnet test Nexus1.Runtime.sln -c Release --no-build
```

NO HIDDEN POST-ACTION

Solution integration and validation stay in reviewed repository scripts. The template itself does not silently run restore, fetch tools, mutate unrelated projects, or declare success.

The generated project graph is the first invariant

The project graph is more important than the number of folders. Each reference must preserve the direction defined in Chapter 4. The template emits the projects and their intended references; architecture tests verify the real compiled graph after integration.

Generated project	May reference	Must not reference
Nexus1.RootCause.Domain	Dependency-poor approved BuildingBlocks only.	Application, Infrastructure, Host, contracts, ServiceDefaults, transport, persistence.
Nexus1.RootCause.Application	Domain and narrow application abstractions.	Host, transport-specific consumers, concrete database/broker clients.
Nexus1.RootCause.Infrastructure	Application, Domain, producer-owned Contracts, approved technical packages.	Host implementation or another context's internals.
Nexus1.Contracts.RootCause	BCL and explicitly approved serialization annotations only.	RootCause Domain/Application/Infrastructure/Host or shared business abstractions.
Nexus1.RootCause.Host	Application, Infrastructure, required Contracts, Nexus1.ServiceDefaults.	Business rules implemented in endpoints, startup, middleware, or hosted loops.
RootCause.UnitTests	Domain/Application test surface and deterministic fixtures.	Mandatory network, database, broker, container, or shared mutable test state.
RootCause.ComponentTests	Host boundary and real adapters under controlled infrastructure.	Another service's private projects or an in-memory path that bypasses owned adapters.

BLOCKING ARCHITECTURE RULE
If a generated project cannot compile without a forbidden reference, the template is wrong. Do not add an exception merely to make the first instance build.

ServiceDefaults exposes mechanics through a small surface

Chapter 5 allowed a narrow Nexus1.ServiceDefaults package for host-level conventions. Chapter 7 defines the surface the template expects; Chapters 8-11 will implement and test the composition, configuration, health, and telemetry behaviour. The API names are reference contracts, not proof that the library exists.

GENERATED COMPOSITION ROOT - REFERENCE [EXTEND][NEW]

```
var builder = WebApplication.CreateBuilder(args);

builder.AddNexusServiceDefaults(options =>
{
    options.ServiceName = "root-cause";
    options.ConfigurationSection = "RootCause";
});

builder.Services.AddRootCauseApplication();
builder.Services.AddRootCauseInfrastructure(builder.Configuration);

var app = builder.Build();

app.MapNexusServiceEndpoints();
app.MapRootCauseEndpoints();

await app.RunAsync();
```

Hook	Template expectation	Chapter that supplies evidence
AddNexusServiceDefaults	Register technical conventions without scanning service domain assemblies for behaviour.	8 - composition; 10 - configuration; 11 - health.

Hook	Template expectation	Chapter that supplies evidence
AddRootCauseApplication	Service-owned registration of use cases, validators, and authorization abstractions.	8-9 - dependency direction and layers.
AddRootCauseInfrastructure	Service-owned selection of persistence, messaging, clock, identity, and adapter implementations.	8-10 and later infrastructure Parts.
MapNexusServiceEndpoints	Expose technical liveness/readiness/diagnostic endpoints under a stable convention.	11 - truthful health; IX - operational evidence.
MapRootCauseEndpoints	Expose service-owned HTTP capabilities and local authorization.	9 and Part IV - edge and public contracts.

A generated host begins truthful, not green

The host may be alive before it is ready. The template therefore creates separate liveness and readiness routes, but it cannot register a fake database, broker, schema, or identity check. A capability declaration marked required remains a pending readiness obligation until its service-owned adapter registers and proves the corresponding check. Chapter 11 defines exact startup and dependency semantics.

Signal	Template default	Forbidden shortcut
Liveness	Process loop and endpoint infrastructure can respond without checking remote success.	Restarting the process because a database, broker, or consumer is temporarily unavailable.
Readiness	Unready until required local configuration and declared startup obligations are satisfied.	Returning healthy because no service-specific readiness checks were registered.
Diagnostics	Expose bounded technical detail with correlation and no secrets.	Dumping configuration, credentials, contract payloads, or private domain state.
Telemetry	Stable service/resource identity and hooks for logs, metrics, and traces.	Claiming observability before the selected flow and failure signals are asserted.

Telemetry is a hook, not an observability claim

The .NET observability surface uses logging, metrics, and distributed tracing primitives. The template can standardize service identity, ActivitySource and Meter naming, correlation propagation hooks, exporter configuration boundaries, and prohibited-field tests. It cannot know which RootCause decision, queue lag, case transition, authorization denial, or repair action matters. Those signals remain service-owned and must be asserted by later end-to-end and failure evidence.

SERVICE-OWNED DIAGNOSTICS SEED - [NEW]

```
internal static class RootCauseDiagnostics
{
    internal const string ServiceName = "root-cause";
    internal const string InstrumentationName = "Nexus1.RootCause";

    internal static readonly ActivitySource Activities =
        new(InstrumentationName);

    internal static readonly Meter Metrics =
        new(InstrumentationName);
}

// Names are stable technical identity. Span names, metrics, and fields
// are added by RootCause where their meaning is owned and testable.
```

The template seeds tests that can fail honestly

A generated test project is not evidence merely because it compiles. The golden path must seed tests that detect structural absence: an unresolved template token, illegal project reference, empty discovery run, misleading health endpoint, unvalidated required configuration, or a passport that claims a higher gate than its evidence. Service-specific invariant and adapter tests are then added by the owner.

Seeded test/evidence surface	What the template can assert	What specialization must add
Template contract tests	template.json schema, expected files, replacement token removal, collision behaviour, normalized repeatability.	Approved changes to the generated manifest and migration notes.
Architecture tests	Assembly graph, forbidden references, namespace ownership, shared-surface vocabulary ban.	New approved adapters or project types with an ADR-backed rule update.
Unit-test seed	The project is discovered and contains one explicit failing/pending specialization marker.	RootCause invariants, use-case decisions, validation, authorization.
Component-test seed	Host factory and controlled infrastructure fixture locations exist.	RootCauseDb, messaging, identity, migrations, readiness, restart/recovery.
Contract-test seed	Producer-owned contract assembly and fixture locations are discoverable.	Serialization fixtures, compatibility window, consumer inventory.
Container check	Docker context, non-root policy placeholders, port/config metadata, image-pin obligation.	Actual image build, startup, health, shutdown, SBOM and vulnerability evidence.
Service passport	Every known evidence item is present and pending; no unsupported gate is named.	Executed artifact links, reviewer identity, validity, residual risk.

ZERO-TEST PROTECTION

A successful test command with zero discovered required tests is a blocking failure. Discovery counts and required assemblies belong in machine-readable evidence.

Evidence begins at generation

Evidence ID	Required artifact	Acceptance rule
TPL-SCHEMA-001	Validated template.json and template-pack metadata under the pinned SDK.	No unknown/invalid field; identity, short name, source token, and package version recorded.
TPL-GEN-001	Generation log plus complete staged file manifest and declared inputs.	Exit success, no collision, no unexpected binary/secret/symlink, exact template version.
TPL-TOKEN-001	Filename/content scan for TemplateService and other reserved generator tokens.	No unresolved token outside explicitly allowed test fixtures.
TPL-GRAPH-001	Generated project/reference manifest compared with architecture policy.	Exact projects and permitted edges; no implicit or forbidden reference.
TPL-BUILD-001	Authoritative solution restore/build and test-discovery result after integration.	Feeds, SDK, packages and commit identified; warnings/errors and zero tests block.
TPL-REPEAT-001	Two isolated generations with normalized manifests and hashes.	Equivalent output for the same template version and declared inputs.
TPL-PASS-001	Generated service passport validated against gate/status rules.	Every required evidence ID exists; runtime items remain pending; no G1 overclaim.
TPL-REVIEW-001	Named review of generated RootCause fragment and declared deviations.	Service and template owners accept scope, exceptions, and residual risk.

Repeatability excludes unstable output

A byte-for-byte comparison is useful only after explicitly normalizing fields that are expected to differ. The NEXUS-1 template should not emit timestamps, random identifiers, machine paths, user names, restored packages, or environment-derived configuration. If a generated artifact needs an identifier, it should derive from the declared service identity or be created later by an owning migration or deployment process whose evidence records it.

Output source	Policy	Reason
Service/context name	Declared input; appears in namespaces, paths, assembly names, passport.	Deterministic ownership identity.
Template version	Pinned package input; recorded in manifest and passport.	Supports reproduction and drift analysis.
Timestamp	Forbidden in generated source and manifest identity.	Makes equal input produce unequal output.
Random GUID	Forbidden unless derived deterministically or created by a later owned process.	Hides unexplained state inside scaffolding.
Machine/user path	Forbidden; staging paths are excluded from retained evidence.	Leaks environment detail and prevents independent repetition.
Package restore result	Not generated; captured by build evidence after integration.	Separates source generation from toolchain/network execution.

Instantiate RootCause without changing its meaning

The first accepted instance is RootCause because ADR-001 selected it. Generation creates the physical locations and pending evidence catalogue already described by Chapters 4-6. It does not create RootCauseCase, decide how evidence is accepted, invent a verdict API, choose a database schema, or implement AlarmFloodDetected.v1 consumption. Those are specializations under later chapters and preservation rules RC-INV-001, RC-INV-002, and RC-OWN-001.

Generated RootCause artifact	Immediate status	Next owning work
Domain/Application projects	STRUCTURE ONLY	Place inherited aggregate/use-case code and prove invariants in Chapters 8-9.
Infrastructure project	HOOKS ONLY	Implement RootCauseDb, messaging, identity and migrations in Parts III-V.
Contracts project	LOCATION ONLY	Define and test RootCauseVerdictIssued.v1 in Part IV.
RootCause Host	COMPOSITION SHAPE	Prove dependency direction, configuration, health and startup in Chapters 8-11.
Test projects	DISCOVERABILITY PENDING	Add executable invariant, architecture, component, contract and failure evidence.
Container/operations files	PLACEHOLDERS	Pin image digest and prove build, startup, shutdown, runbooks, alerts and recovery later.
Service passport	G0 ACCEPTED / G1 PENDING	Resolve TPL/BUILD/ARCH evidence without claiming runtime gates.

CURRENT VERDICT

Chapter 7 defines how RootCause should be generated. It does not prove that RootCause has been generated, compiled, run, or accepted at G1 in this environment.

Template upgrades are migrations

A generated service is not continually overwritten by the template. When version 0.2 changes health conventions, container policy, analyzer settings, or evidence schema, existing services need a reviewed migration. Some changes belong in centrally referenced ServiceDefaults or root policy and arrive through dependency/version updates. Others require local edits. The template release must publish both the new-instance manifest and a migration guide that states which existing services are affected.

Change type	Delivery path	Required evidence
Root build/analyzer policy	Directory.Build.props, package policy, or repository tooling update.	Full solution build, architecture/tests, documented exceptions.
ServiceDefaults behaviour	Versioned technical package update with compatibility notes.	Component/health/telemetry/security tests for affected hosts.
Generated local source	Explicit migration patch or guide applied by each service owner.	Diff review, service tests, passport revalidation, rollback plan.
New optional capability	New declaration schema plus adapter/evidence extension; no automatic enablement.	Template matrix, declaration validation, capability-specific tests.
Domain/application convention	Normally documentation or local refactoring, not a shared runtime abstraction.	Owner approval and preservation evidence for each affected context.

NO REGENERATION

Re-running a newer template over an owned service is forbidden. Upgrade through an explicit migration whose diff, evidence impact, and reversal are reviewable.

False greens in service generation

Apparently successful result	Hidden failure	Required correction
dotnet new reports success	Files may contain stale tokens, illegal references, collisions, or misleading placeholders.	Run manifest, token, graph, declaration, and security validation.
Generated projects compile alone	They may rely on different package policy or avoid the authoritative solution.	Restore/build Nexus1.Runtime.sln under the pinned root policy.
The host returns HTTP 200	No owned adapter, identity, readiness, contract or failure path is proved.	Keep runtime evidence pending and execute Chapters 8-12 gates.
Every service looks identical	The template may have erased service-specific language and capability differences.	Audit generated domain/application content for prohibited meaning.
A new template version exists	Existing services may drift without migration guidance or evidence impact analysis.	Publish instance manifest, migration guide, compatibility and rollback evidence.
A sample service works	The sample may contain manual repairs or dependencies not present in the template.	Generate independently from the packaged template and compare manifests.

ADR-005 - Adopt a generated golden path

ADR field	ADR-005 record
Status	Accepted for the reference implementation design; execution evidence pending.

ADR field	ADR-005 record
Context	Chapter 4 defined a context-first project graph and Chapter 5 limited shared code. Recreating that structure manually for every service invites drift, false health, missing tests, and inconsistent evidence.
Decision	Use a versioned multi-project .NET source template, deterministic staging orchestrator, structural validator, narrow ServiceDefaults surface, capability declaration, and generated service passport.
Ownership	Template/platform owners own mechanical defaults and migration guidance. Each service owner owns generated source, capabilities, domain meaning, adapters, evidence and deviations.
Current claim	RootCause remains G0 accepted with G1 specified. TPL-SCHEMA/GENTOKEN/GRAPH/BUILD/REPEAT/PASS/REVIEW evidence is pending.
Consequences	New services start consistently; generation is slower than copying a folder; template changes require tests and migrations; local meaning stays visible.
Rejected alternatives	Copy/paste checklist, one Web API template, shared application framework, code generator that owns aggregates/contracts, hidden post-actions, regeneration over owned code.
Reversal condition	Replace when the mechanism cannot preserve the required project graph or its maintenance cost exceeds its evidence value; retain the blocking ownership and dependency rules.

Risk register for the golden path

Risk	Control	Evidence owner
Template becomes a shared domain framework	Vocabulary scan, dependency rules, minimal empty Domain/Application projects.	Architecture + domain owners.
Option matrix becomes untestable	One stable skeleton; capabilities live in a validated declaration and evidence matrix.	Template maintainers.
Generated service overclaims readiness	Passport starts G0/G1-pending; no runtime evidence is pre-populated as accepted.	Service owner + gate reviewer.
Hidden post-action mutates repository	Generate in staging; reviewed orchestrator promotes an explicit manifest; no network/secret actions.	Repository engineering + security.
Template and root policy diverge	Pack/generate/build against the authoritative solution in CI on every release.	Build/toolchain owner.
Generated code is treated as immutable	Ownership-transfer rule and migration guides; no regeneration over service source.	Service engineering owner.
Template upgrade strands services	Version inventory, impact matrix, migration/rollback guide, revalidation triggers.	Template + service owners.
A sample hides manual steps	Independent generation from the package and normalized manifest comparison.	CI/evidence maintainer.
Defaults leak secrets or sensitive telemetry	Static scans, negative fixtures, bounded diagnostics and prohibited-field tests.	Security + observability owners.

Official-source validation

The reference syntax was checked against current Microsoft documentation on 18 July 2026. The official .NET template documentation confirms that templates use ordinary source files

packs can be installed from directories or packages. ASP.NET Core documentation distinguishes liveness from readiness. .NET observability documentation identifies ILogger, Meter, and ActivitySource as the native logging, metrics, and tracing surfaces.

Official source	Stable point used here	NEXUS-1 boundary
Microsoft Learn - Custom templates for dotnet new	Source files, template.json, sourceName, package installation and Template package shape.	Schema and generated output still require pinned-SDK validation.
Microsoft Learn - Health checks in ASP.NET Core	Liveness and readiness can represent different process/startup states.	Chapter 11 defines service-specific checks; no dependency health is invented here.
Microsoft Learn - .NET observability with OpenTelemetry	ILogger, Meter, ActivitySource/Activity provide native signal surfaces.	RootCause owns signal meaning, fields, cardinality, assertions and operational use.

The series baseline remains net8.0 as established in Chapter 4. A documentation page showing a newer framework version does not silently upgrade the book. Exact SDK patch, template engine behaviour, package versions, container images and generated build results must be captured by the executable repository.

Technical material still required

The golden path is specified but not yet executable in this manuscript environment. The template pack, content tree, declaration schema, orchestration scripts, manifest normalizer, token/secret/reference validators, template contract tests, ServiceDefaults implementation, root solution integration, container policy, and RootCause specialization must be created and run under the pinned toolchain.

TECHNICAL MATERIAL STATUS - [TEMPLATE PACKAGE + COMPILE EVIDENCE REQUIRED]

Create the complete template source and pack; validate template.json; implement staged generation and atomic promotion; generate RootCause twice; compare normalized manifests; scan tokens/secrets/references; integrate the exact project list; restore/build/test the authoritative solution; and publish TPL-SCHEMA-001 through TPL-REVIEW-001. Until then, G1 remains specified but not evidenced.

Deliverable	Minimum technical content	Blocking evidence
Template pack	Pack project, content tree, template.json, package version, locked source manifest.	TPL-SCHEMA-001
Service declaration	Schema, owner/capability/status validation, unknown-field and missing-obligation tests.	TPL-PASS-001
Repository orchestrator	Collision-safe staging, explicit promotion manifest, solution integration, rollback on failure.	TPL-GEN-001 / TPL-GRAPH-001
Validators	Token, secret, symlink, binary, namespace, project-reference and package-source checks.	TPL-TOKEN-001 / TPL-GRAPH-001
Generated RootCause	Context projects, host, tests, ops/container metadata, pending service passport.	TPL-GEN-001 / TPL-REVIEW-001
Build/test report	Pinned SDK/feed/package identity, full solution build, discovery counts, architecture result.	TPL-BUILD-001
Repeatability report	Two clean generations, normalization policy, file hashes and explained differences.	TPL-REPEAT-001

CHAPTER 7 - CODE AUDIT

The golden path becomes an executable repository transaction

The template generates ordinary owned source. The orchestrator owns staging, validation, promotion, and evidence; it never invents RootCause business behaviour.

EVIDENCE STATUS

The listings are compile-oriented reference assets. No dotnet executable is available in this environment, so template pack, generation, build, and repeatability results remain PENDING.

EXECUTABLE ASSET 7-A - TEMPLATE.JSON

```
{
  "$schema": "https://json.schemastore.org/template",
  "author": "NEXUS-1",
  "classifications": ["NEXUS-1", "Service", "Distributed"],
  "identity": "Nexus1.Templates.Service",
  "name": "NEXUS-1 golden-path service",
  "shortName": "nexus-service",
  "sourceName": "TemplateService",
  "preferNameDirectory": false,
  "tags": { "language": "C#", "type": "project" },
  "symbols": {
    "ServiceSlug": {
      "type": "parameter",
      "datatype": "text",
      "isRequired": true,
      "description": "Lower-case operational service identity"
    }
  },
  "sources": [{
    "source": "./",
    "target": "./",
    "exclude": [
      ".template.config/**", "**/bin/**", "**/obj/**",
      "**/*.user", "**/.vs/**"
    ]
  }]
}
```

Only ownership identity is a template parameter. Capabilities remain in the generated declaration, preventing an unbounded conditional-output matrix. No postActions are declared.

EXECUTABLE ASSET 7-B - TEMPLATE PACK PROJECT

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>netstandard2.0</TargetFramework>
    <PackageType>Template</PackageType>
    <PackageId>Nexus1.ServiceTemplate</PackageId>
    <PackageVersion>0.1.0-alpha</PackageVersion>
    <Title>NEXUS-1 Service Template</Title>
    <Authors>NEXUS-1</Authors>
    <Description>Golden-path repository fragment.</Description>
    <PackageTags>dotnet-new;templates;nexus-1</PackageTags>
    <IncludeContentInPack>true</IncludeContentInPack>
    <IncludeBuildOutput>false</IncludeBuildOutput>
    <ContentTargetFolders>content</ContentTargetFolders>
  </PropertyGroup>
  <ItemGroup>
    <Content Include="templates/**/*"
      Exclude="templates/**/bin/**;templates/**/obj/**" />
    <Compile Remove="**/*" />
  </ItemGroup>
</Project>
```

CAPABILITY DECLARATION

Capabilities add obligations, never exemptions

EXECUTABLE ASSET 7-C - ROOT-CAUSE.SERVICE.JSON

```
{
  "schemaVersion": "1.0",
  "service": "root-cause",
  "context": "RootCause",
  "template": {
    "identity": "Nexus1.Templates.Service",
    "version": "0.1.0-alpha",
    "sha256": "PENDING"
  },
  "owners": {
    "domain": "PENDING",
    "engineering": "PENDING",
    "operations": "PENDING"
  },
  "capabilities": {
    "http": "required",
    "messagingConsumer": "required",
    "messagingProducer": "required",
    "persistence": "required",
    "scheduledWork": "not-declared"
  },
  "targetGate": "G1"
}
```

EXECUTABLE ASSET 7-D - DECLARATION VALIDATOR

```
public static void Validate(ServiceDeclaration declaration)
{
    if (!Regex.IsMatch(
        declaration.Service, "^[a-z][a-z0-9-]{2,62}$"))
        throw new DeclarationViolation("Invalid service slug");

    if (!Regex.IsMatch(
        declaration.Context, "^[A-Z][A-Za-z0-9]+$"))
        throw new DeclarationViolation("Invalid context name");

    RequireOwner(declaration.Owners.Domain, "domain");
    RequireOwner(declaration.Owners.Engineering, "engineering");

    foreach (var capability in declaration.Capabilities)
        if (!AllowedCapabilities.Contains(capability.Key))
            throw new DeclarationViolation(
                $"Unknown capability: {capability.Key}");

    var requiredEvidence = CapabilityMatrix
        .RequiredEvidence(declaration.Capabilities);
    declaration.SeedEvidence(requiredEvidence, status: Pending);
}
```

For the current manuscript, owner fields intentionally remain PENDING and validation must therefore block promotion. The example shows the target contract, not a successful RootCause generation.

CORE OBLIGATIONS

Boundary, authorization hooks, observability, test discovery, ownership, and passport creation are unconditional. A capability can add an adapter and evidence IDs; it cannot remove them.

STAGED GENERATION

Repository mutation begins only after a complete inspection

EXECUTABLE ASSET 7-E - ORCHESTRATION TRANSACTION

```
public async Task<GenerationResult> GenerateAsync(
    GenerationRequest request,
    CancellationToken cancellationToken)
{
    declarationValidator.Validate(request.Declaration);

    await using var stage = await staging.CreateAsync(
        request.Service, cancellationToken);

    await templateEngine.GenerateAsync(
        package: request.TemplatePackage,
        shortName: "nexus-service",
        name: request.Declaration.Context,
        output: stage.Path,
        arguments: new()
        {
            ["ServiceSlug"] = request.Declaration.Service
        },
        cancellationToken);

    var inspection = await validator.InspectAsync(
        stage.Path, request.Declaration, cancellationToken);
    inspection.ThrowIfRejected();

    var manifest = await manifestBuilder.CreateAsync(
        stage.Path, request, cancellationToken);

    return new GenerationResult(stage, manifest, inspection);
}
```

EXECUTABLE ASSET 7-F - EXPLICIT PROMOTION

```
public async Task PromoteAsync(
    GenerationResult result,
    RepositoryRoot repository,
    CancellationToken cancellationToken)
{
    result.Inspection.ThrowIfRejected();

    foreach (var file in result.Manifest.Files)
    {
        var target = repository.Resolve(file.RepositoryPath);
        if (File.Exists(target))
            throw new PromotionViolation($"Collision: {target}");
    }

    await repository.ApplyAtomicallyAsync(async transaction =>
    {
        foreach (var file in result.Manifest.Files)
            await transaction.CopyVerifiedAsync(
                result.Stage.Resolve(file.StagePath),
                file.RepositoryPath,
                file.Sha256,
                cancellationToken);

        await transaction.AddProjectsAsync(
            result.Manifest.Projects, cancellationToken);
        await transaction.WriteEvidenceAsync(
            result.Manifest, cancellationToken);
    });
}
```

ATOMIC MEANS REVIEWABLE

The repository adapter must implement rollback or an atomic workspace publication strategy. The loop alone is not atomic; the transaction boundary is a required adapter contract.

STRUCTURAL INSPECTION

Generated files are hostile input until proven otherwise

EXECUTABLE ASSET 7-G - STAGED TREE INSPECTION

```
public async Task<InspectionReport> InspectAsync(
    DirectoryInfo stage,
    ServiceDeclaration declaration,
    CancellationToken cancellationToken)
{
    var findings = new List<Finding>();

    foreach (var entry in stage.EnumerateFileSystemInfos(
        "*", SearchOption.AllDirectories))
    {
        RejectSymlink(entry, findings);
        RejectOutsideRoot(stage, entry, findings);

        if (entry is not FileInfo file) continue;
        RejectBinary(file, findings);
        RejectSecret(file, findings);
        RejectReservedToken(file, "TemplateService", findings);
        RejectForbiddenPackageSource(file, findings);
    }

    findings.AddRange(await projectGraph.ValidateAsync(
        stage, declaration.Context, cancellationToken));
    findings.AddRange(passport.ValidateSeed(stage, declaration));

    return InspectionReport.From(findings);
}
```

EXECUTABLE ASSET 7-H - PATH CONTAINMENT AND SYMLINK GUARD

```
private static void RejectOutsideRoot(
    DirectoryInfo root,
    FileSystemInfo entry,
    ICollection<Finding> findings)
{
    var rootPath = Path.GetFullPath(root.FullName)
        .TrimEnd(Path.DirectorySeparatorChar)
        + Path.DirectorySeparatorChar;
    var entryPath = Path.GetFullPath(entry.FullName);

    if (!entryPath.StartsWith(
        rootPath, StringComparison.Ordinal))
        findings.Add(Block("TPL-GEN-001", entryPath, "Path escape"));
}

private static void RejectSymlink(
    FileSystemInfo entry,
    ICollection<Finding> findings)
{
    if ((entry.Attributes & FileAttributes.ReparsePoint) != 0 ||
        entry.LinkTarget is not null)
        findings.Add(Block("TPL-GEN-001", entry.FullName, "Symlink"));
}
```

Secret detection combines reviewed patterns, entropy checks, and allow-listed fixtures. A match is a blocking finding until reviewed; the validator must not print secret contents into CI logs.

PROJECT GRAPH

The generated graph is compared with an exact allow-list

EXECUTABLE ASSET 7-I - EXPECTED ROOTCAUSE EDGES

```
private static readonly IReadOnlyDictionary<string, string[]> Allowed =
    new Dictionary<string, string[]>
    {
        ["Nexus1.RootCause.Domain"] =
            ["Nexus1.BuildingBlocks.Domain"],
        ["Nexus1.RootCause.Application"] =
            ["Nexus1.RootCause.Domain"],
        ["Nexus1.RootCause.Infrastructure"] =
            [
                "Nexus1.RootCause.Application",
                "Nexus1.RootCause.Domain",
                "Nexus1.Contracts.AlarmManagement"
            ],
        ["Nexus1.Contracts.RootCause"] = [],
        ["Nexus1.RootCause.Host"] =
            [
                "Nexus1.RootCause.Application",
                "Nexus1.RootCause.Infrastructure",
                "Nexus1.Contracts.RootCause",
                "Nexus1.ServiceDefaults"
            ]
    };
```

EXECUTABLE ASSET 7-J - GRAPH EQUALITY TEST

```
[Fact]
public async Task TPL_GRAPH_001_generated_edges_are_exact()
{
    var graph = await ProjectGraph.LoadAsync(Fixture.Stage);

    graph.ProjectNames.Should().BeEquivalentTo(
        Allowed.Keys,
        options => options.WithStrictOrdering());

    foreach (var project in Allowed)
        graph.ReferencesOf(project.Key)
            .Should().BeEquivalentTo(project.Value);

    graph.References.Should().NotContain(edge =>
        edge.From.EndsWith(".Domain") &&
        edge.To.Contains(".Infrastructure"));
}
```

The final repository test resolves evaluated MSBuild project references, not only literal XML. Generated props, imports, and conditional references must therefore appear in the inspected graph.

NO CONVENIENT EXCEPTION

If the first generated service requires a forbidden edge, the template or boundary is wrong. The test is changed only through an explicit architecture decision.

REPEATABILITY

Two isolated generations produce the same normalized manifest

EXECUTABLE ASSET 7-K - DETERMINISTIC FILE MANIFEST

```
public async Task<GenerationManifest> CreateAsync(
    DirectoryInfo root,
    GenerationRequest request,
    CancellationToken cancellationToken)
{
    var files = new List<GeneratedFile>();
    foreach (var file in root.EnumerateFiles(
        "*", SearchOption.AllDirectories)
        .OrderBy(x => Relative(root, x),
            StringComparer.Ordinal))
    {
        await using var stream = file.OpenRead();
        files.Add(new(
            Path: Relative(root, file).Replace('\\', '/'),
            Length: file.Length,
            Sha256: Convert.ToHexString(
                await SHA256.HashDataAsync(stream, cancellationToken)));
    }

    return new(
        TemplateId: request.TemplateId,
        TemplateVersion: request.TemplateVersion,
        TemplateSha256: request.TemplateSha256,
        Inputs: request.DeclaredInputs,
        Files: files);
}
```

EXECUTABLE ASSET 7-L - REPEATABILITY PROOF

```
[Fact]
public async Task TPL_REPEAT_001_same_inputs_have_same_manifest()
{
    var request = Fixture.RootCauseRequest();

    var first = await generator.GenerateAsync(
        request, TestContext.Current.CancellationToken);
    var second = await generator.GenerateAsync(
        request, TestContext.Current.CancellationToken);

    Normalize(first.Manifest).Should().BeEquivalentTo(
        Normalize(second.Manifest),
        options => options.WithStrictOrdering());
}

private static object Normalize(GenerationManifest manifest) => new
{
    manifest.TemplateId,
    manifest.TemplateVersion,
    manifest.TemplateSha256,
    Inputs = manifest.Inputs.OrderBy(x => x.Key),
    Files = manifest.Files.OrderBy(x => x.Path)
};
```

No timestamp, staging path, user name, machine identity, or random GUID belongs in the identity manifest. Execution time and runner identity belong in the surrounding evidence record.

TEMPLATE CONTRACT TESTS

The packaged template is tested from outside its source tree

EXECUTABLE ASSET 7-M - CLEAN PACKAGE/INSTALL/GENERATE TEST

```
#!/usr/bin/env bash
set -euo pipefail

out="$(mktemp -d)"
pack="$out/pack"
first="$out/first"
second="$out/second"

dotnet pack templates/Nexus1.ServiceTemplate.Pack \
  -c Release -o "$pack"

nupkg="$pack/Nexus1.ServiceTemplate.0.1.0-alpha.nupkg"
sha256sum "$nupkg" > "$out/template.sha256"

dotnet new install "$nupkg"
trap 'dotnet new uninstall Nexus1.ServiceTemplate' EXIT

dotnet new nexus-service -n RootCause \
  --ServiceSlug root-cause -o "$first"
dotnet new nexus-service -n RootCause \
  --ServiceSlug root-cause -o "$second"

dotnet run --project tools/Nexus1.TemplateValidator -- \
  inspect "$first" services/root-cause.service.json
dotnet run --project tools/Nexus1.TemplateValidator -- \
  compare "$first" "$second"
```

TEST ASSET - EXACT GENERATED SKELETON

```
[Fact]
public void TPL_GEN_001_contains_the_required_repository_fragment()
{
    RelativeFiles(stage).Should().Contain(
    [
        "src/Contexts/RootCause/Nexus1.RootCause.Domain/" +
        "Nexus1.RootCause.Domain.csproj",
        "src/Contexts/RootCause/Nexus1.RootCause.Application/" +
        "Nexus1.RootCause.Application.csproj",
        "src/Contexts/RootCause/Nexus1.RootCause.Infrastructure/" +
        "Nexus1.RootCause.Infrastructure.csproj",
        "src/Hosts/Nexus1.RootCause.Host/" +
        "Nexus1.RootCause.Host.csproj",
        "tests/Nexus1.RootCause.UnitTests/" +
        "Nexus1.RootCause.UnitTests.csproj",
        "ops/RootCause/service-passport.json"
    ]
    );
}
```

CLEAN-ROOM RULE

The generated projects must not require template source paths, local caches, hidden imports, or manual repair. The package hash and installed identity are retained with TPL-SCHEMA-001.

REPOSITORY INTEGRATION

One CI path proves generation, promotion, build, and evidence shape

EXECUTABLE ASSET 7-N - EXPLICIT SOLUTION PROJECT LIST

```
private static readonly string[] RootCauseProjects =
[
    "src/Contexts/RootCause/Nexus1.RootCause.Domain/" +
    "Nexus1.RootCause.Domain.csproj",
    "src/Contexts/RootCause/Nexus1.RootCause.Application/" +
    "Nexus1.RootCause.Application.csproj",
    "src/Contexts/RootCause/Nexus1.RootCause.Infrastructure/" +
    "Nexus1.RootCause.Infrastructure.csproj",
    "src/Contracts/Nexus1.Contracts.RootCause/" +
    "Nexus1.Contracts.RootCause.csproj",
    "src/Hosts/Nexus1.RootCause.Host/" +
    "Nexus1.RootCause.Host.csproj",
    "tests/Nexus1.RootCause.UnitTests/" +
    "Nexus1.RootCause.UnitTests.csproj",
    "tests/Nexus1.RootCause.ComponentTests/" +
    "Nexus1.RootCause.ComponentTests.csproj"
];

foreach (var project in RootCauseProjects)
    await solution.AddExactAsync(project, cancellationToken);
```

CI ASSET - GOLDEN-PATH EVIDENCE JOB

```
- name: Generate and validate RootCause twice
  run: ./eng/verify-golden-path.sh

- name: Build authoritative solution
  run: |
    dotnet restore Nexus1.Runtime.sln --locked-mode
    dotnet build Nexus1.Runtime.sln -c Release --no-restore
    dotnet test Nexus1.Runtime.sln -c Release --no-build \
      --logger "trx;LogFileName=template-tests.trx"

- name: Retain TPL evidence
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: golden-path-${{ github.sha }}
    path: |
      artifacts/template/**
      **/template-tests.trx
    retention-days: 90
```

Official-source validation

Checked 29 July 2026. Microsoft documents `template.json` under `.template.config`, `sourceName` replacement, Template package metadata, content-only packing, and installation from a local nupkg. Microsoft also warns that templates can execute arbitrary code; NEXUS-1 therefore pins and reviews the package and declares no hidden post-actions.

Source: <https://learn.microsoft.com/dotnet/core/tools/custom-templates>

REVISION RESULT

Chapter 7 now supplies the template and declaration contracts, staged transaction, structural and graph validators, deterministic manifest, clean package test, explicit integration list, and CI evidence path. G1 remains pending until those assets execute under the pinned SDK.

Implementation sequence

1. Commit ADR-005, the template responsibility boundary, capability declaration schema, and TPL evidence catalogue.
2. Create the template-pack project and the complete multi-project repository-fragment source tree.
3. Validate template.json against its schema and the pinned SDK; pack the template with no runtime binaries.
4. Implement the staging orchestrator with name/collision checks, explicit input capture, and no hidden network or secret access.
5. Implement token, binary, symlink, secret, namespace, package-source, project-graph, and passport validators.
6. Generate RootCause into two clean staging directories and compare normalized manifests for repeatability.
7. Promote one validated fragment, add the exact project list to Nexus1.Runtime.sln, and preserve repository-root policy.
8. Implement the minimum ServiceDefaults surface and the service-owned composition hooks without business behaviour.
9. Restore, build and test the authoritative solution; record discovery counts, architecture results and every exception.
10. Review the generated RootCause instance, keep runtime evidence pending, and accept G1 only if every inherited blocker passes.

What this chapter has proved

Chapter 7 has defined the golden path as a versioned source template, a deterministic repository orchestrator, a structural validator, a narrow ServiceDefaults contract, a capability declaration, and an evidence-producing acceptance flow. It has constrained what the template may own, defined the generated project graph, separated generation from integration, and treated template upgrades as migrations.

It has not generated or compiled RootCause. The reference template.json, package project, code and commands remain design artifacts until the pinned repository executes them. The current claim therefore does not advance: RootCause is G0 accepted, its G1 boundary and golden path are specified, and TPL/BUILD/ARCH evidence remains pending.

CHAPTER CLOSING

A golden path is successful when it makes the safe mechanical route easy, makes deviation visible, and leaves every statement of business truth with the generated service owner.

Chapter checkpoint

You are ready for Chapter 8 when the template can be discussed without confusing generation, composition, service meaning, or evidence.

Question	Expected answer
What does the golden path own?	Repeatable project, host, configuration, health, telemetry, test, container and evidence mechanics.
What must it never own?	Domain language, invariants, use cases, source truth, contract meaning, authorization or repair policy.
Why use a staging directory?	A failed or invalid generation must not partially mutate the authoritative repository.
Why is dotnet new success insufficient?	It proves only template-engine completion, not tokens, references, build, tests, ownership, health or evidence.
Who owns generated code?	The service team after acceptance; template upgrades arrive through reviewed migrations, never regeneration over owned source.
Why are capabilities declared separately?	To avoid an untestable option matrix and to attach explicit evidence obligations without changing the stable skeleton.
What is the current RootCause gate?	G0 accepted; G1 and the golden path are specified, but generation/compile/architecture evidence is pending.
What must Chapter 8 add?	Executable host composition rules that preserve dependency direction and keep business decisions outside the composition root.

NEXT CHAPTER

Chapter 8 defines Host Composition and Dependency Direction. It turns the generated registration hooks into a composition root, proves which layer may instantiate which adapter, constrains background services and endpoint wiring, and makes startup and shutdown ownership explicit without moving RootCause decisions into Program.cs.

PART II - THE SERVICE FOUNDATION

8. Host Composition and Dependency Direction

The composition root may know every concrete adapter so that the rest of the service does not. Its job is to assemble one valid object graph, expose explicit entry points, and own process lifetime - never to decide what RootCause means.

CHAPTER ARGUMENT

Keep object creation and adapter selection at the outer boundary. Make compile-time references point toward policy, make runtime lifetimes explicit, and make every entry point enter the same application-owned decisions.

Chapter 7 created reference hooks: AddNexusServiceDefaults, AddRootCauseApplication, AddRootCauseInfrastructure, MapNexusServiceEndpoints, and MapRootCauseEndpoints. Those names are useful only if their implementation preserves the project graph from Chapter 4 and the sharing limits from Chapter 5. A registration extension that reaches across layers, scans assemblies for business behaviour, or hides a second container would turn the golden path into a distributed monolith generator.

Chapter 8 therefore treats composition as architecture, not startup ceremony. The host is the one place allowed to select concrete adapters, attach framework lifetimes, start background entry points, and connect transport to application ports. Domain and Application code remain ignorant of the host, the service container, ASP.NET Core, persistence, messaging, and deployment environment.

The code in this chapter is a reference contract. The manuscript does not contain the generated RootCause repository or pinned SDK, so it has not compiled Program.cs, built the provider, resolved an entry point, started a worker, or observed shutdown. The current claim remains G0 accepted and G1 specified. Composition evidence is pending.

The composition root has a narrow job

Responsibility	Host must do	Host must not do
Select adapters	Bind application-owned ports to service-owned infrastructure implementations.	Choose an adapter from request data or a business condition.
Declare lifetimes	Register singleton, scoped, and transient services with explicit reasons.	Promote scoped state into a singleton or rely on accidental defaults.
Expose entry points	Map HTTP groups, technical endpoints, and hosted adapters.	Implement commands, invariants, retries, repair policy, or response meaning.
Own process lifecycle	Build once, start once, honor cancellation, and dispose the root provider once.	Create nested providers, block constructors, or continue work after cancellation.
Select environment mechanics	Read technical configuration and choose declared technical implementations.	Fork domain rules by environment or hide a production-only object graph.
Produce evidence	Publish registration, endpoint, worker, startup, and shutdown manifests.	Treat a successful process start as proof that the graph is correct.

COMPOSITION-ROOT TEST

If a line in Program.cs answers a business question, it belongs elsewhere. If a concrete adapter is selected elsewhere, the composition boundary has leaked.

Dependency direction is a compile-time rule

Runtime calls travel in many directions: an endpoint calls a use case, a use case calls a repository port, and an infrastructure adapter calls a database. Compile-time dependencies must still point toward owned policy. The application owns the port; infrastructure implements it. The host references both only to join them. This is dependency inversion without pretending that adapters do not exist.

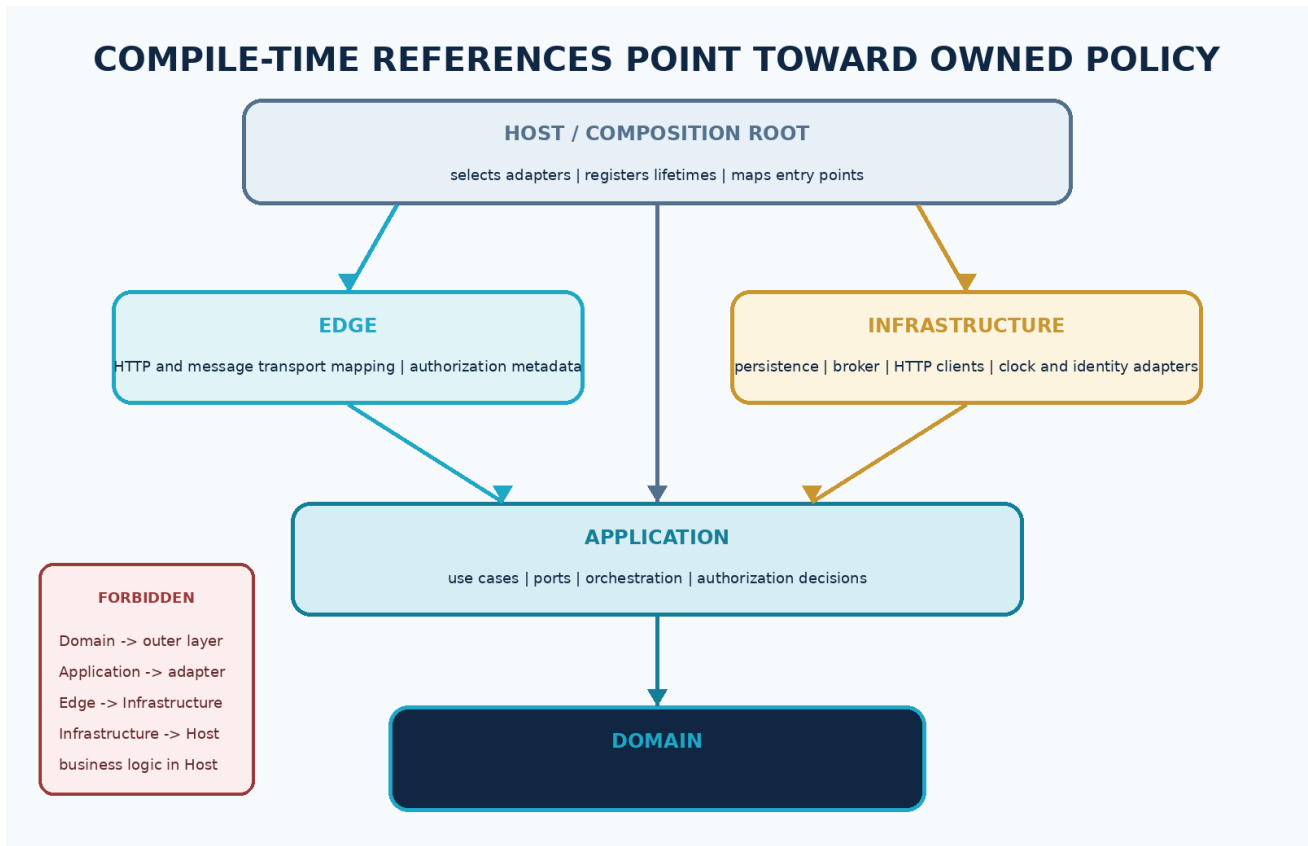


Figure 8.1 - The Host sees outer adapters so inner policy remains independent of them.

Project	May reference	Must remain invisible
Nexus1.RootCause.Domain	BCL and explicitly approved dependency-poor building blocks.	Application, Edge, Infrastructure, Host, contracts, ASP.NET Core, data and broker packages.
Nexus1.RootCause.Application	Domain and application-owned ports.	Host, Edge, concrete persistence, concrete messaging, HTTP clients, environment APIs.
Nexus1.RootCause.Edge	Application, public contracts, transport and authorization abstractions.	Infrastructure implementations, database contexts, broker clients, Host internals.
Nexus1.RootCause.Infrastructure	Application ports, Domain types, required producer-owned contracts, approved technical packages.	Host, Edge implementation, another context's private projects.
Nexus1.RootCause.Host	Application, Edge, Infrastructure, required contracts, ServiceDefaults.	Business implementation, private projects of another service, direct data ownership.

DIRECTION IS NOT A DIAGRAM PREFERENCE

A forbidden project reference is a failed G1 blocker. Dependency injection cannot repair a compile-time dependency that points outward.

Ports belong to the policy that needs them

`IRootCauseCaseStore` is not an infrastructure abstraction merely because `SQL` implements it. The

Application. Infrastructure supplies `SqlRootCauseCaseStore`. The host performs the binding. This keeps the use case testable without moving the service's language into a shared repository interface.

APPLICATION-OWNED PORT, INFRASTRUCTURE-OWNED ADAPTER - [NEW][COMPILE REQUIRED]

```
// Nexus1.RootCause.Application
public interface IRootCauseCaseStore
{
    Task<RootCauseCase?> FindAsync(
        RootCauseCaseId id,
        CancellationToken cancellationToken);

    Task SaveAsync(
        RootCauseCase rootCauseCase,
        CancellationToken cancellationToken);
}

// Nexus1.RootCause.Infrastructure
internal sealed class SqlRootCauseCaseStore(
    RootCauseDbContext db) : IRootCauseCaseStore
{
    // Persistence mapping remains adapter-owned.
}
```

NO GENERIC REPOSITORY ESCAPE HATCH

A generic `IRepository<T>` in `BuildingBlocks` would erase `RootCause` transaction, concurrency, and aggregate-loading semantics. Define the narrow port the use case actually requires.

Registration modules reveal ownership

Each project exposes one deliberate registration surface. Application registers its use cases and application policies. Infrastructure binds application ports to concrete adapters. Edge registers transport-only collaborators. `ServiceDefaults` registers only approved host mechanics. The host calls those surfaces explicitly; it does not scan every assembly and infer service meaning from naming conventions.

OWNED REGISTRATION MODULES - REFERENCE [NEW][COMPILE REQUIRED]

```
// Application/RootCauseApplicationRegistration.cs
public static class RootCauseApplicationRegistration
{
    public static IServiceCollection AddRootCauseApplication(
        this IServiceCollection services)
    {
        services.AddScoped<IExplainAlarmFlood, ExplainAlarmFloodHandler>();
        services.AddScoped<IAcceptRootCauseVerdict, AcceptRootCauseVerdictHandler>();
        return services;
    }
}

// Infrastructure/RootCauseInfrastructureRegistration.cs
public static class RootCauseInfrastructureRegistration
{
    public static IServiceCollection AddRootCauseInfrastructure(
        this IServiceCollection services,
        IConfiguration configuration)
    {
        services.AddScoped<IRootCauseCaseStore, SqlRootCauseCaseStore>();
        services.AddScoped<IAlarmFloodInbox, SqlAlarmFloodInbox>();
        services.AddHttpClient<IAlarmManagementClient, AlarmManagementClient>();
        services.AddHostedService<AlarmFloodInboxWorker>();
        return services;
    }
}
```

Rule	Reason	Validator
One public Add... surface per owned module	The host can see composition without concrete registration noise.	Registration API inventory.

Rule	Reason	Validator
No assembly scanning for business handlers	A naming convention must not become hidden domain wiring.	Package and reflection scan policy.
No BuildServiceProvider during registration	A nested or temporary root creates duplicate singletons and disposal ambiguity.	Source scan plus provider-build test.
No GetService in application code	Runtime service location hides dependencies and mixes inversion strategies.	Forbidden-symbol architecture test.
No silent duplicate binding for a singular port	Last-registration-wins behaviour must not choose business capability accidentally.	Descriptor uniqueness validator.
Factories are synchronous and side-effect free	Registration must not perform I/O, migrations, remote calls, or blocking startup work.	Factory review and boot test.

Program.cs is declarative and boring

The generated composition root extends Chapter 7's reference contract. Its sequence is visible: create the builder, enable provider validation, add technical defaults, add service-owned modules, build once, map technical and service endpoints, and run. The exact APIs remain subject to compilation under the pinned repository.

ROOTCAUSE COMPOSITION ROOT - REFERENCE [EXTEND][COMPILE REQUIRED]

```
var builder = WebApplication.CreateBuilder(args);

builder.Host.UseDefaultServiceProvider((context, options) =>
{
    options.ValidateScopes = true;
    options.ValidateOnBuild = true;
});

builder.AddNexusServiceDefaults(options =>
{
    options.ServiceName = "root-cause";
    options.ConfigurationSection = "RootCause";
});

builder.Services.AddRootCauseApplication();
builder.Services.AddRootCauseInfrastructure(builder.Configuration);
builder.Services.AddRootCauseEdge();

var app = builder.Build();

app.MapNexusServiceEndpoints();
app.MapRootCauseEndpoints();

await app.RunAsync();
```

READING PROGRAM.CS

The file names the modules and entry points, but not their business behaviour. A reviewer can see the complete outer composition without learning how a RootCause verdict is produced.

Lifetimes are part of the architecture

A correct reference graph can still fail at runtime when a longer-lived object captures shorter-lived state. NEXUS-1 therefore requires every non-trivial registration to declare its lifetime rationale. Scoped is the default for application use cases and transactional adapters, not because scoped is universally safe, but because one request or one explicit worker scope needs one coherent object graph.

Service kind	Expected lifetime	Reason and prohibition
Domain value or policy	Usually constructed directly or transient	Keep state explicit; do not use the container as a domain object registry.
Application use case	Scoped	One execution graph; must not be retained by a singleton or static field.
DbContext, unit of work, repository adapter	Scoped	One transactional boundary; never resolve from the root provider.
Typed outbound client	Factory-managed	Handler lifetime is not request scope; do not store request-specific state in handlers.
Hosted service wrapper	Singleton	Framework lifetime; inject only singleton-safe collaborators and a scope factory.
Immutable technical options	Container-managed	Configuration ownership is defined in Chapter 10; never register mutable business state.
Shared cache or coordinator	Singleton only by explicit decision	Must be thread-safe, bounded, observable, disposable, and free of request-scoped state.

CAPTIVE DEPENDENCY

A singleton that holds a scoped service has changed that service's effective lifetime. Scope validation must fail the composition before runtime evidence is accepted.

Provider validation is necessary, not sufficient

ValidateScopes detects scoped services resolved from the root or captured by singletons. ValidateOnBuild asks the provider to validate creatable registrations when it builds. Neither proves external configuration, endpoint metadata, open-generic paths, factory side effects, runtime branching, worker iteration, or graceful shutdown. NEXUS-1 therefore combines provider validation with explicit root resolution and runtime smoke evidence.

COMPOSITION SMOKE TEST - [NEW][COMPILE REQUIRED]

```
[Fact]
public async Task Production_composition_builds_and_resolves_declared_roots()
{
    await using var app = await RootCauseHostFixture.BuildAsync(
        environment: "Production",
        useControlledAdapters: true);

    await using var scope = app.Services.CreateAsyncScope();

    scope.ServiceProvider.GetRequiredService<IExplainAlarmFlood>();
    scope.ServiceProvider.GetRequiredService<IAcceptRootCauseVerdict>();

    Assert.Single(app.Services
        .GetServices<IHostedService>()
        .OfType<AlarmFloodInboxWorker>());
}
```

The fixture must build the same registration path as the real host. Controlled adapters may replace external effects, but they may not bypass application or infrastructure projects, suppress required registrations, or create a separate test-only composition model.

Scope ownership follows the entry point

ASP.NET Core creates a scope for an HTTP request. A BackgroundService is registered as a singleton and receives no automatic per-iteration scope. Those facts produce two different lifetime boundaries. The endpoint accepts a framework-owned request scope; the hosted adapter creates and disposes a scope for each bounded unit of work.

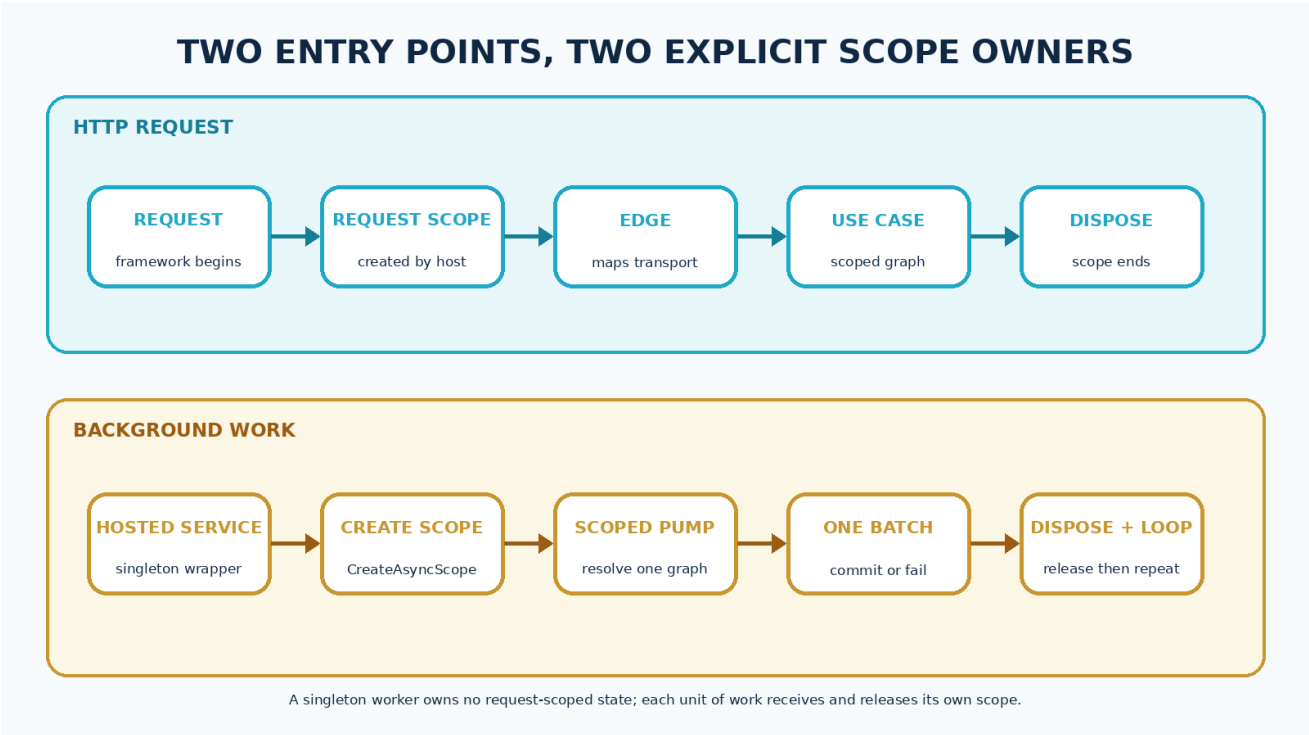


Figure 8.2 - HTTP and background entry points enter the same application layer through different scope owners.

Entry point	Scope owner	Unit of work	Disposal boundary
HTTP endpoint	ASP.NET Core request pipeline	One authorized request or explicitly split operation.	End of request scope.
Message pump	Hosted adapter through <code>IServiceScopeFactory</code>	One bounded fetch/handle/commit batch as later messaging policy defines.	After each batch, before delay or retry.
Startup initializer	Explicit startup coordinator only if declared	One idempotent, bounded technical initialization task.	Before readiness; never an unreviewed migration.
Shutdown drain	Host lifecycle and adapter	Stop intake, finish or abandon according to declared policy.	Before host timeout; residual work remains recoverable.

A background worker is an adapter, not a use case

The singleton worker owns the loop, cancellation, delay, logging, and scope boundary. It delegates message interpretation, idempotency, transaction, and `RootCause` decisions to a scoped application-facing pump. The `GetRequiredService` call is confined to the explicit lifetime boundary created by the adapter; it is not available to Domain or Application code.

HOSTED ADAPTER WITH PER-BATCH SCOPE - REFERENCE [NEW][COMPILE REQUIRED]

```
internal sealed class AlarmFloodInboxWorker(
    IServiceScopeFactory scopeFactory,
    ILogger<AlarmFloodInboxWorker> logger)
    : BackgroundService
{
    protected override async Task ExecuteAsync(
        CancellationToken stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            await using var scope = scopeFactory.CreateAsyncScope();
            var pump = scope.ServiceProvider
                .GetRequiredService<IAlarmFloodInboxPump>();

            await pump.RunOneBatchAsync(stoppingToken);
        }
    }

    public override async Task StopAsync(CancellationToken cancellationToken)
    {
        logger.LogInformation("Stopping AlarmFlood inbox worker");
        await base.StopAsync(cancellationToken);
    }
}
```

SHUTDOWN TRUTH

StopAsync is an opportunity for bounded graceful work, not a guarantee that every process exits cleanly. Later messaging chapters must make interrupted work safe to replay.

Endpoint wiring stays at the edge

MapRootCauseEndpoints is service-owned Edge code. It may group routes, declare authorization metadata, bind transport values, invoke an application port, and map an application outcome to an HTTP contract. It must not load a DbContext, call a broker, mutate a domain entity directly, or treat transport validation as the domain invariant.

THIN HTTP EDGE - REFERENCE [NEW][COMPILE REQUIRED]

```
public static RouteGroupBuilder MapRootCauseEndpoints(
    this IEndpointRouteBuilder endpoints)
{
    var group = endpoints
        .MapGroup("/api/root-causes")
        .WithTags("RootCause")
        .RequireAuthorization("rootcause.operator");

    group.MapPost("/explanations", ExplainAsync)
        .WithName("ExplainAlarmFlood");

    return group;
}

private static async Task<IResult> ExplainAsync(
    ExplainAlarmFloodRequest request,
    IExplainAlarmFlood useCase,
    CancellationToken cancellationToken)
{
    var command = RootCauseHttpMapping.ToCommand(request);
    var outcome = await useCase.ExecuteAsync(command, cancellationToken);
    return RootCauseHttpMapping.ToResult(outcome);
}
```

Edge may own	Application must own	Later chapter evidence
Route, verb, media type, binding and status-code mapping.	Use-case orchestration and decision outcome.	Part IV contract and compatibility tests.
Authentication/authorization metadata attachment.	Resource and domain authorization decision through explicit ports.	Security tests and denial evidence.

Edge may own	Application must own	Later chapter evidence
Transport-shape validation and safe error serialization.	Invariants, state transitions and conflict semantics.	Chapter 9 layer tests and Part IV problem contracts.
Correlation and request cancellation propagation.	Domain meaning of idempotency, retry, or cancellation effects.	Parts V-VII distributed-flow evidence.

ONE APPLICATION PATH

HTTP, message, and administrative entry points may have different transport adapters. They must not duplicate or bypass the application-owned decision merely to simplify one transport.

Middleware is technical policy, not a secret application layer

Host middleware may own correlation, authentication integration, exception-to-problem conversion, request logging, rate-limit mechanics, and technical headers. It must not infer RootCause authorization from route strings, repair cases, choose verdict policy, or swallow failures that the application needs to classify. Middleware order is composition evidence because changing it can change security and failure behaviour.

Order concern	Required evidence	False green
Forwarded headers / trusted proxy	Known deployment boundary and spoofing test.	Accept every forwarded header in every environment.
Authentication before authorization	Protected endpoint inventory and anonymous-denial test.	Authorization metadata exists but is never enforced.
Correlation and exception mapping	Trace/error contract assertion with prohibited-field scan.	Logs contain an ID while responses leak private exception detail.
Rate limiting or admission control	Declared policy, client identity basis and overload result.	Generic limiter silently changes business priority.

Registration order must not become hidden precedence

The built-in container can hold multiple registrations for one service type, and a single resolution can select the last registration. NEXUS-1 treats that as dangerous for singular business ports. A repository, authorization policy, clock, publisher, or case-number allocator must have one declared binding unless the application explicitly requests an ordered collection and defines the collection semantics.

Pattern	Policy	Evidence
One implementation for a singular port	Exactly one descriptor after all modules are registered.	CMP-DI-001 descriptor uniqueness report.
Several observers or enrichers	Application owns IEnumerable<T> ordering/independence semantics.	Collection order and partial-failure tests.
Environment-specific technical adapter	One explicit selection at composition; both graphs are validated.	Per-environment composition manifest diff.
Test replacement	Replace one known descriptor in the real graph; fail if target is missing or duplicated.	Fixture mutation log and production-path assertion.
TryAdd or silent fallback	Allowed only for a documented technical default with an override test.	Default/override pair evidence.

NO ORDER-BASED BUSINESS SELECTION

If module order chooses the RootCause implementation, the business decision is implicit and

unreviewable. Make the selection a named composition decision or move it into application policy.

The composition manifest makes the graph reviewable

Source code is authoritative, but a normalized composition manifest makes drift comparable. The build harness records declared roots, port bindings, lifetimes, workers, endpoint groups, middleware order, environment-specific differences, and the source/package identities that produced the graph. It contains type and policy identity, never secrets or resolved configuration values.

NORMALIZED ROOTCAUSE COMPOSITION MANIFEST - [NEW]

```
service: root-cause
target_framework: net8.0
composition:
  application:
    - service: IExplainAlarmFlood
      implementation: ExplainAlarmFloodHandler
      lifetime: scoped
  ports:
    - port: IRootCauseCaseStore
      implementation: SqlRootCauseCaseStore
      owner: infrastructure
      lifetime: scoped
  endpoints:
    - kind: http
      group: /api/root-causes
      authorization: rootcause.operator
    - kind: worker
      implementation: AlarmFloodInboxWorker
      lifetime: singleton
      work_scope: per-batch
  validation:
    scopes: required
    on_build: required
evidence_status: pending
```

Manifest check	Blocking condition	Evidence ID
Declared root resolves	Missing or ambiguous use-case, endpoint, worker, or technical root.	CMP-DI-001
Project direction matches graph	Domain/Application reference an outer project or Host behaviour leaks inward.	CMP-REF-001
Lifetime graph is safe	Root resolution, captive scope, root-scoped disposable, or leaked worker scope.	CMP-LIFE-001
Endpoint inventory is explicit	Unowned route, missing authorization metadata, or technical endpoint collision.	CMP-HTTP-001
Worker inventory is explicit	Duplicate worker, missing per-work scope, ignored cancellation, or unknown shutdown owner.	CMP-WORKER-001
Environment diff is approved	Business binding changes or unreviewed adapter disappears in production.	CMP-ENV-001

Architecture tests enforce the compile boundary

Project-reference validation should inspect both project files and compiled assemblies. Project files detect declared edges; assembly metadata detects the references that survived compilation. Namespace conventions and folder names are supporting signals, not architecture evidence.

COMPILED DEPENDENCY-DIRECTION TEST - [NEW][COMPILE REQUIRED]

```
public sealed class DependencyDirectionTests
{
    private static readonly HashSet<string> DomainForbidden = new(
        StringComparer.Ordinal)
    {
        "Nexus1.RootCause.Application",
        "Nexus1.RootCause.Edge",
        "Nexus1.RootCause.Infrastructure",
        "Nexus1.RootCause.Host"
    };

    [Fact]
    public void Domain_has_no_outer_project_reference()
    {
        var references = typeof(RootCauseCase).Assembly
            .GetReferencedAssemblies();

        Assert.DoesNotContain(
            references,
            reference => reference.Name is not null
                && DomainForbidden.Contains(reference.Name));
    }
}
```

Test surface	What must fail	Important limitation
Project reference graph	Any forbidden ProjectReference, including transitive policy defined by the repository.	Does not show unused package code or runtime resolution.
Compiled assembly references	Outer service assembly referenced by Domain/Application.	Only sees compiled references; reflection/dynamic loading needs separate policy.
Forbidden API scan	IServiceProvider, GetService, BuildServiceProvider, static service access outside approved composition adapters.	Must distinguish the hosted-service scope boundary from general service location.
Registration descriptor scan	Duplicate singular ports, invalid lifetimes, unapproved implementation assembly.	Factories need explicit metadata or controlled execution.
Host-source purity scan	Domain namespaces, DbContext use, broker calls, request-specific business branching in Host.	Reviewer still judges meaning; syntax alone is incomplete.

EVIDENCE MUST BIND TO THE BUILD

A dependency report from source commit A cannot accept binaries from commit B. Record the solution, SDK, restore graph, commit, configuration, and assembly hashes together.

Boot and shutdown are executable composition tests

A composition test starts the real host path with controlled external dependencies, proves the declared endpoint and worker inventory, observes readiness remaining truthful, signals shutdown, and records whether scoped/disposable components were released. It does not need to prove RootCause business outcomes yet, but it must fail on an invalid graph or leaked lifetime.

Evidence	Required assertion	Acceptance
CMP-BOOT-001	Provider builds once; declared roots resolve; process starts without hidden remote work.	Exit and structured boot report match the manifest.
CMP-HTTP-001	Expected route groups and authorization metadata exist; no collision or unowned endpoint.	Endpoint inventory reviewed against capability declaration.
CMP-WORKER-001	Exactly one declared worker starts and creates/disposes a scope per controlled batch	Lifecycle trace contains start, scope, batch and dispose events.

Evidence	Required assertion	Acceptance
CMP-STOP-001	Shutdown signal stops intake, propagates cancellation and completes within the test budget.	No new work starts after stopping; residual work is reported.
CMP-LIFE-001	Scoped and disposable probes have expected creation/disposal counts.	No captive or root-scoped business object remains.

RootCause composition status

Chapter 8 can now instantiate the design at specification level without claiming runtime success. The following status table connects Chapter 7's generated skeleton to the exact composition evidence still required for G1.

Item	Specified state	Evidence state
Program.cs composition sequence	Provider validation, defaults, Application, Infrastructure, Edge, build, map, run.	Pending compile and CMP-BOOT-001.
Application registrations	IExplainAlarmFlood and IAcceptRootCauseVerdict are scoped roots.	Pending implementation and CMP-DI-001.
Persistence and inbox ports	Application-owned ports bind to RootCause infrastructure adapters.	Pending RootCauseDb, adapter code and CMP-REF-001.
HTTP entry point	One authorized /api/root-causes group delegates to IExplainAlarmFlood.	Pending contract code and CMP-HTTP-001.
Message entry point	Singleton AlarmFloodInboxWorker creates a per-batch scope.	Pending broker/inbox code and CMP-WORKER-001.
Lifecycle	Build once, explicit scopes, cancellation, bounded stop and container disposal.	Pending CMP-LIFE-001 / CMP-STOP-001.
Current gate	G0 accepted; G1 template and composition specified.	TPL, CMP, BUILD and architecture evidence remain pending.

CURRENT VERDICT

Chapter 8 does not advance RootCause to G1. It defines the composition claim that the generated repository must compile, build, resolve, start, inspect, stop, and review.

False greens in host composition

False green	Why it passes	Why NEXUS-1 rejects it
The host starts	Lazy paths and unused registrations are never resolved.	Declared roots, endpoints, workers and shutdown remain unproved.
ValidateOnBuild passes	Open or runtime-selected paths and external effects may be absent.	Composition smoke and manifest evidence are still required.
Unit tests resolve handlers	Tests build a smaller or hand-written container.	The production registration path and module order are not exercised.
Program.cs is short	Complexity moved into opaque scanning or a god registration class.	The graph is hidden rather than constrained.
A worker processes one message	A root-scoped dependency survives or cancellation is ignored.	Lifetime and shutdown correctness remain failed.
Dependency diagram is correct	Project/assembly references are not measured.	An illustration cannot replace the reference report.

ADR-006 - One explicit composition root

ADR field	ADR-006 record
Context	RootCause needs HTTP and message entry points, transactional scoped adapters, technical defaults, and controlled lifecycle without outer dependencies leaking inward.
Decision	Use the Host project as the only process composition root. Use explicit project-owned registration modules, provider validation, one root provider, request/per-work scopes, and evidence manifests.
Consequences	The Host references outer projects and must be reviewed carefully; Application owns ports; Infrastructure owns adapters; workers remain thin; composition drift becomes testable.
Rejected	Assembly-wide business scanning, service locator, nested containers, static provider, environment-selected domain behaviour, host-owned use cases, singleton scoped-state capture.
Review trigger	New entry point, adapter replacement, lifetime change, container change, middleware reorder, environment graph change, or new hosted service.

Risk register for composition

Risk	Early signal	Blocking control
Captive scoped dependency	Singleton constructor accepts repository, DbContext, use case, or scoped adapter.	ValidateScopes plus descriptor and disposal probes.
Service locator spreads	IServiceProvider or GetService appears outside Host/approved worker scope boundary.	Forbidden API test and review.
Duplicate singular port	Module order changes resolved implementation.	Exact-one descriptor validator.
Host owns business branching	Program, middleware, or endpoint chooses RootCause policy.	Host purity scan and application-path review.
Worker leaks work scope	Disposable counts grow or state crosses batches.	Per-batch scope trace and forced-stop test.
Environment graphs diverge	Production lacks a registration present in development or swaps business behaviour.	Normalized per-environment manifest and approved diff.
Startup performs remote mutation	Constructor/factory connects, migrates, publishes, or blocks.	Side-effect-free registration rule; explicit startup obligation later.
Shutdown loses ownership	New work begins after stopping or cancellation is swallowed.	CMP-STOP-001 with residual-work report.

Official-source validation

The reference design was checked against current Microsoft documentation on 18 July 2026. The stable concepts used here are constructor injection and container-owned disposal; avoiding service locator and BuildServiceProvider during registration; enabling scope validation; treating AddHostedService registrations as singleton hosted services that create explicit scopes for scoped work; using grouped routes for shared endpoint metadata; and honoring the Generic Host shutdown lifecycle.

Official source	Stable point used here	NEXUS-1 boundary
Microsoft Learn - Dependency injection guidelines	Avoid static service access, service locator and BuildServiceProvider in registration; enable scope validation.	NEXUS-1 adds ownership, uniqueness, graph manifests and evidence acceptance.
Microsoft Learn - Dependency injection in ASP.NET Core	Container creates/disposes services; Add... methods group registrations; request scope and	Project direction and business ownership

Official source	Stable point used here	NEXUS-1 boundary
Microsoft Learn - Scoped services in BackgroundService	AddHostedService registers a singleton; create an async scope for scoped work.	Per-batch meaning, idempotency and recovery remain RootCause-owned and pending.
Microsoft Learn - .NET Generic Host	Stopping signals propagate and graceful shutdown is bounded/configurable.	A clean stop is evidence; crash safety still requires durable workflow design.
Microsoft Learn - Minimal API route handlers	MapGroup organizes common prefixes and attaches shared metadata.	Route grouping does not own use-case or contract meaning.

The series baseline remains net8.0 as established in Chapter 4. Current documentation may display a newer product version; it does not silently upgrade the book. Exact APIs, SDK patch, packages, generated binaries, provider behaviour and lifecycle results must be captured by the pinned executable repository.

Technical material still required

Host composition is now specified but not executable in this manuscript environment. The RootCause projects, registration modules, ports, adapters, endpoint group, worker, provider options, architecture tests, composition fixture, manifest extractor, boot/shutdown probes, environment diff and evidence publisher must be implemented under the pinned toolchain.

TECHNICAL MATERIAL STATUS - [COMPOSITION TEST HARNESS + BUILD/RUNTIME EVIDENCE REQUIRED]

Compile the exact Program.cs and registration modules; validate project and assembly references; build the production provider with scope/build validation; fail duplicate singular ports; resolve every declared root; enumerate endpoints and hosted services; run request and per-batch scope probes; start and stop the real host path; compare environment manifests; and publish CMP-REF-001 through CMP-REVIEW-001. Until then, G1 remains specified but not evidenced.

Deliverable	Minimum technical content	Blocking evidence
Composition source	Program.cs, project-owned registration modules, provider options, endpoint mapping and worker registration.	CMP-REF-001 / CMP-DI-001
Architecture harness	Project graph, assembly references, forbidden API scan, Host purity and descriptor uniqueness.	CMP-REF-001 / CMP-LIFE-001
Runtime fixture	Production registration path, controlled external adapters, root resolution, endpoint/worker inventory.	CMP-BOOT-001 / CMP-HTTP-001 / CMP-WORKER-001
Lifecycle probes	Creation/disposal counters, request scope, per-batch scope, cancellation, no-new-work-after-stop assertion.	CMP-LIFE-001 / CMP-STOP-001
Composition manifest	Normalized roots, bindings, lifetimes, entry points, middleware order and environment diff.	CMP-ENV-001
Review record	Pinned SDK/restore/build identity, evidence links, exceptions, reviewer and current gate.	CMP-REVIEW-001 / TPL-BUILD-001

CHAPTER 8 - CODE AUDIT

One production composition path becomes testable

The host exposes one reusable builder path. Program.cs, tests, and evidence extraction all use that path, so a smaller test-only container cannot produce a false green.

EVIDENCE STATUS

These listings are compile-oriented reference assets. No dotnet executable is available here; provider build, endpoint inventory, worker lifecycle, and shutdown evidence remain PENDING.

EXECUTABLE ASSET 8-A - HOST COMPOSITION MODULE

```
public static class RootCauseHost
{
    public static WebApplicationBuilder CreateBuilder(
        string[] args,
        Action<RootCauseHostOptions>? configure = null)
    {
        var builder = WebApplication.CreateBuilder(args);
        builder.Host.UseDefaultServiceProvider( (_, options) =>
        {
            options.ValidateScopes = true;
            options.ValidateOnBuild = true;
        });

        builder.AddNexusServiceDefaults(o =>
        {
            o.ServiceName = "root-cause";
            o.ConfigurationSection = "RootCause";
        });

        builder.Services.AddRootCauseApplication();
        builder.Services.AddRootCauseInfrastructure(
            builder.Configuration);
        builder.Services.AddRootCauseEdge();
        configure?.Invoke(
            new RootCauseHostOptions(builder.Services));
        return builder;
    }

    public static WebApplication Build(WebApplicationBuilder builder)
    {
        var app = builder.Build();
        app.MapNexusServiceEndpoints();
        app.MapRootCauseEndpoints();
        return app;
    }
}
```

EXECUTABLE ASSET 8-B - DELIBERATELY BORING PROGRAM.CS

```
var builder = RootCauseHost.CreateBuilder(args);
await using var app = RootCauseHost.Build(builder);
await app.RunAsync();

public partial class Program;
```

The public partial Program enables integration bootstrapping. It does not introduce a second composition model; the same CreateBuilder and Build methods remain authoritative.

OWNED REGISTRATIONS

Application declares roots; Infrastructure binds ports

EXECUTABLE ASSET 8-C - APPLICATION MODULE

```
public static class RootCauseApplicationRegistration
{
    public static IServiceCollection AddRootCauseApplication(
        this IServiceCollection services)
    {
        services.TryAddScoped<
            IExplainAlarmFlood, ExplainAlarmFloodHandler>();
        services.TryAddScoped<
            IAcceptRootCauseVerdict, AcceptRootCauseVerdictHandler>();
        services.TryAddScoped<
            IAlarmFloodInboxPump, AlarmFloodInboxPump>();
        return services;
    }
}
```

EXECUTABLE ASSET 8-D - INFRASTRUCTURE MODULE

```
public static class RootCauseInfrastructureRegistration
{
    public static IServiceCollection AddRootCauseInfrastructure(
        this IServiceCollection services,
        IConfiguration configuration)
    {
        services.AddOptions<RootCauseStoreOptions>()
            .Bind(configuration.GetRequiredSection("RootCause:Store"))
            .ValidateDataAnnotations()
            .ValidateOnStart();

        services.TryAddScoped<
            IRootCauseCaseStore, SqlRootCauseCaseStore>();
        services.TryAddScoped<
            IAlarmFloodInbox, SqlAlarmFloodInbox>();
        services.AddHostedService<AlarmFloodInboxWorker>();
        return services;
    }
}
```

TryAdd prevents accidental replacement but does not prove exact uniqueness. The descriptor gate below rejects zero or multiple bindings for every singular application port.

EXECUTABLE ASSET 8-E - CONTROLLED ADAPTER OVERRIDE

```
public sealed class RootCauseHostOptions(IServiceCollection services)
{
    public void UseControlledAdapters(
        IRootCauseCaseStore store,
        IAlarmFloodInbox inbox)
    {
        services.Replace(
            ServiceDescriptor.Scoped<IRootCauseCaseStore>(
                _ => store));
        services.Replace(
            ServiceDescriptor.Scoped<IAlarmFloodInbox>(
                _ => inbox));
    }
}
```

TEST SUBSTITUTION BOUNDARY

Controlled adapters may replace external effects. They may not bypass Application registrations, remove authorization metadata, suppress the worker, or create a separate provider.

DESCRIPTOR VALIDATION

Singular ports and lifetime rules fail before Build

EXECUTABLE ASSET 8-F - EXACT-ONE DESCRIPTOR GATE

```
public static class CompositionDescriptorValidator
{
    private static readonly Type[] SingularPorts =
    [
        typeof(IExplainAlarmFlood),
        typeof(IAcceptRootCauseVerdict),
        typeof(IAAlarmFloodInboxPump),
        typeof(IRootCauseCaseStore),
        typeof(IAAlarmFloodInbox)
    ];

    public static void Validate(IServiceCollection services)
    {
        foreach (var port in SingularPorts)
        {
            var descriptors = services
                .Where(d => d.ServiceType == port)
                .ToArray();
            if (descriptors.Length != 1)
                throw new CompositionViolation(
                    $"{port.Name}: expected 1, found {descriptors.Length}");
        }

        RejectScopedImplementationForSingleton(services);
        RejectImplementationFactoryWithUnknownIdentity(services);
    }
}
```

TEST ASSET - DUPLICATE BINDING IS BLOCKING

```
[Fact]
public void CMP_DI_001_duplicate_singular_port_is_rejected()
{
    var builder = RootCauseHost.CreateBuilder([]);
    builder.Services.AddScoped<
        IRootCauseCaseStore, AnotherRootCauseCaseStore>();

    var act = () =>
        CompositionDescriptorValidator.Validate(builder.Services);

    act.Should().Throw<CompositionViolation>()
        .WithMessage("*/IRootCauseCaseStore*found 2*");
}
```

TEST ASSET - PROVIDER VALIDATES SCOPES AND ROOTS

```
[Fact]
public async Task CMP_LIFE_001_provider_builds_and_roots_resolve()
{
    var builder = RootCauseHost.CreateBuilder([]);
    CompositionDescriptorValidator.Validate(builder.Services);

    await using var app = RootCauseHost.Build(builder);
    await using var scope = app.Services.CreateAsyncScope();

    scope.ServiceProvider
        .GetRequiredService<IExplainAlarmFlood>();
    scope.ServiceProvider
        .GetRequiredService<IAcceptRootCauseVerdict>();
    scope.ServiceProvider
        .GetRequiredService<IAAlarmFloodInboxPump>();
}
```

ARCHITECTURE ENFORCEMENT

Compile direction and forbidden service location are separate gates

EXECUTABLE ASSET 8-G - COMPILED REFERENCE RULES

```
[Fact]
public void CMP_REF_001_inner_assemblies_do_not_reference_outer_layers()
{
    AssertNoReference(
        typeof(RootCauseCase).Assembly,
        "Nexus1.RootCause.Application",
        "Nexus1.RootCause.Edge",
        "Nexus1.RootCause.Infrastructure",
        "Nexus1.RootCause.Host");

    AssertNoReference(
        typeof(IExpainAlarmFlood).Assembly,
        "Nexus1.RootCause.Edge",
        "Nexus1.RootCause.Infrastructure",
        "Nexus1.RootCause.Host");

    AssertNoReference(
        typeof(SqlRootCauseCaseStore).Assembly,
        "Nexus1.RootCause.Host");
}

private static void AssertNoReference(
    Assembly assembly, params string[] forbidden)
{
    var names = assembly.GetReferencedAssemblies()
        .Select(x => x.Name)
        .Where(x => x is not null);
    names.Should().NotIntersectWith(forbidden);
}
```

EXECUTABLE ASSET 8-H - FORBIDDEN API POLICY

```
private static readonly ForbiddenApiRule[] Rules =
[
    Forbid("System.IServiceProvider", outside:
        "Nexus1.RootCause.Host"),
    Forbid("Microsoft.Extensions.DependencyInjection." +
        "ServiceProviderServiceExtensions.GetService", outside:
        "Nexus1.RootCause.Infrastructure.Workers"),
    Forbid("Microsoft.Extensions.DependencyInjection." +
        "ServiceCollectionContainerBuilderExtensions." +
        "BuildServiceProvider", outside: "<none>")
];

[Fact]
public void service_locator_and_nested_provider_are_absent()
{
    ForbiddenApiScanner.Scan(SolutionAssemblies, Rules)
        .Should().BeEmpty();
}
```

WHY BOTH TESTS EXIST

Assembly references prove compiled direction. Forbidden-API inspection proves that allowed outer assemblies do not spread runtime service location or nested providers.

HTTP INVENTORY

Routes and authorization metadata become inspectable evidence

EXECUTABLE ASSET 8-I - WEBAPPLICATIONFACTORY USING PRODUCTION PATH

```
public sealed class RootCauseFactory
    : WebApplicationFactory<Program>
{
    protected override void ConfigureWebHost(
        IWebHostBuilder builder)
    {
        builder.UseEnvironment("Chapter8Test");
        builder.ConfigureServices(services =>
        {
            services.Replace(ServiceDescriptor.Scoped<
                IRootCauseCaseStore, ControlledCaseStore>());
            services.Replace(ServiceDescriptor.Scoped<
                IAlarmFloodInbox, ControlledInbox>());
        });
    }
}
```

EXECUTABLE ASSET 8-J - ENDPOINT METADATA ASSERTION

```
[Fact]
public void CMP_HTTP_001_RootCause_routes_are_owned_and_authorized()
{
    using var factory = new RootCauseFactory();
    _ = factory.CreateClient(); // forces server start

    var endpoints = factory.Services
        .GetRequiredService<EndpointDataSource>()
        .Endpoints
        .OfType<RouteEndpoint>()
        .Where(e => e.RoutePattern.RawText!
            .StartsWith("/api/root-causes"))
        .ToArray();

    endpoints.Should().ContainSingle(e =>
        e.RoutePattern.RawText ==
            "/api/root-causes/explanations");

    endpoints.Should().OnlyContain(e =>
        e.Metadata.GetMetadata<IAuthorizeData>()?.Policy ==
            "rootcause.operator");
}
```

BLOCKING COLLISION CHECK

```
var collisions = endpoints
    .GroupBy(e => (
        Pattern: e.RoutePattern.RawText,
        Methods: string.Join(",",
            e.Metadata.GetMetadata<HttpMethodMetadata>()?
                .HttpMethods.OrderBy(x => x) ?? [])))
    .Where(group => group.Count() > 1)
    .ToArray();

collisions.Should().BeEmpty();
```

Endpoint inventory proves route and metadata ownership. It does not prove contract compatibility or resource authorization decisions; those remain later security and contract evidence.

WORKER LIFECYCLE

A singleton worker creates one disposable scope per bounded batch

EXECUTABLE ASSET 8-K - THIN WORKER WITH BOUNDED DELAY

```
internal sealed class AlarmFloodInboxWorker(
    IServiceScopeFactory scopeFactory,
    TimeProvider time,
    IWorkerLifecycleProbe probe)
    : BackgroundService
{
    protected override async Task ExecuteAsync(
        CancellationToken stoppingToken)
    {
        await probe.StartedAsync(stoppingToken);
        while (!stoppingToken.IsCancellationRequested)
        {
            await using var scope = scopeFactory.CreateAsyncScope();
            await probe.ScopeCreatedAsync(stoppingToken);
            try
            {
                var pump = scope.ServiceProvider
                    .GetRequiredService<IAlarmFloodInboxPump>();
                var result = await pump.RunOneBatchAsync(stoppingToken);
                if (result is BatchResult.Empty)
                    await Task.Delay(
                        TimeSpan.FromMilliseconds(200),
                        time,
                        stoppingToken);
            }
            finally
            {
                await probe.ScopeDisposingAsync(CancellationToken.None);
            }
        }
    }
}
```

EXECUTABLE ASSET 8-L - PER-BATCH SCOPE TEST

```
[Fact]
public async Task CMP_WORKER_001_each_batch_has_a_new_scope()
{
    var probe = new RecordingLifecycleProbe();
    var pump = new ControlledPump(
        results: [Handled, Handled, Empty]);
    await using var host = Fixture.Host(probe, pump);

    await host.StartAsync();
    await probe.WaitForBatchesAsync(3, TestTimeout);
    using var stopTimeout =
        new CancellationTokenSource(TestTimeout);
    await host.StopAsync(stopTimeout.Token);

    probe.ScopeCreated.Should().Be(3);
    probe.ScopeDisposed.Should().Be(3);
    probe.BatchScopeIds.Should().OnlyHaveUniqueItems();
}
```

MEANING REMAINS DOWNSTREAM

The worker owns loop, cancellation, scope, and idle delay. The scoped pump owns message claim, idempotency, transaction, and RootCause application invocation.

SHUTDOWN

Stop intake, propagate cancellation, and report residual work

EXECUTABLE ASSET 8-M - SHUTDOWN PROBE TEST

```
[Fact]
public async Task CMP_STOP_001_no_batch_starts_after_stop()
{
    var probe = new RecordingLifecycleProbe();
    var pump = new BlockingControlledPump();
    await using var host = Fixture.Host(probe, pump);

    await host.StartAsync();
    await pump.FirstBatchStarted.WaitAsync(TestTimeout);

    using var stopTimeout =
        new CancellationTokencSource(TimeSpan.FromSeconds(5));
    var stop = host.StopAsync(stopTimeout.Token);
    await pump.CancellationObserved.WaitAsync(TestTimeout);
    await stop;

    var countAtStop = probe.BatchStarted;
    await Task.Delay(TimeSpan.FromMilliseconds(100));

    probe.BatchStarted.Should().Be(countAtStop);
    probe.ScopeCreated.Should().Be(probe.ScopeDisposed);
    probe.ResidualWork.Should().BeEmpty();
}
```

EXECUTABLE ASSET 8-N - LIFECYCLE EVIDENCE RECORD

```
{
    "evidenceId": "CMP-STOP-001",
    "service": "root-cause",
    "commit": "PENDING",
    "environment": "Chapter8Test",
    "startedAtUtc": null,
    "stopRequestedAtUtc": null,
    "cancellationObservedAtUtc": null,
    "completedAtUtc": null,
    "batchesStartedAfterStop": null,
    "scopesCreated": null,
    "scopesDisposed": null,
    "residualWork": [],
    "accepted": false
}
```

The manuscript intentionally leaves measurements null. A test listing is E1 implementation shape; only retained execution output tied to a build can advance the claim.

COMPOSITION EVIDENCE

Normalized manifests expose environment drift

EXECUTABLE ASSET 8-O - MANIFEST EXTRACTION

```
public CompositionManifest Extract(
    IServiceCollection descriptors,
    EndpointDataSource endpoints,
    string environment)
{
    return new(
        Service: "root-cause",
        Environment: environment,
        Registrations: descriptors
            .Where(IsReviewedRegistration)
            .Select(d => new Registration(
                Service: TypeName(d.ServiceType),
                Implementation: ImplementationIdentity(d),
                Lifetime: d.Lifetime.ToString()))
            .OrderBy(x => x.Service)
            .ThenBy(x => x.Implementation)
            .ToArray(),
        Endpoints: endpoints.Endpoints
            .OfType<RouteEndpoint>()
            .Select(ToEndpointRecord)
            .OrderBy(x => x.Pattern)
            .ThenBy(x => x.Methods)
            .ToArray(),
        Workers: descriptors
            .Where(d => d.ServiceType == typeof(IHostedService))
            .Select(ImplementationIdentity)
            .OrderBy(x => x)
            .ToArray());
}
```

EXECUTABLE ASSET 8-P - APPROVED ENVIRONMENT DIFF

```
[Fact]
public void CMP_ENV_001_production_and_test_differ_only_in_adapters()
{
    var production = Extract("Production");
    var controlled = Extract("Chapter8Test");
    var diff = CompositionDiff.Between(production, controlled);

    diff.Should().OnlyContain(change =>
        change.Path is
            "Registrations[IRootCauseCaseStore].Implementation" or
            "Registrations[IAAlarmFloodInbox].Implementation");

    diff.Should().NotContain(change =>
        change.Path.Contains("IExplainAlarmFlood") ||
        change.Path.Contains("Endpoints") ||
        change.Path.Contains("Workers"));
}
```

CI ASSET - COMPOSITION EVIDENCE

```
- name: Verify RootCause composition
  run: >
    dotnet test tests/Nexus1.RootCause.CompositionTests
    -c Release --no-restore
    --logger "trx;LogFileName=composition.trx"

- name: Retain composition evidence
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: root-cause-composition-${{ github.sha }}
    path: |
      **/composition.trx
    artifacts/composition/**
    retention-days: 90
```

Official-source validation

Checked 29 July 2026. Microsoft documents WebApplicationFactory for the SUT entry point, public partial Program for integration access, AddHostedService as singleton registration with explicit scopes for scoped work, and MapGroup metadata for grouped route authorization.

Sources: <https://learn.microsoft.com/aspnet/core/test/integration-tests> ; <https://learn.microsoft.com/dotnet/core/extensions/scoped-service> ; <https://learn.microsoft.com/aspnet/core/fundamentals/minimal-apis/route-handlers>

REVISION RESULT

Chapter 8 now supplies one production builder path, owned registrations, descriptor and architecture gates, endpoint inventory, per-batch lifecycle and shutdown probes, environment manifest diff, and CI evidence retention. CMP/BUILD evidence remains explicitly pending.

Implementation sequence

1. Commit ADR-006, the allowed project-reference graph, lifetime policy, singular-port rule and composition evidence catalogue.
2. Implement Application-owned ports and use-case registration without referencing Host, Edge or Infrastructure.
3. Implement Infrastructure adapters and one explicit registration module with no I/O, scanning or nested provider.
4. Implement Edge endpoint groups that map transport and authorization metadata to application ports without domain decisions.
5. Implement Program.cs as the single composition root and enable provider scope/build validation explicitly.
6. Implement the AlarmFloodInboxWorker as a singleton adapter that creates and disposes one scope per bounded batch.
7. Add project, assembly, forbidden-API, descriptor-uniqueness and Host-purity architecture tests.
8. Build the real production registration path with controlled external adapters and resolve every declared root.
9. Start the host, inventory endpoints/workers, exercise request and worker scopes, signal shutdown and record disposal/cancellation evidence.
10. Normalize per-environment composition manifests, review every difference, publish CMP evidence and accept G1 only if every blocker passes.

What this chapter has proved

Chapter 8 has defined one explicit Host composition root, compile-time dependency direction, project-owned registration modules, application-owned ports, lifetime rules, provider validation, separate HTTP and background scope ownership, thin endpoint and worker adapters, a normalized composition manifest, architecture tests, runtime smoke evidence, ADR-006, and a composition risk register.

It has not compiled or run RootCause. The Program.cs, registrations, provider, endpoint group, hosted worker, manifests, lifecycle probes and evidence commands remain reference artifacts until the pinned repository executes them. RootCause remains G0 accepted; its G1 template and composition are specified; TPL/CMP/BUILD evidence remains pending.

CHAPTER CLOSING

A composition root is successful when it knows enough about concrete mechanics to keep every business decision, lifetime assumption, and dependency leak visible - and nowhere else needs to know how the process was assembled.

Chapter checkpoint

You are ready for Chapter 9 when the runtime graph can be described without confusing compile-time direction, object lifetime, transport entry points, or business ownership.

Question	Expected answer
What is the composition root?	The Host boundary that selects adapters, lifetimes, entry points and process lifecycle, then builds one provider.
Why may Host reference Infrastructure?	It must select concrete adapters so Application and Domain do not reference them.
Who owns a persistence port?	The Application policy that needs the capability; Infrastructure implements it and Host binds it.
Why is a hosted worker singleton?	That is the framework registration lifetime; it creates an explicit scope for each bounded unit of scoped work.
Is ValidateOnBuild enough?	No. Root resolution, duplicate bindings, endpoints, workers, environment branches, lifecycle and shutdown require evidence.
What may an endpoint own?	Transport binding, authorization metadata and outcome mapping - not use-case or domain decisions.
What is the current RootCause gate?	G0 accepted; G1 template and composition specified; compile, build, runtime and review evidence pending.
What must Chapter 9 add?	Executable internal-layer boundaries and concrete RootCause behaviour that preserve the composition rules defined here.

NEXT CHAPTER

Chapter 9 defines Domain, Application, Infrastructure, and Edge Inside a Service. It places RootCause language, invariants, use cases, ports, adapters, transaction boundaries, and transport mapping in their owning layers, then tests that behaviour cannot drift back into the Host or shared packages.

PART II - THE SERVICE FOUNDATION

9. Domain, Application, Infrastructure, and Edge Inside a Service

A service boundary does not remove internal architecture. RootCause still needs a place where truth is decided, a place where use cases are sequenced, a place where technical effects are realized, and a place where external models are translated.

CHAPTER ARGUMENT

Keep four kinds of ownership distinct: Domain owns valid meaning, Application owns use-case intent, Infrastructure owns technical realization, and Edge owns transport translation. Host only composes them.

Chapter 8 proved the intended compile-time direction: Host may reference every outer project, Edge and Infrastructure depend inward through Application-owned ports, Application depends on Domain, and Domain depends on no service framework. Chapter 9 now fills that graph with RootCause behaviour while preserving the stable rules from Chapter 2.

Those inherited rules are not placeholders. RC-INV-001 says a closed RootCauseCase cannot accept new evidence, hypotheses, or ordinary state changes. RC-INV-002 says a case cannot close with a verdict unless evidence exists. RC-OWN-001 says only RootCause may issue or correct the verdict. Deterministic decisions exclude delivery timing, retry count, and explanatory text; abstention remains a valid result. These rules determine the code shape below.

The manuscript still cannot claim implementation. The generated repository, exact domain types, EF Core model, RootCauseDb, migrations, contract serializers, outbox tables, identities, and test harness do not exist here. Every code listing is a reference contract marked for compilation and execution under the pinned toolchain.

Four layers are four kinds of ownership

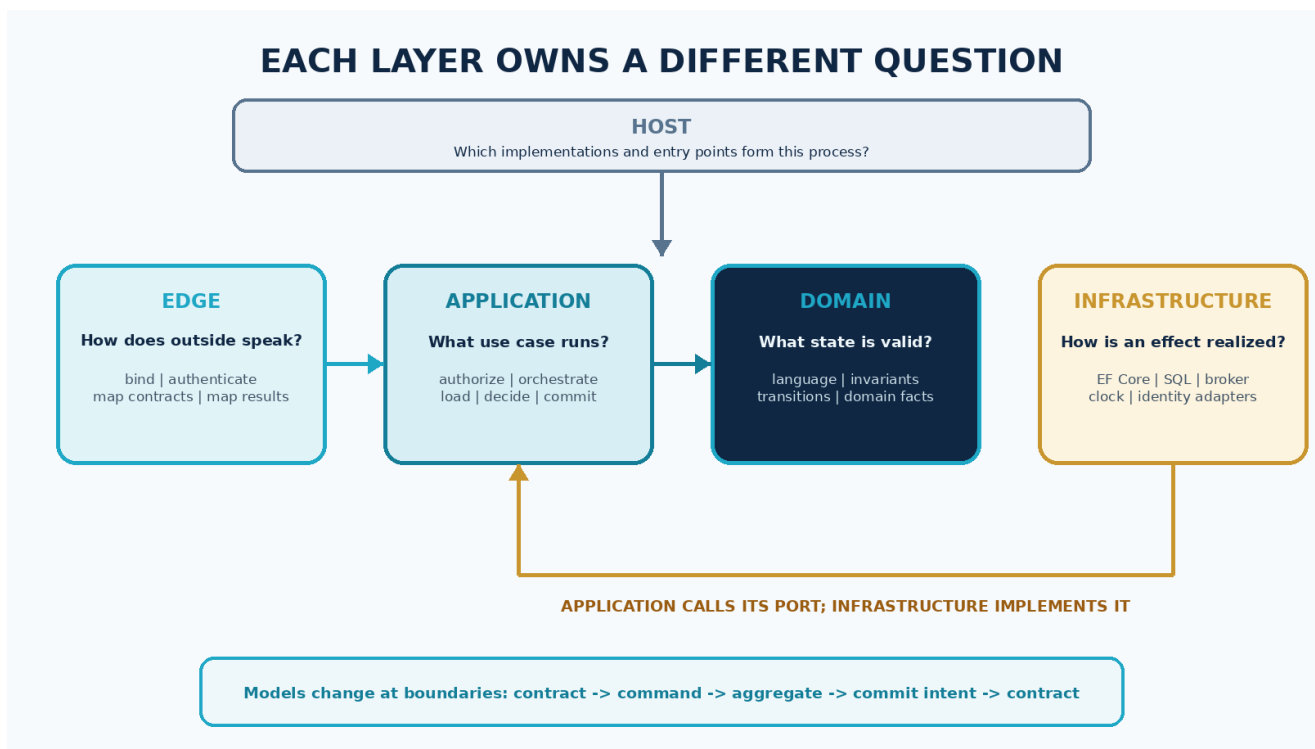


Figure 9.1 - Each layer ans

Layer	Owns	Must not own
Domain	RootCause language, aggregate state, invariants, deterministic decisions, internal domain facts.	HTTP, messages, serialization, EF Core, SQL, configuration, service provider, remote retries.
Application	Use cases, local authorization intent, orchestration, ports, transaction intent, explicit outcomes.	Transport status codes, DbContext, broker clients, JSON, database schema, another service's decisions.
Infrastructure	EF Core mappings, repositories, unit of work, outbox storage, clocks/identity/remote technical adapters.	RootCause invariant decisions, HTTP contracts, use-case branching, Host lifecycle.
Edge	HTTP/message binding, public-contract mapping, authentication context, outcome-to-transport mapping.	Aggregate mutation, DbContext access, transaction ownership, hidden authorization replacement.
Host	Registration, lifetimes, endpoint/worker attachment, configuration and process lifecycle.	Any business decision, persistence query, transport mapping, or domain model.

LAYERING TEST

A class belongs where its decision is owned, not where its method happens to be called. Moving code to another folder does not move authority.

Domain protects valid RootCause state

The Domain project contains ordinary C# types with no ASP.NET Core, EF Core, serializer, message-envelope, options, logging, or dependency-injection references. It may expose behaviour and read-only state. It does not expose settable persistence properties or public-contract DTOs. Frameworks adapt to the model; the model does not become a framework extension point.

ROOTCAUSE AGGREGATE - INHERITED INVARIANT SHAPE [NEW][COMPILE REQUIRED]

```
public sealed class RootCauseCase
{
    private readonly List<EvidenceItem> _evidence = [];
    private readonly List<RootCauseHypothesis> _hypotheses = [];

    public RootCauseCaseId Id { get; }
    public RootCauseCaseStatus Status { get; private set; }
    public RootCauseVerdict? FinalVerdict { get; private set; }

    public void AddEvidence(EvidenceItem evidence)
    {
        EnsureOpen();
        ArgumentNullException.ThrowIfNull(evidence);
        _evidence.Add(evidence);
    }

    public void AddHypothesis(RootCauseHypothesis hypothesis)
    {
        EnsureOpen();
        ArgumentNullException.ThrowIfNull(hypothesis);
        _hypotheses.Add(hypothesis);
    }

    public RootCauseVerdictIssued CloseWithVerdict(
        RootCauseVerdict verdict,
        DateTimeOffset occurredAt)
    {
        EnsureOpen();
        if (_evidence.Count == 0)
            throw DomainRuleViolation.Create("RC-INV-002");

        FinalVerdict = verdict;
        Status = RootCauseCaseStatus.Closed;
        return new RootCauseVerdictIssued(Id, verdict, occurredAt);
    }

    private void EnsureOpen()
    {
        if (Status == RootCauseCaseStatus.Closed)
            throw DomainRuleViolation.Create("RC-INV-001");
    }
}
```

WHAT IS INTENTIONALLY ABSENT

No correction method is invented here. RC-OWN-001 requires RootCause to own correction, but the exact correction authorization, reason, history and public fact need a separate explicit design before code is added.

Domain facts are internal facts

RootCauseVerdictIssued above is an internal domain fact. It is not RootCauseVerdictIssued.v1 and it is never serialized directly. The Domain type can evolve with the aggregate. The producer-owned public contract is mapped at an outbound edge and follows its own versioning and compatibility rules.

Internal model	Public model	Boundary rule
RootCauseVerdictIssued	RootCauseVerdictIssuedV1	Explicit semantic mapping; no reflection serializer over the Domain type.
RootCauseCaseId	caseId string/UUID field	Parse/format at Edge; Domain keeps a validated typed identity.
RootCauseVerdict	versioned verdict payload	Only fields promised by the contract leave the service.
DomainRuleViolation	problem response or message handling result	Internal exception/type names never become public error contracts accidentally.
RootCauseCase	HTTP response or projection contract	Aggregate is never returned or serialized across the service boundary.

Abstention is a domain result, not a transport failure

The preservation ledger explicitly allows insufficient evidence. A remote workflow must not force a verdict merely because it is waiting. Model the decision as issue-or-abstain, and keep explanatory text outside the deterministic state transition. Transport may later represent abstention as an accepted incomplete result or a public fact, but it cannot translate it into an infrastructure exception.

DETERMINISTIC DECISION BOUNDARY - [NEW][COMPILE REQUIRED]

```
public abstract record VerdictDecision
{
    private VerdictDecision() { }

    public sealed record Issue(RootCauseVerdict Verdict)
        : VerdictDecision;

    public sealed record Abstain(AbstentionReason Reason)
        : VerdictDecision;
}

public sealed record VerdictDecisionInput(
    IReadOnlyList<EvidenceSnapshot> OrderedEvidence,
    DecisionEngineVersion EngineVersion,
    ConfigurationIdentity Configuration);

// Arrival time, retry count, broker attempt and LLM text are not inputs.
```

Decision input	Permitted	Reason
Ordered evidence snapshot	Yes	The causal evidence being evaluated is explicit and replayable.
Engine/model version	Yes	A changed result can be tied to a changed deterministic implementation.
Configuration identity	Yes	Decision-affecting configuration is named and reproducible.
Message arrival time or broker attempt	No	Transport timing and retry cannot silently change truth.
Generated explanation	No	Explanation is read-only and cannot mutate deterministic case state.
Remote subscriber status	No	Audit, Compliance, and Reporting availability cannot choose the RootCause verdict.

REPLAY PROPERTY

The same ordered evidence, engine version and configuration identity must produce the same issue-or-abstain result. If it does not, the decision is not yet evidenced as deterministic.

Aggregate tests prove inherited rules directly

BLOCKING AGGREGATE TESTS - [NEW][COMPILE REQUIRED]

```
[Fact]
public void Closed_case_rejects_new_evidence()
{
    var sut = RootCauseCaseFixture.OpenWithEvidence();
    sut.CloseWithVerdict(Verdicts.Known, Time.UtcNow);

    var error = Assert.Throws<DomainRuleViolation>(
        () => sut.AddEvidence(Evidence.New()));

    Assert.Equal("RC-INV-001", error.RuleId);
}

[Fact]
public void Case_without_evidence_cannot_close()
{
    var sut = RootCauseCaseFixture.OpenWithoutEvidence();

    var error = Assert.Throws<DomainRuleViolation>(
        () => sut.CloseWithVerdict(Verdicts.Known, Time.UtcNow));

    Assert.Equal("RC-INV-002", error.RuleId);
}
```

These tests do not need a container, database, clock, broker, serializer, web host, or environment. If an aggregate test requires those dependencies, the protected rule has leaked out of Domain or the fixture is exercising the wrong layer.

Application owns the use case

Application translates an authorized intention into one coherent local operation. It identifies the use case, requests local authorization, loads the aggregate through a port, invokes Domain behaviour, stages the producer-owned public intent through a port, commits once, and returns an application outcome. It does not know HTTP status codes, DbContext, JSON, SQL, queues, or deployment environment.

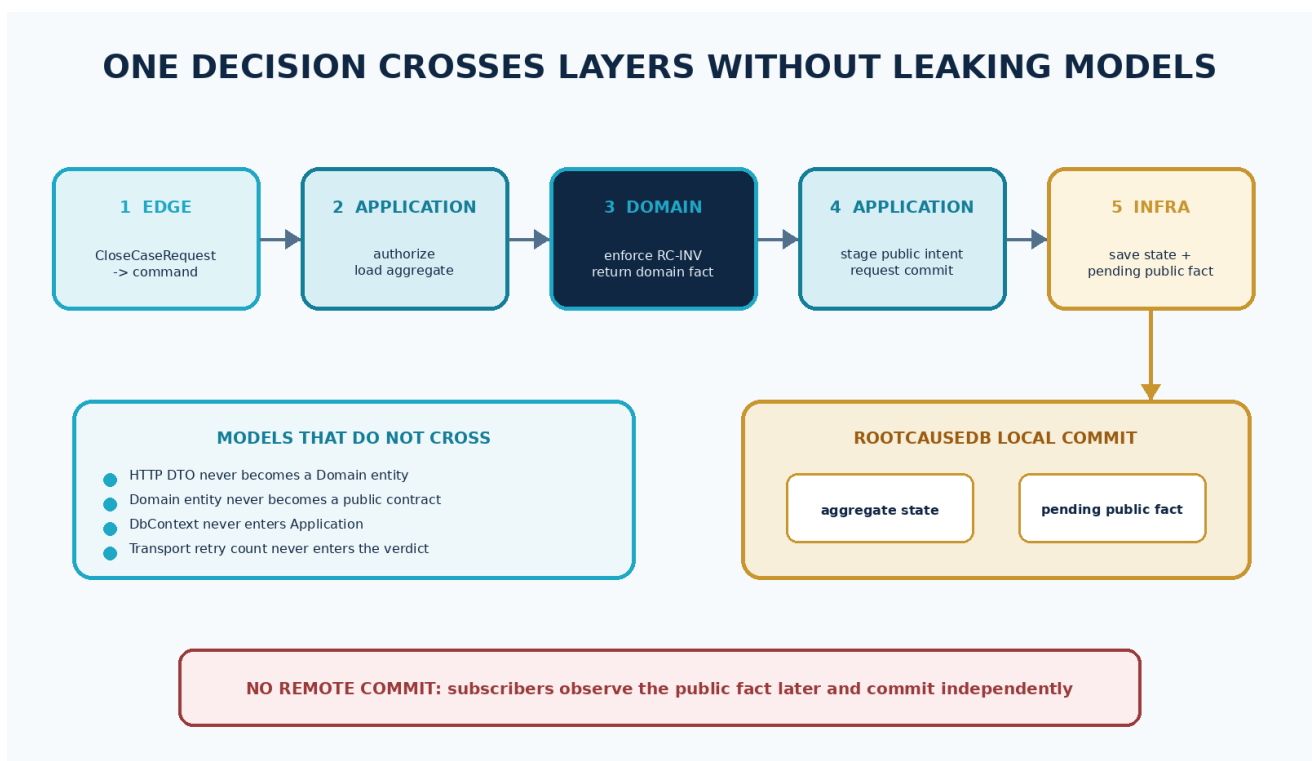


Figure 9.2 - A close-case decision changes models at each boundary and commits only RootCause-owned state.

APPLICATION USE CASE - REFERENCE [NEW][COMPILE REQUIRED]

```
public sealed class CloseRootCauseCaseHandler(
    IRootCauseCaseStore cases,
    IRootCauseAccessPolicy access,
    IRootCausePublicFactWriter publicFacts,
    IRootCauseUnitOfWork unitOfWork,
    TimeProvider timeProvider)
    : ICloseRootCauseCase
{
    public async Task<CloseCaseOutcome> ExecuteAsync(
        CloseRootCauseCaseCommand command,
        ExecutionActor actor,
        CausalityContext causality,
        CancellationToken cancellationToken)
    {
        if (!await access.MayIssueVerdictAsync(
            actor, command.CaseId, cancellationToken))
            return CloseCaseOutcome.Denied();

        var rootCauseCase = await cases.FindAsync(
            command.CaseId, cancellationToken);
        if (rootCauseCase is null)
            return CloseCaseOutcome.NotFound();

        try
        {
            var fact = rootCauseCase.CloseWithVerdict(
                command.Verdict,
                timeProvider.GetUtcNow());

            await publicFacts.StageAsync(
                VerdictIssuedFact.From(fact, actor, causality),
                cancellationToken);
            await unitOfWork.CommitAsync(cancellationToken);

            return CloseCaseOutcome.Closed(rootCauseCase.Id);
        }
        catch (DomainRuleViolation error)
        {
            return CloseCaseOutcome.Conflict(error.RuleId);
        }
    }
}
```

ONE LOCAL DECISION

The handler commits RootCause state and a pending public fact locally. It does not wait for Audit, Compliance, Reporting, or a broker acknowledgement before deciding that the local transaction succeeded.

Application ports express needs, not technologies

A port is named for the capability the use case needs. It should reveal transaction and ownership semantics without exposing provider APIs. Repository, access policy, public-fact writer, unit of work, and time are separate because their evidence and substitution boundaries differ.

APPLICATION-OWNED PORTS - [NEW][COMPILE REQUIRED]

```

public interface IRootCauseCaseStore
{
    Task<RootCauseCase?> FindAsync(
        RootCauseCaseId id,
        CancellationToken cancellationToken);
}

public interface IRootCauseAccessPolicy
{
    Task<bool> MayIssueVerdictAsync(
        ExecutionActor actor,
        RootCauseCaseId id,
        CancellationToken cancellationToken);
}

public interface IRootCausePublicFactWriter
{
    Task StageAsync(
        VerdictIssuedFact fact,
        CancellationToken cancellationToken);
}

public interface IRootCauseUnitOfWork
{
    Task CommitAsync(CancellationToken cancellationToken);
}

```

Port	What Application assumes	What remains adapter-owned
IRootCauseCaseStore	Loads one RootCause aggregate by typed identity in the current work scope.	EF queries, tracking, schema, indexes, connection and provider failures.
IRootCauseAccessPolicy	Evaluates local RootCause capability for an explicit actor and resource.	Identity token parsing, claims transport, policy-store integration and audit sink.
IRootCausePublicFactWriter	Stages the producer-owned meaning derived from an accepted domain fact.	V1 DTO mapping, serialization, envelope, outbox row and broker publication.
IRootCauseUnitOfWork	Atomically commits staged RootCause state/effects or reports failure.	DbContext, transaction API, savepoints, retries, concurrency and database errors.
TimeProvider	Provides an explicit occurred/decision time input where time is required.	System clock source and test clock control; time is not a hidden verdict input.

PORT OWNERSHIP

Application owns the interface because Application needs the capability. Infrastructure or Edge owns the implementation because it knows the technology or public contract.

Application outcomes are not HTTP results

Closed, NotFound, Denied, Conflict, and Abstained are use-case outcomes. Edge decides how an HTTP contract or message handler represents them. A use case returning IResult, status code, ProblemDetails, broker disposition, or serialized payload has moved transport ownership inward.

Application outcome	Possible HTTP mapping	Possible message handling
Closed	200/202 contract selected by API decision.	Commit inbox effect; publication remains local outbox work.
NotFound	404 only if the public contract exposes non-existence.	Classify according to producer/consumer contract; do not guess retry.
Denied	403 or concealment policy selected by Edge/security design.	Reject/quarantine with auditable service identity and policy result.
Conflict(ruleId)	Stable public problem code mapped from an internal rule catalogue.	Non-retryable or repair path only after explicit messaging policy.

Application outcome	Possible HTTP mapping	Possible message handling
Abstained	Accepted incomplete representation with visible reason semantics.	Publish or retain incomplete state as the versioned workflow defines.

Infrastructure realizes the local unit of work

Infrastructure may know EF Core, the relational provider, SQL schema, the outbox table, clock/identity integrations, and producer-owned contract packages. It implements Application ports. It may not decide whether evidence is sufficient, whether a case can close, who may issue a verdict, or what HTTP response should be returned.

EF CORE ADAPTER BOUNDARY - REFERENCE [NEW][COMPILE REQUIRED]

```
internal sealed class RootCauseDbContext(
    DbContextOptions<RootCauseDbContext> options)
    : DbContext(options)
{
    internal DbSet<RootCauseCase> Cases => Set<RootCauseCase>();
    internal DbSet<PendingPublicFactRow> PendingPublicFacts =>
        Set<PendingPublicFactRow>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.ApplyConfiguration(new RootCauseCaseConfiguration());
        modelBuilder.ApplyConfiguration(new PendingPublicFactConfiguration());
    }
}

internal sealed class EfRootCauseUnitOfWork(
    RootCauseDbContext db) : IRootCauseUnitOfWork
{
    public Task CommitAsync(Cancellation_token cancellation_token) =>
        db.SaveChangesAsync(cancellation_token);
}
```

The scoped DbContext represents one short unit of work and is never shared across threads or work scopes. Every asynchronous EF operation is awaited before the context is used again. The same tracked context must stage both the aggregate change and pending public-fact row before SaveChangesAsync if the local atomicity claim is to hold.

Infrastructure rule	Blocking failure	Evidence
DbContext remains internal and scoped	Application, Edge, Host business code, singleton, static field, or parallel operation receives it.	LAY-INF-001 reference/API scan and lifetime test.
Domain model has no provider attributes	EF/SQL annotations or generated persistence base class enter Domain.	LAY-DOM-001 package/reference scan.
One SaveChanges local commit	Aggregate succeeds without pending fact or pending fact exists without state change.	LAY-TRAN-001 relational transaction test.
No remote work in transaction	Broker/HTTP call is awaited before local commit or transaction spans another service.	Forbidden dependency and fault test.
Provider behaviour is tested	In-memory substitute passes while relational constraints/transactions fail.	Provider-backed component test and rollback assertion.

LOCAL ATOMICITY ONLY

A database transaction may atomically save RootCause state and its pending public fact. It cannot atomically commit Audit, Compliance, Reporting, or a broker without introducing a different and explicitly rejected boundary.

Direct mapping is a decision, not a default

EF Core can map private fields and controlled constructors, allowing a rich Domain aggregate to remain persistence-ignorant. If provider requirements force public setters, leaky base types, or infrastructure-shaped collections, introduce a persistence model and explicit mapper instead. The choice is reviewed against invariant protection, query needs, mapping cost, and migration risk; neither approach is automatically purer.

Mapping approach	Use when	Reject when
Map aggregate directly	EF configuration can preserve constructors, backing fields, typed IDs and invariant methods.	Provider pressure distorts Domain or enables invalid materialized state.
Separate persistence model	Schema/query shape differs materially or provider concerns cannot remain outside Domain.	Mapper duplicates rules, drops history, or creates a second source of truth.

Edge translates; it does not decide

The HTTP Edge binds public input, extracts authenticated actor and causality, maps to a command, invokes the use case, and maps the application outcome to a public response. It can reject malformed transport input before Application. It cannot implement RC-INV-001/002, infer RootCause authority from a route, or load the database directly.

HTTP EDGE ADAPTER - REFERENCE [NEW][COMPILE REQUIRED]

```
private static async Task<IResult> CloseAsync(
    string caseId,
    CloseRootCauseRequest request,
    ClaimsPrincipal principal,
    ICloseRootCauseCase useCase,
    CancellationToken cancellationToken)
{
    var command = RootCauseHttpMapping.ToCommand(caseId, request);
    var actor = RootCauseHttpMapping.ToActor(principal);
    var causality = RootCauseHttpMapping.ToCausality();

    var outcome = await useCase.ExecuteAsync(
        command, actor, causality, cancellationToken);

    return outcome switch
    {
        CloseCaseOutcome.Closed value =>
            TypedResults.Ok(RootCauseHttpMapping.ToResponse(value)),
        CloseCaseOutcome.NotFound _ => TypedResults.NotFound(),
        CloseCaseOutcome.Denied _ => TypedResults.Forbid(),
        CloseCaseOutcome.Conflict value =>
            TypedResults.Conflict(RootCauseHttpMapping.ToProblem(value)),
        _ => TypedResults.Problem(statusCode: 500)
    };
}
```

PUBLIC ERROR DISCIPLINE

The fallback above is a defensive transport result, not a license to hide an unmodelled application outcome. Every reachable outcome must receive a reviewed contract mapping and test.

The message edge removes envelope mechanics

AlarmFloodDetected.v1 arrives with message ID, correlation, causation, producer identity, occurred time, schema version, and payload. The message Edge validates the public contract and maps it to an Application command. Domain receives typed RootCause language, not broker headers or a retry count. Duplicate absorption, inbox state, disposition, and replay are specified in later messaging chapters and remain pending here.

INBOUND PUBLIC-CONTRACT MAPPING - [NEW][COMPILE REQUIRED]

```
internal static OpenOrAdvanceRootCauseCaseCommand ToCommand(
    AlarmFloodDetectedV1 message)
{
    return new OpenOrAdvanceRootCauseCaseCommand(
        AlarmFloodId.Parse(message.AlarmFloodId),
        AlarmReference.Parse(message.AlarmReference),
        EvidenceDescriptor.FromContract(message.Evidence));
}

// message.Attempt, delivery time and broker headers are not domain inputs.
// Correlation and causation are passed separately as execution context.
```

Edge mapping test	Required assertion	Forbidden shortcut
Valid close request	DTO maps to the exact typed command; actor and causality stay separate.	Pass ClaimsPrincipal or HttpContext into Application/Domain.
Malformed identity	Binding fails before use case and emits the stable public error contract.	Construct an invalid typed ID and rely on persistence failure.
Denied actor	Application Denied maps to reviewed public semantics without revealing private state.	Treat authenticated service identity as automatic authorization.
Domain conflict	Rule ID maps through an allow-list to a stable problem code.	Serialize exception type/message/stack trace.
AlarmFloodDetected.v1	Only promised contract fields enter the command; envelope identities remain execution context.	Deserialize directly into RootCauseCase or Domain event.
VerdictIssued mapping	Internal application fact maps explicitly to RootCauseVerdictIssuedV1.	Serialize Domain fact or aggregate by reflection.

The public-fact writer joins Edge and Infrastructure through ports

The outbound mapping problem must preserve both dependency direction and local atomicity. Application requests that a VerdictIssuedFact be staged. An Edge implementation maps that internal application fact to RootCauseVerdictIssuedV1. It calls an Application-owned IPendingPublicFactStore port implemented by Infrastructure, which adds the serialized row to the same scoped DbContext. Application then commits once through IRootCauseUnitOfWork.

Step	Owning layer	Model crossing
Aggregate returns internal fact	Domain	RootCauseCase -> RootCauseVerdictIssued; no public schema.
Handler adds actor/causality intent	Application	Domain fact -> VerdictIssuedFact application model.
Map to V1 and serialize	Edge	VerdictIssuedFact -> RootCauseVerdictIssuedV1 bytes.
Stage pending row	Infrastructure	Bytes/envelope metadata -> PendingPublicFactRow in RootCauseDb.
Commit once	Application intent / Infrastructure realization	Tracked aggregate and pending row commit or roll back together.
Publish later	Infrastructure messaging adapter	Outbox row -> broker; Part V owns delivery evidence.

NO CIRCULAR PROJECT REFERENCE

Edge and Infrastructure collaborate only through Application-owned ports. Neither project references the other, and Host selects both implementations.

Layer tests fail at the owning boundary

Layer suite	Primary proof	Must not require
Domain unit	RC-INV-001/002, deterministic replay, abstention, typed identity/value rules.	Container, database, HTTP, broker, serializer, environment or wall clock.
Application unit	Authorization sequence, port calls, explicit outcomes, one commit intent, no commit on failure.	EF Core, real network, public status codes or production configuration.
Infrastructure component	Mapping, relational constraints, local transaction rollback, scoped DbContext and pending fact row.	Another service database or successful broker delivery.
Edge contract/component	Binding, actor/causality extraction, V1 mapping, status/problem mapping and prohibited fields.	Direct database access or aggregate mutation.
Architecture	Project/package/API direction, forbidden framework types and no Domain/contract serialization leak.	Runtime success as a substitute for static evidence.

APPLICATION ORCHESTRATION TEST - [NEW][COMPILE REQUIRED]

```
[Fact]
public async Task Conflict_does_not_stage_or_commit()
{
    var fixture = CloseCaseFixture.ForClosedCase();

    var outcome = await fixture.Sut.ExecuteAsync(
        fixture.Command,
        fixture.AuthorizedActor,
        fixture.Causality,
        CancellationToken.None);

    Assert.IsType<CloseCaseOutcome.Conflict>(outcome);
    Assert.Equal(0, fixture.PublicFacts.StageCount);
    Assert.Equal(0, fixture.UnitOfWork.CommitCount);
}
```

Architecture rules make layer leakage executable

Evidence ID	Blocking rule	Required artifact
LAY-DOM-001	Domain references no Application, Edge, Infrastructure, Host, Contracts, ASP.NET Core, EF Core or serializer package.	Project/assembly/package scan plus public API manifest.
LAY-APP-001	Application references Domain and owned abstractions only; no IResult, HttpContext, DbContext, SQL, broker or public DTO.	Forbidden-type and assembly-reference test.
LAY-INF-001	Infrastructure implements Application ports; DbContext is internal/scoped; no Host or HTTP Edge dependency.	Reference, API, lifetime and descriptor report.
LAY-EDGE-001	Edge maps transport/contracts to Application and never references DbContext/provider or mutates aggregates.	Reference scan, endpoint/message mapping tests.
LAY-MAP-001	Domain objects/facts are never directly serialized; every public V1 mapping is explicit and fixture-tested.	Serializer source scan and golden contract fixtures.
LAY-TRAN-001	Aggregate state and pending public fact commit/roll back together in RootCauseDb.	Provider-backed transaction and fault test.
LAY-RULE-001	RC-INV-001/002 and deterministic/abstention rules have discovered unit tests tied to stable IDs.	Test report with zero-test protection.
LAY-REVIEW-001	Domain, application, data, contract and security owners accept mappings, gaps and resolutions.	Signed review record bound to build identity.

NO COVERAGE THEATRE

A line-coverage percentage cannot prove that a rule lives in the correct layer. Each stable preservation rule needs a named test that fails when the rule is weakened or bypassed.

RootCause layer status

Layer	Specified artifact	Evidence state
Domain	RootCauseCase, RC-INV-001/002, domain fact, issue-or-abstain decision input.	Reference code only; LAY-DOM-001 / LAY-RULE-001 pending.
Application	CloseRootCauseCase handler, access/store/fact/unit-of-work ports, explicit outcomes.	Reference code only; LAY-APP-001 pending.
Infrastructure	RootCauseDbContext, repository/unit of work, pending public-fact storage.	No database/provider/migration; LAY-INF-001 / LAY-TRAN-001 pending.
Edge	HTTP mapping, AlarmFloodDetectedV1 mapping, RootCauseVerdictIssuedV1 mapping.	No serializer/contract execution; LAY-EDGE-001 / LAY-MAP-001 pending.
Host	Chapter 8 composition binds layer implementations.	No compile/provider/runtime evidence; CMP evidence pending.
Current gate	G0 accepted; G1 template, composition and internal-layer design specified.	TPL/CMP/LAY/BUILD evidence remains pending.

CURRENT VERDICT

Chapter 9 does not advance RootCause to G1 or G2. It strengthens the exact code and test claim the generated repository must prove without pretending that RootCauseDb or a public event path exists.

False greens inside a service

False green	Why it looks correct	Why it fails
Four project folders exist	The solution resembles the diagram.	Types, packages and decisions can still point outward or bypass a layer.
Aggregate is an EF entity	Persistence works quickly.	Public setters/provider requirements may permit invalid state or pollute Domain.
Handler unit tests pass	Fakes return expected outcomes.	No relational transaction, mapping, serialization or public contract is proved.
Endpoint is thin	It delegates to a service.	The delegated service may still be transport-shaped or mutate DbContext directly.
SaveChanges succeeds	Aggregate state is stored.	Pending public fact may be absent, separately committed or semantically wrong.
DTO equals Domain model	Mapping code disappears.	Public compatibility and private evolution are now coupled.

ADR-007 - Preserve internal layers and explicit model mapping

ADR field	ADR-007 record
Context	RootCause must preserve inherited aggregate rules and decision authority while exposing HTTP/events and persisting a local source of truth.
Decision	Retain Domain, Application, Infrastructure and Edge responsibilities inside the service. Use Application-owned ports, explicit model mapping and one local commit for aggregate state plus pending public fact.
Consequences	More mapping and test surfaces are visible; Domain stays framework-poor; public contracts evolve independently; technical adapters remain replaceable; atomicity is local only.
Rejected	Anemic DTO Domain, active-record DbContext in handlers, generic repository in BuildingBlocks, Domain/public DTO reuse, Host business services, remote transaction, reflection contract mapping.
Review trigger	New invariant, use case, public contract/version, persistence model, provider, transaction boundary, entry point, authorization capability or correction path.

Risk register for internal layers

Risk	Early signal	Blocking control
Domain becomes persistence-shaped	Public setters, EF attributes, provider base classes or invalid constructor state.	LAY-DOM-001 plus aggregate materialization tests.
Application becomes transport-shaped	IResult, HttpContext, status codes, DTOs or broker disposition in handlers.	LAY-APP-001 forbidden-type test.
Infrastructure steals decisions	Repository filters or SQL procedures determine evidence sufficiency/verdict.	Domain rule tests plus provider mutation review.
Edge bypasses use case	Endpoint/consumer injects DbContext/repository or calls aggregate directly.	LAY-EDGE-001 and endpoint descriptor scan.
Contract leaks private model	Serializer accepts Domain entity/fact or schema follows private member rename.	LAY-MAP-001 golden fixtures and source scan.
Local commit is split	Aggregate and pending fact use different contexts/SaveChanges or remote broker call.	LAY-TRAN-001 fault/rollback test.
Correction is improvised	Setter/update endpoint overwrites FinalVerdict or original public fact.	Stop condition until correction ADR, reason, authorization and history exist.
In-memory tests overclaim	Provider-specific constraints, transactions and mappings never execute.	Relational provider-backed component evidence.

Official-source validation

The technical mechanics were checked against current Microsoft documentation on 18 July 2026. The stable points used here are that DbContext is a short-lived, non-thread-safe unit-of-work object; AddDbContext registers a scoped context by default; a single SaveChanges call uses a transaction when the provider supports it; Minimal API parameters can bind transport values and DI services; and IResult/TypedResults provide transport response implementations.

Official source	Stable point used here	NEXUS-1 boundary
Microsoft Learn - DbContext lifetime/configuration	Short unit of work, disposal, no parallel/thread sharing, scoped AddDbContext default.	Context stays internal to Infrastructure and one explicit work scope.
Microsoft Learn - EF Core transactions	One SaveChanges is transactional when supported; manual control only when req	Atomic claim is provider-tested and limited to

Official source	Stable point used here	NEXUS-1 boundary
Microsoft Learn - Minimal API parameter binding	Route/body/header values and DI services can bind to handler parameters.	Edge still maps external models to Application commands explicitly.
Microsoft Learn - Minimal API responses	IResult and TypedResults implement HTTP response behaviour.	Transport results remain in Edge and do not become Application outcomes.

The series baseline remains net8.0. Current documentation may show newer EF Core or ASP.NET Core versions; that does not upgrade the manuscript. Exact package versions, provider behaviour, SQL mapping, transaction semantics and response APIs must be compiled and executed under the pinned repository.

Technical material still required

The four-layer design is specified but no RootCause implementation exists in this manuscript environment. The aggregate/value types, deterministic decision engine, application outcomes/handlers/ports, authorization policy, EF mappings, relational schema, repository/unit of work, pending public-fact model, V1 mapping/serialization, HTTP/message adapters and test projects must be created.

TECHNICAL MATERIAL STATUS - [DOMAIN/APPLICATION/ADAPTER CODE + COMPILE/TEST EVIDENCE REQUIRED]

Implement the exact RootCause layer projects; compile every reference listing; run RC-INV-001/002 and deterministic/abstention tests; validate forbidden references/types; execute Application orchestration tests; map and fixture-test AlarmFloodDetectedV1 and RootCauseVerdictIssuedV1; run provider-backed mapping and local rollback tests; verify no direct Domain serialization; and publish LAY-DOM-001 through LAY-REVIEW-001. G1 remains specified but not evidenced.

Deliverable	Minimum technical content	Blocking evidence
Domain implementation	Aggregate, value types, rule IDs, issue/abstain model, internal facts and pure unit tests.	LAY-DOM-001 / LAY-RULE-001
Application implementation	Commands/outcomes, handlers, ports, local authorization intent and orchestration tests.	LAY-APP-001
Infrastructure implementation	DbContext, mappings, repository, unit of work, pending fact storage and relational fixture.	LAY-INF-001 / LAY-TRAN-001
Edge implementation	HTTP/message binding, actor/causality extraction, V1 mapping, serializer and contract fixtures.	LAY-EDGE-001 / LAY-MAP-001
Architecture harness	Assembly/package/API/forbidden-type scans tied to the exact build.	LAY-DOM-001 through LAY-EDGE-001
Review record	Pinned build identity, test reports, mapping review, exceptions, owners and current gate.	LAY-REVIEW-001 / TPL-BUILD-001

CHAPTER 9 - CODE AUDIT

One use case crosses four explicit ownership boundaries

The additions below follow one close-case request from Domain decision to Application orchestration, local persistence, pending public fact, and Edge response. Each model changes only at an explicit mapper or port.

EVIDENCE STATUS

These are compile-oriented reference assets aligned to net8.0. This environment has no dotnet executable or provider fixture, so compilation, migration, SQL transaction, serializer, and HTTP execution evidence remain PENDING.

EXECUTABLE ASSET 9-A - TYPED IDENTITY, EVIDENCE, AND VERDICT

```
public readonly record struct RootCauseCaseId(Guid Value)
{
    public static RootCauseCaseId Parse(string value) =>
        Guid.TryParse(value, out var id)
        ? new(id)
        : throw new DomainRuleViolation(
            "RC-ID-001", "Invalid RootCause case identity.");
}

public sealed record EvidenceItem(
    EvidenceId Id,
    AlarmReference Alarm,
    EvidenceDescriptor Descriptor);

public sealed record RootCauseVerdict(
    RootCauseCode Cause,
    ConfidenceBand Confidence);

public enum RootCauseCaseStatus { Open, Closed }
```

EXECUTABLE ASSET 9-B - AGGREGATE OWNS VALID STATE CHANGES

```
public sealed class RootCauseCase
{
    private readonly List<EvidenceItem> _evidence = [];
    private readonly List<object> _facts = [];

    private RootCauseCase() { } // EF materialization only
    public RootCauseCase(RootCauseCaseId id) => Id = id;

    public RootCauseCaseId Id { get; private set; }
    public RootCauseCaseStatus Status { get; private set; }
    public RootCauseVerdict? FinalVerdict { get; private set; }
    public IReadOnlyCollection<EvidenceItem> Evidence => _evidence;
    internal IReadOnlyCollection<object> Facts => _facts;

    public void AddEvidence(EvidenceItem item)
    {
        EnsureOpen();
        ArgumentNullException.ThrowIfNull(item);
        if (_evidence.All(x => x.Id != item.Id))
            _evidence.Add(item);
    }

    public void CloseWithVerdict(
        RootCauseVerdict verdict, DateTimeOffset decidedAt)
    {
        EnsureOpen();
        if (_evidence.Count == 0)
            throw new DomainRuleViolation(
                "RC-INV-002", "A verdict requires evidence.");

        FinalVerdict = verdict;
        Status = RootCauseCaseStatus.Closed;
        _facts.Add(new VerdictIssued(Id, verdict, decidedAt));
    }

    private void EnsureOpen()
    {
        if (Status == RootCauseCaseStatus.Closed)
            throw new DomainRuleViolation(
                "RC-INV-001", "A closed case cannot change.");
    }
}
```

DOMAIN DECISION

Determinism includes abstention and excludes delivery metadata

EXECUTABLE ASSET 9-C - DECISION INPUT AND PURE ENGINE

```
public sealed record RootCauseDecisionInput(
    IReadOnlyList<EvidenceItem> Evidence,
    IReadOnlyList<RootCauseHypothesis> Hypotheses);

public abstract record VerdictDecision
{
    private VerdictDecision() { }
    public sealed record Issue(RootCauseVerdict Verdict)
        : VerdictDecision;
    public sealed record Abstain(AbstentionReason Reason)
        : VerdictDecision;
}

public sealed class DeterministicVerdictPolicy
{
    public VerdictDecision Decide(RootCauseDecisionInput input)
    {
        var supported = input.Hypotheses
            .Select(h => new {
                Hypothesis = h,
                Score = input.Evidence.Count(h.Supports)
            })
            .OrderByDescending(x => x.Score)
            .ThenBy(x => x.Hypothesis.Code.Value,
                StringComparer.Ordinal)
            .ToArray();

        if (supported.Length == 0 || supported[0].Score == 0)
            return new VerdictDecision.Abstain(
                AbstentionReason.InsufficientEvidence);

        return new VerdictDecision.Issue(
            new RootCauseVerdict(
                supported[0].Hypothesis.Code,
                ConfidenceBand.FromScore(supported[0].Score)));
    }
}
```

EXECUTABLE ASSET 9-D - REPLAY AND ABSTENTION TESTS

```
[Fact]
public void LAY_RULE_001_same_input_produces_same_decision()
{
    var input = DecisionFixture.StableInput();
    var policy = new DeterministicVerdictPolicy();

    var first = policy.Decide(input);
    var replay = policy.Decide(input);

    replay.Should().BeEquivalentTo(first);
}

[Fact]
public void LAY_RULE_002_no_supported_hypothesis_abstains()
{
    var result = new DeterministicVerdictPolicy()
        .Decide(DecisionFixture.Unsupported());

    result.Should().Be(
        new VerdictDecision.Abstain(
            AbstentionReason.InsufficientEvidence));
}
```

EXCLUDED INPUTS

Attempt number, delivery time, broker headers, correlation identifiers, retry count, and explanatory prose never enter RootCauseDecisionInput.

APPLICATION ORCHESTRATION

Outcomes are explicit; ports describe required effects

EXECUTABLE ASSET 9-E - COMMAND, OUTCOME, AND PORTS

```
public sealed record CloseRootCauseCase(
    RootCauseCaseId CaseId);

public abstract record CloseCaseOutcome
{
    private CloseCaseOutcome() { }
    public sealed record Closed(RootCauseVerdict Verdict)
        : CloseCaseOutcome;
    public sealed record Abstained(AbstentionReason Reason)
        : CloseCaseOutcome;
    public sealed record Denied : CloseCaseOutcome;
    public sealed record NotFound : CloseCaseOutcome;
    public sealed record Conflict(string RuleId)
        : CloseCaseOutcome;
}

public interface IRootCauseCaseStore
{
    Task<RootCauseCase?> FindAsync(
        RootCauseCaseId id, CancellationToken cancellationToken);
}

public interface IRootCauseUnitOfWork
{
    Task CommitAsync(CancellationToken cancellationToken);
}

public interface IRootCausePublicFactWriter
{
    void Stage(VerdictIssued fact);
}
```

EXECUTABLE ASSET 9-F - ONE USE CASE, ONE COMMIT INTENT

```
public sealed class CloseRootCauseCaseHandler(
    IRootCauseCaseStore cases,
    IRootCauseAccessPolicy access,
    IRootCausePublicFactWriter facts,
    IRootCauseUnitOfWork unitOfWork,
    DeterministicVerdictPolicy policy,
    TimeProvider time)
{
    public async Task<CloseCaseOutcome> ExecuteAsync(
        CloseRootCauseCase command,
        ExecutionActor actor,
        CancellationToken cancellationToken)
    {
        if (!await access.MayIssueVerdictAsync(
            actor, command.CaseId, cancellationToken))
            return new CloseCaseOutcome.Denied();

        var aggregate = await cases.FindAsync(
            command.CaseId, cancellationToken);
        if (aggregate is null)
            return new CloseCaseOutcome.NotFound();

        return policy.Decide(aggregate.ToDecisionInput()) switch
        {
            VerdictDecision.Abstain x =>
                new CloseCaseOutcome.Abstained(x.Reason),
            VerdictDecision.Issue x =>
                await CloseAsync(aggregate, x.Verdict,
                    cancellationToken),
            _ => throw new UnreachableException()
        };
    }

    private async Task<CloseCaseOutcome> CloseAsync(
        RootCauseCase aggregate,
        RootCauseVerdict verdict,
        CancellationToken cancellationToken)
    {
        try
        {
            aggregate.CloseWithVerdict(verdict, time.GetUtcNow());
        }
        catch (DomainRuleViolation violation)
        {
            return new CloseCaseOutcome.Conflict(violation.RuleId);
        }
        facts.Stage(aggregate.Facts.OfType<VerdictIssued>().Single());
        await unitOfWork.CommitAsync(cancellationToken);
        return new CloseCaseOutcome.Closed(verdict);
    }
}
```

APPLICATION TESTS

Call order and absence of effects become observable

EXECUTABLE ASSET 9-G - SUCCESS PATH PROVES ORCHESTRATION

```
[Fact]
public async Task LAY_APP_001_close_stages_fact_then_commits_once()
{
    var trace = new CallTrace();
    var aggregate = RootCauseCaseFixture.OpenWithEvidence();
    var sut = HandlerFixture.Create(
        trace: trace,
        aggregate: aggregate,
        decision: new VerdictDecision.Issue(Verdicts.Known));

    var result = await sut.ExecuteAsync(
        new(aggregate.Id), Actors.Engineer, CancellationToken.None);

    result.Should().Be(
        new CloseCaseOutcome.Closed(Verdicts.Known));
    trace.Entries.Should().Equal(
        "authorize", "load", "stage-fact", "commit");
    trace.Count("commit").Should().Be(1);
}
```

EXECUTABLE ASSET 9-H - NON-SUCCESS PATHS CANNOT COMMIT

```
[Theory]
[InlineData(CaseScenario.Denied)]
[InlineData(CaseScenario.NotFound)]
[InlineData(CaseScenario.Abstained)]
public async Task LAY_APP_002_non_success_has_no_durable_effect(
    CaseScenario scenario)
{
    var fixture = HandlerFixture.For(scenario);

    _ = await fixture.Sut.ExecuteAsync(
        fixture.Command,
        Actors.Engineer,
        CancellationToken.None);

    fixture.Facts.Staged.Should().BeEmpty();
    fixture.UnitOfWork.CommitCalls.Should().Be(0);
}

[Fact]
public async Task LAY_APP_003_domain_conflict_is_an_explicit_outcome()
{
    var fixture = HandlerFixture.For(CaseScenario.AlreadyClosed);

    var result = await fixture.Execute();

    result.Should().Be(
        new CloseCaseOutcome.Conflict("RC-INV-001"));
    fixture.UnitOfWork.CommitCalls.Should().Be(0);
}
```

TEST SEAM

Application tests use controlled ports, a fixed TimeProvider, and the real Domain aggregate. They do not instantiate DbContext, HttpContext, a serializer, or a broker client.

INFRASTRUCTURE PERSISTENCE

Aggregate state and pending public fact share one transaction

EXECUTABLE ASSET 9-I - EF MAPPING PROTECTS THE AGGREGATE SURFACE

```
internal sealed class RootCauseCaseConfiguration
    : IEntityTypeConfiguration<RootCauseCase>
{
    public void Configure(EntityTypeBuilder<RootCauseCase> b)
    {
        b.ToTable("RootCauseCases", "RootCause");
        b.HasKey(x => x.Id);
        b.Property(x => x.Id)
            .HasConversion(x => x.Value, x => new(x));
        b.Property(x => x.Status)
            .HasConversion<string>().HasMaxLength(24);
        b.OwnsOne(x => x.FinalVerdict, verdict =>
        {
            verdict.Property(x => x.Cause)
                .HasColumnName("FinalCauseCode");
            verdict.Property(x => x.Confidence)
                .HasColumnName("FinalConfidence");
        });
        b.Navigation(x => x.Evidence)
            .UsePropertyAccessMode(PropertyAccessMode.Field);
    }
}
```

EXECUTABLE ASSET 9-J - PENDING FACT ROW AND LOCAL COMMIT

```
internal sealed class PendingPublicFact
{
    public Guid Id { get; init; }
    public required string Type { get; init; }
    public required string Payload { get; init; }
    public DateTimeOffset OccurredAt { get; init; }
    public DateTimeOffset? PublishedAt { get; private set; }
}

internal sealed class EfRootCauseUnitOfWork(
    RootCauseDbContext db) : IRootCauseUnitOfWork
{
    public async Task CommitAsync(
        CancellationToken cancellationToken)
    {
        // Aggregate updates and staged PendingPublicFacts are
        // tracked by the same scoped DbContext.
        await db.SaveChangesAsync(cancellationToken);
    }
}

internal sealed class EfPublicFactWriter(
    RootCauseDbContext db,
    IRootCauseFactSerializer serializer)
    : IRootCausePublicFactWriter
{
    public void Stage(VerdictIssued fact) =>
        db.PendingPublicFacts.Add(serializer.ToRow(fact));
}
```

ATOMICITY BOUNDARY

The local commit proves aggregate state plus a pending fact row. It does not prove broker delivery. Dispatch and publisher confirmation remain later Message Backbone obligations.

INFRASTRUCTURE FIXTURE

Rollback and materialization are provider-backed obligations

EXECUTABLE ASSET 9-K - RELATIONAL SUCCESS FIXTURE

```
[Fact]
public async Task LAY_TRAN_001_state_and_fact_commit_together()
{
    await using var fixture = await RootCauseSqlFixture.StartAsync();
    await fixture.SeedOpenCaseAsync(Cases.A, Evidence.Known);

    var result = await fixture.ExecuteCloseAsync(Cases.A);

    result.Should().BeOfType<CloseCaseOutcome.Closed>();
    await using var verification = fixture.CreateDbContext();
    var aggregate = await verification.RootCauseCases
        .SingleAsync(x => x.Id == Cases.A);
    var fact = await verification.PendingPublicFacts
        .SingleAsync();
    aggregate.Status.Should().Be(RootCauseCaseStatus.Closed);
    fact.Type.Should().Be("RootCauseVerdictIssued.v1");
    fact.PublishedAt.Should().BeNull();
}
```

EXECUTABLE ASSET 9-L - FAILED FACT INSERT ROLLS BACK STATE

```
[Fact]
public async Task LAY_TRAN_002_fact_failure_rolls_back_case()
{
    await using var fixture = await RootCauseSqlFixture.StartAsync();
    await fixture.SeedOpenCaseAsync(Cases.A, Evidence.Known);
    fixture.FailPendingFactInsert();

    var act = () => fixture.ExecuteCloseAsync(Cases.A);

    await act.Should().ThrowAsync<DbUpdateException>();
    await using var verification = fixture.CreateDbContext();
    var aggregate = await verification.RootCauseCases
        .AsNoTracking().SingleAsync(x => x.Id == Cases.A);
    aggregate.Status.Should().Be(RootCauseCaseStatus.Open);
    (await verification.PendingPublicFacts.CountAsync())
        .Should().Be(0);
}
```

EXECUTABLE ASSET 9-M - MATERIALIZATION CANNOT BYPASS RULES

```
[Fact]
public async Task LAY_INF_001_closed_case_rehydrates_as_closed()
{
    await using var fixture = await RootCauseSqlFixture.StartAsync();
    await fixture.SeedClosedCaseAsync(Cases.A);

    var aggregate = await fixture.LoadAsync(Cases.A);

    var act = () => aggregate.AddEvidence(Evidence.New());
    act.Should().Throw<DomainRuleViolation>()
        .Which.RuleId.Should().Be("RC-INV-001");
}
```

BLOCKING EVIDENCE

These tests require the pinned SQL provider, real migrations, constraints, and transaction semantics. An EF InMemory green result is not accepted as LAY-INF-001 or LAY-TRAN-001.

EDGE TRANSLATION

Public contracts never become Domain citizens

EXECUTABLE ASSET 9-N - EXPLICIT V1 MAPPING

```
internal static class RootCauseContractMapper
{
    public static CloseRootCauseCase ToCommand(
        this CloseRootCauseCaseRequest request) =>
        new(RootCauseCaseId.Parse(request.CaseId));

    public static RootCauseVerdictIssuedV1 ToContract(
        this VerdictIssued fact,
        CausalityContext causality) =>
        new(
            MessageId: Guid.NewGuid().ToString("D"),
            CaseId: fact.CaseId.Value.ToString("D"),
            CauseCode: fact.Verdict.Cause.Value,
            Confidence: fact.Verdict.Confidence.Value,
            OccurredAt: fact.DecidedAt,
            CorrelationId: causality.CorrelationId);
}
```

EXECUTABLE ASSET 9-O - HTTP ENDPOINT OWNS TRANSPORT MAPPING

```
group.MapPost("/{caseId}/close", async (
    string caseId,
    ClaimsPrincipal principal,
    ICloseRootCauseCase useCase,
    CancellationToken cancellationToken) =>
{
    var command = new CloseRootCauseCase(
        RootCauseCaseId.Parse(caseId));
    var actor = ExecutionActor.From(principal);
    var outcome = await useCase.ExecuteAsync(
        command, actor, cancellationToken);

    return outcome switch
    {
        CloseCaseOutcome.Closed x =>
            TypedResults.Ok(CloseCaseResponse.From(x)),
        CloseCaseOutcome.Abstained x =>
            TypedResults.Accepted(value:
                AbstainedResponse.From(x)),
        CloseCaseOutcome.Denied =>
            TypedResults.Forbid(),
        CloseCaseOutcome.NotFound =>
            TypedResults.NotFound(),
        CloseCaseOutcome.Conflict x =>
            TypedResults.Conflict(
                ProblemDetailsMapper.From(x)),
        _ => throw new UnreachableException()
    };
}).RequireAuthorization("rootcause.operator");
```

EXECUTABLE ASSET 9-P - CONTRACT EXCLUSION TEST

```
[Fact]
public void LAY_MAP_001_public_event_has_no_delivery_inputs()
{
    var json = JsonSerializer.Serialize(
        ContractFixture.VerdictIssuedV1());

    json.Should().Contain("\"caseId\"");
    json.Should().Contain("\"causeCode\"");
    json.Should().NotContain("attempt");
    json.Should().NotContain("retryCount");
    json.Should().NotContain("brokerHeader");
    json.Should().NotContain("explanation");
}
```

ARCHITECTURE EVIDENCE

A layer claim is a build-bound, reviewable record

EXECUTABLE ASSET 9-Q - DEPENDENCY AND FRAMEWORK GATES

```
[Fact]
public void LAY_DOM_001_domain_is_framework_poor() =>
    Types.InAssembly(typeof(RootCauseCase).Assembly)
        .ShouldNot().HaveDependencyOnAny(
            "Microsoft.EntityFrameworkCore",
            "Microsoft.AspNetCore",
            "RabbitMQ.Client",
            "System.Text.Json")
        .GetResult().IsSuccessful.Should().BeTrue();

[Fact]
public void LAY_EDGE_001_edge_does_not_use_infrastructure() =>
    Types.InAssembly(typeof(RootCauseEndpoints).Assembly)
        .ShouldNot().HaveDependencyOn(
            "Nexus.RootCause.Infrastructure")
        .GetResult().IsSuccessful.Should().BeTrue();

[Fact]
public void LAY_DOM_002_domain_is_not_serialized_directly()
{
    ForbiddenApiScan.InProjects("*.Edge", "/*.Infrastructure")
        .RejectGenericArgumentFrom(
            "System.Text.Json.JsonSerializer",
            namespacePrefix: "Nexus.RootCause.Domain");
}
```

EXECUTABLE ASSET 9-R - NORMALIZED EVIDENCE RECORD

```
{
  "evidenceId": "LAY-REVIEW-001",
  "chapter": 9,
  "targetFramework": "net8.0",
  "restoreLockSha256": null,
  "commitSha": null,
  "tests": {
    "LAY-DOM-001": "PENDING",
    "LAY-RULE-001": "PENDING",
    "LAY-APP-001": "PENDING",
    "LAY-INF-001": "PENDING",
    "LAY-TRAN-001": "PENDING",
    "LAY-EDGE-001": "PENDING",
    "LAY-MAP-001": "PENDING"
  },
  "provider": null,
  "migration": null,
  "accepted": false
}
```

Chapter 9 code-audit result

The chapter now exposes a continuous implementation path: typed Domain model, deterministic issue-or-abstain policy, Application-owned ports and outcomes, provider-backed local atomicity, explicit V1/HTTP translation, and build-bound architecture evidence. The additions preserve RC-INV-001, RC-INV-002, RC-OWN-001 and the rule that Host only composes.

CURRENT VERDICT

The code removes implementation ambiguity but does not manufacture execution evidence. LAY/TPL/CMP/BUILD records remain pending until the pinned repository compiles and runs them.

Implementation sequence

1. Commit ADR-007 and bind RC-INV-001, RC-INV-002, RC-OWN-001, deterministic replay and abstention to stable test identities.
2. Implement Domain value types, RootCauseCase behaviour, internal facts and pure invariant tests with no framework packages.
3. Implement Application commands, outcomes, ports and CloseRootCauseCase orchestration with local authorization and explicit commit intent.
4. Implement the HTTP and AlarmFloodDetectedV1 Edge mappings without exposing transport or public DTOs inward.
5. Implement RootCauseVerdictIssuedV1 mapping through the Application-owned public-fact writer and pending-store ports.
6. Implement RootCauseDbContext, persistence mapping, repository, unit of work and pending public-fact row in Infrastructure.
7. Add project, assembly, package, public API, forbidden-type and direct-Domain-serialization architecture tests.
8. Run Domain/Application tests, then provider-backed mapping, transaction rollback and Edge contract/component tests.
9. Generate normalized layer/API/mapping manifests and bind every result to the pinned restore/build identity.
10. Review gaps and exceptions; accept G1 only when TPL, CMP, LAY and BUILD blockers all pass without weakening preservation rules.

What this chapter has proved

Chapter 9 has placed inherited RootCause rules in Domain, use-case authorization/orchestration and ports in Application, EF Core and durable local effects in Infrastructure, and public transport/contract mapping in Edge. It has separated internal facts from V1 contracts, preserved issue-or-abstain determinism, defined one local commit for aggregate state plus pending public fact, and created layer-specific evidence obligations.

It has not compiled or executed those artifacts. RootCauseDb, migrations, provider mappings, HTTP/message serializers, outbox schema, identities, tests and public event flow remain absent. RootCause remains G0 accepted; G1 template, composition and internal-layer behaviour are specified; TPL/CMP/LAY/BUILD evidence remains pending.

CHAPTER CLOSING

A service has internal integrity when each layer protects one kind of ownership, every model changes explicitly at a boundary, and no framework convenience can bypass the aggregate rule that owns the truth.

Chapter checkpoint

You are ready for Chapter 10 when every RootCause rule, use case, adapter and external model has one owning layer and one explicit boundary crossing.

Question	Expected answer
What belongs in Domain?	RootCause language, valid state, invariants, deterministic issue/abstain decisions and internal facts.
What belongs in Application?	Use cases, local authorization intent, orchestration, ports, transaction intent and explicit outcomes.
What belongs in Infrastructure?	EF Core mappings, RootCauseDb adapters, unit of work, pending-fact storage and technical integrations.
What belongs in Edge?	HTTP/message binding, actor/causality extraction, public-contract mapping and transport results.
Why not serialize the aggregate?	Private evolution and invariant state would become a public compatibility promise.
What commits atomically?	Only RootCause-owned aggregate state and its pending public fact in one RootCauseDb transaction.
What is the current RootCause gate?	G0 accepted; G1 template, composition and layer behaviour specified; executable evidence pending.
What must Chapter 10 add?	Typed, validated, secret-safe configuration and explicit environment boundaries without changing business rules.

NEXT CHAPTER

Chapter 10 defines Configuration and Environment Boundaries. It separates deploy-time mechanics from domain policy, validates required settings before readiness, prevents secret/configuration leakage across layers, and proves that environment differences do not silently select different RootCause truth.

PART II - THE SERVICE FOUNDATION

10. Configuration and Environment Boundaries

Configuration is executable input. It selects technical collaborators, resource limits, endpoints and accepted policy identities; if it remains an untyped global dictionary, it can also change truth without review, leak secrets, or let one environment behave like a different service.

CHAPTER ARGUMENT

Host owns provider order, binding, validation and environment-specific composition. Edge and Infrastructure consume narrow technical options. Application receives explicit policy snapshots. Domain never receives IConfiguration, IOptions, IHostEnvironment, a secret or an environment name.

Chapter 9 ended with a ConfigurationIdentity inside the deterministic decision input. That identity was not decorative metadata. If a threshold, evidence-ordering rule, engine mode or other decision-affecting value can change, RootCause must name the accepted policy snapshot that produced each issue-or-abstain result. Delivery timing, retry count, environment label and generated explanation remain excluded from truth.

The baseline remains net8.0. The code below is reference material for the future generated repository. No RootCause configuration sections, production secret provider, environment manifests, startup tests or replay fixtures exist in this manuscript environment, so the chapter specifies evidence without claiming it has run.

Configuration is a boundary, not a global service

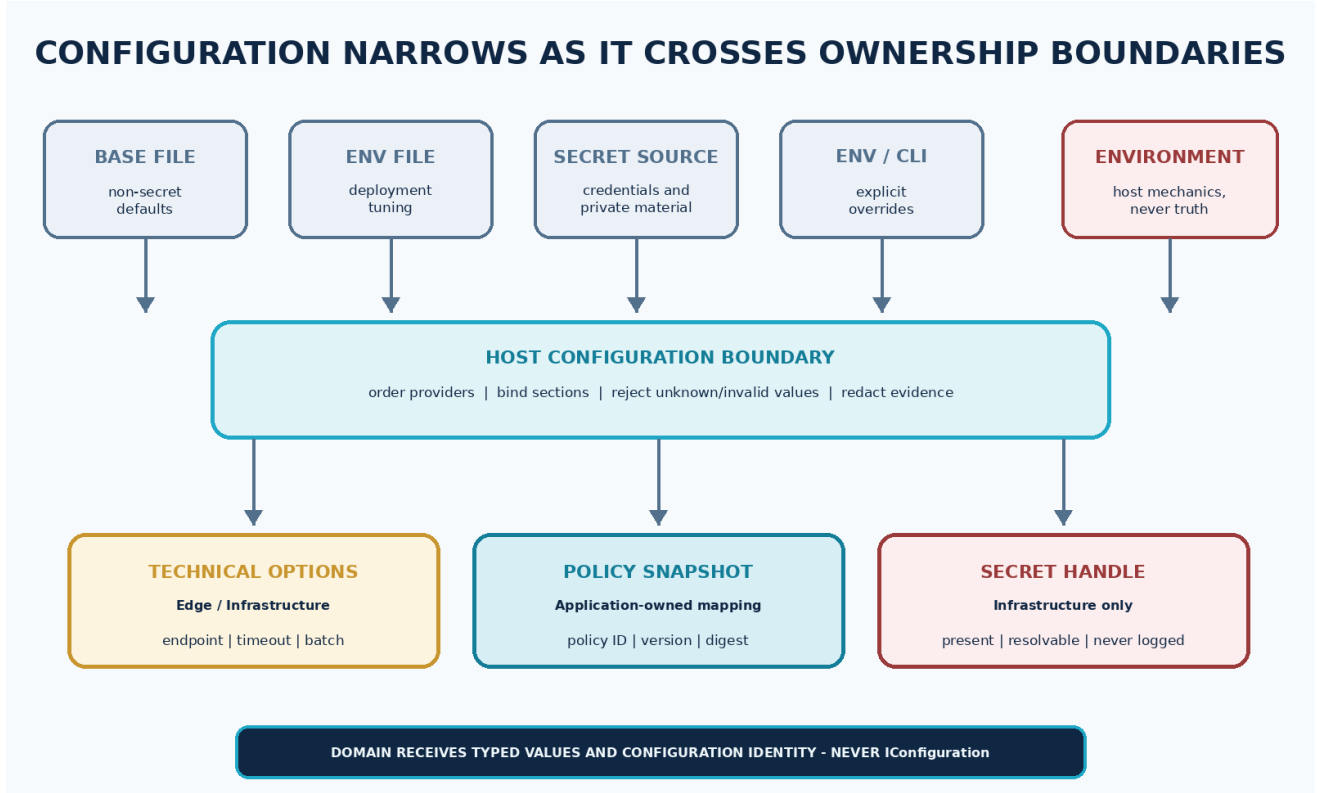


Figure 10.1 - Raw provider values narrow into owned technical options, policy snapshots and secret handles.

Layer or owner	Owns	Must not receive
Deployment	Environment label, provider material, secret injection, resource attachments and rollout identity.	Domain objects, business-rule switches, arbitrary application internals or permission to weaken validation.
Host	Provider order, section binding, validation-on-start, environment-specific composition and safe manifest generation.	Verdict authority, aggregate mutation, policy meaning or raw configuration passed through the graph.
Edge	Transport-owned timeouts, limits, endpoint exposure and public-adapter settings.	Database credentials, decision policy parsing, environment-selected contract meaning or IConfiguration in handlers.
Infrastructure	Database, broker, secret-provider and technical-client options plus rotation mechanics.	RootCause invariant switches, HTTP semantics, unredacted values in telemetry or Domain dependencies.
Application	Typed decision-policy mapping, accepted configuration identity and use-case-visible capability settings.	IConfiguration, provider names, secret strings, EnvironmentName or adapter configuration classes.
Domain	Validated typed values/value objects and explicit ConfigurationIdentity where truth depends on them.	IOptions, IConfiguration, host environment, files, variables, reload notifications, credentials or endpoints.

BOUNDARY TEST

If a class asks IConfiguration for a key, that class now owns provider spelling, precedence, conversion, fallback and secret handling. In NEXUS-1, only Host registration and narrowly reviewed adapter bootstrap code may make that claim.

Classify settings before choosing an API

A configuration key is not adequately designed until its owner, sensitivity, mutability, validation, failure mode and evidence are known. The same string-shaped value may be a harmless display label, a credential, a restart-bound endpoint or a decision-affecting policy input. Those categories cannot share one reload or logging rule.

Configuration class	Examples	Change contract	Required evidence
Host mechanics	Environment name, URLs, process mode, diagnostics surface.	Selected before or during Host construction; restart for structural change.	Host manifest and composition test.
Adapter technical	Database endpoint, command timeout, broker topic, batch size.	Restart-bound unless the adapter has a tested atomic reload contract.	Typed binding, validation and adapter test.
Operational tuning	Safe log level, polling cadence, bounded concurrency.	Reload only when partial/invalid change behavior is explicit and observable.	Reload acceptance/rejection test.
Decision policy	Evidence ordering, deterministic engine mode, policy version.	Immutable accepted snapshot; new identity for every semantic change.	Canonical digest and replay fixture.
Secret	Database credential, signing key, broker credential.	Never logged or committed; rotation is provider/adapter-owned and tested.	Secret scan, redaction and rotation evidence.
Build identity	Commit, lockfile, SDK and artifact digests.	Immutable for the process; configuration cannot rewrite it.	BUILD/TPL manifest linkage.

NO KEY WITHOUT A CONTRACT

A new setting is an architecture change. Review its owner, default, range, sensitivity, override path, mutability, rollback behavior and evidence before adding it to appsettings.json or a deployment manifest.

Provider order is part of the runtime contract

The default ASP.NET Core builder composes ordered configuration providers. For duplicate keys, later providers override earlier providers. The effective value is therefore the result of both data and order. A correct value from the wrong provider can still be a governance failure, especially when a command-line override or inherited environment variable silently defeats the reviewed deployment value.

Default source	Relative priority	NEXUS-1 use
appsettings.json	Lowest application source	Non-secret, environment-neutral technical defaults only.
appsettings.{Environment}.json	Overrides base file	Environment-specific mechanics and tuning; never weakened invariants or hidden authorization.
User secrets in Development	Overrides JSON files	Developer-only sensitive material; not production and not an encrypted trusted store.
Non-prefixed environment variables	Overrides files and user secrets	Deployment injection; hierarchical keys use double underscore across platforms.
Command-line arguments	Highest default application source	Explicit operational override only when deployment policy allows and records it.
Fallback host configuration	Below application configuration	Host bootstrap values; application configuration wins when the same key exists.

PROVIDER AND ENVIRONMENT CONVENTION - [NEW][COMPILE REQUIRED]

```
// Default builder order is retained and evidenced.
var builder = WebApplication.CreateBuilder(args);

// Cross-platform hierarchy uses '__' in environment variable names:
// RootCauseDb__CommandTimeoutSeconds=30
// RootCausePublicFacts__Topic=rootcause-verdict-issued-v1

// Never set a decision rule from DOTNET_ENVIRONMENT or
// ASPNETCORE_ENVIRONMENT. Environment selects Host mechanics only.
```

A production evidence bundle records the approved configuration schema, source kinds, order and redacted presence state. It does not dump IConfiguration.AsEnumerable(), connection strings, tokens, certificates or secret hashes. Provider diagnostics are useful only after a secret-safe allow-list is applied.

Typed options make adapter contracts reviewable

Each technical adapter binds one named section into one options type that lives with that adapter. Required values, numeric ranges, formats and cross-field rules are validated before the service can become ready. Unknown keys and conversion failures are treated as defects because a misspelled timeout that falls back to zero is not a harmless deployment detail.

INFRASTRUCTURE-OWNED TECHNICAL OPTIONS - [NEW][COMPILE REQUIRED]

```
public sealed class RootCauseDbOptions
{
    public const string Section = "RootCauseDb";

    [Required]
    public string ConnectionString { get; init; } = string.Empty;

    [Range(1, 300)]
    public int CommandTimeoutSeconds { get; init; } = 30;

    [Range(0, 10)]
    public int MaxRetryCount { get; init; } = 3;
}

public sealed class RootCausePublicFactOptions
{
    public const string Section = "RootCausePublicFacts";

    [Required, MinLength(1)]
    public string Topic { get; init; } = string.Empty;

    [Range(1, 500)]
    public int MaxBatchSize { get; init; } = 100;

    [Range(100, 60_000)]
    public int PollIntervalMilliseconds { get; init; } = 1_000;
}
```

HOST BINDING AND FAIL-FAST VALIDATION - [NEW][COMPILE REQUIRED]

```
builder.Services
    .AddOptions<RootCauseDbOptions>()
    .Bind(
        builder.Configuration.GetSection(RootCauseDbOptions.Section),
        binder => binder.ErrorOnUnknownConfiguration = true)
    .ValidateDataAnnotations()
    .Validate(
        options => !string.IsNullOrEmpty(options.ConnectionString),
        "RootCauseDb connection material is required.")
    .ValidateOnStart();

builder.Services
    .AddOptions<RootCausePublicFactOptions>()
    .Bind(
        builder.Configuration.GetSection(RootCausePublicFactOptions.Section),
        binder => binder.ErrorOnUnknownConfiguration = true)
    .ValidateDataAnnotations()
    .ValidateOnStart();
```

VALIDATION BOUNDARY

ValidateOnStart proves that registered option values can be created and pass registered validators. It does not prove database reachability, topic existence, permissions, network path or downstream health. Chapter 11 assigns those dependency checks to readiness and startup policy.

Binding success is not semantic acceptance

Data annotations protect basic shape. Adapter-specific validators must protect cross-field and provider rules: a retry delay must fit the request budget, a topic must match the producer contract, a certificate reference must use an approved scheme, and a database endpoint must not point to another service's source of truth. Binding a string cannot prove any of those meanings.

Validation level	Example rejection	Owner
Presence and conversion	Missing section, malformed integer, invalid enum, unknown key.	Host binding contract.
Range and format	Zero timeout, negative batch size, unsupported URI/scheme.	Adapter options/validator.

Validation level	Example rejection	Owner
Cross-field	Retry budget exceeds timeout; TLS required but disabled; incompatible mode pair.	Adapter validator.
Ownership	RootCause connection points at Alarm or Audit database; consumer topic is producer-owned elsewhere.	Architecture/deployment review.
Decision policy	Unrecognized policy ID/version, digest mismatch, non-canonical evidence ordering.	Application policy mapper and governance.
Connectivity/capability	Credential cannot authenticate; schema/topic unavailable; permission denied.	Readiness/startup dependency policy in Chapter 11.

The net8.0 binder can be configured to reject unknown configuration and conversion failures through `ErrorOnUnknownConfiguration`. That strictness must be compiled under the pinned package graph, because options extension overloads and source-generation choices depend on the exact `Microsoft.Extensions` packages selected by the repository.

Environment can change mechanics, not meaning

Development, Staging and Production are Host labels. They may select diagnostics, local substitutes, endpoints, resource limits, secret sources and observability exporters. They may not select different `RootCause` invariants, change who owns the verdict, turn insufficient evidence into a verdict, alter public-contract meaning, or make retry count a decision input.

Environment-dependent difference	Allowed?	Reason
Developer exception detail at a local Edge	Yes, with exposure controls	Transport diagnostics can vary without changing Application/Domain outcomes.
Database endpoint and credential source	Yes	Infrastructure location and access material are deployment mechanics.
Command timeout and bounded concurrency	Yes, within validated safety limits	Operational capacity may vary; business truth may not.
Use fake adapter in explicit test host	Yes, in a separately evidenced test composition	The substitute is a Host choice, not a Domain branch.
Disable RC-INV-002 in Development	No	A case without evidence is invalid in every environment.
Production-only verdict threshold without policy identity	No	Environment would silently become a hidden decision input.
Skip local pending-public-fact commit in tests	No	The test would exercise a different correctness boundary.
Expose secret/configuration dump only in Development	No by default	Developer environments still leak into logs, screenshots and support bundles.

ENVIRONMENT-INVARIANCE TEST - [NEW][COMPILE REQUIRED]

```
[Theory]
[InlineData("Development")]
[InlineData("Staging")]
[InlineData("Production")]
public void Same_policy_identity_preserves_domain_decision(
    string environmentName)
{
    var host = RootCauseTestHost.Build(
        environmentName,
        policy: Policies.AcceptedV7);

    var actual = host.Decide(Evidence.OrderedKnownCase);

    Assert.Equal(Expected.IssueKnownVerdict, actual.Decision);
    Assert.Equal(Policies.AcceptedV7.Identity, actual.Configuration);
}
```

FORBIDDEN DEPENDENCY

Domain and Application architecture tests must reject IHostEnvironment, IWebHostEnvironment, IConfiguration and every IOptions interface. A test that only searches for IsDevelopment() misses constructor injection and wrapper aliases; scan public APIs, fields, parameters and referenced assemblies.

Decision configuration is a versioned input

Decision-affecting configuration is not ordinary options. Host binds and validates the external representation. An Application-owned mapper then constructs an immutable policy snapshot with a stable policy ID, version and canonical digest. Domain receives only the typed decision values and ConfigurationIdentity. The accepted identity is stored with the decision and emitted in the producer-owned public fact so replay can select the same snapshot.

VERSIONED DECISION-POLICY BOUNDARY - [NEW][COMPILE REQUIRED]

```
public sealed record DecisionPolicySettings
{
    public const string Section = "RootCauseDecisionPolicy";

    public required string PolicyId { get; init; }
    public required int Version { get; init; }
    public required string EvidenceOrdering { get; init; }
    public required string ExpectedSha256 { get; init; }
}

public sealed record DecisionPolicySnapshot(
    DecisionPolicyId PolicyId,
    DecisionPolicyVersion Version,
    EvidenceOrdering Ordering,
    ConfigurationIdentity Identity);

// Application mapping validates allowed values and computes a canonical
// digest from named, ordered, non-secret policy fields. It then compares
// the result with ExpectedSha256 before the snapshot can be accepted.
```

Policy rule	Why it blocks	Evidence
Canonical fields only	Raw JSON property order, whitespace, culture and provider formatting cannot define identity.	Canonicalization fixture and digest vector.
No secret material	A secret in the digest input leaks rotation into business replay and may expose a comparison oracle.	Policy schema and secret-classification scan.
Identity stored with result	Current configuration cannot reliably replay a historical decision.	Persistence/public-contract mapping test.
Unknown identity fails closed	Falling back to latest policy rewrites historical truth.	Replay test for missing policy artifact.
Semantic change creates identity	Editing a value under the same version makes evidence ambiguous.	Immutable artifact and review control.
Environment is excluded	Development/Production labels are deployment context, not causal evidence.	Cross-environment decision equivalence test.

REPLAY CONTRACT

Given the same ordered evidence, engine version and recorded ConfigurationIdentity, RootCause must produce the same issue-or-abstain result. If the policy artifact is missing or its digest differs, replay fails explicitly; it never substitutes the current environment's settings.

Secrets are capabilities, not configuration evidence

Passwords, tokens, private keys and credential-bearing connection strings must not be committed in source or appsettings files. Development Secret Manager keeps values outside the project tree but does not encrypt them and is not a production trust boundary. Environment variables are easy to inject but are commonly plain text and may be visible to a compromised process. Production uses a controlled secret provider or workload identity selected by deployment policy.

Secret control	Required behavior	Forbidden evidence
Source control	Commit schema/examples with placeholders; scan history and generated artifacts.	Real credential, private key, token or password-bearing connection string.
Local development	Use development-only secret storage with isolated non-production values.	Claim that Secret Manager encrypts or safely stores production credentials.
Deployment	Inject through an approved provider/identity with least privilege and auditable access.	Bake the secret into image, Helm values, pipeline log or command transcript.
Application use	Expose the minimum adapter capability; keep raw secret inside Infrastructure/bootstrap scope.	Return secret through API, domain model, exception, health response or public fact.
Observability	Record key classification, required/present state and provider category only.	Value, substring, hash, length, certificate body, token claims or connection string.
Rotation	Define whether adapter reconnects, refreshes or requires restart; test old/new overlap and failure.	Assume IOptionMonitor alone rotates an active database/broker client.

SECRET-SAFE CONFIGURATION EVIDENCE - [NEW][COMPILE REQUIRED]

```
public sealed record SafeConfigurationEntry(
    string Key,
    string Classification,
    string Presence,
    string ChangeContract);

// The allow-list contains no values and never enumerates unknown keys.
var manifest = new[]
{
    new SafeConfigurationEntry(
        "RootCauseDb:ConnectionString",
        "secret",
        hasDatabaseCredential ? "present" : "missing",
        "restart-bound"),
    new SafeConfigurationEntry(
        "RootCauseDecisionPolicy",
        "decision-policy",
        acceptedPolicy.Identity.Value,
        "versioned-immutable")
};
```

NO SECRET FINGERPRINT

Hashing a low-entropy password or a structured credential is not redaction. Configuration evidence records approved identity for non-secret policy artifacts and presence/classification for secrets, never the secret value or a derivative of it.

Reload is an acceptance protocol

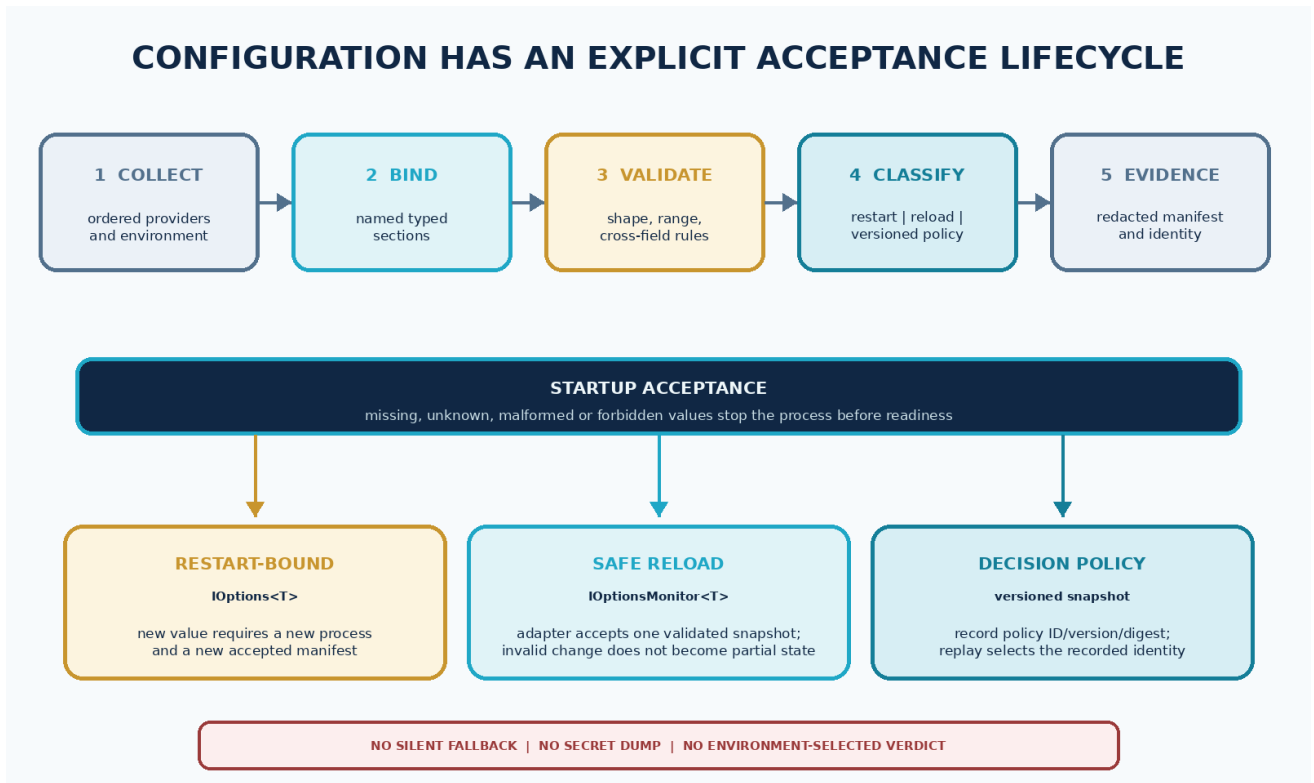


Figure 10.2 - Startup acceptance and post-start change behavior are distinct, explicit contracts.

`IOptions<T>` exposes a startup-created value and does not read changes after startup. `IOptionsSnapshot<T>` is scoped and recalculated when resolved for a new scope. `IOptionsMonitor<T>` is singleton-capable and supports current values and change notifications when the underlying provider supports them. Those mechanics do not decide whether a NEXUS-1 setting is safe to change.

Consumption model	Permitted NEXUS-1 use	Blocking caution
<code>IOptions<T></code>	Restart-bound adapter options and immutable process configuration.	A changed source does not change the already consumed value; restart/rollout evidence is required.
<code>IOptionsSnapshot<T></code>	Scoped presentation or request mechanics with a reviewed snapshot boundary.	It is scoped, not available to singleton dependencies, and must not choose Domain truth per request.
<code>IOptionsMonitor<T></code>	Explicitly reloadable operational adapters with atomic snapshot validation and observable acceptance.	Provider may not notify reliably; invalid/multi-key changes need a defined last-known-good or fail policy.
Versioned policy store	Decision-affecting immutable snapshots selected by accepted identity.	Never replace with <code>CurrentValue</code> or latest-on-replay semantics.
Secret-provider client	Credential retrieval/rotation through the Infrastructure adapter's provider contract.	Options notification alone does not prove live clients discarded old credentials safely.

A reloadable group must be accepted as one validated snapshot. Updating three independent variables can expose a temporary mixed state. If atomic cross-field change matters, publish one versioned document/reference and switch identity once, or require a restart. The framework does not automatically provide a last-known-good acceptance layer for every options monitor scenario; that policy must be implemented and fault-tested.

Startup evidence is redacted and reproducible

A successful process start is not enough. The build evidence bundle needs a safe configuration manifest tied to the build identity and deployment candidate: environment name, approved provider kinds/order, typed section schema versions, validation result, restart/reload classification, accepted decision-policy identity and secret pre:

Evidence ID	Blocking rule	Required artifact
CFG-OWN-001	Domain/Application expose no configuration, options, host-environment or secret-provider framework types.	Assembly/public API/forbidden-type scan.
CFG-SRC-001	Provider kinds and order match the approved Host contract; override cases are deterministic.	Provider-order manifest and precedence tests.
CFG-BIND-001	Every owned section binds strictly; unknown keys and malformed values fail.	Schema fixtures and negative binding report.
CFG-VAL-001	Missing, out-of-range and cross-field-invalid required settings stop startup before readiness.	Host startup matrix with expected failures.
CFG-ENV-001	Environment differences change mechanics only; identical policy identity preserves Domain outcomes.	Cross-environment composition and decision tests.
CFG-DEC-001	Every decision records a canonical accepted ConfigurationIdentity and replays with it.	Digest vectors, persistence mapping and replay tests.
CFG-SEC-001	Secrets are absent from source, image, logs, errors, manifests, API/public facts and test snapshots.	Secret scan and canary redaction test.
CFG-REL-001	Every reloadable group has atomic validation, acceptance/rejection telemetry and rollback/restart behavior.	Change, partial-write and provider-notification fault tests.
CFG-MAN-001	Safe manifest is value-free, tied to build/deployment identity and reviewable.	Normalized redacted manifest and schema.
CFG-REVIEW-001	Platform, service, security and domain owners accept sources, policies, exceptions and residual risk.	Signed review bound to exact build identity.

EVIDENCE RULE

A screenshot of a successful deployment is not configuration evidence. Re-run the negative startup matrix, precedence fixtures, secret canary and policy replay against the exact pinned build and deployment candidate.

Configuration tests fail at the owning boundary

Suite	Primary proof	Must not require
Binding unit	Known keys bind; unknown/malformed/missing values fail with stable diagnostics.	Database, broker, network or production secret.
Validator unit	Ranges, formats, cross-field and ownership constraints reject unsafe combinations.	Web server or environment-specific branch.
Host component	Provider order, environment composition and ValidateOnStart fail/accept matrix.	Another service source of truth.
Policy replay	Canonical digest vectors and same identity/evidence/engine produce same result.	Current environment settings or wall clock.
Secret canary	Known synthetic secret never appears in logs, errors, health, manifests or HTTP/message output.	Real credential or production provider.
Reload fault	Valid snapshot accepted once; invalid/partial change rejected or process withdrawn by policy.	Assumption that file watcher behavior equals all deployment providers.
Architecture	Forbidden configuration/host types remain outside Host/adapters and secrets stay Infrastructure-owned.	Runtime success as a substitute for static evidence.

BLOCKING STARTUP AND REPLAY TESTS - [NEW][COMPILE REQUIRED]

```
[Fact]
public async Task Missing_database_connection_fails_startup()
{
    var result = await RootCauseHostFixture.StartAsync(new Dictionary<string, string?>
    {
        ["RootCauseDb:CommandTimeoutSeconds"] = "30"
    });

    Assert.False(result.Started);
    Assert.Contains("RootCauseDb", result.SafeDiagnostic);
    Assert.DoesNotContain("ConnectionString=", result.SafeDiagnostic);
}

[Fact]
public void Unknown_decision_policy_identity_never_falls_back_to_latest()
{
    var replay = ReplayFixture.WithoutPolicy("rootcause-policy/7");

    Assert.Throws<ConfigurationIdentityUnavailable>(
        () => replay.Execute());
}
```

False greens at the configuration boundary

False green	Why it looks correct	Why it fails
The app starts with defaults	No deployment values are required locally.	A missing production key may silently select unsafe or unusable behavior.
ValidateDataAnnotations passes	Required and range attributes are present.	Cross-field, ownership, provider and connectivity semantics remain unproved.
Environment file is reviewed	Production settings are visible in source.	Environment variables or command-line arguments may override them later.
Secrets are not in appsettings.json	Repository scan is clean.	Values can still leak through environment dumps, exceptions, manifests or generated files.
IOptionsMonitor updates	A test observes CurrentValue change.	Active clients, partial changes, invalid snapshots and notification loss are not proved safe.
Replay uses current settings	Recent decisions reproduce.	Historical identity is missing and a future change silently rewrites the result.
Development differs for convenience	Local work is faster.	The test host may bypass the invariant, transaction or public-fact boundary under review.

ADR-008 - Make configuration typed, owned and evidenced

ADR field	ADR-008 record
Context	RootCause needs environment-specific technical composition and secrets without allowing providers, reloads or environment labels to change inherited truth invisibly.
Decision	Host owns ordered providers, strict typed binding, validation-on-start and safe manifests. Adapters own narrow technical options. Decision-affecting settings become immutable Application policy snapshots with recorded ConfigurationIdentity. Domain/Application reject configuration framework types and raw secrets.
Consequences	More schemas, validators, negative startup tests and deployment evidence are required. Failures occur earlier; secret leakage and silent fallback are reduced; historical replay remains possible.
Rejected	Global IConfiguration access, environment-selected business rules, secrets in files/source, unvalidated string lookup, automatic live reload for all settings, latest-policy replay and raw configuration dumps.

ADR field	ADR-008 record
Review trigger	New key/provider/environment, changed default/range, secret source or rotation, reload behavior, policy field/version, adapter endpoint, fallback, validation or manifest schema.

Risk register for configuration and environments

Risk	Early signal	Blocking control
Silent provider override	Effective value differs from reviewed file or deployment value.	CFG-SRC-001 precedence fixtures and safe provider manifest.
Environment becomes policy	IsDevelopment/IsProduction appears in Domain/Application or decision mapper.	CFG-OWN-001 / CFG-ENV-001 architecture and equivalence tests.
Unknown key ignored	Misspelled value leaves a default active.	Strict binding and CFG-BIND-001 negative fixtures.
Invalid value fails late	First request/worker access throws OptionsValidationException.	ValidateOnStart and CFG-VAL-001 host matrix.
Secret disclosure	Configuration dump, exception or support bundle contains canary.	CFG-SEC-001 canary, scans and value-free manifest.
Reload creates mixed state	Related keys change across different notifications or active clients disagree.	Versioned atomic snapshot or restart-bound classification.
Policy identity reused	Semantic value changes without version/digest change.	Immutable policy artifact, digest vectors and review gate.
Historical policy missing	Replay depends on latest deployment configuration.	Retained policy store and explicit fail-closed replay test.
Test composition cheats	Test environment replaces durable or invariant boundary implicitly.	Explicit Host manifest and cross-environment composition diff.

Official-source validation

The configuration mechanics were checked against current Microsoft documentation on 18 July 2026. The stable points used here are that later configuration providers override earlier providers; environment variables use double underscore for portable hierarchy; typed options separate related settings and support validation; ValidateOnStart moves registered validation to startup; IOptions, IOptionsSnapshot and IOptionsMonitor have different lifetime/reload semantics; environment is fixed for the running process; Secret Manager is development-only and unencrypted; and environment variables may be exposed as plain text.

Official source	Stable point used here	NEXUS-1 boundary
Microsoft Learn - Configuration providers in .NET	Provider order controls override; environment variables override files/secrets; double underscore maps hierarchy.	Provider order is evidenced and deployment overrides remain explicit.
Microsoft Learn - Options pattern in .NET	Typed options, validation, ValidateOnStart and distinct options lifetime/reload interfaces.	Owned adapter sections fail fast; reload requires an additional acceptance contract.
Microsoft Learn - ASP.NET Core runtime environments	Environment is selected at startup, cannot change while running and defaults to Production when unset.	Environment remains Host context and never becomes a verdict input.
Microsoft Learn - Safe storage of app secrets	Do not store secrets in source/config files; Secret Manager is unencrypted development storage; environment variables may be plain text.	Production secrets use a controlled provider/identity and never enter evidence values.
Microsoft Learn - .NET 8 binder compatibility	With ErrorOnUnknownConfiguration, unknown or unconvertible bound values can throw.	Strict binding rejects misspellings and malformed configuration under the pinned baseline.

Microsoft documentation now centers power supported .NET releases while the series baseline

CHAPTER 10 - CODE AUDIT

Configuration becomes a typed and reviewable runtime contract

The Host binds external representation into narrow technical options and immutable policy artifacts. No raw configuration object crosses into Application or Domain.

EVIDENCE STATUS

The listings are compile-oriented net8.0 reference assets. No dotnet executable, secret provider, deployment candidate, or provider-backed startup fixture exists here; CFG evidence therefore remains PENDING.

EXECUTABLE ASSET 10-A - OWNED TECHNICAL SECTIONS

```
public sealed class RootCauseDbOptions
{
    public const string SectionName = "RootCauseDb";

    [Required]
    public string ConnectionString { get; init; } = "";

    [Range(1, 300)]
    public int CommandTimeoutSeconds { get; init; } = 30;

    [Range(0, 10)]
    public int MaxRetryCount { get; init; } = 3;
}

public sealed class PublicFactDispatchOptions
{
    public const string SectionName = "RootCausePublicFacts";

    [Required, MinLength(1)]
    public string Topic { get; init; } = "";

    [Range(1, 500)]
    public int MaxBatchSize { get; init; } = 100;

    [Range(100, 60_000)]
    public int PollIntervalMilliseconds { get; init; } = 1_000;
}
```

EXECUTABLE ASSET 10-B - STRICT BINDING AT THE HOST

```
internal static class RootCauseOptionsRegistration
{
    public static IServiceCollection AddRootCauseOptions(
        this IServiceCollection services,
        IConfiguration configuration)
    {
        services.AddOptions<RootCauseDbOptions>()
            .Bind(configuration.GetRequiredSection(
                RootCauseDbOptions.SectionName),
                binder => binder.ErrorOnUnknownConfiguration = true)
            .ValidateDataAnnotations()
            .ValidateOnStart();

        services.AddOptions<PublicFactDispatchOptions>()
            .Bind(configuration.GetRequiredSection(
                PublicFactDispatchOptions.SectionName),
                binder => binder.ErrorOnUnknownConfiguration = true)
            .ValidateDataAnnotations()
            .ValidateOnStart();

        return services;
    }
}
```

VALIDATION

Cross-field rules fail before the service becomes ready

EXECUTABLE ASSET 10-C - ADAPTER-OWNED VALIDATOR

```
internal sealed class RootCauseDbOptionsValidator
    : IValidateOptions<RootCauseDbOptions>
{
    public ValidateOptionsResult Validate(
        string? name, RootCauseDbOptions options)
    {
        var failures = new List<string>();

        if (!options.ConnectionString.Contains(
            "Encrypt=True", StringComparison.OrdinalIgnoreCase))
            failures.Add("CFG-VAL-DB-001: encryption is required.");

        var retryBudget = options.MaxRetryCount * 5;
        if (retryBudget >= options.CommandTimeoutSeconds)
            failures.Add(
                "CFG-VAL-DB-002: retry budget must be below timeout.");

        return failures.Count == 0
            ? ValidateOptionsResult.Success
            : ValidateOptionsResult.Fail(failures);
    }
}
```

EXECUTABLE ASSET 10-D - STARTUP MATRIX

```
[Theory]
[MemberData(nameof(InvalidConfigurations))]
public async Task CFG_VAL_001_invalid_configuration_stops_startup(
    IReadOnlyDictionary<string, string?> values,
    string expectedRule)
{
    var result = await RootCauseHostFixture.StartAsync(values);

    result.Started.Should().BeFalse();
    result.SafeDiagnostic.Should().Contain(expectedRule);
    result.SafeDiagnostic.Should().NotContain("Password=");
    result.SafeDiagnostic.Should().NotContain("User Id=");
}

public static TheoryData<
    IReadOnlyDictionary<string, string?>, string>
    InvalidConfigurations => new()
{
    { Config.Without("RootCauseDb:ConnectionString"),
      "RootCauseDb" },
    { Config.With("RootCauseDb:CommandTimeoutSeconds", "0"),
      "CommandTimeoutSeconds" },
    { Config.With("RootCauseDb:UnexpectedKey", "value"),
      "UnexpectedKey" },
    { Config.WithUnencryptedConnection(), "CFG-VAL-DB-001" }
};
```

BOUNDARY

Validation proves shape and internal compatibility. Database reachability, schema availability, and permissions belong to Chapter 11 readiness/startup dependency evidence.

PROVIDER PRECEDENCE

The effective source order is inspected, not assumed

EXECUTABLE ASSET 10-E - EXPLICIT PROVIDER COMPOSITION

```
public static WebApplicationBuilder AddApprovedConfiguration(
    this WebApplicationBuilder builder)
{
    builder.Configuration.Sources.Clear();
    builder.Configuration
        .AddJsonFile("appsettings.json",
            optional: false, reloadOnChange: false)
        .AddJsonFile(
            $"appsettings.{builder.Environment.EnvironmentName}.json",
            optional: true, reloadOnChange: false);

    if (builder.Environment.IsDevelopment())
        builder.Configuration.AddUserSecrets<Program>(
            optional: true, reloadOnChange: false);

    builder.Configuration
        .AddEnvironmentVariables(prefix: "NEXUS1_")
        .AddCommandLine(Environment.GetCommandLineArgs()[1..]);

    return builder;
}
```

EXECUTABLE ASSET 10-F - VALUE-FREE SOURCE MANIFEST

```
public sealed record ConfigurationSourceRecord(
    int Precedence,
    string ProviderKind,
    bool CanReload,
    string Purpose);

internal static IReadOnlyList<ConfigurationSourceRecord>
    DescribeSources(ConfigurationManager configuration) =>
    configuration.Sources.Select((source, index) =>
        new ConfigurationSourceRecord(
            index,
            source.GetType().FullName,
            ?? source.GetType().Name,
            CanReload(source),
            ApprovedPurpose.For(source)))
        .ToArray();
// No key values, connection strings, token hints, or secret names.
```

EXECUTABLE ASSET 10-G - DETERMINISTIC OVERRIDE TEST

```
[Fact]
public void CFG_SRC_001_later_approved_provider_wins()
{
    var configuration = ConfigFixture.Build(
        json: ("RootCauseDb:CommandTimeoutSeconds", "30"),
        environment: ("RootCauseDb:CommandTimeoutSeconds", "45"),
        commandLine: ("RootCauseDb:CommandTimeoutSeconds", "60"));

    configuration["RootCauseDb:CommandTimeoutSeconds"]
        .Should().Be("60");
    ConfigManifest.From(configuration).ProviderKinds
        .Should().Equal(ApprovedProviders.InOrder);
}
```


DECISION POLICY

Canonical bytes produce a stable replay identity

EXECUTABLE ASSET 10-H - IMMUTABLE APPLICATION ARTIFACT

```
public sealed record RootCauseDecisionPolicy(
    string PolicyId,
    int Version,
    int MinimumEvidenceCount,
    decimal IssueThreshold,
    IReadOnlyList<string> EvidenceOrder,
    ConfigurationIdentity Identity);

public readonly record struct ConfigurationIdentity(
    string PolicyId, int Version, string Sha256);

internal sealed record RootCauseDecisionPolicyDocument(
    string PolicyId,
    int Version,
    int MinimumEvidenceCount,
    decimal IssueThreshold,
    string[] EvidenceOrder);
```

EXECUTABLE ASSET 10-I - CANONICAL SERIALIZER AND DIGEST

```
internal static class DecisionPolicyAcceptance
{
    private static readonly JsonSerializerOptions Canonical = new()
    {
        PropertyNamingPolicy = JsonNamingPolicy.CamelCase,
        WriteIndented = false
    };

    public static RootCauseDecisionPolicy Accept(
        RootCauseDecisionPolicyDocument document)
    {
        var normalized = document with
        {
            EvidenceOrder = document.EvidenceOrder
                .Distinct(StringComparer.Ordinal)
                .Order(StringComparer.Ordinal)
                .ToArray()
        };

        var bytes = JsonSerializer.SerializeToUtf8Bytes(
            normalized, Canonical);
        var digest = Convert.ToHexString(
            SHA256.HashData(bytes)).ToLowerInvariant();

        var identity = new ConfigurationIdentity(
            normalized.PolicyId, normalized.Version, digest);
        return new RootCauseDecisionPolicy(
            normalized.PolicyId,
            normalized.Version,
            normalized.MinimumEvidenceCount,
            normalized.IssueThreshold,
            normalized.EvidenceOrder,
            identity);
    }
}
```

EXECUTABLE ASSET 10-J - HISTORICAL LOOKUP NEVER FALLS FORWARD

```
public sealed class DecisionPolicyRegistry(
    IReadOnlyDictionary<ConfigurationIdentity,
    RootCauseDecisionPolicy> retained)
{
    public RootCauseDecisionPolicy Require(
        ConfigurationIdentity identity) =>
        retained.TryGetValue(identity, out var policy)
        ? policy
        : throw new ConfigurationIdentityUnavailable(identity);
}
```

ENVIRONMENT BOUNDARY

Different mechanics must preserve the same decision

EXECUTABLE ASSET 10-K - EXPLICIT ENVIRONMENT COMPOSITION

```
internal static void AddEnvironmentAdapters(
    WebApplicationBuilder builder)
{
    switch (builder.Environment.EnvironmentName)
    {
        case Environments.Development:
            builder.Services.AddSingleton<ISecretResolver,
                DevelopmentSecretResolver>();
            builder.Services.AddSingleton<IClockDiagnostics,
                VerboseClockDiagnostics>();
            break;

        case "Production":
            builder.Services.AddSingleton<ISecretResolver,
                WorkloadIdentitySecretResolver>();
            builder.Services.AddSingleton<IClockDiagnostics,
                ProductionClockDiagnostics>();
            break;

        default:
            throw new InvalidOperationException(
                "Unapproved environment identity.");
    }
}
```

EXECUTABLE ASSET 10-L - NORMALIZED COMPOSITION ALLOW-LIST

```
private static readonly string[] AllowedEnvironmentDifferences =
[
    "Nexus.RootCause.Secrets.ISecretResolver",
    "Nexus.RootCause.Diagnostics.IClockDiagnostics",
    "Nexus.RootCause.Infrastructure.IRootCauseDbConnection"
];

[Fact]
public void CFG_ENV_001_only_approved_adapters_may_differ()
{
    var development = CompositionManifest.For("Development");
    var production = CompositionManifest.For("Production");

    development.Diff(production).ChangedServiceTypes
        .Should().BeEquivalentTo(AllowedEnvironmentDifferences);
}
```

EXECUTABLE ASSET 10-M - POLICY EQUIVALENCE ACROSS ENVIRONMENTS

```
[Fact]
public void CFG_ENV_002_same_identity_preserves_domain_decision()
{
    var identity = PolicyFixtures.AcceptedIdentity;
    var input = DecisionFixture.StableInput(identity);

    var development = EnvironmentFixture
        .For("Development").Decide(input);
    var production = EnvironmentFixture
        .For("Production").Decide(input);

    production.Should().BeEquivalentTo(development);
}
```

SECRETS

Capabilities are resolved late and never enter evidence

EXECUTABLE ASSET 10-N - SECRET REFERENCE, NOT SECRET VALUE

```
public readonly record struct SecretReference(string Value)
{
    public override string ToString() => "[secret-reference]";
}

public interface ISecretResolver
{
    ValueTask<SecretLease> ResolveAsync(
        SecretReference reference,
        CancellationToken cancellationTokens);
}

public sealed class SecretLease : IAsyncDisposable
{
    private readonly char[] _value;
    internal SecretLease(char[] value) => _value = value;
    public ReadOnlyMemory<char> Value => _value;

    public ValueTask DisposeAsync()
    {
        Array.Clear(_value);
        return ValueTask.CompletedTask;
    }
}
```

EXECUTABLE ASSET 10-O - CENTRAL REDACTION POLICY

```
internal static class ConfigurationRedaction
{
    private static readonly string[] SensitiveFragments =
        ["password", "token", "secret", "connectionstring",
        "privatekey", "credential"];

    public static object SafeEntry(string key, string? value) =>
        new
        {
            Key = key,
            Classification = SensitiveFragments.Any(
                x => key.Contains(x,
                    StringComparison.OrdinalIgnoreCase))
                ? "SECRET"
                : "NON_SECRET",
            IsPresent = value is not null,
            Value = (string?)null
        };
}
```

EXECUTABLE ASSET 10-P - SYNTHETIC CANARY TEST

```
[Fact]
public async Task CFG_SEC_001_secret_canary_never_leaves_process()
{
    const string canary = "NEXUS1-CANARY-do-not-publish";
    await using var host = await HostFixture.StartWithSecret(canary);

    var surfaces = await host.CaptureLogsMetricsErrorsAndManifest();

    surfaces.Should().AllSatisfy(surface =>
        surface.Should().NotContain(canary));
    host.SecretResolver.ResolvedReferences
        .Should().ContainSingle();
}
```

ROTATION CONTRACT

A new secret lease changes technical capability, not decision identity. The adapter must refresh or restart explicitly, emit value-free telemetry, and retain the previous lease only for a bounded overlap.

RELOAD ACCEPTANCE

A candidate snapshot is validated before atomic publication

EXECUTABLE ASSET 10-Q - SAFE MONITOR WRAPPER

```
public sealed class AcceptedSnapshot<T> : IDisposable
    where T : class
{
    private T _current;
    private readonly IDisposable _subscription;
    private readonly IValidateOptions<T> _validator;

    public AcceptedSnapshot(
        IOptionMonitor<T> monitor,
        IValidateOptions<T> validator,
        IRejectedConfigurationSink rejected)
    {
        _validator = validator;
        _current = RequireValid(monitor.CurrentValue);
        _subscription = monitor.OnChange((candidate, name) =>
        {
            var result = _validator.Validate(name, candidate);
            if (result.Succeeded)
                Interlocked.Exchange(ref _current, candidate);
            else
                rejected.Record(typeof(T).Name, result.Failures);
        });
    }

    public T Current => Volatile.Read(ref _current);

    private T RequireValid(T candidate)
    {
        var result = _validator.Validate(null, candidate);
        if (result.Failed)
            throw new OptionsValidationException(
                typeof(T).Name, typeof(T), result.Failures);
        return candidate;
    }

    public void Dispose() => _subscription.Dispose();
}
```

EXECUTABLE ASSET 10-R - INVALID RELOAD RETAINS ACCEPTED VALUE

```
[Fact]
public void CFG_REL_001_invalid_reload_is_rejected_atomically()
{
    var fixture = ReloadFixture.Valid(initialBatchSize: 100);
    using var snapshot = fixture.CreateAcceptedSnapshot();

    fixture.Publish(batchSize: 0);

    snapshot.Current.MaxBatchSize.Should().Be(100);
    fixture.Rejections.Should().ContainSingle()
        .Which.SecretValues.Should().BeEmpty();
}
```

CLASSIFICATION

Database endpoints and secret-provider identities are restart-bound. Dispatch batch size may use validated reload. Decision policy never uses monitor reload; it is accepted as a new versioned artifact and referenced by ConfigurationIdentity.

CONFIGURATION EVIDENCE

Manifests expose shape and provenance without values

EXECUTABLE ASSET 10-S - ARCHITECTURE GATE

```
[Theory]
[InlineData("Nexus.RootCause.Domain")]
[InlineData("Nexus.RootCause.Application")]
public void CFG_OWN_001_inner_projects_have_no_configuration_api(
    string assemblyName)
{
    ForbiddenApiScan.InAssembly(assemblyName).Reject(
        "Microsoft.Extensions.Configuration.IConfiguration",
        "Microsoft.Extensions.Options.IOptions`1",
        "Microsoft.Extensions.Options.IOptionsSnapshot`1",
        "Microsoft.Extensions.Options.IOptionsMonitor`1",
        "Microsoft.Extensions.Hosting.IHostEnvironment",
        "Microsoft.AspNetCore.Hosting.IWebHostEnvironment");
}
```

EXECUTABLE ASSET 10-T - VALUE-FREE REVIEW RECORD

```
{
  "evidenceId": "CFG-REVIEW-001",
  "chapter": 10,
  "targetFramework": "net8.0",
  "commitSha": null,
  "restoreLockSha256": null,
  "deploymentCandidate": null,
  "providers": [
    { "order": 0, "kind": "Json", "reload": false },
    { "order": 1, "kind": "EnvironmentJson", "reload": false },
    { "order": 2, "kind": "SecretProvider", "reload": false },
    { "order": 3, "kind": "EnvironmentVariables", "reload": false },
    { "order": 4, "kind": "CommandLine", "reload": false }
  ],
  "sections": [
    { "name": "RootCauseDb", "owner": "Infrastructure",
      "change": "restart", "valuesIncluded": false },
    { "name": "RootCausePublicFacts", "owner": "Infrastructure",
      "change": "validated-reload", "valuesIncluded": false },
    { "name": "RootCauseDecisionPolicy", "owner": "Application",
      "change": "new-version", "valuesIncluded": false }
  ],
  "tests": {
    "CFG-OWN-001": "PENDING", "CFG-SRC-001": "PENDING",
    "CFG-BIND-001": "PENDING", "CFG-VAL-001": "PENDING",
    "CFG-ENV-001": "PENDING", "CFG-DEC-001": "PENDING",
    "CFG-SEC-001": "PENDING", "CFG-REL-001": "PENDING"
  },
  "accepted": false
}
```

Chapter 10 code-audit result

The chapter now shows strict owned sections, fail-fast validation, explicit provider order, canonical policy identity, environment-equivalence tests, late secret resolution, atomic reload acceptance, inner-layer API gates, and a value-free evidence record.

CURRENT VERDICT

Implementation ambiguity is reduced, but no execution claim is made. CFG/TPL/CMP/LAY/BUILD evidence remains pending until these assets compile and run in the pinned repository.

source generators, provider behavior and host startup semantics must compile and execute under the repository's pinned SDK and lockfiles.

Technical material still required

The ownership, provider, options, environment, policy-identity, secret and reload contracts are specified, but no generated RootCause service implements them here. The exact configuration schema, defaults, deployment keys, secret provider, validators, canonical policy digest, redacted manifest, startup matrix, environment equivalence fixtures, reload acceptance wrapper and evidence publication path remain to be built.

TECHNICAL MATERIAL STATUS - [OPTIONS/SECRET/ENVIRONMENT CODE + STARTUP/REPLAY EVIDENCE REQUIRED]

Implement strict net8.0 options binding and validators; define approved providers/order and safe manifests; add development and production secret integrations without value leakage; create immutable decision-policy artifacts and canonical ConfigurationIdentity mapping; prohibit configuration framework types in Domain/Application; execute precedence, negative startup, cross-environment, secret-canary, reload-fault and historical-replay tests; and publish CFG-OWN-001 through CFG-REVIEW-001. RootCause remains G0 accepted and G1 specified, not evidenced.

Deliverable	Minimum technical content	Blocking evidence
Configuration schema	Owned sections, defaults, sensitivity, change contract, provider path and deprecation rules.	CFG-SRC-001 / CFG-BIND-001
Options implementation	Adapter classes, strict binding, annotations/custom validators and startup registration.	CFG-VAL-001
Environment boundary	Explicit Host compositions and a difference allow-list with invariant-equivalence fixtures.	CFG-OWN-001 / CFG-ENV-001
Decision policy	Immutable artifacts, canonical serializer/digest, identity persistence and retained replay lookup.	CFG-DEC-001
Secret integration	Development storage, production provider/identity, redaction, canary and rotation contract.	CFG-SEC-001
Reload protocol	Atomic validated snapshots, rejection/withdrawal policy, telemetry and client-refresh behavior.	CFG-REL-001
Safe evidence	Value-free manifest tied to exact build/deployment candidate and signed review.	CFG-MAN-001 / CFG-REVIEW-001

Implementation sequence

1. Commit ADR-008 and inventory every RootCause configuration key with owner, class, sensitivity, default, validation, provider and change contract.
2. Define the approved net8.0 provider order and deployment override policy; create precedence fixtures before adding service-specific values.
3. Implement Infrastructure and Edge options types with strict binding, data-annotation/custom validators and ValidateOnStart registration.
4. Add architecture tests that prohibit IConfiguration, all IOptions forms, host-environment types and secret providers from Domain/Application.
5. Create immutable decision-policy artifacts, canonical field ordering, digest vectors, Application mapping and ConfigurationIdentity persistence.
6. Implement secret-safe development and production provider paths, least-privilege access, synthetic-canary redaction tests a

7. Classify each group as restart-bound, safely reloadable or versioned policy; implement and fault-test the accepted change protocol.
8. Generate a normalized value-free configuration manifest bound to build identity and the deployment candidate.
9. Run missing/unknown/malformed/cross-field startup cases, cross-environment decision equivalence, provider override and historical replay tests.
10. Review exceptions and residual risk; accept configuration evidence only when CFG-OWN-001 through CFG-REVIEW-001 pass against the exact build.

What this chapter has proved

Chapter 10 has placed provider order, binding, validation and environment-specific composition in Host; narrow technical options in Edge/Infrastructure; accepted policy mapping and identity in Application; and only typed decision input in Domain. It has separated secrets from evidence, restart from reload, deployment mechanics from business meaning, and current configuration from historical replay.

It has not compiled the options classes, selected a production secret provider, created decision-policy artifacts, exercised startup or reload behavior, or emitted a safe manifest. RootCause remains G0 accepted; G1 template, composition, internal layers and configuration boundaries are specified; TPL/CMP/LAY/CFG/BUILD evidence remains pending.

CHAPTER CLOSING

A configuration value is safe only when its owner, source, validation, sensitivity, change contract and evidence are explicit - and when changing the environment cannot silently change the truth.

Chapter checkpoint

You are ready for Chapter 11 when every configuration key has an owner and acceptance protocol, every secret is value-free in evidence, and every decision can replay using its recorded ConfigurationIdentity.

Question	Expected answer
Who owns provider order and binding?	Host, with approved provider kinds, precedence tests, strict typed sections and startup validation.
Where may IConfiguration/IOptions appear?	Host registration and narrowly reviewed Edge/Infrastructure adapter bootstrap; never Domain/Application.
What may environments change?	Technical composition, diagnostics, endpoints, resources, secret sources and bounded operational tuning - not truth.
How is decision configuration handled?	As an immutable accepted policy snapshot with canonical ConfigurationIdentity stored beside the decision.
Can secrets appear in evidence?	Only classification and present/missing state; never values, substrings, hashes, lengths or derived material.
Does ValidateOnStart prove dependency health?	No. It proves option creation/validation; Chapter 11 owns readiness and startup dependency policy.
When is live reload allowed?	Only for an explicitly classified group with atomic validation, acceptance/rejection behavior and fault evidence.
What is the current RootCause gate?	G0 accepted; G1 architecture and configuration boundaries specified; executable evidence pending.
What must Chapter 11 add?	Separate liveness, readiness and startup dependency checks with bounded failure and recovery semantics.

NEXT CHAPTER

Chapter 11 defines Liveness, Readiness, and Startup Dependencies. It prevents a healthy process from advertising unsafe service readiness, keeps dependency outages from causing self-inflicted restart storms, and binds every probe to a specific ownership and recovery claim.

PART II - THE SERVICE FOUNDATION

11. Liveness, Readiness, and Startup Dependencies

A probe is not a dashboard ornament. Its answer drives an external controller: wait, route traffic, withdraw traffic, or kill the process. If three different actions share one undifferentiated health boolean, a database outage can become a restart storm and a responsive process can advertise work it cannot safely accept.

CHAPTER ARGUMENT

Startup proves bounded initialization, liveness recommends restart only for an in-process terminal fault, and readiness controls whether this instance may receive new work. Each probe includes only the dependencies needed for its action and exposes no secret or internal topology.

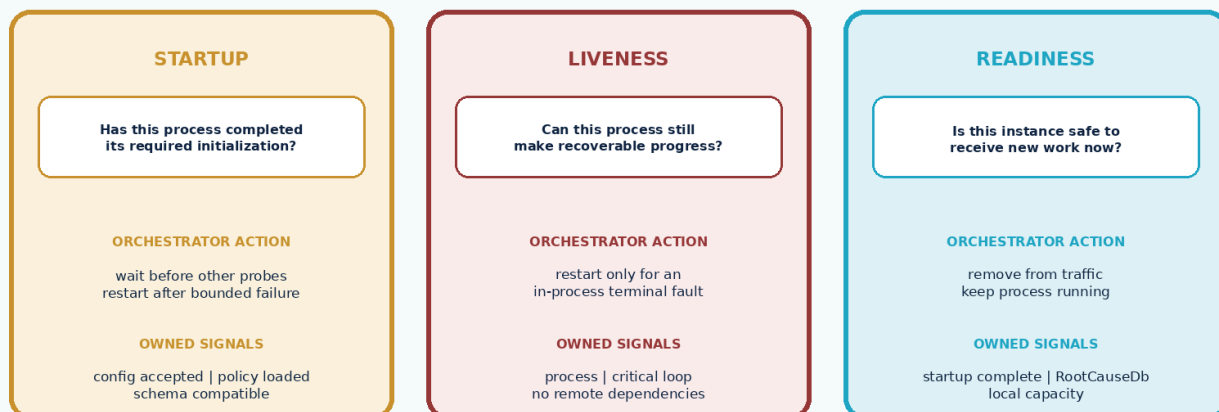
Chapter 10 established that typed configuration is validated before readiness and that ValidateOnStart does not prove dependency health. Chapter 11 now owns the next boundary: which local capabilities must exist before RootCause can serve, which temporary losses should withdraw intake without killing the process, and which self-faults are actually recoverable through restart.

The inherited service rules remain unchanged. RootCause owns verdict truth; it commits aggregate state and a pending public fact locally; Audit, Compliance and Reporting are asynchronous consumers; broker acknowledgement is not part of the local decision; environment and retry count do not choose the verdict. Probe design must preserve those boundaries rather than recreate a distributed availability dependency graph.

No health-check project, startup coordinator, RootCauseDb, broker, deployment manifest or fault harness exists in this manuscript environment. The code and manifests below are reference contracts for net8.0 and require compilation plus orchestrator-level execution before any operational claim is accepted.

Three probes answer three control-plane questions

THREE PROBES MAKE THREE DIFFERENT CONTROL-PLANE CLAIMS

**A FAILED DEPENDENCY IS NOT AUTOMATICALLY A DEAD PROCESS**

Database loss withdraws readiness; broker loss degrades publication; deadlock alone recommends restart

NO AGGREGATE RULE | NO SECRET | NO TRANSITIVE REMOTE HEALTH

Figure 11.1 - Startup, liveness and readiness drive different orchestrator actions and therefore need different inputs.

Probe	Question	Failure action	Must not include
Startup	Has this process completed the required one-time initialization for this build/configuration identity?	Keep liveness/readiness suppressed; restart or fail rollout after the declared startup budget.	Optional remote consumers, broker delivery, reporting availability or unbounded retry.
Liveness	Is the process and every restart-recoverable critical loop still capable of making progress?	Restart only after the configured failure tolerance.	Database, broker, DNS, secret store, Audit, Compliance, Reporting or general downstream reachability.
Readiness	Can this instance safely accept its advertised new work now?	Remove the instance from matching traffic while it keeps running and recovering.	Transitive dependency trees, non-critical analytics, another service's database or cosmetic degradation.
Diagnostics	What is degraded, since when, and under which safe reason code?	Alert, investigate and support recovery; it does not directly define restart/traffic action.	Secrets, connection strings, exception messages, hostnames, SQL, tokens or customer/domain payload.

ACTION-FIRST DESIGN

Select a probe input only after naming the controller action a failure should cause. If restart cannot repair the condition, it does not belong in liveness. If the condition does not make new work unsafe, it does not belong in readiness.

Liveness measures self-recoverability, not the world

A liveness success means the current process should not be restarted. The endpoint itself proves that the HTTP pipeline can respond, but a RootCause host may also contain critical background roles whose silent termination leaves the process responsive but permanently incomplete. Include such a role only when it is process-local, required, observable and restart is the documented recovery.

Signal	Liveness result	Reason
HTTP health pipeline responds	Pass unless a stronger self-fault exists	The process can execute the low-cost probe path.
Critical in-process loop stopped unexpectedly	Fail after an explicit terminal state	Restart recreates the loop and process-owned state.
Critical loop is waiting on broker/database	Pass	External recovery, not process replacement, is the useful action.
RootCauseDb unavailable	Pass	Readiness withdraws intake; restarting every replica adds load and does not repair the database.
Broker unavailable	Pass	Local outbox preserves accepted facts; publisher remains observable and retryable.
Audit/Compliance/Reporting unavailable	Pass	They are independent consumers and must not control RootCause process life.
Internal deadlock or unrecoverable corrupted worker state	Fail when proven	Restart is expected to restore progress; fault evidence must demonstrate this.

PROCESS-LOCAL LIVENESS STATE - [NEW][COMPILE REQUIRED]

```
public sealed class ProcessLivenessCheck(
    CriticalProcessState state) : IHealthCheck
{
    public Task<HealthCheckResult> CheckHealthAsync(
        HealthCheckContext context,
        CancellationToken cancellationToken = default)
    {
        var snapshot = state.Current;

        return Task.FromResult(snapshot.RestartRecommended
            ? HealthCheckResult.Unhealthy(snapshot.SafeReasonCode)
            : HealthCheckResult.Healthy("process-progress"));
    }
}

// External dependency failures never set RestartRecommended.
// Only an explicit process-local terminal fault may set it.
```

NO HEARTBEAT THEATRE

A timestamp that a worker updates on every polling iteration can remain fresh while useful work is impossible, or become stale while the worker correctly waits on a dependency. Model explicit states and recovery actions; do not turn an arbitrary elapsed time into a restart verdict.

Readiness protects the advertised capability

RootCause readiness is not 'all integrations are green.' It asks whether this instance can safely accept the commands routed to it. For a close-case request, the accepted policy snapshot must exist, RootCauseDb must be reachable and schema-compatible, and the local pending-public-fact path must have safe capacity. Broker delivery can be unavailable because the local transaction does not wait for the broker.

Capability/dependency	Blocks RootCause HTTP readiness?	Boundary rationale
Startup initialization state	Yes	The process cannot accept work before configuration, policy and schema acceptance completes.
RootCauseDb connectivity	Yes	The command path must load and commit RootCause-owned state locally.
Schema compatibility	Yes	A reachable but incompatible schema cannot safely execute the build.
Pending-public-fact capacity	Yes at the accepted safety limit	Local outbox saturation removes the guarantee that state and public intent commit together.
Message broker	No while local outbox remains safe	Publication is asynchronous; broker loss is degraded delivery, not failed local intake.
Audit, Compliance or Reporting	No	They observe later and commit independently.
Alarm service	No direct check	Inbound messages/commands carry the contract; RootCause does not query Alarm health to accept HTTP work.
Telemetry exporter	No	Observability loss alerts separately; it cannot rewrite service correctness.

LOCAL BUFFER NUANCE

Broker outage alone does not withdraw command readiness. If the durable pending-fact store reaches a reviewed capacity or age limit that makes further local commits unsafe, the local capacity check withdraws readiness. The signal is owned locally, not a direct broker ping.

Health-check registration keeps actions separate

ASP.NET Core health registrations can carry tags, and each mapped endpoint can select registrations through a predicate. NEXUS-1 uses that mechanism to keep process life, startup completion and intake capability from collapsing into one aggregate result.

TAGGED PROBE REGISTRATION - [NEW][COMPILE REQUIRED]

```
builder.Services.AddSingleton<StartupState>();
builder.Services.AddSingleton<CriticalProcessState>();

builder.Services.AddHealthChecks()
    .AddCheck<ProcessLivenessCheck>(
        "process-progress",
        failureStatus: HealthStatus.Unhealthy,
        tags: ["live"])
    .AddCheck<StartupCompletionCheck>(
        "startup-complete",
        failureStatus: HealthStatus.Unhealthy,
        tags: ["startup", "ready"])
    .AddCheck<RootCauseIntakeReadinessCheck>(
        "rootcause-intake",
        failureStatus: HealthStatus.Unhealthy,
        tags: ["ready"]);

app.MapHealthChecks("/health/startup", ProbeOptions.For("startup"));
app.MapHealthChecks("/health/live", ProbeOptions.For("live"));
app.MapHealthChecks("/health/ready", ProbeOptions.For("ready"));
```

ACTION-ORIENTED ENDPOINT MAPPING - [NEW][COMPILE REQUIRED]

```
internal static HealthCheckOptions For(string tag) => new()
{
    Predicate = registration => registration.Tags.Contains(tag),
    ResultStatusCodes =
    {
        [HealthStatus.Healthy] = StatusCodes.Status200OK,
        [HealthStatus.Degraded] = StatusCodes.Status503ServiceUnavailable,
        [HealthStatus.Unhealthy] = StatusCodes.Status503ServiceUnavailable
    },
    ResponseWriter = SafeProbeResponse.WriteAsync
};

// SafeProbeResponse writes only status and approved reason codes.
// No exception, dependency address, secret, SQL or configuration value.
```

Degraded is intentionally mapped to 503 for these controller endpoints because a routing controller needs a binary action. A separate authenticated diagnostics surface may retain Healthy/Degraded/Unhealthy detail. Never let the same status label mean 'keep routing' in one deployment and 'withdraw' in another without an explicit contract.

Startup is a bounded initialization protocol

Startup begins after Host construction has accepted Chapter 10 configuration. It loads the exact decision-policy artifact, verifies its digest, verifies local schema compatibility and confirms that the local commit path can be created. It may retry a transient direct dependency inside one declared deadline. It never blocks forever, waits for asynchronous consumers, or publishes business facts as a side effect of a probe.

Startup item	Policy	Failure behavior
Typed configuration validation	Required before process startup completes.	Fail process with safe configuration code; do not expose probe values.
Accepted decision policy identity	Required and immutable for the process/deployment candidate.	Fail closed; latest-policy fallback is forbidden.
RootCauseDb schema compatibility	Required for the exact build.	Fail rollout/startup; do not serve with an unknown schema.
Schema migration	Deployment-owned job or explicitly elected migrator. never everv replica by default.	Keep service not ready until compatible; prevent concurrent unreviewed migration.

Startup item	Policy	Failure behavior
RootCauseDb transient reachability	Retry only inside bounded startup budget when policy permits.	Remain live/not ready; exit or let startup-probe policy restart after deadline.
Broker reachability	Deferred; outbox publisher recovers after service starts.	Record degraded delivery; do not block RootCause intake startup.
Audit/Compliance/Reporting reachability	Forbidden startup dependency.	No check or retry; they are asynchronous consumers.
Cache/warm-up	Required only when absence makes advertised work unsafe; otherwise lazy/bounded.	Not ready while required warm-up runs; safe code on deadline.

BOUNDED STARTUP COORDINATOR - REFERENCE [NEW][COMPILE REQUIRED]

```
public sealed class RootCauseStartupInitializer(
    StartupState state,
    IAcceptedDecisionPolicyLoader policies,
    IRootCauseSchemaVerifier schema,
    IHostApplicationLifetime lifetime,
    IOptions<StartupOptions> options) : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        state.Begin();
        using var deadline = CancellationTokenSource
            .CreateLinkedTokenSource(stoppingToken);
        deadline.CancelAfter(options.Value.MaximumDuration);

        try
        {
            var policy = await policies.LoadAcceptedAsync(deadline.Token);
            await schema.VerifyCompatibleAsync(deadline.Token);
            state.Complete(policy.Identity);
        }
        catch (OperationCanceledException) when (!stoppingToken.IsCancellationRequested)
        {
            state.Fail("startup.deadline-exceeded");
            lifetime.StopApplication();
        }
        catch (Exception error)
        {
            state.Fail(StartupFailureCodes.From(error));
            lifetime.StopApplication();
        }
    }
}
```

HOSTED-SERVICE CONSTRAINT

StartAsync remains short because hosted services start sequentially. Long initialization runs asynchronously and reports through StartupState. The exact BackgroundService scheduling, cancellation, shutdown and failure behavior must be compiled and fault-tested under the pinned net8.0 Host.

Schema compatibility is not a connectivity ping

Opening a database connection proves only a narrow transport capability. RootCause also needs an accepted schema/contract version for the exact build. The startup verifier reads a small service-owned compatibility marker or runs a provider-specific compatibility query. It does not apply arbitrary migrations from a health endpoint and does not inspect another service's schema.

Database check	What it proves	What it does not prove
Connection/open	Endpoint, network path, credential and provider can open a session now.	Required tables, indexes, constraints, migration history or transaction behavior.
Schema compatibility marker	This build recognizes the service-owned schema version/identity.	All queries, mappings and provider semantics work.

Database check	What it proves	What it does not prove
Readiness command	A low-cost, read-only service-owned query finishes within budget.	Full write path or business invariant correctness.
Provider component suite	Mappings, constraints, transactions, rollback and pending-fact atomicity work.	Production reachability at probe time.
Migration job evidence	The approved transition ran once and produced the expected schema identity.	Every running replica should migrate on startup.

Probe work is cheap, bounded and cancellation-aware

A controller may call each endpoint frequently and concurrently across many replicas. Every check must complete comfortably inside the orchestrator timeout, honor the supplied cancellation token, avoid retries, avoid mutations and avoid allocating an unbounded scope graph. A timed-out check is a controller failure signal, not permission for work to continue in the background.

Budget rule	Blocking failure	Required evidence
Probe deadline exceeds check deadline	Application check keeps running after controller timeout.	Cancellation/fault test with no orphan operation.
No internal retry	One probe invocation amplifies a dependency incident with repeated calls.	Call-count assertion under timeout/failure.
Low-cost query	Readiness scans tables, publishes messages, writes state or runs migrations.	Query/command allow-list and latency distribution.
Bounded concurrency	Probe storms exhaust pools or serialize behind business traffic indefinitely.	Load test at declared probe frequency/replica count.
No shared DbContext	Singleton health check retains or concurrently uses a scoped/non-thread-safe context.	Scope/lifetime test and descriptor scan.
Explicit cache TTL	Stale healthy result outlives the withdrawal budget.	Clock-controlled cache/failure transition test.

SCOPED, CANCELLATION-AWARE READINESS CHECK - [NEW][COMPILE REQUIRED]

```
internal sealed class RootCauseIntakeReadinessCheck(
    IServiceScopeFactory scopes,
    StartupState startup) : IHealthCheck
{
    public async Task<HealthCheckResult> CheckHealthAsync(
        HealthCheckContext context,
        CancellationToken cancellationToken = default)
    {
        if (!startup.Current.IsComplete)
            return HealthCheckResult.Unhealthy("startup-incomplete");

        await using var scope = scopes.CreateAsyncScope();
        var probe = scope.ServiceProvider
            .GetRequiredService<IRootCauseIntakeProbe>();

        var result = await probe.CheckAsync(cancellationToken);
        return result.IsSafe
            ? HealthCheckResult.Healthy("intake-ready")
            : HealthCheckResult.Unhealthy(result.SafeReasonCode);
    }
}
```

Probe timing is an action budget

Orchestrator fields such as period, timeout, failure threshold and success threshold define when control-plane action occurs. They are not generic defaults to copy. Derive them from startup distributions, withdrawal objectives, dependency latency, graceful termination and the time a bad instance may remain in service. Validate the arithmetic in deployment tests.

ILLUSTRATIVE ORCHESTRATOR CONTRACT - [NEW][DEPLOYMENT TEST REQUIRED]

```
startupProbe:
  httpGet: { path: /health/startup, port: management }
  periodSeconds: 5
  timeoutSeconds: 2
  failureThreshold: 30      # 150-second startup budget

livenessProbe:
  httpGet: { path: /health/live, port: management }
  periodSeconds: 10
  timeoutSeconds: 2
  failureThreshold: 3      # restart after three terminal self-faults

readinessProbe:
  httpGet: { path: /health/ready, port: management }
  periodSeconds: 5
  timeoutSeconds: 2
  failureThreshold: 2      # withdraw after two failures
  successThreshold: 2      # require stable recovery before re-entry
```

NUMBERS ARE NOT DEFAULTS

The example explains arithmetic, not a production recommendation. Measure real startup and check latency, then set thresholds per deployment. A copied threshold without an objective, distribution and fault test is configuration theatre.

State transitions make recovery auditable

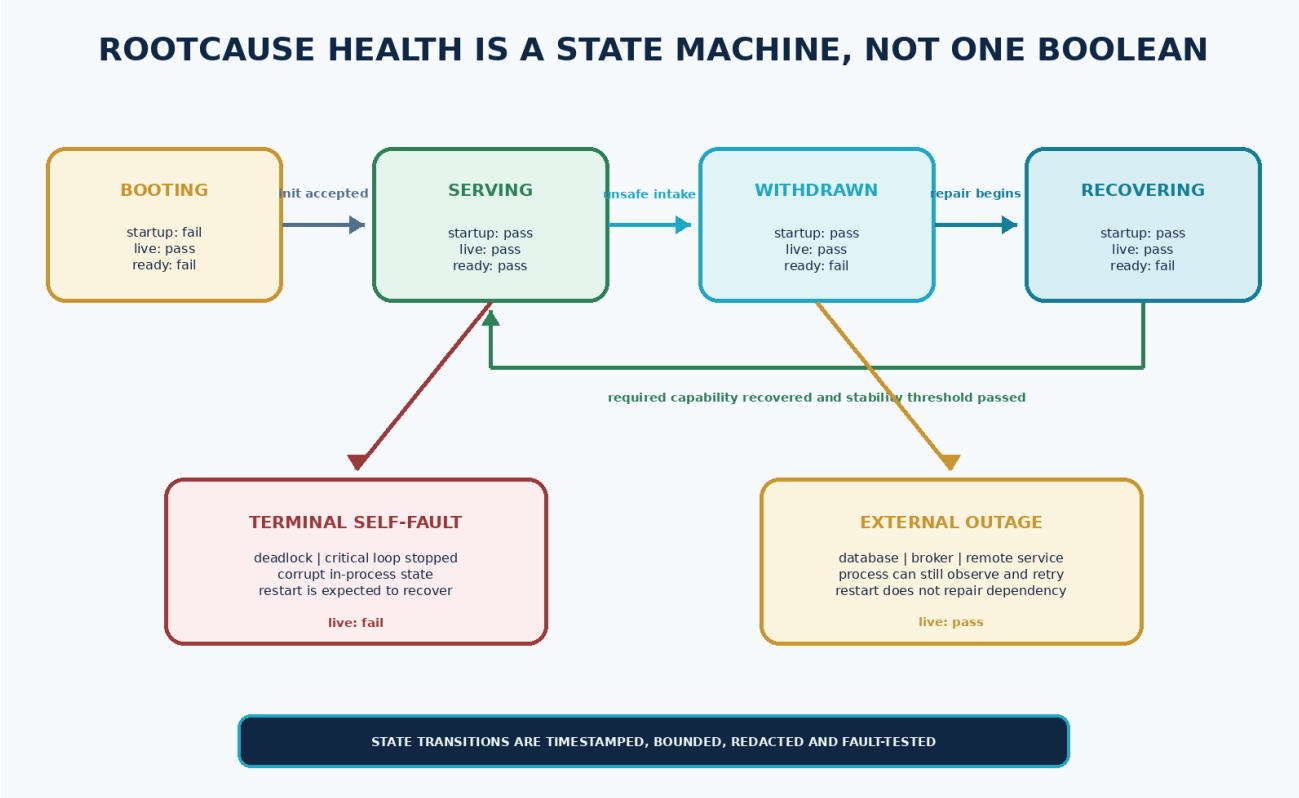


Figure 11.2 - RootCause remains live through recoverable dependency outages and withdraws readiness until intake is safe.

Startup and readiness transitions are explicit state changes with monotonic timestamps, safe reason codes, configuration/build identity and recovery counters. A temporary failure withdraws traffic; a stable success threshold returns it. Liveness changes only when the process-local state says restart is the selected recovery. Health state never changes the aggregate or emits a RootCause business fact.

Scenario	Startup	Liveness	Readiness	Expected action
Booting within budget	Fail/pending	Pass	Fail	Wait; do not route; do not restart while startup

Scenario	Startup	Liveness	Readiness	Expected action
Serving normally	Pass	Pass	Pass	Route traffic.
RootCauseDb unavailable after startup	Pass	Pass	Fail	Withdraw; retain process; recover/recheck.
Broker unavailable, outbox safe	Pass	Pass	Pass	Continue local intake; alert degraded publication.
Broker unavailable, outbox safety limit reached	Pass	Pass	Fail	Withdraw new writes; preserve process and evidence.
Audit/Reporting unavailable	Pass	Pass	Pass	No RootCause probe dependency; consumer alert owns failure.
Policy artifact missing at boot	Fail	Pass until controlled exit	Fail	Fail rollout/startup; no latest fallback.
Critical process loop terminal	Pass	Fail	Fail	Restart after tolerance; prove restart recovery.
Graceful shutdown/drain	Pass	Pass until termination	Fail	Stop new traffic, finish bounded in-flight work, exit.

Dependency health is not transitive

If RootCause checks Alarm, and Alarm checks Telemetry, and Telemetry checks Reporting, one Reporting incident can withdraw the entire system. Each service checks only the direct capability required for the work it advertises. A dependency's own readiness remains its controller's responsibility; RootCause observes call outcomes, circuit state, queue/backlog state and local safety, not the dependency's private health endpoint.

Anti-pattern	Why it spreads failure	Replacement
Call every downstream /health/ready	Builds a transitive availability graph and couples private topology/probe semantics.	Check the direct operation/capability locally needed, with bounded policy.
Broker ping in liveness	Broker incident restarts all publishers and increases reconnect load.	Keep live; expose publisher degradation and local outbox capacity.
Database of another service	Violates data ownership and makes foreign schema availability a local probe input.	Use public contract outcome or local durable state only.
Telemetry exporter blocks readiness	Observability incident removes otherwise correct capacity.	Alert/exporter health separately unless regulation explicitly forbids unobserved work.
One global health aggregator	A cosmetic Degraded status can trigger routing or restart unexpectedly.	Separate controller endpoints from authenticated diagnostics.

Probe endpoints are a security boundary

Controller endpoints often need unauthenticated access from a node agent, but that does not make them public diagnostics APIs. Bind them to a dedicated management port or network path, restrict ingress at the platform layer, and return only minimal status. Host-header restrictions alone are not a complete network control. Rich details require authentication and a separate support contract.

Surface	Allowed output	Forbidden output
/health/startup	HTTP status plus optional stable startup reason code.	Policy payload/digest inputs, schema names, connection details or exception.
/health/live	HTTP status and generic process-progress code.	Thread dump, worker state payload, stack trace, dependency state or secret.
/health/ready	HTTP status and approved local capability rea	Database host, SQL, broker address, backlog

Surface	Allowed output	Forbidden output
Authenticated diagnostics	Redacted component state, age, counts, build/config identity and runbook code.	Credential, token, connection string, raw exception, public/business event or private topology beyond need.
Logs/metrics	Transition reason code, duration, counter and build/deployment identity.	One log per probe request, secret/config value, request payload or unbounded cardinality.

Health evidence is executable

Evidence ID	Blocking rule	Required artifact
HLT-OWN-001	Health/startup code remains Host/Infrastructure; Domain/Application expose no health framework or orchestrator types.	Assembly/public API/forbidden-type scan.
HLT-LIVE-001	Liveness contains only restart-recoverable process-local signals and excludes external dependencies.	Registration manifest and dependency-fault matrix.
HLT-READY-001	Readiness represents the exact advertised RootCause intake capability and local safety limits.	Capability map, endpoint registration and fault tests.
HLT-START-001	Required initialization is bounded, cancellation-aware, identity-bound and excludes asynchronous consumers.	Startup success/failure/deadline matrix.
HLT-DEP-001	No transitive health calls, foreign databases or remote consumer gates exist.	Outbound-call/connection/reference scan.
HLT-BUDGET-001	Every check completes inside controller timeout, performs no retry/mutation and leaves no orphan operation.	Latency/load/cancellation/call-count report.
HLT-STATE-001	Withdrawal, recovery, terminal self-fault and shutdown transitions are deterministic and timestamped.	Clock-controlled state-transition tests.
HLT-EXPOSE-001	Controller responses and telemetry contain only allow-listed codes; endpoints are network-restricted.	Canary disclosure test and deployment policy.
HLT-FAULT-001	Database, broker, consumer, overload, deadlock, timeout and recovery produce the expected controller action.	Orchestrator/component fault-injection report.
HLT-REVIEW-001	Platform, service, data, security and operations owners accept probe actions, budgets and residual risk.	Signed review bound to build/deployment identity.

PROBE EVIDENCE RULE

A 200 response proves only the tested instant. Evidence must force each dependency and self-fault, observe the controller action and recovery, and bind results to the exact build, configuration identity and deployment manifest.

Fault tests verify action, not only status

BOUNDARY FAULT TESTS - [NEW][EXECUTION REQUIRED]

```
[Fact]
public async Task Broker_outage_does_not_fail_liveness_or_safe_intake()
{
    await fixture.Broker.StopAsync();
    fixture.Outbox.SetSafeCapacity();

    Assert.Equal(Healthy, await fixture.Get("/health/live"));
    Assert.Equal(Healthy, await fixture.Get("/health/ready"));
    Assert.Equal("publisher-unavailable", fixture.Diagnostics.ReasonCode);
}

[Fact]
public async Task Database_outage_withdraws_without_restart_signal()
{
    await fixture.RootCauseDb.StopAsync();

    Assert.Equal(Healthy, await fixture.Get("/health/live"));
    Assert.Equal(Unhealthy, await fixture.Get("/health/ready"));
    Assert.False(fixture.ProcessState.Current.RestartRecommended);
}
```

Test layer	Primary proof	Must not substitute
State unit	Startup/readiness/liveness transitions, safe codes, deadlines and recovery hysteresis.	Wall-clock sleeps or real network.
Health component	Tags/predicates, status mapping, cancellation, scopes and safe response body.	Direct invocation only; endpoint pipeline must run.
Dependency component	Real provider timeout, connection loss, schema mismatch and outbox-capacity query.	In-memory database or success-only fake.
Orchestrator integration	Startup suppression, readiness withdrawal/re-entry and liveness restart after thresholds.	HTTP status assertion without controller observation.
Load/fault	Probe cost, concurrency, pool impact, notification storms and recovery under incident load.	Single sequential check on a developer machine.
Security	Management-path restriction and synthetic secret/topology canary absent from every output.	Manual response inspection.

False greens in health design

False green	Why it looks correct	Why it fails
One /health endpoint returns 200	Monitoring sees a simple signal.	The controller action is ambiguous and dependencies may trigger restart storms.
Database ping is in liveness	The process depends on data.	Restart cannot repair the database and amplifies the incident.
Readiness checks every service	The whole workflow appears protected.	Transitive availability coupling withdraws healthy local capacity.
Startup retries forever	Eventually a dependency may recover.	Rollout hangs with no bounded failure, evidence or operator decision.
Probe uses production query	It proves realistic access.	Frequent/mutating/expensive work can damage the dependency being checked.
Health body includes exception	Operators get detail quickly.	Secrets, topology and provider internals cross an unauthenticated boundary.
Status test passes	Endpoint mapping is correct.	No controller threshold, removal, restart or stable recovery has been observed.
Background worker heartbeat is fresh	The loop is running.	Useful progress, external wait and terminal self-fault are not distinguished.

ADR-009 - Separate health probes by controller action

ADR field	ADR-009 record
Context	RootCause must initialize policy/schema safely, accept work only with local commit capability, survive recoverable dependency outages and restart only when replacement can repair a process-local fault.
Decision	Expose distinct startup, liveness and readiness endpoints. Tag/filter ASP.NET Core checks by action. Keep external dependencies out of liveness; make readiness capability-specific and local; bound startup/retries; use minimal redacted management responses; prove controller actions with fault injection.
Consequences	More states, endpoints, budgets, deployment configuration and fault tests are required. Restart storms and transitive health coupling are reduced; degraded publication remains visible without breaking local intake.
Rejected	Single global health boolean, database/broker/remote liveness, transitive health calls, every-replica migration, unbounded startup retry, mutation in probes, raw exception responses and status-only testing.
Review trigger	New entry point, background role, direct dependency, startup task, migration policy, local buffer limit, probe output, timeout/threshold, management path, controller or recovery action.

Risk register for probes and startup

Risk	Early signal	Blocking control
Restart storm	Dependency outage increases container restarts/reconnects.	HLT-LIVE-001 exclusion matrix and broker/database fault test.
False readiness	Requests fail after a ready response due to missing local capability.	HLT-READY-001 capability map and live provider faults.
Permanent withdrawal	Healthy dependency returns but instance never re-enters traffic.	HLT-STATE-001 recovery/hysteresis tests and transition telemetry.
Startup deadlock	Rollout remains pending beyond declared budget.	HLT-START-001 deadline, cancellation and safe exit evidence.
Probe overload	Health traffic consumes pools/CPU or slows business work.	HLT-BUDGET-001 load, timeout and call-count bounds.
Transitive coupling	Remote health endpoint or foreign database appears in check graph.	HLT-DEP-001 outbound/reference/connection scan.
Information disclosure	Probe response/log contains host, SQL, exception or canary.	HLT-EXPOSE-001 allow-list and synthetic-secret test.
Concurrent migration	Multiple replicas apply schema changes during rollout.	Deployment-owned migration gate and schema identity verification.
Controller mismatch	Same app status causes different action across environments.	Normalized deployment manifest and orchestrator integration matrix.

Official-source validation

The mechanics were checked against current Microsoft and Kubernetes documentation on 18 July 2026. The stable points used here are that ASP.NET Core health checks return Healthy, Degraded or Unhealthy; tags and endpoint predicates select subsets; readiness and liveness are separate scenarios; IHealthCheck receives cancellation; hosted-service StartAsync should remain short; a startup probe suppresses liveness/readiness until success; liveness failure can restart a container; and readiness failure removes the Pod from matching service traffic while the process continues.

Official source	Stable point used here	NEXUS-1 boundary
Microsoft Learn - ASP.NET Core health checks	Health middleware, custom IHealthCheck, tags/predicates and separate readiness/liveness endpoints.	Three action-oriented endpoints select only their owned checks.
Microsoft Learn - hosted services	StartAsync is short/sequential; long work belongs in asynchronous execution; shutdown is cancellation-bound.	Startup work reports through state and remains bounded/cancellation-aware.
Microsoft .NET API - IHealthCheck	CheckHealthAsync returns component health and receives a cancellation token.	Every dependency check honors controller budget and leaves no orphan operation.
Kubernetes - liveness, readiness and startup probes	Startup gates later probes; liveness failure restarts; readiness failure removes traffic without stopping the container.	Probe inputs are selected by the external action their failure causes.
Kubernetes - probe thresholds/timeouts	Period, timeout, failure/success thresholds define action timing and startup budget.	Deployment numbers are measured, explicit and fault-tested rather than copied.

Current source pages may describe newer supported ASP.NET Core or Kubernetes releases. The book remains pinned to net8.0 and does not select an orchestrator version here. Exact packages, Host scheduling, endpoint behavior, container platform semantics and manifests must be compiled and executed under the repository and deployment baselines.

Technical material still required

Probe ownership, state, dependency, budget, exposure and controller-action contracts are specified, but RootCause has no implementation or execution evidence here. StartupState, CriticalProcessState, health registrations, intake/schema/outbox checks, management listener, safe response writer, migration gate, deployment probes and fault harness must be built.

TECHNICAL MATERIAL STATUS - [HEALTH/STARTUP/DEPENDENCY CODE + FAULT/RECOVERY EVIDENCE REQUIRED]

Implement net8.0 startup, liveness and readiness endpoints with tagged registrations; create bounded initialization and schema/policy identity checks; keep broker and remote consumers out of liveness/readiness except local outbox safety; enforce cancellation, cost and disclosure limits; execute database, broker, consumer, overload, deadlock, timeout, shutdown and recovery scenarios in the real controller; and publish HLT-OWN-001 through HLT-REVIEW-001. RootCause remains G0 accepted and G1 specified, not evidenced.

Deliverable	Minimum technical content	Blocking evidence
Health state model	Thread-safe startup, withdrawal, recovery, terminal-self-fault and shutdown states with safe codes.	HLT-STATE-001
Probe endpoints	Tagged checks, endpoint predicates, status mapping, management path and safe writer.	HLT-OWN-001 / HLT-EXPOSE-001
Startup protocol	Policy identity, schema compatibility, bounded retry/deadline, cancellation and controlled failure.	HLT-START-001
Readiness adapters	RootCauseDb/local outbox capability checks using scoped, low-cost provider operations.	HLT-READY-001 / HLT-BUDGET-001
Liveness contract	Process-local restart recommendation and critical-role terminal-state detection only.	HLT-LIVE-001 / HLT-DEP-001
Deployment contract	Measured periods/timeouts/thresholds, startup budget, recovery threshold and management routing.	HLT-BUDGET-001 / HLT-FAULT-001
Fault/review bundle	Controller-observed failure/recovery matrix, manifests, exceptions, residual risk and sign-off.	HLT-FAULT-001 / HLT-REVIEW-001

CHAPTER 11 - CODE AUDIT

Three probes become three explicit state machines

Startup completion, restart recommendation, and intake eligibility are stored separately. Each transition carries a stable safe reason code and drives one controller action.

EVIDENCE STATUS

These listings are compile-oriented net8.0 reference assets. No dotnet executable, SQL fixture, container runtime, or orchestrator is available here; health and recovery evidence remains PENDING.

EXECUTABLE ASSET 11-A - IMMUTABLE PROBE SNAPSHOTS

```
public sealed record StartupSnapshot(
    bool Completed,
    string ReasonCode,
    DateTimeOffset ChangedAt,
    string BuildIdentity,
    ConfigurationIdentity Configuration);

public sealed record IntakeSnapshot(
    bool Accepting,
    string ReasonCode,
    DateTimeOffset ChangedAt);

public sealed record ProcessSnapshot(
    bool RestartRecommended,
    string ReasonCode,
    DateTimeOffset ChangedAt);
```

EXECUTABLE ASSET 11-B - ATOMIC STATE PUBLICATION

```
public sealed class HealthState(TimeProvider time)
{
    private StartupSnapshot _startup =
        StartupSnapshotFactory.Pending(time.GetUtcNow());
    private IntakeSnapshot _intake =
        new(false, "startup-pending", time.GetUtcNow());
    private ProcessSnapshot _process =
        new(false, "process-starting", time.GetUtcNow());

    public StartupSnapshot Startup => Volatile.Read(ref _startup);
    public IntakeSnapshot Intake => Volatile.Read(ref _intake);
    public ProcessSnapshot Process => Volatile.Read(ref _process);

    public void CompleteStartup(
        string build, ConfigurationIdentity configuration)
    {
        Interlocked.Exchange(ref _startup,
            new(true, "startup-complete", time.GetUtcNow(),
                build, configuration));
    }

    public void SetIntake(bool accepting, string safeReason) =>
        Interlocked.Exchange(ref _intake,
            new(accepting, safeReason, time.GetUtcNow()));

    public void RecommendRestart(string safeReason) =>
        Interlocked.Exchange(ref _process,
            new(true, safeReason, time.GetUtcNow()));
}
```

BOUNDED STARTUP

Initialization reaches one terminal decision before its deadline

EXECUTABLE ASSET 11-C - STARTUP COORDINATOR

```
public sealed class RootCauseStartupInitializer(
    HealthState state,
    IAcceptedDecisionPolicyLoader policies,
    IRootCauseSchemaVerifier schema,
    IHostApplicationLifetime lifetime,
    IOptions<StartupOptions> options,
    ILogger<RootCauseStartupInitializer> logger)
    : BackgroundService
{
    protected override async Task ExecuteAsync(
        CancellationToken stoppingToken)
    {
        using var deadline = CancellationTokenSource
            .CreateLinkedTokenSource(stoppingToken);
        deadline.CancelAfter(options.Value.Budget);

        try
        {
            var accepted = await policies.RequireAsync(deadline.Token);
            await schema.VerifyCompatibleAsync(deadline.Token);
            state.CompleteStartup(
                BuildIdentity.Current, accepted.Identity);
            state.SetIntake(true, "intake-capable");
        }
        catch (OperationCanceledException)
            when (!stoppingToken.IsCancellationRequested)
        {
            state.SetIntake(false, "startup-deadline");
            logger.LogCritical("Startup failed: {ReasonCode}",
                "startup-deadline");
            lifetime.StopApplication();
        }
        catch (Exception error)
        {
            state.SetIntake(false, "startup-rejected");
            logger.LogCritical(error,
                "Startup failed: {ReasonCode}", "startup-rejected");
            lifetime.StopApplication();
        }
    }
}
```

EXECUTABLE ASSET 11-D - MISSING POLICY NEVER FALLS FORWARD

```
[Fact]
public async Task HLT_START_001_unknown_policy_fails_bounded_startup()
{
    var fixture = StartupFixture.MissingPolicy(
        new ConfigurationIdentity("rootcause-policy", 7, "sha256"));

    await fixture.RunAsync();

    fixture.State.Startup.Completed.Should().BeFalse();
    fixture.State.Intake.Accepting.Should().BeFalse();
    fixture.Lifetime.StopCalls.Should().Be(1);
    fixture.PolicyLoader.LatestFallbackCalls.Should().Be(0);
}
```

STARTUP BOUNDARY

The coordinator checks accepted policy and service-owned schema compatibility. Broker, Audit, Compliance, Reporting, and general downstream reachability are not startup dependencies.

READINESS CAPABILITY

A scoped check answers whether new RootCause work is safe

EXECUTABLE ASSET 11-E - LOW-COST READINESS CHECK

```
public sealed class RootCauseIntakeReadinessCheck(
    IServiceScopeFactory scopes,
    HealthState state,
    IOptions<ReadinessBudgetOptions> options)
    : IHealthCheck
{
    public async Task<HealthCheckResult> CheckHealthAsync(
        HealthCheckContext context,
        CancellationToken cancellationToken = default)
    {
        if (!state.Startup.Completed)
            return HealthCheckResult.Unhealthy("startup-pending");
        if (!state.Intake.Accepting)
            return HealthCheckResult.Unhealthy(
                state.Intake.ReasonCode);

        using var budget = CancellationTokenSource
            .CreateLinkedTokenSource(cancellationToken);
        budget.CancelAfter(options.Value.DatabaseCheck);

        try
        {
            await using var scope = scopes.CreateAsyncScope();
            var capability = scope.ServiceProvider
                .GetRequiredService<IRootCauseCommitCapability>();
            var outbox = scope.ServiceProvider
                .GetRequiredService<ILocalOutboxSafety>();

            await capability.VerifyReadOnlyAsync(budget.Token);
            return await outbox.HasCapacityAsync(budget.Token)
                ? HealthCheckResult.Healthy("intake-capable")
                : HealthCheckResult.Unhealthy("outbox-capacity");
        }
        catch (OperationCanceledException)
        {
            return HealthCheckResult.Unhealthy("db-check-timeout");
        }
        catch (Exception)
        {
            return HealthCheckResult.Unhealthy("db-unavailable");
        }
    }
}
```

EXECUTABLE ASSET 11-F - CHECK CONTRACT TEST

```
[Fact]
public async Task HLT_READY_001_check_is_read_only_and_scoped()
{
    var fixture = ReadinessFixture.Capable();

    var result = await fixture.CheckAsync();

    result.Status.Should().Be(HealthStatus.Healthy);
    fixture.Database.ReadCommands.Should().Be(1);
    fixture.Database.WriteCommands.Should().Be(0);
    fixture.Scopes.Created.Should().Be(1);
    fixture.Scopes.Disposed.Should().Be(1);
    fixture.Database.RetryCalls.Should().Be(0);
}
```

PROCESS LIVENESS

Only a terminal process-local fault recommends restart

EXECUTABLE ASSET 11-G - CRITICAL ROLE STATE

```
public sealed record CriticalRoleSnapshot(
    string Role,
    CriticalRoleStatus Status,
    string ReasonCode,
    DateTimeOffset ChangedAt);

public enum CriticalRoleStatus
{
    Starting, Progressing, WaitingOnDependency,
    Stopping, TerminalFault
}

public sealed class CriticalRoleRegistry
{
    private ImmutableDictionary<string, CriticalRoleSnapshot>
        _roles = ImmutableDictionary<string,
            CriticalRoleSnapshot>.Empty;

    public IReadOnlyCollection<CriticalRoleSnapshot> Current =>
        Volatile.Read(ref _roles).Values;

    public void Publish(CriticalRoleSnapshot snapshot) =>
        ImmutableInterlocked.AddOrUpdate(
            ref _roles, snapshot.Role, snapshot,
            (_, _) => snapshot);
}
```

EXECUTABLE ASSET 11-H - LIVENESS EXCLUDES DEPENDENCIES

```
public sealed class ProcessLivenessCheck(
    CriticalRoleRegistry roles) : IHealthCheck
{
    public Task<HealthCheckResult> CheckHealthAsync(
        HealthCheckContext context,
        CancellationToken cancellationToken = default)
    {
        var terminal = roles.Current
            .FirstOrDefault(x =>
                x.Status == CriticalRoleStatus.TerminalFault);

        var result = terminal is null
            ? HealthCheckResult.Healthy("process-progress")
            : HealthCheckResult.Unhealthy(terminal.ReasonCode);
        return Task.FromResult(result);
    }
}
```

EXECUTABLE ASSET 11-I - DEPENDENCY WAIT REMAINS LIVE

```
[Fact]
public async Task HLT_LIVE_001_broker_wait_does_not_request_restart()
{
    var roles = new CriticalRoleRegistry();
    roles.Publish(new(
        "outbox-dispatcher",
        CriticalRoleStatus.WaitingOnDependency,
        "broker-unavailable",
        TimeProvider.System.GetUtcNow()));

    var result = await new ProcessLivenessCheck(roles)
        .CheckHealthAsync(new HealthCheckContext());

    result.Status.Should().Be(HealthStatus.Healthy);
}
```


PROBE EXPOSURE

Tags, predicates, and a minimal writer preserve action boundaries

EXECUTABLE ASSET 11-J - TAGGED REGISTRATION

```
services.AddSingleton<HealthState>();
services.AddSingleton<CriticalRoleRegistry>();
services.AddHealthChecks()
    .AddCheck<StartupCompletionCheck>(
        "startup-complete",
        failureStatus: HealthStatus.Unhealthy,
        tags: ["startup", "ready"])
    .AddCheck<ProcessLivenessCheck>(
        "process-progress",
        failureStatus: HealthStatus.Unhealthy,
        tags: ["live"])
    .AddCheck<RootCauseIntakeReadinessCheck>(
        "rootcause-intake",
        failureStatus: HealthStatus.Unhealthy,
        tags: ["ready"]);
```

EXECUTABLE ASSET 11-K - THREE ENDPOINTS, THREE SELECTIONS

```
app.MapHealthChecks("/health/startup",
    ProbeOptions.ForTag("startup"));
app.MapHealthChecks("/health/live",
    ProbeOptions.ForTag("live"));
app.MapHealthChecks("/health/ready",
    ProbeOptions.ForTag("ready"));

internal static class ProbeOptions
{
    public static HealthCheckOptions ForTag(string tag) => new()
    {
        Predicate = registration =>
            registration.Tags.Contains(tag),
        ResponseWriter = SafeHealthResponse.WriteAsync,
        ResultStatusCodes =
        {
            [HealthStatus.Healthy] = StatusCodes.Status200OK,
            [HealthStatus.Degraded] =
                StatusCodes.Status503ServiceUnavailable,
            [HealthStatus.Unhealthy] =
                StatusCodes.Status503ServiceUnavailable
        }
    };
}
```

EXECUTABLE ASSET 11-L - VALUE-FREE RESPONSE

```
internal static class SafeHealthResponse
{
    public static Task WriteAsync(
        HttpContext context, HealthReport report)
    {
        context.Response.ContentType = "application/json";
        return context.Response.WriteAsJsonAsync(new
        {
            status = report.Status.ToString(),
            checks = report.Entries.Select(x => new
            {
                name = x.Key,
                status = x.Value.Status.ToString(),
                reason = SafeReasonCodes.Normalize(
                    x.Value.Description)
            })
        });
    }
}
```

BUDGET ENFORCEMENT

Timeout and cancellation leave no orphan operation

EXECUTABLE ASSET 11-M - CANCELLATION TEST

```
[Fact]
public async Task HLT_BUDGET_001_timeout_cancels_dependency_operation()
{
    var fixture = ReadinessFixture.BlockedDatabase(
        budget: TimeSpan.FromMilliseconds(50));

    var result = await fixture.CheckAsync();

    result.Status.Should().Be(HealthStatus.Unhealthy);
    result.Description.Should().Be("db-check-timeout");
    fixture.Database.CancellationObserved.Should().BeTrue();
    fixture.Database.ActiveOperations.Should().Be(0);
    fixture.Database.RetryCalls.Should().Be(0);
}
```

EXECUTABLE ASSET 11-N - CONCURRENT PROBE COST REMAINS BOUNDED

```
[Fact]
public async Task HLT_BUDGET_002_concurrent_checks_use_bounded_work()
{
    var fixture = ReadinessFixture.Capable();

    var checks = Enumerable.Range(0, 20)
        .Select(_ => fixture.CheckAsync());
    var results = await Task.WhenAll(checks);

    results.Should().OnlyContain(
        x => x.Status == HealthStatus.Healthy);
    fixture.Database.MaxConcurrentCommands
        .Should().BeLessThanOrEqualTo(4);
    fixture.Database.WriteCommands.Should().Be(0);
}
```

EXECUTABLE ASSET 11-O - DISCLOSURE TEST

```
[Fact]
public async Task HLT_EXPOSE_001_public_response_is_value_free()
{
    var response = await fixture.Client.GetStringAsync(
        "/health/ready");

    response.Should().Contain("db-unavailable");
    response.Should().NotContain("Server=");
    response.Should().NotContain("Password=");
    response.Should().NotContain("SqlException");
    response.Should().NotContain("RootCauseDb");
}
```

MEASUREMENT RULE

The tests express obligations, not production thresholds. Timeout, period, failure threshold, and recovery threshold are derived from measured distributions in the target deployment.

FAULT ACTIONS

A test proves withdrawal, survival, restart, and recovery

EXECUTABLE ASSET 11-P - DATABASE OUTAGE AND RECOVERY

```
[Fact]
public async Task HLT_FAULT_001_database_loss_withdraws_without_restart()
{
    await using var fixture = await ControllerFixture.StartAsync();
    await fixture.Database.StopAsync();

    await fixture.Controller.WaitUntilWithdrawnAsync();

    (await fixture.GetStatus("/health/live"))
        .Should().Be(HttpStatusCode.OK);
    (await fixture.GetStatus("/health/ready"))
        .Should().Be(HttpStatusCode.ServiceUnavailable);
    fixture.Pod.RestartCount.Should().Be(0);

    await fixture.Database.StartAsync();
    await fixture.Controller.WaitUntilReadyAsync(
        requiredSuccesses: 2);

    fixture.Pod.AcceptsNewTraffic.Should().BeTrue();
}
```

EXECUTABLE ASSET 11-Q - TERMINAL ROLE FAULT RESTARTS

```
[Fact]
public async Task HLT_FAULT_002_terminal_loop_fault_requests_replacement()
{
    await using var fixture = await ControllerFixture.StartAsync();
    fixture.Roles.PublishTerminalFault(
        "inbox-pump", "inbox-loop-terminal");

    await fixture.Controller.WaitForRestartAsync(
        requiredFailures: 3);

    fixture.Pod.RestartCount.Should().Be(1);
    await fixture.Controller.WaitUntilReadyAsync();
    fixture.RestartedInstance.ProcessSnapshot
        .RestartRecommended.Should().BeFalse();
}
```

EXECUTABLE ASSET 11-R - BROKER LOSS RESPECTS OUTBOX SAFETY

```
[Theory]
[InlineData(OutboxCapacity.Safe, HttpStatusCode.OK)]
[InlineData(OutboxCapacity.Exhausted,
    HttpStatusCode.ServiceUnavailable)]
public async Task HLT_FAULT_003_broker_loss_uses_local_safety(
    OutboxCapacity capacity, HttpStatusCode expectedReadiness)
{
    await using var fixture = await ControllerFixture.StartAsync();
    await fixture.Broker.StopAsync();
    fixture.Outbox.SetCapacity(capacity);

    (await fixture.GetStatus("/health/live"))
        .Should().Be(HttpStatusCode.OK);
    (await fixture.GetStatus("/health/ready"))
        .Should().Be(expectedReadiness);
    fixture.Pod.RestartCount.Should().Be(0);
}
```

HEALTH EVIDENCE

Controller behavior is bound to the exact build and manifest

EXECUTABLE ASSET 11-S - ARCHITECTURE BOUNDARY

```
[Theory]
[InlineData("Nexus.RootCause.Domain")]
[InlineData("Nexus.RootCause.Application")]
public void HLT_OWN_001_inner_layers_have_no_health_types(
    string assemblyName)
{
    ForbiddenApiScan.InAssembly(assemblyName).Reject(
        "Microsoft.Extensions.Diagnostics.HealthChecks.IHealthCheck",
        "Microsoft.Extensions.Hosting.IHostedService",
        "Microsoft.AspNetCore.Http.HttpContext",
        "KubernetesClient");
}
```

EXECUTABLE ASSET 11-T - DEPLOYMENT EVIDENCE RECORD

```
{
  "evidenceId": "HLT-REVIEW-001",
  "chapter": 11,
  "targetFramework": "net8.0",
  "commitSha": null,
  "deploymentManifestSha256": null,
  "controllerVersion": null,
  "probes": {
    "startup": { "path": "/health/startup",
      "timeoutSeconds": null, "failureThreshold": null },
    "liveness": { "path": "/health/live",
      "timeoutSeconds": null, "failureThreshold": null },
    "readiness": { "path": "/health/ready",
      "timeoutSeconds": null, "failureThreshold": null,
      "successThreshold": null }
  },
  "tests": {
    "HLT-OWN-001": "PENDING", "HLT-STATE-001": "PENDING",
    "HLT-START-001": "PENDING", "HLT-LIVE-001": "PENDING",
    "HLT-READY-001": "PENDING", "HLT-BUDGET-001": "PENDING",
    "HLT-EXPOSE-001": "PENDING", "HLT-FAULT-001": "PENDING"
  },
  "accepted": false
}
```

Chapter 11 code-audit result

The chapter now exposes atomic health state, bounded initialization, scoped and read-only readiness, process-local liveness, tag-selected endpoints, value-free responses, cancellation and concurrency tests, plus controller-observed fault and recovery scenarios.

CURRENT VERDICT

The code clarifies implementation, but real probe timing and controller action are not claimed. HLT/TPL/CMP/LAY/CFG/BUILD evidence remains pending until the pinned deployment executes it.

Implementation sequence

1. Commit ADR-009 and list every process role, advertised capability, direct dependency, recovery action and controller action.
2. Implement thread-safe StartupState and CriticalProcessState with stable reason codes, timestamps and build/configuration identity.
3. Implement bounded policy/schema initialization; keep StartAsync short and make deadline/cancellation/failure behavior explicit.
4. Implement RootCause intake readiness using scoped, low-cost RootCauseDb and local pending-fact capacity checks.
5. Implement process-local liveness that excludes database, broker, DNS, secret store and every remote consumer.
6. Map separate startup/live/ready endpoints with tags/predicates, binary controller status and a value-free response writer.
7. Define a deployment-owned migration gate and prove schema identity before RootCause becomes ready.
8. Measure check/startup distributions; derive controller timeouts, periods and thresholds; publish the arithmetic in the manifest.
9. Run component and orchestrator fault scenarios for database, broker, outbox capacity, consumer, overload, deadlock, timeout, shutdown and recovery.
10. Scan outputs/dependencies, review exceptions and accept only when HLT-OWN-001 through HLT-REVIEW-001 pass against the exact build/deployment.

What this chapter has proved

Chapter 11 has separated initialization, restart recommendation and traffic eligibility; limited readiness to the RootCause capability being advertised; kept broker and remote consumers outside process life; made local outbox safety explicit; bounded probe cost and startup; and attached every status to a controller action, recovery transition and evidence ID.

It has not compiled the health code, selected the production management network, measured real latency, verified schema/provider checks, configured an orchestrator or executed fault recovery. RootCause remains G0 accepted; G1 template, composition, layers, configuration and health boundaries are specified; TPL/CMP/LAY/CFG/HLT/BUILD evidence remains pending.

CHAPTER CLOSING

A healthy service is not one whose dependencies are all green. It is one whose process knows when restart can help, whose readiness protects the work it advertises, and whose startup reaches a bounded, evidenced decision.

Chapter checkpoint

You are ready for Chapter 12 when every probe has one question, one controller action, one dependency allow-list and one fault test that proves failure and recovery.

Question	Expected answer
What does startup prove?	Required initialization for the exact build/configuration identity completed within a bounded protocol.
What does liveness prove?	The process and restart-recoverable critical roles can still make progress; external dependencies are excluded.
What does readiness prove?	This instance can safely accept the specific RootCause work advertised now.
Does broker loss fail readiness?	Not while the local durable pending-fact path remains within its accepted safety limits.
Does database loss fail liveness?	No. It withdraws readiness; restart does not repair the database.
May probes call remote /health endpoints?	No transitive health graph; check only direct local capability needed for advertised work.
What may a controller response expose?	Binary status and approved reason code only; richer redacted detail belongs on an authenticated surface.
How are timeouts and thresholds chosen?	From measured distributions and explicit startup, withdrawal, restart and recovery objectives.
What is the current RootCause gate?	G0 accepted; G1 architecture through health boundaries specified; executable evidence pending.
What must Chapter 12 add?	A distributed-service testing pyramid that binds pure rules, adapters, contracts and system faults to the gates.

NEXT CHAPTER

Chapter 12 defines The Testing Pyramid for a Distributed Service. It maps each preservation rule and boundary claim to the cheapest trustworthy test, prevents mock-heavy false greens, and builds one evidence chain from Domain invariants to provider and orchestrator faults.

PART II - THE SERVICE FOUNDATION

12. The Testing Pyramid for a Distributed Service

A distributed test suite is trustworthy only when every claim lands on the cheapest test that can actually falsify it. Speed belongs near owned meaning; fidelity belongs where provider, protocol, process or controller semantics begin.

CHAPTER ARGUMENT

Build a wide base of deterministic Domain and Application tests, then add narrow real-boundary evidence for persistence, contracts, HTTP, messaging, deployment actions and recovery. Never let mocks, coverage or an end-to-end happy path impersonate the missing layer.

Chapter 11 ended with one question per probe, one controller action, one dependency allow-list and one fault test. This chapter generalizes that discipline. A test does not become stronger because it starts more containers or crosses more services. It becomes fit for a claim when its system under test, real boundaries, controlled doubles, observations and exclusions match the mechanism being asserted.

The inherited RootCause rules remain unchanged. RootCause owns verdict truth; an accepted decision policy is part of the decision identity; environment and retry count do not choose the verdict; aggregate state and pending public fact commit locally; Audit, Compliance and Reporting observe asynchronously; broker acknowledgement is outside the local decision. The test structure must protect those meanings without rebuilding the monolith as one distributed test fixture.

No generated RootCause projects, pinned test packages, production database selection, broker fixture, container image, deployment manifest or controller exist in this manuscript environment. Every code fragment and command below is therefore a reference contract for net8.0. It requires package selection, compilation, discovery and execution before any TST evidence can pass.

The pyramid maps fidelity, not importance

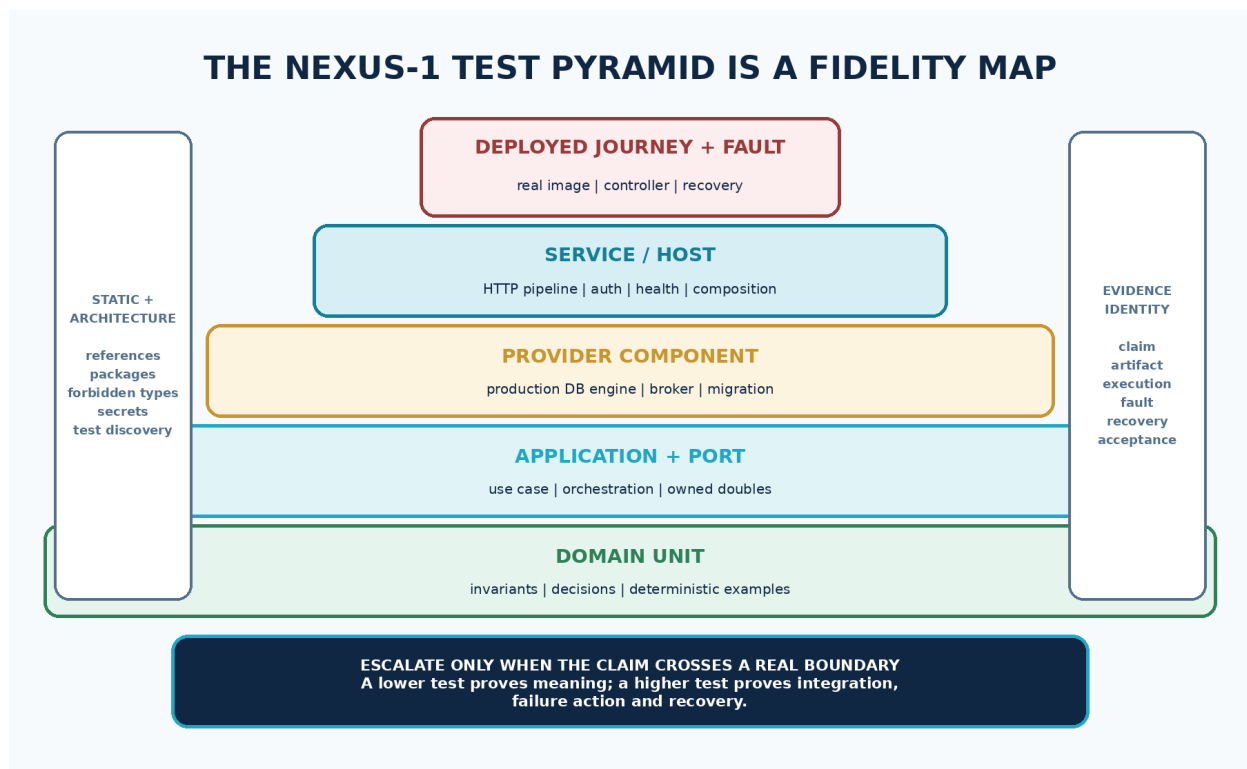


Figure 12.1 - Tests widen around owned meaning and narrow as runtime fidelity, cost, state and diagnostic burden increase

A preservation rule can be important and still belong in a unit test. RC-INV-001 does not need a broker to prove that retry count cannot change a verdict. Conversely, a small database mapping deserves a provider component test if its correctness depends on collation, transaction isolation, generated values, constraints or provider translation. Layer is selected by the falsifier, not by organizational prestige.

Test surface	Primary claim	Must be real	Does not prove
Static / architecture	Source, package, assembly, ownership and evidence structure obey declared rules.	Parsed source/project graph or compiled metadata according to the rule.	Runtime behaviour, provider semantics or recovery.
Domain unit	Invariants and decisions are deterministic for controlled inputs.	Domain code, value objects, policy snapshot and result.	DI, serialization, SQL, HTTP, broker or process lifecycle.
Application / port	Use case orders owned decisions and calls ports under explicit outcomes.	Application code and owned port contracts.	Adapter implementation, provider translation or network behaviour.
Provider component	One adapter works against the selected production-class provider.	Provider engine/protocol, migrations, serializer or broker as claimed.	Complete host, authorization, controller action or cross-service recovery.
Service / host	Composition and the request/response or background pipeline work together.	Real host registrations, middleware, endpoint mapping and selected adapters.	Kestrel/TLS/container/network when TestServer is used.
Deployed fault	The built image and controller perform the expected action and recovery.	Artifact, network, dependencies, probe/controller and fault mechanism.	Every domain permutation or production scale.
Cross-service journey	A sparse critical flow preserves contracts and observable outcome across owners.	Producer/consumer artifacts and public boundaries in scope.	Exhaustive correctness or ownership of private internals.

SELECTION RULE

Name the claim and the mechanism that could violate it. Choose the lowest test surface that executes that mechanism. Add a higher test only for a new boundary, action or recovery claim—not to repeat permutations already proved below.

Every test carries a claim boundary

A folder called IntegrationTests does not define evidence. Each suite needs a manifestable contract: the system under test, the real components, allowed doubles, initial state, action, observations, excluded claims, isolation strategy, timeout and evidence identity. Without that boundary, a green result is easy to overstate and a failure is hard to diagnose.

Test-contract field	Required question	Blocking ambiguity
Claim ID	Which preservation, architecture, health, data, contract or recovery rule is falsified by failure?	A generic test name cannot be traced to a gate.
System under test	Which assembly, adapter, host, image or deployed slice owns the behaviour?	The fixture silently expands into the whole platform.
Real boundary	Which provider/protocol/controller semantics are actually exercised?	A fake is mistaken for the production mechanism.
Controlled doubles	Which owned ports are stubbed, why, and what may not be inferred from them?	Mock interaction is reported as transport success.
State and action	What minimal initial state is created and what single meaningful act occurs?	Shared fixtures and multiple acts hide causality.
Observations	Which returned value, durable state, public fact, status, telemetry or controller action is asserted?	Only an incidental call count or HTTP status is checked.
Isolation / cleanup	How are names, data, clock, concurrency and resources managed and bounded?	Order dependence or cleanup failure contaminates later tests.

Test-contract field	Required question	Blocking ambiguity
Evidence identity	Which commit, SDK, packages, image/provider versions, configuration and result files produced the claim?	A result cannot be repeated or reviewed.

RootCause claims land on different surfaces

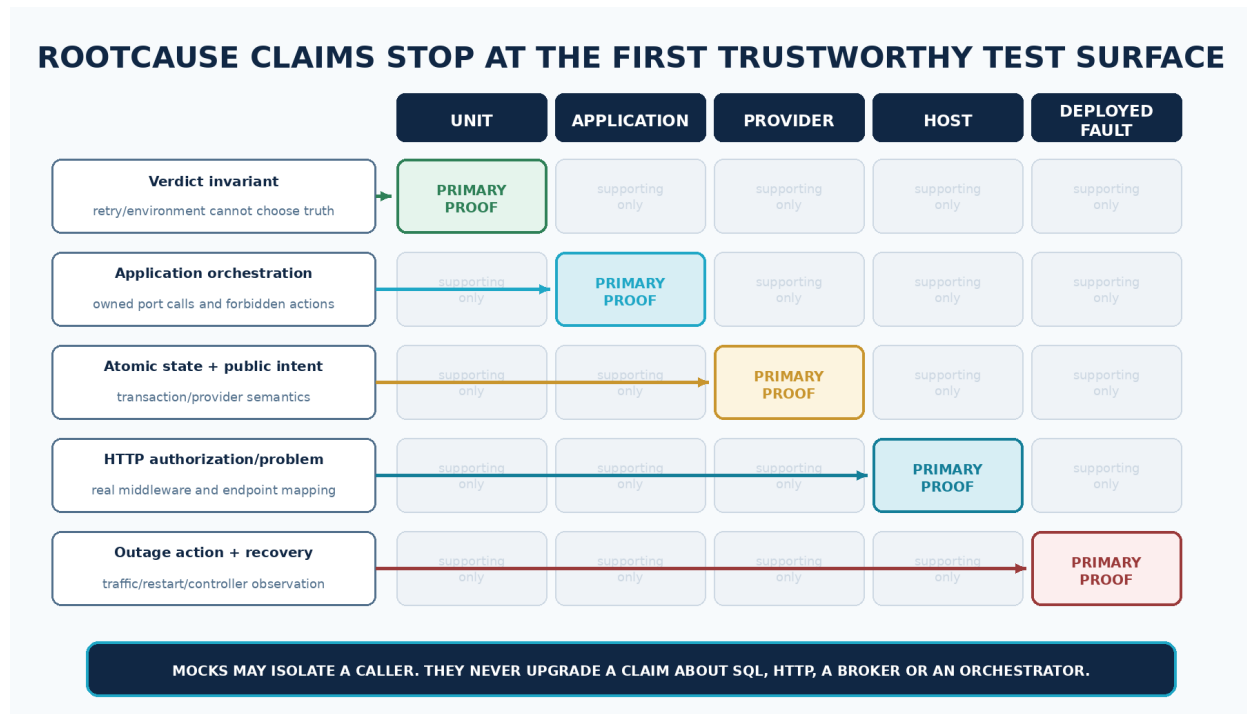


Figure 12.2 - The first trustworthy surface owns the primary proof; higher layers add only the boundaries they really execute.

RootCause claim	Cheapest primary proof	Required higher companion
Decision policy produces the same verdict for the same accepted facts.	Domain unit examples plus generated/property cases with controlled clock and identity.	Serialization fixture only if policy/facts cross a public contract.
Environment and retry count never choose verdict truth.	Negative Domain/Application tests over all admitted environment/retry values.	Configuration scan proving those values do not enter the decision port.
Aggregate state and pending public fact commit atomically.	Component test against the selected production database engine.	Crash/restart and publisher-recovery fault evidence.
Duplicate Alarm fact cannot repeat the decision effect.	Application test for duplicate identity plus real persistence uniqueness/concurrency test.	Broker redelivery/restart scenario when the consumer exists.
Close-case HTTP requires local authorization and stable problem mapping.	Host test through real middleware and endpoint registration.	Deployed identity/network policy test for platform enforcement.
Broker outage does not kill safe local intake.	Component test for local commit/outbox capacity semantics.	Controller-observed deployed broker fault and recovery.
Database outage withdraws readiness but not liveness.	Health state/component tests for local capability.	Deployment test observing traffic removal, retained process and re-entry.
Audit/Compliance/Reporting cannot control RootCause truth.	Architecture/reference scan and application forbidden-call tests.	Cross-service outage test only to prove RootCause remains independent.

Domain tests own preservation meaning

The largest suite belongs beside the Domain because that is where NEXUS-1 can vary inputs cheaply without hiding infrastructure. Tests use the smallest accepted facts, a fixed decision-policy identity, deterministic time and explicit identifiers. They assert returned decisions and state transitions, not private helper calls. A table-driven test

explore algebraic or boundary properties; neither may invent a weaker invariant than the preservation ledger.

PRESERVATION-FOCUSED DOMAIN TEST - [NEW][COMPILE REQUIRED]

```
public sealed class RootCauseDecisionTests
{
    [Theory]
    [InlineData("Development", 0)]
    [InlineData("Production", 0)]
    [InlineData("Production", 7)]
    public void Decide_SameAcceptedFacts_IgnoresEnvironmentAndRetry(
        string environment,
        int retryCount)
    {
        var policy = AcceptedPolicy.Fixed("policy-sha256");
        var facts = RootCauseFacts.MinimalAcceptedCase();

        var result = RootCauseDecision.Decide(
            facts,
            policy,
            DecisionContext.ForTest(environment, retryCount));

        Assert.Equal(ExpectedVerdict.RootCauseConfirmed, result.Verdict);
        Assert.Equal(policy.Identity, result.PolicyIdentity);
    }
}
```

TEST THE PUBLIC MEANING

If a refactor changes private methods but the same owned decision and state transition remain observable, the test should normally survive. A unit suite coupled to implementation shape becomes a tax on safe change and still may miss the invariant.

Domain-test family	Required examples	Forbidden shortcut
Invariant partitions	Minimum/maximum, absent evidence, contradictory evidence, delayed truth, already-closed case.	One representative happy path.
Determinism	Same facts + accepted policy identity -> same decision regardless of environment/retry/host.	System clock, random ID or current configuration read inside the test act.
State transitions	Allowed transition, forbidden transition, idempotent duplicate and terminal-state behaviour.	Setting aggregate internals directly.
Policy identity	Accepted snapshot present, unknown/mismatched identity rejected, no latest-policy fallback.	Mock returning whichever policy makes the assertion pass.
Public-intent mapping	Owned domain outcome maps to one pending public fact without leaking private model.	Serializing the domain object as the integration event.

Application tests isolate orchestration at owned ports

Application tests prove that a use case loads the correct owned state, invokes the Domain decision, checks local authorization, commits the state/public-intent unit and returns a stable result. Stubs provide explicit port outcomes; spies or mocks are used only when the interaction itself is the contract, such as forbidding a second commit after a concurrency failure. The production adapter remains unproved until its component suite runs.

Double	Legitimate NEXUS-1 use	Cannot prove
Dummy	Required argument that is never used in the scenario.	Any behaviour or interaction.
Stub	Returns a controlled owned-port outcome such as case state, authorization decision or policy snapshot.	Database query, network, serialization or retry semantics.

Double	Legitimate NEXUS-1 use	Cannot prove
Fake	Small working alternative when its semantic differences are explicit, such as a deterministic clock.	Parity with an external provider unless separately validated.
Spy	Records an owned-port interaction for an assertion when ordering/count is meaningful.	That the production adapter completes the interaction.
Mock	Defines and verifies a narrow interaction expectation at an owned boundary.	Business outcome, SQL behaviour, HTTP middleware, broker acknowledgement or controller recovery.

MOCK BOUNDARY

Mock the output of an owned port when isolating its caller. Do not mock DbSet query behaviour, HttpClient internals or broker channels and then label the result a provider, protocol or delivery test.

Architecture tests are a second axis

The pyramid describes behavioural fidelity. Static and architecture checks cut through every layer: project references, package ownership, forbidden framework types in Domain/Application, cross-database access, service-internal types in Contracts, remote health clients, unresolved template tokens, secrets and zero-test discovery. They are fast, but they prove structure only. A legal dependency graph can still contain incorrect behaviour.

COMPILED DEPENDENCY RULE - [NEW][PACKAGE + COMPILE REQUIRED]

```
// Reference policy test; exact inspection library is not selected here.
[Fact]
public void Domain_HasNoInfrastructureOrHealthFrameworkReferences()
{
    var graph = CompiledAssemblyGraph.Load(typeof(RootCauseCase).Assembly);

    graph.MustNotReference("Microsoft.EntityFrameworkCore");
    graph.MustNotReference("Microsoft.Extensions.Diagnostics.HealthChecks");
    graph.MustNotReference("RabbitMQ.Client");
    graph.MustNotExposeTypesFrom("Nexus1.ServiceDefaults");
}
```

Provider component tests use the production mechanism

A query, transaction or migration claim belongs against the selected production database engine and provider version. EF Core's in-memory provider is not relational, does not support transactions or raw SQL, and can evaluate queries differently. SQLite can also differ in collation, SQL dialect, provider functions and translation. Either may be useful as an explicitly bounded fake, but neither can satisfy TST-DATA-001 for a different production engine.

Persistence claim	Required real-provider scenario	Primary observation
Schema identity	Apply the accepted migration chain to empty database and upgrade supported prior identity.	Exact migration/schema marker accepted by the build.
Query semantics	Execute case-sensitive/insensitive, null, date, ordering and provider-function boundaries used by RootCause.	Expected result and generated/provider path where material.
Atomic commit	Force failure between state/public-intent operations inside the local transaction.	Both durable or neither durable; no orphan truth/public intent.
Optimistic concurrency	Two actors write the same aggregate version under controlled timing.	One accepted transition; loser receives stable concurrency outcome.
Idempotency/uniqueness	Repeat the same inbound fact identity concurrently and after restart.	One owned effect and durable duplicate record/decision.

Persistence claim	Required real-provider scenario	Primary observation
Rollback/cleanup	Cancel, timeout and provider-fault the operation.	No partial state, leaked transaction, orphan connection or hidden retry.

The component harness may provision ephemeral resources with containers, a local installed engine or an isolated CI service. Tool choice is subordinate to evidence. The image/provider version or digest, migration identity, readiness condition, credentials scope, database name, timeout and cleanup ownership must be recorded. Container running is not provider readiness; use a bounded semantic readiness check.

PRODUCTION-PROVIDER LEASE CONTRACT - [NEW][IMPLEMENTATION REQUIRED]

```
public interface IRootCauseProviderLease : IAsyncDisposable
{
    string RedactedProviderIdentity { get; }
    string DatabaseIdentity { get; }
    Task StartAndWaitUntilUsableAsync(CancellationTokent cancellationToken);
    Task ApplyAcceptedMigrationsAsync(CancellationTokent cancellationToken);
    RootCauseDbContext CreateDbContext();
}

// The concrete fixture selects the production provider/image from the
// repository baseline. It never substitutes EFCore.InMemory for TST-DATA-001.
// Readiness, startup timeout, unique database identity and cleanup are bounded.
```

Messaging component tests stop at the adapter boundary

A broker fixture can prove connection, topology declaration, serialization, headers, publisher confirmation, acknowledgement, negative acknowledgement, redelivery and adapter retry/dead-letter behaviour. It does not prove the RootCause verdict or a downstream consumer's private state. Producer contract fixtures and consumer compatibility tests remain public-boundary evidence; shared compiled DTOs between producer and consumer are not compatibility evidence because both can break together.

Messaging scenario	Fixture action	Required assertion
Publish after local commit	Commit pending fact, start publisher, receive broker confirmation.	Pending record becomes published only after accepted confirmation.
Broker unavailable	Remove connectivity before publish attempts.	Local truth remains; retry is bounded/observable; readiness follows local capacity policy.
Publisher crash window	Terminate after broker accept but before local published marker.	Redelivery/republish cannot create a second business effect downstream.
Consumer duplicate	Deliver identical message identity twice and across restart.	Inbox/idempotency admits one owned effect and records duplicate safely.
Poison contract	Deliver invalid/unsupported schema and retry-exhausted payload.	No business mutation; bounded quarantine/dead-letter evidence and safe diagnostic code.
Ordering boundary	Reorder facts where the contract permits transport reordering.	Consumer policy preserves or rejects meaning explicitly; no accidental arrival-order truth.

Host tests exercise the real ASP.NET Core composition

ASP.NET Core integration tests can create a test web host and TestServer through WebApplicationFactory. NEXUS-1 uses this surface to exercise Program composition, DI registrations, middleware order, local authorization, validation, problem responses, endpoint tags and health mapping. Controlled adapters may be replaced for a host-focused test, but that replacement is declared. TestServer does not prove Kestrel sockets, TLS, container ports, ingress or network policy.

HTTP AUTHORIZATION BOUNDARY - [NEW][COMPILE REQUIRED]

```
public sealed class RootCauseHostTests(
    RootCauseWebFactory factory) : IClassFixture<RootCauseWebFactory>
{
    [Fact]
    public async Task CloseCase_UnauthorizedIdentity_DoesNotLoadOrCommitCase()
    {
        using var client = factory.CreateClient(
            new WebApplicationFactoryClientOptions { AllowAutoRedirect = false });

        using var response = await client.PostAsJsonAsync(
            "/root-cause/cases/case-7/close",
            new { policyIdentity = "policy-sha256" });

        Assert.Equal(HttpStatusCode.Forbidden, response.StatusCode);
        Assert.Equal(0, factory.CaseStore.LoadCount);
        Assert.Equal(0, factory.UnitOfWork.CommitCount);
    }
}
```

Host claim	Host-test proof	Deployment companion
Composition	Required registrations resolve; forbidden adapters are absent; startup state is explicit.	Real image starts with accepted configuration and secrets.
HTTP contract	Route, method, media type, validation and stable problem response.	Ingress/proxy limits, TLS and public routing where applicable.
Authorization	Real authentication/authorization middleware with accepted test identity contract.	Platform identity issuance, audience, key/credential and network enforcement.
Health mapping	Tagged startup/live/ready checks map to intended status and value-free body.	Controller performs wait/route/withdraw/restart actions and recovery.
Background role	Host starts/stops role with cancellation and controlled dependency outcomes.	Process termination, drain, restart policy and durable recovery.

Contract tests belong to the producer and the boundary

A public contract test starts from a producer-owned schema or fixture and proves serialization, required fields, identifiers, version metadata, compatibility policy and prohibited private fields. Consumers run the accepted fixture against their deserializer and handling boundary without referencing the producer's private model. Later Part IV chapters will define the exact envelopes and compatibility windows; this chapter fixes the test responsibility now.

Contract evidence	Producer proves	Consumer proves	False green
Canonical fixture	Current payload conforms to owned schema and stable examples.	Accepted fixture deserializes and maps to a local input.	Both compile against one shared DTO assembly.
Compatibility corpus	Supported prior versions remain readable/published according to policy.	Every claimed supported version has an executable fixture.	Only latest producer and latest consumer are tested together.
Negative corpus	Unknown/invalid field, identifier or version receives defined boundary outcome.	No local state change on rejected payload.	Deserializer exception alone is considered safe handling.
Privacy/ownership scan	No domain internals, secrets or foreign service-owned meaning leak.	Consumer does not depend on producer implementation types.	Snapshot accepted without semantic review.

Deployed tests prove controller action and recovery

The narrow top runs the built artifact with production-class dependencies and the selected controller. It observes from outside the process: startup suppression, traffic eligibility, restart count, durable state, public intent, telemetry, recovery and residual backlog. A fault test is incomplete when it stops after

the first failure status. It must prove the selected action, bounded impact, stable recovery and state reconciliation.

Fault scenario	Expected action	Recovery proof
RootCauseDb outage after startup	Readiness withdraws; liveness passes; process is retained.	Database returns, required success threshold passes, traffic re-enters, no partial commit.
Broker outage with safe local capacity	RootCause intake remains ready/live; publication degradation is visible.	Publisher reconnects, drains pending facts without duplicate downstream effect.
Broker outage to accepted capacity limit	Readiness withdraws before unsafe new commits; liveness passes.	Capacity drains, readiness re-enters under hysteresis, accepted facts publish.
Terminal in-process worker fault	Liveness fails after tolerance; controller replaces process.	New process resumes durable work and no owned state is lost.
Policy identity missing at boot	Startup never succeeds; rollout/process exits within declared budget.	Correct artifact/config starts; no latest-policy fallback occurred.
Graceful shutdown during work	Readiness withdraws, new work stops, bounded in-flight work finishes or cancels safely.	Replacement starts from durable truth; no unbounded drain or orphan operation.

FAULT EVIDENCE RULE

Capture before, fault, controller action, bounded impact, recovery threshold, after-state and reconciliation. A screenshot of Unhealthy or one successful retry does not prove the service recovered safely.

Test data, time and isolation are architecture

Distributed suites become unreliable when they share mutable databases, queues, clocks, ports or identities without an ownership rule. NEXUS-1 prefers minimal explicit seed builders, unique test-run/resource identities, deterministic clocks and IDs below the provider boundary, and lifecycle scopes that match the resource. Cleanup is bounded and observable, but correctness is asserted before cleanup so deletion cannot erase the evidence of a bad state.

Isolation concern	Required control	Detection
Database state	Unique database/schema/tenant identity or proven transactional reset compatible with provider semantics.	Parallel contamination test and retained failed-state diagnostic.
Broker topology	Unique exchange/queue/routing identities or isolated virtual host; exclusive/auto-delete only when semantics permit.	Cross-test delivery canary and post-run resource inventory.
Clock / timeout	Injected monotonic/domain clock for logic; real bounded time only for provider/controller behaviour.	No wall-clock sleeps in unit/application tests; timeout evidence at boundary.
Identifiers	Deterministic IDs for meaning tests; collision-safe run IDs for external resources.	Repeated and parallel run comparison.
Order	Each test creates its own prerequisite state and admits arbitrary runner order.	Randomized/repeated order or isolated invocation produces same result.
Parallelism	Declare collections/resources that serialize only for a real shared constraint.	Parallel lane plus resource ownership report; no blanket disable.
Cleanup	Idempotent bounded disposal with ownership labels and failure reporting.	Leak scan independent from assertions; no deletion of foreign resources.

Pipeline lanes reflect cost and evidence validity

One command cannot prove every gate, but every lane must be discoverable from the authoritative repository and emit machine-readable evidence. The pinned net8.0 baseline uses its selected VSTest-compatible runner until an explicit too

describes newer Microsoft Testing Platform behaviour; those options must not be copied into the net8.0 workflow without a baseline migration.

Lane	Required suites	Trigger / blocker
Pull request	Restore/build, static/architecture, Domain, Application, contract schema, focused host/component smoke.	Every change; zero discovery, failure or missing report blocks merge.
Main branch	Full real-provider component, migrations, HTTP host, contract corpus and deterministic repeat.	Every accepted merge; package/provider identity and isolation failures block candidate.
Deployment candidate	Built image, configuration/secret validation, startup/health, identity, database/broker and fault/recovery matrix.	Every candidate/environment; controller action or reconciliation failure blocks promotion.
Scheduled / risk	Load, soak, replay, upgrade/downgrade, backup/restore, security and wider fault experiments.	Cadence or material change; stale required evidence regresses the gate.
Migration rehearsal	Source/target comparison, one-writer enforcement, cutover/reversal and history retention.	Every migration cohort; unreconciled truth blocks cutover.

EVIDENCE-PRODUCING TEST LANES - [REFERENCE][EXECUTION REQUIRED]

```
# Reference net8.0/VSTest workflow; commands are not execution evidence.
dotnet restore Nexus1.Runtime.sln --locked-mode
dotnet build Nexus1.Runtime.sln -c Release --no-restore

dotnet test tests/RootCause.UnitTests/RootCause.UnitTests.csproj -c Release --no-build --logger
"trx;LogFileName=unit.trx"

dotnet test tests/RootCause.ComponentTests/RootCause.ComponentTests.csproj -c Release --no-build --
logger "trx;LogFileName=component.trx"

# Repository tooling verifies required assemblies, discovery counts,
# result files, failures, skips/exceptions and evidence-manifest identity.
```

ZERO-TEST PROTECTION

Exit code zero is insufficient when a required test assembly was not discovered, a filter selected nothing or a report was not produced. The evidence validator compares expected projects/suites with discovered counts and result identities.

Coverage is a navigation signal, not acceptance

Line or branch coverage can reveal unvisited code, but a percentage cannot prove that assertions protect the right meaning or that provider/fault boundaries were exercised. NEXUS-1 does not set a universal percentage threshold here. Critical pure decision code may add mutation testing after the test runner/tool is pinned; surviving meaningful mutations are review input. Coverage and mutation never replace contract, provider, security or recovery evidence.

Metric / technique	Useful question	Unsafe inference
Line coverage	Which executable lines were never visited by this suite?	Visited lines preserve invariants or contain effective assertions.
Branch coverage	Which decision branches lack an executed scenario?	Boundary values, concurrency and failure semantics are complete.
Mutation testing	Would selected changes to pure logic be detected by assertions?	External adapters, performance, authorization or recovery are proved.
Test count	Did expected suites discover approximately the declared inventory?	More tests mean stronger evidence or no duplicate theatre.
Duration	Did cost or startup materially regress for this lane?	Fast means deterministic or slow means high

Metric / technique	Useful question	Unsafe inference
Flake rate	Which tests fail non-deterministically and need quarantine/remediation evidence?	Retries convert a flaky result into a valid pass.

Test evidence is executable and reviewable

Evidence ID	Blocking rule	Required artifact
TST-OWN-001	Every blocking claim has an owner, SUT, primary surface, observation and explicit exclusions.	Claim-to-test manifest and orphan/duplicate-claim report.
TST-UNIT-001	RootCause invariants, determinism, transitions and policy identity pass without infrastructure.	Unit report plus preservation-rule mapping.
TST-APP-001	Use cases preserve authorization, local commit intent, forbidden actions and port outcomes.	Application report and double inventory.
TST-ARCH-001	Compiled/source/project graph enforces layers, ownership, forbidden dependencies and test locations.	Architecture/static scan with approved exceptions.
TST-DATA-001	Migrations, queries, transactions, concurrency, uniqueness and rollback pass on production provider.	Provider identity, isolated database report and failure artifacts.
TST-MSG-001	Serialization, topology, confirmation, redelivery, poison and recovery preserve adapter contracts.	Broker/contract fixture report and public corpus.
TST-HOST-001	Composition, HTTP/auth/problem mapping, health and hosted roles execute through the real host pipeline.	Host report, endpoint/registration manifest and controlled replacements.
TST-FAULT-001	Controller-observed dependency/process faults cause the accepted action and reconciled recovery.	Fault timeline, manifests, state/telemetry before-during-after.
TST-DET-001	Required suites are isolated, order-independent, bounded and repeatable under declared parallelism.	Repeat/parallel/leak report with run/resource identities.
TST-DISC-001	Every required assembly and suite is discovered; zero tests, missing reports and hidden skips block.	Expected-versus-discovered inventory and machine results.
TST-REVIEW-001	Engineering, architecture, data, security and operations accept scope, freshness, exceptions and residual risk.	Signed/identified review bound to build/deployment identity.

EVIDENCE IDENTITY

Retain the commit, SDK/test runner, packages, image/provider versions, configuration digest, migration/contract identity, seed/fault manifest, timestamps, results and redacted diagnostics. A detached green badge is not a service gate.

False greens in distributed testing

False green	Why it looks convincing	Why it fails
All unit tests pass	Owned logic has broad examples.	No DI, SQL, HTTP, broker, image, controller or recovery boundary ran.
EF InMemory test passes	DbContext queries and SaveChanges execute.	Relational translation, constraints, transaction and provider semantics differ or are absent.
WebApplicationFactory test passes	The full ASP.NET Core pipeline appears to run.	Replaced adapters and TestServer do not prove Kestrel, TLS, container or platform policy.
Producer and consumer share DTO	Both projects compile and deserialize together.	A breaking change can update both simultaneously without compatibility detection.
End-to-end happy path passes	Many real components produced a result.	A narrow path can miss invariants, duplicates, authorization, isolation and recovery.
Fault returns expected 503	Fail	

False green	Why it looks convincing	Why it fails
Retries make a flaky test green	Pipeline eventually succeeds.	The original evidence remains non-deterministic and retry can conceal contamination.
Coverage exceeds target	Most lines were visited.	Assertion quality, provider fidelity and missing failure modes are unknown.
Test command exits zero	Runner reports success.	Required assemblies may be absent, filtered to zero or skipped without policy.

ADR-010 - Test every claim at its first trustworthy surface

ADR field	ADR-010 record
Status	Accepted for the NEXUS-1 reference design; executable repository evidence remains pending.
Context	Distributed suites can become slow monoliths, mock-heavy false greens or end-to-end theatre while ownership and failure claims remain unproved.
Decision	Map every blocking claim to the cheapest trustworthy test; keep Domain/Application tests infrastructure-free; use real providers/protocols for their semantics; reserve deployed tests for artifact/controller/fault/recovery; enforce discovery, isolation and evidence identity.
Consequences	More explicit suites and fixtures are maintained; failures localize better; real-provider cost is paid only where needed; evidence cannot be upgraded by naming or coverage.
Rejected	One IntegrationTests bucket, all-mocked adapters, EF InMemory as production proof, shared DTO compatibility, test-count/coverage gates alone, broad happy-path E2E, retry-to-green.
Review trigger	New provider/protocol/entry point, preservation rule, authorization policy, controller action, test runner, parallelism/resource model or evidence gate.

Risk register for the test foundation

Risk	Early signal	Blocking control
Slow-suite collapse	Developers skip local tests or one failure takes minutes to locate.	Claim mapping, separate lanes, focused provider cases and timing telemetry.
Mock-heavy design	Tests assert call counts while adapters lack real fixtures.	Double inventory plus TST-DATA/MSG/HOST evidence.
Provider drift	Image/package/migration identity differs across developer and CI runs.	Pinned accepted identities and report validation.
Shared-state flakiness	Order/parallel reruns change results or resources leak.	Unique ownership, deterministic time, TST-DET-001 repeat/parallel/leak suite.
False compatibility	Producer/consumer upgrade together and old fixtures disappear.	Producer-owned version corpus, independent consumer runs and retention policy.
Fault without recovery	Pipeline stops after failure status or manual intervention.	Before/during/action/recovery/after/reconciliation template.
Evidence loss	Green UI remains but reports/manifests cannot be retrieved.	Retained machine artifacts, hashes, catalogue links and freshness rules.
Test-only architecture	Production code gains abstractions solely to mock framework/provider internals.	Owned ports follow application boundary; real provider tests cover adapters.
Overclaimed gate	G2/G3 language appears after design-only or host-only tests.	DOD-SCOPE-001 plus TST-OWN/DISC/REVIEW validation.

CHAPTER 12 - CODE AUDIT

Every blocking claim receives one first trustworthy test

A typed claim catalogue prevents one green suite from impersonating another. Each claim names its owner, system under test, primary surface, observation, exclusions, and evidence identity.

EVIDENCE STATUS

These listings are compile-oriented net8.0 reference assets. No dotnet runner, provider, broker, image, or controller executes here; all TST evidence remains PENDING.

EXECUTABLE ASSET 12-A - CLAIM-TO-TEST MODEL

```
public enum TestSurface
{
    Architecture, Domain, Application, Provider,
    Contract, Host, Deployment
}

public sealed record BlockingClaim(
    string ClaimId,
    string Owner,
    string SystemUnderTest,
    TestSurface PrimarySurface,
    string Observation,
    IReadOnlySet<string> Exclusions,
    string EvidenceId);

public static class RootCauseClaims
{
    public static readonly BlockingClaim ClosedCaseIsImmutable =
        new("RC-INV-001", "RootCause.Domain", "RootCauseCase",
            TestSurface.Domain, "DomainRuleViolation/RC-INV-001",
            new HashSet<string> { "SQL", "HTTP", "broker" },
            "TST-UNIT-001");

    public static readonly BlockingClaim StateAndFactAreAtomic =
        new("RC-TX-001", "RootCause.Infrastructure",
            "RootCauseDb transaction", TestSurface.Provider,
            "rollback leaves open case and zero pending facts",
            new HashSet<string> { "EF InMemory", "mock DbSet" },
            "TST-DATA-001");
}
```

EXECUTABLE ASSET 12-B - CATALOGUE VALIDATION

```
[Fact]
public void TST_OWEN_001_every_blocking_claim_has_one_primary_surface()
{
    var claims = ClaimCatalogue.Load();

    claims.Should().OnlyContain(x =>
        !string.IsNullOrEmpty(x.Owner)
        && !string.IsNullOrEmpty(x.SystemUnderTest)
        && !string.IsNullOrEmpty(x.Observation)
        && !string.IsNullOrEmpty(x.EvidenceId));
    claims.GroupBy(x => x.ClaimId)
        .Should().OnlyContain(group => group.Count() == 1);
    claims.Select(x => x.EvidenceId)
        .Should().NotContainNulls();
}
```

DETERMINISTIC FAST SUITES

Fixtures control identity and time without hiding Domain code

EXECUTABLE ASSET 12-C - SHARED DETERMINISTIC FIXTURE

```
public sealed class RootCauseFixture
{
    public static readonly RootCauseCaseId CaseId =
        new(Guid.Parse("11111111-1111-1111-1111-111111111111"));
    public static readonly DateTimeOffset Now =
        DateTimeOffset.Parse("2026-07-18T10:00:00Z");
    public static readonly TimeProvider Time =
        new FrozenTimeProvider(Now);
    public static readonly RootCauseDecisionPolicy Policy =
        PolicyFixtures.Accepted("rootcause-policy", 7, "policy-sha256");

    public RootCauseCase OpenWithEvidence() =>
        RootCauseCase.Open(CaseId,
            EvidenceFixture.Accepted(),
            Policy.Identity);
}
```

EXECUTABLE ASSET 12-D - PRESERVATION MATRIX

```
[Theory]
[InlineData("Development", 0)]
[InlineData("Production", 0)]
[InlineData("Production", 7)]
public void TST_UNIT_001_environment_and_retry_do_not_choose_truth(
    string environment, int retryCount)
{
    var fixture = new RootCauseFixture();
    var input = fixture.OpenWithEvidence().ToDecisionInput();
    var envelope = DecisionRequestEnvelope.ForTest(
        input, environment, retryCount);

    var domainInput = envelope.ToDomainInput();
    var result = new DeterministicVerdictPolicy()
        .Decide(domainInput);

    result.Should().BeEquivalentTo(
        new VerdictDecision.Issue(Verdicts.Known));
    domainInput.GetType().GetProperties().Select(x => x.Name)
        .Should().NotContain(["Environment", "RetryCount"]);
}
```

EXECUTABLE ASSET 12-E - APPLICATION INTERACTION PROOF

```
[Fact]
public async Task TST_APP_001_denied_actor_never_loads_or_commits()
{
    var trace = new CallTrace();
    var fixture = HandlerFixture.Denied(trace);

    var result = await fixture.ExecuteAsync();

    result.Should().BeOfType<CloseCaseOutcome.Denied>();
    trace.Entries.Should().Equal("authorize");
    fixture.Cases.LoadCalls.Should().Be(0);
    fixture.Facts.StageCalls.Should().Be(0);
    fixture.UnitOfWork.CommitCalls.Should().Be(0);
}
```

DOUBLE INVENTORY

The fixed TimeProvider is a fake with explicit semantics. Port spies prove caller interaction only; they do not claim SQL, serializer, HTTP, or broker behavior.

ARCHITECTURE AND DISCOVERY

Compiled boundaries and expected suites fail closed

EXECUTABLE ASSET 12-F - PROJECT AND API GATES

```
[Fact]
public void TST_ARCH_001_domain_has_no_outer_dependencies()
{
    Architecture.For("Nexus.RootCause.Domain")
        .RejectProjectReferences(
            "Nexus.RootCause.Application",
            "Nexus.RootCause.Infrastructure",
            "Nexus.RootCause.Edge",
            "Nexus.RootCause.Host")
        .RejectNamespaces(
            "Microsoft.EntityFrameworkCore",
            "Microsoft.AspNetCore",
            "Microsoft.Extensions.Configuration",
            "Microsoft.Extensions.Options",
            "System.Text.Json")
        .AssertSuccess();
}
```

EXECUTABLE ASSET 12-G - EXPECTED SUITE INVENTORY

```
public sealed record ExpectedTestAssembly(
    string Name, int MinimumDiscoveredTests, TestSurface Surface);

public static readonly ExpectedTestAssembly[] ExpectedSuites =
[
    new("Nexus.RootCause.Domain.Tests", 20, TestSurface.Domain),
    new("Nexus.RootCause.Application.Tests", 15, TestSurface.Application),
    new("Nexus.RootCause.Architecture.Tests", 10, TestSurface.Architecture),
    new("Nexus.RootCause.Provider.Tests", 12, TestSurface.Provider),
    new("Nexus.RootCause.Contract.Tests", 10, TestSurface.Contract),
    new("Nexus.RootCause.Host.Tests", 10, TestSurface.Host),
    new("Nexus.RootCause.Deployment.Tests", 8, TestSurface.Deployment)
];
```

EXECUTABLE ASSET 12-H - ZERO DISCOVERY IS A FAILURE

```
[Fact]
public void TST_DISC_001_results_match_expected_inventory()
{
    var result = TestEvidence.ReadTrx("artifacts/tests");

    foreach (var expected in ExpectedSuites)
    {
        var actual = result.RequireAssembly(expected.Name);
        actual.Discovered.Should()
            .BeGreaterThanOrEqualTo(expected.MinimumDiscoveredTests);
        actual.Executed.Should().Be(actual.Discovered);
        actual.Skipped.Should().Be(0);
        actual.Failed.Should().Be(0);
    }
}
```

PRODUCTION PROVIDER

A bounded lease proves the selected database semantics

EXECUTABLE ASSET 12-I - ISOLATED PROVIDER LEASE

```
public interface IRootCauseProviderLease : IAsyncDisposable
{
    string RedactedProviderIdentity { get; }
    string DatabaseIdentity { get; }
    Task StartAndWaitUntilUsableAsync(
        CancellationToken cancellationToken);
    Task ApplyMigrationsAsync(CancellationToken cancellationToken);
    RootCauseDbContext CreateDbContext();
}

public sealed class RootCauseSqlFixture : IAsyncLifetime
{
    private IRootCauseProviderLease _lease = null!;
    public async Task InitializeAsync()
    {
        _lease = ProviderLeaseFactory.Create(
            database: $"rootcause_{Guid.NewGuid():N}");
        using var budget = new CancellationTokenSource(
            TimeSpan.FromSeconds(60));
        await _lease.StartAndWaitUntilUsableAsync(budget.Token);
        await _lease.ApplyMigrationsAsync(budget.Token);
    }
    public async Task DisposeAsync() => await _lease.DisposeAsync();
    public RootCauseDbContext CreateDbContext() =>
        _lease.CreateDbContext();
}
```

EXECUTABLE ASSET 12-J - PROVIDER ROLLBACK PROOF

```
[Fact]
public async Task TST_DATA_001_fact_failure_rolls_back_aggregate()
{
    await using var fixture = await RootCauseSqlFixture.StartAsync();
    await fixture.SeedOpenCaseAsync(Cases.A, Evidence.Known);
    fixture.FailPendingFactInsert();

    var act = () => fixture.ExecuteCloseAsync(Cases.A);

    await act.Should().ThrowAsync<DbUpdateException>();
    await using var verification = fixture.CreateDbContext();
    var aggregate = await verification.RootCauseCases
        .AsNoTracking().SingleAsync(x => x.Id == Cases.A);
    aggregate.Status.Should().Be(RootCauseCaseStatus.Open);
    (await verification.PendingPublicFacts.CountAsync())
        .Should().Be(0);
}
```

EXECUTABLE ASSET 12-K - REAL CONCURRENCY BOUNDARY

```
[Fact]
public async Task TST_DATA_002_two_writers_yield_one_verdict()
{
    await using var fixture = await RootCauseSqlFixture.StartAsync();
    await fixture.SeedOpenCaseAsync(Cases.A, Evidence.Known);

    var results = await Task.WhenAll(
        fixture.ExecuteCloseInNewScopeAsync(Cases.A),
        fixture.ExecuteCloseInNewScopeAsync(Cases.A));

    results.Count(x => x is CloseCaseOutcome.Closed).Should().Be(1);
    results.Count(x => x is CloseCaseOutcome.Conflict).Should().Be(1);
    (await fixture.CountPendingVerdictFactsAsync()).Should().Be(1);
}
```

PROVIDER IDENTITY

The evidence records provider package, engine image/digest, migration identity, database identity, readiness condition, timeout, and cleanup result. Credentials never enter evidence.

CONTRACTS AND MESSAGING

A corpus and real adapter test protect public meaning

EXECUTABLE ASSET 12-L - CONTRACT CORPUS TEST

```
[Theory]
[MemberData(nameof(ContractCorpus.RootCauseVerdictIssuedV1))]
public void TST_CONTRACT_001_v1_payload_matches_public_schema(
    string fixtureName, string json)
{
    var schema = ContractSchemas.Load(
        "RootCauseVerdictIssued.v1.schema.json");

    schema.Validate(json).Should().BeEmpty(fixtureName);
    var contract = JsonSerializer.Deserialize<
        RootCauseVerdictIssuedV1>(json, ContractJson.Strict);
    contract.Should().NotBeNull();
    JsonSerializer.Serialize(contract, ContractJson.Strict)
        .Should().BeJsonEquivalentTo(json);
}
```

EXECUTABLE ASSET 12-M - FORBIDDEN INTERNAL FIELDS

```
[Fact]
public void TST_CONTRACT_002_public_event_excludes_internal_models()
{
    var properties = typeof(RootCauseVerdictIssuedV1)
        .GetProperties().Select(x => x.Name);

    properties.Should().NotContain([
        "Aggregate", "DomainEvents", "EvidenceItems",
        "RetryCount", "BrokerHeaders", "DbVersion"
    ]);
}
```

EXECUTABLE ASSET 12-N - BROKER REDELIVERY IS IDEMPOTENT

```
[Fact]
public async Task TST_MSG_001_duplicate_delivery_has_one_owned_effect()
{
    await using var fixture = await RabbitFixture.StartAsync();
    var message = MessageFixture.AlarmFloodDetectedV1(
        messageId: "message-fixed-001");

    await fixture.PublishAsync(message);
    await fixture.PublishAsync(message);
    await fixture.WaitUntilConsumedAsync(message.MessageId);

    fixture.Inbox.Accepted(message.MessageId).Should().Be(1);
    fixture.RootCauseCases.Created.Should().Be(1);
    fixture.Acknowledgements.Should().Be(2);
}
```

BOUNDARY

The schema corpus proves compatibility; the broker fixture proves the selected adapter's redelivery behavior. Neither substitutes for the consumer's transaction and inbox assertions.

PRODUCTION-PATH HOST

HTTP tests exercise composition, middleware, and authorization

EXECUTABLE ASSET 12-O - PRODUCTION BUILDER FIXTURE

```
public sealed class RootCauseWebFactory
    : WebApplicationFactory<Program>
{
    protected override void ConfigureWebHost(IWebHostBuilder builder)
    {
        builder.UseEnvironment("Chapter12Test");
        builder.ConfigureServices(services =>
        {
            services.RemoveAll<IRootCauseDbConnection>();
            services.AddSingleton<IRootCauseDbConnection>(
                _ => ProviderFixture.Connection);
            services.RemoveAll<IExternalPublisher>();
            services.AddSingleton<IExternalPublisher,
                ControlledPublisher>();
        });
    }
}
```

EXECUTABLE ASSET 12-P - UNAUTHORIZED REQUEST HAS NO SIDE EFFECT

```
[Fact]
public async Task TST_HOST_001_unauthorized_actor_never_loads_case()
{
    using var factory = new RootCauseWebFactory();
    using var client = factory.CreateClient(
        new WebApplicationFactoryClientOptions
        {
            AllowAutoRedirect = false
        });

    using var response = await client.PostAsJsonAsync(
        "/api/root-causes/cases/case-7/close",
        new CloseRootCauseCaseRequest("case-7"));

    response.StatusCode.Should().Be(HttpStatusCode.Unauthorized);
    factory.Services.GetRequiredService<CaseStoreProbe>()
        .LoadCalls.Should().Be(0);
}
```

EXECUTABLE ASSET 12-Q - PROBLEM CONTRACT AND CORRELATION

```
[Fact]
public async Task TST_HOST_002_conflict_is_safe_problem_details()
{
    using var client = AuthenticatedClient.ForClosedCase();

    var response = await client.CloseCaseAsync(Cases.Closed);
    var problem = await response.Content
        .ReadFromJsonAsync<ProblemDetails>();

    response.StatusCode.Should().Be(HttpStatusCode.Conflict);
    problem!.Type.Should().Be(ProblemTypes.RootCauseConflict);
    problem.Extensions.Should().ContainKey("correlationId");
    problem.Detail.Should().NotContain("SqlException");
    problem.Detail.Should().NotContain("ConnectionString");
}
```

DEPLOYMENT FAULT LABORATORY

The controller action and recovery are the assertions

EXECUTABLE ASSET 12-R - OUTAGE SCENARIO CONTRACT

```
public sealed record FaultScenario(
    string Id,
    Func<DeploymentFixture, Task> Inject,
    ControllerAction ExpectedAction,
    Func<DeploymentFixture, Task> Recover,
    Func<DeploymentFixture, Task> Reconcile);

public enum ControllerAction
{
    KeepServing, WithdrawTraffic, RestartInstance,
    FailStartup
}
```

EXECUTABLE ASSET 12-S - DATABASE OUTAGE

```
[Fact]
public async Task TST_FAULT_001_database_outage_withdraws_then_recovers()
{
    await using var fixture = await DeploymentFixture.StartCandidateAsync();
    await fixture.Database.StopAsync();

    await fixture.Controller.WaitForAsync(
        ControllerAction.WithdrawTraffic);

    fixture.Instance.RestartCount.Should().Be(0);
    fixture.Instance.Liveness.Should().BeHealthy();
    fixture.Instance.Readiness.Should().BeUnhealthy();

    await fixture.Database.StartAsync();
    await fixture.Controller.WaitForStableReadinessAsync(
        requiredSuccesses: 2);
    await fixture.ReconcileOwnedStateAsync();

    fixture.Instance.AcceptsTraffic.Should().BeTrue();
    fixture.Reconciliation.PartialCommits.Should().Be(0);
}
```

EXECUTABLE ASSET 12-T - RESTART RECOVERY

```
[Fact]
public async Task TST_FAULT_002_terminal_loop_fault_resumes_durable_work()
{
    await using var fixture = await DeploymentFixture.StartCandidateAsync();
    var durableWork = await fixture.SeedPendingInboxWorkAsync();

    await fixture.InjectTerminalInboxFaultAsync();
    await fixture.Controller.WaitForAsync(
        ControllerAction.RestartInstance);
    await fixture.WaitUntilProcessedAsync(durableWork);

    fixture.Instance.RestartCount.Should().Be(1);
    fixture.Inbox.OwnedEffects(durableWork).Should().Be(1);
    fixture.Reconciliation.LostWork.Should().BeEmpty();
}
```


EVIDENCE VALIDATION

A green exit code is rejected when identity or discovery is missing

EXECUTABLE ASSET 12-U - MACHINE EVIDENCE GATE

```
public sealed class TestEvidenceGate
{
    public EvidenceVerdict Validate(
        BuildIdentity build,
        IReadOnlyList<TestRun> runs,
        ClaimCatalogue claims)
    {
        var failures = new List<string>();
        failures.AddRange(DiscoveryRules.Validate(runs, ExpectedSuites));
        failures.AddRange(IdentityRules.RequireSameBuild(runs, build));
        failures.AddRange(ClaimRules.RequireFreshEvidence(claims, runs));
        failures.AddRange(ResultRules.RejectRetriesSkipsAndFlakes(runs));
        failures.AddRange(ResourceRules.RequireCleanup(runs));

        return failures.Count == 0
            ? EvidenceVerdict.Accepted(build)
            : EvidenceVerdict.Rejected(build, failures);
    }
}
```

EXECUTABLE ASSET 12-V - REVIEW RECORD

```
{
  "evidenceId": "TST-REVIEW-001",
  "chapter": 12,
  "targetFramework": "net8.0",
  "commitSha": null,
  "restoreLockSha256": null,
  "runner": null,
  "providerIdentity": null,
  "brokerIdentity": null,
  "imageDigest": null,
  "controllerVersion": null,
  "tests": {
    "TST-OWN-001": "PENDING", "TST-UNIT-001": "PENDING",
    "TST-APP-001": "PENDING", "TST-ARCH-001": "PENDING",
    "TST-DATA-001": "PENDING", "TST-MSG-001": "PENDING",
    "TST-HOST-001": "PENDING", "TST-FAULT-001": "PENDING",
    "TST-DET-001": "PENDING", "TST-DISC-001": "PENDING"
  },
  "discovered": null,
  "executed": null,
  "skipped": null,
  "retries": null,
  "cleanupAccepted": false,
  "accepted": false
}
```

Chapter 12 code-audit result

The chapter now shows an executable claim catalogue, deterministic Domain/Application suites, compiled architecture and discovery gates, real provider and broker fixtures, contract corpus, production-path host tests, deployment fault recovery, and a fail-closed evidence validator.

CURRENT VERDICT

The test design is explicit, but no green result is claimed. TST/TPL/CMP/LAY/CFG/HLT/BUILD evidence remains pending until the pinned repository and deployment execute these assets.

Official-source validation

The mechanics were checked against current Microsoft, EF Core, Docker and Testcontainers for .NET documentation on 18 July 2026. Stable points used here are that unit tests should avoid infrastructure; ASP.NET Core integration tests can use a test host/TestServer and WebApplicationFactory; real providers are required for exact database behaviour; EF Core InMemory is discouraged as a database fake; container running is not equivalent to readiness; and test execution/reporting depends on the pinned runner and SDK.

Official source	Stable point used here	NEXUS-1 boundary
Microsoft Learn - unit-test best practices	Keep unit tests isolated from infrastructure; focus readable tests on behaviour.	Domain/Application projects contain no provider dependencies.
Microsoft Learn - ASP.NET Core integration tests	WebApplicationFactory creates a TestServer/test host; broader tests are costlier and should target important infrastructure paths.	Host tests prove composition/pipeline, not container/network/controller semantics.
Microsoft Learn - EF Core testing strategy	Provider doubles can differ; test against the production database for exact behaviour; avoid InMemory for provider claims.	TST-DATA-001 requires the declared production provider identity.
Microsoft Learn - dotnet test	Runner behaviour/options and result reporting depend on the SDK/test platform; pre-.NET 10 uses VSTest.	Book remains net8.0; runner/package changes require explicit baseline migration.
Testcontainers for .NET - wait strategies	Running/listening may not mean usable; waits can be semantic, bounded and cancellation-aware.	Fixtures record a provider-specific usable condition and timeout.
Docker Docs - Compose startup order	A running dependency is not necessarily ready; service_healthy depends on an explicit healthcheck.	Compose ordering may prepare a lab but does not replace RootCause fault/action assertions.

Current source pages may describe newer supported .NET, ASP.NET Core, EF Core, test-runner, container or Compose releases. This volume remains pinned to net8.0 and selects no test framework/package/provider/orchestrator version beyond the existing baseline. Exact APIs, packages, images, runner options and manifests must compile and execute in the repository before acceptance.

Technical material still required

The claim map, suite boundaries, provider policy, fault matrix, evidence identities and pipeline lanes are specified, but no RootCause test project or external resource has executed here. The reference code has not compiled; no test was discovered; no real database, broker, host, image or controller produced evidence.

TECHNICAL MATERIAL STATUS - [TEST PROJECTS + REAL PROVIDER/FAULT EXECUTION EVIDENCE REQUIRED]

Implement the net8.0 Domain, Application, Architecture, Component, Contract, Host and Deployment test projects; pin the test SDK/framework and every provider/image; build real database and broker leases; create deterministic data/isolation utilities; execute authorization, migration, transaction, concurrency, duplicate, outage, shutdown and recovery scenarios; validate discovery/results/evidence identity; and publish TST-OWN-001 through TST-REVIEW-001. RootCause remains G0 accepted and G1 specified, not evidenced.

Deliverable	Minimum technical content	Blocking evidence
Claim/test manifest	Every preservation and boundary rule maps to one primary surface, owner, observation and exclusions.	TST-OWN-001
Fast suites	Domain/Application deterministic fixtures, doubles inventory, architecture/static checks.	TST-UNIT-001 / TST-APP-001 / TST-ARCH-001
Provider suites	Production database migrations/transactions/concurrency plus broker/serialization/delivery fixtures.	TST-DATA-001 / TST-MSG-001
Host suite	Real composition, middleware, authorization, HTTP, lifecycle, etc.	TST-MSG-001

Deliverable	Minimum technical content	Blocking evidence
Fault laboratory	Built artifact/controller scenarios with action, recovery, reconciliation and telemetry.	TST-FAULT-001
Deterministic harness	Unique resources, controlled time/IDs, parallel/repeat/leak/cleanup evidence.	TST-DET-001
Evidence validator	Expected discovery, machine results, identities, retained diagnostics, freshness and review.	TST-DISC-001 / TST-REVIEW-001

Implementation sequence

1. Commit ADR-010 and create the claim-to-test manifest from every inherited preservation, layer, configuration and health rule.
2. Pin the net8.0 test SDK/framework, runner configuration, package locks, expected test projects and machine-result schema.
3. Build deterministic RootCause Domain/Application fixtures with accepted policy, clock, identifiers, minimal facts and owned port doubles.
4. Implement architecture/static checks for references, packages, ownership, forbidden infrastructure/remote-health types, secrets and unresolved tokens.
5. Select the production database/broker baselines and implement isolated, bounded provider leases with semantic readiness and cleanup ownership.
6. Execute migrations, query, transaction, concurrency, idempotency, serialization, confirmation, redelivery and poison component scenarios.
7. Build WebApplicationFactory host tests for DI, middleware, authorization, HTTP/problem contracts, health and hosted-service lifecycle.
8. Build the real image/deployment laboratory and execute dependency, capacity, terminal fault, shutdown and recovery scenarios through the controller.
9. Run repeat, parallel, order, leak and zero-discovery checks; retain redacted machine evidence bound to the exact identities.
10. Review TST-OWN-001 through TST-REVIEW-001 with engineering, architecture, data, security and operations; keep the gate pending until all blockers pass.

What this chapter has proved

Chapter 12 has mapped test cost to claim fidelity; assigned Domain, Application, architecture, provider, host, contract and deployed-fault responsibilities; bounded doubles; required the production provider for provider semantics; separated TestServer from deployment claims; made isolation/discovery/evidence identity blocking; and attached every layer to TST evidence.

It has not compiled or executed a RootCause test suite, selected packages/providers, built fixtures, discovered tests, generated coverage, started a container, built an image, injected a fault or observed recovery. Part II is now structurally specified from golden path through test evidence, but RootCause remains G0 accepted and G1 specified; TPL/CMP/LAY/CFG/HLT/TST/BUILD evidence remains pending.

PART II CLOSING

A service foundation is not a template plus a green test command. It is a constrained shape whose composition, layers, configuration, health and test evidence make each owned claim falsifiable at the boundary where it can actually fail.

Chapter checkpoint

You are ready for Part III when each RootCause claim has one primary test surface, every real boundary has a faithful component or deployment test, and no green result is allowed to imply an unexecuted mechanism.

Question	Expected answer
What chooses a test layer?	The cheapest surface that executes the mechanism capable of falsifying the claim.
What belongs in Domain unit tests?	Owned invariants, deterministic decisions, transitions and policy identity without infrastructure.
When is a mock legitimate?	At an owned port to isolate its caller, with explicit exclusions; never as proof of provider/protocol behaviour.
What proves SQL/provider semantics?	A component suite against the selected production database engine/provider and accepted migrations.
What does WebApplicationFactory prove?	ASP.NET Core composition and request pipeline in TestServer, not Kestrel/TLS/container/network/controller behaviour.
What completes a fault test?	Observed failure, selected controller action, bounded impact, stable recovery and durable reconciliation.
Why is coverage insufficient?	It shows visited code, not assertion quality, provider fidelity, compatibility, security or recovery.
How are zero tests prevented?	Expected assemblies/suites and discovered counts are compared with machine-readable result artifacts.
What is the current RootCause gate?	G0 accepted; Part II G1 foundation specified; executable TPL/CMP/LAY/CFG/HLT/TST/BUILD evidence pending.
What must Part III add?	Executable service-owned database boundaries, beginning with the transition from seventeen schemas to owned databases.

NEXT CHAPTER

Chapter 13 opens Part III, Data Ownership in Executable Form, with From Seventeen Schemas to Service-Owned Databases. It turns the accepted context boundaries into a migration inventory: ownership, credentials, schema lifecycle, cross-schema reads/writes, transition seams and the evidence required before any table moves.

PART III

Data Ownership in Executable Form

A database becomes service-owned only when schema, credentials, migration, commit and recovery authority can evolve independently.

CHAPTER 13**Ownership map**

inventory -> classification -> cutover

CHAPTER 14**AlarmManagementDb**

aggregate truth + pending fact

CHAPTER 15**RootCauseDb**

evidence graph + verdict lineage

CHAPTER 16**Migration lifecycle**

artifact -> expand/contract -> restore

CHAPTER 17**Cross-service identity**

stable IDs without shared tables

CHAPTER 18**Local transaction**

one owner, one atomic boundary

ENTRY RULE

No cross-database foreign key, write, transaction or migration authority may survive the ownership boundary.

BEGIN WITH CHAPTER 13 / INVENTORY IS NOT OWNERSHIP

PART III - DATA OWNERSHIP IN EXECUTABLE FORM

13. From Seventeen Schemas to Service-Owned Databases

A database boundary is executable only when one service owns the facts, one credential owns the writes, one migration stream owns the schema, and every reader knows whether it is consuming truth or a projection. Seventeen legacy schemas establish none of those things by themselves.

CHAPTER ARGUMENT

Treat the seventeen schemas as a discovery inventory, not a target service map. Classify every fact, writer, reader, transaction, constraint, job and history obligation; then move one bounded ownership unit through a reconciled one-writer cutover with reversible, reviewable evidence.

Part II specified a RootCause service foundation without claiming a generated repository or runtime. Part III now turns the accepted context boundaries into data authority that can be enforced by databases, credentials, migrations and operational lifecycle. The starting point is a legacy estate described as seventeen schemas. That number is a deployment fact. It is not a domain model, a team topology or evidence that seventeen independently deployable services exist.

The inherited preservation rule remains DATA-OWN-001: no service writes another service's source-of-truth database. RootCause owns verdict truth; AlarmManagement owns alarm intake and flood detection; Audit, Compliance and Reporting observe public facts without controlling those decisions. A service also violates ownership when it reads another schema and reimplements its rule locally, even if it never issues an UPDATE. Write isolation and decision authority must move together.

This chapter does not create AlarmManagementDb or RootCauseDb, choose a relational provider, generate EF Core migrations, copy a row, revoke a credential or run a cutover. It defines the inventory, transition states and blocking evidence required before those actions can be accepted. Chapters 14 and 15 build the two owned models; Chapter 16 owns their independent migration lifecycle; Chapter 17 handles cross-service identity; Chapter 18 completes local transaction and concurrency boundaries.

A schema is a clue, not an owner

Legacy schemas often encode chronology, vendors, security groupings, reporting convenience or one application's package structure. One schema can mix facts from several bounded contexts; one bounded context can be scattered across schemas. Views, synonyms, triggers, procedures and scheduled jobs can cross those visible borders. Renaming each schema as a service database would preserve the coupling under newer labels.

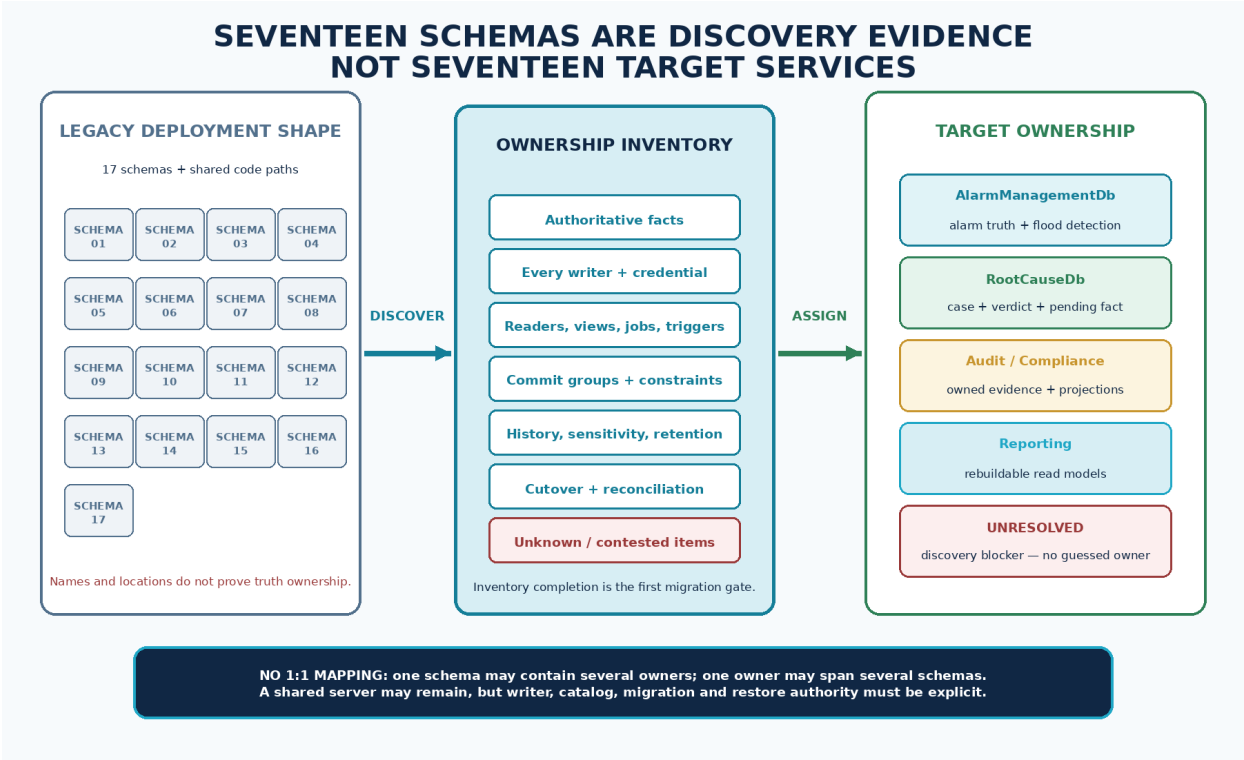


Figure 13.1 - The seventeen schemas enter an ownership inventory; they do not map one-for-one to target services or databases.

Legacy observation	Unsafe inference	Required discovery
A table sits in schema RootCause.	RootCause owns every column and business rule.	Which facts are authoritative, who decides them, and which code paths write them?
Only one application account is documented.	Only one writer exists.	Logins, roles, connection strings, jobs, procedures, triggers, ETL and manual paths.
A foreign key crosses schemas.	The entities belong in one service.	Which invariant needs atomic enforcement and whether the boundary is drawn correctly.
Reporting selects operational tables.	The read is harmless because it is read-only.	Freshness need, query load, sensitive fields, failure coupling and projection replacement.
A stored procedure updates several schemas.	It is merely shared utility code.	Decision owner, commit group, callers, side effects, retry/idempotency and error semantics.
A schema has no current application calls.	It is safe to delete or archive.	Jobs, ad-hoc access, legal retention, restore obligations and dormant incident procedures.

ADR BOUNDARY

Do not assign an owner from a schema or table name alone. Unknown and contested items remain explicit blockers until business and runtime evidence agree.

Build the inventory before the target diagram

The inventory is object-level where necessary and fact-level where a table mixes meanings. It combines static catalog extraction with runtime and organizational evidence. Catalog metadata finds objects and declared dependencies; query telemetry, job definitions, configuration repositories and interviews find active and hidden paths. Production telemetry alone cannot prove absence, because monthly, incident-only and manually invoked paths may not appear during the observation window.

Inventory field	Minimum record	Why it blocks movement
Data identity	Database/catalog, schema, object, columns or fact slice, key and current schema identity.	Prevents similarly named objects or replicas from being treated as one source.

Inventory field	Minimum record	Why it blocks movement
Semantic owner	Bounded context and accountable decision owner for each authoritative fact.	A technical team cannot safely move a rule it does not own.
Writers	Application, job, procedure, trigger, operator and credential; frequency and failure path.	A hidden writer creates split truth after cutover.
Readers	Query/API/report/ETL path, fields, purpose, freshness, availability and load.	Every direct reader needs a contract, projection or accepted temporary seam.
Commit group	Objects and invariants currently committed atomically; isolation/concurrency assumptions.	Splitting an invariant without redesign produces partial business state.
Database dependencies	FK, view, synonym, function, sequence, trigger, procedure, job and cross-catalog reference.	Invisible execution order and side effects defeat independent lifecycle.
Data qualities	Volume, change rate, null/duplicate rules, sensitivity, retention, lineage and history.	Backfill, masking, validation and archive strategy depend on them.
Operations	RPO/RTO, backup/restore, maintenance, peak windows, owner and incident runbook.	Ownership includes failure and restore, not only CRUD.
Transition	Target class, seam, cursor/watermark, cutover unit, reconciliation and reversal owner.	A destination without a bounded path is only an architecture sketch.
Evidence status	Discovery source, timestamp, reviewer, confidence, exception and open question.	Unverified assumptions cannot silently become migration facts.

DATA-UNIT RECORD - [REFERENCE][LEGACY DISCOVERY REQUIRED]

```
# Reference inventory shape – values are illustrative, not discovered evidence.
dataUnit: root-cause.case-and-verdict
currentLocations: [legacy.schema_04, legacy.schema_11]
semanticOwner: RootCause
classification: source-of-truth
writers:
  - path: legacy-root-cause-command
    credential: inventory-reference-only
readers:
  - purpose: reporting-projection
    replacement: RootCauseCaseDecided public fact
commitGroup: case-state-plus-pending-public-fact
cutover:
  mode: one-writer-cohort
  watermark: required
  reconcile: per-key-meaning-plus-history
status: UNVERIFIED
```

Classify the data unit before assigning a destination

Classification	Authority rule	Migration consequence
Source of truth	One context decides and one owned store records the accepted fact.	Requires single-writer cutover, invariant proof, history decision and write revocation.
Derived projection	Rebuildable from owned public facts or an accepted snapshot plus stream.	May be rebuilt in parallel; lag and rebuild completeness are explicit, never promoted to truth.
Reference copy	A bounded subset copied for local decisions that do not steal the source rule.	Needs owner identity, freshness/expiry, missing/stale behaviour and correction path.
Append-only evidence	Records observations or actions under an evidence owner's retention/immutability policy.	Preserve lineage, order semantics, redaction, retention and legal hold; do not rewrite as business truth.
Technical coordination	Lease, checkpoint, inbox/outbox, lock or migration marker with no foreign decision authority.	Moves with the owning process and its recovery semantics; never used as a hidden public API.
Unknown / contested	Ownership or meaning cannot yet be demonstrated.	No destructive move or authority transfer; discovery remains blocking.

The classification applies to meaning, not necessarily to a whole table. A legacy Case table might contain RootCause verdict truth, impo

denormalizations. The target does not have to reproduce that row shape. Each fact receives an owner; foreign identifiers remain references; derived fields move to projections or are recomputed only when their meaning and history permit it.

Four ownerships must be named

The phrase database owner is too vague for migration review. NEXUS-1 records semantic ownership, write ownership, schema-change ownership and operational custody. They should converge on the service team and its platform contract. When they cannot, the exception states who may decide, write, migrate, restore and approve a cutover; an operations team hosting the server does not acquire business decision authority.

Ownership dimension	Accountability	Executable control
Semantic / decision	Defines authoritative facts, invariants, policy identity and public meaning.	Context map, preservation ledger, domain tests and contract review.
Write	Chooses the only accepted command path and handles concurrency/idempotency.	Service-specific runtime principal, grants/denials and negative cross-write tests.
Schema / migration	Versions object shape and approves forward/repair operations.	Owned migration assembly/artifact, history identity and deployment gate.
Operational custodian	Backs up, restores, monitors, patches and meets RPO/RTO.	Runbook, restore rehearsal, alerts, escalation and evidence retention.

The first target ownership map

The accepted contexts establish destinations for meaning, not a promise that every database is already built. AlarmManagementDb and RootCauseDb are source-of-truth targets. Audit and Compliance need owned evidence or assessment stores only when their write models are defined. Reporting consumes projections. Unknown legacy facts stay outside the accepted target until discovery resolves them.

Context	Owned authoritative meaning	Target data role	Must not become
AlarmManagement	Alarm intake identity, accepted alarm state and flood-detection outcome.	AlarmManagementDb source of truth plus pending public facts.	A writer of RootCause verdicts or a generic shared alarm schema.
RootCause	Case, accepted evidence, verdict, policy identity and pending public fact.	RootCauseDb source of truth.	A direct writer to Audit/Compliance/Reporting or a reader that re-decides AlarmManagement truth.
Audit	Audit-owned immutable observations, actor/action lineage and retention metadata.	Evidence store fed by public facts and local audit actions.	A synchronous prerequisite for RootCause truth or a duplicate command owner.
Compliance	Compliance-owned assessments, rule identities and review outcomes.	Assessment store/projection with explicit source lineage.	A hidden RootCause policy engine accessed through shared tables.
Reporting	Report definitions, projection checkpoints and derived analytical models.	Rebuildable read stores optimized per query.	An authoritative correction surface or permanent direct operational reader.
Unresolved legacy	Not yet accepted.	Quarantined discovery backlog with accountable reviewer.	A guessed service/database created to make the diagram complete.

PHYSICAL HOSTING

Service-owned databases may share a physical database server when that is operationally appropriate. Independence is demonstrated by private catalogs/schemas, principals, migration history, backup/restore scope and denied foreign writes—not by buying one server per service.

Catalogue every cross-schema dependency

The migration graph is built from execution paths, not just foreign keys. Direct writes and cross-schema commit groups are the highest-risk edges, but read views, procedures, triggers, shared sequences, jobs and operator scripts can hold the same coupling. Each edge receives a source, destination, business purpose, mechanism, credential, frequency, failure behaviour, replacement and removal gate.

Dependency edge	Risk carried	Preferred target
Foreign service write	Two command owners, hidden invariants and coupled releases.	Owned command/API only when synchronous response is required; otherwise public fact and local reaction.
Cross-schema transaction	Atomicity spans future owners and can block independent failure/release.	Redraw boundary or replace with local commits plus durable process/compensation and reconciliation.
Cross-schema FK	Lifecycle, delete and restore order remain coupled.	Stable foreign identity plus local validation/projection; Chapter 17 owns the complete rule.
Shared view / direct SELECT	Schema, performance, availability and privacy coupling.	Owner API for current need or event-fed projection for local query.
Stored procedure / trigger	Side effects and decision code hide inside the database.	Assign code and fact ownership; expose one owned command path; remove cross-owner mutation.
ETL / CDC / batch job	Lag, duplicate, delete and reordering semantics may be implicit.	Bounded transition seam or owned projection pipeline with checkpoint/replay evidence.
Shared sequence / key generator	Allocation availability and collision policy couple owners.	Service-owned identifiers or an explicit identity authority; never an accidental database dependency.
Shared runtime credential	Permission inheritance defeats ownership even if code intends isolation.	Per-service principal; separate migration authority; negative grants and rotation evidence.
Manual correction	Operator bypass creates untracked source changes.	Audited owner command/runbook with authorization, reason and reconciliation.

Credentials make ownership enforceable

The final boundary is not a coding convention. Each service receives a runtime principal scoped to its owned database objects and operations. Migration authority is separate because schema change requires privileges that the running service should not retain. Projection readers receive only their accepted read surface. Shared administrator credentials are not embedded in services, jobs or diagnostics, and cross-owner denial is tested from the actual deployment identity.

Principal	Allowed	Denied / absent	Evidence
RootCause runtime	RootCause data operations and owned technical coordination.	AlarmManagement/Audit/Compliance/Reporting writes; schema alteration.	Effective-permission inventory plus successful owned and failed foreign operations.
AlarmManagement runtime	AlarmManagement data operations and owned technical coordination.	RootCause verdict/case writes; schema alteration.	Same positive/negative permission test from deployed identity.
Migration executor	Apply reviewed migration artifact to one owned database and history.	Normal request processing; unrelated databases; standing interactive use.	Short-lived identity, artifact hash, approval, transcript and resulting schema identity.
Projection reader/builder	Only accepted owner API/fact stream or transition read surface.	Source writes and privileged columns not required by projection.	Field/grant review, redaction test, checkpoint and freshness metrics.
Operator break-glass	Explicit bounded repair under incident/change authority.	Routine application use or anonymous shared access.	Time-bound grant, actor/reason, commands, before/after state and reconciliation.

CREDENTIAL GATE

A repository scan that finds no foreign connection string is useful but insufficient. DATA-CRED-001 passes only when effective runtime permissions prove owned access and reject foreign writes with the identities used by the deployed services

A transition seam is temporary and owned

Extraction needs a seam between legacy and target states. It may be an anti-corruption adapter, read-only view, export job, change stream, owner API or public-fact projection. The seam has an owner, purpose, admitted fields, ordering/duplicate/delete semantics, start and end condition, security scope, telemetry and removal date. Calling it temporary without an executable removal gate turns it into a permanent shared database API.

Transition pattern	Use when	Non-negotiable boundary
Backfill + public fact tail	Historical state is copied once and new owner facts maintain the target/projection.	Cursor and event checkpoint overlap safely; duplicates/deletes/late facts reconcile.
Read-only anti-corruption adapter	New code must interpret a legacy read model before the owner moves.	No foreign write; legacy semantics translated at one replaceable edge; owner and sunset recorded.
Owner API before physical move	Decision/write authority can move while storage is temporarily hosted in legacy infrastructure.	Only the new owner writes; catalog/credentials/migrations are isolated enough to enforce that fact.
Cohort routing	Entities/tenants can move in bounded groups.	A durable routing authority maps each key to exactly one writer; cross-cohort invariants are understood.
Projection replacement	A foreign direct reader needs query-shaped data, not command authority.	Projection is explicitly stale/rebuildable and cannot correct source truth.
Freeze-and-copy	Change volume and downtime permit a bounded write pause.	Freeze is verified, copy/reconcile completes inside budget, and reopening selects exactly one writer.

Application-level dual-write is rejected as a steady state. A request that writes legacy and target stores cannot make both commits one durable business fact across independent databases; retries and partial failure create ambiguity. Shadow copies may be populated by a controlled migration pipeline, but the source-of-truth writer remains singular and recorded throughout.

The one-writer cutover state machine

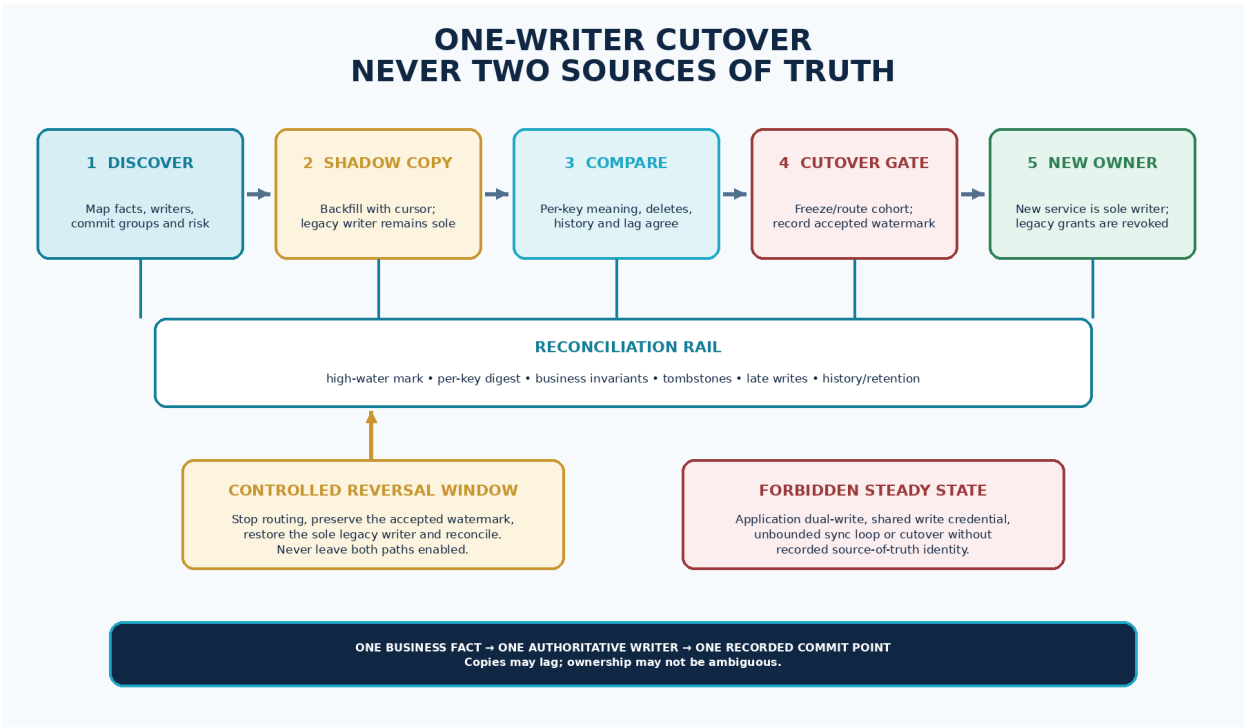


Figure 13.2 - Backfill and comparison may run in parallel; authoritative business writes never do.

State	Writer authority	Entry evidence	Exit blocker
Discover	Legacy writer(s), explicitly inventoried.	Objects, meanings, writers/readers, commit groups and operations recorded.	Unknown writer, owner or invariant.
Shadow copy	Legacy remains sole authoritative writer.	Target schema, bounded cursor, history/delete policy and isolated credentials ready.	Unbounded copy, missing checkpoint or target mutation by business commands.
Compare	Legacy remains sole authoritative writer.	Backfill/tail caught up within declared lag; per-key and business reconciliation runs.	Mismatch above threshold, unexplained orphan or history loss.
Cutover gate	Exactly one writer per admitted cohort during freeze/route change.	Approved watermark, health/capacity, reversal steps, owner roster and monitoring window.	Ambiguous routing, stale credentials or no durable cutover record.
New owner	New service is sole authoritative writer.	Legacy writes revoked; commands reach new owner; reconciliation remains green.	Residual writer, unexplained divergence or unowned operational lifecycle.
Retire seam	New owner remains sole writer.	All readers replaced, retention satisfied, restore path accepted and observation window complete.	Active dependency, legal hold, rollback reliance or missing archive evidence.

Reconciliation proves meaning, not just row count

Equal row counts can hide different keys, values, deletes, order, policy identities and history. Reconciliation is defined before copy begins and runs repeatedly through the comparison and observation windows. It records the source watermark and target checkpoint so reviewers know which state was compared. Differences are classified, explained and either repaired or accepted by the semantic owner; a tolerance cannot silently legalize lost truth.

Reconciliation layer	Required comparison	Failure exposed
Coverage	Key ranges/partitions, high-water mark, target checkpoint and explicit excluded cohorts.	A successful job copied only an easy subset.
Identity	Per-key presence, stable owner ID, duplicate mapping and orphan set.	Rows match in count while entities differ or collide.
Meaning	Canonical per-key digest or field comparison after documented normalization.	Formatting/default/time-zone conversion changes business facts.
Invariant	Domain rules such as valid state transition, one accepted verdict/policy identity and one owned effect.	Individually plausible rows form impossible business state.
Delete / correction	Tombstone, hard-delete, supersession and late-correction semantics.	Target resurrects deleted data or misses corrected truth.
History / evidence	Required event/version chronology, actor/reason, retention and legal-hold continuity.	Current state matches while accountable history is lost.
Freshness	Observed lag distribution and maximum against consumer/cutover budget.	A projection is correct only after an unsafe delay.
Operational	Query/commit latency, capacity, backup, restore and monitoring during representative load.	Correct data lands in a store that cannot operate safely.

Do not split a commit group mechanically

A legacy transaction that touches several schemas is not automatically a good distributed transaction and not automatically safe to break. The inventory identifies the business invariant it protects. If all writes form one owned aggregate, the target boundary may need to keep them local. If the transaction combines distinct owners, the redesign chooses a local commit in each context, durable public facts, idempotent reactions, an explicit process state and compensation or manual reconciliation where reversal is meaningful.

Legacy atomicity claim	Question before extraction	Target direction
Case state + pending public fact	Are both RootCause-owned and required for recoverable publication?	Keep one local RootCauseDb transaction; publication/acknowledgement follows asynchronously.
Alarm update + RootCause verdict	Does one context truly own both decisions, or did the monolith hide two owners?	Separate owned commits and public contract; RootCause alone decides verdict from accepted facts.
Operational row + reporting summary	Is the summary authoritative or rebuildable?	Commit operational truth locally; build reporting projection asynchronously.
Business state + audit observation	Must audit availability choose whether business truth exists?	Commit owner state/public intent locally; Audit records observation independently with recovery evidence.
Cross-schema FK delete	Which owner decides deletion and what must consumers retain?	Owner publishes lifecycle fact; consumers apply local retention/reference policy without cross-database cascade.

SCOPE BOUNDARY

This chapter records the commit group and target direction. Chapter 18 specifies exact local transaction, optimistic concurrency and commit mechanics; Part IV specifies the public-fact and delivery contracts.

Readers migrate to contracts or projections

A foreign SELECT is an availability and change contract disguised as SQL. Current decisions that need owner validation use an owner API only when the latency and failure coupling are accepted. Query-heavy, analytical or cross-context views use local projections fed by public facts and, when necessary, a bounded snapshot. The projection owns its schema and checkpoint but never acquires authority to correct the source fact.

- Record the consumer purpose and minimum fields; do not copy an entire source row for convenience.
- State the accepted freshness, missing-data and stale-data behaviour for every reference copy or projection.
- Carry stable owner-issued identities and source contract/policy identity; do not recreate foreign database keys as local truth.
- Design rebuild, replay, duplicate, delete, correction and version compatibility before removing the direct read.
- Separate operational projections from long-retention analytics when their workload, privacy, cost and restore needs differ.
- Revoke the old SELECT path after the consumer proves its replacement; a dormant grant remains a regression path.

A migration unit needs a complete dossier

The safe unit of movement is not necessarily a table. It is the smallest set of authoritative facts, invariants, history and technical coordination that can move under one writer and one reconciliation definition. Each dossier binds design to execution and becomes the input to the detailed database chapters.

Dossier section	Required content
Identity and scope	Data-unit ID, semantic owner, keys/cohorts, current locations, classification and exclusions.
Current execution	All writers/readers/credentials, procedures/jobs/triggers, commit groups, failure and correction paths.

Dossier section	Required content
Target authority	Owning service/database, command path, runtime/migration principals, schema/migration owner and public contracts.
Transform	Field/meaning mapping, defaults, invalid/duplicate policy, history, sensitive data and retention decisions.
Transfer	Snapshot/cursor/tail mechanism, capacity budget, throttling, retries, idempotency and checkpoint durability.
Reconcile	Coverage, per-key meaning, invariants, deletes/corrections, history, freshness and operational checks.
Cutover	Cohort/freeze/routing plan, watermark, one-writer assertion, credential revocation and observation window.
Reversal / repair	Allowed reversal point, sole-writer restoration, forward-repair path, backups and decision authority.
Retirement	Reader replacement, grant/seam removal, archive/delete/legal hold, restore ownership and evidence retention.
Acceptance	Evidence IDs, artifact/schema identities, timestamps, reviewers, exceptions and residual risk.

Data-ownership evidence gates

Evidence ID	Blocking rule	Required artifact
DATA-INV-001	Every in-scope object/fact, writer, reader, dependency, commit group and operation has a confidence-marked record.	Catalog/runtime/config/job inventory plus unresolved-item register.
DATA-OWN-001	Each authoritative fact has one accepted semantic owner and no foreign decision implementation.	Fact-to-context map, preservation review and contested-owner disposition.
DATA-WRITE-001	Every cohort has exactly one authoritative writer in every migration state.	Writer/routing manifest, cutover watermark and dual-writer negative result.
DATA-READ-001	Every foreign read is removed or bound to an accepted temporary seam, owner API or local projection.	Reader-purpose inventory, replacement contract and removal evidence.
DATA-CRED-001	Runtime and migration principals are separate, least-privileged and deny foreign writes.	Effective grants/denials, identity-bound positive/negative tests and rotation record.
DATA-SEAM-001	Every transition seam has bounded semantics, telemetry, security, owner and executable retirement gate.	Seam contract, field/lag/delete rules, dashboard and sunset condition.
DATA-CUT-001	Cutover is cohort-bounded, one-writer, observable and tied to a durable source-of-truth identity.	Approved runbook, state timeline, watermark, routing/grant change and observation record.
DATA-REC-001	Coverage, identity, meaning, invariants, deletes, history, freshness and operations reconcile.	Machine comparison plus explained/accepted difference register.
DATA-REV-001	Reversal or forward repair preserves one writer and names the decision authority and terminal state.	Rehearsal, backup/restore identity, triggers, steps and reconciliation result.
DATA-LIFE-001	Owned database migration, backup, restore, retention, monitoring and retirement lifecycles are assigned.	Lifecycle/runbook catalogue and restore evidence; detailed in Chapter 16.
DATA-REVIEW-001	Semantic owner, data, security, operations and consuming teams accept evidence and residual risk.	Identity-bound review with exceptions and expiry.

GATE HONESTY

DATA evidence must identify the exact source/target schema, code/artifact, credential, cohort, watermark, time window and reviewer. A diagram, successful copy command or equal row count cannot advance a migration gate by itself.

ADR-011 - Treat the seventeen schemas as a discovery inventory

ADR field	ADR-011 record
Status	Accepted for the NEXUS-1 reference design; legacy inventory and execution evidence remain pending.
Context	The monolith exposes seventeen schemas, but schema boundaries may mix owners and hide writers, readers, transactions, jobs and operational obligations.
Decision	Do not map schemas one-for-one to services. Inventory facts and execution paths; assign one semantic/write/schema lifecycle owner; move bounded data units through shadow, compare and one-writer cutover; replace foreign access with contracts/projections; reconcile meaning and revoke legacy authority.
Consequences	Discovery precedes extraction; unknowns remain visible; some schemas split or combine; temporary seams require retirement; credentials and restore lifecycle become architecture evidence.
Rejected	Seventeen databases/services by rename, permanent schema-per-service in a shared-write catalog, application dual-write, shared runtime credential, row-count-only validation, direct reporting reads as the final state.
Review trigger	New writer/reader/job, changed context authority, provider/topology choice, cross-owner invariant, retention/security obligation, cutover model or restore boundary.

Risk register for data extraction

Risk	Early signal	Blocking control
Schema-to-service rename	Target diagram has seventeen boxes before fact ownership is reviewed.	DATA-INV/OWN inventory and explicit many-to-many mapping.
Hidden writer	Target diverges after a batch, incident or manual correction.	Credential/config/job/trigger discovery, runtime observation and post-cutover denial.
Dual truth	Legacy and target accept commands for the same key/cohort.	Durable routing authority, one-writer state machine and DATA-WRITE-001.
Semantic transform loss	Counts match but verdict, policy, delete or history differs.	Canonical per-key/invariant/history reconciliation and owner sign-off.
Read coupling survives	A report or service still queries the old catalog after cutover.	Reader inventory, projection/API replacement and grant removal.
Transaction fracture	Partial business state appears across newly separate owners.	Commit-group analysis, local transaction redesign and durable process/reconciliation.
Privilege regression	Service can connect with administrator/shared credentials.	Separate principals, effective-permission test, secret rotation and break-glass audit.
Seam becomes permanent	No owner, traffic metric or removal condition exists.	DATA-SEAM-001 with expiry, dashboard and retirement blocker.
Restore incompatibility	Source and projection restore to unrelated points with no recovery rule.	Owned RPO/RTO, checkpoints/replay strategy and Chapter 16 restore rehearsal.
Premature deletion	Dormant job, legal hold or historical evidence appears after retirement.	Retention/legal/restore review and recoverable archive before destructive action.

Official-source validation

The architecture and lifecycle mechanics were checked against current Microsoft documentation on 18 July 2026. Stable points used here are that microservices manage private data stores; services should not directly share a data store or tables; a physical database server may be shared; source-of-truth and eventually consistent copies must be distinguished; least permissions should be granted to database principals; EF Core production migrations should be inspected and tested; and a local SaveChanges call is transactional when the provider supports transactions.

CHAPTER 13 - CODE AUDIT

The legacy estate becomes a typed discovery inventory

Schema names are observations. The extraction model records objects, semantic candidates, writers, readers, credentials, commit groups, dependencies, history, and confidence before any target database is proposed.

EVIDENCE STATUS

The following assets are compile-oriented reference material. No legacy catalog, runtime trace, organizational ownership record, provider baseline, or migration rehearsal is available here; DATA evidence remains PENDING.

EXECUTABLE ASSET 13-A - INVENTORY RECORDS

```
public enum DataRole
{
    AuthoritativeFact, DerivedProjection, ForeignReference,
    EvidenceRecord, TechnicalCoordination, Unknown
}

public sealed record LegacyObject(
    string Database, string Schema, string Name,
    string Kind, long? RowCount, long? SizeBytes,
    string DefinitionDigest);

public sealed record AccessPath(
    string ObjectIdentity, string Principal,
    string Mechanism, AccessMode Mode,
    string SourceArtifact, Confidence Confidence);

public sealed record OwnershipCandidate(
    string ObjectIdentity, string FactGroup,
    DataRole Role, string? SemanticOwner,
    IReadOnlySet<string> CommitGroups,
    Confidence Confidence, string EvidenceId);

public enum AccessMode { Read, Write, Execute, Ddl }
public enum Confidence { Confirmed, Observed, Declared, Inferred, Unknown }
```

EXECUTABLE ASSET 13-B - FAIL-CLOSED INVENTORY GATE

```
[Fact]
public void DATA_INV_001_unknown_writer_or_owner_blocks_movement()
{
    var inventory = LegacyInventory.Load("artifacts/data/inventory.json");

    var blockers = inventory.Objects
        .Where(x => inventory.AccessPaths.For(x).Any(p =>
            p.Mode == AccessMode.Write
            && p.Confidence == Confidence.Unknown)
            || inventory.Ownership.For(x).SemanticOwner is null)
        .ToArray();

    blockers.Should().BeEmpty(
        "unknown authority must remain visible, not defaulted");
}
```


CATALOG EXTRACTION

Static metadata and observed access are separate evidence streams

EXECUTABLE ASSET 13-C - SQL SERVER CANDIDATE CATALOG QUERY

```

SELECT
  DB_NAME()                                AS database_name,
  s.name                                    AS schema_name,
  o.name                                    AS object_name,
  o.type_desc                              AS object_kind,
  SUM(CASE WHEN p.index_id IN (0,1)
    THEN p.rows ELSE 0 END)                AS row_count,
  SUM(a.total_pages) * 8192                AS size_bytes,
  CONVERT(varchar(64), HASHBYTES('SHA2_256',
    COALESCE(m.definition, '')), 2)        AS definition_digest
FROM sys.objects o
JOIN sys.schemas s ON s.schema_id = o.schema_id
LEFT JOIN sys.sql_modules m ON m.object_id = o.object_id
LEFT JOIN sys.partitions p ON p.object_id = o.object_id
LEFT JOIN sys.allocation_units a
  ON a.container_id = p.partition_id
WHERE o.is_ms_shipped = 0
GROUP BY s.name, o.name, o.type_desc, m.definition
ORDER BY s.name, o.name;

```

EXECUTABLE ASSET 13-D - EXTRACTION BOUNDARY

```

public interface ILegacyEstateExtractor
{
    Task<IReadOnlyList<LegacyObject>> ExtractCatalogAsync(
        CancellationToken cancellationToken);
    Task<IReadOnlyList<DependencyEdge>> ExtractDependenciesAsync(
        CancellationToken cancellationToken);
    Task<IReadOnlyList<DatabasePrincipal>> ExtractPrincipalsAsync(
        CancellationToken cancellationToken);
    Task<IReadOnlyList<AccessPath>> ExtractObservedAccessAsync(
        ObservationWindow window,
        CancellationToken cancellationToken);
}

public sealed record InventorySnapshot(
    string SourceIdentity,
    DateTimeOffset CapturedAt,
    ObservationWindow RuntimeWindow,
    IReadOnlyList<LegacyObject> Objects,
    IReadOnlyList<DependencyEdge> Dependencies,
    IReadOnlyList<DatabasePrincipal> Principals,
    IReadOnlyList<AccessPath> AccessPaths);

```

EXECUTABLE ASSET 13-E - STATIC/OBSERVED MISMATCH REPORT

```

[Fact]
public void DATA_INV_002_declared_and_observed_writers_reconcile()
{
    var snapshot = InventoryFixture.Load();
    var declared = snapshot.AccessPaths
        .Where(x => x.Mode == AccessMode.Write
            && x.Confidence == Confidence.Declared)
        .Select(AccessKey.From).ToHashSet();
    var observed = snapshot.AccessPaths
        .Where(x => x.Mode == AccessMode.Write
            && x.Confidence == Confidence.Observed)
        .Select(AccessKey.From).ToHashSet();

    declared.Except(observed)
        .Concat(observed.Except(declared))
        .Should().OnlyContain(x => x.HasReviewedDisposition);
}

```

CANDIDATE-SPECIFIC SQL

The T-SQL query is a SQL Server discovery adapter, not proof that the provider baseline has been accepted. The inventory model and evidence IDs remain provider-neutral.

DEPENDENCY GRAPH

Executable edges include more than foreign keys

EXECUTABLE ASSET 13-F - DEPENDENCY EDGE MODEL

```
public enum DependencyKind
{
    ForeignKey, CrossDatabaseReference, View, Synonym,
    ProcedureCall, TriggerWrite, SharedSequence,
    ScheduledJob, EtlPath, OperatorScript
}

public sealed record DependencyEdge(
    string Source, string Target, DependencyKind Kind,
    string Principal, string Purpose,
    string FailureBehavior, string Replacement,
    string RemovalGate, Confidence Confidence);
```

EXECUTABLE ASSET 13-G - CROSS-OWNER BLOCKERS

```
public static IReadOnlyList<MigrationBlocker> FindBlockers(
    InventorySnapshot inventory,
    OwnershipMap ownership)
{
    return inventory.Dependencies
        .Where(edge =>
            !ownership.OwnersOf(edge.Source)
                .SetEquals(ownership.OwnersOf(edge.Target)))
        .Select(edge => edge.Kind switch
        {
            DependencyKind.TriggerWrite or
            DependencyKind.ProcedureCall =>
                MigrationBlocker.ForeignWrite(edge),
            DependencyKind.ForeignKey =>
                MigrationBlocker.CrossOwnerConstraint(edge),
            DependencyKind.View or
            DependencyKind.Synonym =>
                MigrationBlocker.ForeignRead(edge),
            _ => MigrationBlocker.Unreviewed(edge)
        })
        .ToArray();
}
```

EXECUTABLE ASSET 13-H - NO HIDDEN EXECUTABLE EDGE

```
[Fact]
public void DATA_OWN_001_every_cross_owner_edge_has_replacement()
{
    var inventory = InventoryFixture.Load();
    var ownership = OwnershipMap.Load();

    var blockers = FindBlockers(inventory, ownership);

    blockers.Should().OnlyContain(x =>
        x.Edge.Confidence != Confidence.Unknown
        && !string.IsNullOrEmpty(x.Edge.Replacement)
        && !string.IsNullOrEmpty(x.Edge.RemovalGate));
}
```

OWNERSHIP CATALOGUE

Facts move; legacy row shapes do not become the target model

EXECUTABLE ASSET 13-I - FACT-LEVEL OWNERSHIP RECORD

```
public sealed record FactOwnership(
    string FactId,
    string LegacyObject,
    string Meaning,
    DataRole Role,
    string SemanticOwner,
    string WriteOwner,
    string SchemaLifecycleOwner,
    string OperationalCustodian,
    Sensitivity Sensitivity,
    HistoryPolicy History,
    string DecisionRecord);

public sealed record OwnershipMap(
    IReadOnlyDictionary<string, FactOwnership> Facts)
{
    public IReadOnlySet<string> OwnersOf(string objectIdentity) =>
        Facts.Values
            .Where(x => x.LegacyObject == objectIdentity)
            .Select(x => x.SemanticOwner)
            .ToHashSet(StringComparer.Ordinal);
}
```

EXECUTABLE ASSET 13-J - CONTESTED FACTS REMAIN BLOCKED

```
[Fact]
public void DATA_OWN_002_only_accepted_fact_groups_enter_target_design()
{
    var map = OwnershipMap.Load();

    map.Facts.Values.Should().OnlyContain(fact =>
        fact.Role != DataRole.Unknown
        && !string.IsNullOrEmpty(fact.SemanticOwner)
        && fact.SemanticOwner == fact.WriteOwner
        && !string.IsNullOrEmpty(fact.DecisionRecord));
}
```

EXECUTABLE ASSET 13-K - ROOTCAUSE OWNERSHIP SLICE

```
{
    "factId": "rootcause.case.verdict",
    "legacyObjects": [
        "Legacy.RootCause.Case",
        "Legacy.RootCause.VerdictHistory"
    ],
    "meaning": "RootCause-owned issued or corrected verdict",
    "role": "AuthoritativeFact",
    "semanticOwner": "RootCause",
    "writeOwner": "RootCause",
    "schemaLifecycleOwner": "RootCause",
    "operationalCustodian": "Platform/DataOperations",
    "commitGroup": [
        "RootCauseCase", "PendingPublicFact"
    ],
    "foreignReaders": ["Audit", "Compliance", "Reporting"],
    "replacement": "RootCauseVerdictIssued.v1 projections",
    "status": "PROPOSED_NOT_EVIDENCED"
}
```

FOUR OWNERS

Operational custody may remain with a platform team, but it does not grant semantic, write, or migration authority. Exceptions require a named decision record and expiry.

CREDENTIAL BOUNDARY

Runtime and migration authority are separate principals

EXECUTABLE ASSET 13-L - SQL SERVER CANDIDATE GRANTS

```
CREATE ROLE [rootcause_runtime];
CREATE ROLE [rootcause_migrator];

GRANT SELECT, INSERT, UPDATE, DELETE
  ON SCHEMA::[RootCause] TO [rootcause_runtime];
DENY ALTER, CONTROL
  ON SCHEMA::[RootCause] TO [rootcause_runtime];

GRANT ALTER, CONTROL
  ON SCHEMA::[RootCause] TO [rootcause_migrator];

DENY SELECT, INSERT, UPDATE, DELETE
  ON SCHEMA::[AlarmManagement] TO [rootcause_runtime];
DENY SELECT, INSERT, UPDATE, DELETE
  ON SCHEMA::[Audit] TO [rootcause_runtime];
DENY SELECT, INSERT, UPDATE, DELETE
  ON SCHEMA::[Compliance] TO [rootcause_runtime];
DENY SELECT, INSERT, UPDATE, DELETE
  ON SCHEMA::[Reporting] TO [rootcause_runtime];
```

EXECUTABLE ASSET 13-M - EFFECTIVE PERMISSION PROOF

```
[Fact]
public async Task DATA_CRED_001_runtime_can_write_only_owned_schema()
{
    await using var runtime = await DatabaseSession
        .ConnectAsAsync(Principals.RootCauseRuntime);

    await runtime.ExecuteAsync(
        "UPDATE RootCause.RootCauseCases SET Status='Open' WHERE 1=0");

    var foreignWrite = () => runtime.ExecuteAsync(
        "UPDATE AlarmManagement.AlarmFloods SET Status='Open' WHERE 1=0");
    await foreignWrite.Should().ThrowAsync<SqlException>();

    var ddl = () => runtime.ExecuteAsync(
        "ALTER TABLE RootCause.RootCauseCases ADD Forbidden int NULL");
    await ddl.Should().ThrowAsync<SqlException>();
}
```

EXECUTABLE ASSET 13-N - MIGRATOR IS ABSENT FROM RUNTIME GRAPH

```
[Fact]
public void DATA_CRED_002_host_has_no_migration_credential()
{
    var manifest = CompositionManifest.ForProduction();

    manifest.SecretReferences.Should()
        .Contain(SecretReferences.RootCauseRuntime);
    manifest.SecretReferences.Should()
        .NotContain(SecretReferences.RootCauseMigrator);
    manifest.Services.Should().NotContain(x =>
        x.Implementation.Contains("Migrator"));
}
```

ONE-WRITER CUTOVER

Authority changes through a durable state machine

EXECUTABLE ASSET 13-O - CUTOVER STATES AND INVARIANT

```

public enum CutoverState
{
    Discover, ShadowCopy, Compare, CutoverGate,
    NewOwner, LegacyRevoked, Retired, Reversed
}

public sealed class OwnershipCutover
{
    public CutoverState State { get; private set; }
    public WriterAuthority Writer { get; private set; }
    public CutoverWatermark? Watermark { get; private set; }

    public void AcceptNewOwner(
        CutoverWatermark watermark,
        ReconciliationVerdict reconciliation)
    {
        RequireState(CutoverState.CutoverGate);
        if (!reconciliation.Accepted)
            throw new CutoverBlocked("DATA-REC-001");
        if (Writer != WriterAuthority.Legacy)
            throw new CutoverBlocked("DATA-WRITE-001");

        Watermark = watermark;
        Writer = WriterAuthority.Target;
        State = CutoverState.NewOwner;
    }

    public void RevokeLegacy(LegacyGrantEvidence evidence)
    {
        RequireState(CutoverState.NewOwner);
        if (!evidence.WritesDenied)
            throw new CutoverBlocked("DATA-CRED-001");
        State = CutoverState.LegacyRevoked;
    }
}

```

EXECUTABLE ASSET 13-P - COHORT ROUTING GATE

```

[Fact]
public void DATA_CUT_001_each_cohort_has_exactly_one_writer()
{
    var routing = CutoverFixture.RoutingManifest();

    routing.Cohorts.Should().OnlyContain(cohort =>
        cohort.AcceptedWriters.Count == 1);
    routing.Cohorts.SelectMany(x => x.AcceptedWriters)
        .Should().OnlyContain(x =>
            x is WriterAuthority.Legacy
            or WriterAuthority.Target);
}

```

NO DUAL WRITE

Shadow and compare never authorize target business commands. The authority switch is a durable routing/watermark decision, followed by explicit legacy credential revocation.

RECONCILIATION ORACLE

Coverage, meaning, deletes, history, and invariants are separate

EXECUTABLE ASSET 13-Q - LAYERED RESULT

```
public sealed record ReconciliationResult(
    ReconciliationLayer Layer,
    long Compared,
    long Matched,
    IReadOnlyList<Mismatch> Mismatches,
    string SourceWatermark,
    string TargetWatermark);

public enum ReconciliationLayer
{
    Coverage, PerKeyMeaning, Invariants,
    DeletesAndCorrections, History, Freshness, Operations
}

public sealed record ReconciliationVerdict(
    bool Accepted,
    IReadOnlyList<ReconciliationResult> Results,
    IReadOnlyList<string> BlockingReasons);
```

EXECUTABLE ASSET 13-R - PER-KEY CANONICAL COMPARISON

```
public static KeyDigest CanonicalDigest(RootCauseTransferRow row)
{
    var canonical = string.Join("|",
        row.CaseId.ToString("D"),
        row.Status,
        row.FinalCauseCode ?? "<null>",
        row.FinalConfidence?.ToString(
            CultureInfo.InvariantCulture) ?? "<null>",
        row.PolicyIdentity,
        row.LastChangedAt.UtcDateTime.ToString("0"));

    return new(row.CaseId,
        Convert.ToHexString(SHA256.HashData(
            Encoding.UTF8.GetBytes(canonical))));
}
```

EXECUTABLE ASSET 13-S - MISMATCH BLOCKS CUTOVER

```
[Fact]
public async Task DATA_REC_001_unexplained_mismatch_blocks_authority()
{
    var source = ReconciliationFixture.Legacy();
    var target = ReconciliationFixture.TargetWithMissingCorrection();
    var oracle = new RootCauseReconciliationOracle();

    var verdict = await oracle.CompareAsync(source, target);

    verdict.Accepted.Should().BeFalse();
    verdict.Results.Single(x =>
        x.Layer == ReconciliationLayer.DeletesAndCorrections)
        .Mismatches.Should().NotBeEmpty();
    CutoverFixture.Gate(verdict)
        .Should().Throw<CutoverBlocked>();
}
```

REVERSAL AND RETIREMENT

Temporary seams expire only after authority and recovery evidence

EXECUTABLE ASSET 13-T - SEAM CONTRACT

```
public sealed record TransitionSeam(
    string SeamId,
    string Owner,
    IReadOnlySet<string> AdmittedFields,
    string DuplicateRule,
    string DeleteRule,
    string OrderingRule,
    DateOnly RemovalDate,
    IReadOnlySet<string> RemovalEvidence);

[Fact]
public void DATA_SEAM_001_expired_or_unowned_seam_blocks_release()
{
    var seams = TransitionSeamCatalogue.Load();
    var today = new DateOnly(2026, 7, 29);

    seams.Should().OnlyContain(seam =>
        !string.IsNullOrEmpty(seam.Owner)
        && seam.RemovalDate >= today
        && seam.RemovalEvidence.Count > 0);
}
```

EXECUTABLE ASSET 13-U - RECOVERY DECISION

```
public abstract record RecoveryDecision
{
    private RecoveryDecision() { }
    public sealed record ReverseToLegacy(
        CutoverWatermark Watermark,
        LegacyGrantEvidence RestoredWriter) : RecoveryDecision;
    public sealed record ForwardRepair(
        IReadOnlyList<RepairCommand> Commands) : RecoveryDecision;
}

public interface ICutoverRecovery
{
    Task ExecuteAsync(
        RecoveryDecision decision,
        CancellationToken cancellationToken);
    Task<RestoreEvidence> RehearseRestoreAsync(
        BackupIdentity backup,
        CancellationToken cancellationToken);
}
```

EXECUTABLE ASSET 13-V - EVIDENCE RECORD

```
{
    "evidenceId": "DATA-REVIEW-001",
    "chapter": 13,
    "inventoryIdentity": null,
    "runtimeObservationWindow": null,
    "providerBaseline": null,
    "sourceSchemaDigest": null,
    "targetSchemaDigest": null,
    "cutoverWatermark": null,
    "tests": {
        "DATA-INV-001": "PENDING", "DATA-OWN-001": "PENDING",
        "DATA-CRED-001": "PENDING", "DATA-READ-001": "PENDING",
        "DATA-SEAM-001": "PENDING", "DATA-WRITE-001": "PENDING",
        "DATA-CUT-001": "PENDING", "DATA-REC-001": "PENDING",
        "DATA-REV-001": "PENDING", "DATA-LIFE-001": "PENDING"
    },
    "foreignWritesDenied": false,
    "restoreRehearsed": false,
    "accepted": false
}
```

Chapter 13 code-audit result

The chapter now exposes a typed estate inventory, static/runtime access reconciliation, cross-owner dependency blockers, fact-level ownership, separate runtime/migration credentials, one-writer cutover, layered reconciliation, and explicit seam/recovery evidence.

CURRENT VERDICT

The migration is now implementable in shape, but no legacy object or owner has been discovered or moved. DATA/TST/TPL/CMP/LAY/CFG/HLT/BUILD evidence remains pending.

Official source	Stable point used here	NEXUS-1 boundary
Microsoft Azure Architecture Center - Data considerations for microservices	Each service manages private data; other services do not directly access it; a physical server may be shared; copies/projections need explicit consistency and source-of-truth rules.	Ownership is fact/write/lifecycle isolation, not necessarily one physical server per service.
Microsoft Learn - Database Engine permissions	Permissions bind principals to securables; roles/grants/denials are inspectable; least permission is the guiding principle.	Per-service runtime and separate migration identities must prove owned access and foreign denial.
Microsoft Learn - EF Core Applying Migrations	Production migration scripts should be inspected and tested; scripts and bundles are deployment options; runtime schema mutation gives the application elevated rights and operational risk.	Chapter 16 selects and proves the net8.0/provider-specific lifecycle; this chapter only requires independent authority and evidence.
Microsoft Learn - EF Core Transactions	A supported provider applies one SaveChanges call transactionally; shared transactions require the same relational connection and transaction.	Do not extrapolate a local database transaction into cross-service atomicity; Chapter 18 proves the local commit boundary.

Current source pages may describe EF Core releases newer than the book baseline. This volume remains pinned to net8.0 and has not selected a provider or migration package. Exact APIs, generated migrations, permission syntax and deployment artifacts must be checked against the selected net8.0-compatible versions before execution.

Technical material still required

The inventory schema, classifications, target ownership map, dependency types, credential model, one-writer state machine, reconciliation layers and evidence gates are specified. No legacy catalog, runtime query log, job definition, connection inventory, data sample or organizational ownership record was available in this manuscript environment. Therefore none of the seventeen schemas has been assigned, copied or retired.

TECHNICAL MATERIAL STATUS - [LEGACY DATA INVENTORY + OWNERSHIP/CUTOVER/RECONCILIATION EVIDENCE REQUIRED]

Extract and review the seventeen-schema catalog; enumerate all application, job, trigger, procedure, ETL, operator and credential paths; classify facts and commit groups; resolve semantic owners; select the net8.0-compatible provider/migration baseline; define per-service runtime and migration principals; build target schemas and transfer tools; execute shadow copy, per-key/invariant/history reconciliation, one-writer cutover, foreign-write denial, reversal/repair and restore rehearsals; then publish DATA-INV-001 through DATA-REVIEW-001. RootCause remains G0 accepted and G1 specified, not implemented or migration-evidenced.

Deliverable	Minimum technical content	Blocking evidence
Legacy estate inventory	Catalog objects plus writers/readers/credentials/jobs/triggers/procedures, commit groups, data/operations profile and confidence.	DATA-INV-001
Fact ownership map	Authoritative/derived/reference/evidence/technical/unknown classification and semantic owner review.	DATA-OWN-001
Access boundary	Per-service runtime principal, separate migration identity, effective grants/denials and secret rotation.	DATA-CRED-001
Reader/seam migration	Contract/projection replacement, field/freshness/delete rules, telemetry and retirement condition.	DATA-READ-001 / DATA-SEAM-001
Transfer and cutover	Bounded snapshot/tail, checkpoint, cohort/routing, watermark, one writer and legacy revocation.	DATA-WRITE-001 / DATA-CUT-001
Reconciliation	Coverage, per-key meaning, invariants, deletes/corrections, history, freshness and operational result.	DATA-REC-001
Recovery/lifecycle	Reversal or forward repair, backup/restore, RPO/RTO, retention, monitoring and retirement.	DATA-REV-001 / DATA-LIFE-001

CHAPTERS 2-13 - SECOND AUDIT

The thematic audits are complete; execution identity is the remaining gap

Each chapter already owns an eight-page technical audit. The second pass therefore does not duplicate those examples. It adds one common harness that compiles, discovers, executes and correlates them without allowing a chapter to self-certify.

EXECUTABLE ASSET X13-A - CLOSED CHAPTER AUDIT MATRIX

```
public sealed record ChapterAuditContract(
    int Chapter,
    string ClaimNamespace,
    string PrimaryBoundary,
    string RequiredSuite,
    string EvidenceId);

public static class ChaptersTwoToThirteen
{
    public static readonly ChapterAuditContract[] Required =
    {
        new(2, "PRES", "preservation", "Preservation.Tests", "PRES-REVIEW-001"),
        new(3, "SLICE", "selection", "Slice.Tests", "SLICE-REVIEW-001"),
        new(4, "BUILD", "repository", "Architecture.Tests", "BUILD-REVIEW-001"),
        new(5, "SHARE", "package", "Package.Tests", "SHARE-REVIEW-001"),
        new(6, "DDD", "passport", "Passport.Tests", "DDD-REVIEW-001"),
        new(7, "TPL", "template", "Template.Tests", "TPL-REVIEW-001"),
        new(8, "CMP", "composition", "Composition.Tests", "CMP-REVIEW-001"),
        new(9, "LAY", "layers", "Layer.Tests", "LAY-REVIEW-001"),
        new(10, "CFG", "configuration", "Configuration.Tests", "CFG-REVIEW-001"),
        new(11, "HLT", "health", "Health.Tests", "HLT-REVIEW-001"),
        new(12, "TST", "testing", "Acceptance.Tests", "TST-REVIEW-001"),
        new(13, "DATA", "data-move", "Migration.Tests", "DATA-REVIEW-001")
    };
}
```

FINDING

No thematic hole was found in Chapters 2-13. The missing proof was cross-cutting: there was no single identified run proving that every printed asset compiled and that every required suite discovered non-zero tests.

PRINTED-ASSET INVENTORY

Every listing maps to one repository path and test suite

A PDF code block is review material, not compilation evidence. The repository manifest assigns each asset a stable ID, source path, kind and required tests.

EXECUTABLE ASSET X13-B - ASSET DESCRIPTOR AND VALIDATOR

```
public enum AuditAssetKind
{
    Production,
    Test,
    Configuration,
    Script,
    EvidenceSchema
}

public sealed record PrintedAsset(
    int Chapter,
    string AssetId,
    AuditAssetKind Kind,
    string RelativePath,
    string ContentSha256,
    IReadOnlyList<string> RequiredTestIds);

public static class PrintedAssetManifest
{
    public static void Validate(
        IReadOnlyCollection<PrintedAsset> assets)
    {
        assets.Should().OnlyContain(
            x => x.Chapter is >= 2 and <= 13);
        assets.Select(x => x.AssetId)
            .Should().OnlyHaveUniqueItems();
        assets.Select(x => x.RelativePath)
            .Should().OnlyHaveUniqueItems();

        foreach (var asset in assets)
        {
            SafeRepositoryPath.AssertRelativeFile(
                asset.RelativePath);
            HexDigest.AssertSha256(asset.ContentSha256);
            if (asset.Kind is AuditAssetKind.Production
                or AuditAssetKind.Test)
                asset.RequiredTestIds.Should().NotBeEmpty();
        }
    }
}
```

EXECUTABLE ASSET X13-C - REPOSITORY COVERAGE TEST

```
public sealed class PrintedAssetManifestTests
{
    [Fact]
    public void X13_ASSET_001_every_chapter_has_assets_and_tests()
    {
        var assets = AuditFixture.LoadAssets();
        PrintedAssetManifest.Validate(assets);

        ChaptersTwoToThirteen.Required
            .Should().OnlyContain(chapter =>
                assets.Any(x => x.Chapter == chapter.Chapter)
                && assets.Any(x => x.Chapter == chapter.Chapter
                    && x.Kind == AuditAssetKind.Test));
    }
}
```

PINNED BUILD PLAN

Restore, build and test are one ordered evidence transaction

The SDK, test platform and package locks are selected before commands run. Locked restore failure, build failure or report failure ends the transaction; later stages cannot conceal an earlier failure.

EXECUTABLE ASSET X13-D - BUILD PLAN

```
public sealed record AuditCommand(
    string FileName,
    IReadOnlyList<string> Arguments,
    TimeSpan Timeout);

public static class ChaptersAuditBuildPlan
{
    public static readonly AuditCommand[] Commands =
    [
        new("dotnet",
            ["restore", "Nexus1.sln",
             "--locked-mode", "--nologo"],
            TimeSpan.FromMinutes(5)),
        new("dotnet",
            ["build", "Nexus1.sln", "--no-restore",
             "--configuration", "Release",
             "-p:ContinuousIntegrationBuild=true",
             "--nologo"],
            TimeSpan.FromMinutes(10)),
        new("dotnet",
            ["test", "Nexus1.sln", "--no-build",
             "--configuration", "Release",
             "--results-directory", "artifacts/test-results",
             "--logger",
             "trx;LogFilePrefix=chapters-02-13"],
            TimeSpan.FromMinutes(20))
    ];
}
```

EXECUTABLE ASSET X13-E - ORDERED RUNNER

```
public sealed class AuditBuildRunner(IProcessRunner processes)
{
    public async Task<IReadOnlyList<ProcessEvidence>> RunAsync(
        CancellationToken cancellationToken)
    {
        var evidence = new List<ProcessEvidence>();
        foreach (var command in ChaptersAuditBuildPlan.Commands)
        {
            var result = await processes.RunAsync(
                command, cancellationToken);
            evidence.Add(result);
            if (result.ExitCode != 0 || result.TimedOut)
                throw new AuditCommandFailed(
                    command.FileName, result);
        }
        return evidence;
    }
}
```

PINNED RUNNER SEMANTICS

The evidence records whether dotnet test used VSTest or Microsoft.Testing.Platform. TRX availability belongs to the pinned test-platform configuration; the manuscript does not assume that runner selection is interchangeable.

DISCOVERY AND REPORT INTEGRITY

A green exit code cannot hide zero, skipped or missing suites

The report reader binds every discovered test to a required chapter suite. Retries, missing reports and duplicate test identities remain visible rather than being folded into a single pass count.

EXECUTABLE ASSET X13-F - NORMALIZED SUITE RESULT

```
public sealed record ChapterSuiteResult(
    int Chapter,
    string SuiteId,
    string ReportSha256,
    int Discovered,
    int Executed,
    int Passed,
    int Failed,
    int Skipped,
    int Retries,
    IReadOnlySet<string> TestIds);

public static class ChapterSuiteGate
{
    public static void Accept(ChapterSuiteResult result)
    {
        if (String.IsNullOrEmpty(result.ReportSha256))
            throw new UnidentifiedTestReport(result.SuiteId);
        if (result.Discovered <= 0)
            throw new ZeroDiscovery(result.SuiteId);
        if (result.Executed != result.Discovered)
        {
            || result.Passed != result.Executed
            || result.Failed != 0
            || result.Skipped != 0
            || result.Retries != 0)
            throw new IncompleteChapterSuite(result.SuiteId);
        if (result.TestIds.Count != result.Discovered)
            throw new DuplicateOrMissingTestIdentity(
                result.SuiteId);
        }
    }
}
```

EXECUTABLE ASSET X13-G - ZERO DISCOVERY REGRESSION

```
public sealed class ChapterSuiteGateTests
{
    [Fact]
    public void X13_DISC_001_zero_tests_never_become_green()
    {
        var result = AuditFixture.Suite(chapter: 7) with
        {
            Discovered = 0,
            Executed = 0,
            Passed = 0,
            TestIds = new HashSet<string>()
        };

        var act = () => ChapterSuiteGate.Accept(result);
        act.Should().Throw<ZeroDiscovery>();
    }
}
```

ENVIRONMENT EQUIVALENCE

The same tests in a different environment are different evidence

The identity remains value-free and secret-free. It captures the toolchain, source, lock files, test platform and any provider or broker image needed by the selected chapter suites.

EXECUTABLE ASSET X13-H - AUDIT ENVIRONMENT IDENTITY

```
public sealed record AuditEnvironmentIdentity(
    string CommitSha,
    string SdkVersion,
    string RuntimeIdentifier,
    string TestPlatform,
    string PackageLockSha256,
    string SourceManifestSha256,
    IReadOnlyDictionary<string, string> ImageDigests,
    string EnvironmentSha256);

public static class AuditEnvironmentGate
{
    public static void AssertExact(
        AuditEnvironmentIdentity approved,
        AuditEnvironmentIdentity observed)
    {
        var approvedImages = approved.ImageDigests
            .OrderBy(x => x.Key, StringComparer.Ordinal);
        var observedImages = observed.ImageDigests
            .OrderBy(x => x.Key, StringComparer.Ordinal);

        if (observed.CommitSha != approved.CommitSha
            || observed.SdkVersion != approved.SdkVersion
            || observed.RuntimeIdentifier
                != approved.RuntimeIdentifier
            || observed.TestPlatform != approved.TestPlatform
            || observed.PackageLockSha256
                != approved.PackageLockSha256
            || observed.SourceManifestSha256
                != approved.SourceManifestSha256
            || !observedImages.SequenceEqual(approvedImages)
            || observed.EnvironmentSha256
                != approved.EnvironmentSha256)
            throw new AuditEnvironmentMismatch();
    }
}
```

EXECUTABLE ASSET X13-I - ENVIRONMENT MISMATCH TEST

```
public sealed class AuditEnvironmentGateTests
{
    [Fact]
    public void X13_ENV_001_unapproved_sdk_blocks_reuse()
    {
        var approved = AuditFixture.Environment();
        var observed = approved with
        {
            SdkVersion = "different-sdk",
            EnvironmentSha256 = Hashes.Other
        };

        var act = () => AuditEnvironmentGate.AssertExact(
            approved, observed);
        act.Should().Throw<AuditEnvironmentMismatch>();
    }
}
```

NO SECRET VALUES

Connection strings, passwords and tokens never enter the manifest. Only provider identity, image digest and value-free configuration provenance are retained.

CROSS-CHAPTER REFERENTIAL INTEGRITY

Every required chapter owns one result in the same run

A collection of unrelated green files is not a book-level audit. The gate requires one run identity, exact chapter coverage and no duplicate evidence ID.

EXECUTABLE ASSET X13-J - BOOK AUDIT EVIDENCE

```
public enum BookAuditStatus { Pending, Passed, Failed }

public sealed record ChapterAuditEvidence(
    string EvidenceId,
    int Chapter,
    BookAuditStatus Status,
    string RunId,
    string EnvironmentSha256,
    string AssetManifestSha256,
    string ReportSha256,
    int DiscoveredTests);

public static class ChaptersAuditEvidenceGate
{
    public static void Accept(
        IReadOnlyCollection<ChapterAuditEvidence> evidence)
    {
        var required = ChaptersTwoToThirteen.Required;
        var requiredChapters = required
            .Select(x => x.Chapter).ToHashSet();
        if (!evidence.Select(x => x.Chapter)
            .ToHashSet().SetEquals(requiredChapters))
            throw new MissingOrUnexpectedChapterEvidence();
        if (evidence.GroupBy(x => x.Chapter)
            .Any(group => group.Count() != 1))
            throw new DuplicateChapterEvidence();
        if (evidence.GroupBy(x => x.EvidenceId,
            StringComparer.Ordinal)
            .Any(group => group.Count() != 1))
            throw new DuplicateEvidenceIdentity();

        var runs = evidence.Select(x => x.RunId)
            .Distinct(StringComparer.Ordinal).ToArray();
        var environments = evidence
            .Select(x => x.EnvironmentSha256)
            .Distinct(StringComparer.Ordinal).ToArray();
        if (runs.Length != 1 || environments.Length != 1)
            throw new MixedAuditRun();

        foreach (var chapter in required)
        {
            var item = evidence.Single(
                x => x.Chapter == chapter.Chapter);
            if (item.EvidenceId != chapter.EvidenceId
                || item.Status != BookAuditStatus.Passed
                || item.DiscoveredTests <= 0)
                throw new UnacceptedChapterEvidence(
                    chapter.Chapter);
        }
    }
}
```

ATOMIC ACCEPTANCE

The book-level gate either accepts all twelve chapter results from one identified run or accepts none. A later chapter cannot hide a missing or failed earlier suite.

FAILURE-PATH ACCEPTANCE

The common gate is tested against false-green evidence

The validator is not trusted because it exists. Its tests explicitly falsify the shortcuts most likely to make a cross-chapter audit look complete.

EXECUTABLE ASSET X13-K - GATE REGRESSION SUITE

```
public sealed class ChaptersAuditEvidenceGateTests
{
    [Fact]
    public void X13_GATE_001_missing_chapter_is_rejected()
    {
        var evidence = AuditFixture.BookEvidence()
            .Where(x => x.Chapter != 6).ToArray();
        var act = () =>
            ChaptersAuditEvidenceGate.Accept(evidence);
        act.Should()
            .Throw<MissingOrUnexpectedChapterEvidence>();
    }

    [Fact]
    public void X13_GATE_002_pending_result_is_rejected()
    {
        var evidence = AuditFixture.Replace(
            chapter: 10,
            x => x with { Status = BookAuditStatus.Pending });
        var act = () =>
            ChaptersAuditEvidenceGate.Accept(evidence);
        act.Should().Throw<UnacceptedChapterEvidence>();
    }

    [Fact]
    public void X13_GATE_003_mixed_environment_is_rejected()
    {
        var evidence = AuditFixture.Replace(
            chapter: 12,
            x => x with
            {
                EnvironmentSha256 = Hashes.Other
            });
        var act = () =>
            ChaptersAuditEvidenceGate.Accept(evidence);
        act.Should().Throw<MixedAuditRun>();
    }

    [Fact]
    public void X13_GATE_004_zero_discovery_is_rejected()
    {
        var evidence = AuditFixture.Replace(
            chapter: 4,
            x => x with { DiscoveredTests = 0 });
        var act = () =>
            ChaptersAuditEvidenceGate.Accept(evidence);
        act.Should().Throw<UnacceptedChapterEvidence>();
    }
}
```

HONEST SCOPE

These tests validate the gate's logic. They do not execute the chapter assets, start providers, prove production deployment or turn the PENDING record green.

CHAPTERS 1-13 EVIDENCE CLOSURE

The second audit records what changed and what did not run

EXECUTABLE ASSET X13-L - CONSOLIDATED RE-AUDIT RECORD

```
{
  "evidenceId": "X13-REVIEW-001",
  "scope": "chapters-01-13",
  "chapter1HandoffManifestSha256": null,
  "assetManifestSha256": null,
  "commitSha": null,
  "sdkVersion": null,
  "testPlatform": null,
  "packageLockSha256": null,
  "environmentSha256": null,
  "runId": null,
  "chapterEvidence": {
    "1": "PENDING", "2": "PENDING",
    "3": "PENDING", "4": "PENDING",
    "5": "PENDING", "6": "PENDING",
    "7": "PENDING", "8": "PENDING",
    "9": "PENDING", "10": "PENDING",
    "11": "PENDING", "12": "PENDING",
    "13": "PENDING"
  },
  "discoveredTests": null,
  "executedTests": null,
  "passedTests": null,
  "failedTests": null,
  "skippedTests": null,
  "accepted": false
}
```

Re-audit result

Chapter 1 now has an evidence-oriented audit. Chapters 2-13 retain their existing technical additions and gain one common asset, build, discovery, environment and book-level evidence boundary. No existing material was removed or compressed.

Checked against the current .NET CLI model: SDK selection is pinned through global.json, locked restore must fail on dependency drift, ContinuousIntegrationBuild normalizes official-build paths, and test-runner identity remains explicit.

CURRENT VERDICT

The missing structures are now supplied, but no repository run was performed. Every HND and X13 result remains PENDING; the book makes no new green claim.

Deliverable	Minimum technical content	Blocking evidence
Acceptance	Identity-bound evidence catalogue, multidisciplinary review, exceptions, residual risk and expiry.	DATA-REVIEW-001

Implementation sequence

1. Accept ADR-011 and publish the inventory schema, classification vocabulary, evidence identities and unresolved-item policy.
2. Extract the seventeen-schema catalog, dependencies, sizes and database principals; inventory application configuration, stored code, jobs, ETL and operator paths.
3. Observe representative runtime access and reconcile it with static discovery; label dormant/uncertain paths instead of declaring them absent.
4. Split mixed tables into fact-level data units; identify authoritative meaning, semantic owner, commit group, history, sensitivity and operational obligations.
5. Map AlarmManagement, RootCause, Audit, Compliance and Reporting units to source, projection, reference or evidence roles; keep contested units blocked.
6. For the first bounded unit, design the target owner command path, database scope, runtime/migration principals, transition seam and reader replacements.
7. Define snapshot/cursor/tail, transformation, checkpoint, delete/correction/history semantics and the complete reconciliation oracle before copying.
8. Rehearse shadow copy, comparison, cohort cutover, legacy write revocation, failure injection, controlled reversal/forward repair and restore.
9. Execute the approved one-writer cutover with durable routing/watermark evidence; monitor, reconcile and resolve every unexplained difference.
10. Remove foreign reads, temporary seams and legacy grants only after retention, restore and review gates pass; publish DATA-INV-001 through DATA-REVIEW-001.

What this chapter has proved

Chapter 13 has converted the phrase service-owned database into a reviewable migration contract: facts are classified; semantic, write, schema and operational ownership are separated; hidden dependency types are inventoried; credentials enforce the boundary; temporary seams have retirement gates; one writer is preserved through cutover; and reconciliation covers identity, meaning, invariants, history and operations.

It has not discovered the actual seventeen-schema estate, assigned any real object, selected a provider, generated a migration, created AlarmManagementDb or RootCauseDb, copied data, compared a key, changed a grant, rehearsed recovery or retired a dependency. RootCause remains G0 accepted and G1 specified. DATA evidence is pending, and the detailed database implementations begin next.

PART III OPENING

Data ownership is not achieved when a table changes address. It is achieved when authority, write path, migration identity, failure lifecycle and every consuming contract can change independently without creating a second truth.

Chapter checkpoint

You are ready to build the first owned database when the source facts, writers, readers, commit groups and operational obligations are known; one semantic owner is accepted; the transition preserves one writer; and reconciliation/reversal can falsify a bad move before production truth changes hands.

Question	Expected answer
Do seventeen schemas imply seventeen services?	No. They are discovery inputs; one schema may mix owners and one owner may span schemas.
What is the migration unit?	A bounded set of authoritative facts, invariants, history and technical coordination that can move under one writer and one reconciliation definition.
What makes a database service-owned?	Semantic/write/schema lifecycle authority, private runtime/migration principals, denied foreign writes and owned backup/restore/retirement.
May services share a physical database server?	Yes, if catalogs/schemas, credentials, migrations and lifecycle remain isolated; shared tables or write authority are not accepted.
Why is application dual-write rejected?	Independent commits and retries can diverge, leaving two candidate truths with no atomic business commit.
What proves a safe copy?	Bounded coverage plus per-key meaning, invariants, deletes/corrections, history, freshness and operational reconciliation.
When can authority cut over?	After comparison passes, routing/freeze and watermark are durable, exactly one writer is enabled and reversal/repair is rehearsed.
What replaces foreign SQL reads?	An owner API for accepted current coupling or a local projection/reference copy with explicit freshness and rebuild semantics.
What is the current RootCause gate?	G0 accepted; G1 specified; no RootCauseDb, migration, inventory or cutover execution evidence exists.
What does Chapter 14 add?	The executable design of AlarmManagementDb: owned model, mappings, constraints, persistence ports, migrations and provider evidence.

NEXT CHAPTER

Chapter 14, Building AlarmManagementDb, takes the first accepted source-of-truth target and defines its executable persistence boundary: aggregates and tables, mappings, constraints, identities, concurrency, pending public facts, migration ownership and the provider-level evidence required before AlarmManagement can own production data.

PART III - DATA OWNERSHIP IN EXECUTABLE FORM

14. Building AlarmManagementDb

AlarmManagementDb is the first service-owned source store in the reference slice. It must preserve accepted alarm identity, current alarm truth, deterministic flood decisions, exact decision evidence and pending public facts in one local transaction—without exposing its tables as an integration API.

CHAPTER ARGUMENT

Model persistence around AlarmManagement meanings and failure rules, not around the seventeen legacy schemas. One accepted source event produces at most one observation; concurrent detector work reloads owned truth; a flood decision records its configuration and evidence identity; and publication intent commits beside the decision before any broker interaction begins.

Chapter 13 established the migration contract: facts are classified, owners are explicit, credentials enforce the boundary, cutover preserves one writer and reconciliation proves meaning. This chapter supplies the target persistence design for the first accepted source-of-truth destination.

AlarmManagement owns alarm intake identity, accepted alarm state and flood detection. RootCause may consume AlarmFloodDetected.v1 later, but it never reads or writes AlarmManagementDb.

The preservation rule ALM-DET-001 remains binding: the same ordered accepted inputs and configuration identity produce the same flood decision. DATA-OWN-001 still prohibits foreign writes. FLOW-IDEM-001 limits a repeated delivery to one business effect. The design must therefore store more than a current Alarm row: it needs durable input identity, reproducible decision evidence, concurrency state and a local publication intent.

No AlarmManagement projects, provider package, database image, connection, migration, production credential or legacy mapping exists in this manuscript environment. Every C# fragment below is a net8.0 reference contract. It has not compiled. Provider-specific types, SQL, error codes, collation, time precision, isolation behaviour and migration output remain blocking choices that real component tests must settle.

Start from one owned use case

The persistence model is driven by AcceptAlarmObservation, not by a generic CRUD surface. The application authenticates and validates the intake boundary, normalizes a producer-owned source identity, loads the owned alarm and detector scope, invokes the deterministic Domain policy, records the accepted observation, persists any state transition and detection evidence, and adds one pending public fact when the Domain emits a new flood decision. One SaveChanges call is the intended commit point.

Step	Owned decision	Durable result	Must not occur
Identify	Map source system + source event/alarm IDs to stable local identities.	Unique intake identity and owned AlarmId.	Use a broker delivery tag, table sequence or RootCause ID as alarm truth.
Order	Create the canonical comparison key for accepted facts.	Occurred time plus stable tie-breakers and stored policy identity.	Let arrival thread, host clock or retry count choose detection order.
Load	Read the current managed alarm and detector scope/version.	One aggregate/detector snapshot for the attempted decision.	Read RootCause, Reporting or another schema inside the decision.
Decide	Apply ALM-DET-001 to the exact ordered input set and configuration identity.	Alarm transition; optional FloodDetection and evidence membership.	Recalculate a different result because environment or queue order changed.
Commit	Persist owned truth and publication intent atomically.	Observation, state, detector version, detection/evidence and pending fact—all or none.	Publish to broker or write Audit/RootCause inside the database transaction.

Step	Owned decision	Durable result	Must not occur
Return	Report Accepted, Duplicate or Conflict using stable application outcomes.	Identity/correlation and enough detail for safe retry/diagnosis.	Expose EF/provider exception text or private row shape.

One database, six private responsibilities

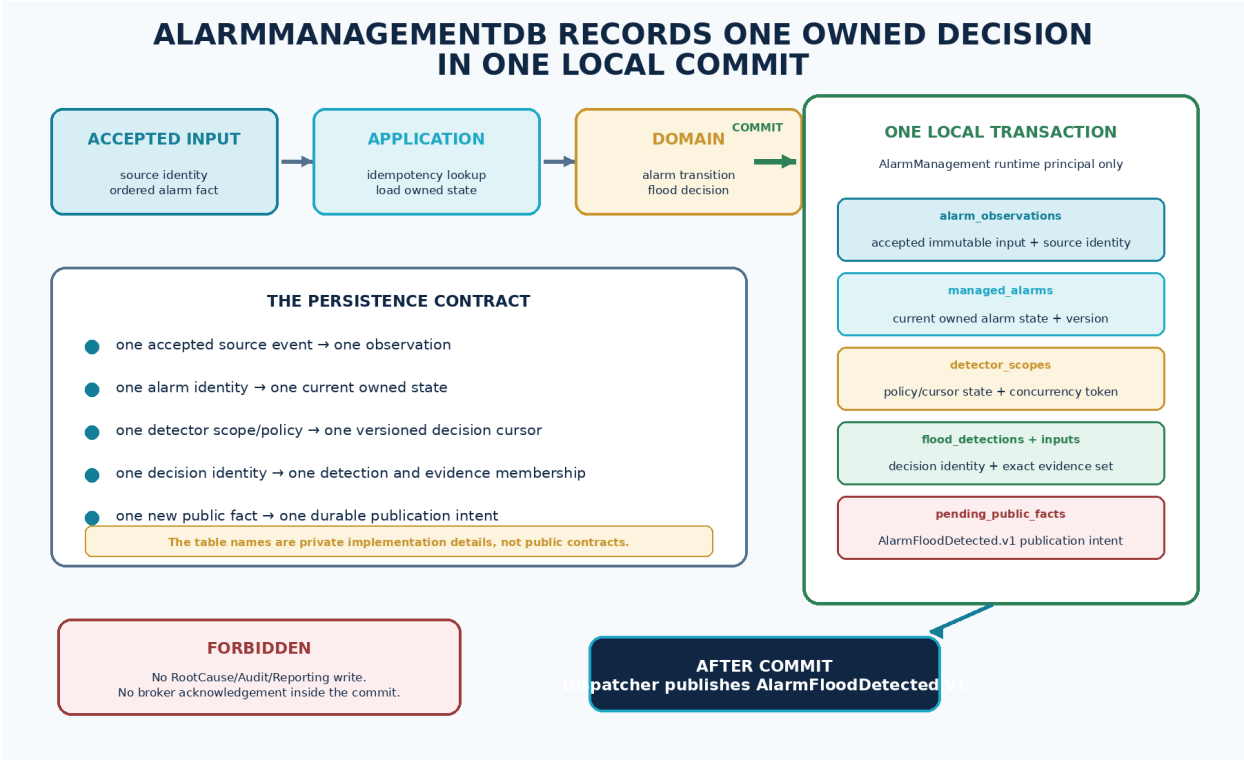


Figure 14.1 - AlarmManagementDb keeps accepted input, current truth, detector concurrency, decision evidence and public intent inside one owned commit.

Private relation	Responsibility	Key / identity	Mutability
alarm_observations	Immutable accepted alarm facts after boundary validation and normalization.	ObservationId PK; unique SourceSystem + SourceEventId.	Insert-only; correction is a new accepted fact with lineage.
managed_alarms	Current AlarmManagement-owned state for one source alarm.	AlarmId PK; unique SourceSystem + SourceAlarmId.	Versioned state transition; no foreign updates.
detector_scopes	Per detection scope/policy cursor and concurrency serialization point.	ScopeId PK; unique ScopeKey; ConcurrencyVersion token.	Updated on every decision-affecting accepted input.
flood_detections	Immutable record that a specific policy decided a flood for a bounded input set.	DetectionId PK; unique DecisionIdentity.	Insert-only; correction/supersession is explicit new history.
flood_detection_inputs	Exact ordered observation membership used by one detection.	DetectionId + Position PK; unique DetectionId + ObservationId.	Insert-only with owning detection.
pending_public_facts	Durable producer intent for a public fact after local truth commits.	PublicFactId PK; unique producer fact identity.	Dispatch metadata may advance; payload/meaning is immutable.

PRIVATE SCHEMA RULE

Relation and column names are replaceable implementation details. Public consumers receive AlarmFloodDetected.v1 under the producer-owned contract; they never receive entity classes, EF migrations or direct database access.

The aggregate boundary is not the DbContext

A DbContext tracks persistence work; it is not the domain aggregate and does not own flood policy. AlarmManagement Domain defines ManagedAlarm, DetectorScope, FloodDetection and the accepted policy snapshot. Application owns purpose-built ports. Infrastructure maps those meanings to rows and EF Core. The mapping may use persistence records to keep EF details isolated, but it cannot become an alternate decision engine.

Layer	Owns	Forbidden persistence shortcut
Domain	Alarm transitions, canonical decision input, flood policy, decision/evidence identity.	DbContext, IQueryable, provider types, annotations, connection or migration APIs.
Application	AcceptAlarmObservation use case, AlarmManagement-specific store/unit-of-work ports and stable outcomes.	Generic IRepository<T>, provider exception codes or raw SQL semantics.
Infrastructure	DbContext, entity configurations, row/domain mapping, unique/concurrency translation and migrations.	Choosing whether a flood exists or changing policy identity on retry.
Host / Edge	Registration, boundary mapping, authorization metadata and problem/outcome mapping.	Resolving DbContext in endpoints or performing multi-step business persistence.

PURPOSE-BUILT APPLICATION PORT - [NEW][COMPILE REQUIRED]

```
// Nexus1.AlarmManagement.Application – no EF Core dependency.
public interface IAlarmManagementStore
{
    Task<AlarmDecisionState> LoadDecisionStateAsync(
        AlarmSourceIdentity alarm,
        DetectorScopeKey scope,
        CancellationToken cancellationToken);

    void AddAcceptedObservation(AcceptedAlarmObservation observation);
    void Track(ManagedAlarm alarm);
    void Track(DetectorScope detector);
    void Add(FloodDetection detection);
    void Add(PendingPublicFact publicFact);

    Task CommitAsync(CancellationToken cancellationToken);
}
```

NO GENERIC REPOSITORY

The port names the atomic AlarmManagement operation. Exposing IQueryable or SaveChanges through a shared abstraction would leak query, tracking and transaction semantics while hiding the decision boundary.

Identity is layered and explicit

Database identity, source identity, public-fact identity and correlation identity solve different problems. They are never collapsed into one convenient GUID. SourceEventId stops a duplicate intake. AlarmId is the local aggregate identity. DecisionIdentity proves that one exact ordered evidence set under one configuration produced one decision. PublicFactId identifies the producer fact for later dispatch/idempotency. Correlation and causation explain the flow but do not provide uniqueness by themselves.

Identity	Created / owned by	Durable uniqueness	Not used as
ObservationId	AlarmManagement at accepted intake.	Primary key; maps one normalized accepted source fact.	Producer message identity or ordering key.
SourceSystem + SourceEventId	Upstream contract; normalized by AlarmManagement.	Unique index prevents repeated delivery becoming a second observation.	Local aggregate primary key or authorization identity.
AlarmId	AlarmManagement		

Identity	Created / owned by	Durable uniqueness	Not used as
ScopeId / ScopeKey	AlarmManagement detector partitioning policy.	One versioned state row per canonical scope.	Physical shard/server address exposed to consumers.
DecisionIdentity	Domain from scope + policy identity + exact ordered evidence identity.	Unique index prevents a repeated computation becoming a second flood.	Hash of mutable database serialization or environment.
PublicFactId	AlarmManagement producer when a new public fact is admitted.	Unique pending-public-fact identity retained through dispatch.	Broker delivery tag or downstream inbox identity.
Correlation / causation	Boundary and flow context.	Indexed for evidence navigation; duplicates are legitimate.	Source-event deduplication or database key.

The persistence model is explicit

The reference uses EF Core metadata in Infrastructure and keeps the Domain free of mapping attributes. Fluent configurations are one class per persistence type so keys, required fields, lengths, relationships, delete behaviour, concurrency tokens and indexes are reviewable. Applying all configurations from the Infrastructure assembly is accepted only when configuration constructors are parameterless and configuration order is irrelevant.

ALARMMANAGEMENTDBCONTEXT - [NEW][EF PACKAGE + COMPILE REQUIRED]

```
internal sealed class AlarmManagementDbContext(
    DbContextOptions<AlarmManagementDbContext> options) : DbContext(options)
{
    internal DbSet<AlarmObservationRow> AlarmObservations => Set<AlarmObservationRow>();
    internal DbSet<ManagedAlarmRow> ManagedAlarms => Set<ManagedAlarmRow>();
    internal DbSet<DetectorScopeRow> DetectorScopes => Set<DetectorScopeRow>();
    internal DbSet<FloodDetectionRow> FloodDetections => Set<FloodDetectionRow>();
    internal DbSet<FloodDetectionInputRow> FloodDetectionInputs => Set<FloodDetectionInputRow>();
    internal DbSet<PendingPublicFactRow> PendingPublicFacts => Set<PendingPublicFactRow>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.ApplyConfigurationsFromAssembly(
            typeof(AlarmManagementDbContext).Assembly);
    }
}
```

DbSet visibility remains internal to Infrastructure. Application receives the purpose-built store, not the DbContext. The Host binds one scoped DbContext/store per unit of work. Background intake creates a scope per bounded batch or message according to the composition rules from Chapter 8; it never captures the scoped context in a singleton worker.

Keys, relationships and constraints carry meaning

PROVIDER-NEUTRAL MAPPING INTENT - [NEW][COMPILE REQUIRED]

```
internal sealed class AlarmObservationConfiguration
    : IEntityTypeConfiguration<AlarmObservationRow>
{
    public void Configure(EntityTypeBuilder<AlarmObservationRow> b)
    {
        b.ToTable("alarm_observations");
        b.HasKey(x => x.ObservationId);
        b.Property(x => x.SourceSystem).HasMaxLength(80).IsRequired();
        b.Property(x => x.SourceEventId).HasMaxLength(160).IsRequired();
        b.Property(x => x.SourceAlarmId).HasMaxLength(160).IsRequired();
        b.Property(x => x.ScopeKey).HasMaxLength(200).IsRequired();
        b.Property(x => x.PolicyIdentity).HasMaxLength(128).IsRequired();
        b.Property(x => x.NormalizedState).HasMaxLength(40).IsRequired();
        b.HasIndex(x => new { x.SourceSystem, x.SourceEventId }).IsUnique();
        b.HasIndex(x => new { x.ScopeKey, x.OccurredAtUtc, x.SourceSystem, x.SourceEventId });
    }
}

internal sealed class DetectorScopeConfiguration
    : IEntityTypeConfiguration<DetectorScopeRow>
{
    public void Configure(EntityTypeBuilder<DetectorScopeRow> b)
    {
        b.ToTable("detector_scopes");
        b.HasKey(x => x.ScopeId);
        b.Property(x => x.ScopeKey).HasMaxLength(200).IsRequired();
        b.HasIndex(x => x.ScopeKey).IsUnique();
        b.Property(x => x.PolicyIdentity).HasMaxLength(128).IsRequired();
        b.Property(x => x.ConcurrencyVersion).IsConcurrencyToken();
    }
}
```

Lengths and names are reference limits to be validated against actual upstream contracts and provider indexing rules. Time precision, case sensitivity, collation, GUID storage, generated-value behaviour and check-constraint SQL are provider concerns. The model contract states the requirement; the selected provider configuration and migration must demonstrate the exact physical enforcement.

Rule	Database enforcement	Application / Domain companion
One accepted source event	Unique SourceSystem + SourceEventId.	Duplicate outcome reloads the existing accepted result; provider violation is classified narrowly.
One source alarm	Unique SourceSystem + SourceAlarmId on managed_alarms.	Source alias/merge is an explicit correction process, not a key update.
One detector state per scope	Unique ScopeKey and concurrency token.	Every decision-affecting input advances a new token before commit.
One flood decision	Unique DecisionIdentity.	Domain derives identity from canonical scope, policy and ordered evidence membership.
Exact evidence order	Composite PK DetectionId + Position; unique DetectionId + ObservationId.	Domain supplies contiguous positions; component test rejects duplicate/missing membership.
Owned relationships	FKs exist only within AlarmManagementDb; restrict/cascade behaviour is explicit per relation.	No RootCause/Audit/Reporting entity navigation or cross-database FK.
Valid state vocabulary	Required bounded string plus provider-specific check constraint when selected.	Domain refuses unknown transition before persistence; database protects alternate paths.
Immutable evidence/public meaning	No normal update path; restrictive privileges where supported.	Repository exposes add/read only; correction creates linked new history.

Ordering is a domain contract

ALM-DET-001 cannot rely on whichever message reached a worker first. AlarmManagement defines a total order for admitted facts: occurred-at instant normalized to UTC, then an owner-approved source ordering field when one exists, then stable SourceSystem and SourceEventId tie-breakers. RecordedAtUtc is operational evidence and never substitutes for domain occurrence. A late fact is

admitted or rejected by policy and may cause a new explicit decision; it never rewrites the evidence identity of a decision already issued.

Ordering field	Purpose	Failure rule
OccurredAtUtc	Primary domain occurrence order after strict timezone/precision normalization.	Missing/invalid time receives a boundary outcome; host time is not silently substituted.
SourceSequence	Optional producer-owned ordering inside one source stream when its semantics are guaranteed.	Never compare incomparable sequences from different sources.
SourceSystem + SourceEventId	Stable deterministic tie-breaker and replay identity.	Normalization/case rules are pinned and tested on the production provider.
RecordedAtUtc	Operational evidence for intake delay and observability.	Not part of flood truth unless the accepted policy explicitly says so.
PolicyIdentity	Binds thresholds, window, severity filters, lateness and correction behaviour.	No fallback to latest policy during replay or concurrency retry.
DecisionIdentity	Binds scope, policy and exact ordered observation identities into one result identity.	Canonicalization/hash version is explicit; serialization drift cannot invent a second decision.

DETERMINISM BOUNDARY

The database stores the inputs and identities required to replay the decision; it does not decide the flood. A SQL COUNT over arrival order is not a replacement for the accepted Domain policy.

One SaveChanges is the intended commit point

When the selected relational provider supports transactions, EF Core applies all tracked changes in one SaveChanges call transactionally. AlarmManagement uses that default for the normal intake path: observation, current alarm state, detector concurrency token, optional detection/membership and pending public fact succeed together or the database remains unchanged. An explicit transaction is introduced only when a proven use case requires multiple SaveChanges calls or provider operations, and its retry strategy is designed deliberately.

ATOMIC INTAKE SHAPE - [NEW][COMPILE + REAL PROVIDER TEST REQUIRED]

```
public async Task<AcceptAlarmOutcome> Handle(
    AcceptAlarmObservation command,
    CancellationToken cancellationToken)
{
    var state = await store.LoadDecisionStateAsync(
        command.AlarmIdentity,
        command.Scope,
        cancellationToken);

    if (state.HasAccepted(command.SourceEventIdentity))
        return AcceptAlarmOutcome.Duplicate(state.ExistingObservationId);

    var result = AlarmFloodPolicy.Accept(
        state,
        command.ToAcceptedObservation(),
        acceptedPolicy);

    store.AddAcceptedObservation(result.Observation);
    store.Track(result.Alarm);
    store.Track(result.Detector);

    if (result.NewDetection is { } detection)
    {
        store.Add(detection);
        store.Add(PendingPublicFact.From(detection));
    }

    await store.CommitAsync(cancellationToken); // one local SaveChangesAsync
    return AcceptAlarmOutcome.Accepted(result.Observation.ObservationId);
}
```


Failure point	Required durable state	Outcome
Before SaveChanges	No new row or version.	Safe cancellation/failure; caller may retry with same source identity.
Observation insert fails	No alarm/detector/detection/public-fact change.	Provider failure translated; no partial accepted truth.
Detector concurrency conflict	The entire attempted commit rolls back.	Dispose context, reload owned truth and perform bounded semantic retry.
Unique source identity conflict	The winning accepted observation remains; loser adds no second effect.	Classify exact constraint, reload and return Duplicate.
Unique decision identity conflict	One detection/public fact exists.	Reload and verify semantic equivalence; mismatch is a blocking defect.
Commit succeeds; process crashes	Owned truth and pending public fact both survive.	Later dispatcher resumes; broker state cannot roll back the local decision.
Broker unavailable	Local commit remains; pending fact stays durable under capacity policy.	No database rollback; readiness/backpressure follows Chapter 11 rules.

Optimistic concurrency serializes one detector scope

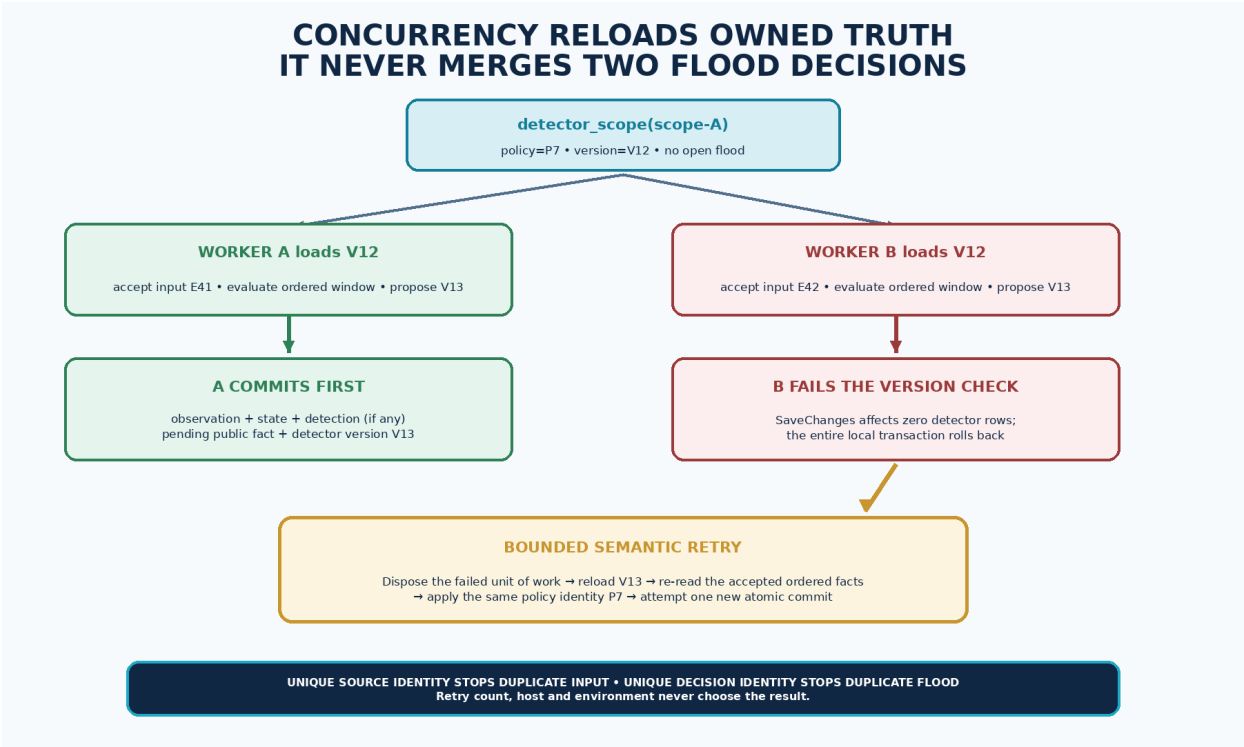


Figure 14.2 - A concurrency loser rolls back the whole local attempt, reloads the winning truth and replays the same policy under a bounded retry.

The `detector_scope` row is a concurrency serialization point, not a global lock. Two service instances may evaluate the same starting version, but only one can update that version. EF Core includes the original concurrency token in the update predicate; a zero-row update raises `DbUpdateConcurrencyException`. The loser discards its tracked attempt, creates a fresh unit of work, reloads the accepted facts and policy identity, and re-runs the Domain decision. It does not merge tracked row values or keep a pending detection created from stale evidence.

Concurrency rule	Reference policy	Blocking evidence
Token scope	DetectorScope concurrency token covers every change that can alter flood result; ManagedAlarm has its own version for alarm transitions.	Generated SQL/update predicate and two-writer provider test.
Retry ownership	Application use case owns a small bounded semantic retry; infrastructure exposes classified Conflict/Duplicate outcomes.	Attempt count, elapsed budget and final state/result evidence.

Concurrency rule	Reference policy	Blocking evidence
Fresh unit of work	Failed tracked graph is disposed; retry reloads current database values and exact accepted policy.	No reuse of stale CurrentValues or pending public fact.
No decision by retry	Environment, host, delay and retry count are excluded from the flood policy and DecisionIdentity.	ALM-DET-001 replay matrix across concurrent schedules.
Exhaustion	After the accepted retry budget, return stable contention outcome and retain no partial change.	No hidden infinite loop; metric/trace and safe client/queue policy.
Provider token	Application-managed GUID token is the portable reference; native rowversion/xmin-style choice requires provider ADR and tests.	Exact selected mapping, generated SQL and restart behaviour.

Duplicate intake is not a concurrency exception

EF Core reports optimistic update/delete conflicts through `DbUpdateConcurrencyException`. Concurrent inserts that violate a unique source or decision constraint normally surface as provider-specific database errors. Infrastructure must therefore classify only known constraint identities or provider error codes; it must not catch every `DbUpdateException` and label it Duplicate. Unknown database failures remain failures and preserve diagnostics without leaking secrets or values.

NARROW PROVIDER-ERROR TRANSLATION - [NEW][PROVIDER ADAPTER REQUIRED]

```
public async Task CommitAsync(CancellationTokentoken)
{
    try
    {
        await db.SaveChangesAsync(cancellationTokentoken);
    }
    catch (DbUpdateConcurrencyException ex)
    {
        throw new AlarmDecisionConflictException(
            "The owned detector state changed before commit.", ex);
    }
    catch (DbUpdateException ex) when (
        providerErrors.IsUniqueViolation(ex, KnownConstraint.SourceEventIdentity))
    {
        throw new AlarmObservationAlreadyAcceptedException(ex);
    }
}
```

DO NOT OVER-CATCH

Constraint-name/error-code classification is part of the selected provider adapter and component suite. Message-text matching and catch-all Duplicate outcomes are rejected.

Queries and indexes follow owned operations

Indexes are justified by the command and evidence paths, not generated by habit. The intake lookup needs the source-event unique index. Managed alarm loading needs the source-alarm unique index. Detector replay needs scope plus canonical order. Pending publication needs a bounded status/next-attempt scan. Audit/reconciliation needs decision, correlation and time access. Every additional index adds write and migration cost, so its query, order and selectivity are captured before acceptance.

Owned query	Candidate index	Verification
Duplicate source-event lookup	UNIQUE (source_system, source_event_id).	Parallel duplicate test; explain/plan uses seek; normalization/collation correct.
Current alarm by source identity	UNIQUE (source_system, source_alarm_id).	One row under concurrent first sight; correction policy explicit.
Detector state by scope	UNIQUE (scope_key).	Concurrency update includes PK/unique lookup and original version.

Owned query	Candidate index	Verification
Window facts in canonical order	(scope_key, occurred_at_utc, source_system, source_event_id), with source sequence if admitted.	Provider plan, boundary timestamps, late facts and deterministic ties.
Detection evidence	UNIQUE decision_identity; membership PK (detection_id, position).	Exact ordered replay and duplicate-decision scenario.
Pending dispatch	Provider-appropriate index over pending state, next attempt and creation order.	Bounded batch query under backlog; no table scan at accepted capacity.
Evidence navigation	Correlation/causation and occurred time indexes only when a real support/reconciliation query exists.	Representative volume plan and retention/partition impact.

History and correction remain first-class

alarm_observations and flood_detections are evidence-bearing records. A late alarm, upstream correction or policy supersession does not update the old observation or detection as if it never existed. It creates a new accepted fact with causation/supersession lineage and, when Domain policy admits it, a new decision/public fact. managed_alarms and detector_scopes remain current operational state; the append-only relations explain how that state was reached.

Data class	Retention / correction rule	Restore implication
Accepted observation	Retain for the accepted replay/audit window; correction appends linked observation.	Restore must preserve source identities or replay creates duplicates.
Managed alarm current state	Keep while operationally relevant; terminal retention is policy-owned.	May be reconstructed only if the complete accepted history and policy identities are available/proved.
Detector scope	Keep version/current policy cursor while scope can receive input.	Restore with observations/detections to one consistent database point.
Flood detection + membership	Immutable decision evidence; supersession references prior DetectionId.	Restore/reconciliation proves exact membership and decision identity.
Pending public fact	Retain until accepted dispatch/operational retention; payload meaning immutable.	Restore cannot republish as a new fact ID; duplicates remain downstream-safe.
Migration history	Owned by AlarmManagement migration stream and never shared with RootCause.	Restore target must match an accepted schema/migration identity before traffic.

The initial migration belongs to AlarmManagement

The initial EF migration is generated only after the provider, compatible EF Core 8 packages, relational names, precision/collation rules and constraints are pinned. It lives with the AlarmManagement persistence owner, carries its own model snapshot/history and is deployed by a separate migration identity. The running service does not call EnsureCreated or auto-migrate at startup in production. Chapter 16 defines the complete lifecycle, scripts/bundles, expand-contract policy and restore gates.

INITIAL MIGRATION WORKFLOW - [REFERENCE][PROVIDER + EXECUTION REQUIRED]

```
# Reference commands; not execution evidence.
dotnet ef migrations add InitialAlarmManagement --project
src/Contexts/AlarmManagement/Nexus1.AlarmManagement.Infrastructure --startup-project
src/Hosts/Nexus1.AlarmManagement.Host --context AlarmManagementDbContext

dotnet ef migrations script 0 InitialAlarmManagement --project
src/Contexts/AlarmManagement/Nexus1.AlarmManagement.Infrastructure --startup-project
src/Hosts/Nexus1.AlarmManagement.Host --context AlarmManagementDbContext --output
artifacts/migrations/alarm-management-initial.sql
```

Migration check	Required proof	False green
Model completeness	Every required relation, key, unique/concurrency constraint FK action length/precision and index is pre:	Migration generated successfully from an

Migration check	Required proof	False green
Empty create	Apply from zero on a clean production-provider database; compare accepted schema identity.	EnsureCreated produced tables without migration history.
Upgrade	Apply from every supported prior migration with representative data and time/lock budget.	Only clean create passes.
Review	Inspect generated migration and SQL for drops, implicit defaults, provider types, locks and destructive changes.	Generated code is trusted without human/provider review.
Runtime permissions	Runtime principal performs owned operations but cannot alter schema; migration principal is short-lived and scoped.	Service starts with db_owner/administrator credentials.
Failure / repair	Interrupt at controlled points; identify resulting migration state and rehearse forward repair/reversal policy.	One happy application with no failure evidence.
Restore compatibility	Restore backup, validate migration identity, start compatible artifact and replay pending facts safely.	Backup file exists but has never restored.

Credentials and operations complete ownership

Capability	Runtime principal	Migration / operations principal
DML	Only required SELECT/INSERT/UPDATE on AlarmManagement-owned objects; DELETE only where policy explicitly allows.	Repair/maintenance operations through reviewed runbook, never routine application path.
DDL	Denied/absent.	Apply reviewed AlarmManagement migration artifact to its database only.
Foreign databases	No RootCause/Audit/Compliance/Reporting write; normally no direct read.	No unrelated database authority.
Secrets	Resolved through the Chapter 10 secret handle; values absent from logs, manifests and evidence.	Short-lived/rotated identity; connection detail redacted from migration transcript.
Health	Readiness checks a bounded local capability and capacity signal; no deep remote service query.	Operations validates schema identity, backup age and restore evidence out of request path.
Backup / restore	No authority to modify backup policy.	Owned RPO/RTO, encrypted backup, restore rehearsal and post-restore reconciliation.

PERMISSION EVIDENCE

DATA-CRED-001 and ALMDB-SEC-001 require positive owned operations and negative schema/foreign-write attempts from the actual deployed identities. Configuration inspection alone cannot prove effective database permissions.

Provider component tests are blocking

The model is accepted only on the selected production-class database engine. EF Core InMemory or a different relational provider cannot prove generated SQL, collation, time precision, constraints, transactions, concurrency, query plans or migrations. Component fixtures create an isolated AlarmManagementDb, apply the accepted migration chain, run the scenario, capture redacted provider/schema identity and dispose only their owned resource.

Scenario	Fault / contention	Required observation
First accepted alarm	None.	Observation + managed alarm + detector version commit; no public fact unless policy decides flood.
Duplicate source event	Sequential, concurrent and after process restart.	One observation and one business effect; stable Duplicate result.
Concurrent different inputs	Two contexts load the same detector version.	One commit wins; loser fully rolls back, reloads and replays; final decision equals serialized truth.

Scenario	Fault / contention	Required observation
Flood boundary	At threshold, below/above, same timestamp ties and accepted late fact.	ALM-DET-001 result, exact evidence membership and policy/decision identity.
Failure inside commit	Provider fault after some commands begin.	All owned state/public intent or none; no orphan observation/detection/fact.
Unique decision race	Same decision derived concurrently.	One detection and one pending public fact; semantic mismatch blocks.
Migration create/upgrade	Empty and supported prior schema; representative volume.	Exact schema identity, preserved data, accepted lock/time budget.
Permissions	Runtime attempts DDL and foreign write; migration identity attempts unrelated DB.	Owned path succeeds; forbidden paths fail and are evidenced safely.
Restore/replay	Restore at accepted point with pending facts.	Compatible artifact starts, no new fact identity, duplicate downstream effect prevented.
Backlog query	Representative pending-public-fact volume.	Bounded plan/latency and capacity/readiness policy; no unbounded scan.

AlarmManagementDb evidence gates

Evidence ID	Blocking rule	Required artifact
ALMDB-OWN-001	Every relation/column is AlarmManagement truth, evidence or technical coordination; no foreign authority is stored as owned truth.	Fact-to-relation map and DATA-OWN-001 review.
ALMDB-MAP-001	EF model explicitly defines tables, keys, relationships, lengths, precision, indexes and delete behaviour.	Model debug/schema manifest plus mapping tests.
ALMDB-INV-001	Database and Domain constraints agree for identity, state, evidence membership and immutability.	Constraint catalogue and negative provider tests.
ALMDB-IDEM-001	Repeated source event and decision identity create at most one owned effect/public fact.	Sequential/concurrent/restart duplicate report.
ALMDB-CON-001	Concurrent detector work cannot commit from stale scope state; bounded replay preserves ALM-DET-001.	Two-writer schedules, generated predicate and final replay comparison.
ALMDB-TXN-001	Observation, current state, detector version, detection/evidence and pending fact commit atomically.	Injected-failure transaction report and durable before/after state.
ALMDB-MIG-001	Accepted migration creates/upgrades the selected production provider without unreviewed/destructive drift.	Migration/source hash, reviewed SQL, create/upgrade/failure results.
ALMDB-SEC-001	Runtime and migration identities are separate, least-privileged and reject foreign/schema operations.	Effective permission inventory and identity-bound positive/negative tests.
ALMDB-OPS-001	Backup/restore, schema compatibility, retention, capacity and pending-fact recovery meet owned lifecycle policy.	Restore/replay/capacity evidence and runbook.
ALMDB-TEST-001	All provider-dependent claims execute on the selected engine/version with isolated repeatable fixtures.	Provider identity, migration identity, result files and retained failure diagnostics.
ALMDB-REVIEW-001	AlarmManagement, architecture, data, security and operations accept scope, evidence and residual risk.	Identity-bound review with exceptions/expiry.

ADR-012 - Persist alarm truth and flood evidence together

ADR field	ADR-012 record
Status	Accepted for the NEXUS-1 reference design; provider, migrations and execution evidence remain pending.

ADR field	ADR-012 record
Context	A current Alarm table alone cannot prove idempotent intake, deterministic ordered detection, concurrent replay, exact evidence or recoverable publication.
Decision	AlarmManagementDb stores immutable accepted observations, versioned current alarm and detector state, immutable flood decisions with ordered evidence membership, and pending public facts. One normal SaveChanges call is the local commit; detector scope uses optimistic concurrency; source and decision identities are unique.
Consequences	More relations and evidence are retained; decisions are replayable; contention is localized per scope; storage/retention cost and provider-specific constraint/index work must be operated.
Rejected	One mutable alarm table, arrival-order SQL COUNT, shared schema writes, generic repository, broker publish inside transaction, application dual-write, catch-all duplicate handling, runtime auto-migration.
Review trigger	Flood policy/order, scope partitioning, source identity, provider/token, retention, outbox contract, transaction boundary, restore model or public event meaning changes.

Risk register for AlarmManagementDb

Risk	Early signal	Blocking control
Arrival-order truth	Different workers/retries produce different flood membership.	Canonical stored order, policy identity and ALM-DET/ALMDB-CON replay tests.
Duplicate observation	Same source event creates repeated count/state transition.	Unique source identity plus classified duplicate outcome across restart/concurrency.
Concurrency hotspot	One broad scope causes repeated conflicts and latency.	Domain-valid scope partition review, metrics, bounded retry and capacity test; never weaken truth silently.
Orphan public fact	Flood state exists without recoverable publication intent or vice versa.	One local SaveChanges and ALMDB-TXN-001 injected-failure test.
Provider drift	Mapping works on fake/local provider but production SQL/constraints differ.	Pinned production provider component suite and schema identity.
Catch-all idempotency	Foreign-key/timeout/data errors return Duplicate.	Known constraint/error classification and unknown-failure propagation.
Mutable evidence	Late/corrected fact rewrites old observation/detection.	Append-only repository path, restrictive permissions and history tests.
Over-indexing	Write/migration cost grows without owned query evidence.	Query-to-index catalogue, plan/volume tests and index review.
Runtime schema privilege	Service calls Migrate/EnsureCreated or holds DDL rights.	Separate migration identity, startup scan and permission denial.
Restore duplicates	Restored pending fact is assigned a new identity or source events replay as new.	Identity-preserving restore/replay rehearsal and downstream idempotency.

Official-source validation

The EF Core mechanics were checked against current Microsoft documentation on 18 July 2026. Stable points used here are that Fluent API can configure the model and extracted IEntityTypeConfiguration classes; composite and unique indexes are explicit; primary/composite keys identify entities; a configured concurrency token is included in the update predicate and conflicts raise DbUpdateConcurrencyException; one SaveChanges call is transactional when the provider supports transactions; and production migrations should be inspected and tested before application.

Official source	Stable point used here	NEXUS-1 boundary
Microsoft Learn - EF Core model configuration	Fluent API has the strongest configuration precedence; mappings can be grouped in IEntityTypeConfiguration classes and applied from an <code>DbContext</code>	Infrastructure owns explicit mappings; Domain remains free of EF annotations/packages.

Official source	Stable point used here	NEXUS-1 boundary
Microsoft Learn - EF Core indexes and keys	Composite/unique indexes and primary/composite keys are explicit model concepts; uniqueness violations are database-enforced.	Source/decision uniqueness is modelled, then proved on the selected provider.
Microsoft Learn - EF Core concurrency	Concurrency tokens participate in update/delete predicates; zero affected rows raise DbUpdateConcurrencyException; native tokens are provider-specific and application-managed tokens are possible.	Portable GUID token is a reference only; exact token and semantic replay require provider evidence.
Microsoft Learn - EF Core transactions	A provider-supported single SaveChanges call applies tracked changes transactionally; manual transactions interact with retry strategies and savepoints.	Normal intake uses one SaveChanges; any multi-save transaction is a separate Chapter 18/provider decision.
Microsoft Learn - EF Core applying migrations	Generated migrations must be inspected/tested; SQL scripts/bundles support controlled deployment; production runtime migration retains elevated-risk concerns.	AlarmManagement owns migrations and separate authority; Chapter 16 selects the exact net8.0 deployment path.

Current documentation may display EF Core versions newer than the book baseline and may mention behaviour introduced after EF Core 8. This volume remains pinned to net8.0. It does not import EF Core 9/10/11 migration locking, pending-model checks, JSON/complex-type indexes or other newer features into the reference until the repository explicitly upgrades its SDK and packages.

Technical material still required

The owned responsibilities, data relations, identities, mappings, constraints, ordering contract, transaction, concurrency replay, migration boundary, credentials, component scenarios and evidence IDs are specified. No provider or EF package version has been pinned; no AlarmManagement project has compiled; no migration/model snapshot/SQL exists; and no database engine has enforced a key, transaction or concurrency token.

TECHNICAL MATERIAL STATUS - [PROVIDER SELECTION + ALARMMANAGEMENTDB MIGRATION/CONCURRENCY EXECUTION EVIDENCE REQUIRED]

Create the net8.0 AlarmManagement Domain/Application/Infrastructure/Host and test projects; pin compatible EF Core and production-provider packages/images; implement the aggregate, store, row mappings and narrow error classifier; generate/review the initial migration; apply it to empty and supported prior databases; execute deterministic ordering, duplicate, concurrent detector, atomic failure, query-plan, permission, backup/restore and pending-public-fact recovery scenarios; publish ALMDB-OWN-001 through ALMDB-REVIEW-001 and linked DATA/TST evidence. AlarmManagementDb remains specified, not built or migrated. RootCause remains G0 accepted and G1 specified.

Deliverable	Minimum technical content	Blocking evidence
Domain/application model	Accepted observation, ManagedAlarm/DetectorScope/FloodDetection, canonical order/policy/decision identity and purpose-built store.	ALM-DET-001 / ALMDB-OWN-001
EF model	DbContext, persistence rows/mapping, keys, relationships, constraints, lengths/precision, concurrency and indexes.	ALMDB-MAP-001 / ALMDB-INV-001
Provider adapter	Pinned package/engine, options, GUID/native token choice and exact constraint/error classification.	ALMDB-TEST-001
Initial migration	Migration, snapshot, reviewed SQL/bundle/script and exact schema identity.	ALMDB-MIG-001
Correctness suite	Duplicate/restart, deterministic boundaries, two-writer replay, unique decision and atomic fault scenarios.	ALMDB-IDEM-001 / ALMDB-CON-001 / ALMDB-TXN-001
Access/operations	Runtime/migration identities, denied foreign/DDDL actions, query/capacity, retention and backup/restore/replay.	ALMDB-SEC-001 / ALMDB-OPS-001

Deliverable	Minimum technical content	Blocking evidence
Acceptance	Provider/schema/artifact identities, machine results, diagnostics, multidisciplinary review and residual risk.	ALMDB-REVIEW-001

Implementation sequence

1. Accept ADR-012 and freeze the AlarmManagement fact, source identity, canonical order, detector scope, policy identity and decision-identity contracts.
2. Generate the AlarmManagement service projects from the accepted repository shape; keep Domain/Application free of EF/provider dependencies.
3. Pin net8.0-compatible EF Core and production-provider packages, engine image/version, connection options and provider ADR.
4. Implement ManagedAlarm, DetectorScope, FloodDetection, evidence membership and the purpose-built AlarmManagement store/unit-of-work port.
5. Implement AlarmManagementDbContext, persistence rows/mapping, constraints, relationships, GUID/native concurrency token and query-driven indexes.
6. Generate and inspect InitialAlarmManagement; create the provider database from zero and upgrade every supported prior identity with representative data.
7. Implement narrow source/decision unique-violation and DbUpdateConcurrencyException translation; unknown failures remain failures.
8. Execute deterministic boundary, duplicate, concurrent schedule, atomic failure, restart and pending-public-fact component scenarios on the real provider.
9. Provision runtime/migration identities; prove owned operations, denied DDL/foreign writes, secret handling, capacity/health and query plans.
10. Rehearse backup/restore/schema compatibility and pending-fact replay; publish ALMDB-OWN-001 through ALMDB-REVIEW-001 without advancing unproved gates.

What this chapter has proved

Chapter 14 has defined the executable persistence contract for AlarmManagementDb: one owned use case; immutable accepted inputs; versioned current alarm and detector state; deterministic policy/evidence identities; exact flood membership; one local commit with pending public fact; optimistic concurrency with semantic replay; explicit keys, constraints, indexes, migrations, credentials and provider evidence gates.

It has not selected or run a provider, compiled EF code, generated a migration, created a database, enforced a constraint, won a concurrency race, copied legacy data, published AlarmFloodDetected.v1, restored a backup or advanced AlarmManagement/RootCause runtime evidence. The design is ready to implement; ALMDB, DATA, TST and migration evidence remain pending.

OWNERSHIP MADE CONCRETE

AlarmManagementDb is not a renamed legacy schema. It is the atomic record of what AlarmManagement accepted, what its policy decided, which evidence produced that decision and which public fact must eventually leave the boundary.

CHAPTER 14 - CODE AUDIT

AlarmManagement truth becomes an explicit persistence boundary

The code below makes the chapter's critical seams visible: canonical identities, immutable evidence, versioned detector state, exact decision membership, and publication intent are prepared for one local commit without turning EF Core into the domain model.

EVIDENCE STATUS

These assets are compile-oriented reference material. The current manuscript environment has no dotnet SDK, selected provider, database image, generated migration, or executed component suite. ALMDB evidence therefore remains **PENDING**.

EXECUTABLE ASSET 14-A - OWNER-DEFINED IDENTITIES

```
public readonly record struct ObservationId(Guid Value);
public readonly record struct AlarmId(Guid Value);
public readonly record struct DetectorScopeId(Guid Value);
public readonly record struct FloodDetectionId(Guid Value);
public readonly record struct PublicFactId(Guid Value);

public sealed record SourceEventIdentity(
    string SourceSystem,
    string SourceEventId);

public sealed record CanonicalAlarmOrder(
    DateTimeOffset OccurredAtUtc,
    string SourceOrderToken,
    string SourceSystem,
    string SourceEventId) : IComparable<CanonicalAlarmOrder>
{
    public int CompareTo(CanonicalAlarmOrder? other)
    {
        if (other is null) return 1;
        var byTime = OccurredAtUtc.CompareTo(other.OccurredAtUtc);
        if (byTime != 0) return byTime;
        var byToken = StringComparer.Ordinal.Compare(
            SourceOrderToken, other.SourceOrderToken);
        if (byToken != 0) return byToken;
        var bySource = StringComparer.Ordinal.Compare(
            SourceSystem, other.SourceSystem);
        return bySource != 0 ? bySource
            : StringComparer.Ordinal.Compare(SourceEventId, other.SourceEventId);
    }
}
```

EXECUTABLE ASSET 14-B - COLLISION-SAFE DECISION IDENTITY

```
public static class DecisionIdentity
{
    public static string Create(
        string scopeKey,
        string policyIdentity,
        IEnumerable<ObservationId> orderedEvidence)
    {
        var fields = new[] { "v1", scopeKey, policyIdentity }
            .Concat(orderedEvidence.Select(x => x.Value.ToString("D")));
        var canonical = string.Concat(fields.Select(x =>
            $"{Encoding.UTF8.GetByteCount(x)}:{x}"));
        return Convert.ToHexString(
            SHA256.HashData(Encoding.UTF8.GetBytes(canonical)));
    }
}
```

PERSISTENCE RECORDS

Rows mirror owned responsibilities, not public contracts

EXECUTABLE ASSET 14-C - ACCEPTED INPUT AND CURRENT STATE ROWS

```
internal sealed class AlarmObservationRow
{
    public Guid ObservationId { get; init; }
    public required string SourceSystem { get; init; }
    public required string SourceEventId { get; init; }
    public required string SourceAlarmId { get; init; }
    public required string ScopeKey { get; init; }
    public required string SourceOrderToken { get; init; }
    public required string PolicyIdentity { get; init; }
    public required string NormalizedState { get; init; }
    public DateTimeOffset OccurredAtUtc { get; init; }
    public DateTimeOffset RecordedAtUtc { get; init; }
}

internal sealed class ManagedAlarmRow
{
    public Guid AlarmId { get; init; }
    public required string SourceSystem { get; init; }
    public required string SourceAlarmId { get; init; }
    public required string CurrentState { get; set; }
    public Guid ConcurrencyVersion { get; set; }
}

internal sealed class DetectorScopeRow
{
    public Guid ScopeId { get; init; }
    public required string ScopeKey { get; init; }
    public required string PolicyIdentity { get; set; }
    public Guid ConcurrencyVersion { get; set; }
}
```

EXECUTABLE ASSET 14-D - IMMUTABLE DECISION AND PRODUCER INTENT ROWS

```
internal sealed class FloodDetectionRow
{
    public Guid DetectionId { get; init; }
    public required string DecisionIdentity { get; init; }
    public required string ScopeKey { get; init; }
    public required string PolicyIdentity { get; init; }
    public DateTimeOffset DecidedAtUtc { get; init; }
    public List<FloodDetectionInputRow> Inputs { get; } = [];
}

internal sealed class FloodDetectionInputRow
{
    public Guid DetectionId { get; init; }
    public int Position { get; init; }
    public Guid ObservationId { get; init; }
}

internal sealed class PendingPublicFactRow
{
    public Guid PublicFactId { get; init; }
    public required string FactType { get; init; }
    public required string SchemaVersion { get; init; }
    public required string PartitionKey { get; init; }
    public required string PayloadJson { get; init; }
    public DateTimeOffset CreatedAtUtc { get; init; }
    public DateTimeOffset? PublishedAtUtc { get; set; }
    public int AttemptCount { get; set; }
}
```

PRIVATE BY CONSTRUCTION

The row types stay internal to Infrastructure. The public AlarmFloodDetected.v1 DTO is mapped into PayloadJson at the producer boundary; no consumer references these classes.

EF CORE MODEL

Keys, constraints, order, and concurrency are reviewable metadata

EXECUTABLE ASSET 14-E - OBSERVATION AND DETECTOR MAPPINGS

```
internal sealed class AlarmObservationConfiguration
    : IEntityTypeConfiguration<AlarmObservationRow>
{
    public void Configure(EntityTypeBuilder<AlarmObservationRow> b)
    {
        b.ToTable("alarm_observations");
        b.HasKey(x => x.ObservationId);
        b.Property(x => x.SourceSystem).HasMaxLength(80).IsRequired();
        b.Property(x => x.SourceEventId).HasMaxLength(160).IsRequired();
        b.Property(x => x.SourceAlarmId).HasMaxLength(160).IsRequired();
        b.Property(x => x.ScopeKey).HasMaxLength(200).IsRequired();
        b.Property(x => x.SourceOrderToken).HasMaxLength(80).IsRequired();
        b.Property(x => x.PolicyIdentity).HasMaxLength(128).IsRequired();
        b.Property(x => x.NormalizedState).HasMaxLength(40).IsRequired();
        b.HasIndex(x => new { x.SourceSystem, x.SourceEventId })
            .HasDatabaseName("ux_alarm_observations_source_identity")
            .IsUnique();
        b.HasIndex(x => new {
            x.ScopeKey, x.OccurredAtUtc, x.SourceOrderToken,
            x.SourceSystem, x.SourceEventId
        }).HasDatabaseName("ix_alarm_observations_detector_order");
    }
}

internal sealed class DetectorScopeConfiguration
    : IEntityTypeConfiguration<DetectorScopeRow>
{
    public void Configure(EntityTypeBuilder<DetectorScopeRow> b)
    {
        b.ToTable("detector_scopes");
        b.HasKey(x => x.ScopeId);
        b.Property(x => x.ScopeKey).HasMaxLength(200).IsRequired();
        b.HasIndex(x => x.ScopeKey).IsUnique();
        b.Property(x => x.PolicyIdentity).HasMaxLength(128).IsRequired();
        b.Property(x => x.ConcurrencyVersion)
            .IsConcurrencyToken().ValueGeneratedNever();
    }
}
```

EXECUTABLE ASSET 14-F - EXACT EVIDENCE MEMBERSHIP MAPPING

```
internal sealed class FloodDetectionConfiguration
    : IEntityTypeConfiguration<FloodDetectionRow>
{
    public void Configure(EntityTypeBuilder<FloodDetectionRow> b)
    {
        b.ToTable("flood_detections");
        b.HasKey(x => x.DetectionId);
        b.Property(x => x.DecisionIdentity).HasMaxLength(64).IsRequired();
        b.HasIndex(x => x.DecisionIdentity)
            .HasDatabaseName("ux_flood_detections_decision_identity")
            .IsUnique();
        b.HasMany(x => x.Inputs).WithOne()
            .HasForeignKey(x => x.DetectionId)
            .OnDelete(DeleteBehavior.Cascade);
    }
}

internal sealed class FloodDetectionInputConfiguration
    : IEntityTypeConfiguration<FloodDetectionInputRow>
{
    public void Configure(EntityTypeBuilder<FloodDetectionInputRow> b)
    {
        b.ToTable("flood_detection_inputs");
        b.HasKey(x => new { x.DetectionId, x.Position });
        b.HasIndex(x => new { x.DetectionId, x.ObservationId }).IsUnique();
        b.HasOne<AlarmObservationRow>().WithMany()
            .HasForeignKey(x => x.ObservationId)
            .OnDelete(DeleteBehavior.Restrict);
    }
}
```

ATOMIC STAGING

Domain result and public intent enter one tracked unit of work

EXECUTABLE ASSET 14-G - PUBLIC FACT FACTORY

```
internal static class AlarmFloodPublicFactFactory
{
    public static PendingPublicFactRow Create(
        FloodDetection detection,
        TimeProvider clock)
    {
        var contract = new AlarmFloodDetectedV1(
            EventId: detection.PublicFactId.Value,
            DetectionId: detection.Id.Value,
            ScopeKey: detection.Scope.Value,
            PolicyIdentity: detection.PolicyIdentity.Value,
            DecisionIdentity: detection.DecisionIdentity,
            OrderedObservationIds: detection.Evidence
                .OrderBy(x => x.Position)
                .Select(x => x.ObservationId.Value)
                .ToArray());

        return new PendingPublicFactRow
        {
            PublicFactId = contract.EventId,
            FactType = "AlarmFloodDetected",
            SchemaVersion = "v1",
            PartitionKey = contract.ScopeKey,
            PayloadJson = JsonSerializer.Serialize(
                contract, AlarmContractsJsonContext.Default.AlarmFloodDetectedV1),
            CreatedAtUtc = clock.GetUtcNow()
        };
    }
}
```

EXECUTABLE ASSET 14-H - ONE SAVECHANGES COMMIT

```
internal sealed class EfAlarmManagementStore(
    AlarmManagementDbContext db,
    ITrackedAlarmStates trackedStates,
    IDatabaseErrorClassifier databaseErrors,
    TimeProvider clock) : IAlarmManagementStore
{
    public void Stage(AcceptAlarmResult result)
    {
        db.AlarmObservations.Add(AlarmRows.From(result.Observation));

        // These rows were loaded and tracked by this unit of work.
        // Apply changes without replacing the original concurrency values.
        var tracked = trackedStates.For(result.Detector.Scope);
        tracked.Alarm.Apply(result.Alarm);
        tracked.Detector.Apply(
            result.Detector, nextVersion: Guid.NewGuid());

        if (result.NewDetection is { } detection)
        {
            db.FloodDetections.Add(AlarmRows.From(detection));
            db.PendingPublicFacts.Add(
                AlarmFloodPublicFactFactory.Create(detection, clock));
        }
    }

    public async Task CommitAsync(CancellationTokens cancellationTokens)
    {
        try
        {
            await db.SaveChangesAsync(cancellationTokens);
        }
        catch (DbUpdateConcurrencyException ex)
        {
            throw new AlarmDecisionConflictException(ex);
        }
        catch (DbUpdateException ex)
        {
            when (databaseErrors.TryUnique(ex, out var conflict))
            {
                throw conflict switch
                {
                    UniqueConflict.SourceEvent =>
                        new AlarmObservationAlreadyAcceptedException(ex),
                    UniqueConflict.Decision =>
                        new AlarmDecisionAlreadyRecordedException(ex),
                    _ => throw new UnreachableException()
                };
            }
        }
    }
}
```

COMMIT BOUNDARY

There is no broker client and no RootCause/Audit DbContext in this store. A successful SaveChangesAsync makes both owned truth and producer intent durable: dispatch starts later.

SEMANTIC RETRY

A concurrency loser discards its graph and reloads owned truth

EXECUTABLE ASSET 14-I - FRESH-UNIT-OF-WORK COMMAND HANDLER

```
public sealed class AcceptAlarmObservationHandler(
    IAlarmManagementUnitOfWorkFactory units,
    IAcceptedAlarmPolicy policy,
    IAlarmDuplicateResolver duplicates)
{
    private const int MaxAttempts = 3;

    public async Task<AcceptAlarmOutcome> Handle(
        AcceptAlarmObservation command,
        CancellationToken cancellationToken)
    {
        for (var attempt = 1; attempt <= MaxAttempts; attempt++)
        {
            await using var unit = await units.CreateAsync(cancellationToken);
            var state = await unit.Store.LoadDecisionStateAsync(
                command.AlarmIdentity,
                command.Scope,
                cancellationToken);

            if (state.TryGetObservation(
                command.SourceEventIdentity, out var existing))
                return AcceptAlarmOutcome.Duplicate(existing.Id);

            var result = AlarmFloodPolicy.Accept(
                state,
                command.ToAcceptedObservation(),
                policy.Resolve(state.PolicyIdentity));

            unit.Store.Stage(result);
            try
            {
                await unit.Store.CommitAsync(cancellationToken);
                return AcceptAlarmOutcome.Accepted(result.Observation.Id);
            }
            catch (AlarmObservationAlreadyAcceptedException)
            {
                return await duplicates.ResolveAsync(
                    command.SourceEventIdentity, cancellationToken);
            }
            catch (AlarmDecisionConflictException)
            {
                when (attempt < MaxAttempts)
                {
                    // Dispose this unit; next loop reloads all semantic truth.
                }
            }
        }

        return AcceptAlarmOutcome.Contended(MaxAttempts);
    }
}
```

EXECUTABLE ASSET 14-J - RETRY INVARIANTS

```
[Fact]
public async Task ALMDB_CON_001_retry_never_reuses_stale_state()
{
    var units = new RecordingUnitOfWorkFactory(
        conflictsBeforeSuccess: 1);
    var handler = AlarmFixture.CreateHandler(units);

    await handler.Handle(
        AlarmFixture.AcceptedObservation("source-event-42"),
        CancellationToken.None);

    units.Created.Should().HaveCount(2);
    units.Created[0].WasDisposed.Should().BeTrue();
    units.Created[1].LoadedVersion.Should()
        .Be(units.WinningDatabaseVersion);
    units.Created[1].StagedDecisionIdentity.Should()
        .Be(AlarmFixture.SequentialOracleDecisionIdentity);
}
```

RETRY IS NOT A MERGE

CurrentValues.SetValues, retry-count-dependent policy, and reuse of the losing pending fact would all preserve a stale decision. The whole unit of work is replaced.

PROVIDER ADAPTER

Only known constraint identities become idempotency outcomes

EXECUTABLE ASSET 14-K - PROVIDER-NEUTRAL CLASSIFICATION PORT

```
internal enum UniqueConflict { SourceEvent, Decision }

internal interface IDatabaseErrorClassifier
{
    bool TryUnique(
        DbUpdateException exception,
        out UniqueConflict conflict);
}

internal static class KnownConstraints
{
    public const string SourceEvent =
        "ux_alarm_observations_source_identity";
    public const string Decision =
        "ux_flood_detections_decision_identity";
}
```

EXECUTABLE ASSET 14-L - POSTGRESQL CANDIDATE ADAPTER

```
internal sealed class PostgreSqlErrorClassifier
    : IDatabaseErrorClassifier
{
    public bool TryUnique(
        DbUpdateException exception,
        out UniqueConflict conflict)
    {
        conflict = default;
        if (exception.InnerException is not PostgresException
            {
                SqlState: PostgresErrorCodes.UniqueViolation,
                ConstraintName: { } constraint
            })
            return false;

        return constraint switch
        {
            KnownConstraints.SourceEvent =>
                Set(UniqueConflict.SourceEvent, out conflict),
            KnownConstraints.Decision =>
                Set(UniqueConflict.Decision, out conflict),
            _ => false
        };
    }

    private static bool Set(
        UniqueConflict value, out UniqueConflict target)
    {
        target = value;
        return true;
    }
}
```

EXECUTABLE ASSET 14-M - CLASSIFIER SAFETY TEST

```
[Theory]
[InlineData(KnownConstraints.SourceEvent, UniqueConflict.SourceEvent)]
[InlineData(KnownConstraints.Decision, UniqueConflict.Decision)]
public void ALMDB_IDEM_001_known_unique_constraint_is_classified(
    string name, UniqueConflict expected)
{
    var error = ProviderExceptionFixture.UniqueViolation(name);
    classifier.TryUnique(error, out var actual).Should().BeTrue();
    actual.Should().Be(expected);
}

[Fact]
public void unknown_database_failure_is_not_a_duplicate()
{
    var error = ProviderExceptionFixture.ForeignKeyViolation();
    classifier.TryUnique(error, out _).Should().BeFalse();
}
```

CANDIDATE, NOT PROVIDER SELECTION

The Npgsql adapter demonstrates the required shape because PostgreSQL exposes a structured constraint name. If another engine is selected, its exact codes and metadata need a different adapter and provider-backed tests; message-text matching remains forbidden.

TRANSACTIONAL COMPONENT PROOF

The selected provider must expose all-or-none durable state

EXECUTABLE ASSET 14-N - CONCURRENT DUPLICATE ON REAL DATABASE

```
[Fact]
public async Task ALMDB_IDEM_001_concurrent_duplicate_has_one_effect()
{
    await fixture.ResetAndMigrateAsync();
    var command = AlarmFixture.AcceptedObservation("source-event-42");
    var start = new Barrier(2);

    var outcomes = await Task.WhenAll(
        fixture.HandleInNewProcessScopeAsync(command, start),
        fixture.HandleInNewProcessScopeAsync(command, start));

    outcomes.Should().ContainSingle(x =>
        x.Kind == AcceptAlarmOutcomeKind.Accepted);
    outcomes.Should().ContainSingle(x =>
        x.Kind == AcceptAlarmOutcomeKind.Duplicate);

    await using var assertDb = fixture.CreateCleanDbContext();
    (await assertDb.AlarmObservations.CountAsync()).Should().Be(1);
    (await assertDb.ManagedAlarms.CountAsync()).Should().Be(1);
    (await assertDb.DetectorScopes.CountAsync()).Should().Be(1);
}
```

EXECUTABLE ASSET 14-O - INJECTED COMMAND FAILURE ROLLS BACK

```
[Fact]
public async Task ALMDB_TXN_001_failed_commit_leaves_no_partial_truth()
{
    await fixture.ResetAndMigrateAsync();
    fixture.DatabaseCommandFault.FailOnceBefore(
        table: "pending_public_facts");

    var act = () => fixture.HandleInNewProcessScopeAsync(
        AlarmFixture.FloodProducingObservation(),
        start: null);

    await act.Should().ThrowAsync<InjectedDatabaseFault>();

    await using var assertDb = fixture.CreateCleanDbContext();
    (await assertDb.AlarmObservations.CountAsync()).Should().Be(0);
    (await assertDb.FloodDetections.CountAsync()).Should().Be(0);
    (await assertDb.PendingPublicFacts.CountAsync()).Should().Be(0);
    (await assertDb.DetectorScopes.CountAsync()).Should().Be(0);
}
```

EXECUTABLE ASSET 14-P - SUCCESSFUL COMMIT SURVIVES BROKER OUTAGE

```
[Fact]
public async Task ALMDB_TXN_002_broker_is_not_part_of_local_commit()
{
    await fixture.ResetAndMigrateAsync();
    broker.RejectAllPublishes();

    await fixture.HandleInNewProcessScopeAsync(
        AlarmFixture.FloodProducingObservation(), start: null);

    await using var assertDb = fixture.CreateCleanDbContext();
    (await assertDb.FloodDetections.CountAsync()).Should().Be(1);
    (await assertDb.PendingPublicFacts
        .CountAsync(x => x.PublishedAtUtc == null)).Should().Be(1);
    broker.PublishCalls.Should().Be(0);
}
```

CONCURRENCY AND EVIDENCE

Two writers must converge to deterministic AlarmManagement truth

EXECUTABLE ASSET 14-Q - TWO-WRITER SEMANTIC REPLAY

```
[Fact]
public async Task ALMDB_CON_001_concurrent_inputs_equal_sequential_oracle()
{
    await fixture.ResetAndMigrateAsync();
    var first = AlarmFixture.Observation("A", occurredSecond: 10);
    var second = AlarmFixture.Observation("B", occurredSecond: 11);
    var bothLoaded = new Barrier(2);

    var outcomes = await Task.WhenAll(
        fixture.HandleInNewProcessScopeAsync(first, bothLoaded),
        fixture.HandleInNewProcessScopeAsync(second, bothLoaded));

    outcomes.Should().OnlyContain(x =>
        x.Kind == AcceptAlarmOutcomeKind.Accepted);

    var durable = await fixture.ReadDecisionSnapshotAsync();
    var oracle = AlarmFloodPolicy.Replay(
        new[] { first, second }.OrderBy(x => x.CanonicalOrder),
        AlarmFixture.AcceptedPolicy);

    durable.PolicyIdentity.Should().Be(oracle.PolicyIdentity);
    durable.OrderedObservationIds.Should()
        .Equal(oracle.OrderedObservationIds);
    durable.DecisionIdentity.Should().Be(oracle.DecisionIdentity);
    durable.PendingFactIds.Should().OnlyHaveUniqueItems();
}
```

EXECUTABLE ASSET 14-R - ACCEPTANCE EVIDENCE RECORD

```
{
  "evidenceId": "ALMDB-REVIEW-001",
  "chapter": 14,
  "artifactCommit": null,
  "sdk": "net8.0",
  "efCorePackage": null,
  "provider": null,
  "providerVersion": null,
  "migrationIdentity": null,
  "schemaDigest": null,
  "tests": {
    "ALMDB-OWN-001": "PENDING",
    "ALMDB-MAP-001": "PENDING",
    "ALMDB-INV-001": "PENDING",
    "ALMDB-IDEM-001": "PENDING",
    "ALMDB-CON-001": "PENDING",
    "ALMDB-TXN-001": "PENDING",
    "ALMDB-MIG-001": "PENDING",
    "ALMDB-SEC-001": "PENDING",
    "ALMDB-OPS-001": "PENDING",
    "ALMDB-TEST-001": "PENDING"
  },
  "runtimeDdlDenied": false,
  "foreignWritesDenied": false,
  "restoreRehearsed": false,
  "accepted": false
}
```

Chapter 14 code-audit result

The chapter now shows the main implementation seams from accepted alarm identity to durable flood evidence and pending publication, including the two failure paths senior developers must reason about separately: optimistic update conflict and unique insert violation.

CURRENT VERDICT

AlarmManagementDb is implementable in shape, but it has not compiled, migrated, or executed. Provider choice, generated SQL, permissions, fault injection, restore, and every ALMDB evidence gate remain explicitly PENDING.

Chapter checkpoint

You are ready to implement AlarmManagementDb when the input identity/order/policy contracts are accepted, every relation carries one owned responsibility, the normal use case has one local commit, concurrency retries reload semantic truth, and the selected provider can falsify every mapping, constraint, migration and recovery claim.

Question	Expected answer
What is AlarmManagementDb's source truth?	Accepted alarm observations, current owned alarm/detector state, flood decisions/evidence and pending producer facts.
Why is one current Alarm table insufficient?	It cannot prove duplicate intake, ordered deterministic detection, exact evidence, concurrent replay or recoverable publication.
What preserves ALM-DET-001?	Canonical accepted order, exact policy identity, evidence membership, decision identity and semantic replay after conflicts.
What makes intake idempotent?	Unique SourceSystem + SourceEventId plus a stable Duplicate outcome and one owned effect across concurrency/restart.
What serializes concurrent flood decisions?	A versioned detector-scope concurrency token; losers roll back, reload current truth and reapply the same policy.
What commits atomically?	Observation, alarm/detector state, optional detection/evidence and pending public fact in one local SaveChanges.
Does broker publication belong in that transaction?	No. Only pending producer intent commits locally; dispatch/confirmation/recovery follow after commit.
Why must provider tests be real?	Keys, SQL, collation, time, unique errors, transactions, concurrency, plans and migrations are provider-dependent.
What is the current status?	AlarmManagementDb is specified, not built/migrated; RootCause remains G0 accepted and G1 specified.
What does Chapter 15 add?	RootCauseDb: case/evidence/verdict persistence, policy identity, concurrency and atomic case-state/pending-verdict-fact commit.

NEXT CHAPTER

Chapter 15, Building RootCauseDb, applies the same ownership discipline to the decision-rich consumer: RootCauseCase state, accepted evidence, hypotheses, verdict and correction history, policy identity, inbound idempotency, concurrency and the pending RootCauseVerdictIssued.v1 fact—without allowing AlarmManagement, Audit, Compliance or Reporting to share the commit.

PART III - DATA OWNERSHIP IN EXECUTABLE FORM

15. Building RootCauseDb

RootCauseDb is the owned decision record for cases, admitted evidence, hypotheses, verdicts and corrections. It consumes AlarmFloodDetected.v1 without borrowing AlarmManagement tables, and it commits each case transition beside any pending RootCause public fact in one local transaction.

CHAPTER ARGUMENT

Persist the facts required to enforce RootCause meaning: a closed case does not change through the ordinary path; a case cannot close without evidence; only RootCause issues or corrects a verdict; duplicate delivery has one business effect; and concurrent evidence is resolved by reloading the current case and reapplying the same policy—not by merging stale tracked values.

Chapter 14 defined AlarmManagementDb as the producer-owned record of accepted alarms and flood decisions. RootCause receives only AlarmFloodDetected.v1 under the public contract. It may retain the producer fact identity, contract version and evidence lineage needed to explain causality, but it never treats AlarmManagement table keys, schema names or direct reads as its source truth. DATA-OWN-001 and CONTRACT-001 remain binding at both the database and language boundaries.

The inherited RootCause ledger is stricter than generic persistence. RC-INV-001 prohibits new ordinary evidence, hypotheses or state changes after closure. RC-INV-002 prohibits closure without evidence. RC-OWN-001 reserves verdict issue and correction for RootCause. FLOW-IDEM-001 limits repeated delivery to one RootCause business effect, while FLOW-CAUS-001 preserves correlation and causation across the consumer and later publication.

No RootCause implementation projects, selected provider, compatible EF package set, database image, connection, migration, runtime identity, broker delivery or legacy mapping exists in this manuscript environment. The C# below is a net8.0 reference contract and has not compiled. Provider-specific SQL, identifiers, collations, timestamp precision, isolation, native tokens, error codes and migration output remain blocking evidence.

Begin with one owned consumer use case

The model is driven by HandleAlarmFloodDetected, not by CRUD screens or a generic message repository. The consumer validates the producer contract, normalizes the producer fact identity, classifies duplicate intake, resolves or creates the RootCauseCase under an accepted case-correlation rule, admits evidence, invokes RootCause policy, records hypothesis/verdict changes, stages a public fact when RootCause issues or corrects a verdict, and performs one local SaveChanges. Broker acknowledgement follows only after the local outcome is durable.

Step	Owned decision	Durable result	Must not occur
Validate	Check AlarmFloodDetected.v1 schema/version and required producer identity.	Normalized inbound fact or stable rejected/poison outcome.	Deserialize AlarmManagement private entities or infer missing truth from its database.
Deduplicate	Find producer + public fact identity before applying case policy.	One inbound receipt and one RootCause business effect.	Use broker delivery tag, correlation ID or process memory as uniqueness.
Resolve	Apply accepted correlation policy to one RootCauseCase identity.	One case/version snapshot and explicit policy identity.	Let Reporting, arrival thread or a mutable lookup silently choose a different case.
Decide	Admit evidence and apply RC-INV-001/002 and RC-OWN-001.	Evidence, hypotheses and optional verdict/correction with exact lineage.	Close without evidence, reopen ordinary flow, or let an adapter issue the verdict.

Step	Owned decision	Durable result	Must not occur
Commit	Persist receipt, case truth and producer intent atomically.	All owned rows and pending fact —or none.	Write AlarmManagement/Audit/Compliance or publish to the broker inside the transaction.
Return	Report Applied, Duplicate, Closed, Conflict or Rejected without provider leakage.	Stable outcome plus case/fact identities for retry and diagnosis.	Expose EF exception text, private row shape or authorization-sensitive evidence.

One database, six private responsibilities

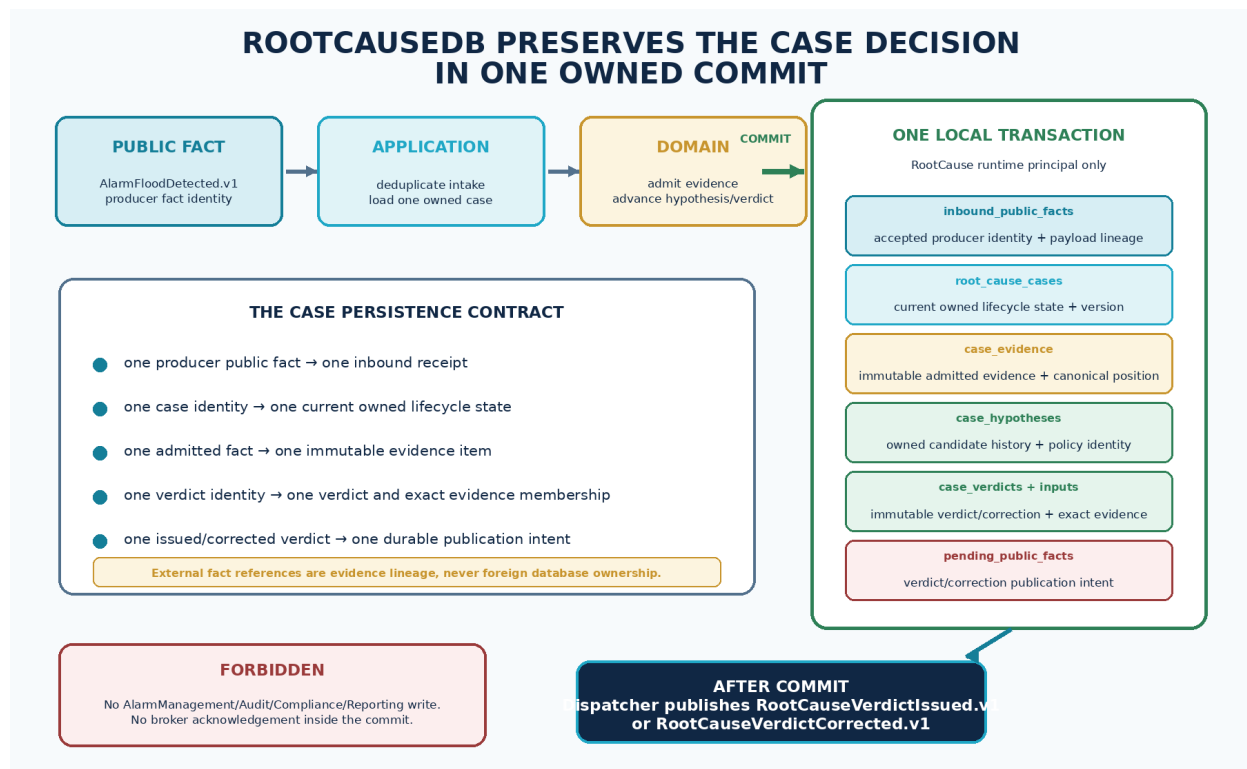


Figure 15.1 - RootCauseDb keeps inbound identity, case truth, evidence, hypothesis/verdict history and public intent inside one owned commit.

Private relation	Responsibility	Key / identity	Mutability
inbound_public_facts	Accepted producer fact, contract identity, content fingerprint and causality lineage.	InboundFactId PK; unique Producer + PublicFactId + ContractVersion.	Insert-only; processing outcome may advance without rewriting payload meaning.
root_cause_cases	Current RootCause-owned lifecycle state, policy cursor and concurrency point.	Caseld PK; accepted correlation key may be unique within a case policy.	Versioned transitions; ordinary path cannot mutate closed state.
case_evidence	Immutable evidence admitted by RootCause, linked to one case and inbound lineage.	EvidenceId PK; unique Caseld + EvidenceIdentity.	Insert-only; rejection/correction appends explicit history.
case_hypotheses	Candidate cause history and status under a policy identity.	HypothesisId PK; unique Caseld + HypothesisIdentity + Revision.	Append/version transition; no destructive replacement of evaluated history.
case_verdicts + inputs	Issued or corrected RootCause decision and exact ordered evidence/hypothesis membership.	VerdictId PK; unique VerdictIdentity; membership composite keys.	Immutable decision record; correction links a new verdict to the prior one.
pending_public_facts	Durable producer intent for verdict/correction facts after local commit.	PublicFactId PK; unique producer fact identity retained through dispatch.	Payload/meaning immutable; dispatch metadata may advance.

PRIVATE SCHEMA RULE

RootCause relation names, persistence rows and EF migrations are private implementation details. Public consumers receive RootCauseVerdictIssued.v1 or RootCauseVerdictCorrected.v1; they never receive RootCauseCase entities or direct RootCauseDb access.

The aggregate boundary is the case, not the DbContext

RootCause Domain owns case transitions, evidence admission, hypothesis evaluation, verdict authority and correction rules. Application owns the consumer use case and a purpose-built unit-of-work port. Infrastructure maps a complete case decision graph to EF Core rows. Host/Edge validates transport and authorization metadata. DbContext tracks persistence; it is not a policy engine, evidence classifier or case-reopening mechanism.

Layer	Owns	Forbidden persistence shortcut
Domain	RootCauseCase lifecycle, evidence/hypothesis/verdict rules, policy and decision identities.	DbContext, IQueryable, provider annotations, contract DTOs or broker metadata.
Application	HandleAlarmFloodDetected, case-correlation policy, RootCause store/unit-of-work port and stable outcomes.	Generic IRepository<T>, raw SQL semantics, provider error codes or direct producer database reads.
Infrastructure	DbContext, explicit mapping, graph load/save, uniqueness/concurrency translation and migrations.	Closing/reopening a case, inventing evidence identity or changing policy on retry.
Host / Edge	Consumer registration, envelope validation, authorization metadata and outcome/ack mapping.	Resolving DbContext in handlers or acknowledging before owned state is durable.

PURPOSE-BUILT ROOTCAUSE PORT - [NEW][COMPILE REQUIRED]

```
// Nexus1.RootCause.Application - no EF Core dependency.
public interface IRootCauseDecisionStore
{
    Task<RootCauseDecisionState> LoadAsync(
        RootCauseCaseIdentity caseIdentity,
        ProducerFactIdentity inboundFact,
        CancellationToken cancellationToken);

    void AddInboundFact(AcceptedInboundPublicFact inboundFact);
    void AddEvidence(CaseEvidence evidence);
    void Track(RootCauseCase rootCauseCase);
    void Add(HypothesisRevision hypothesis);
    void Add(RootCauseVerdict verdict);
    void Add(PendingPublicFact publicFact);

    Task CommitAsync(CancellationToken cancellationToken);
}
```

NO AGGREGATE FRAGMENTS

The store loads enough owned state to evaluate one case consistently. A generic repository that returns partial collections or IQueryable lets callers bypass RC-INV-001/002, changes transaction semantics and makes replay depend on accidental query shape.

Identity is layered; evidence references stay references

A producer public fact identity proves which AlarmManagement fact RootCause received. CaseId identifies the local aggregate. EvidenceIdentity prevents the same admitted meaning from being attached twice. HypothesisIdentity and revision preserve candidate history. VerdictIdentity binds one exact policy and evidence set to one RootCause decision. PublicFactId identifies the RootCause-produced fact. Correlation/causation e

Identity	Created / owned by	Durable use	Never used as
Producer + PublicFactId + ContractVersion	AlarmManagement contract; normalized by RootCause.	Unique inbound receipt and duplicate outcome.	RootCause case key, evidence authorization or broker delivery tag.
Caseld	RootCause case-correlation policy.	Primary key and concurrency/lifecycle boundary.	AlarmManagement alarm/detection identity or public contract version.
External detection reference	Producer public fact.	Lineage attached to admitted evidence.	Foreign key into AlarmManagementDb or proof that payload is valid.
EvidenceIdentity	RootCause from normalized evidence meaning + lineage policy.	Unique within case; exact verdict membership input.	Hash of mutable EF serialization, host time or retry count.
HypothesisIdentity + revision	RootCause Domain.	Candidate history and deterministic comparison.	A final verdict or mutable row version.
VerdictIdentity	RootCause from Caseld + policy + ordered evidence/hypothesis identity.	Unique decision/correction record and replay comparison.	Database-generated sequence or message delivery ID.
PublicFactId	RootCause producer when issuing/correcting verdict.	Unique pending dispatch identity retained through recovery.	Downstream inbox identity or case authorization.
Correlation / causation	Boundary flow context.	Indexed navigation and trace lineage; duplicates allowed.	Case correlation, deduplication or database primary key by itself.

The persistence model makes relationships explicit

The reference uses EF Core metadata in Infrastructure and keeps Domain free of mapping attributes. Keys, foreign keys, requiredness, lengths, delete behaviour, concurrency tokens and indexes are configured explicitly. Case-owned child relationships are ordinary relational relationships with RootCauseDb-local foreign keys; no relationship crosses into AlarmManagement, Audit, Compliance or Reporting.

ROOTCAUSEDBCONTEXT - [NEW][EF PACKAGE + COMPILE REQUIRED]

```
internal sealed class RootCauseDbContext(
    DbContextOptions<RootCauseDbContext> options) : DbContext(options)
{
    internal DbSet<InboundPublicFactRow> InboundPublicFacts => Set<InboundPublicFactRow>();
    internal DbSet<RootCauseCaseRow> Cases => Set<RootCauseCaseRow>();
    internal DbSet<CaseEvidenceRow> Evidence => Set<CaseEvidenceRow>();
    internal DbSet<CaseHypothesisRow> Hypotheses => Set<CaseHypothesisRow>();
    internal DbSet<CaseVerdictRow> Verdicts => Set<CaseVerdictRow>();
    internal DbSet<VerdictEvidenceRow> VerdictEvidence => Set<VerdictEvidenceRow>();
    internal DbSet<PendingPublicFactRow> PendingPublicFacts => Set<PendingPublicFactRow>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.ApplyConfigurationsFromAssembly(
            typeof(RootCauseDbContext).Assembly);
    }
}
```

DbSet visibility remains internal to Infrastructure. Application receives IRootCauseDecisionStore. The Host binds one scoped context/store per consumed message or bounded batch item. A singleton consumer never captures the scoped context; duplicate classification and acknowledgement occur through explicit application outcomes.

Keys, relationships and delete rules carry RootCause meaning

PROVIDER-NEUTRAL MAPPING INTENT - [NEW][COMPILE REQUIRED]

```
internal sealed class RootCauseCaseConfiguration
    : IEntityConfiguration<RootCauseCaseRow>
{
    public void Configure(EntityTypeBuilder<RootCauseCaseRow> b)
    {
        b.ToTable("root_cause_cases");
        b.HasKey(x => x.CaseId);
        b.Property(x => x.State).HasMaxLength(32).IsRequired();
        b.Property(x => x.PolicyIdentity).HasMaxLength(128).IsRequired();
        b.Property(x => x.CorrelationKey).HasMaxLength(220).IsRequired();
        b.Property(x => x.ConcurrencyVersion).IsConcurrencyToken();
        b.HasIndex(x => x.CorrelationKey).IsUnique();

        b.HasMany(x => x.Evidence).WithOne().HasForeignKey(x => x.CaseId)
            .OnDelete(DeleteBehavior.Restrict);
        b.HasMany(x => x.Hypotheses).WithOne().HasForeignKey(x => x.CaseId)
            .OnDelete(DeleteBehavior.Restrict);
        b.HasMany(x => x.Verdicts).WithOne().HasForeignKey(x => x.CaseId)
            .OnDelete(DeleteBehavior.Restrict);
    }
}
```

The exact maximum lengths and the unique correlation rule are reference limits until the real contract and case-correlation ADR are accepted. Restrict is intentional for evidence-bearing history; any purge or archival process is a separate owned lifecycle operation, not cascade deletion from an ordinary application command. Provider configuration and the migration must prove the final physical behaviour.

Rule	Database enforcement	Domain / Application companion
One producer fact	Unique Producer + PublicFactId + ContractVersion.	Duplicate reloads the prior RootCause outcome; only known constraint is classified.
One current case	Caseld PK; accepted correlation identity unique within pinned policy scope.	Correlation policy is versioned and never inferred from delivery order.
Evidence belongs to one case	FK Caseld; unique Caseld + EvidenceIdentity.	Domain validates meaning, admissibility and closed-case rule before insert.
No closure without evidence	Required verdict-to-case relationship and at least one membership proved in transaction tests.	RC-INV-002 is enforced before persistence; SQL schema alone cannot express all policy.
One verdict meaning	Unique VerdictIdentity; membership PK VerdictId + Position.	Domain derives identity from case, policy and exact ordered evidence/hypothesis set.
Correction preserves history	New VerdictId references superseded verdict; no normal update/delete path.	RC-OWN-001 authorizes correction and emits a separate corrected public fact.
Owned relationships only	All FKs terminate inside RootCauseDb with explicit delete behaviour.	No foreign navigation or cross-database constraint to AlarmManagement/Audit/Compliance.
Closed-case immutability	No provider trigger assumed by reference design; restrictive rights where feasible.	Aggregate refuses ordinary change; component tests attempt every forbidden path.

Evidence admission and ordering are domain contracts

RootCause does not treat every received field as accepted evidence. The boundary proves contract identity and authenticity; Application normalizes the public fact; Domain applies admissibility, case state and policy rules. Accepted evidence receives a stable EvidenceIdentity and canonical position. RecordedAtUtc remains operational telemetry. Producer occurrence time, producer decision identity and owner-approved tie-breakers are stored explicitly when they affect policy.

Field / identity	Purpose	Failure rule
Producer PublicFactId	Inbound idempotency and immutable producer lineage.	Missing/invalid identity is rejected or poisoned; broker tag never substitutes.
ProducerOccurredAtUtc	Domain-relevant occurrence when the con	Normalize explicitly; do not silently replace

Field / identity	Purpose	Failure rule
ProducerDecisionIdentity	Reference to the AlarmManagement decision represented by the public fact.	Stored as evidence lineage, never used for a direct database join.
RecordedAtUtc	RootCause intake delay, operations and audit navigation.	Not a case/verdict ordering field unless an accepted RootCause policy says so.
PolicyIdentity	Pins evidence admission, correlation, hypothesis and verdict rules.	No fallback to latest policy during replay, correction or recovery.
EvidenceIdentity + Position	Exact canonical input membership for hypothesis/verdict replay.	Canonicalization version is explicit; duplicate/missing positions fail component tests.
VerdictIdentity	Binds case, policy, evidence/hypothesis membership and correction lineage.	Serialization/provider drift cannot create a second logical verdict.

DECISION BOUNDARY

SQL can preserve keys, membership and current state; it does not decide evidence admissibility, close a case or choose a root cause. Those decisions remain inside RootCause Domain under RC-INV-001/002 and RC-OWN-001.

One SaveChanges is the intended case commit

When the selected relational provider supports transactions, one normal SaveChanges call is the RootCause commit point. The inbound receipt, admitted evidence, current case/version, optional hypothesis/verdict/correction and pending producer fact succeed together or the database remains unchanged. Explicit multi-SaveChanges transactions are introduced only by a proven use case and must account for retry-strategy and provider savepoint behaviour.

ATOMIC ROOTCAUSE CONSUMER SHAPE - [NEW][COMPILE + REAL PROVIDER TEST REQUIRED]

```
public async Task<HandleFloodOutcome> Handle(
    AlarmFloodDetectedV1 message,
    CancellationToken cancellationToken)
{
    var identity = ProducerFactIdentity.From(message);
    var state = await store.LoadAsync(
        caseCorrelation.Resolve(message), identity, cancellationToken);

    if (state.HasProcessed(identity))
        return state.PreviousOutcome(identity);

    var accepted = rootCausePolicy.Accept(
        state.Case, message.ToCandidateEvidence(), state.PolicyIdentity);

    store.AddInboundFact(accepted.InboundFact);
    store.Track(accepted.Case);
    store.AddEvidence(accepted.Evidence);

    foreach (var hypothesis in accepted.NewHypothesisRevisions)
        store.Add(hypothesis);

    if (accepted.NewVerdict is { } verdict)
    {
        store.Add(verdict);
        store.Add(PendingPublicFact.From(verdict));
    }

    await store.CommitAsync(cancellationToken); // one local SaveChangesAsync
    return HandleFloodOutcome.Applied(accepted.Case.Id, accepted.Outcome);
}
```

Failure point	Required durable state	Stable outcome / recovery
Before SaveChanges	No inbound receipt, evidence, case version or public intent.	Safe failure; same producer identity may retry.
Inbound insert fails	No case/evidence/hypothesis/verdict/public-fact	Known unique conflict reloads previous

Failure point	Required durable state	Stable outcome / recovery
Case concurrency conflict	Entire attempted graph rolls back.	Dispose unit of work, reload current case/evidence and perform bounded semantic replay.
Evidence identity conflict	Winner remains; loser adds no second evidence or effect.	Reload and prove equivalence or return Conflict; mismatch blocks.
Verdict identity conflict	One immutable verdict and pending fact exist.	Reload and compare policy/membership/correction lineage; mismatch blocks.
Commit succeeds; process crashes	RootCause truth and pending public fact both survive.	Consumer returns Duplicate on redelivery; dispatcher resumes independently.
Broker unavailable	Local commit remains; pending fact is recoverable under capacity policy.	No database rollback; acknowledgement/readiness follows Chapter 11 flow rules.

Optimistic concurrency serializes one RootCauseCase

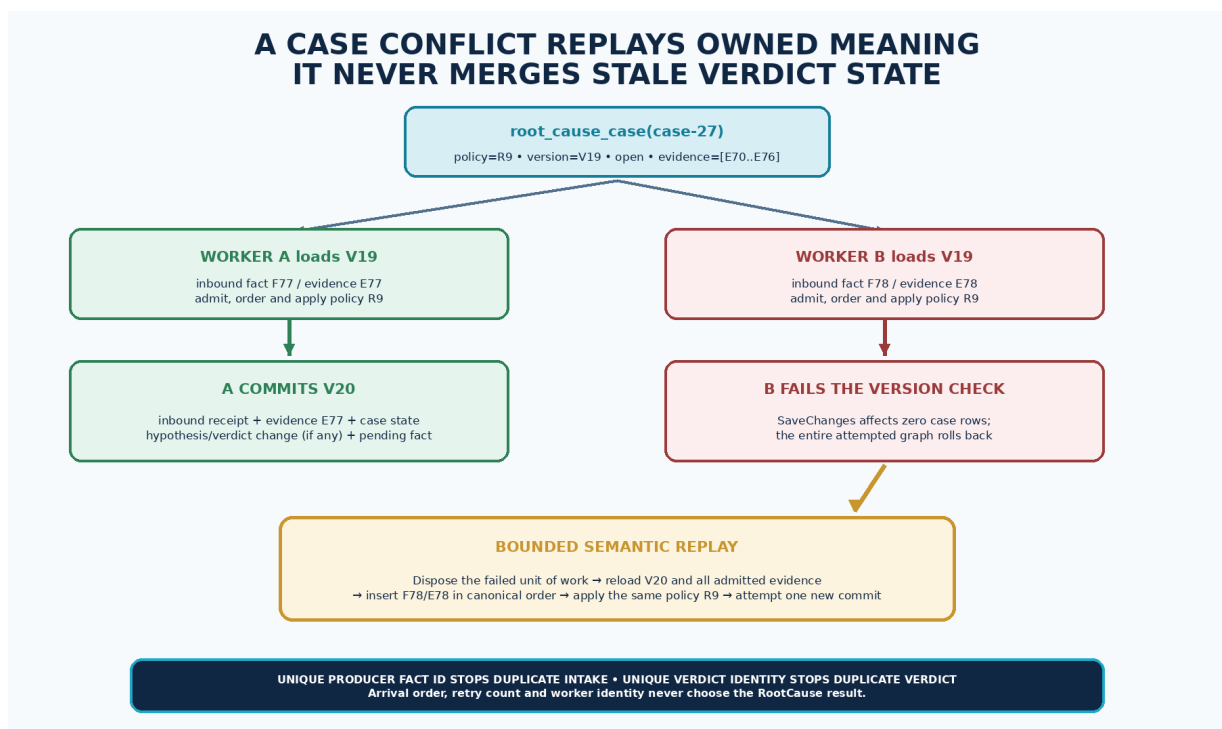


Figure 15.2 - A concurrency loser discards its stale case graph, reloads the winner and replays the same RootCause policy.

The `root_cause_cases` row is the case serialization point, not a global lock. Two consumers may evaluate the same V19 state, but only one can update the original concurrency token. A zero-row update produces `DbUpdateConcurrencyException`. The loser disposes the tracked graph, creates a fresh unit of work, reloads current case, evidence, policy and prior inbound outcomes, and re-runs the use case. It never merges stale verdict/hypothesis collections with database values.

Concurrency rule	Reference policy	Blocking evidence
Token scope	One case token covers every decision-affecting case/evidence/hypothesis/verdict transition.	Generated predicate and two-writer provider test; no unprotected decision path.
Fresh graph	Failed <code>DbContext</code> is disposed; replay loads current owned graph and accepted policy identity.	No reuse of <code>CurrentValues</code> , stale children or pending public fact from failed attempt.
No decision by retry	Worker, schedule, delay and retry count are excluded from evidence and <code>VerdictIdentity</code> .	Replay matrix reaches the same final case/verdict for equivalent admitted facts.
Bounded exhaustion	After accepted attempt/time budget, return stable Conflict/Deferred without partial state.	Metrics, trace, pending workload and safe client/consumer policy.
Provider token	Application-managed GUID is portable reference; nati	Exact mapping, generated SQL, token renewal

Duplicate intake is not a concurrency exception

EF Core reports optimistic update/delete conflicts through `DbUpdateConcurrencyException`. Concurrent inserts that violate producer-fact, evidence or verdict uniqueness normally surface through provider-specific database exceptions. Infrastructure classifies only known constraint identities or error codes. It does not catch every `DbUpdateException` and label it Duplicate, because foreign-key, timeout, storage and permission failures would become false success.

NARROW PROVIDER-ERROR TRANSLATION - [NEW][PROVIDER ADAPTER REQUIRED]

```
public async Task CommitAsync(CancellationTokentoken)
{
    try
    {
        await db.SaveChangesAsync(cancellationtoken);
    }
    catch (DbUpdateConcurrencyException ex)
    {
        throw new RootCauseCaseConflictException(
            "Owned RootCause case changed before commit.", ex);
    }
    catch (DbUpdateException ex) when (
        providerErrors.IsUniqueViolation(ex, KnownConstraint.InboundFactIdentity))
    {
        throw new RootCauseFactAlreadyHandledException(ex);
    }
}
```

UNKNOWN MEANS FAILURE

Only exact selected-provider constraint/error classification may become Duplicate. Message text matching, catch-all translation and leaking raw database errors are rejected.

Verdict and correction history remain immutable

A verdict is evidence-bearing `RootCause` history. Correction never updates the original verdict as if it did not occur. The correction path authorizes RC-OWN-001, loads the closed case and prior verdict, admits correction evidence under a pinned policy, creates a linked corrected verdict with its own `VerdictIdentity` and membership, and stages `RootCauseVerdictCorrected.v1`. Ordinary `AlarmFloodDetected.v1` handling cannot silently reopen the case.

History class	Retention / correction rule	Restore / replay implication
Inbound public fact	Retain producer identity, accepted contract and processing outcome for idempotency window.	Restore must preserve identities or redelivery creates a second effect.
Case current state	Keep while operationally/legal relevant; closure and correction status are explicit.	Rebuild only from complete accepted history whose policy identities are available.
Admitted evidence	Immutable with lineage; rejection/correction is a new owned record.	Replay proves exact identity/order and cannot depend on producer database availability.
Hypothesis revisions	Retain evaluated candidate history required by decision/explanation policy.	Restore preserves revision identity and relationship to verdict inputs.
Verdict + membership	Immutable issue/correction record; corrected verdict references superseded <code>VerdictId</code> .	Reconciliation proves exact evidence/hypothesis membership and public-fact identity.
Pending public fact	Retain until accepted dispatch/operational policy; payload meaning immutable.	Recovery republishes the same fact identity; downstream duplicates remain one effect.
Migration history	Owned by <code>RootCause</code> migration stream and never shared with <code>AlarmManagement</code> .	Restored database must match accepted schema/migration identity before consumption.

Queries and indexes follow case operations

Indexes are accepted only for owned commands, replay, dispatch and evidence navigation. Inbound lookup needs the producer identity unique index. Case resolution needs the accepted correlation key. Aggregate loading needs case-keyed evidence, hypotheses and verdicts in canonical order. Pending dispatch needs a bounded state/next-attempt scan. Review/reconciliation may need correlation, causation, policy, verdict and occurrence indexes—but only with a real supported query and volume.

Owned query	Candidate index	Verification
Duplicate inbound lookup	UNIQUE (producer, public_fact_id, contract_version).	Parallel/restart duplicate test; exact constraint classification.
Resolve current case	UNIQUE accepted correlation key, possibly scoped by correlation-policy identity.	One case under concurrent first fact; no ambiguous lookup.
Load case evidence	(case_id, canonical_position) plus UNIQUE (case_id, evidence_identity).	Exact order, late evidence policy and representative plan.
Hypothesis history	(case_id, hypothesis_identity, revision) and status query only if required.	Latest/complete history query and retention volume.
Verdict replay	UNIQUE verdict_identity; membership PK (verdict_id, position).	Exact decision/correction comparison and duplicate-verdict race.
Pending dispatch	Provider-appropriate pending state + next attempt + creation order.	Bounded backlog plan/latency; no unbounded table scan.
Evidence navigation	Correlation/causation/policy/time indexes only for owned review/reconciliation.	Representative query, retention horizon and write-cost review.

The initial migration belongs to RootCause

InitialRootCause is generated only after the production provider, net8.0-compatible EF Core packages, names, precision/collation rules, constraints, correlation policy and concurrency token are pinned. It lives with RootCause Infrastructure, carries its own model snapshot/history and is deployed by a separate migration identity. The running consumer does not call EnsureCreated or auto-migrate at startup. Chapter 16 defines the complete independent migration lifecycle.

INITIAL ROOTCAUSE MIGRATION WORKFLOW - [REFERENCE][PROVIDER + EXECUTION REQUIRED]

```
# Reference commands; not execution evidence.
dotnet ef migrations add InitialRootCause \
  --project src/Contexts/RootCause/Nexus1.RootCause.Infrastructure \
  --startup-project src/Hosts/Nexus1.RootCause.Host \
  --context RootCauseDbContext

dotnet ef migrations script 0 InitialRootCause \
  --project src/Contexts/RootCause/Nexus1.RootCause.Infrastructure \
  --startup-project src/Hosts/Nexus1.RootCause.Host \
  --context RootCauseDbContext \
  --output artifacts/migrations/rootcause-initial.sql
```

Migration check	Required proof	False green
Model completeness	All owned relations, keys, FKs, delete rules, unique/concurrency constraints, precision and indexes.	Migration succeeds from an incomplete case graph.
Empty create	Apply from zero on clean selected-provider database; compare schema/model identity.	EnsureCreated produced tables without migration history.
Upgrade	Apply from every supported prior identity with representative cases, evidence and pending facts.	Only a clean database passes.
Review	Inspect generated migration and SQL for drops, cascade, defaults, locks and destructive drift.	Generated code is trusted without human/provider review.
Runtime permissions	Consumer can perform owned DML but cannot alter schema; migration principal is short-lived/scoped.	Runtime starts with owner/administrator rights.
Failure / repair	Interrupt at controlled points: identify state and rehi	

Migration check	Required proof	False green
Restore compatibility	Restore backup, validate migration/policy identity and replay pending facts safely.	Backup exists but has never been restored and reconciled.

Credentials and operations complete RootCause ownership

Capability	Runtime principal	Migration / operations principal
DML	Only required SELECT/INSERT/UPDATE on RootCause-owned objects; DELETE only through approved lifecycle.	Repair/maintenance through reviewed runbook, never routine request path.
DDL	Denied/absent.	Apply reviewed RootCause migration artifact to RootCauseDb only.
Foreign databases	No AlarmManagement/Audit/Compliance/Reporting write; normally no direct read.	No unrelated database authority.
Secrets	Resolve Chapter 10 secret handle; values absent from logs, traces and evidence.	Short-lived/rotated identity; connection detail redacted from migration transcript.
Consumer readiness	Bounded RootCauseDb capability, pending backlog/capacity and broker intake state.	Validate schema identity, backup age and restore evidence out of request path.
Backup / restore	No authority to modify backup policy.	Owned RPO/RTO, encrypted backup, restore rehearsal and post-restore idempotency reconciliation.

PERMISSION EVIDENCE

DATA-CRED-001 and RCDB-SEC-001 require successful owned operations and denied DDL/foreign-schema attempts from the actual deployed identities. Configuration review alone does not prove effective database authority.

Selected-provider component tests are blocking

RootCauseDb is accepted only on the selected production-class engine. EF Core InMemory or a different relational provider cannot prove generated SQL, collation, timestamp precision, relationship/delete behaviour, unique exceptions, transactions, concurrency predicates, query plans or migrations. Every fixture creates an isolated database, applies the accepted migration chain, captures redacted provider/schema identities and disposes only its owned resource.

Scenario	Fault / contention	Required observation
First flood fact	No existing case under accepted correlation policy.	Inbound receipt, open case, admitted evidence and V1 commit; no verdict without policy/evidence rule.
Duplicate producer fact	Sequential, concurrent and after restart.	One inbound receipt and one RootCause effect; stable previous outcome.
Closed-case late fact	Ordinary consumer receives new or duplicate evidence after verdict.	RC-INV-001 rejection/route; no silent reopen or mutation.
Closure boundary	No evidence, one evidence, minimum/maximum policy boundary.	RC-INV-002 and exact verdict/evidence membership.
Concurrent different evidence	Two contexts load the same case version.	One commits; loser rolls back, reloads/replays; serializable final truth.
Unique verdict race	Same logical verdict derived concurrently.	One verdict and pending public fact; semantic mismatch blocks.
Correction	Authorized correction with new evidence/policy; retry after crash.	Prior verdict immutable; linked corrected verdict and stable corrected public fact identity.
Failure inside commit	Provider fault after commands begin.	All owned state/public intent or none; no orphan receipt/evidence/verdict/fact.
Migration create/upgrade	Empty and supported prior schema; representative volume	Exact schema identity, preserved history and accepted

Scenario	Fault / contention	Required observation
Permissions	Runtime attempts DDL/foreign write; migration identity targets unrelated DB.	Owned path succeeds; forbidden paths fail with safe evidence.
Restore/replay	Restore at accepted point with pending facts and redelivered inbound facts.	No new inbound/verdict fact identity; downstream effect remains idempotent.
Backlog query	Representative pending-public-fact and open-case volume.	Bounded plan/latency and capacity policy; no unbounded scan.

RootCauseDb evidence gates

Evidence ID	Blocking rule	Required artifact
RCDB-OWN-001	Every relation/column is RootCause truth, evidence or technical coordination; external facts remain references.	Fact-to-relation map and RC-OWN-001/DATA-OWN-001 review.
RCDB-MAP-001	EF model explicitly defines case graph, keys, FKs, delete behaviour, lengths, precision and indexes.	Model debug/schema manifest plus mapping tests.
RCDB-INV-001	Database and Domain agree on closed case, evidence-before-verdict, membership, ownership and immutability.	Constraint catalogue and negative provider/Domain tests.
RCDB-IDEM-001	Repeated producer fact creates at most one receipt and one RootCause effect across restart/concurrency.	Sequential/concurrent/restart duplicate report.
RCDB-DET-001	Same case, policy and canonical evidence/hypothesis set produce the same verdict identity/result.	Deterministic replay and canonicalization matrix.
RCDB-CON-001	Concurrent case work cannot commit from stale state; replay preserves RootCause invariants.	Two-writer schedules, predicate and final-state comparison.
RCDB-TXN-001	Inbound receipt, case state, evidence, hypothesis/verdict and pending fact commit atomically.	Injected-failure report and durable before/after state.
RCDB-MIG-001	Accepted migration creates/upgrades the selected provider without unreviewed/destructive drift.	Migration/source hash, reviewed SQL and create/upgrade/failure results.
RCDB-SEC-001	Runtime/migration identities are separate, least-privileged and reject foreign/schema operations.	Effective permission inventory and identity-bound positive/negative tests.
RCDB-OPS-001	Backup/restore, retention, policy/schema compatibility, capacity and pending-fact recovery meet lifecycle policy.	Restore/replay/capacity evidence and runbook.
RCDB-TEST-001	Provider-dependent claims execute on selected engine/version with repeatable isolated fixtures.	Provider/migration identities, result files and retained diagnostics.
RCDB-REVIEW-001	RootCause, architecture, data, security and operations accept scope/evidence/residual risk.	Identity-bound review with exceptions and expiry.

ADR-013 - Persist RootCause decisions with exact lineage

ADR field	ADR-013 record
Status	Accepted for the NEXUS-1 reference design; provider, migrations, implementation and execution evidence remain pending.
Context	A mutable Case/Verdict table cannot prove inbound idempotency, RC-INV-001/002, deterministic replay, exact evidence, correction history, concurrency or recoverable publication.
Decision	RootCauseDb stores immutable inbound facts and admitted evidence, versioned current case state, hypothesis revisions, immutable verdicts/corrections with exact ordered membership and pending public facts. One normal SaveChanges is the local commit; the case uses optimistic concurrency; inbound/evidence/verdict identities are unique.
Consequences	More history and relationships are retained; case decisions are replayable; contention is localized per case; correlation, storage, retention, privacy and provider-specific constraint/index work must be operated.

ADR field	ADR-013 record
Rejected	Shared AlarmManagement/RootCause schema, direct producer reads, generic repository, mutable verdict overwrite, arrival-order decision, catch-all Duplicate, distributed database transaction, runtime auto-migration.
Review trigger	Case-correlation, evidence admission/order, hypothesis/verdict policy, correction/retention, contract meaning, provider/token, transaction/publication or restore model changes.

Risk register for RootCauseDb

Risk	Early signal	Blocking control
Foreign truth	Case outcome changes because AlarmManagement table/query changes.	Public-contract-only input and RCDB-OWN-001 permission review.
Duplicate effect	Same producer fact adds repeated evidence/hypothesis/verdict.	Unique inbound identity plus RCDB-IDEM-001 across restart/concurrency.
Silent reopen	Late ordinary evidence changes a closed case.	RC-INV-001 transition tests and explicit correction route.
Empty verdict	Case closes with no admitted evidence/membership.	RC-INV-002, exact membership and transaction failure tests.
Arrival-order verdict	Workers/schedules produce different final root cause.	Canonical evidence order, pinned policy and RCDB-DET-001 replay.
Concurrency hotspot	Broad correlation collapses unrelated evidence into one contended case.	Case-correlation ADR, metrics, bounded replay and capacity test.
Orphan public fact	Verdict exists without recoverable producer intent or vice versa.	One local SaveChanges and RCDB-TXN-001 injected-failure tests.
Correction erases history	Original verdict row/payload is updated.	Immutable verdict store, linked correction and negative history tests.
Catch-all idempotency	FK/timeout/permission failures return Duplicate.	Known constraint/error classifier and unknown failure propagation.
Provider drift	Fake/local mapping passes while production SQL differs.	Pinned provider component suite and schema/migration identity.
Runtime schema privilege	Consumer calls Migrate/EnsureCreated or holds DDL rights.	Separate migration identity and startup/permission scan.
Restore duplicates	Restored receipts/pending facts receive new identities.	Identity-preserving restore/replay and downstream idempotency rehearsal.

Official-source validation

The EF Core mechanics were checked against current Microsoft documentation on 18 July 2026. Stable points used here are that EF relationship mappings connect primary/foreign keys and navigations; explicit model-building configuration has full fidelity and precedence; composite and unique keys/indexes are explicit; concurrency tokens participate in update predicates and conflicts raise `DbUpdateConcurrencyException`; one `SaveChanges` is transactional when the provider supports transactions; and generated migrations must be inspected and tested before production application.

Official source	Stable point used here	NEXUS-1 boundary
Microsoft Learn - EF Core relationships	Relational relationships use PK/FK constraints; model-building API explicitly configures navigations, keys and delete behaviour.	All case-graph relationships terminate inside RootCauseDb; no cross-service FK.
Microsoft Learn - EF Core keys and indexes	Primary/composite keys identify rows; unique indexes enforce selected column-set uniqueness; newer complex-type examples are version-sensitive.	net8.0 reference uses scalar row properties; inbound/evidence/verdict uniqueness is provider-proved.
Microsoft Learn - EF Core concurrency	Tracked concurrency tokens participate in update/delete predicates; zero affected rows raise <code>DbUpdateConcurrencyException</code> ; insert uniqueness is provider-specific.	Case token is a reference design; fresh semantic replay and exact classifier require selected-provider evidence.

Official source	Stable point used here	NEXUS-1 boundary
Microsoft Learn - EF Core transactions	A provider-supported single SaveChanges applies tracked changes transactionally; manual transactions/savepoints interact with provider and retry behaviour.	Normal RootCause intake uses one SaveChanges; multi-save transaction needs a separate accepted decision.
Microsoft Learn - EF Core applying migrations	Generated migrations should be inspected/tested; reviewed scripts support controlled production deployment; runtime migration has elevated-risk concerns.	RootCause owns a separate migration stream/identity; Chapter 16 defines lifecycle and deployment gates.

Current Microsoft pages may display EF Core releases newer than the book baseline and may mention keys/indexes over complex types or migration behaviour introduced after EF Core 8. This volume remains pinned to net8.0. It does not import EF Core 9/10/11 features until the repository explicitly changes its SDK/packages and supplies new compatibility evidence.

Technical material still required

The RootCause responsibilities, private relations, identities, case graph, constraints, evidence admission/order, transaction, concurrency replay, correction history, migration boundary, credentials, scenarios and evidence IDs are specified. No provider or EF package version has been pinned; no RootCause project has compiled; no migration/model snapshot/SQL exists; no database has enforced a RootCause key, relationship, transaction or concurrency token; and no public fact has been consumed or published.

TECHNICAL MATERIAL STATUS - [ROOTCAUSEDDB PROVIDER + CASE/VERDICT MIGRATION/CONCURRENCY EXECUTION EVIDENCE REQUIRED]

Create the net8.0 RootCause Domain/Application/Infrastructure/Host and test projects; pin compatible EF Core and production-provider packages/images; implement the aggregate, consumer use case, case-correlation/evidence policies, purpose-built store, row mappings and narrow error classifier; generate/review InitialRootCause; execute closed-case, evidence-before-verdict, duplicate/restart, deterministic ordering, concurrent case, atomic failure, correction, query-plan, permission, backup/restore and pending-fact recovery scenarios; publish RCDB-OWN-001 through RCDB-REVIEW-001 with linked RC/DATA/FLOW/TST evidence. RootCauseDb remains specified, not built or migrated. RootCause remains G0 accepted and G1 specified; runtime evidence has not advanced.

Deliverable	Minimum technical content	Blocking evidence
Domain/application model	RootCauseCase, evidence admission, hypothesis revisions, verdict/correction identity, correlation policy and store/use case.	RC-INV-001/002 / RC-OWN-001 / RCDB-OWN-001
EF model	DbContext, rows/mappings, keys, local FKs, delete rules, constraints, lengths/precision, token and indexes.	RCDB-MAP-001 / RCDB-INV-001
Provider adapter	Pinned engine/packages/options, exact constraint/error classification and token behaviour.	RCDB-TEST-001
Initial migration	Migration, snapshot, reviewed SQL/script/bundle and exact schema identity.	RCDB-MIG-001
Correctness suite	Closed/empty-verdict, duplicate/restart, deterministic replay, two-writer, unique verdict, correction and atomic fault scenarios.	RCDB-IDEM-001 / RCDB-DET-001 / RCDB-CON-001 / RCDB-TXN-001
Access/operations	Runtime/migration identities, denied foreign/DDI, query/capacity, privacy/retention and backup/restore/replay.	RCDB-SEC-001 / RCDB-OPS-001
Acceptance	Provider/schema/artifact identities, machine results, diagnostics, multidisciplinary review and residual risk.	RCDB-REVIEW-001

CHAPTER 15 - CODE AUDIT

RootCause decisions become a durable, replayable case graph

The code below joins the chapter's critical seams: producer-fact identity, case-owned evidence, immutable verdict lineage, exact membership and one pending public fact are derived from canonical domain meaning.

EVIDENCE STATUS

These assets are compile-oriented reference material. The current manuscript environment has no dotnet SDK, selected provider, database image, generated migration, or executed component suite. RCDB evidence therefore remains PENDING.

EXECUTABLE ASSET 15-A - OWNER-DEFINED IDENTITIES

```
public readonly record struct RootCauseCaseId(Guid Value);
public readonly record struct EvidenceId(Guid Value);
public readonly record struct HypothesisId(Guid Value);
public readonly record struct RootCauseVerdictId(Guid Value);
public readonly record struct PublicFactId(Guid Value);
public readonly record struct PolicyIdentity(string Value);

public sealed record ProducerFactIdentity(
    string Producer,
    Guid PublicFactId,
    string ContractVersion);

public sealed record CanonicalEvidenceInput(
    EvidenceId EvidenceId,
    DateTimeOffset ProducerOccurredAtUtc,
    string SourceOrderToken,
    string SourceSystem,
    string SourceEventId) : IComparable<CanonicalEvidenceInput>
{
    public int CompareTo(CanonicalEvidenceInput? other)
    {
        if (other is null) return 1;
        var time = ProducerOccurredAtUtc.CompareTo(other.ProducerOccurredAtUtc);
        if (time != 0) return time;
        var token = StringComparer.Ordinal.Compare(
            SourceOrderToken, other.SourceOrderToken);
        if (token != 0) return token;
        var source = StringComparer.Ordinal.Compare(
            SourceSystem, other.SourceSystem);
        return source != 0 ? source
            : StringComparer.Ordinal.Compare(SourceEventId, other.SourceEventId);
    }
}
```

EXECUTABLE ASSET 15-B - LENGTH-PREFIXED SEMANTIC IDENTITY

```
public static class StableIdentity
{
    public static string Create(params string[] fields)
    {
        var canonical = string.Concat(fields.Select(value =>
            $"{Encoding.UTF8.GetByteCount(value)}:{value}"));
        return Convert.ToHexString(
            SHA256.HashData(Encoding.UTF8.GetBytes(canonical)));
    }

    public static string Verdict(
        RootCauseCaseId caseId,
        PolicyIdentity policy,
        IEnumerable<EvidenceId> orderedEvidence,
        IEnumerable<HypothesisId> orderedHypotheses) =>
        Create(
            "verdict-v1",
            caseId.Value.ToString("D"),
            policy.Value,
            string.Join("|", orderedEvidence.Select(x => x.Value.ToString("D"))),
            string.Join("|", orderedHypotheses.Select(x => x.Value.ToString("D"))));
}
```

DOMAIN INVARIANTS

Only the aggregate can close or correct a case

EXECUTABLE ASSET 15-C - ROOTCAUSECASE TRANSITION BOUNDARY

```

public sealed class RootCauseCase
{
    private readonly List<CaseEvidence> _evidence = [];
    private readonly List<HypothesisRevision> _hypotheses = [];
    private readonly List<RootCauseVerdict> _verdicts = [];

    public RootCauseCaseId Id { get; }
    public CaseState State { get; private set; } = CaseState.Open;
    public PolicyIdentity PolicyIdentity { get; private set; }
    public Guid ConcurrencyVersion { get; private set; }
    public IReadOnlyList<CaseEvidence> Evidence => _evidence;
    public IReadOnlyList<RootCauseVerdict> Verdicts => _verdicts;

    public void Admit(CaseEvidence evidence)
    {
        if (State == CaseState.Closed)
            throw new ClosedCaseCannotChange();
        if (_evidence.Any(x => x.Id == evidence.Id))
            return;

        _evidence.Add(evidence);
        _evidence.Sort((a, b) => a.CanonicalOrder.CompareTo(b.CanonicalOrder));
        Touch();
    }

    public RootCauseVerdict Issue(VerdictDraft draft)
    {
        if (State == CaseState.Closed)
            throw new ClosedCaseCannotChange();
        var verdict = Append(draft, supersedes: null);
        State = CaseState.Closed;
        Touch();
        return verdict;
    }

    public RootCauseVerdict Correct(
        VerdictDraft draft,
        RootCauseCorrectionAuthority authority)
    {
        if (State != CaseState.Closed || _verdicts.Count == 0)
            throw new CaseHasNoVerdictToCorrect();
        authority.DemandRootCauseOwnership();
        var correction = Append(draft, _verdicts[^1].Id);
        Touch(); // State remains Closed.
        return correction;
    }

    private RootCauseVerdict Append(
        VerdictDraft draft,
        RootCauseVerdictId? supersedes)
    {
        if (_evidence.Count == 0 || draft.OrderedEvidenceIds.Count == 0)
            throw new VerdictRequiresEvidence();
        var admitted = _evidence.Select(x => x.Id).ToHashSet();
        if (draft.OrderedEvidenceIds.Any(id => !admitted.Contains(id)))
            throw new VerdictReferencesUnadmittedEvidence();

        var verdict = RootCauseVerdict.Create(
            Id, PolicyIdentity, draft, supersedes);
        _verdicts.Add(verdict); // Existing verdict rows never change.
        return verdict;
    }

    private void Touch() => ConcurrencyVersion = Guid.NewGuid();
}

```

EXECUTABLE ASSET 15-D - PINNED INVARIANT TESTS

```

[Fact]
public void RC_INV_001_closed_case_rejects_late_evidence()
{
    var sut = CaseFixture.ClosedWithEvidence();
    var act = () => sut.Admit(CaseFixture.NewEvidence("late"));
    act.Should().Throw<ClosedCaseCannotChange>();
}

[Fact]
public void RC_INV_002_empty_case_cannot_issue_verdict()
{
    var sut = CaseFixture.EmptyOpenCase();
    var act = () => sut.Issue(CaseFixture.VerdictDraft());
    act.Should().Throw<VerdictRequiresEvidence>();
}

[Fact]
public void RC_OWN_001_correction_preserves_original_verdict()
{
    var sut = CaseFixture.ClosedWithEvidence();
    var original = sut.Verdicts.Single();
    var correction = sut.Correct(
        CaseFixture.CorrectionDraft(), CaseFixture.RootCauseAuthority);
    sut.Verdicts.Should().Equal(original, correction);
    correction.Supersedes.Should().Be(original.Id);
}

```

STATE IS NOT RECONSTRUCTED BY EF

Materialization must rebuild a valid aggregate from owned history. Adapters do not set Closed, invent evidence membership, or overwrite a verdict to simulate correction.

PERSISTENCE RECORDS

Rows preserve case responsibilities and immutable lineage

EXECUTABLE ASSET 15-E - INBOUND IDENTITY AND CURRENT CASE ROWS

```
internal sealed class InboundPublicFactRow
{
    public Guid InboundFactId { get; init; }
    public required string Producer { get; init; }
    public Guid PublicFactId { get; init; }
    public required string ContractVersion { get; init; }
    public required string PayloadFingerprint { get; init; }
    public required string OutcomeKind { get; set; }
    public Guid? RootCauseCaseId { get; set; }
    public DateTimeOffset RecordedAtUtc { get; init; }
}

internal sealed class RootCauseCaseRow
{
    public Guid CaseId { get; init; }
    public required string CorrelationKey { get; init; }
    public required string State { get; set; }
    public required string PolicyIdentity { get; set; }
    public Guid ConcurrencyVersion { get; set; }
    public List<CaseEvidenceRow> Evidence { get; } = [];
    public List<CaseHypothesisRow> Hypotheses { get; } = [];
    public List<CaseVerdictRow> Verdicts { get; } = [];
}

internal sealed class CaseEvidenceRow
{
    public Guid EvidenceId { get; init; }
    public Guid CaseId { get; init; }
    public required string EvidenceIdentity { get; init; }
    public Guid InboundFactId { get; init; }
    public int CanonicalPosition { get; init; }
    public required string NormalizedKind { get; init; }
    public required string NormalizedValue { get; init; }
}

internal sealed class CaseHypothesisRow
{
    public Guid HypothesisId { get; init; }
    public Guid CaseId { get; init; }
    public required string HypothesisIdentity { get; init; }
    public int Revision { get; init; }
    public required string Status { get; init; }
}
```

EXECUTABLE ASSET 15-F - VERDICT MEMBERSHIP AND PRODUCER INTENT ROWS

```
internal sealed class CaseVerdictRow
{
    public Guid VerdictId { get; init; }
    public Guid CaseId { get; init; }
    public required string VerdictIdentity { get; init; }
    public required string PolicyIdentity { get; init; }
    public Guid? SupersedesVerdictId { get; init; }
    public DateTimeOffset IssuedAtUtc { get; init; }
    public List<VerdictEvidenceRow> Evidence { get; } = [];
}

internal sealed class VerdictEvidenceRow
{
    public Guid VerdictId { get; init; }
    public int Position { get; init; }
    public Guid EvidenceId { get; init; }
}

internal sealed class PendingPublicFactRow
{
    public Guid PublicFactId { get; init; }
    public required string FactType { get; init; }
    public required string SchemaVersion { get; init; }
    public required string PartitionKey { get; init; }
    public required string PayloadJson { get; init; }
    public DateTimeOffset CreatedAtUtc { get; init; }
    public DateTimeOffset? PublishedAtUtc { get; set; }
    public int AttemptCount { get; set; }
}
```

PRIVATE BY CONSTRUCTION

Every row type stays internal to RootCause Infrastructure. AlarmManagement, Audit, Compliance and Reporting receive public facts; they never reference these classes.

EF CORE MODEL

Keys, relationships and version checks carry owned meaning

EXECUTABLE ASSET 15-G - CASE, INBOUND AND EVIDENCE MAPPINGS

```
internal sealed class RootCauseCaseConfiguration
    : IEntityTypeConfiguration<RootCauseCaseRow>
{
    public void Configure(EntityTypeBuilder<RootCauseCaseRow> b)
    {
        b.ToTable("root_cause_cases");
        b.HasKey(x => x.CaseId);
        b.Property(x => x.CorrelationKey).HasMaxLength(220).IsRequired();
        b.Property(x => x.State).HasMaxLength(32).IsRequired();
        b.Property(x => x.PolicyIdentity).HasMaxLength(128).IsRequired();
        b.Property(x => x.ConcurrencyVersion)
            .IsConcurrencyToken().ValueGeneratedNever();
        b.HasIndex(x => x.CorrelationKey)
            .HasDatabaseName("ux_root_cause_cases_correlation")
            .IsUnique();
    }
}

internal sealed class InboundPublicFactConfiguration
    : IEntityTypeConfiguration<InboundPublicFactRow>
{
    public void Configure(EntityTypeBuilder<InboundPublicFactRow> b)
    {
        b.ToTable("inbound_public_facts");
        b.HasKey(x => x.InboundFactId);
        b.HasIndex(x => new {
            x.Producer, x.PublicFactId, x.ContractVersion
        })
            .HasDatabaseName("ux_inbound_public_facts_identity").IsUnique();
        b.Property(x => x.PayloadFingerprint).HasMaxLength(64).IsRequired();
        b.Property(x => x.OutcomeKind).HasMaxLength(32).IsRequired();
    }
}

internal sealed class CaseEvidenceConfiguration
    : IEntityTypeConfiguration<CaseEvidenceRow>
{
    public void Configure(EntityTypeBuilder<CaseEvidenceRow> b)
    {
        b.ToTable("case_evidence");
        b.HasKey(x => x.EvidenceId);
        b.HasIndex(x => new { x.CaseId, x.EvidenceIdentity })
            .HasDatabaseName("ux_case_evidence_identity").IsUnique();
        b.HasIndex(x => new { x.CaseId, x.CanonicalPosition }).IsUnique();
        b.HasOne<RootCauseCaseRow>().WithMany(x => x.Evidence)
            .HasForeignKey(x => x.CaseId).OnDelete(DeleteBehavior.Restrict);
        b.HasOne<InboundPublicFactRow>().WithMany()
            .HasForeignKey(x => x.InboundFactId).OnDelete(DeleteBehavior.Restrict);
    }
}
```

EXECUTABLE ASSET 15-H - IMMUTABLE VERDICT AND EXACT MEMBERSHIP MAPPINGS

```
internal sealed class CaseVerdictConfiguration
    : IEntityTypeConfiguration<CaseVerdictRow>
{
    public void Configure(EntityTypeBuilder<CaseVerdictRow> b)
    {
        b.ToTable("case_verdicts");
        b.HasKey(x => x.VerdictId);
        b.Property(x => x.VerdictIdentity).HasMaxLength(64).IsRequired();
        b.Property(x => x.PolicyIdentity).HasMaxLength(128).IsRequired();
        b.HasIndex(x => x.VerdictIdentity)
            .HasDatabaseName("ux_case_verdicts_identity").IsUnique();
        b.HasOne<RootCauseCaseRow>().WithMany(x => x.Verdicts)
            .HasForeignKey(x => x.CaseId).OnDelete(DeleteBehavior.Restrict);
        b.HasOne<CaseVerdictRow>().WithMany()
            .HasForeignKey(x => x.SupersedesVerdictId)
            .OnDelete(DeleteBehavior.Restrict);
    }
}

internal sealed class VerdictEvidenceConfiguration
    : IEntityTypeConfiguration<VerdictEvidenceRow>
{
    public void Configure(EntityTypeBuilder<VerdictEvidenceRow> b)
    {
        b.ToTable("verdict_evidence");
        b.HasKey(x => new { x.VerdictId, x.Position });
        b.HasIndex(x => new { x.VerdictId, x.EvidenceId }).IsUnique();
        b.HasOne<CaseVerdictRow>().WithMany(x => x.Evidence)
            .HasForeignKey(x => x.VerdictId).OnDelete(DeleteBehavior.Restrict);
        b.HasOne<CaseEvidenceRow>().WithMany()
            .HasForeignKey(x => x.EvidenceId).OnDelete(DeleteBehavior.Restrict);
    }
}
```

MAPPING IS NECESSARY, NOT SUFFICIENT

The database preserves identities, membership and concurrency predicates. Only the aggregate admits evidence, closes a case and authorizes an immutable correction.

ATOMIC STAGING

Case truth and public intent enter one tracked unit of work

EXECUTABLE ASSET 15-I - VERDICT PUBLIC-FACT FACTORY

```
internal static class RootCauseVerdictPublicFactFactory
{
    public static PendingPublicFactRow Create(
        RootCauseCase aggregate,
        RootCauseVerdict verdict,
        TimeProvider clock)
    {
        var orderedEvidence = verdict.OrderedEvidenceIds.ToArray();
        var evidenceIds = orderedEvidence.Select(x => x.Value).ToArray();
        var (factType, payload) = verdict.Supersedes is { } prior
            ? (
                "RootCauseVerdictCorrected",
                JsonSerializer.Serialize(
                    new RootCauseVerdictCorrectedV1(
                        verdict.PublicFactId.Value,
                        aggregate.Id.Value,
                        verdict.Id.Value,
                        prior.Value,
                        aggregate.PolicyIdentity.Value,
                        verdict.VerdictIdentity,
                        evidenceIds),
                    RootCauseContractsJsonContext.Default
                        .RootCauseVerdictCorrectedV1))
            : (
                "RootCauseVerdictIssued",
                JsonSerializer.Serialize(
                    new RootCauseVerdictIssuedV1(
                        verdict.PublicFactId.Value,
                        aggregate.Id.Value,
                        verdict.Id.Value,
                        aggregate.PolicyIdentity.Value,
                        verdict.VerdictIdentity,
                        evidenceIds),
                    RootCauseContractsJsonContext.Default
                        .RootCauseVerdictIssuedV1));

        return new PendingPublicFactRow
        {
            PublicFactId = verdict.PublicFactId.Value,
            FactType = factType,
            SchemaVersion = "v1",
            PartitionKey = aggregate.Id.Value.ToString("D"),
            PayloadJson = payload,
            CreatedAtUtc = clock.GetUtcNow()
        };
    }
}
```

EXECUTABLE ASSET 15-J - ONE SAVECHANGES COMMIT

```
internal sealed class EfRootCauseDecisionStore(
    RootCauseDbContext db,
    ITrackedRootCauseCase trackedCase,
    IDatabaseErrorClassifier databaseErrors,
    TimeProvider clock) : IRootCauseDecisionStore
{
    public void Stage(HandleFloodResult result)
    {
        db.InboundPublicFacts.Add(RootCauseRows.From(result.InboundFact));
        trackedCase.Apply(result.Case, nextVersion: Guid.NewGuid());

        db.Evidence.AddRange(result.NewEvidence.Select(RootCauseRows.From));
        db.Hypotheses.AddRange(
            result.NewHypothesisRevisions.Select(RootCauseRows.From));

        if (result.NewVerdict is { } verdict)
        {
            db.Verdicts.Add(RootCauseRows.From(verdict));
            db.VerdictEvidence.AddRange(
                verdict.OrderedEvidenceIds.Select((id, position) =>
                    RootCauseRows.Membership(verdict.Id, id, position)));
            db.PendingPublicFacts.Add(
                RootCauseVerdictPublicFactFactory.Create(
                    result.Case, verdict, clock));
        }
    }

    public async Task CommitAsync(Cancellation token)
    {
        try
        {
            await db.SaveChangesAsync(token);
        }
        catch (DbUpdateConcurrencyException ex)
        {
            throw new RootCauseCaseConflictException(ex);
        }
        catch (DbUpdateException ex)
            when (databaseErrors.TryUnique(ex, out var conflict))
        {
            throw conflict switch
            {
                UniqueConflict.InboundFact =>
                    new InboundFactAlreadyProcessedException(ex),
                UniqueConflict.Evidence =>
                    new EvidenceAlreadyAdmittedException(ex),
                UniqueConflict.Verdict =>
                    new VerdictAlreadyRecordedException(ex),
                _ => throw new UnreachableException()
            };
        }
    }
}
```

SEMANTIC RETRY

A losing writer reloads the complete owned case graph

EXECUTABLE ASSET 15-K - FRESH-UNIT-OF-WORK CONSUMER HANDLER

```
public sealed class HandleAlarmFloodDetectedHandler(
    IRootCauseUnitOfWorkFactory units,
    IRootCausePolicy policy,
    IRootCauseDuplicateResolver duplicates,
    ICaseCorrelationPolicy correlation)
{
    private const int MaxAttempts = 3;

    public async Task<HandleFloodOutcome> Handle(
        AlarmFloodDetectedV1 message,
        CancellationToken cancellationToken)
    {
        var inbound = ProducerFactIdentity.From(message);
        var caseId = correlation.Resolve(message);

        for (var attempt = 1; attempt <= MaxAttempts; attempt++)
        {
            await using var unit =
                await units.CreateAsync(cancellationToken);
            var state = await unit.Store.LoadAsync(
                caseId, inbound, cancellationToken);

            if (state.TryGetPriorOutcome(inbound, out var previous))
                return HandleFloodOutcome.Duplicate(previous);

            var result = policy.Apply(
                state,
                message.ToCandidateEvidence(),
                state.PolicyIdentity);
            unit.Store.Stage(result);

            try
            {
                await unit.Store.CommitAsync(cancellationToken);
                return HandleFloodOutcome.Applied(
                    result.Case.Id, result.NewVerdict?.Id);
            }
            catch (InboundFactAlreadyProcessedException)
            {
                return await duplicates.ResolveAsync(
                    inbound, cancellationToken);
            }
            catch (RootCauseCaseConflictException)
            {
                when (attempt < MaxAttempts)
                {
                    // Disposal removes every stale tracked row and pending fact.
                }
            }
        }

        return HandleFloodOutcome.Contended(MaxAttempts);
    }
}
```

EXECUTABLE ASSET 15-L - RETRY INVARIANTS

```
[Fact]
public async Task RCDB_CON_001_retry_uses_fresh_semantic_state()
{
    var units = new RecordingUnitOfWorkFactory(
        conflictsBeforeSuccess: 1);
    var handler = RootCauseFixture.CreateHandler(units);

    await handler.Handle(
        RootCauseFixture.Flood("fact-42"),
        CancellationToken.None);

    units.Created.Should().HaveCount(2);
    units.Created[0].WasDisposed.Should().BeTrue();
    units.Created[1].LoadedVersion.Should()
        .Be(units.WinningDatabaseVersion);
    units.Created[1].StagedVerdictIdentity.Should()
        .Be(RootCauseFixture.SequentialOracleVerdictIdentity);
    units.Created[0].PendingFactWasReused.Should().BeFalse();
}
```

RETRY IS NOT ENTITY MERGING

CurrentValues.SetValues, retry-count-dependent policy and reuse of the losing verdict/public fact all preserve state meaning. The whole unit of work is replaced.

PROVIDER ADAPTER

Only named uniqueness violations become idempotency outcomes

EXECUTABLE ASSET 15-M - PROVIDER-NEUTRAL CLASSIFICATION PORT

```
internal enum UniqueConflict { InboundFact, Evidence, Verdict }

internal interface IDatabaseErrorClassifier
{
    bool TryUnique(
        DbUpdateException exception,
        out UniqueConflict conflict);
}

internal static class KnownRootCauseConstraints
{
    public const string InboundFact =
        "ux_inbound_public_facts_identity";
    public const string Evidence =
        "ux_case_evidence_identity";
    public const string Verdict =
        "ux_case_verdicts_identity";
}
```

EXECUTABLE ASSET 15-N - POSTGRESQL CANDIDATE ADAPTER

```
internal sealed class PostgreSQLRootCauseErrorClassifier
    : IDatabaseErrorClassifier
{
    public bool TryUnique(
        DbUpdateException exception,
        out UniqueConflict conflict)
    {
        conflict = default;
        if (exception.InnerException is not PostgresException
            {
                SqlState: PostgresErrorCodes.UniqueViolation,
                ConstraintName: { } constraint
            })
            return false;

        return constraint switch
        {
            KnownRootCauseConstraints.InboundFact =>
                Set(UniqueConflict.InboundFact, out conflict),
            KnownRootCauseConstraints.Evidence =>
                Set(UniqueConflict.Evidence, out conflict),
            KnownRootCauseConstraints.Verdict =>
                Set(UniqueConflict.Verdict, out conflict),
            _ => false
        };
    }

    private static bool Set(
        UniqueConflict value,
        out UniqueConflict target)
    {
        target = value;
        return true;
    }
}
```

EXECUTABLE ASSET 15-O - CLASSIFIER SAFETY TESTS

```
[Theory]
[InlineData(KnownRootCauseConstraints.InboundFact,
    UniqueConflict.InboundFact)]
[InlineData(KnownRootCauseConstraints.Evidence,
    UniqueConflict.Evidence)]
[InlineData(KnownRootCauseConstraints.Verdict,
    UniqueConflict.Verdict)]
public void known_unique_constraint_is_classified(
    string name, UniqueConflict expected)
{
    var error = ProviderExceptionFixture.UniqueViolation(name);
    classifier.TryUnique(error, out var actual).Should().BeTrue();
    actual.Should().Be(expected);
}

[Fact]
public void foreign_key_timeout_or_permission_is_not_duplicate()
{
    var error = ProviderExceptionFixture.ForeignKeyViolation();
    classifier.TryUnique(error, out _).Should().BeFalse();
}
```

CANDIDATE, NOT PROVIDER SELECTION

The Npgsql shape is explicit because PostgreSQL exposes structured constraint names. A different engine requires its own exact adapter and provider-backed tests; matching exception message text remains forbidden.

TRANSACTIONAL AND CONCURRENCY PROOF

A real database must falsify partial or stale RootCause truth

EXECUTABLE ASSET 15-P - ATOMIC FAILURE AND TWO-WRITER ORACLE

```
[Fact]
public async Task RCDB_TXN_001_failed_public_fact_insert_rolls_back_case()
{
    await fixture.ResetAndMigrateAsync();
    fixture.DatabaseCommandFault.FailOnceBefore(
        table: "pending_public_facts");

    var act = () => fixture.HandleInNewProcessScopeAsync(
        RootCauseFixture.VerdictProducingFlood("fact-A"), start: null);
    await act.Should().ThrowAsync<InjectedDatabaseFault>();

    var durable = await fixture.ReadCountsAsync();
    durable.InboundFacts.Should().Be(0);
    durable.Cases.Should().Be(0);
    durable.Evidence.Should().Be(0);
    durable.Verdicts.Should().Be(0);
    durable.PendingFacts.Should().Be(0);
}

[Fact]
public async Task RCDB_CON_001_concurrent_inputs_equal_sequential_oracle()
{
    await fixture.ResetAndMigrateAsync();
    var first = RootCauseFixture.Flood("A", occurredSecond: 10);
    var second = RootCauseFixture.Flood("B", occurredSecond: 11);
    var bothLoaded = new Barrier(2);

    var outcomes = await Task.WhenAll(
        fixture.HandleInNewProcessScopeAsync(first, bothLoaded),
        fixture.HandleInNewProcessScopeAsync(second, bothLoaded));
    outcomes.Should().OnlyContain(x =>
        x.Kind == HandleFloodOutcomeKind.Applied);

    var durable = await fixture.ReadCaseSnapshotAsync();
    var oracle = RootCausePolicy.Replay(
        new[] { first, second }.OrderBy(x => x.CanonicalOrder),
        RootCauseFixture.AcceptedPolicy());
    durable.OrderedEvidenceIds.Should().Equal(oracle.OrderedEvidenceIds);
    durable.VerdictIdentity.Should().Be(oracle.VerdictIdentity);
    durable.PendingFactIds.Should().OnlyHaveUniqueItems();
}
```

EXECUTABLE ASSET 15-Q - DUPLICATE EFFECT ON THE REAL PROVIDER

```
[Fact]
public async Task RCDB_IDEM_001_concurrent_duplicate_has_one_effect()
{
    var flood = RootCauseFixture.Flood("same-public-fact");
    var start = new Barrier(2);
    await Task.WhenAll(
        fixture.HandleInNewProcessScopeAsync(flood, start),
        fixture.HandleInNewProcessScopeAsync(flood, start));

    var durable = await fixture.ReadCountsAsync();
    durable.InboundFacts.Should().Be(1);
    durable.Evidence.Should().Be(1);
    durable.PendingFacts.Should().BeLessThanOrEqualTo(1);
}
```

EXECUTABLE ASSET 15-R - ACCEPTANCE EVIDENCE RECORD

```
{
    "evidenceId": "RCDB-REVIEW-001", "chapter": 15,
    "artifactCommit": null, "sdk": "net8.0",
    "efCorePackage": null, "provider": null,
    "providerVersion": null, "migrationIdentity": null,
    "schemaDigest": null,
    "tests": {
        "RCDB-OWN-001": "PENDING", "RCDB-MAP-001": "PENDING",
        "RCDB-INV-001": "PENDING", "RCDB-IDEM-001": "PENDING",
        "RCDB-DET-001": "PENDING", "RCDB-CON-001": "PENDING",
        "RCDB-TXN-001": "PENDING", "RCDB-MIG-001": "PENDING",
        "RCDB-SEC-001": "PENDING", "RCDB-OPS-001": "PENDING",
        "RCDB-TEST-001": "PENDING"
    },
    "runtimeDdlDenied": false, "foreignWritesDenied": false,
    "restoreRehearsed": false, "accepted": false
}
```

Chapter 15 code-audit result

The chapter now exposes the implementation path from producer-fact intake to immutable verdict lineage and recoverable publication, including the two failure families senior developers must distinguish: optimistic case conflict and unique insert violation.

CURRENT VERDICT

RootCauseDb is implementable in shape, but it has not compiled, migrated, or executed. Provider choice, generated SQL, permissions, fault injection, restore, and every RCDB evidence gate remain explicitly PENDING.

Implementation sequence

1. Accept ADR-013 and freeze case-correlation, producer identity, evidence admission/order, policy, verdict and correction contracts.
2. Generate the RootCause projects from the accepted repository shape; keep Domain/Application free of EF/provider and AlarmManagement private dependencies.
3. Pin net8.0-compatible EF Core and production-provider packages, engine image/version, connection options, concurrency token and provider ADR.
4. Implement RootCauseCase, evidence/hypothesis/verdict/correction history and the HandleAlarmFloodDetected use case with a purpose-built store.
5. Implement RootCauseDbContext, persistence rows/mappings, RootCauseDb-local relationships, constraints and query-driven indexes.
6. Generate and inspect InitialRootCause; create from zero and upgrade every supported prior identity with representative case history.
7. Implement exact inbound/evidence/verdict unique classification and DbUpdateConcurrencyException translation; unknown failures remain failures.
8. Execute RC invariants, duplicate/restart, deterministic replay, concurrent schedules, correction, atomic fault and pending-fact scenarios on the real provider.
9. Provision runtime/migration identities; prove owned operations, denied DDL/foreign access, secrets, capacity/readiness and query plans.
10. Rehearse backup/restore/schema-policy compatibility and inbound/pending-fact replay; publish RCDB evidence without advancing unproved runtime gates.

What this chapter has proved

Chapter 15 has defined the executable persistence contract for RootCauseDb: producer-fact-only integration; one owned consumer use case; immutable inbound identity and admitted evidence; versioned case state; hypothesis and verdict/correction history; exact policy/evidence membership; one local commit with pending public fact; optimistic case concurrency with semantic replay; explicit RootCauseDb-local relationships, constraints, indexes, migration, credentials and evidence gates.

It has not selected or run a provider, compiled RootCause code, generated a migration, created a database, consumed AlarmFloodDetected.v1, enforced a case invariant, won a concurrency race, issued/corrected a verdict, published RootCauseVerdictIssued.v1, restored a backup or advanced G1 runtime evidence. The design is ready to implement; RCDB, RC, DATA, FLOW, TST and migration evidence remain pending.

DECISION OWNERSHIP MADE DURABLE

RootCauseDb is not a cache of AlarmManagement facts and not a mutable verdict table. It is the atomic, replayable record of what RootCause accepted, how its case evolved, which evidence supported its decision and which immutable public fact must leave the boundary.

Chapter checkpoint

You are ready to implement RootCauseDb when the producer/correlation/evidence/policy identities are accepted, the case graph enforces RootCause ownership, the normal consumer path has one local commit, conflicts reload semantic truth, correction preserves history, and the selected provider can falsify every relationship, constraint, migration, recovery and permission claim.

Question	Expected answer
What is RootCauseDb's source truth?	RootCause-owned case state, admitted evidence, hypotheses, verdict/correction history, inbound outcomes and pending producer facts.
What remains external reference data?	AlarmFloodDetected.v1 producer identity, contract version and detection lineage; never an AlarmManagementDb FK or direct read.
What preserves RC-INV-001?	Closed state in the aggregate, explicit correction route, case concurrency and negative late-evidence provider tests.
What preserves RC-INV-002?	Evidence admission plus exact verdict membership before closure; empty-verdict paths fail without partial commit.
What preserves RC-OWN-001?	Only the RootCause application/domain issue or correct verdicts; credentials/contracts forbid foreign writes/commands.
What makes consumption idempotent?	Unique producer + PublicFactId + contract version, stored prior outcome and one effect across concurrency/restart.
What serializes concurrent decisions?	A versioned RootCauseCase token; losers roll back, reload the full owned graph and reapply the same policy.
What commits atomically?	Inbound receipt, case state, admitted evidence, hypothesis/verdict change and pending public fact in one local SaveChanges.
How is a verdict corrected?	A new authorized, linked immutable verdict/correction and public fact; the original is never overwritten.
What is the current status?	RootCauseDb is specified, not built/migrated; RootCause remains G0 accepted and G1 specified without runtime evidence.
What does Chapter 16 add?	Independent migration lifecycles, artifact identities, compatibility matrices, deployment sequencing, rollback/repair and restore gates.

NEXT CHAPTER

Chapter 16, Independent Migrations and Database Lifecycle, turns the AlarmManagementDb and RootCauseDb designs into separately versioned schema streams: reviewed artifacts, compatibility ranges, expand/contract sequencing, deployment and rollback authority, failure repair, backup/restore gates and evidence that one service can evolve its database without coordinating a shared-schema release.

PART III - DATA OWNERSHIP IN EXECUTABLE FORM

16. Independent Migrations and Database Lifecycle

A service owns its data only when it can version, review, deploy, repair and restore its schema without joining a shared database release.

AlarmManagementDb and RootCauseDb therefore carry different migration histories, artifacts, credentials, compatibility windows and operational decisions - even when a public contract causes one application release to precede another.

CHAPTER ARGUMENT

Treat every service schema change as an owner-scoped release artifact. Pin its source, provider and tool identities; declare the application-schema compatibility window; prove create, upgrade, interruption and restore paths; and let only the owning service's migration authority apply it. Public contract compatibility may coordinate traffic, but never creates a shared migration stream.

Chapters 14 and 15 specified separate AlarmManagementDbContext and RootCauseDbContext models. Each owns its model snapshot, migration source, history table, selected provider, initial migration, runtime principal and migration principal. ALMDB-MIG-001 and RCDB-MIG-001 already require create and supported-upgrade evidence. This chapter adds the release lifecycle that makes those local designs independently operable.

The current manuscript contains no AlarmManagement or RootCause project, provider package, model snapshot, migration source, generated SQL, bundle, database, migration identity or execution result. The commands and manifests below are net8.0 reference contracts. They do not assert that an artifact exists, compiles or runs. RootCause remains G0 accepted and G1 specified; neither service has gained runtime migration evidence.

Database ownership includes release ownership

Two connection strings are not sufficient separation. If both services must use one migration project, one history table, one deployment credential, one schema window or one rollback decision, the release train still owns the data. Independent ownership requires an authority and evidence boundary for every transition from model change to verified database state.

Lifecycle concern	AlarmManagement owner	RootCause owner	Forbidden coupling
Source	AlarmManagement EF model, snapshot and migration source.	RootCause EF model, snapshot and migration source.	One shared snapshot or migration assembly.
History	AlarmManagementDb migration history and accepted current identity.	RootCauseDb migration history and accepted current identity.	A shared history table or cross-database ordering number.
Artifact	Provider-pinned script/bundle with ALMDB manifest and hash.	Provider-pinned script/bundle with RCDB manifest and hash.	One package that can mutate both databases.
Authority	Short-lived AlarmManagement migration principal and approver.	Short-lived RootCause migration principal and approver.	Runtime identity, global DDL account or another service's operator.
Compatibility	Accepted AlarmManagement app-to-schema range.	Accepted RootCause app-to-schema range.	Assuming newest application always matches newest schema.
Recovery	Owner-specific stop, repair, rollback or restore decision.	Owner-specific stop, repair, rollback or restore decision.	Rolling both databases back because one migration failed.
Evidence	ALMDB-MIG/SEC/OPS/TEST artifacts.	RCDB-MIG/SEC/OPS/TEST artifacts.	A green shared pipeline with no service/database identity.

Two migration streams, one public boundary

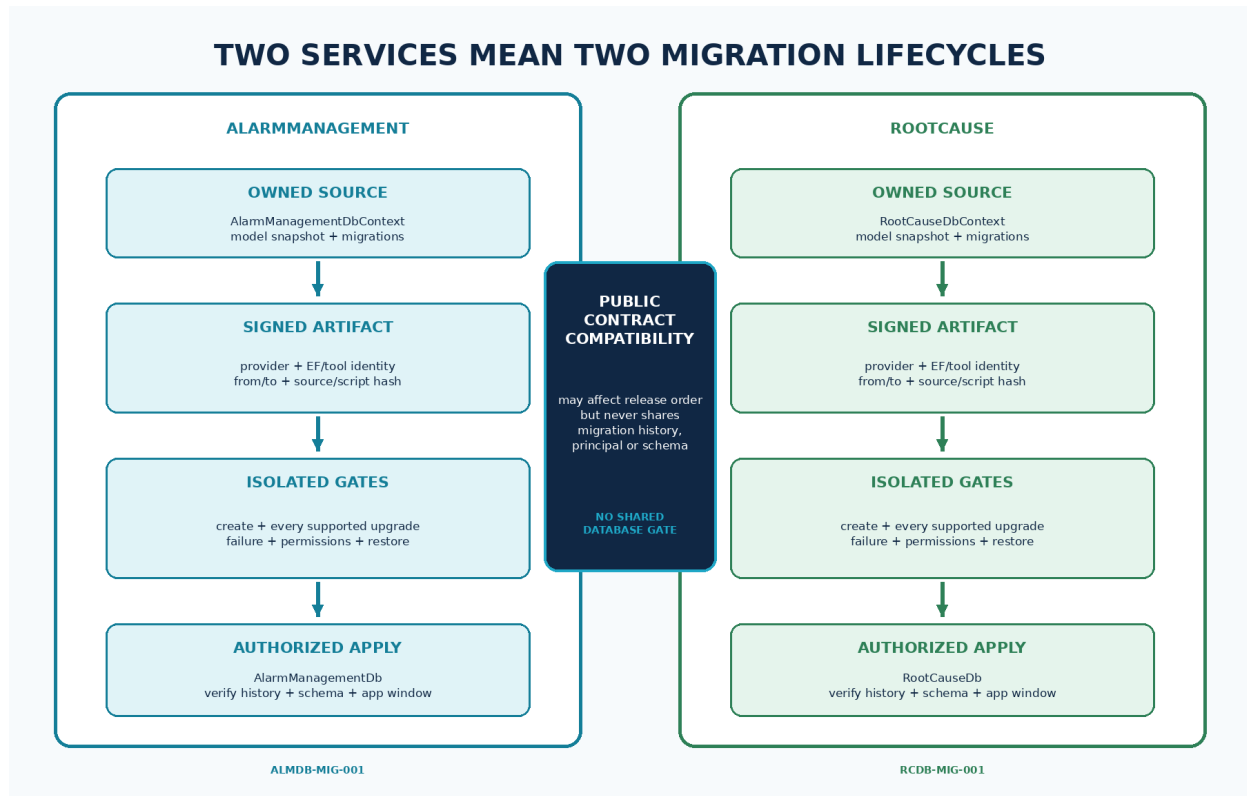


Figure 16.1 - AlarmManagement and RootCause generate, test, authorize and verify different migration artifacts. Public contract compatibility can order releases without sharing database authority.

A RootCause release may require AlarmFloodDetected.v2 before it can enable a new case policy. That is a public contract dependency, not permission for RootCause to migrate AlarmManagementDb. AlarmManagement first deploys a backward-compatible producer and schema within its own window; RootCause then deploys a compatible consumer and its own schema. Either database can remain unchanged when only the other owner's private model evolves.

INDEPENDENCE TEST

Ask whether RootCauseDb can add, validate and deploy a private index while AlarmManagementDb remains on its current migration, and whether AlarmManagement can repair its own failed schema change without holding RootCause's migration credential. If either answer is no, the lifecycle is still shared.

A migration artifact is more than generated SQL

The reviewed deployment unit pairs immutable bytes with a manifest. The manifest binds the artifact to one service, one DbContext, one exact source revision, one from/to migration range, one model snapshot, one provider/engine family, one EF package/tool set and one accepted compatibility claim. An unlabelled script cannot prove which database state it expects or which application can safely run afterward.

Manifest field	Required meaning	Blocking check
Owner / database / context	Exactly one service, logical database and DbContext migration stream.	Reject multi-owner or wildcard target.
From / to migration identity	Last accepted applied identity and intended final identity, including create-from-zero when applicable.	Preflight queries actual history; unexpected state blocks.
Source + snapshot hash	Repository revision, migration files and model snapshot used to generate the artifact.	Rebuild comparison or signed provenance matches.
Runtime baseline	net8.0 plus exact EF runtime/design/tool and provider package versions.	No floating tool restore or silent EF major upgrade.

Manifest field	Required meaning	Blocking check
Provider target	Engine family/version, identifier rules, collation/precision assumptions and transaction behavior.	Run on the same supported provider/version family.
Artifact identity	Exact SQL script or migration bundle hash, platform identity if executable, and generation command.	Bytes applied equal bytes reviewed and published.
Change classification	Additive, data-moving, constraint/index, destructive or irreversible semantics.	Destructive/data-loss path needs explicit approval and recovery choice.
Compatibility window	Application artifacts allowed before, during and after the transition.	Deployment controller blocks an incompatible app/schema pair.
Budgets	Lock, duration, log/storage growth, backfill rate and maintenance/traffic posture.	Representative test plus production preflight fits limits.
Repair / recovery	Known interruption states, forward repair, reviewed reversal where safe, restore point and verification.	Operator can choose from evidence, not improvise under failure.
Approval / evidence	Reviewer identities, expiry, test reports and residual risk.	Authorization is bound to this artifact and target state.

MIGRATION ARTIFACT MANIFEST - [REFERENCE ONLY][REAL VALUES REQUIRED]

```

service: RootCause
database: RootCauseDb
context: RootCauseDbContext
fromMigration: 202607010900_InitialRootCause
toMigration: 202607181430_AddVerdictCorrectionLineage
sourceRevision: REQUIRED_REPOSITORY_COMMIT
modelSnapshotSha256: REQUIRED_MODEL_SNAPSHOT_SHA256
artifactSha256: REQUIRED_ARTIFACT_SHA256
target:
  framework: net8.0
  efRuntime: REQUIRED_PINNED_EF8_VERSION
  efTool: REQUIRED_PINNED_EF8_TOOL_VERSION
  provider: REQUIRED_PROVIDER_AND_VERSION
compatibility:
  before: [REQUIRED_CURRENT_APP_ARTIFACT]
  during: [REQUIRED_CURRENT_APP_ARTIFACT, REQUIRED_NEXT_APP_ARTIFACT]
  after: [REQUIRED_NEXT_APP_ARTIFACT]
evidence: [RCDB-MIG-001, MIG-COMPAT-001, MIG-FAIL-001]
status: SPECIMEN_ONLY_NOT_GENERATED

```

Keep the migration project owner-local

EF Core can keep migrations beside Infrastructure or in a separate owner-specific migrations project. Either choice is acceptable when its project and startup project are explicit, the output folder belongs to one DbContext, the model snapshot remains in that stream and the build cannot discover another service's provider or connection by accident. A separate project is packaging, not an excuse to centralize migrations across services.

OWNER-SCOPED EF CORE COMMANDS - [REFERENCE ONLY][COMPILE + PROVIDER REQUIRED]

```

# Reference generation only - exact package/tool versions must be pinned.
dotnet ef migrations add AddVerdictCorrectionLineage \
  --project src/Nexus1.RootCause.Infrastructure \
  --startup-project src/Nexus1.RootCause.Host \
  --context RootCauseDbContext \
  --output-dir Persistence/Migrations

# Exact range for one known starting state.
dotnet ef migrations script \
  202607010900_InitialRootCause \
  202607181430_AddVerdictCorrectionLineage \
  --project src/Nexus1.RootCause.Infrastructure \
  --startup-project src/Nexus1.RootCause.Host \
  --context RootCauseDbContext \
  --output artifacts/rootcause.sql

# A fleet script may be idempotent, but history checks do not prove data meaning.
dotnet ef migrations script --idempotent \
  --project src/Nexus1.RootCause.Infrastructure \
  --startup-project src/Nexus1.RootCause.Host \
  --context RootCauseDbContext \
  --output artifacts/rootcause-idempote

```

Generated files are reviewed and committed like source. The generated Up and Down operations, SQL identifiers, default values, nullability, data conversions, indexes, constraint timing and destructive warnings are inspected. The artifact is then regenerated under the pinned toolchain. A source diff alone is insufficient because provider output and production execution semantics remain to be proved.

The production application never becomes the migrator

Production Host startup must not call `EnsureCreated`, `Migrate` or `MigrateAsync`. The runtime principal can execute owned DML and required reads but has no DDL permission. A deployment job or operator obtains the owner-specific migration principal for one authorized window, targets one validated database identity, applies one reviewed artifact and relinquishes the credential. Application replicas remain outside that authority.

Identity	Allowed	Denied	Evidence
AlarmManagement runtime	Owned AlarmManagement DML and approved metadata reads.	DDL, RootCauseDb access, migration history mutation and privilege grant.	ALMDB-SEC-001 positive/negative tests.
AlarmManagement migrator	Approved artifact against validated AlarmManagementDb during authorized window.	RootCauseDb, unrelated schemas, runtime traffic and broad server administration.	Target-bound credential and ALMDB-MIG-001 audit.
RootCause runtime	Owned RootCause DML and approved metadata reads.	DDL, AlarmManagementDb access, migration history mutation and privilege grant.	RCDB-SEC-001 positive/negative tests.
RootCause migrator	Approved artifact against validated RootCauseDb during authorized window.	AlarmManagementDb, unrelated schemas, runtime traffic and broad server administration.	Target-bound credential and RCDB-MIG-001 audit.
Deployment controller	Request short-lived identity, verify manifest/evidence and record outcome.	Read service data, edit SQL after approval or reuse credential across owners.	Identity-bound authorization log and expiry.

NET8.0 CAVEAT

The EF Core documentation now describes automatic migration locking and pending-model checks that begin with EF Core 9. This book remains on net8.0/EF Core 8. It does not inherit those behaviors. A selected-provider deployment mechanism must prove single-executor coordination, model/snapshot drift detection and interruption handling explicitly.

Compatibility is an executable release contract

A migration ending successfully does not prove that the next or previous application can use the resulting schema. Each application artifact declares the schema identities it can read and write. The controller compares that declaration with the database preflight and the transition phase. The window is intentionally finite; it is not the claim that every historical application works forever.

Application artifact	Schema S0 current	Schema S1 expanded	Schema S2 contracted	Release use
N	Read/write accepted.	Read/write accepted; ignores additive shape.	Blocked: old write/read path removed.	Rollback target only before contract.
N+1 compatibility build	Optional only when code can tolerate absence of new shape.	Read/write accepted; old and new meaning reconciled.	Read/write accepted after proof.	Bridge during expand and backfill.
N+2 post-contract	Blocked.	Optional read/write if old shape still present.	Read/write accepted.	Normal target after old artifact retired.
Migration worker	May apply exact S0 to S1 artifact.	May backfill/checkpoint and later apply S1 to S2 after authorization.	No-op verification only.	Never serves application traffic.

Read compatibility and write compatibility are separate. An old reader may tolerate an additive nullable column while an old writer violates a new required invariant. A new reader may understand

both representations while a new writer must write only one canonical value. The manifest records the exact behavior; labels such as backward compatible or zero downtime are not evidence.

Expand and contract preserve a mixed-version window

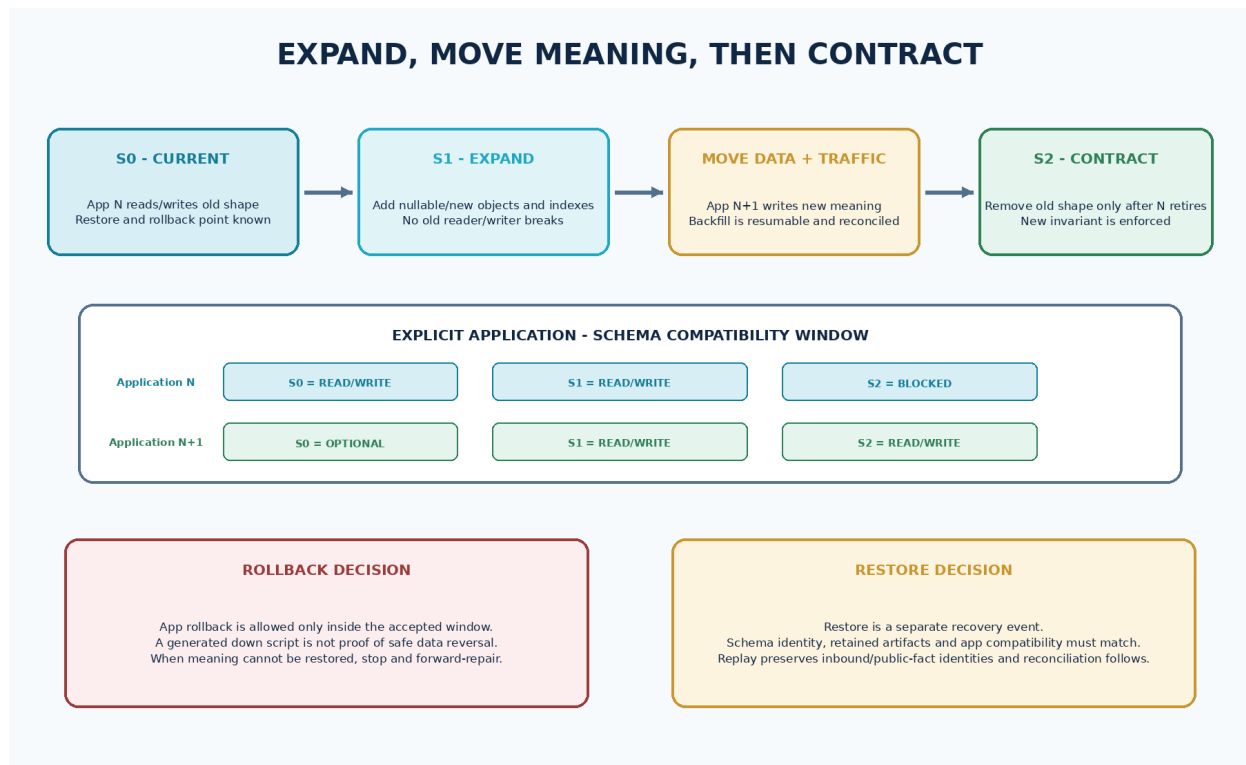


Figure 16.2 - Additive schema, compatible application, bounded data movement and delayed contraction create a reversible application window without pretending that destructive schema reversal is safe.

1. Freeze the current S0 schema identity, representative data, accepted application artifact and restore point.
2. Apply S1 expand operations that do not break N: add nullable columns/tables, additive indexes or new constraints in a non-enforcing phase where the provider supports it.
3. Deploy the compatibility build that reads accepted old/new states and writes the canonical transition shape without cross-service database writes.
4. Run a resumable owner-local backfill with checkpoints, bounded batches, throttling and reconciliation. Public facts remain the cross-service integration path.
5. Prove every row and application instance is on the accepted representation; retire N and close the rollback window deliberately.
6. Apply S2 contract operations, enforce the final invariant and publish the new minimum application/schema identity.

CONTRACT GATE

Do not drop, rename in place, tighten nullability or enforce a new invariant merely because the backfill job reported completion. Contract requires independent reconciliation, old-artifact retirement, backup/restore readiness and an explicit irreversible-change authorization.

Backfills are production programs

A data-moving migration is not hidden inside one unbounded transaction. The owner defines the source of truth, transformation/policy identity, stable partition key, checkpoint, batch size, retry semantics, lock/log budget, pause switch and reconciliation query. The operation is idempotent by business identity, not by elapsed time or process memory.

RESUMABLE OWNER-LOCAL BACKFILL - [REFERENCE ONLY][COMPILE + PROVIDER TEST REQUIRED]

```

public async Task<BackfillBatchResult> ExecuteBatch(
    BackfillCheckpoint checkpoint,
    CancellationToken cancellationToken)
{
    var rows = await source.LoadAfter(
        checkpoint.LastStableKey, options.BatchSize, cancellationToken);

    foreach (var row in rows)
    {
        var expected = projector.Project(row, policyIdentity);
        await target.UpsertIfEquivalent(expected, cancellationToken);
    }

    var proof = await reconciler.Compare(rows, cancellationToken);
    if (!proof.IsEquivalent)
        return BackfillBatchResult.Blocked(proof.Differences);

    return BackfillBatchResult.Advance(rows.LastOrDefault()?.StableKey);
}

```

Backfill property	Required rule	Failure evidence
Stable selection	Keyset/partition cursor with deterministic ordering; no OFFSET drift or host-clock cutoff.	Restart and concurrent-write schedules neither skip nor repeat meaning.
Idempotent write	Same source identity/policy yields the same target meaning; mismatch blocks.	Replayed batches do not create a second effect or overwrite different truth.
Bounded work	Batch/time/lock/log budgets and pause/resume controls are explicit.	Representative volume stays within SLO and maintenance posture.
Concurrent traffic	Accepted application versions define writes during movement and conflict resolution.	Mixed-version writer matrix converges to the canonical representation.
Reconciliation	Counts are diagnostic; identity- and value-level equivalence proves completion.	Missing, extra and semantically different rows are retained and triaged.
Retention	Checkpoint, policy, artifact, errors and proof survive process restart.	Operator resumes the same job identity instead of inventing a new run.

Deployment is a guarded owner-local state machine

Only one owner-specific migration execution is admitted for a database transition. The guard may be supplied by the deployment platform or a selected-provider mechanism, but the EF Core 8 baseline does not assume automatic locking. The executor records durable state before and after each phase so a second operator can distinguish not-started, applying, interrupted, applied and verified without rerunning blindly.

Phase	Blocking precondition	Action	Durable outcome
Resolve	Target service/database identity and environment are unambiguous.	Read migration history, schema fingerprint, active app artifacts and capacity posture.	Signed preflight report or stop.
Authorize	Manifest, artifact hash, tests, window, budgets and recovery choice are accepted.	Issue short-lived owner migration credential and single-executor lease.	Authorization bound to target/from/to/hash/expiry.
Prepare	Backup/restore point is usable; traffic posture matches migration class.	Quiesce writers only when required; confirm storage/log/replica health.	Start record with evidence links.
Apply	Actual state still equals accepted from identity.	Execute exact reviewed artifact; retain statement/tool/provider diagnostics.	Applied, failed-before-change or interrupted/unknown.
Verify	Execution ended without losing target identity.	Check history, schema fingerprint, constraints/indexes, data reconciliation and app window.	Verified target identity or blocked failure state.
Release	Compatible application and readiness probes pass.	Enable traffic, observe workload and close credential/lease.	Owner-local release record and residual-risk timer.

Failure repair begins by identifying the durable state

A tool exit code is not a complete database state. Provider DDL transaction behavior varies, scripts may be split into batches and an interrupted connection may leave the executor uncertain whether the server committed. The operator first freezes traffic when required, preserves logs and database evidence, reads history and physical schema, and classifies the state. Re-execution is allowed only when the accepted artifact is proved safe from that exact state.

Observed state	Do not	Required decision
Failed before first change	Assume a schema defect or mark target applied.	Correct environment/authority issue; re-authorize same immutable artifact if evidence permits.
History unchanged; physical schema unchanged	Regenerate a different script casually.	Prove no durable change, then retry under a new execution record.
History unchanged; physical schema partially changed	Run the full artifact again or insert a history row manually.	Use reviewed provider-specific repair/forward script from exact observed state, then reconcile.
History advanced; physical schema incomplete	Trust history alone or release application.	Block traffic; repair schema/data and history through an approved recovery procedure.
History and schema at target; data movement incomplete	Declare success because DDL finished.	Resume the same backfill job identity and reconcile before contract/release.
Target reached; application incompatible	Roll schema backward automatically.	Deploy an accepted compatible app, forward-repair, or restore through the separate recovery gate.
Executor outcome unknown	Retry from a timeout callback.	Query target state and provider activity; classify before any second execution.

HISTORY TABLE BOUNDARY

EF migration history tells which migration identities EF recorded as applied. It does not prove that the physical schema, data transformation, permissions, indexes, replicas or application compatibility are correct. Verification compares all of those surfaces.

Rollback, forward repair and restore are different controls

Rollback is not a generic undo button. An application rollback is usually the first option while the expanded schema still supports N. A schema reversal requires a separately generated, reviewed and executed artifact and may be impossible when data was transformed or discarded. Forward repair preserves the reached state and corrects it with another authorized change. Restore replaces the database state and introduces RPO, replay and compatibility consequences.

Control	Use when	Must prove	Typical rejection
Application rollback	Schema remains inside the old artifact's read/write window.	Old binary/config/contract can run and will not corrupt new representation.	S2 removed old shape or new writers introduced unsupported meaning.
Schema down/reversal	Every reversal operation and data consequence is known and accepted.	Reviewed down SQL, representative data result, lock budget and final app compatibility.	Drop/data conversion is irreversible or restored meaning is ambiguous.
Forward repair	Target is reachable and preserving current data is safer than reversal.	Exact observed-state input, repair idempotency, verification and compatible app path.	Observed state cannot be identified or repair expands damage.
Database restore	Failure/loss requires returning to an accepted backup point.	Backup integrity, schema/migration identity, app window, RPO/RTO and post-restore checks.	Restored schema has no supported app/artifact or backup is unverified.
Restore plus replay	Inbound/public facts after backup can be safely redelivered.	Stable business identities, idempotent consumers/producers, ordering policy and reconciliation.	Replay creates new identities or required external history is unavailable.

Restore must rejoin the release timeline

A backup is useful only with its database identity, last applied migration, schema fingerprint, application compatibility window, policy/contract identities and pending-message state. After restore, traffic remains blocked until the owner proves that an available application can run the restored schema, the required migration artifact chain is retained, runtime/migration permissions are correct and inbox/outbox or pending-public-fact identities can be reconciled.

1. Restore into an isolated target and verify backup integrity, database owner and encryption/key access.
2. Read migration history and compare the physical schema to the retained schema fingerprint for that backup point.
3. Select an application artifact explicitly compatible with the restored schema; never point the newest binary at it by default.
4. Apply only retained, reviewed migration artifacts for every required forward transition and re-run their verification gates.
5. Reconcile AlarmManagement accepted/pending facts or RootCause inbound/pending facts and preserve their original business identities during replay.
6. Prove RPO/RTO, application readiness and downstream idempotency before enabling traffic; record any accepted loss or residual risk.

Cross-service sequencing stays at the contract boundary

Independent does not mean unordered. A public fact version, authentication policy or readiness dependency may impose a release sequence. The sequence is expressed as consumer/producer contract compatibility and traffic enablement. It never becomes a shared database transaction, shared migration number or a requirement to apply private schema changes in lockstep.

Change	AlarmManagementDb action	RootCauseDb action	Coordination
Private AlarmManagement index	Apply owner artifact and verify plan/capacity.	None.	No public contract change; no RootCause release gate.
Private RootCause verdict index	None.	Apply owner artifact and verify query/replay.	No AlarmManagement credential or schema change.
Add optional AlarmFloodDetected.v2 field	Expand schema/code, publish compatible v1/v2 contract and evidence.	May stay unchanged or deploy tolerant consumer later.	Contract matrix and traffic telemetry; databases migrate independently.
RootCause requires v2 meaning	Producer must already publish accepted v2 while preserving agreed window.	Expand/migrate/deploy v2 consumer, then enable policy.	Enablement waits on public fact evidence, not shared migration.
AlarmManagement migration fails	Owner stops/repairs/restores its database.	Continue only under accepted input/backlog/dependency policy.	Readiness and backlog policy; never migrate RootCause as compensation.
RootCause restore	Continue producing under capacity/backpressure policy.	Restore, migrate, replay and reconcile RootCause identities.	Public fact retention/replay contract; no AlarmManagementDb restore.

Observe identities and states, not secrets

Migration telemetry carries service, environment, database fingerprint, from/to identity, artifact hash, execution ID, phase, duration, lock/wait and row/backfill counts. It records reviewer/authorization references and verification results. Connection strings, credentials, private row payloads and generated SQL containing sensitive literals are excluded or protected. Application health reports compatibility state without exposing DDL authority.

Signal	Purpose	Alert / block condition
migration_preflight	Actual history/schema/app identity versus manifest.	Unknown target, drift, incompatible app or missing recovery evidence.

Signal	Purpose	Alert / block condition
migration_execution	Owner/from/to/hash/phase/duration/outcome.	Concurrent executor, expired authority, timeout or unknown completion.
migration_lock_wait	Provider/platform coordination and workload contention.	Accepted budget exceeded or writer impact breaches posture.
backfill_progress	Stable cursor, batch rate, retries and estimated remaining work.	No progress, repeated mismatch, unbounded log/lock or pause failure.
migration_reconciliation	History, schema, constraints/indexes and data equivalence.	Any mismatch; traffic/contract stays blocked.
schema_compatibility	Active app artifacts allowed for current/transition schema.	Blocked pair appears or rollback window closes unexpectedly.
restore_replay	Backup point, schema path, replay identities, duplicates and missing effects.	New business identity, unexplained loss/extra effect or RPO/RTO breach.

Test the lifecycle from every supported state

Migration testing is a state matrix, not one empty-database smoke test. Every supported from identity is restored with representative data, scale, pending work and application versions. The exact reviewed artifact runs on the selected engine/version. Failure injection interrupts the provider/tool at planned points. The suite retains before/after history, schema, data reconciliation, time/lock/log metrics and permission results.

Scenario	Starting state	Acceptance
Create from zero	Empty selected-provider database with migration principal only.	Exact target schema/history and compatible app startup; runtime DDL denied.
Every supported upgrade	Each retained migration identity plus representative owned history.	Preserved meaning, exact target and accepted time/lock/storage budget.
Mixed application window	N and N+1 instances against S1 under real writes.	Declared read/write matrix holds; convergence/reconciliation succeeds.
Backfill restart	Interrupted at every checkpoint boundary and amid concurrent writes.	Resume has no missing/extra/different meaning; same job identity advances.
Migration interruption	Failure before, during and after provider DDL/data statements.	Observed state classified; approved retry/repair/restore reaches verified target.
Permission boundary	Runtime, correct migrator, foreign-service and expired identities.	Only correct owner migrator mutates schema; all denials are safe/audited.
Application rollback	S1 with new writes and prior artifact available.	Rollback allowed only where compatibility and data semantics remain accepted.
Schema reversal	Representative S1/S2 data where a down path is claimed.	Reviewed reversal preserves declared meaning or the path is rejected.
Restore then migrate	Backup at every retained recovery point.	Schema/app/artifact identities align; forward chain and verification succeed.
Restore then replay	Pending and already-processed inbound/public facts around backup point.	Same business identities produce no duplicate/missing effect after reconciliation.
Independent release	One database migrates while the other remains unchanged.	No foreign credential, history or schema mutation; public contract SLO remains accepted.

Migration lifecycle evidence gates

Evidence ID	Blocking rule	Required artifact
MIG-OWN-001	One service owns source, history, artifact, authority and recovery decision for one database.	Owner map plus independent-release and denied-foreign tests.
MIG-ID-001	Reviewed bytes bind to source/snapshot, from/to, net8/EF8 toolchain, provider and target.	Signed manifest, hashes and reproducible generation record.
MIG-COMPAT-001	Every deployment/rollback uses an explicit application read/write to schema window.	Executable compatibility matrix and mixed-version results.

Evidence ID	Blocking rule	Required artifact
MIG-EXP-001	Destructive/enforcing change follows accepted expand, move, reconcile and contract phases.	Phase evidence, old-artifact retirement and contract authorization.
MIG-DATA-001	Backfill is bounded, resumable, idempotent and reconciled under concurrent traffic.	Checkpoint/results/metrics and value-level comparison.
MIG-LOCK-001	Exactly one authorized executor transitions a database; EF8 automatic locking is not assumed.	Provider/platform coordination and concurrent-executor test.
MIG-FAIL-001	Every injected interruption is classified from durable history/schema/data state.	Failure matrix, retained diagnostics and approved repair result.
MIG-ROLL-001	Application rollback, schema reversal and forward repair are separately authorized and tested.	Compatibility/reversal/repair reports with rejected unsafe paths.
MIG-REST-001	Restore re-establishes schema/app compatibility and preserves replay identities within RPO/RTO.	Restore-migrate-replay rehearsal and reconciliation.
MIG-SEC-001	Runtime has no DDL; owner migration authority is target-bound, short-lived and rejects foreign scope.	Effective rights plus positive/negative identity tests.
MIG-OBS-001	Preflight, execution, verification, backfill and restore states are attributable without secrets.	Metrics/traces/audit sample and alert tests.
MIG-TEST-001	Exact artifacts execute from zero and every supported prior identity on the selected provider/version.	Repeatable provider fixtures, results and retained diagnostics.
MIG-REVIEW-001	Owner, architecture, data, security and operations accept the artifact, evidence and residual risk.	Identity-bound review, exceptions and expiry.

ALMDB-MIG-001 and RCDB-MIG-001 remain the service-specific acceptance records. They reference MIG-ID-001 through MIG-TEST-001 for lifecycle proof and keep their own provider, schema, data and operator evidence. A shared MIG gate is a common quality rule, not a shared migration artifact or authority.

ADR-014 - Version and deploy each service schema independently

ADR field	ADR-014 record
Status	Accepted for the NEXUS-1 reference design; provider, artifact mechanism, implementation and execution evidence remain pending.
Context	Separate databases still become release-coupled when migrations share source, history, credentials, compatibility assumptions or recovery decisions.
Decision	AlarmManagementDb and RootCauseDb own different model snapshots, migration histories, provider-pinned reviewed artifacts, manifests, application-schema windows, principals, deployment state machines and recovery evidence. Production applications never auto-migrate.
Consequences	Private schemas can evolve independently; every artifact carries more provenance/test/operations work; contract dependencies are expressed outside the database; expand-contract and retained recovery chains become product responsibilities.
Rejected	Shared migration project/history/principal, multi-database artifact, runtime EnsureCreated/Migrate, newest-app assumption, unbounded data migration, blind retry, automatic down migration and coordinated restore of unrelated service databases.
Review trigger	Provider/EF baseline, schema ownership, deployment platform, compatibility policy, destructive change, backfill, public contract, credential, retention, backup/restore or replay model changes.

Risk register for independent migrations

Risk	Early signal	Blocking control
Shared release train	One migration job/hash/credential targets both databases.	MIG-OWN-001 owner map and denied-foreign tests.
Artifact drift	Applied bytes cannot be tied to reviewed source	MIG-ID-001 manifest, hashes and immutable

Risk	Early signal	Blocking control
Incompatible startup	Newest binary starts against unknown/restored schema.	MIG-COMPAT-001 preflight and readiness block.
Runtime DDL	Host invokes EnsureCreated/Migrate or runtime can alter schema.	Code/config scan, least privilege and MIG-SEC-001.
Concurrent migration	Two jobs/replicas attempt the same EF8 transition.	Single-executor control and MIG-LOCK-001 failure test.
Long blocking change	Lock/log/replica lag exceeds preflight budget.	Expand-contract, representative load and abort/pause decision.
Backfill divergence	Counts match while values/identities differ.	MIG-DATA-001 identity/value reconciliation.
Blind retry	Timeout causes immediate second execution.	Durable-state classification and MIG-FAIL-001 repair matrix.
False rollback safety	Generated Down is treated as lossless proof.	Separate reversal tests; prefer compatible app rollback or forward repair.
Unusable restore	Backup lacks schema/app/artifact identity.	MIG-REST-001 restore-migrate-replay rehearsal.
Secret leakage	SQL/log/telemetry contains credentials or private payload.	Redaction, protected artifacts and MIG-OBS-001 review.
Contract disguised as schema	Consumer release waits for direct producer database change/read.	Public contract matrix and credential/schema isolation.

Official-source validation

The EF Core mechanics were checked against current Microsoft documentation on 18 July 2026. Stable points used here are that migrations are generated by comparing the current model with a model snapshot, source files are retained in version control and applied identities are recorded in a history table; generated migrations must be inspected and tested; reviewed SQL scripts are the recommended production deployment path; exact range, rollback-direction and idempotent scripts are supported; migration bundles are deployable artifacts; and application-startup migration is inappropriate for production for authority, concurrency, review and rollback reasons.

Official source	Stable point used here	NEXUS-1 boundary
Microsoft Learn - Migrations overview	Snapshot comparison generates migration source; applied migrations are recorded in history.	Each DbContext owns a separate snapshot/history; history is necessary but not sufficient verification.
Microsoft Learn - Applying migrations	Inspect/test first; SQL scripts are recommended for production and can target a from/to range.	Exact reviewed bytes and manifest are the baseline; selected deployment mechanism still needs provider evidence.
Microsoft Learn - Idempotent scripts	A script can consult migration history and apply only missing migrations.	Useful for mixed starting identities, but it does not prove physical/data/app compatibility.
Microsoft Learn - Migration bundles	A bundle packages migrations into an executable and can be built in CI for later application.	Bundle platform/config/transaction/failure behavior must be pinned, reviewed and tested before selection.
Microsoft Learn - Runtime migration	Production startup migration is discouraged; pre-EF9 instances can migrate concurrently and runtime needs elevated rights.	net8/EF8 Hosts never migrate; deployment has separate owner authority and single-executor proof.
Microsoft Learn - Managing/separate projects	Migration source is versioned; target/startup projects and a separate migrations project can be explicit.	Project separation remains owner-local and cannot centralize multiple service schemas.

The current Applying Migrations page also documents automatic migration locking and pending-model enforcement beginning with EF Core 9. Those statements are intentionally excluded from this net8.0/EF Core 8 reference. The repository must explicitly change framework/packages and regenerate compatibility evidence before adopting them. Sources:

<https://learn.microsoft.com/en-us/ef/core/managing-schemas/migrations/> and

<https://learn.microsoft.com/en-us/ef/core/managing-schemas/migrations/applying>

Technical material still required

The independent ownership model, artifact manifest, compatibility window, expand-contract phases, data-movement contract, executor states, failure classification, rollback/repair/restore choices, cross-service boundary, telemetry, scenarios and evidence IDs are specified. No migration project or artifact

database has been created, upgraded, interrupted, repaired, reversed or restored; and no runtime or migration permission has been exercised.

TECHNICAL MATERIAL STATUS - [PROVIDER-PINNED MIGRATION ARTIFACT + COMPATIBILITY/FAILURE/RESTORE EXECUTION EVIDENCE REQUIRED]

Create the owner-specific AlarmManagement and RootCause migration projects/streams; pin net8.0-compatible EF Core runtime/design/tool packages, production providers and engine images; generate separate initial and next migrations, snapshots, exact-range SQL and manifests; choose and prove the deployment mechanism and EF8 single-executor control; execute create, every supported upgrade, mixed application-schema, expand/backfill/contract, interruption/repair, permission, application rollback, safe reversal where claimed, backup/restore/migrate/replay and independent-release scenarios; publish MIG-OWN-001 through MIG-REVIEW-001 linked to ALMDB-MIG-001 and RCDB-MIG-001. Both databases remain specified, not built or migrated. RootCause remains G0 accepted and G1 specified; runtime evidence has not advanced.

Deliverable	Minimum technical content	Blocking evidence
Owner streams	Separate projects/folders, DbContexts, snapshots, history configuration and source controls.	MIG-OWN-001 / ALMDB-MIG-001 / RCDB-MIG-001
Pinned toolchain	net8/EF8 runtime/design/tool/provider versions and selected engine images/options.	MIG-ID-001 / MIG-TEST-001
Artifacts	Create and exact supported-range SQL or accepted bundles with manifests/hashes/review.	MIG-ID-001 / MIG-REVIEW-001
Compatibility	Read/write matrices for current, expand, contract and restored schema identities.	MIG-COMPAT-001 / MIG-EXP-001
Data movement	Resumable job, checkpoint, budgets, traffic policy and reconciliation.	MIG-DATA-001
Deployment control	Target preflight, separate principal, single executor, phases, audit and readiness.	MIG-LOCK-001 / MIG-SEC-001 / MIG-OBS-001
Failure/recovery	Interrupted-state repairs, app rollback, reviewed reversal where safe, forward repair and restore/replay.	MIG-FAIL-001 / MIG-ROLL-001 / MIG-REST-001
Acceptance	Selected-provider results from zero/every supported state and multidisciplinary residual-risk review.	MIG-TEST-001 / MIG-REVIEW-001

Implementation sequence

1. Accept ADR-014 and freeze AlarmManagementDb and RootCauseDb ownership, retained migration states, recovery points and application-schema compatibility policy.
2. Create separate owner-local migration streams and pin net8.0-compatible EF runtime/design/tool/provider packages plus selected engine versions.
3. Generate initial migrations and model snapshots; inspect source and create-from-zero provider SQL; record exact schema fingerprints.
4. Implement the artifact manifest, immutable publication/hashing and owner-specific review/authorization workflow.
5. Implement preflight, target identity, separate short-lived principal, EF8 single-executor coordination and durable execution/verification states.
6. Define S0/S1/S2 application read/write matrices and build an expand-contract change with a resumable, reconciled backfill.
7. Generate/test exact artifacts from every supported prior migration identity with representative data, traffic, lock/log/storage budgets and provider failure injection.
8. Exercise application rollback, reviewed schema reversal only where safe, forward repair and unknown-outcome classification without blind retry.

9. Restore every retained backup point, select a compatible application, apply retained artifacts, replay stable fact identities and reconcile effects within RPO/RTO.
10. Prove one service can migrate privately while the other remains unchanged; publish MIG, ALMDB and RCDB evidence without advancing unproved runtime gates.

What this chapter has proved

Chapter 16 has defined independent migration lifecycles for AlarmManagementDb and RootCauseDb: owner-local source/history; provider-pinned reviewed artifacts and manifests; separate runtime/migration authority; explicit application-schema compatibility; expand-move-reconcile-contract sequencing; bounded backfill; single-executor deployment; durable failure classification; distinct application rollback, schema reversal, forward repair and restore; identity-preserving replay; contract-only cross-service coordination; and evidence gates tied to each service's own database.

It has not selected EF/provider/tool versions, generated or reviewed SQL, created a migration project, applied a schema, moved data, exercised a credential, interrupted or repaired a migration, rolled back an application, reversed a schema, restored a backup, replayed a fact or advanced runtime evidence. The lifecycle is ready to implement; ALMDB, RCDB and MIG evidence remains pending.

SCHEMA FREEDOM REQUIRES OPERATIONAL DISCIPLINE

A service is free to change its private database only after it can state exactly which artifact changes which known schema, which applications remain safe, who may apply it, how failure is classified and how the owner restores meaning. Independence removes shared release coupling; it does not remove evidence.

Close the transition with one owner record

Verification closes the execution, not merely the tool process. The owner publishes one immutable close-out record that lets deployment, operations and a future restore determine exactly which durable state was accepted and which obligations remain open.

Close-out field	Recorded proof	Why it matters later
Target identity	Service, database fingerprint, from/to migration, source/snapshot/tool/provider and artifact hashes.	A later deploy or restore can select the exact retained chain.
Durable result	History and physical schema fingerprints, constraints/indexes, backfill checkpoint and value-level reconciliation.	A green exit code cannot conceal incomplete schema or data movement.
Compatibility state	Active application artifacts, read/write window, traffic posture and explicit rollback-window status.	Readiness and rollback never infer safety from newest labels.
Authority and execution	Execution ID, migration principal, authorization/lease expiry, reviewer and protected diagnostics.	The change remains attributable without retaining broad credentials.
Recovery posture	Backup point, reversal/forward-repair decision, retained artifacts, RPO/RTO result and residual-risk review date.	The next incident begins from an accepted recovery choice, not memory.

CHAPTER 16 - CODE AUDIT

A migration becomes an owner-scoped, verifiable release

The code below makes the lifecycle executable: immutable artifact identity, application-schema compatibility, owner-only authority, bounded execution, durable recovery and restore evidence are evaluated before a private schema changes.

EVIDENCE STATUS

These assets are compile-oriented reference material. No EF toolchain, provider, engine image, generated migration, deployment platform, backup or executed test suite is present here. MIG, ALMDB-MIG and RCDB-MIG evidence remains PENDING.

EXECUTABLE ASSET 16-A - IMMUTABLE MIGRATION ARTIFACT MANIFEST

```
public readonly record struct MigrationIdentity(string Value);
public readonly record struct SchemaIdentity(string Value);
public readonly record struct Sha256Digest(string Value);

public sealed record RuntimeBaseline(
    string Framework,
    string EfRuntime,
    string EfDesign,
    string EfTool);

public sealed record ProviderTarget(
    string ProviderPackage,
    string ProviderVersion,
    string EngineFamily,
    string EngineVersion);

public sealed record MigrationArtifactManifest(
    string Owner,
    string Database,
    string DbContext,
    MigrationIdentity FromMigration,
    MigrationIdentity ToMigration,
    string SourceRevision,
    Sha256Digest ModelSnapshot,
    Sha256Digest Artifact,
    RuntimeBaseline Runtime,
    ProviderTarget Provider,
    ChangeClassification Change,
    MigrationBudgets Budgets,
    CompatibilityWindow Compatibility,
    IReadOnlyList<string> EvidenceIds)
{
    public void DemandTarget(
        string owner,
        string database,
        string dbContext)
    {
        if (!StringComparer.Ordinal.Equals(Owner, owner)
            || !StringComparer.Ordinal.Equals(Database, database)
            || !StringComparer.Ordinal.Equals(DbContext, dbContext))
            throw new ForeignMigrationTarget();
        if (FromMigration == ToMigration)
            throw new EmptyMigrationRange();
    }
}
```

EXECUTABLE ASSET 16-B - BYTE IDENTITY IS CHECKED BEFORE AUTHORIZATION

```
public static class ArtifactDigest
{
    public static async Task<Sha256Digest> ReadAsync(
        string path,
        CancellationToken cancellationToken)
    {
        await using var stream = File.OpenRead(path);
        var bytes = await SHA256.HashDataAsync(
            stream, cancellationToken);
        return new Sha256Digest(Convert.ToHexString(bytes));
    }

    public static async Task DemandAsync(
        string artifactPath,
        Sha256Digest expected,
        CancellationToken cancellationToken)
    {
        var actual = await ReadAsync(
            artifactPath, cancellationToken);
        if (!CryptographicOperations.FixedTimeEquals(
            Convert.FromHexString(actual.Value),
            Convert.FromHexString(expected.Value)))
            throw new MigrationArtifactHashMismatch(expected, actual);
    }
}
```

EXECUTABLE OWNERSHIP

Each DbContext resolves to one private migration stream

EXECUTABLE ASSET 16-C - OWNER-LOCAL MIGRATION CATALOGUE

```
public sealed record MigrationStreamDescriptor(
    string Owner,
    string Database,
    string DbContext,
    string MigrationsAssembly,
    string HistoryTable,
    string RuntimePrincipal,
    string MigrationPrincipal,
    string Provider);

public sealed class MigrationOwnershipCatalogue
{
    private readonly IReadOnlyDictionary<string, MigrationStreamDescriptor> _byOwner;

    public MigrationOwnershipCatalogue(
        IEnumerable<MigrationStreamDescriptor> descriptors)
    {
        var streams = descriptors.ToArray();
        if (streams.GroupBy(x => x.Owner, StringComparer.Ordinal)
            .Any(x => x.Count() != 1))
            throw new DuplicateMigrationOwner();
        if (streams.GroupBy(x => x.MigrationsAssembly, StringComparer.Ordinal)
            .Any(x => x.Select(y => y.Owner).Distinct().Count() != 1))
            throw new SharedMigrationAssembly();
        if (streams.GroupBy(x => x.MigrationPrincipal, StringComparer.Ordinal)
            .Any(x => x.Select(y => y.Owner).Distinct().Count() != 1))
            throw new SharedMigrationPrincipal();
        if (streams.Any(x => StringComparer.Ordinal.Equals(
            x.RuntimePrincipal, x.MigrationPrincipal)))
            throw new RuntimeHasMigrationAuthority();

        _byOwner = streams.ToDictionary(
            x => x.Owner, StringComparer.Ordinal);
    }

    public MigrationStreamDescriptor Demand(
        string owner,
        string database,
        string dbContext)
    {
        if (!_byOwner.TryGetValue(owner, out var stream)
            || !StringComparer.Ordinal.Equals(stream.Database, database)
            || !StringComparer.Ordinal.Equals(stream.DbContext, dbContext))
            throw new ForeignMigrationTarget();
        return stream;
    }
}
```

EXECUTABLE ASSET 16-D - OWNERSHIP GATES

```
[Fact]
public void MIG_OWN_001_each_owner_has_one_stream()
{
    var catalogue = new MigrationOwnershipCatalogue(
        MigrationFixture.SeparateAlarmAndRootCauseStreams());

    catalogue.Demand(
        "RootCause", "RootCauseDb", "RootCauseDbContext")
        .MigrationsAssembly.Should()
        .Be("Nexus1.RootCause.Infrastructure");
}

[Fact]
public void shared_migration_principal_is_rejected()
{
    var streams = MigrationFixture.Streams(
        alarmMigrator: "global-ddl",
        rootCauseMigrator: "global-ddl");
    var act = () => new MigrationOwnershipCatalogue(streams);
    act.Should().Throw<SharedMigrationPrincipal>();
}

[Fact]
public void RootCause_artifact_cannot_target_AlarmManagementDb()
{
    var catalogue = MigrationFixture.AcceptedCatalogue();
    var act = () => catalogue.Demand(
        "RootCause", "AlarmManagementDb", "RootCauseDbContext");
    act.Should().Throw<ForeignMigrationTarget>();
}

[Fact]
public void runtime_and_migration_principals_must_differ()
{
    var act = () => new MigrationOwnershipCatalogue(
        MigrationFixture.RuntimeWithDdlAuthority());
    act.Should().Throw<RuntimeHasMigrationAuthority>();
}
```

SEPARATION IS EVALUATED, NOT INFERRED

Different connection-string names do not prove ownership. The catalogue rejects a shared migrations assembly, cross-owner migration principal and runtime DDL identity.

APPLICATION-SCHEMA COMPATIBILITY

A transition is safe only for exact application and schema identities

EXECUTABLE ASSET 16-E - READ/WRITE COMPATIBILITY MATRIX

```
[Flags]
public enum DatabaseAccess { None = 0, Read = 1, Write = 2 }

public sealed record CompatibilityRule(
    string ApplicationArtifact,
    SchemaIdentity Schema,
    TransitionPhase Phase,
    DatabaseAccess Allowed);

public sealed class CompatibilityMatrix(
    IReadOnlyCollection<CompatibilityRule> rules)
{
    public void Demand(
        string applicationArtifact,
        SchemaIdentity schema,
        TransitionPhase phase,
        DatabaseAccess required)
    {
        var rule = rules.SingleOrDefault(x =>
            StringComparer.Ordinal.Equals(
                x.ApplicationArtifact, applicationArtifact)
            && x.Schema == schema
            && x.Phase == phase);
        if (rule is null || (rule.Allowed & required) != required)
            throw new IncompatibleApplicationSchema(
                applicationArtifact, schema, phase, required);
    }

    public void DemandActiveFleet(
        IEnumerable<string> activeApplications,
        SchemaIdentity schema,
        TransitionPhase phase,
        DatabaseAccess required)
    {
        foreach (var application in activeApplications.Distinct(
            StringComparer.Ordinal))
            Demand(application, schema, phase, required);
    }
}
```

EXECUTABLE ASSET 16-F - MIXED-VERSION PREFLIGHT TESTS

```
[Fact]
public void MIG_COMPAT_001_expand_accepts_old_and_new_writers()
{
    var matrix = CompatibilityFixture.AcceptedS0S1S2();
    matrix.DemandActiveFleet(
        ["rootcause-app-v1", "rootcause-app-v2"],
        new SchemaIdentity("S1-expanded"),
        TransitionPhase.MixedFleet,
        DatabaseAccess.Read | DatabaseAccess.Write);
}

[Fact]
public void contraction_blocks_while_old_writer_is_active()
{
    var matrix = CompatibilityFixture.AcceptedS0S1S2();
    var act = () => matrix.DemandActiveFleet(
        ["rootcause-app-v1", "rootcause-app-v2"],
        new SchemaIdentity("S2-contracted"),
        TransitionPhase.Contracting,
        DatabaseAccess.Write);
    act.Should().Throw<IncompatibleApplicationSchema>()
        .Where(x => x.ApplicationArtifact == "rootcause-app-v1");
}

[Fact]
public void restored_schema_selects_a_retained_compatible_app()
{
    var matrix = CompatibilityFixture.AcceptedS0S1S2();
    matrix.Demand(
        "rootcause-app-v1",
        new SchemaIdentity("S0-current"),
        TransitionPhase.Restored,
        DatabaseAccess.Read | DatabaseAccess.Write);
}
```

NEWEST IS NOT A COMPATIBILITY RULE

Readiness evaluates the exact active application artifacts and observed schema. A label such as latest, current or backward compatible cannot satisfy this gate.

FAIL-CLOSED PREFLIGHT

Reviewed bytes must match the observed database before execution

EXECUTABLE ASSET 16-G - TARGET AND ARTIFACT VERIFICATION

```
public sealed record DatabaseObservation(
    string Owner,
    string Database,
    string DbContext,
    MigrationIdentity CurrentMigration,
    SchemaIdentity Schema,
    Sha256Digest PhysicalFingerprint,
    IReadOnlyList<string> ActiveApplications);

public sealed class MigrationPreflight(
    MigrationOwnershipCatalogue ownership,
    CompatibilityMatrix compatibility,
    IPhysicalSchemaVerifier physicalSchema)
{
    public async Task<PreflightReport> EvaluateAsync(
        MigrationArtifactManifest manifest,
        DatabaseObservation observed,
        string artifactPath,
        CancellationToken cancellationToken)
    {
        manifest.DemandTarget(
            observed.Owner, observed.Database, observed.DbContext);
        ownership.Demand(
            observed.Owner, observed.Database, observed.DbContext);
        if (observed.CurrentMigration != manifest.FromMigration)
            throw new UnexpectedMigrationIdentity(
                manifest.FromMigration, observed.CurrentMigration);

        await ArtifactDigest.DemandAsync(
            artifactPath, manifest.Artifact, cancellationToken);
        await physicalSchema.DemandAsync(
            observed, cancellationToken);
        compatibility.DemandActiveFleet(
            observed.ActiveApplications,
            manifest.Compatibility.DuringSchema,
            TransitionPhase.MixedFleet,
            manifest.Compatibility.RequiredAccess);

        return PreflightReport.Accepted(
            observed, manifest.Artifact, manifest.Compatibility);
    }
}
```

EXECUTABLE ASSET 16-H - PREFLIGHT BLOCKS DRIFT WITHOUT APPLYING SQL

```
[Fact]
public async Task MIG_ID_001_hash_mismatch_blocks_before_authority()
{
    fixture.ReplaceArtifactBytesAfterReview();

    var act = () => fixture.PreflightAsync();
    await act.Should().ThrowAsync<MigrationArtifactHashMismatch>();
    fixture.MigrationCredentialRequests.Should().Be(0);
    fixture.SqlApplyCalls.Should().Be(0);
}

[Fact]
public async Task unexpected_history_or_physical_schema_blocks()
{
    fixture.Database.ReportsMigration("unexpected-hotfix");

    var act = () => fixture.PreflightAsync();
    await act.Should().ThrowAsync<UnexpectedMigrationIdentity>();
    fixture.SqlApplyCalls.Should().Be(0);
}

[Fact]
public async Task foreign_database_fingerprint_blocks()
{
    fixture.Database.ReplaceIdentity("AlarmManagementDb");

    var act = () => fixture.PreflightAsync();
    await act.Should().ThrowAsync<ForeignMigrationTarget>();
    fixture.MigrationCredentialRequests.Should().Be(0);
}
```

AUTHORIZATION FOLLOWS EVIDENCE

The migration credential is requested only after owner, database, context, from-state, physical fingerprint, artifact hash and compatibility have all been accepted.

SINGLE EXECUTOR AND AUTHORITY

One short-lived owner lease controls one reviewed transition

EXECUTABLE ASSET 16-I - OWNER-SCOPED EXECUTOR

```
public sealed class MigrationExecutor(
    MigrationPreflight preflight,
    IMigrationLeaseProvider leases,
    IMigrationAuthority authority,
    IReviewedArtifactApplier applier,
    IMigrationVerifier verifier,
    IMigrationJournal journal)
{
    public async Task<MigrationExecutionOutcome> ExecuteAsync(
        MigrationExecutionRequest request,
        CancellationToken cancellationToken)
    {
        var accepted = await preflight.EvaluateAsync(
            request.Manifest,
            request.ObservedDatabase,
            request.ArtifactPath,
            cancellationToken);

        await using var lease = await leases.TryAcquireAsync(
            new MigrationLeaseKey(
                request.Manifest.Owner,
                request.Manifest.Database),
            request.Manifest.Budgets.MaximumDuration,
            cancellationToken);
        if (lease is null)
            return MigrationExecutionOutcome.AlreadyExecuting();

        await using var credential = await authority.DemandAsync(
            request.Manifest.Owner,
            request.Manifest.Database,
            request.Authorization,
            cancellationToken);

        var execution = await journal.StartAsync(
            accepted, credential.Identity, lease.Identity,
            cancellationToken);
        try
        {
            await applier.ApplyExactAsync(
                request.ArtifactPath, credential, cancellationToken);
            var proof = await verifier.VerifyAsync(
                request.Manifest, cancellationToken);
            await journal.CompleteAsync(
                execution.Id, proof, cancellationToken);
            return MigrationExecutionOutcome.Completed(proof);
        }
        catch (Exception exception)
            when (exception is not OperationCanceledException)
        {
            await journal.RecordUnknownOrFailedAsync(
                execution.Id, exception, CancellationToken.None);
            throw; // Recovery classification is a separate operation.
        }
    }
}
```

EXECUTABLE ASSET 16-J - CONCURRENCY AND PERMISSION GATES

```
[Fact]
public async Task MIG_LOCK_001_two_executors_apply_once()
{
    var start = new Barrier(2);
    var outcomes = await Task.WhenAll(
        fixture.ExecuteInNewScopeAsync(start),
        fixture.ExecuteInNewScopeAsync(start));

    outcomes.Should().ContainSingle(x => x.Kind
        == MigrationExecutionOutcomeKind.Completed);
    outcomes.Should().ContainSingle(x => x.Kind
        == MigrationExecutionOutcomeKind.AlreadyExecuting);
    fixture.ArtifactApplyCalls.Should().Be(1);
}

[Fact]
public async Task MIG_SEC_001_runtime_identity_cannot_apply_DDL()
{
    var act = () => fixture.ApplyWithRuntimeIdentityAsync();
    await act.Should().ThrowAsync<DatabasePermissionDenied>();
    fixture.Database.MigrationIdentity.Should()
        .Be(MigrationFixture.FromMigration);
}

[Fact]
public void production_host_has_no_migration_service()
{
    var services = RootCauseHostComposition.BuildProductionServices();
    services.Should().NotContain(x =>
        x.ServiceType == typeof(IMigrationExecutor));
}
```

EF8 LOCKING IS NOT ASSUMED

The lease is a selected-provider or deployment-platform control with its own contention and expiry tests. Application replicas never acquire it and never retain DDL.

EXPAND, MOVE, RECONCILE, CONTRACT

Bounded data movement carries its checkpoint in the same commit

EXECUTABLE ASSET 16-K - RESUMABLE OWNER-LOCAL BACKFILL

```
public sealed class VerdictLineageBackfill(
    RootCauseDbContext db,
    BackfillOptions options,
    TimeProvider clock)
{
    public async Task<BackfillBatchResult> ExecuteBatchAsync(
        CancellationToken cancellationToken)
    {
        var checkpoint = await db.BackfillCheckpoints.SingleAsync(
            x => x.Job == "verdict-lineage-v1", cancellationToken);
        var rows = await db.Verdicts
            .Where(x => x.VerdictId.CompareTo(checkpoint.LastVerdictId) > 0)
            .Where(x => x.LineageIdentity == null)
            .OrderBy(x => x.VerdictId)
            .Take(options.BatchSize)
            .ToListAsync(cancellationToken);

        foreach (var row in rows)
            row.LineageIdentity = StableIdentity.Create(
                "lineage-v1",
                row.CaseId.ToString("D"),
                row.VerdictIdentity,
                row.SupersedesVerdictId?.ToString("D") ?? "none");

        if (rows.Count != 0)
            checkpoint.LastVerdictId = rows[^1].VerdictId;
        checkpoint.UpdatedAtUtc = clock.GetUtcNow();

        await db.SaveChangesAsync(cancellationToken);
        return new BackfillBatchResult(
            rows.Count, checkpoint.LastVerdictId);
    }
}
```

EXECUTABLE ASSET 16-L - CONTRACT GATE REQUIRES SEMANTIC RECONCILIATION

```
public sealed record ContractEvidence(
    bool BackfillComplete,
    bool SemanticReconciliationAccepted,
    bool LegacyWritersRetired,
    bool RestorePointVerified,
    bool IrreversibleChangeApproved);

public static class ContractGate
{
    public static void Demand(ContractEvidence evidence)
    {
        if (!evidence.BackfillComplete
            || !evidence.SemanticReconciliationAccepted
            || !evidence.LegacyWritersRetired
            || !evidence.RestorePointVerified
            || !evidence.IrreversibleChangeApproved)
            throw new ContractPhaseBlocked(evidence);
    }
}

[Fact]
public void MIG_EXP_001_matching_counts_without_values_cannot_contract()
{
    var evidence = MigrationFixture.ContractEvidence(
        rowCountsMatch: true,
        semanticValuesMatch: false);
    var act = () => ContractGate.Demand(evidence);
    act.Should().Throw<ContractPhaseBlocked>();
}

[Fact]
public async Task MIG_DATA_001_replayed_batch_is_idempotent()
{
    await fixture.ExecuteSameBatchTwiceAsync();
    var durable = await fixture.ReadLineageSnapshotAsync();
    durable.Rows.Should().OnlyHaveUniqueItems(x => x.VerdictId);
    durable.Should().BeEquivalentTo(fixture.SequentialOracle);
}
```

COUNTS ARE NOT RECONCILIATION

The gate compares identities and derived values under the pinned algorithm. Completion, compatibility, recovery and retirement evidence must all agree before contraction.

INTERRUPTION AND RECOVERY

Recovery begins from observed durable state, never from an exit code

EXECUTABLE ASSET 16-M - DURABLE-STATE RECOVERY CLASSIFIER

```
public sealed record MigrationObservation(
    MigrationIdentity History,
    bool TargetFingerprintMatches,
    bool PartialTargetObjectsExist,
    bool DestructiveStepMayHaveCommitted,
    bool SemanticReconciliationAccepted,
    Sha256Digest ObservedArtifact);

public enum RecoveryDecision
{
    VerifyOnly,
    RetrySameArtifact,
    ForwardRepairRequired,
    RestoreRequired,
    Blocked
}

public static class MigrationRecoveryClassifier
{
    public static RecoveryDecision Classify(
        MigrationArtifactManifest manifest,
        MigrationObservation observed)
    {
        if (observed.ObservedArtifact != manifest.Artifact)
            return RecoveryDecision.Blocked;
        if (observed.History == manifest.ToMigration
            && observed.TargetFingerprintMatches
            && observed.SemanticReconciliationAccepted)
            return RecoveryDecision.VerifyOnly;
        if (observed.DestructiveStepMayHaveCommitted)
            return RecoveryDecision.RestoreRequired;
        if (observed.PartialTargetObjectsExist)
            return RecoveryDecision.ForwardRepairRequired;
        if (observed.History == manifest.FromMigration
            && !observed.PartialTargetObjectsExist)
            return RecoveryDecision.RetrySameArtifact;
        return RecoveryDecision.Blocked;
    }
}
```

EXECUTABLE ASSET 16-N - INJECTED INTERRUPTION MATRIX

```
[Theory]
[InlineData(FaultPoint.BeforeFirstCommand,
    RecoveryDecision.RetrySameArtifact)]
[InlineData(FaultPoint.AfterAdditiveDdl,
    RecoveryDecision.ForwardRepairRequired)]
[InlineData(FaultPoint.AfterDestructiveDdl,
    RecoveryDecision.RestoreRequired)]
[InlineData(FaultPoint.AfterHistoryWrite,
    RecoveryDecision.VerifyOnly)]
public async Task MIG_FAIL_001_observation_selects_recovery(
    FaultPoint fault,
    RecoveryDecision expected)
{
    await fixture.ExecuteUntilFaultAsync(fault);
    var observation = await fixture.ObserveWithFreshConnectionAsync();
    MigrationRecoveryClassifier.Classify(
        fixture.Manifest, observation).Should().Be(expected);
}
```

EXECUTABLE ASSET 16-O - UNKNOWN STATE NEVER RETRIES BLINDLY

```
[Fact]
public async Task timeout_with_unreadable_history_blocks()
{
    await fixture.ExecuteUntilFaultAsync(
        FaultPoint.ConnectionLostDuringCommit);
    fixture.Database.MakeHistoryTemporarilyUnreadable();

    var observation = await fixture.TryObserveAsync();
    observation.IsComplete.Should().BeFalse();
    fixture.RecoveryCommands.Should().BeEmpty();
}

[Fact]
public void generated_Down_is_not_rollback_evidence()
{
    var reversal = MigrationFixture.UnverifiedGeneratedDown();
    reversal.IsAuthorized.Should().BeFalse();
    reversal.RecoveryDecision.Should()
        .Be(RecoveryDecision.Blocked);
}
```

FAILURE IS A DATABASE STATE

A retry, forward repair, reversal or restore is separately authorized only after history, physical schema, data and reviewed artifact identity are observable.

RESTORE AND ACCEPTANCE

A restored database must rejoin a proven release timeline

EXECUTABLE ASSET 16-P - RESTORE, MIGRATE AND REPLAY GATE

```
public sealed class RestoreAcceptanceGate(
    CompatibilityMatrix compatibility,
    IRetainedArtifactChain artifacts,
    IReplayReconciliation replay)
{
    public async Task DemandAsync(
        RestoreObservation restored,
        string selectedApplication,
        CancellationToken cancellationToken)
    {
        if (!restored.BackupIdentityMatches
            || !restored.PhysicalFingerprintMatches
            || !restored.RuntimeDdlDenied
            || !restored.OwnerMigrationAuthorityVerified)
            throw new RestoreNotAccepted(restored);

        compatibility.Demand(
            selectedApplication,
            restored.Schema,
            TransitionPhase.Restored,
            DatabaseAccess.Read | DatabaseAccess.Write);
        await artifacts.DemandChainAsync(
            restored.Schema,
            restored.IntendedTarget,
            cancellationToken);
        await replay.DemandStableIdentityAndEffectsAsync(
            restored.ReplayWindow,
            cancellationToken);
    }
}

[Fact]
public async Task MIG_REST_001_RootCause_restore_is_independent()
{
    var alarmBefore = await fixture.AlarmManagementFingerprintAsync();
    await fixture.RestoreMigrateReplayRootCauseAsync();
    var alarmAfter = await fixture.AlarmManagementFingerprintAsync();

    alarmAfter.Should().Be(alarmBefore);
    fixture.RootCauseReplay.DuplicateEffects.Should().Be(0);
    fixture.RootCauseReplay.PendingFactIds.Should()
        .OnlyHaveUniqueItems();
}
```

EXECUTABLE ASSET 16-Q - ACCEPTANCE EVIDENCE RECORD

```
{
  "evidenceId": "MIG-REVIEW-001", "chapter": 16,
  "artifactCommit": null, "sdk": "net8.0",
  "efRuntime": null, "efTool": null, "provider": null,
  "engineVersion": null, "artifactSha256": null,
  "fromMigration": null, "toMigration": null,
  "tests": {
    "MIG-OWN-001": "PENDING", "MIG-ID-001": "PENDING",
    "MIG-COMPAT-001": "PENDING", "MIG-EXP-001": "PENDING",
    "MIG-DATA-001": "PENDING", "MIG-LOCK-001": "PENDING",
    "MIG-FAIL-001": "PENDING", "MIG-ROLL-001": "PENDING",
    "MIG-REST-001": "PENDING", "MIG-SEC-001": "PENDING",
    "MIG-OBS-001": "PENDING", "MIG-TEST-001": "PENDING",
    "ALMDB-MIG-001": "PENDING", "RCDB-MIG-001": "PENDING"
  },
  "singleExecutorProved": false,
  "restoreRehearsed": false, "accepted": false
}
```

Chapter 16 code-audit result

The chapter now exposes one executable path from reviewed owner-local bytes to compatibility-aware execution, bounded data movement, durable recovery and an identity-preserving restore without coupling the other service database.

CURRENT VERDICT

The lifecycle is implementable in shape, but no migration artifact has been generated or executed. Toolchain, provider, authority, failure, rollback and restore evidence remain explicitly PENDING.

Chapter checkpoint

You are ready to execute independent migrations when each service can generate and publish one provider-pinned artifact, prove every supported transition and compatibility window, apply it with owner-only authority and a single executor, repair failures from observed durable state, and restore/replay without coordinating an unrelated database release.

Question	Expected answer
What makes a migration stream independent?	One service owns its model snapshot, migration source/history, artifact, principal, compatibility window, recovery decision and evidence.
What identifies the deployment unit?	Owner/database/context, from/to, source/snapshot/tool/provider and artifact hashes, change class, budgets, compatibility and recovery evidence.
May production startup migrate?	No. net8/EF8 Hosts have no DDL and do not call EnsureCreated, Migrate or MigrateAsync.
How is concurrency controlled on EF8?	One provider/platform-proved owner executor; EF9 automatic migration locking is not assumed.
What proves compatibility?	An executable read/write matrix for exact application and schema identities before, during, after and after restore.
When is contraction allowed?	After expand, compatible deployment, resumable backfill, value-level reconciliation, old-artifact retirement and irreversible-change approval.
Is an idempotent script sufficient?	No. History-based selection does not prove physical schema, data meaning, provider behavior, permissions or application compatibility.
What follows an interrupted migration?	Freeze as needed, preserve evidence, classify history/schema/data, then use an approved retry, forward repair, reversal or restore path.
What is the safest common rollback?	Application rollback inside a declared expanded-schema window; schema reversal is separate and may be rejected.
What must a restore prove?	Backup and schema identity, compatible application, retained migration chain, correct authority, RPO/RTO and idempotent replay/reconciliation.
Can public contracts coordinate releases?	Yes. They may order compatible producer/consumer deployment, but never share database history, credential, transaction or schema authority.
What is the current status?	AlarmManagementDb and RootCauseDb are specified, not built/migrated; RootCause remains G0 accepted and G1 specified without runtime evidence.
What does Chapter 17 add?	Cross-service identity: public identity ownership, translation, correlation, causation and authorization without leaking private database keys.

NEXT CHAPTER

Chapter 17, Cross-Service Identity, defines how AlarmManagement, RootCause and later consumers preserve public fact identity, correlation, causation and authorized references while keeping database keys private. It will separate identity from location, message delivery, tracing and trust, then specify translation records, replay behavior and evidence for identities that outlive any one schema version.

PART III - DATA OWNERSHIP IN EXECUTABLE FORM

17. Cross-Service Identity Without Cross-Database Foreign Keys

A public identifier is an owner-issued contract that another service may retain, validate and translate. It is not permission to reuse the owner's database key, join its tables, infer its location or authorize a caller. NEXUS-1 preserves identity across facts while every service preserves its own data boundary.

CHAPTER ARGUMENT

Separate six concerns that convenient GUIDs often collapse: private aggregate identity, public subject reference, immutable public-fact identity, correlation/causation, trace context, and caller authority. Issue public meaning at the owner boundary, deduplicate and translate it locally, preserve it through replay and restore, and reject every attempt to turn a reference into a foreign key or security capability.

Chapters 14 and 15 already established the durable tuples. AlarmManagement uses source identity for intake, AlarmId for its private aggregate, DecisionIdentity for one exact flood decision and PublicFactId for a produced fact. RootCause deduplicates Producer + PublicFactId + ContractVersion, owns Caseld, EvidenceIdentity and VerdictIdentity, and publishes a new PublicFactId. Correlation and causation explain the flow but provide neither uniqueness nor authorization by themselves.

Chapter 3 used Guid EventId and Guid Caseld in early architectural reference records. This chapter tightens that shorthand. EventId becomes a typed PublicFactIdentity. A case, verdict or flood value on the wire becomes an owner-scoped public subject reference. Even when the owner temporarily chooses the same bytes as a private key, consumers compile against the public type and contract semantics, never the database representation.

No NEXUS-1 contract assembly, public identity factory, selected serialization, broker envelope, inbox, translation relation, authorization adapter, trace propagation or database constraint has compiled or run in this manuscript environment. The net8.0 code below is a reference contract. It does not claim that an identity has been issued, accepted, mapped, replayed or authorized. RootCause remains G0 accepted and G1 specified without runtime evidence.

Identity is a contract, not a convenient key

Identity class	Owner and durable purpose	Scope / uniqueness	Never substitutes for
Private aggregate ID	Owning service locates and versions its aggregate.	Private database/model scope.	Public contract, foreign key in another database or caller authority.
Source identity	Owner recognizes the same accepted upstream observation.	Source authority + source event/object identity.	Local aggregate ID, public fact ID or ordering.
Decision identity	Domain proves one exact policy/input set produced one decision.	Owner + decision policy/canonical inputs.	Row version, transport message ID or trace ID.
Public subject reference	Owner exposes a stable reference to public meaning such as a flood decision or verdict.	Authority + namespace/kind + value + optional tenant scope.	Owner database key, URL/location or authorization capability.
Public fact identity	Producer identifies one immutable published fact and retains it across dispatch/replay.	Producer + PublicFactId + contract type/version.	Consumer inbox ID, correlation ID or broker delivery tag.
Request/idempotency key	Caller and receiving API bind repeated command attempts to one accepted operation.	Caller/tenant + operation/route + accepted window.	Event/fact identity or authorization.
Correlation	Flow initiator groups related work for navigation and operations.	Workflow context; duplicates are expected.	Deduplication, causation, aggregate selection or permission.
Causation	Producer references the exact prior accepted request/fact that triggered a new fact.	One predecessor reference per emitted fact.	A complete workflow graph or proof of causation.

Identity class	Owner and durable purpose	Scope / uniqueness	Never substitutes for
Trace/span identity	Telemetry system links execution observations across process boundaries.	Trace and request/span lifetime; sampling may vary.	Business identity, durable replay key, correlation contract or authorization.
Transport delivery identity	Broker/protocol identifies an attempt, position, receipt or delivery.	Connection/channel/partition/subscription semantics.	Business effect, public fact identity or producer ownership.
Principal/policy identity	Trust boundary identifies caller/service and the policy decision used locally.	Issuer/audience/tenant/policy/action/resource context.	Resource identity, fact identity, correlation or possession of an opaque ID.

TYPE SEPARATION RULE

Two identities may share the same wire encoding and still have different meaning. Use distinct types, validation and storage constraints so EventId, CaselId, CorrelationId, TracelId and actor subject cannot be passed interchangeably.

One owner issues each public meaning

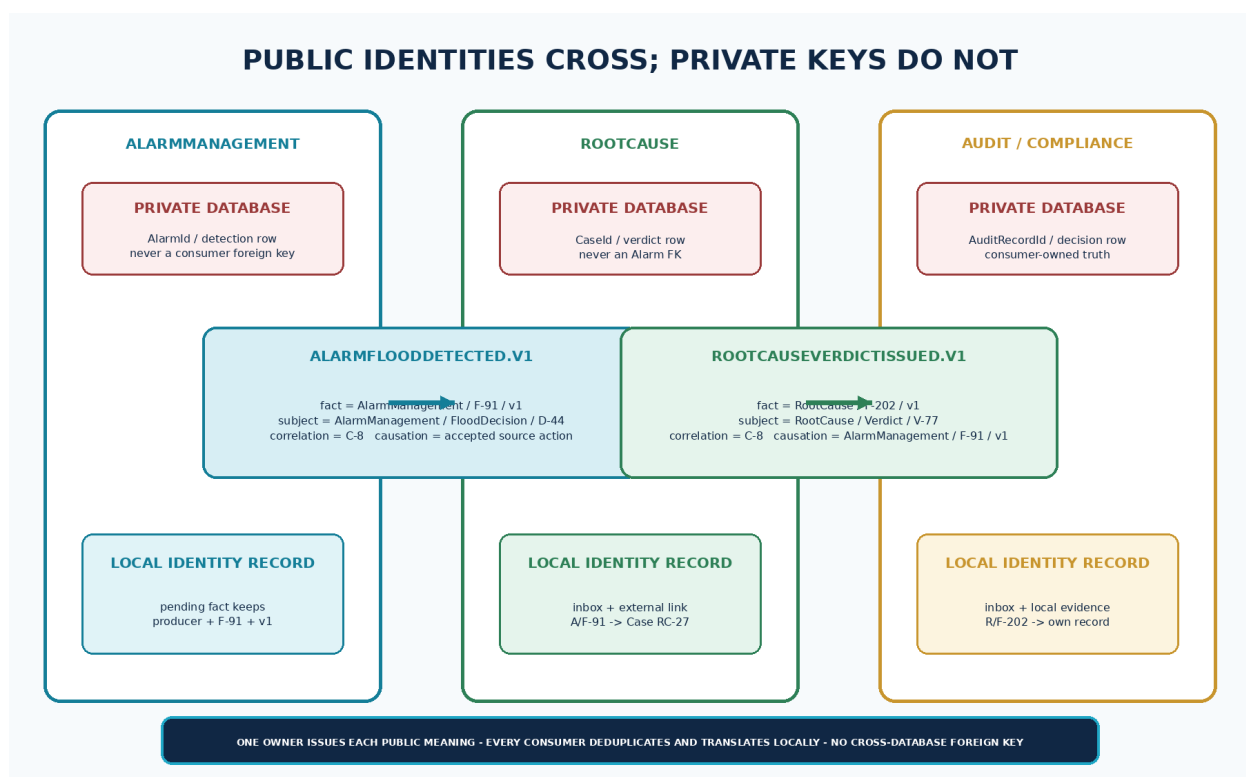


Figure 17.1 - Facts carry owner-scoped public identities and causality. Consumers keep inbox and translation records beside their own truth; private database keys never cross the boundary.

AlarmManagement may expose a FloodDecision reference because it owns that decision. RootCause stores the reference as evidence lineage and maps it to a local case under an accepted policy. RootCause then exposes a Verdict reference because it owns the verdict. Audit and Compliance retain that reference beside their own evidence or policy decision. None of those relations requires a foreign key into the producer database, a distributed join or a shared identity table.

Boundary	Public identity received	Local durable record	Forbidden assumption
Source -> AlarmManagement	SourceSystem + SourceEventId under accepted source contract.	AlarmObservation and private AlarmId/DecisionIdentity.	Source ID is an AlarmId or grants permission to mutate an alarm.
AlarmManagement -> RootCause	AlarmManagement PublicFactIdentity plus optional FloodDecision public reference.	Inbound fact, external reference link, evidence and private CaselId.	FloodDecision value is an AlarmManagementDb FK or chooses the RootCause case alone.
RootCause -> Audit	RootCause verdict fact and public Verdict reference.	Audit inbox and append-only AuditRecordId.	Verdict reference is the RootCause verdict table key or authorizes evidence access

Boundary	Public identity received	Local durable record	Forbidden assumption
RootCause -> Compliance	Same producer fact identity delivered independently.	Compliance inbox, policy/configuration identity and private compliance decision.	Audit consumption, delivery order or shared correlation decides compliance truth.
Any service -> Reporting	Versioned public facts/references under projection contract.	Reporting-owned projection identities and checkpoints.	Projection key becomes source truth or permits a source write.

Scope makes an opaque value meaningful

The string or GUID alone is never the complete identity. A consumer normalizes an authority, namespace or kind, value, contract identity and tenant/audience scope where the contract is multi-tenant. The exact tuple is versioned and length-bounded. Case, whitespace, Unicode normalization and textual UUID rules are contract decisions, not database collation accidents.

Component	Meaning	Validation / storage rule
Authority	Service or issuer that owns the public namespace and lifecycle.	Allow-list accepted producer identity; do not infer from hostname alone.
Namespace / kind	Public semantic class such as FloodDecision, Verdict or RootCauseCase.	Closed vocabulary per contract version; no private table/entity name.
Value	Opaque owner-issued token.	Canonical text/binary form, bounded length, no trimming/case folding unless specified.
Tenant/audience scope	Boundary within which the value is meaningful and authorized.	Opaque public tenant reference or trusted context; never a raw private tenant key.
Contract type/version	Defines the fields and identity interpretation carried by one fact.	Required in dedup tuple; version mismatch is not silently coerced.
Lifecycle state	Active, superseded, tombstoned or expired resolution policy.	Append transition/retained tombstone; value is never reassigned.

The public envelope carries identity and causality explicitly

Chapter 20 will select the serialized message envelope and compatibility rules. Chapter 17 defines the semantic types that any JSON, binary or broker representation must preserve. Transport metadata and trace context remain adjacent execution context; they do not enter Domain payloads or identity equality.

PUBLIC IDENTITY VALUE TYPES - [NEW][COMPILE REQUIRED]

```
public readonly record struct IdentityAuthority(string Value);

public readonly record struct PublicFactIdentity(
    IdentityAuthority Producer,
    Guid FactId,
    string ContractType,
    int ContractVersion);

public readonly record struct PublicSubjectReference(
    IdentityAuthority Authority,
    string Namespace,
    string Value,
    string? TenantScope);
```

FACT ENVELOPE AND ADJACENT DELIVERY CONTEXT - [NEW][COMPILE + SERIALIZATION REQUIRED]

```
public sealed record CausalityContext(
    string CorrelationId,
    PublicFactIdentity? Causation);

public sealed record PublicFactEnvelope<TPayload>(
    PublicFactIdentity Fact,
    PublicSubjectReference? Subject,
    DateTimeOffset OccurredAtUtc,
    CausalityContext Causality,
    TPayload Payload);

// Adjacent transport/telemetry context; never part of public-fact equality.
public sealed record DeliveryContext(
    string? TransportMessageId,
    int Attempt,
    string? TraceParent);
```

The reference deliberately does not include a database key, queue/exchange, URL, host, partition, retry count, recorded time or authentication token in `PublicFactIdentity`. `OccurredAtUtc` has producer-defined contract semantics; the consumer records its own `RecordedAtUtc` separately. `Subject` is optional because some facts identify an occurrence without exposing a durable resource reference.

Tighten the early reference contracts without breaking meaning

Earlier shorthand	Chapter 17 type	Compatibility decision
AlarmFloodDetectedV1.EventId	<code>PublicFactIdentity</code> for the <code>AlarmManagement</code> producer.	Wire field may remain <code>eventId</code> in v1, but code/fixtures bind producer + type/version and reject bare-Guid equality.
RootCauseVerdictIssuedV1.EventId	New <code>RootCause</code> <code>PublicFactIdentity</code> .	Never reuse the causal <code>AlarmManagement</code> fact ID; causation references it explicitly.
RootCauseVerdictIssuedV1.CaseId	<code>RootCauseCase</code> or <code>Verdict</code> <code>PublicSubjectReference</code> chosen by owner contract.	Consumers cannot assume EF key, row location or that the reference authorizes case access.
AlarmGroupId	<code>AlarmManagement</code> public subject/group reference if consumers truly require it.	Do not expose private aggregate/group IDs by convenience; accept owner, purpose and retention first.
CorrelationId / CausationId strings	Typed flow context and full causal <code>PublicFactIdentity</code> reference.	Preserve v1 wire compatibility through mapping; new versions remove ambiguous unscoped causation.

COMPATIBILITY BOUNDARY

A wrapper type can tighten code before a wire-breaking change. If v1 carries a bare GUID, the adapter combines it with the authenticated/accepted producer and contract identity. A v2 envelope may make those fields explicit after Chapter 20 compatibility evidence.

Choose an identifier format without changing the runtime baseline

The public contract requires stable opacity, accepted collision risk and canonical representation; it does not mandate one UUID version. A net8.0 reference may use a service-owned factory backed by `Guid.NewGuid` for random identifiers, then prove canonical lowercase/text or binary representation and uniqueness constraints. Database locality is handled by schema/index design rather than by treating occurrence time as identity.

NET8 PUBLIC-FACT FACTORY CANDIDATE - [NEW][COMPILE + GENERATION TEST REQUIRED]

```
public interface IPublicFactIdFactory
{
    Guid NewFactId();
}

// net8.0 reference candidate. Exact randomness, formatting and collision
// response remain provider/component evidence, not an architectural claim.
internal sealed class Net8PublicFactIdFactory : IPublicFactIdFactory
{
    public Guid NewFactId() => Guid.NewGuid();
}
```

RFC 9562 defines UUIDv7 as time-ordered and recommends it over UUIDv1/v6 when possible. Microsoft's Guid.CreateVersion7 API is documented on the .NET 9 surface. This volume remains pinned to net8.0 and does not silently use that API. Selecting UUIDv7 requires an explicit framework/package/implementation decision, test vectors, clock/monotonicity behavior, privacy review and migration evidence. A sortable ID still does not become domain occurrence order or authorization.

Create the public fact identity before the local commit

An owner allocates `PublicFactIdentity` when the domain result admits a new public fact, before the transaction commits. The pending-public-fact row stores that identity and immutable canonical payload beside the owned state. Dispatch, publisher retry, crash recovery and disaster replay reuse the same identity. A correction or genuinely new fact receives a new identity and references the prior fact through causation or supersession.

IDENTITY IS PART OF THE LOCAL COMMIT - [NEW][COMPILE + PROVIDER TEST REQUIRED]

```
var factIdentity = new PublicFactIdentity(
    producer: AlarmManagementIdentity.Authority,
    factId: publicFactIds.NewFactId(),
    contractType: "AlarmFloodDetected",
    contractVersion: 1);

var envelope = AlarmFloodDetectedMapper.Create(
    factIdentity,
    PublicSubjects.FloodDecision(decision.Identity),
    causality,
    decision);

store.Track(decision);
store.Add(PendingPublicFact.From(envelope));
await store.CommitAsync(cancellationTokens); // owned state + same fact identity

// Dispatcher publishes the stored bytes/identity; it never calls NewFactId().
```

Path	Required identity behavior	Failure if violated
First accepted decision	Allocate one producer fact identity and persist it with state/public intent.	Committed decision has no recoverable public identity or two facts can represent one commit.
Dispatcher retry	Read and republish the exact stored identity/payload.	New ID defeats consumer deduplication and produces duplicate effects.
Crash after broker acceptance	Pending state may retry the same identity until confirmation/reconciliation.	Delivery ambiguity becomes a second public fact.
Domain correction	Create a new fact identity with explicit causation/supersession and immutable prior fact.	Reusing ID with changed payload corrupts history.
Restore/replay	Restore pending identities and replay exactly; never regenerate from current clock/sequence.	Recovered system creates a new public history for old meaning.

Duplicate, collision and new fact are different outcomes

The consumer stores a canonical content fingerprint beside the producer fact tuple. The fingerprint is a diagnostic equality check, not the identity and not a signature. The exact canonicalization version is pinned. Same identity with equivalent content is Duplicate. Same identity with different content is

CollisionOrTampering and blocks/quarantines. New identity with identical content remains a new fact unless an owner-approved business identity proves it is the same effect.

Observed input	Stable outcome	Required action
Same producer + fact ID + contract version; same canonical fingerprint	Duplicate / previously handled.	Return stored outcome; add no second evidence, case change or public fact.
Same identity tuple; different fingerprint or subject scope	IdentityConflict.	Quarantine/block, retain both diagnostics securely and alert; never overwrite receipt.
Different fact ID; same correlation	New fact candidate.	Validate contract/domain business identity; correlation alone never deduplicates.
Different fact ID; byte-identical payload	New fact candidate or producer defect.	Apply domain/source identity policy; do not hash-deduplicate automatically.
Same broker message/delivery ID; different public fact	Transport anomaly.	Use public envelope identity, record transport problem and follow broker policy.
Same public fact after restart/restore	Duplicate / replay.	Reload durable receipt and preserve one business effect.

Consumers translate public references into local meaning

RootCause may need to know that AlarmManagement FloodDecision D-44 is already associated with Case RC-27. It stores a private external-identity link inside RootCauseDb. The link retains the external authority/namespace/value/scope, first-seen public fact, mapping-policy identity, local typed identity, state and version. It is not a cache of the producer row and does not make the producer reference a RootCause primary key.

PRIVATE TRANSLATION RECORD - [NEW][EF MAPPING + MIGRATION REQUIRED]

```
internal sealed class ExternalIdentityLinkRow
{
    public required string Authority { get; init; }
    public required string Namespace { get; init; }
    public required string ExternalValue { get; init; }
    public string? TenantScope { get; init; }

    public required string LocalKind { get; init; }
    public required Guid LocalId { get; init; }
    public required string MappingPolicyIdentity { get; init; }
    public required Guid FirstSeenFactId { get; init; }
    public required string State { get; init; }
    public Guid ConcurrencyVersion { get; set; }
}

// UNIQUE(authority, namespace, external_value, tenant_scope)
// LocalId may reference only a RootCauseDb-local row.
// There is no AlarmManagementDb foreign key or producer lookup.
```

Translation field	Why it is retained	Forbidden behavior
External tuple	Unambiguous owner-scoped reference and unique lookup.	Bare value lookup across authorities, kinds or tenants.
Local kind + ID	Purpose-specific relation to one owned aggregate/evidence record.	Expose the local ID back as the producer identity by accident.
First-seen fact	Explains which accepted fact established the mapping.	Use arrival time or delivery tag as mapping authority.
Mapping policy identity	Makes correlation/translation replayable after rules evolve.	Re-evaluate historical links under latest policy silently.
State / supersession	Preserves merges, tombstones, corrections and operator review.	Destructive remap or reassign an external value.
Concurrency version	Prevents stale administrative/mapping transition.	Merge conflicting local mappings from tracked values.

Concurrent first sight is a semantic conflict

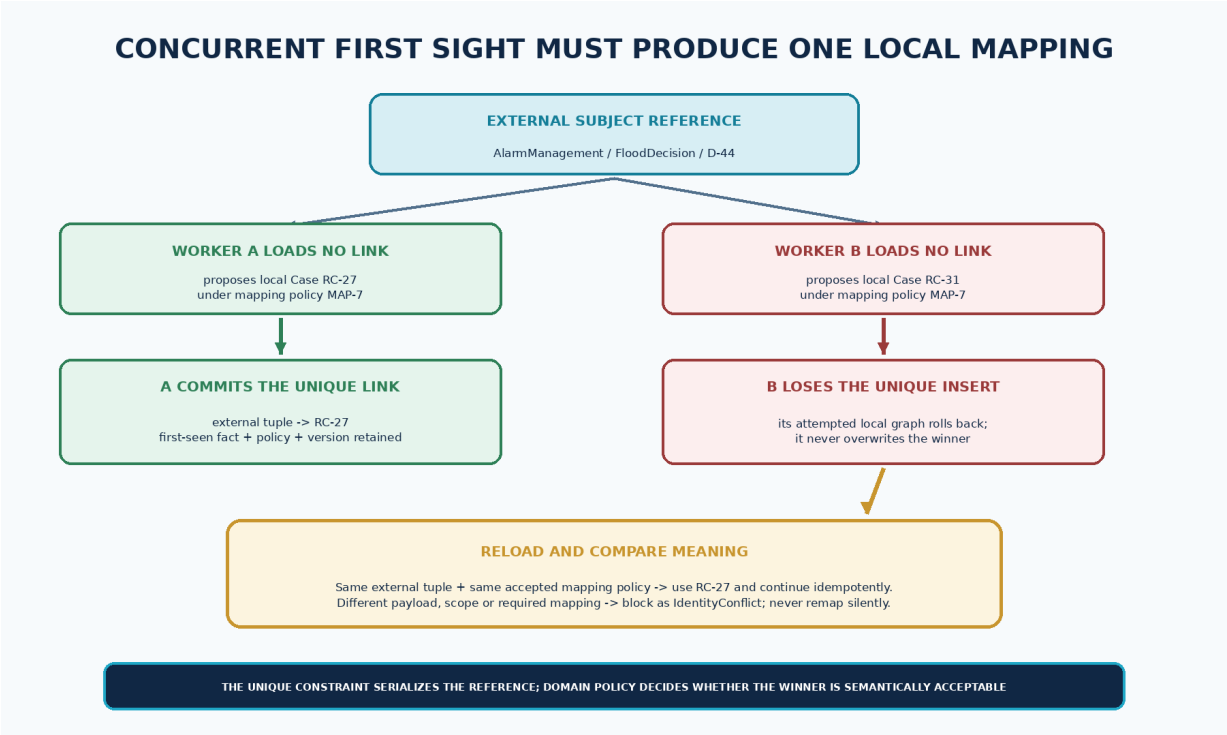


Figure 17.2 - One unique external tuple selects the committed local link. A loser reloads and validates the winner; it never silently remaps a public identity.

Two deliveries can race before any link exists. Both may construct candidate local work, but a unique external tuple admits only one mapping. The loser rolls back its attempted graph, disposes the failed unit of work and reloads the winner. It continues only when the accepted policy proves that the winner represents the same intended local meaning. Different local targets, tenant scope, subject kind or payload lineage produce a stable IdentityConflict for review.

Concurrency case	Database result	Application result
Same fact delivered twice	Unique inbox tuple admits one receipt/effect.	Loser returns prior Duplicate outcome.
Different facts reference same external subject and same mapping	Unique external link admits one row; both facts may remain distinct receipts.	Reload shared local mapping and process each fact under domain rules.
Same external subject proposes different local aggregate	Unique external link rejects loser.	Block IdentityConflict; no last-writer-wins remap.
Same bare value under different authority/tenant	Distinct scoped tuples when contract permits.	Never collapse by value or database collation.
Administrative merge/split races with intake	Versioned link transition rejects stale update.	Reload, authorize explicit merge/split operation and preserve history.

Correlation and causation form a graph, not an identity registry

Correlation connects the broader alarm-to-compliance flow. Causation names the direct accepted predecessor. A new fact normally preserves the incoming correlation and sets causation to the request or public fact that triggered it. A scheduled repair, human command or timeout may start or join correlation under an explicit policy. Fan-out creates several facts with the same causation. Fan-in records the exact contributing fact identities in owned evidence rather than inventing one ambiguous causation string.

Flow situation	Correlation	Causation / evidence	Forbidden shortcut
AlarmManagement emits flood	Preserve accepted source correlation or create owner flow ID.	Reference accepted source request/event where contract permits.	Use AlarmId, trace ID or current time as correlation implicitly.
RootCause handles flood	Preserve AlarmManagement flow correlation.	Causation is full AlarmManagement PublicFactIdentity.	Causation = broker delivery tag or bare GUID without producer/version.

Flow situation	Correlation	Causation / evidence	Forbidden shortcut
RootCause emits verdict	Preserve same flow correlation.	Causation is the accepted flood fact; exact evidence membership remains owned.	Use Caseld as causation or claim correlation proves evidence.
Verdict fans out	Audit and Compliance see the same correlation.	Each delivery references the same RootCause verdict fact.	Consumer order or shared correlation couples their truth.
Human correction	Join prior correlation only under accepted operator workflow.	New command ID and causal/superseded fact plus actor/policy evidence.	Mutate prior fact ID or hide privileged action behind trace context.
Reconciliation repair	Explicit repair correlation links the run.	Retain original fact identities plus repair job/policy identity.	Create new business facts merely to make telemetry contiguous.

Trace context is telemetry context

W3C Trace Context standardizes traceparent and tracestate so tracing systems can connect execution observations. A trace-id identifies a distributed trace; a parent-id identifies the caller's request/span position and trace flags include sampling advice. Those values may be absent, rejected, regenerated, sampled or changed by intermediaries. NEXUS-1 therefore propagates them through the Edge/Host telemetry path and never uses them as PublicFactIdentity, case correlation, idempotency or authorization.

Context	May change when	Durability	NEXUS-1 use
Public fact identity	Never for the same immutable fact.	Pending fact, inbox, replay and restore.	Deduplication and exact fact lineage.
Business correlation	Only through explicit flow/correction/repair policy.	Retained where audit/navigation policy requires.	Alarm-to-compliance flow navigation.
Causation	Each new fact/request chooses its exact predecessor.	Retained with public fact/evidence.	Direct causal graph and explanation.
W3C trace-id	New root, invalid context, trust boundary or tracing decision.	Telemetry retention only; sampling may remove spans.	Distributed execution diagnosis.
parent/span ID	Every hop/span.	Telemetry system only.	Request position and latency/error analysis.
tracestate / flags	Vendor/intermediary/sampling behavior.	Telemetry policy; privacy/security limits.	Tracing interoperability, never business policy.

INFERENCE FROM THE STANDARD

W3C defines trace context for distributed tracing, not business identity or access control. NEXUS-1's prohibition on using trace IDs for deduplication, resource identity or authorization is an architectural inference from that scope and from trace mutation/sampling behavior.

Transport identity describes an attempt

A broker message ID may deliberately copy the public fact ID to aid diagnosis, but that encoding does not merge the semantics. Delivery tag, queue position, partition offset, consumer receipt, redelivery flag and attempt count describe transport. They can change while the public fact remains the same. Consumer adapters validate the public envelope, then use the producer fact tuple for durable deduplication.

Transport field	Valid use	Never used as
Broker message ID	Operational lookup; optionally mirrors fact ID by adapter policy.	Proof of producer/contract or sole inbox uniqueness.
Delivery tag / receipt	Acknowledge/reject one channel/subscription delivery.	Public fact identity or cross-restart reference.
Partition/offset/sequence	Checkpoint transport position and diagnose lag/order.	Domain occurrence order, case identity or authorization.

Transport field	Valid use	Never used as
Redelivery / attempt	Retry/dead-letter policy and metrics.	Payload field, decision input or deterministic evidence.
Queue/topic/exchange	Routing/deployment configuration.	Producer authority, subject namespace or durable location.
Consumer inbox ID	Local row identity for accepted receipt lifecycle.	Producer fact ID exposed to other services.

An identifier never grants authority

Possession of an opaque case, verdict or UUID is not authentication and not authorization. At every API/message boundary the owner authenticates the caller/producer, validates issuer/audience/tenant and locally authorizes an action against the resolved resource and accepted policy. A service identity does not inherit the human's permissions automatically. Forwarded actor context is minimized, signed/trusted under an explicit delegation design and recorded separately from the public subject.

Threat / mistake	Why identity is insufficient	Blocking control
IDOR / reference swapping	A valid-looking reference may name another tenant/resource.	Local subject/tenant/action authorization after resolution; negative adjacent-ID tests.
UUID as bearer capability	RFC 9562 warns UUIDs may be guessable and must not grant access by possession.	Authentication and policy decision independent of identifier entropy.
Correlation spoofing	Caller-supplied correlation is navigation metadata.	Validate/bound it, create trusted local context and never derive permission.
Producer spoofing	Payload producer string alone is untrusted.	Transport/service authentication plus accepted producer/contract mapping.
Confused deputy	Service identity may perform an action the human could not.	Explicit delegation/on-behalf-of policy, minimized claims and owner authorization.
Tenant collision	Same bare external value may exist in another scope.	Tenant/authority included in lookup and policy; no global bare-value cache.
Enumeration / PII leakage	Sequential/meaningful IDs expose volume or sensitive facts.	Opaque public references, response minimization, rate/abuse controls and privacy review.
Trace baggage trust	Telemetry headers may be caller-controlled or vendor-specific.	Allow-list/sanitize telemetry; never map baggage directly to claims/domain facts.

Public identifiers are opaque and privacy-minimized

Do not encode email, username, national identifier, facility name, database shard, host, timestamp precision or business classification into a public ID merely for readability. Opaque does not mean secret. Consumers may log or expose only the minimum form approved for the audience; full identity tuples can be hashed/redacted in general telemetry while protected audit/evidence retains resolvable lineage under policy.

Design choice	Reference rule	Evidence
Opaque value	No private key semantics, PII or location is promised by the token.	Contract/privacy review and log/API snapshot.
Canonical display	One parse/format form; invalid, zero, overlong and non-canonical input fails at Edge.	Test vectors across serializer/language/provider.
Tenant scope	Use an accepted opaque tenant/audience reference and local policy mapping.	Cross-tenant negative and duplicate-value tests.
Logging	Log type/authority and protected or truncated/hash form according to classification.	Synthetic canary scan and access/retention evidence.
Resolution API	Authentication, authorization, rate limits and non-enumerating errors are explicit.	IDOR/enumeration/security tests.
Retention	Public link/tombstone retained only as long as legal/idempotency/audit/replay policy requires.	Deletion/retention/restore exercise.

Identity has a lifecycle but is never reassigned

Public identity lifecycle is append-only meaning. An issued fact identity always names the same immutable fact. A subject reference may remain active, become superseded, merge, split or become tombstoned, but the old value is never recycled for different meaning. The owner publishes explicit correction/supersession facts where consumers require them. Translation records retain enough history to explain which local object represented the reference at an accepted time and policy.

Lifecycle event	Owner behavior	Consumer behavior
Issue	Allocate opaque value under authority/namespace and persist lifecycle/fact identity.	Validate scope, deduplicate fact and create one local translation if needed.
Correction	New public fact ID; same or new subject reference per contract; link prior fact/reason.	Preserve prior receipt/effect and apply authorized correction semantics.
Supersession	Publish old -> new relation and effective policy/time; never rewrite old fact.	Version/append translation state and keep old lineage resolvable.
Merge	Explicitly identify winner/new subject and all retired aliases.	Do not silently collapse local aggregates; execute owner-approved merge workflow.
Split	Create distinct new subject references and mapping facts.	Map each new meaning explicitly; do not infer by payload coincidence.
Tombstone / privacy deletion	Stop resolution/exposure as policy requires while retaining minimum non-reuse/evidence marker where lawful.	Keep minimal idempotency/lineage tombstone; remove content under retention policy.
Retire contract	Retain identity resolution/translation window and consumer inventory.	Migrate contracts without changing historical identity meaning.

Restore and replay must preserve the identity graph

A restored producer keeps pending fact identities. A restored consumer keeps inbox receipts, external links, mapping-policy identities and prior outcomes. If a backup predates a receipt, broker/source replay may redeliver it; the same public fact tuple produces one effect. If the retained window cannot cover the restore point, the runbook must identify which upstream history and reconciliation oracle can re-establish mappings without minting new identities.

Recovery surface	Must survive	Blocking proof
Producer pending facts	PublicFactIdentity, immutable payload/fingerprint, causality and dispatch state.	Restore then republish yields the same fact identity/bytes.
Consumer inbox	Producer tuple, fingerprint, stable outcome and processing policy/version.	Redelivery after restore returns Duplicate or resumes safely.
External links	Scoped external tuple, local ID, first-seen fact, mapping policy and lifecycle history.	No second local mapping or cross-tenant collapse after restore.
Correlation/causation	Required flow/causal graph independent of trace retention.	Audit/explanation reconstructs predecessor and fan-out after telemetry expires.
Authorization evidence	Actor/service/policy/action/resource references required by audit policy.	Restored record explains authority without replaying old access tokens.
Contract/migration identity	Serializer/schema version and database migration compatible with retained tuples.	Chapter 16 restore gate selects an app that can parse and enforce identities.

Test identities as adversarial data

Happy-path uniqueness is insufficient. The suite exercises malformed, zero, overlong, mixed-case, alternate textual, cross-authority, cross-tenant, reused, colliding and concurrently mapped values. It repeats the same scenarios across process restart, provider collation, serialization round trip, contract version, restore and replay. Fixtures state which identities are business inputs and which are transport/telemetry noise.

Scenario	Required result	Evidence retained
Canonical round trip	Accepted identity serializes/parses to one canonical equivalent form across net8 components.	Vectors, contract fixture and equality/hash results.
Malformed / zero / overlong	Edge rejects with stable safe contract before Domain/persistence.	Negative fixture and no durable row/effect.
Cross-authority same value	Distinct only where both authorities are accepted; no bare-value collision.	Scoped unique rows and producer validation.
Cross-tenant same value	Separate mappings and denied cross-tenant resolution/action.	Permission and database query evidence.
Same fact / same content	One receipt/business effect across sequential, parallel, restart and restore replay.	Inbox/outcome/final-state comparison.
Same fact / changed content	IdentityConflict quarantine; no overwrite or business success.	Both fingerprints/diagnostics protected and alert fires.
Different fact / same payload	Domain/source identity policy decides; correlation/hash alone does not deduplicate.	Fact receipts and explicit outcome.
Concurrent first mapping	One external link wins; loser reloads/compares or blocks.	Two-writer schedule, constraint identity and final graph.
Trace mutation/loss	Business identity/correlation/causation and outcome remain correct.	Trace/no-trace/sampled runs with equal durable state.
Delivery ID/attempt changes	Same public fact remains one effect.	Broker/component redelivery matrix.
Unauthorized valid ID	Resolution/action is denied without leaking foreign details.	IDOR/enumeration negative tests and audit decision.
Correction/supersession	New fact identity, immutable prior fact and explicit lifecycle link.	History/translation/consumer result.
Restore/replay	No new public/local identity for previously accepted meaning.	Backup identity, replay inputs and reconciliation report.

Cross-service identity evidence gates

Evidence ID	Blocking rule	Required artifact
ID-OWN-001	Every public subject/fact namespace has one semantic owner, issuer and lifecycle authority.	Identity catalogue, producer/consumer inventory and owner review.
ID-SCOPE-001	Authority, kind/namespace, value, contract and tenant scope are normalized explicitly.	Canonicalization vectors and cross-scope provider tests.
ID-FACT-001	One immutable producer fact keeps the same identity through local commit, dispatch, retry and restore.	Pending-fact/inbox/replay execution report.
ID-REF-001	Public subject references are opaque contract values and never cross-database FKs or location promises.	API/schema/model/permission scan and negative DB-access test.
ID-MAP-001	Each consumer translates externally scoped references into owned local meaning under a pinned policy.	Translation schema, constraint and mapping-policy fixtures.
ID-CON-001	Concurrent first sight cannot create conflicting local mappings or last-writer remaps.	Two-writer schedule and reload/equivalence/block results.
ID-COLL-001	Same fact identity with different canonical content blocks as collision/tampering.	Canonical fingerprint version and mismatch/quarantine test.
ID-IDEM-001	Duplicate/restart/restore delivery produces one consumer business effect and prior stable outcome.	Sequential/parallel/restart/restore matrix.
ID-CAUS-001	Correlation and full causal fact references survive every selected flow boundary without becoming keys.	Alarm-to-compliance causality graph and contract assertions.
ID-TRACE-001	Trace context loss/mutation/sampling cannot change business identity, decision or authorization.	Trace propagation and no-trace equivalence tests.
ID-AUTH-001	Possession of a public ID never grants access; owners authorize principal/action/resource locally.	IDOR, tenant, producer-spoof and delegation tests.
ID-PRIV-001	Identifiers/logs/contracts expose no unaccepted PII, topology or enumeration capability.	Privacy classification, canary scan and abuse tests.
ID-LIFE-001	Issued identities are never reassigned; correction/supersession/tombstone preserve history.	Lifecycle fixtures and retained translation/public-fact history.
ID-REST-001	Backup/restore/replay preserves producer facts, inbox outcomes, translations and causal graph.	Restore-replay-reconcile rehearsal linked to MIG-REST-001.

Evidence ID	Blocking rule	Required artifact
ID-OBS-001	Identity conflicts and flow lineage are attributable without leaking tokens or private IDs.	Redacted log/metric/trace/audit sample and alert test.
ID-TEST-001	Exact net8 serializer/provider/broker paths execute canonical, collision, concurrency and authorization cases.	Machine results bound to build/contract/provider identities.
ID-REVIEW-001	Owners, architecture, security, data, privacy and operations accept scope/evidence/residual risk.	Identity-bound review, exceptions and expiry.

ADR-015 - Issue public identities at the owner boundary and translate locally

ADR field	ADR-015 record
Status	Accepted for the NEXUS-1 reference design; contract implementation, serializer/provider/broker selection and execution evidence remain pending.
Context	Reusing database keys, correlation IDs, trace IDs or transport receipts across services creates hidden schema, trust and lifecycle coupling.
Decision	Each owner issues typed, opaque, scoped public subject and fact identities. Facts retain identity through commit/replay; consumers deduplicate and translate locally. Correlation/causation are explicit flow context; trace and delivery context remain technical; authorization is local and independent of identifier possession.
Consequences	Database keys may change privately; every consumer needs inbox/translation/lifecycle storage and tests; identity catalogues, privacy, collision handling and restore chains become operated contracts.
Rejected	Cross-database FK/join, shared global identity database, bare GUID equality, correlation/trace/delivery as dedup key, UUID bearer capability, silent remap, ID reassignment, regenerated fact ID on retry and payload hash as sole identity.
Review trigger	Owner/tenant/namespace, public payload, ID format/generator, serializer, broker, mapping policy, authorization/delegation, privacy/retention, correction/merge/split, database migration or restore/replay changes.

Risk register for cross-service identity

Risk	Early signal	Blocking control
Private-key leakage	Consumer schema/API names producer table keys or performs joins/lookups.	ID-REF-001 type/schema/permission scan and local translation.
Bare GUID collision	Lookup ignores producer, kind, version or tenant.	ID-SCOPE-001 composite normalization/uniqueness.
Regenerated retry ID	Same pending fact appears under multiple IDs after crash.	ID-FACT/ID-IDEM atomic pending identity and replay test.
Changed payload under same ID	Consumer overwrites inbox or returns Duplicate silently.	ID-COLL-001 fingerprint mismatch quarantine.
Correlation as key	Unrelated facts collapse into one case/effect.	Typed contexts and architecture/negative tests.
Trace dependence	Sampling or invalid trace changes outcome/deduplication.	ID-TRACE-001 no-trace equivalence.
Identifier grants access	Valid reference bypasses resource/tenant authorization.	ID-AUTH-001 IDOR/delegation tests.
Silent remap	External reference changes local target after race/policy update.	Unique link, concurrency token and explicit lifecycle workflow.
Identity PII/topology	Logs/contracts expose human or infrastructure meaning.	Opaque design, classification and ID-PRIV-001 canary scan.
Reassigned/tombstoned value	Old fact resolves to new subject meaning.	ID-LIFE-001 non-reuse and retained tombstone.
Restore creates new graph	Replayed facts mint new producer/local IDs.	ID-REST-001 identity-preserving restore/reconciliation.
Format/runtime drift	UUID/parser/serializer changes during framework upg	Pinned net8 factory/vectors and explicit compatibility

Official-source validation

The external mechanics were checked against current primary sources on 18 July 2026. W3C Trace Context defines traceparent/tracestate for distributed tracing, with trace-id, parent-id and flags that describe trace position and sampling. RFC 9562 defines 128-bit UUID formats, practical local/distributed uniqueness considerations, collision-risk decisions, UUIDv7 time ordering and security guidance that UUIDs must not be treated as capabilities merely because they appear opaque or hard to guess.

Official source	Stable point used here	NEXUS-1 boundary
W3C Recommendation - Trace Context	traceparent carries trace-id, parent-id and trace flags; tracestate carries vendor-specific context for tracing interoperability.	Trace context stays telemetry context and cannot decide business identity, correlation, idempotency or authorization.
RFC 9562 - UUID format/uniqueness	UUIDs provide practical distributed uniqueness without a required central registry, but applications must weigh collision consequences.	Owner scopes identities and still enforces uniqueness/mismatch handling; no universal bare-GUID registry is assumed.
RFC 9562 - security considerations	UUIDs should not be assumed hard to guess and must not be used as security capabilities.	Every valid public reference still requires authentication and local action/resource/tenant authorization.
RFC 9562 - UUIDv7	UUIDv7 embeds Unix-epoch millisecond time and random/counter fields for time ordering and uniqueness.	Time ordering is optional infrastructure behavior, never domain occurrence or permission.
Microsoft Learn - Guid.CreateVersion7	The .NET 9 API creates RFC 9562 UUIDv7 values and uses.UtcNow by default.	The net8.0 baseline does not call it; UUIDv7 requires an explicit compatible implementation/framework decision.

Sources: <https://www.w3.org/TR/trace-context/> ; <https://www.rfc-editor.org/rfc/rfc9562.html> ; <https://learn.microsoft.com/en-us/dotnet/api/system.guid.createversion7?view=net-9.0>

Technical material still required

The identity taxonomy, owner/scoping rule, public envelope semantics, early-contract tightening, net8 factory boundary, atomic fact allocation, duplicate/collision outcomes, local translation, concurrent first mapping, correlation/causation graph, trace/transport separation, authorization/privacy/lifecycle/restore behavior, scenarios and evidence IDs are specified. No contract type or serializer has compiled; no ID factory or canonicalizer has executed; no broker has transported an envelope; no database has enforced an inbox or translation constraint; and no caller, tenant, collision, replay or restore path has been tested.

TECHNICAL MATERIAL STATUS - [PUBLIC IDENTITY CONTRACT + TRANSLATION/COLLISION/REPLAY/AUTHORIZATION EXECUTION EVIDENCE REQUIRED]

Implement typed public identity and causality contracts in the accepted net8.0 contract/application surfaces; pin ID generation/canonical representation, serializer, contract fixtures, provider mappings, broker metadata and accepted producer authentication; add AlarmManagement pending-fact and RootCause/Audit/Compliance inbox/translation/lifecycle rows with exact composite uniqueness and narrow conflict classification; execute canonicalization, duplicate, changed-payload collision, concurrent first-map, cross-authority/tenant, trace-loss, delivery variation, IDOR/delegation, correction/supersession, migration, backup/restore and replay scenarios; publish ID-OWN-001 through ID-REVIEW-001 linked to CONTRACT-001, FLOW-IDEM-001, FLOW-CAUS-001, ALMDB/RCDB and MIG evidence. RootCause remains G0 accepted and G1 specified; runtime evidence has not advanced.

Deliverable	Minimum technical content	Blocking evidence
Contract types	Authority, subject/fact identity, causality, versioned validation and early v1 adapters.	ID-OWN-001 / ID-SCOPE-001 / ID-CAUS-001
ID factory/canonicalizer	Pinned net8 implementation, canonical wire/database form, test vectors and collision poli	ID-SCOPE-001 / ID-COLL-001 / ID-TEST-001

Deliverable	Minimum technical content	Blocking evidence
Producer persistence	Identity allocated before one local commit and retained in immutable pending public fact.	ID-FACT-001 / ALMDB-TXN-001 / RCDB-TXN-001
Consumer persistence	Inbox identity/fingerprint/outcome plus purpose-specific external link/lifecycle mapping.	ID-MAP-001 / ID-IDEM-001 / ID-CON-001
Edge/transport/trace	Producer authentication, envelope mapping, broker metadata separation and W3C propagation.	ID-TRACE-001 / FLOW-CAUS-001
Security/privacy	Local principal/action/resource/tenant authorization, opaque design, logs, retention and abuse controls.	ID-AUTH-001 / ID-PRIV-001
Lifecycle/recovery	Correction, supersession, merge/split/tombstone and identity-preserving migration/restore/replay.	ID-LIFE-001 / ID-REST-001 / MIG-REST-001
Acceptance	Machine results bound to build/contract/provider/broker identities and multidisciplinary review.	ID-OBS-001 / ID-TEST-001 / ID-REVIEW-001

Implementation sequence

1. Accept ADR-015 and publish the owner/authority/namespace/tenant/lifecycle catalogue for every public subject and fact in the selected slice.
2. Replace early bare EventId/CasId application signatures with typed public fact, subject and causality types while preserving v1 wire compatibility through Edge adapters.
3. Pin the net8.0 ID factory, canonical text/binary form, serializer settings and canonical fingerprint version; publish cross-component vectors.
4. Allocate producer fact identity before the AlarmManagement or RootCause local commit and store it with immutable pending public-fact bytes.
5. Implement consumer inbox tuples, fingerprints and stable outcomes; classify only exact duplicate/collision constraints and leave unknown failures as failures.
6. Implement purpose-specific external-identity links with scoped uniqueness, local-only foreign keys, mapping-policy identity, lifecycle history and concurrency token.
7. Propagate typed business correlation/full causal fact references separately from W3C trace and broker delivery context.
8. Authenticate accepted producers/callers and prove owner-local principal/action/resource/tenant authorization; public ID possession never grants access.
9. Execute canonical, malformed, cross-scope, duplicate, collision, concurrent mapping, trace-loss, delivery, correction and privacy scenarios on real serializer/provider/broker paths.
10. Rehearse migration, backup/restore and fact replay while preserving pending facts, inbox outcomes, translations and causal graph; publish ID evidence without advancing unproved runtime gates.

What this chapter has proved

Chapter 17 has defined the cross-service identity contract without cross-database foreign keys: owner-scoped public subject/fact types; early v1 wrapper compatibility; stable identity allocation beside the local commit; duplicate versus changed-payload collision behavior; local translation under pinned policy and concurrency; explicit correlation/full causation; telemetry and transport separation; local authorization independent of identifier possession; opaque privacy-minimized lifecycle; correction/supersession/tombstone; identity-preserving restore/replay; and evidence gates for the selected AlarmManagement -> RootCause -> Audit/Compliance flow.

It has not compiled a contract, selected final serialized fields, executed an ID generator, created an inbox/translation migration, authenticated a producer, propagated a trace, handled a duplicate/collision/race, authorized a public reference, restored an identity graph or advanced runtime evidence. The semantics are ready to

REFERENCES PRESERVE MEANING, NOT STORAGE

A consumer may know exactly which AlarmManagement fact or RootCause verdict it received without knowing a producer table key, database location or access path. That is the useful middle: durable lineage with independent storage and trust.

Close the identity design with one public catalogue record

Catalogue field	Accepted record	Why it must be explicit
Owner / authority	Service/team that issues, corrects, tombstones and supports the identity.	Stops shared/global identity ownership from emerging implicitly.
Kind / scope / format	Namespace, tenant/audience rule, canonical value, maximum length and parser/version.	Prevents bare-value, collation and serialization drift.
Lifecycle	Issue, correction, supersession, merge/split, tombstone, retention and non-reuse policy.	Historical facts remain interpretable after current data changes.
Consumers / purpose	Every consumer, local translation purpose and authorization exposure.	Removes unused identifiers and accidental public database keys.
Evidence / expiry	Contract/provider/broker/security/restore results, reviewer and next review trigger.	The identity claim remains bound to executed artifacts and residual risk.

CHAPTER 17 - CODE AUDIT

Public identity is typed, scoped and canonical

The executable path starts with types that cannot accept a private key, trace id or delivery tag by accident. Authority, contract and canonical value travel together; wire representation is deterministic and authorization remains separate.

EVIDENCE STATUS

These listings are compile-oriented reference material. No selected serializer, provider, broker, authentication path, migration or executed test suite exists in this manuscript environment. ID and CONTRACT evidence remains PENDING.

EXECUTABLE ASSET 17-A - VALIDATED PUBLIC IDENTITY TYPES

```
public readonly record struct IdentityAuthority
{
    public string Value { get; }
    private IdentityAuthority(string value) => Value = value;

    public static IdentityAuthority Parse(string value)
    {
        if (string.IsNullOrEmpty(value)
            || value.Length > 80
            || value != value.Trim()
            || value.Any(char.IsControl))
            throw new InvalidIdentityAuthority(value);
        return new IdentityAuthority(value);
    }
}

public readonly record struct PublicFactIdentity(
    IdentityAuthority Producer,
    Guid FactId,
    string ContractType,
    int ContractVersion)
{
    public void DemandValid()
    {
        if (FactId == Guid.Empty
            || string.IsNullOrEmpty(ContractType)
            || ContractType.Length > 120
            || ContractVersion < 1)
            throw new InvalidPublicFactIdentity(this);
    }
}

public sealed record PublicSubjectReference(
    IdentityAuthority Authority,
    string Namespace,
    string Value,
    string? TenantScope);
```

EXECUTABLE ASSET 17-B - STABLE CANONICAL BYTES

```
public static class PublicFactIdentityWire
{
    public static byte[] Write(PublicFactIdentity value)
    {
        value.DemandValid();
        var buffer = new ArrayBufferWriter<byte>();
        using var json = new Utf8JsonWriter(buffer);
        json.StartObject();
        json.WriteString("producer", value.Producer.Value);
        json.WriteString("factId",
            value.FactId.ToString("D").ToLowerInvariant());
        json.WriteString("contractType", value.ContractType);
        json.WriteNumber("contractVersion", value.ContractVersion);
        json.EndObject();
        json.Flush();
        return buffer.WrittenSpan.ToArray();
    }
}
```

CONTRACT COMPATIBILITY

A v1 bare GUID gains scope only at the trusted edge

EXECUTABLE ASSET 17-C - ACCEPTED PRODUCER CONTEXT

```
public sealed record AcceptedProducerContext(
    IdentityAuthority Producer,
    string ContractType,
    int ContractVersion,
    string AuthenticationEvidenceId);

public sealed class AlarmFloodDetectedV1Adapter
{
    public PublicFactEnvelope<AlarmFloodDetectedV1> Adapt(
        AlarmFloodDetectedV1 input,
        AcceptedProducerContext accepted,
        string correlationId)
    {
        if (!StringComparer.Ordinal.Equals(
            accepted.ContractType, "AlarmFloodDetected")
            || accepted.ContractVersion != 1)
            throw new UnexpectedProducerContract(accepted);

        var fact = new PublicFactIdentity(
            accepted.Producer,
            input.EventId,
            accepted.ContractType,
            accepted.ContractVersion);
        fact.DemandValid();

        return new PublicFactEnvelope<AlarmFloodDetectedV1>(
            fact,
            PublicSubjects.FloodDecision(
                accepted.Producer, input.DecisionId),
            input.OccurredAtUtc,
            new CausalityContext(correlationId, null),
            input);
    }
}
```

EXECUTABLE ASSET 17-D - CANONICAL AND SPOOFING VECTORS

```
[Fact]
public void ID_SCOPE_001_canonical_fact_vector_is_exact()
{
    var fact = IdentityFixture.AlarmFact(
        "2f3ec2f4-5c36-4d31-8e93-6cdd996702a8");

    Encoding.UTF8.GetString(PublicFactIdentityWire.Write(fact))
        .Should().Be(
            "{ \"producer\": \"nexus.alarm-management\", \""
            + "\"factId\": \"2f3ec2f4-5c36-4d31-8e93-6cdd996702a8\", \""
            + "\"contractType\": \"AlarmFloodDetected\", \""
            + "\"contractVersion\": 1 }");
}

[Fact]
public void v1_payload_cannot_choose_its_producer_scope()
{
    var input = IdentityFixture.V1PayloadWithSpoofedExtension(
        producer: "nexus.root-cause");
    var envelope = fixture.Adapter.Adapt(
        input,
        fixture.AuthenticatedAlarmManagement,
        "flow-41");

    envelope.Fact.Producer.Should()
        .Be(IdentityFixture.AlarmManagementAuthority);
}

[Theory]
[InlineData("")]
[InlineData(" nexus.alarm-management")]
[InlineData("nexus.alarm-management\n")]
public void malformed_authority_is_rejected(string value)
{
    var act = () => IdentityAuthority.Parse(value);
    act.Should().Throw<InvalidIdentityAuthority>();
}
```

COMPATIBILITY DOES NOT TRUST PAYLOAD CLAIMS

The v1 adapter combines the legacy EventId with authenticated producer and accepted contract context. A payload field, hostname or queue name cannot choose authority.

PRODUCER IDENTITY

One accepted decision allocates one durable public fact

EXECUTABLE ASSET 17-E - FACT IDENTITY ENTERS THE LOCAL COMMIT

```
public sealed class AlarmFloodFactWriter(
    IPublicFactIdFactory factIds,
    IPublicFactCodec codec,
    TimeProvider clock)
{
    public PendingPublicFactRow Create(
        AlarmFloodDecision decision,
        CausalityContext causality)
    {
        var fact = new PublicFactIdentity(
            AlarmManagementIdentity.Authority,
            factIds.NewFactId(),
            "AlarmFloodDetected",
            1);
        fact.DemandValid();

        var envelope = AlarmFloodDetectedMapper.Create(
            fact,
            PublicSubjects.FloodDecision(
                AlarmManagementIdentity.Authority,
                decision.Identity),
            causality,
            decision,
            clock.GetUtcNow());
        var canonical = codec.SerializeCanonical(envelope);

        return PendingPublicFactRow.Create(
            fact,
            canonical,
            SHA256.HashData(canonical));
    }
}

public async Task<AlarmDecisionOutcome> HandleAsync(
    EvaluateAlarmFlood command,
    CancellationToken cancellationToken)
{
    var prior = await outcomes.FindAsync(
        command.RequestIdentity, cancellationToken);
    if (prior is not null) return prior;

    var decision = policy.Evaluate(command.Observations);
    var pending = facts.Create(decision, command.Causality);
    db.AlarmFloodDecisions.Add(decision);
    db.PendingPublicFacts.Add(pending);
    await db.SaveChangesAsync(cancellationToken);
    return AlarmDecisionOutcome.Accepted(decision.Identity, pending.Fact);
}
```

EXECUTABLE ASSET 17-F - RETRY, CORRECTION AND RESTORE INVARIANTS

```
[Fact]
public async Task ID_FACT_001_dispatch_retry_reuses_stored_bytes_and_id()
{
    var stored = await fixture.CommitOneAlarmDecisionAsync();
    await fixture.DispatchUntilAcceptedAsync(stored.Id, failures: 2);

    fixture.PublicFactIdAllocations.Should().Be(1);
    fixture.Published.Select(x => x.Fact).Distinct()
        .Should().ContainSingle().Which.Should().Be(stored.Fact);
    fixture.Published.Select(x => x.Bytes).Distinct(ByteEqualityComparer.Instance)
        .Should().ContainSingle();
}

[Fact]
public async Task correction_gets_new_id_and_names_prior_fact()
{
    var original = await fixture.CommitOneAlarmDecisionAsync();
    var correction = await fixture.CorrectAsync(original.Fact);

    correction.Fact.Should().NotBe(original.Fact);
    correction.Causality.Causation.Should().Be(original.Fact);
}

[Fact]
public async Task restored_pending_fact_is_not_regenerated()
{
    var before = await fixture.PendingFactSnapshotAsync();
    await fixture.RestoreAndReplayPendingAsync();
    var after = await fixture.PendingFactSnapshotAsync();
    after.Should().BeEquivalentTo(before);
}
```

THE DISPATCHER IS NOT AN IDENTITY FACTORY

Allocation happens before the owner commit. Publish retry, crash recovery and disaster replay read the immutable stored identity and canonical bytes.

CONSUMER DEDUPLICATION

The same tuple may be duplicate or identity conflict

EXECUTABLE ASSET 17-G - INBOX TUPLE AND NARROW UNIQUENESS

```
internal sealed class InboundPublicFactRow
{
    public long Id { get; init; }
    public required string Producer { get; init; }
    public Guid PublicFactId { get; init; }
    public required string ContractType { get; init; }
    public int ContractVersion { get; init; }
    public required byte[] CanonicalFingerprint { get; init; }
    public Guid? LocalOutcomeId { get; set; }
    public required string StableOutcome { get; set; }
}

internal sealed class InboundPublicFactConfiguration
    : IEntityTypeConfiguration<InboundPublicFactRow>
{
    public void Configure(EntityTypeBuilder<InboundPublicFactRow> b)
    {
        b.ToTable("InboundPublicFacts", "rootcause");
        b.HasKey(x => x.Id);
        b.Property(x => x.Producer).HasMaxLength(80);
        b.Property(x => x.ContractType).HasMaxLength(120);
        b.Property(x => x.CanonicalFingerprint).HasMaxLength(32);
        b.HasIndex(x => new
        {
            x.Producer,
            x.PublicFactId,
            x.ContractType,
            x.ContractVersion
        }).IsUnique();
    }
}
```

EXECUTABLE ASSET 17-H - DURABLE DUPLICATE/CONFLICT CLASSIFIER

```
public static FactReplayOutcome Classify(
    InboundPublicFactRow stored,
    PublicFactIdentity incoming,
    ReadOnlySpan<byte> canonicalFingerprint)
{
    var sameTuple =
        StringComparer.Ordinal.Equals(
            stored.Producer, incoming.Producer.Value)
        && stored.PublicFactId == incoming.FactId
        && StringComparer.Ordinal.Equals(
            stored.ContractType, incoming.ContractType)
        && stored.ContractVersion == incoming.ContractVersion;

    if (!sameTuple)
        throw new FactTupleMismatch();
    if (!CryptographicOperations.FixedTimeEquals(
        stored.CanonicalFingerprint,
        canonicalFingerprint))
        return FactReplayOutcome.IdentityConflict;
    return FactReplayOutcome.Duplicate(
        stored.StableOutcome, stored.LocalOutcomeId);
}

[Fact]
public void ID_COLL_001_changed_bytes_under_same_tuple_are_blocked()
{
    var stored = IdentityFixture.StoredReceipt("payload-A");
    var outcome = Classify(
        stored, stored.Identity, Hash("payload-B"));

    outcome.Kind.Should().Be(FactReplayOutcomeKind.IdentityConflict);
    fixture.LocalEffectCount.Should().Be(1);
}
```

A FINGERPRINT IS DIAGNOSTIC, NOT IDENTITY

The exact producer tuple selects the receipt. Equal canonical content returns the stored outcome; changed content under the same tuple blocks and is never overwritten.

CONSUMER TRANSLATION

Concurrent first sight may select one local mapping only

EXECUTABLE ASSET 17-I - PURPOSE-SPECIFIC EXTERNAL LINK

```
public sealed record ExternalIdentityKey(
    string Authority,
    string Namespace,
    string Value,
    string TenantScope);

internal sealed class ExternalIdentityLinkRow
{
    public long Id { get; init; }
    public required string Authority { get; init; }
    public required string Namespace { get; init; }
    public required string ExternalValue { get; init; }
    public required string TenantScope { get; init; }
    public required string LocalKind { get; init; }
    public Guid LocalId { get; init; }
    public required string MappingPolicyIdentity { get; init; }
    public Guid FirstSeenFactId { get; init; }
    public required string State { get; set; }
    public byte[] Version { get; set; } = [];
}

internal static void Configure(EntityTypeBuilder<ExternalIdentityLinkRow> b)
{
    b.ToTable("ExternalIdentityLinks", "rootcause");
    b.HasIndex(x => new
    {
        x.Authority,
        x.Namespace,
        x.ExternalValue,
        x.TenantScope
    }).IsUnique();
    b.Property(x => x.Version).IsRowVersion();
    b.HasOne<RootCauseCaseRow>().WithMany()
        .HasForeignKey(x => x.LocalId);
    // No producer database foreign key or lookup exists.
}
```

EXECUTABLE ASSET 17-J - LOSER RELOADS AND PROVES EQUIVALENCE

```
public async Task<ExternalIdentityLink> GetOrCreateAsync(
    ExternalIdentityProposal proposal,
    CancellationToken cancellationToken)
{
    var created = await TryCreateAsync();
    if (created is not null) return created;

    await using var reload = scopes.CreateAsyncScope();
    var reader = reload.ServiceProvider
        .GetRequiredService<IExternalIdentityLinkReader>();
    var winner = await reader.DemandAsync(
        proposal.Key, cancellationToken);
    if (!proposal.MappingPolicy.IsEquivalent(winner, proposal))
        throw new ExternalIdentityConflict(
            proposal.Key, winner.LocalIdentity);
    return winner;

    async Task<ExternalIdentityLink?> TryCreateAsync()
    {
        await using var attempt = scopes.CreateAsyncScope();
        var store = attempt.ServiceProvider
            .GetRequiredService<IExternalIdentityLinkStore>();
        try
        {
            return await store.CreateAsync(
                proposal, cancellationToken);
        }
        catch (DbUpdateException exception)
            when (conflicts.IsExactExternalTuple(exception))
        {
            return null;
        }
    }
}

[Fact]
public async Task ID_CON_001_racing_first_map_never_remaps()
{
    var outcomes = await fixture.RaceMappingsAsync(
        localA: CaseId.Parse("RC-27"),
        localB: CaseId.Parse("RC-91"));
    outcomes.Should().ContainSingle(x => x.IsAccepted);
    outcomes.Should().ContainSingle(x => x.IsIdentityConflict);
    (await fixture.ReadLinksAsync()).Should().ContainSingle();
}
```

THE DATABASE ARBITRATES; POLICY EXPLAINS

Scoped uniqueness chooses one durable row. The losing attempt uses a fresh unit of work and continues only when pinned mapping policy proves the winner equivalent.

FLOW AND TELEMETRY

Causality survives when tracing and delivery metadata change

EXECUTABLE ASSET 17-K - BUSINESS AND TECHNICAL CONTEXT STAY SEPARATE

```
public readonly record struct CorrelationIdentity(string Value);

public sealed record BusinessCausality(
    CorrelationIdentity Correlation,
    PublicFactIdentity? Causation);

public sealed record DeliveryContext(
    string? TransportMessageId,
    int Attempt,
    string? TraceParent,
    string? TraceState);

public sealed record AcceptedPublicFact<TPayload>(
    PublicFactIdentity Fact,
    PublicSubjectReference? Subject,
    BusinessCausality Causality,
    TPayload Payload);

public sealed class RootCauseConsumer
{
    public Task<RootCauseOutcome> HandleAsync(
        AcceptedPublicFact<AlarmFloodDetectedV1> fact,
        DeliveryContext delivery,
        CancellationToken cancellationToken)
    {
        telemetry.RecordDelivery(fact.Fact, delivery);
        return application.HandleAsync(
            RootCauseCommand.From(fact), cancellationToken);
    }
}
```

EXECUTABLE ASSET 17-L - TRACE-LOSS AND DELIVERY-VARIATION GATES

```
[Theory]
[InlineData(null, null, 1)]
[InlineData("00-a1-b1-01", "broker-7", 1)]
[InlineData("00-c9-d9-00", "broker-91", 4)]
public async Task ID_TRACE_001_technical_context_cannot_change_effect(
    string? traceParent,
    string? transportId,
    int attempt)
{
    var outcome = await fixture.HandleInFreshDatabaseAsync(
        fixture.AcceptedAlarmFact,
        new DeliveryContext(
            transportId, attempt, traceParent, null));

    outcome.BusinessSnapshot.Should()
        .BeEquivalentTo(fixture.ExpectedBusinessSnapshot);
    outcome.Causation.Should()
        .Be(fixture.AcceptedAlarmFact.Fact);
}

[Fact]
public void domain_has_no_trace_or_transport_dependencies()
{
    var assembly = typeof(RootCauseCase).Assembly;
    foreach (var forbidden in new[]
    {
        "System.Diagnostics",
        "OpenTelemetry",
        "MassTransit"
    })
    {
        var result = Types.InAssembly(assembly)
            .Should().NotHaveDependencyOn(forbidden).GetResult();
        result.IsSuccessful.Should().BeTrue(
            $"Domain must not depend on {forbidden}");
    }
}
```

OBSERVABILITY MAY DISAPPEAR; MEANING MAY NOT

Sampling, trace propagation, retry count and broker identity may vary across attempts. Deduplication, correlation, causation and the business result remain stable.

AUTHORIZATION, LIFECYCLE AND PRIVACY

Possessing an opaque reference grants no operation

EXECUTABLE ASSET 17-M - RESOLVE, THEN AUTHORIZE LOCALLY

```
public sealed class VerdictReferenceEndpoint(
    IPublicReferenceResolver references,
    ILocalResourceAuthorizer authorization)
{
    public async Task<IResult> GetAsync(
        ClaimsPrincipal principal,
        string authority,
        string value,
        string tenant,
        CancellationToken cancellationToken)
    {
        var reference = new PublicSubjectReference(
            IdentityAuthority.Parse(authority),
            "RootCauseVerdict",
            value,
            tenant);
        var resolved = await references.ResolveAsync(
            reference, cancellationToken);
        if (resolved is null) return Results.NotFound();

        var decision = await authorization.EvaluateAsync(
            principal,
            action: "verdict:read",
            resource: resolved.LocalResource,
            tenant: tenant,
            cancellationToken);
        if (!decision.Allowed) return Results.Forbid();
        return Results.Ok(VerdictContract.From(resolved));
    }
}

[Fact]
public async Task ID_AUTH_001_valid_reference_cannot_cross_tenant()
{
    var reference = await fixture.CreateVerdictReferenceAsync("tenant-a");
    var response = await fixture.GetAsync(
        fixture.TenantBReader, reference, tenant: "tenant-b");
    response.StatusCode.Should().Be(HttpStatusCode.Forbidden);
    fixture.VerdictReadCount.Should().Be(0);
}
```

EXECUTABLE ASSET 17-N - APPEND-ONLY LIFECYCLE AND PRIVACY GATES

```
public sealed record PublicIdentityTransition(
    PublicSubjectReference Identity,
    IdentityLifecycle From,
    IdentityLifecycle To,
    PublicFactIdentity Cause,
    DateTimeOffset RecordedAtUtc);

public static void DemandTransition(
    IdentityLifecycle current,
    IdentityLifecycle requested)
{
    var allowed = (current, requested) switch
    {
        (IdentityLifecycle.Active, IdentityLifecycle.Superseded) => true,
        (IdentityLifecycle.Active, IdentityLifecycle.Tombstoned) => true,
        (IdentityLifecycle.Superseded, IdentityLifecycle.Tombstoned) => true,
        _ => false
    };
    if (!allowed) throw new IdentityLifecycleViolation(current, requested);
}

[Fact]
public void ID_LIFE_001_tombstoned_value_is_never_reissued()
{
    fixture.Catalogue.AppendTombstone(fixture.PublicVerdict);
    var act = () => fixture.Catalogue.Issue(fixture.PublicVerdict);
    act.Should().Throw<PublicIdentityReuseRejected>();
}

[Fact]
public async Task ID_PRIV_001_logs_do_not_expose_identity_canaries()
{
    await fixture.ExecuteAbuseAndConflictScenariosAsync();
    fixture.Logs.Should().NotContain(fixture.RawIdentityCanary);
    fixture.Logs.Should().NotContain(fixture.RawTenantCanary);
    fixture.Logs.Should().Contain("identityConflict=true");
}
```

IDENTITY AND AUTHORITY REMAIN INDEPENDENT

Resolution establishes local meaning, not permission. Authentication and the exact principal/action/resource/tenant policy are evaluated for every operation.

RESTORE, REPLAY AND ACCEPTANCE

A restored service must preserve the same public history

EXECUTABLE ASSET 17-O - IDENTITY-PRESERVING RECOVERY GATE

```
public sealed class IdentityRestoreAcceptance(
    IIdentityStateFingerprint fingerprints,
    IReplayHarness replay)
{
    public async Task DemandAsync(
        IdentityRecoveryBaseline expected,
        CancellationToken cancellationToken)
    {
        var observed = await fingerprints.ReadAsync(cancellationToken);
        observed.PendingFacts.Should().Be(expected.PendingFacts);
        observed.InboxOutcomes.Should().Be(expected.InboxOutcomes);
        observed.ExternalLinks.Should().Be(expected.ExternalLinks);
        observed.LifecycleHistory.Should().Be(expected.LifecycleHistory);
        observed.CausalGraph.Should().Be(expected.CausalGraph);

        var replayed = await replay.ReplayAsync(
            expected.Window, cancellationToken);
        replayed.NewPublicFactIds.Should().BeEmpty();
        replayed.DuplicateBusinessEffects.Should().Be(0);
        replayed.IdentityConflicts.Should().Be(0);
    }
}

[Fact]
public async Task ID_REST_001_RootCause_identity_restore_is_independent()
{
    var alarmBefore = await fixture.AlarmManagementFingerprintAsync();
    var identityBefore = await fixture.RootCauseIdentityBaselineAsync();

    await fixture.RestoreAndReplayRootCauseAsync();
    await fixture.IdentityAcceptance.DemandAsync(
        identityBefore, CancellationToken.None);

    var alarmAfter = await fixture.AlarmManagementFingerprintAsync();
    alarmAfter.Should().Be(alarmBefore);
}
```

EXECUTABLE ASSET 17-P - ACCEPTANCE EVIDENCE RECORD

```
{
  "evidenceId": "ID-REVIEW-001", "chapter": 17,
  "artifactCommit": null, "sdk": "net8.0",
  "serializer": null, "provider": null, "engineVersion": null,
  "broker": null, "authenticationProfile": null,
  "tests": {
    "ID-OWN-001": "PENDING", "ID-SCOPE-001": "PENDING",
    "ID-FACT-001": "PENDING", "ID-IDEM-001": "PENDING",
    "ID-COLL-001": "PENDING", "ID-MAP-001": "PENDING",
    "ID-CON-001": "PENDING", "ID-CAUS-001": "PENDING",
    "ID-TRACE-001": "PENDING", "ID-AUTH-001": "PENDING",
    "ID-PRIV-001": "PENDING", "ID-LIFE-001": "PENDING",
    "ID-REST-001": "PENDING", "ID-TEST-001": "PENDING"
  },
  "canonicalVectorsAccepted": false,
  "restoreRehearsed": false, "accepted": false
}
```

Chapter 17 code-audit result

The chapter now exposes one executable path from authenticated producer scope and canonical public identity to owner-local persistence, consumer deduplication, translation, causality, authorization and identity-preserving recovery.

CURRENT VERDICT

The identity model is implementable in shape, but no contract package, serializer, provider, broker or authorization path has executed. All runtime and recovery evidence remains explicitly PENDING.

Chapter checkpoint

You are ready to implement cross-service identity when every public value has one owner and scope, producer facts retain one immutable identity through replay, consumers deduplicate and translate locally, correlation/causation explain the flow without becoming keys, trace/delivery context cannot change business meaning, and every valid reference still passes local authorization.

Question	Expected answer
May a consumer store a producer reference?	Yes, as an opaque scoped public reference with lineage and local purpose; never as a cross-database foreign key or source truth.
What identifies one public fact?	Producer authority + PublicFactId + contract type/version, retained with immutable payload/fingerprint.
When is a repeated fact a Duplicate?	The exact producer tuple matches and canonical content/subject scope are equivalent; the prior stable outcome is returned.
What if the same fact ID carries different content?	IdentityConflict/collision or tampering: block/quarantine and never overwrite or report success.
How is an external subject mapped?	A consumer-owned translation row under scoped uniqueness, first-seen fact and pinned mapping policy.
What happens in a concurrent first mapping?	One unique link wins; losers roll back, reload and prove equivalence or block without remapping.
What do correlation and causation do?	Correlation groups the wider flow; causation references the exact direct predecessor. Neither deduplicates or authorizes.
Is trace-id the business correlation ID?	No. W3C trace context is telemetry that may be absent, changed or sampled; durable flow identity is separate.
Can broker message ID or delivery tag be the fact identity?	No. An adapter may mirror a value for operations, but durable equality uses the validated public envelope tuple.
Does an opaque UUID grant access?	No. The owner authenticates and locally authorizes principal, action, resource and tenant for every resolution/command.
May a public identity be reused?	Never for different meaning. Correction/supersession/tombstone appends lifecycle and preserves prior facts.
What must restore preserve?	Pending fact identities, inbox outcomes, translations, mapping policy, lifecycle and correlation/causation independent of trace retention.
What is the current status?	Identity semantics are specified, not implemented/evidenced; RootCause remains G0 accepted and G1 specified.
What does Chapter 18 add?	Local transactions, concurrency and commit boundaries for commands, inbox/outbox work and owned state without distributed transactions.

NEXT CHAPTER

Chapter 18, Local Transactions, Concurrency, and Commit Boundaries, will turn the identity and database rules into exact units of atomicity. It will distinguish command, consumer and dispatcher commits; define optimistic concurrency and unique-conflict handling; place inbox/outbox records beside owned state; classify ambiguous failures; and prove that NEXUS-1 preserves meaning without a transaction spanning services or the broker.

PART III - DATA OWNERSHIP IN EXECUTABLE FORM

18. Local Transactions, Concurrency, and Commit Boundaries

A service preserves meaning by making one owner's state change atomic, not by pretending the whole distributed flow commits together. Every unit of work must name the database rows, identities, conflict policy and recovery proof that belong to its local decision.

CHAPTER ARGUMENT

Use one short local database transaction for one semantic operation. Stage owned state, receipt/outcome and pending public intent together; handle concurrency by re-evaluating meaning; treat a lost commit response as unknown until durable evidence resolves it; and never extend the transaction across a broker, remote service or second owner's database.

Chapters 14 and 15 already specified the durable tuples. AlarmManagement commits its owned decision and pending public fact together. RootCause commits its inbound receipt, case/evidence/verdict change and pending public fact together. Chapter 17 fixed the public identities that must survive dispatch, retry, restore and replay. Chapter 18 turns those rules into exact commit boundaries and failure outcomes.

This chapter does not select the final provider, broker topology, serialized envelope or outbox dispatcher algorithm. Chapter 19 separates domain and integration events; Chapters 20 and 24 own the envelope and compatibility proof; Chapters 26 and 27 implement the transactional outbox and dispatcher recovery; Chapter 28 completes inbox handling. The present contract says what those components must preserve before their mechanics are chosen.

No NEXUS-1 repository, EF Core provider, database engine, execution strategy, transaction interceptor or broker path has run in this manuscript environment. The net8.0 listings are reference candidates. RootCause remains G0 accepted and G1 specified; transaction, concurrency and failure evidence has not advanced.

One transaction changes one owner's truth

Boundary	Atomic local set	Outside the transaction	Success becomes knowable when
AlarmManagement command	Decision/aggregate state + immutable pending AlarmFloodDetected fact + commit intent where selected.	Broker publish, RootCause state, subscriber effects, HTTP client receipt.	The owning database confirms commit or later reconciliation finds the stable intent/outcome.
RootCause consumer	Inbound receipt/outcome + case/evidence/hypothesis/verdict state + pending RootCause fact.	Broker acknowledgement, producer database, Audit/Compliance/Reporting effects.	RootCauseDb confirms commit or redelivery resolves the same producer tuple to its durable result.
Audit/Compliance consumer	Consumer-owned receipt + append-only record or policy decision + local outgoing intent if any.	Other subscriber progress and RootCause private rows.	That consumer's database confirms its own effect.
Dispatcher lease/state	Owner-local selection/lease/attempt marker or confirmed publication status update.	The broker publish call and every subscriber commit.	Local status and broker evidence are reconciled under one public fact identity.
Projection/checkpoint	Projection mutation + owner-local checkpoint when the projection contract requires atomicity.	Source aggregate and other projections.	Projection store confirms its own checkpoint/effect.

ATOMICITY BOUNDARY

A local transaction may include many tables for one owner's invariant. It never includes a second service merely because both effects share a workflow

Command, consumer and dispatcher have different commits

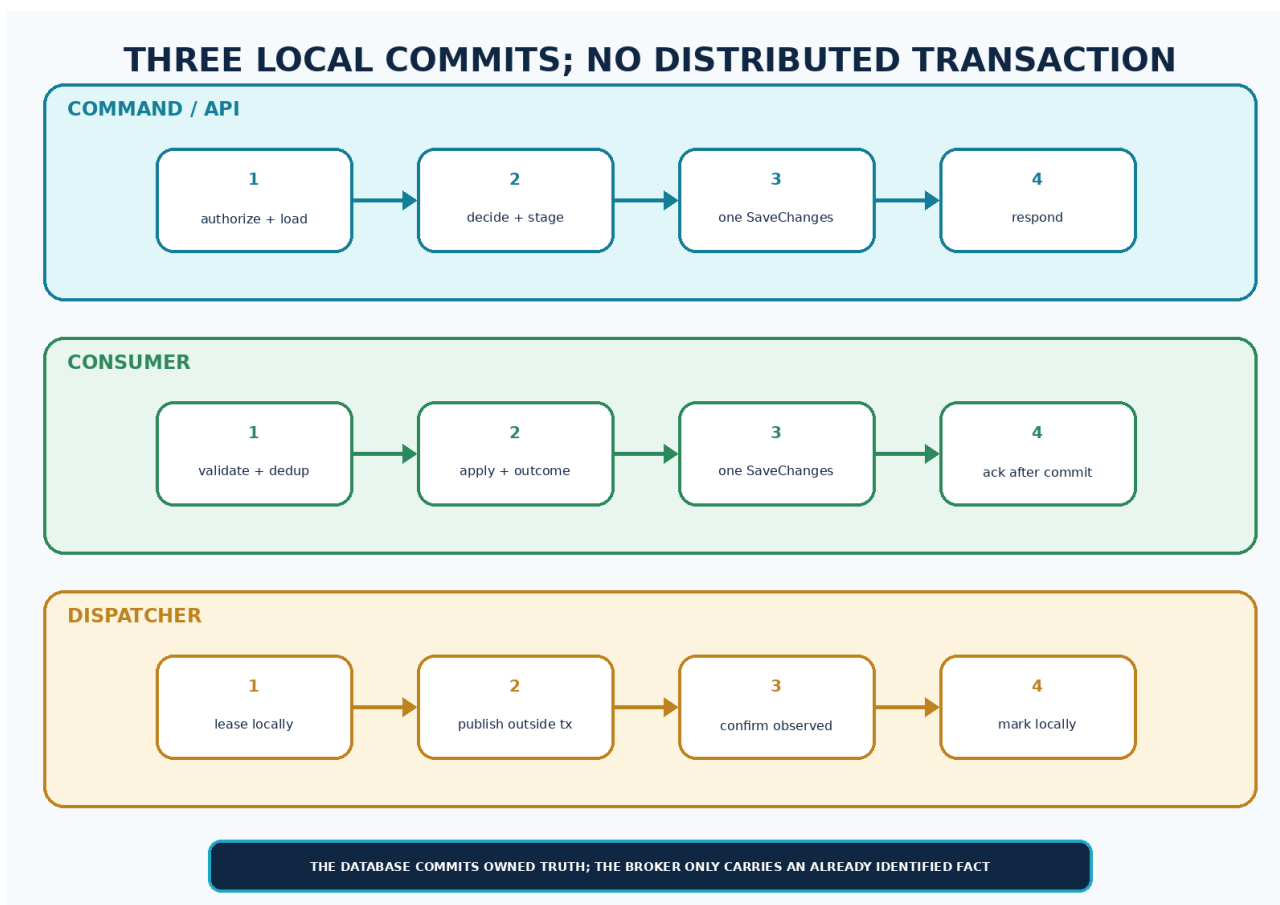


Figure 18.1 - Commands and consumers each commit owned meaning once; dispatch crosses the database/broker gap explicitly.

The first two lanes can usually stage every required row in one tracked DbContext and one SaveChanges. The dispatcher cannot hold a database transaction open while waiting for a broker without creating long locks and still cannot make the broker and database one atomic resource. It therefore accepts a recoverable gap: publish may be repeated under the same PublicFactIdentity, and local status may lag broker acceptance.

Lane	Database commit	External action	Crash-safe consequence
Command/API	Owned mutation + pending fact.	Return transport outcome after commit classification.	A client retry uses request/business identity to discover or reattempt the same meaning.
Consumer	Receipt + stable outcome + owned mutation + pending fact.	Acknowledge only after the commit is known or recoverably reconciled.	Commit followed by lost acknowledgement becomes duplicate redelivery, not a second effect.
Dispatcher	Lease/attempt and final status are separate short local commits.	Publish stored immutable bytes and await selected broker evidence.	Publish followed by lost status update may redeliver the same public fact; consumers absorb it.

A complete vertical slice: AlarmManagement to RootCause

Follow one accepted alarm flood through the complete distributed path. AlarmManagement decides that a correlated burst qualifies as a flood, commits that decision with an immutable pending AlarmFloodDetected fact, and returns an operation outcome. A dispatcher later publishes the stored fact. RootCause validates and deduplicates the delivery, applies its own case/evidence policy, commits a stable consumer outcome, and only then allows acknowledgement. The user-visible workflow is one story; the durable reality is two owner-local commits separated by a recoverable delivery gap.

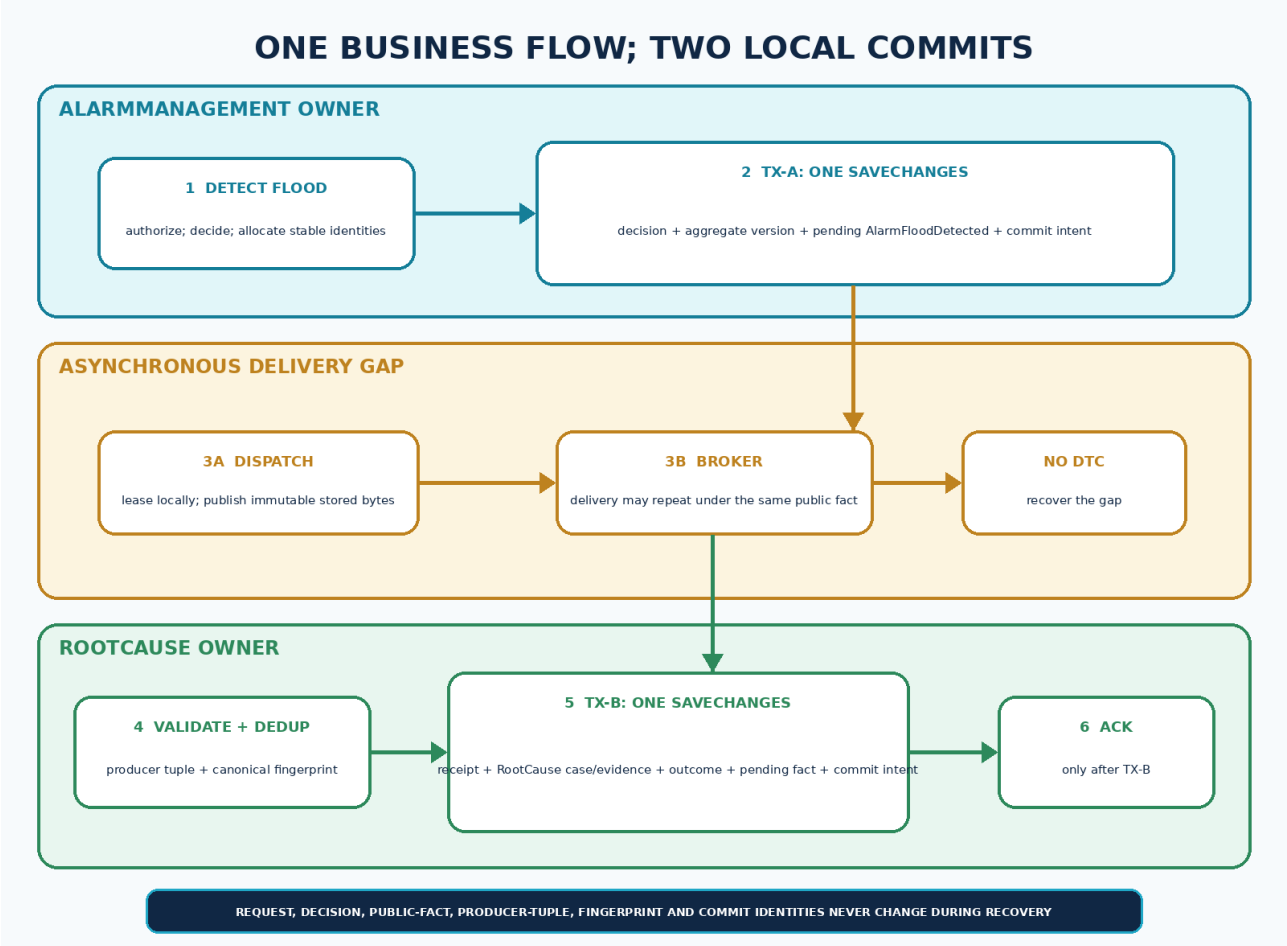


Figure 18.2 - One AlarmFloodDetected flow crosses two owner-local commits and an explicit asynchronous delivery gap.

HOW TO READ THE WALKTHROUGH

TX-A belongs only to AlarmManagementDb. TX-B belongs only to RootCauseDb. The broker call, HTTP response, delivery and acknowledgement sit outside both transactions. Every retry or replay reuses the same accepted meaning and identities.

The identity spine is allocated once and then carried forward

The values below are readable example tokens, not prescribed production formats. Their purpose is to make continuity visible. A real implementation may use UUIDs, ULIDs or another accepted representation, but it must preserve the same roles and immutability rules.

Identity role	Example value	First durable owner	Where it is reused
RequestIdentity	request-am-20482	AlarmManagement API/command record.	HTTP retry and command outcome lookup.
DecisionIdentity	decision-am-9184	AlarmManagement decision row.	Pending fact causality, audit evidence and retry equivalence.
PublicFactIdentity	fact-am-7716	AlarmManagement pending-public-fact row.	Dispatcher attempts, broker deliveries and every consumer receipt.

Identity role	Example value	First durable owner	Where it is reused
Producer fact tuple	AlarmManagement / alarm-flood / 184229 / fact-am-7716	Producer publication history.	RootCause inbox uniqueness, restore and replay.
Canonical fingerprint	sha256:8b2f...	Immutable stored public payload.	RootCause changed-payload detection and duplicate proof.
CommitIntentId	commit-am-5512; commit-rc-7821	Each database owns its own intent.	Fresh outcome verification after timeout or disconnect.
RootCauseCaseId	case-rc-3107	RootCause private state only.	RootCause evidence, hypothesis and verdict history; never the producer key.

Step 1 - AlarmManagement accepts one command meaning

Edge authenticates the caller and validates the request contract. Application authorizes the operation, loads the AlarmManagement aggregate and evaluates the flood policy. If the same RequestIdentity already has a durable outcome, the handler returns that outcome. Otherwise it produces decision-am-9184 and allocates fact-am-7716 once. A transport retry cannot allocate a second decision or public fact merely because the first response was lost.

Input retained for equivalence	Example	Why it matters
Request identity	request-am-20482	Finds a previously committed command outcome.
Correlation scope	site-17 / line-4 / 10-minute window	Defines which alarm burst the policy evaluated.
Policy identity	alarm-flood-policy-v7	Prevents a retry from silently using different decision rules.
Accepted observations	alarm positions 88120-88167	Keeps the evidence set stable and auditable.

Step 2 - TX-A commits the AlarmManagement decision and public intent

AlarmManagement stages every row required to say that the flood decision exists and is eligible for later publication. One scoped AlarmManagementDb context saves the aggregate version, decision, pending immutable fact, request outcome and commit intent. The broker has not been called. RootCause has not been contacted. Success at TX-A proves only AlarmManagement truth.

ALARMMANAGEMENT TX-A - [NEW][REFERENCE WALKTHROUGH; COMPILE + PROVIDER TEST REQUIRED]

```
// Reference walkthrough: TX-A belongs only to AlarmManagementDb.
var decision = floodPolicy.Detect(aggregate, command.AcceptedObservations,
    command.PolicyIdentity, DecisionIdentity.Parse("decision-am-9184"));
var publicFact = AlarmFloodDetected.Create(
    PublicFactIdentity.Parse("fact-am-7716"), decision, canonicalPayloadBytes);

db.AlarmFloodDecisions.Add(decision);
db.PendingPublicFacts.Add(publicFact);           // immutable bytes + fingerprint
db.CommandOutcomes.Add(command.RequestIdentity, decision.Identity);
db.CommitIntents.Add(CommitIntentId.Parse("commit-am-5512"), decision.Identity);

await db.SaveChangesAsync(cancellationToken);    // TX-A: one local atomic set
return DetectFloodOutcome.Accepted(command.RequestIdentity, decision.Identity);
```

TX-A row role	Commits atomically	Failure if absent or separate
AlarmFloodDecision	Policy/result identity and accepted evidence linkage.	A public fact could claim a decision that is not durable.
Aggregate version	The owner state on which the decision was based.	Concurrent writers could overwrite or detach meaning.
PendingPublicFact	fact-am-7716, immutable bytes, fingerprint and causality.	The decision could commit without recoverable publication intent.
CommandOutcome	request-am-20482 to decision-am-9184 mapping.	A lost HTTP response could create a second decision on retry.
CommitIntent	commit-am-5512; commit-rc-7821	A disconnect during commit could not be

WHAT THE API MAY CLAIM

After TX-A is known committed, the API may report that AlarmManagement accepted the operation. It may not report that the fact was published, that RootCause received it, or that a RootCause case now exists.

Step 3 - Dispatch crosses the database-broker gap

The AlarmManagement dispatcher leases fact-am-7716 in a short local transaction, reads the already stored immutable payload bytes and publishes them outside that transaction. A broker confirmation may later support a second short local status update. No ordering of lease, publish, confirmation and status update removes every crash window, so recovery always keeps fact-am-7716 and the exact payload fingerprint unchanged.

Observed moment	Durable evidence	Recovery action	Meaning that must not change
Before publish	Pending fact and optional lease exist; broker evidence absent.	Expire/reclaim lease and publish stored bytes.	PublicFactIdentity and payload.
Broker accepted; local mark lost	Broker may contain the fact while producer still says pending/leased.	Reconcile or republish fact-am-7716.	Decision causality and fingerprint.
Local sent mark exists	Producer publication state is durable.	Continue from durable query/checkpoint.	No claim about RootCause commit.

EXPECTED DUPLICATE

If the dispatcher republishes after losing its local status update, the broker may deliver fact-am-7716 more than once. That repetition is a normal recovery input, not permission to mint a replacement identity or payload.

Step 4 - RootCause validates the public promise before touching Domain

RootCause Edge verifies the authenticated producer, accepted contract version, envelope limits and canonical payload. Application then resolves the complete producer fact tuple and fingerprint. A completed receipt with the same fingerprint returns its stored outcome. The same tuple with different bytes becomes IdentityConflict and is quarantined; it is never treated as a harmless duplicate.

Lookup result	Fingerprint comparison	Application path	Broker disposition
No receipt	No prior fingerprint.	Begin receipt and evaluate the RootCause operation.	Do not acknowledge yet.
Completed receipt	Matches sha256:8b2f...	Return the stable prior outcome; do not reapply Domain behavior.	Acknowledge under adapter policy.
Same producer tuple	Does not match.	IdentityConflict; retain both protected fingerprints and alert.	Reject/quarantine; never return Duplicate.
Receipt outcome unknown	Match may be known but commit classification is not.	Fresh reconciliation under commit-rc-7821.	Allow controlled redelivery; do not claim success.

Step 5 - TX-B commits the RootCause receipt, effect and outcome

RootCause applies its own policy to the accepted alarm flood. In this example it creates case-rc-3107, attaches the accepted flood as evidence, records the first hypothesis state, completes the inbox receipt and stages an optional RootCauseCaseOpened public fact. Those rows form one RootCause-owned atomic set. AlarmManagement rows and broker state are not part of TX-B.

ROOTCAUSE TX-B - [NEW][REFERENCE WALKTHROUGH; COMPILE + IDEMPOTENCY/PROVIDER TEST REQUIRED]

```
// Reference walkthrough: TX-B belongs only to RootCauseDb.
var receipt = inbox.Begin(message.ProducerFactTuple,
    message.CanonicalFingerprint,
    CommitIntentId.Parse("commit-rc-7821"));

var result = rootCauseCases.ApplyAlarmFlood(
    RootCauseCaseId.Parse("case-rc-3107"), message, acceptedPolicyIdentity);

receipt.Complete(result.StableOutcome);
db.RootCauseCases.Add(result.Case);
db.EvidenceLinks.AddRange(result.EvidenceLinks);
db.ConsumerOutcomes.Add(result.StableOutcome);
db.PendingPublicFacts.AddRange(result.PendingPublicFacts);
db.CommitIntents.Add(receipt.CommitIntentId, result.StableOutcome.Identity);

await db.SaveChangesAsync(cancellationToken); // TX-B: one local atomic set
return ConsumeOutcome.Committed(result.StableOutcome);
```

TX-B row role	Commits atomically	Failure if absent or separate
InboxReceipt	Producer tuple, canonical fingerprint and processing state.	Redelivery could create a second effect.
RootCauseCase/evidence	case-rc-3107 and the owner-private interpretation of the flood.	A receipt could say success without the owned state.
ConsumerOutcome	Stable result returned for every equivalent redelivery.	Duplicate handling could change across attempts.
Pending RootCause fact	Optional producer-owned public consequence with its own identity.	A RootCause decision could lose recoverable publication intent.
CommitIntent	commit-rc-7821 and expected outcome identity.	A lost commit response could be guessed rather than verified.

Step 6 - Acknowledgement follows the RootCause commit

Only after TX-B is known committed, or after reconciliation finds its durable receipt/outcome, may the adapter acknowledge according to the accepted broker policy. If TX-B committed but the acknowledgement was lost, redelivery resolves to the completed receipt and returns the same stable outcome. Acknowledgement loss therefore causes repeated delivery, not a second RootCause case.

THE END-TO-END RESULT

AlarmManagement has one accepted decision and one pending/public fact identity. RootCause has one receipt, one stable consumer outcome and one owner-private case effect. The broker may have delivered more than once, yet neither owner committed the other owner's truth.

Follow the same fact through four interruptions

Interruption	What may already be durable	Safe recovery under the same identities	Forbidden simplification
HTTP response lost after TX-A	Decision, command outcome, fact-am-7716 and commit-am-5512.	Retry request-am-20482; return the prior durable outcome.	Create a new decision/fact because the client saw no response.
Dispatcher crashes after broker acceptance	Broker may hold fact-am-7716; local producer status may lag.	Reconcile or republish the same stored bytes and identity.	Mint fact-am-7717 or regenerate the payload.
RootCause commits; acknowledgement is lost	Receipt, case, outcome and optional pending fact in TX-B.	Redelivery finds the same tuple/fingerprint and returns the prior outcome.	Reapply the flood and create a second case.
Connection is lost during TX-B commit	Committed versus rolled back is not yet knowable to the caller.	Fresh lookup by commit-rc-7821 and producer tuple; classify committed, rolled back or still unknown.	Treat timeout/cancellation as proof of rollback.

What the walkthrough proves - and what it deliberately does not

The walkthrough makes concrete	It does not claim
The exact AlarmManagement TX-A atomic set.	That a provider has executed or proved it.
The immutable identity/payload handoff through dispatch and delivery.	Exactly-once broker delivery or a global transaction.
The exact RootCause TX-B receipt/effect/outcome atomic set.	That AlarmManagement can observe or roll back RootCause state.
Recovery after response, status, acknowledgement and commit-result loss.	That timeout, cancellation or absence of a response proves rollback.
One business narrative containing two owner-local truths.	That the complete workflow commits atomically.

The remainder of the chapter generalizes this concrete path into reusable command, consumer, dispatcher, concurrency, retry, ambiguity, cancellation, restore and evidence rules. The example remains explanatory reference material until the net8.0 types compile and the selected provider/broker paths execute under the named fault schedules.

The command path stages meaning before one SaveChanges

Application owns the use case. It authorizes, loads the aggregate, invokes Domain behavior, allocates stable identities, stages the pending public fact and asks one unit of work to commit. Infrastructure realizes that port with the scoped DbContext. Edge maps only the final application outcome; it cannot declare success before the database boundary is classified.

ONE COMMAND, ONE OWNED COMMIT - [NEW][COMPILE + PROVIDER TEST REQUIRED]

```
// Application-owned reference flow; exact types remain to compile.
public async Task<CloseCaseOutcome> HandleAsync(
    CloseRootCauseCase command,
    CancellationToken cancellationToken)
{
    await access.AuthorizeAsync(command.Actor, command.CaseId, cancellationToken);

    var @case = await cases.LoadAsync(command.CaseId, cancellationToken);
    if (@case is null) return CloseCaseOutcome.NotFound();

    var decision = @case.Close(command.Evidence, command.PolicyIdentity);
    var publicFact = facts.CreateVerdictIssued(decision); // stable identity allocated once

    pendingFacts.Add(publicFact); // same scoped DbContext
    commitIntents.Add(command.CommitIntentId, decision.Identity);

    return await unitOfWork.CommitAsync(
        decision.Identity,
        command.CommitIntentId,
        cancellationToken);
}
```

The handler does not call a broker, remote authorization database, email service or another service inside the commit. Authorization inputs required for the decision are obtained before mutation under an accepted freshness policy; the committed audit evidence retains the actor/service/policy identity required to explain the decision without persisting reusable access tokens.

The consumer commit includes receipt, effect, outcome and public intent

A consumer first validates the authenticated producer and public envelope at Edge. Application then resolves the producer fact tuple. A previously completed receipt returns its stable outcome. A new receipt is staged beside the RootCause mutation and any new pending public fact. The broker acknowledgement belongs after the local commit; it never participates in it.

CONSUMER UNIT OF WORK - [NEW][COMPILE + IDEMPOTENCY/FAULT TEST REQUIRED]

```

public async Task<ConsumeOutcome> ConsumeAsync(
    AcceptedAlarmFloodDetected message,
    CancellationToken cancellationToken)
{
    var prior = await inbox.FindAsync(message.Fact, cancellationToken);
    if (prior is not null)
        return prior.Matches(message.CanonicalFingerprint)
            ? prior.StableOutcome
            : ConsumeOutcome.IdentityConflict();

    var receipt = inbox.Begin(message.Fact, message.CanonicalFingerprint);
    var result = await cases.ApplyFloodAsync(message, cancellationToken);

    receipt.Complete(result.StableOutcome);
    foreach (var fact in result.PendingPublicFacts)
        pendingFacts.Add(fact);

    return await unitOfWork.CommitConsumerAsync(
        receipt.CommitIntentId,
        result.DecisionIdentity,
        cancellationToken);
}

```

Consumer moment	Durable state	Broker disposition rule	Forbidden shortcut
Known duplicate	Prior fingerprint/outcome already committed.	Acknowledge according to accepted adapter policy and return stored outcome.	Re-run Domain behavior merely because delivery metadata changed.
New fact, commit known	Receipt, effect, outcome and pending facts are durable.	Acknowledge after commit.	Acknowledge before SaveChanges completes.
Commit known, acknowledgement lost	Same committed receipt/effect remains.	Redelivery resolves Duplicate.	Treat redelivery as a new case/evidence fact.
Commit outcome unknown	Fresh read/reconciliation required.	Do not acknowledge success until policy resolves; allow controlled redelivery.	Assume rollback from timeout/cancellation.
Identity changed under same producer tuple	Collision/tampering evidence retained; no business success.	Reject/quarantine under Chapter 29 policy.	Overwrite fingerprint or return Duplicate.

The dispatcher accepts the database-broker gap

A pending public fact is already part of committed owner truth. A dispatcher may lease rows in one short local transaction, publish outside that transaction, receive a broker confirmation selected in Chapter 27, then mark the local row in another short transaction. No ordering of those calls eliminates every crash window. Correctness comes from immutable stored bytes, stable public identity, bounded leasing, duplicate-safe consumers and reconciliation.

Crash window	What may be true	Required recovery	What never changes
After lease, before publish	Row is leased but broker saw nothing.	Lease expires/reclaims; publish same stored fact.	PublicFactIdentity and payload bytes.
After publish, before confirmation observed	Broker may or may not have accepted.	Retry/reconcile under same fact identity.	Producer fact meaning; duplicate is acceptable.
After confirmation, before local mark	Broker accepted; database still says pending/leased.	Republish may occur; consumers deduplicate.	No new public ID and no payload remap.
After local mark, before process checkpoint	Local sent state is durable.	Restart resumes from durable query/checkpoint.	Subscriber progress remains independent.

PUBLISHER CONFIRM BOUNDARY

A broker confirmation can support the producer's publication evidence. It is not proof that a consumer committed, and it never makes the broker part of the producer database transaction.

One SaveChanges is the normal atomic path

Microsoft documents that, when the provider supports transactions, a single SaveChanges call applies its changes transactionally. NEXUS-1 uses that default as the normal reference path: one scoped context tracks the aggregate, receipt/outcome, translation change and pending public facts, then saves once. Fewer manual transaction surfaces make retries, diagnostics and provider validation more tractable.

EF UNIT-OF-WORK CANDIDATE - [NEW][COMPILE + PROVIDER FAULT TEST REQUIRED]

```
internal sealed class EfRootCauseUnitOfWork(
    RootCauseDbContext db,
    ICommitOutcomeResolver outcomes) : IRootCauseUnitOfWork
{
    public async Task<CommitOutcome> CommitAsync(
        DecisionIdentity decision,
        CommitIntentId intent,
        CancellationToken cancellationToken)
    {
        try
        {
            await db.SaveChangesAsync(cancellationToken); // one local atomic set
            return CommitOutcome.Committed(decision);
        }
        catch (DbUpdateConcurrencyException conflict)
        {
            return CommitOutcome.Stale(conflict.Entries);
        }
        catch (Exception failure) when (outcomes.MaybeCommitAmbiguous(failure))
        {
            return CommitOutcome.Unknown(intent); // resolve from a fresh context
        }
    }
}
```

The broad Exception filter above is a boundary placeholder, not a production classifier. The selected provider/driver must supply the exact transient, timeout, cancellation, connection and commit-ambiguity rules. Unknown exceptions remain failures and retain diagnostics; they are never relabeled Duplicate, ConcurrencyConflict or RolledBack for convenience.

Manual transactions require a new accepted proof

Reason proposed	Default decision	What must be proved before acceptance
Two SaveChanges calls	Refactor to one tracked commit where possible.	Why one call cannot express the invariant; failure at every boundary; retry strategy and durable final state.
Read after an intermediate write	Prefer in-memory values or one provider command path.	Exact isolation, savepoint, lock duration and retry behavior on the supported provider.
Bulk/raw SQL plus EF	Keep one connection/transaction only inside the same owner database.	Every command enlists correctly; model/tracker coherence and rollback are tested.
Two DbContexts	Reject by default; use one context for the local unit.	Same relational connection/DbTransaction, ownership justification and provider-specific evidence.
Second service/database/broker	Reject.	No local test can convert another owner/resource into one atomic local truth; redesign for facts/idempotency/compensation.

When SaveChanges runs inside an existing transaction, EF may create a savepoint and roll back to it on failure. That behavior is not portable permission to retry arbitrary business decisions in place. SQL Server MARS disables EF savepoints, and provider execution strategies interact with manually controlled transactions. Any multi-save design therefore pins provider, connection settings, retry strategy, savepoint behavior and failure schedule.

Optimistic concurrency detects stale state; policy resolves it

A concurrency token adds the original token value to the update/delete predicate. Zero affected rows cause `DbUpdateConcurrencyException`. That mechanism detects a stale write; it does not decide whether the operation may be retried, merged, rejected or converted to a stable prior result. `RootCause` Domain/Application must make that semantic decision after reloading current owned state.

PROVIDER-NEUTRAL TOKEN SHAPE - [NEW][COMPILE + PROVIDER TEST REQUIRED]

```
internal sealed class RootCauseCaseRow
{
    public Guid CaseId { get; init; }
    public required string State { get; set; }
    public Guid ConcurrencyVersion { get; set; }
}

internal sealed class RootCauseCaseRowConfiguration
    : IEntityTypeConfiguration<RootCauseCaseRow>
{
    public void Configure(EntityTypeBuilder<RootCauseCaseRow> entity)
    {
        entity.HasKey(x => x.CaseId);
        entity.Property(x => x.ConcurrencyVersion).IsConcurrencyToken();
    }
}

// Application-managed reference token; regenerate only for protected meaning.
row.ConcurrencyVersion = concurrencyVersions.Next();
```

The provider may later replace the application-managed GUID with a supported native token, but that is an explicit migration and compatibility decision. SQL Server rowversion is provider-specific; other providers expose different primitives or none. The public contract never carries the private concurrency token.

A stale write is replayed from meaning, not patched from memory

1. Roll back or dispose the failed unit of work; never continue with a context whose transaction or tracked graph is uncertain.
2. Create a fresh scoped context and reload the complete owned aggregate/evidence graph required by the decision.
3. Revalidate authorization, policy identity, lifecycle state and the original business inputs under the accepted retry policy.
4. Reapply the Domain operation deterministically with the same source/request/decision identities; do not mint new meaning merely because the first attempt conflicted.
5. Commit once. Bound attempts and return `Conflict`, `Abstained`, `Duplicate`, `Committed` or `CommitOutcomeUnknown` explicitly.

NO LAST-WRITER WINS

Copying proposed scalar values over the freshly loaded row can bypass `RootCause` invariants. An automatic merge is allowed only when a named domain policy proves the combined meaning and its evidence exercises the race.

Concurrency and uniqueness are different failure classes

Observed database signal	Likely class	Required semantic check	Stable outcome
<code>DbUpdateConcurrencyException</code> on case update/delete	Stale aggregate/version.	Reload full graph; reapply same authorized operation.	Committed on retry, <code>Conflict</code> , <code>Abstained</code> or prior stable result.
Provider error for inbox producer tuple	Concurrent first receipt or duplicate.	Load row by exact accepted constraint tuple; compare fingerprint and stored outcome.	Duplicate only if identity/content match; otherwise <code>IdentityConflict</code> .
Provider error for <code>DecisionIdentity</code>	Concurrent equivalent decision or defect.	Compare policy/input identities and defect.	Return prior result if equivalent.

Observed database signal	Likely class	Required semantic check	Stable outcome
Provider error for external translation tuple	Concurrent first mapping.	Reload winner, scope and mapping-policy identity.	Continue only if meaning is equivalent; otherwise IdentityConflict.
Provider error for unrelated constraint	Data/model defect or different conflict.	Keep exact provider diagnostics; no duplicate inference.	Failure/quarantine and alert according to owner policy.

Unique-conflict classification is provider-pinned and narrow

EF documents that inserts which violate uniqueness generally raise a provider-specific exception rather than `DbUpdateConcurrencyException`. A catch-all `DbUpdateException-to-Duplicate` mapping would therefore hide schema defects, nullability failures, foreign-key defects and unrelated unique constraints. Infrastructure classifies only reviewed provider codes and exact constraint identities; Application verifies the business tuple and content.

NARROW UNIQUE CLASSIFIER - [NEW][PROVIDER CODE + CONSTRAINT TEST REQUIRED]

```
public async Task<CommitOutcome> CommitConsumerAsync(
    CommitIntentId intent,
    CancellationTokentoken cancellationToken)
{
    try
    {
        await db.SaveChangesAsync(cancellationToken);
        return CommitOutcome.Committed(intent);
    }
    catch (DbUpdateException error)
        when (constraints.Is(error, RootCauseConstraints.InboxProducerFact))
    {
        // Fresh no-tracking read; the constraint signal alone is insufficient.
        var prior = await receipts.LoadByProducerFactFreshAsync(intent, cancellationToken);
        return prior is not null && prior.IsEquivalentTo(intent)
            ? CommitOutcome.Duplicate(prior.StableOutcome)
            : CommitOutcome.IdentityConflict();
    }
}
```

Classifier input	Must be pinned	Must never be inferred from
Provider/driver identity	Package/version and supported engine family.	Exception message substring alone.
Error identity	SQL state/error number and nested exception shape.	<code>DbUpdateException</code> base type.
Constraint identity	Reviewed migration/schema name or provider-supported metadata.	Current application operation name.
Post-conflict read	Exact scoped unique tuple, fingerprint, lifecycle and outcome.	The failed tracked entity or delivery attempt.
Unknown error	Failure category, diagnostics protection and alert path.	Optimistic assumptions about duplicate/retryability.

Isolation is a provider behavior, not a universal label

Concurrency tokens protect selected rows but do not automatically protect predicates such as no open case exists for this accepted correlation scope. Unique constraints, deterministic keys, provider isolation or an explicit coordination row may protect that invariant. The chosen mechanism must be tied to one owner and exercised under the selected provider's lock/MVCC behavior.

Protection need	Reference mechanism	Evidence required
Existing aggregate stale update	Concurrency token in update/delete predicate.	Two-writer schedule and zero-row classification.
Concurrent first insert	Exact unique constraint plus semantic reload.	Parallel insert, provider error identity and final-state comparison.
Predicate/range invariant	Deterministic key/unique row first; otherwise sele	Phantom/write-skew schedule lock/serialization

Protection need	Reference mechanism	Evidence required
Administrative merge/split	Versioned local workflow and explicit authorization.	Race with intake and retained history.
Long human/external work	No open transaction; persist intent/version and revalidate later.	Timeout, stale approval and changed-state scenarios.

Higher isolation is considered only for short, immediate local work. It never spans a remote call, broker publish, user think time, operator approval or backoff delay. Lock waits, deadlocks and serialization failures remain normal failure inputs with bounded retries and observable budgets.

Retries repeat a whole semantic unit with stable identities

A provider execution strategy may retry transient database work. When an explicit transaction is required, Microsoft directs the application to execute the complete transaction as a retrievable delegate. NEXUS-1 adds a stricter domain rule: the delegate must repeat the same semantic operation under the same request, source, decision, public-fact and commit-intent identities. A retry cannot mint a second fact or silently reevaluate under a different policy snapshot.

Retry input	Stable across attempts	May be refreshed	Must stop retry
Command	Request/idempotency key, actor intent, decision identity inputs, commit intent.	Current aggregate state and authorization evidence under declared policy.	Meaning changed, authorization denied, lifecycle closed, budget exhausted.
Consumer	Producer fact tuple, fingerprint, mapping policy and public causality.	Current local aggregate/inbox/translation state.	Identity collision, unsupported contract, poison data, semantic conflict.
Dispatcher	PublicFactIdentity and stored immutable payload bytes.	Lease/attempt/connection state.	Payload mismatch, expired policy, terminal local state or operator hold.
Projection	Source identity/position and projection contract identity.	Current checkpoint/projection state.	Gap/collision/schema incompatibility or rebuild policy.

FRESH STATE, SAME MEANING

Reloading current state is required after a conflict. Changing the command, policy identity or public fact merely to obtain a successful commit is a new operation and needs a new explicit decision.

Commit ambiguity is not an exception detail

If a connection is lost while the database is committing, Microsoft documents that the transaction outcome is unknown. Blind retry can duplicate an insert or corrupt meaning when the first commit actually succeeded. NEXUS-1 therefore gives each selected operation a stable `CommitIntentId` or an equally strong business/receipt identity that can be verified from a fresh context.

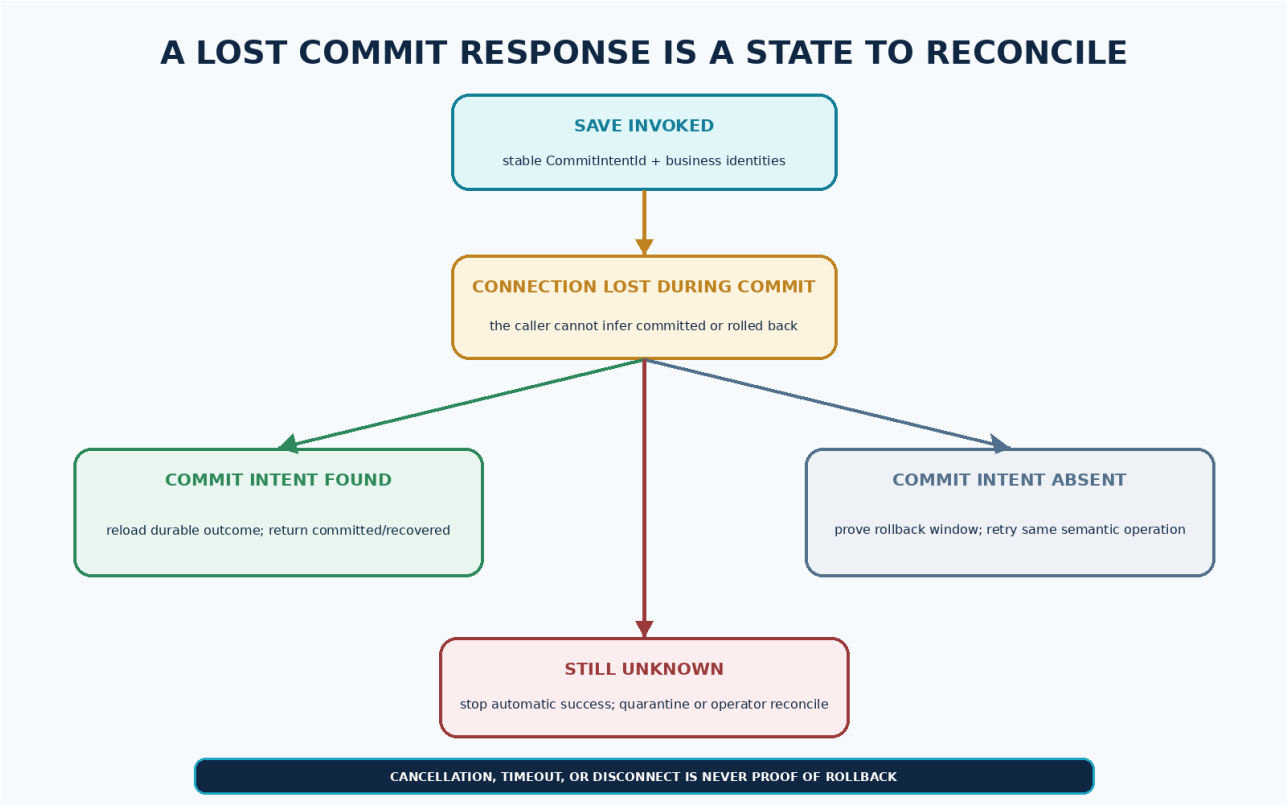


Figure 18.3 - A timeout or disconnect during commit routes to reconciliation, never directly to success or rollback.

Reconciliation result	Database evidence	Application outcome	Next action
Committed	Commit intent and expected business/result identities exist and match.	CommittedRecovered with durable outcome.	Return/ack consistently; continue dispatch asynchronously.
Rolled back	Intent/effect absent under a provider-proved visibility and retry rule.	Retryable with the same semantic identities.	Create fresh context and rerun the complete unit within budget.
Conflicting	Intent exists but result/fingerprint/scope differs.	IdentityConflict or integrity failure.	Quarantine, alert and preserve both diagnostics.
Still unknown	Verification failed or visibility/restore state is insufficient.	CommitOutcomeUnknown.	Stop automatic success; retry verification or require operator reconciliation.

Cancellation and timeout are phase-sensitive

Cancellation is a request to stop work, not evidence that the database rolled back. Before `SaveChanges` begins, cancellation can normally exit without durable mutation. During command execution or commit, the provider may observe cancellation at a point where outcome still needs classification. After a commit is known, a late cancellation cannot turn success into rollback or erase the pending public fact.

Phase	Cancellation/timeout meaning	Required response
Before mutation/staging	No database command started.	Cancel and dispose the scope; no durable outcome claimed.
After staging, before <code>SaveChanges</code>	Only tracked memory changed.	Cancel; dispose context; confirm no external side effect was called.
During <code>SaveChanges</code> before commit known	Provider operation failed/cancelled; outcome may require rollback or ambiguity classification.	Use exact provider signal; fresh reconciliation where commit may have started.
Connection lost during commit	Committed versus rolled back is unknown.	Resolve by durable intent/business identity; never infer from timeout.
After commit known, response/ack cancelled	Owned truth is durable; transport completion is incomplete.	Return/recover using stable operation identity; redelivery/retry must remain idempotent.
Dispatcher publish cancelled	Broker acceptance may be unknown.	Retry same public fact and reconcile broker/local status under Chapter 27 policy.

The failure classifier must preserve uncertainty

Failure	Safe classification only when	Unsafe claim	Blocking evidence
Validation/authorization	No write began and owner policy returned a stable rejection.	Database rollback proves authorization.	Negative application/component tests.
Optimistic concurrency	Exact token update/delete produced zero affected rows.	Every <code>DbUpdateException</code> is concurrency.	Provider SQL/result and two-writer schedule.
Unique conflict	Exact reviewed constraint/provider identity matched and durable tuple was reloaded.	Every unique error means Duplicate.	Constraint matrix and mismatch tests.
Deadlock/serialization/transient	Selected provider marks it retryable and the whole unit is replay-safe.	Retry individual SQL/ <code>SaveChanges</code> blindly.	Fault injection, stable IDs and final-state comparison.
Commit disconnect/timeout	Fresh verification proves commit or rollback.	Timeout means failed or succeeded.	Commit-boundary connection fault and reconciliation report.
Unknown	No narrower accepted class applies.	Map to conflict/duplicate for convenience.	Protected diagnostics, alert and review.

Observability follows the commit state machine

- Counters separate attempts, known commits, known rollbacks, stale-token conflicts, exact unique conflicts, semantic mismatches and `CommitOutcomeUnknown`.
- Histograms record database command/commit duration, lock wait, retry count, reconciliation latency and dispatcher gap age under provider and operation identities.
- Logs retain protected `CommitIntentId`/public fact hashes and exact constraint/error identity without exposing private keys, payload PII or access tokens.
- Traces may connect attempts and reconciliation, but sampling or trace loss cannot change durable commit classification.
- Alerts bind unknown outcomes, repeated semantic conflicts, retry exhaustion and pending-public-fact age to runbook/controller actions.

Restore and replay preserve commit identities

A database restore may return the service to a point before a receipt, commit intent or dispatcher status existed while the broker/source can still replay later history. The Chapter 16 restore gate and Chapter 17 identity graph therefore protect transaction reconciliation: pending facts keep their public IDs, inbox outcomes keep producer tuples/fingerprints, commit intents keep business result identities and concurrency/translation state is reconciled before traffic resumes.

Restored surface	Must remain stable	Reconciliation proof
Command intent/outcome	Request/decision/commit identity and committed result where retained.	Same retry discovers or recreates one accepted owner effect.
Consumer receipt	Producer tuple, fingerprint, stable outcome and local effect link.	Redelivery after restore yields Duplicate or one reconstructed effect.
Pending public fact	PublicFactIdentity, immutable bytes, causality and dispatch state.	Republish uses the same identity; no new history is minted.
Concurrency/translation	Current version, mapping policy and lifecycle history required by retained events.	Replay cannot silently remap or bypass current invariant state.
Dispatcher status	Lease/attempt/confirmation evidence and reconciliation window.	Pending versus published uncertainty is resolved without claiming subscriber commit.

Test the commit boundary under adversarial schedules

Scenario	Required result	Evidence retained
Failure before SaveChanges	No durable rows/effect/public intent.	Before/after state and injected point identity.
Failure within provider command batch	All local rows or none under one SaveChanges.	Provider transaction log/result and durable graph comparison.
Two stale case writers	One commits; loser reloads and re-evaluates; no last-writer overwrite.	Two-writer barrier schedule, tokens and final aggregate truth.
Concurrent same inbox insert	One receipt/effect; equivalent loser returns prior outcome.	Exact constraint identity and sequential/parallel/restart result.
Same producer tuple, changed payload	IdentityConflict; no overwrite or business success.	Both fingerprints protected and alert evidence.
Concurrent same DecisionIdentity	One result/public fact; mismatch blocks.	Policy/input comparison and pending-fact identities.
Deadlock/serialization failure	Whole semantic unit retries with stable IDs inside budget.	Attempt graph, provider code and final-state equivalence.
Connection loss during commit	Known committed/rolled back or explicit unknown after verification.	Commit intent, fresh-read result and operator path.
Commit then HTTP response loss	Client retry observes one result.	Request/commit identity and API retry fixture.
Consumer commit then ack loss	Redelivery returns Duplicate; one local effect/public fact.	Inbox outcome and broker delivery sequence.
Publish then dispatcher crash	Same public fact may republish; consumer remains one effect.	Stored bytes/ID, confirms, attempts and inbox result.
Restore before receipt/status	Replay/reconcile preserves one identity graph.	Backup identity, replay inputs and post-restore state.

Local transaction evidence gates

Evidence ID	Blocking rule	Required artifact
TXN-OWN-001	Every commit names one service/database owner and one semantic operation.	Commit-boundary catalogue and architecture review.
TXN-CMD-001	Command mutation and pending public intent commit together before success.	Command component test with injected failure points.
TXN-CNS-001	Receipt, stable outcome, owned effect and pending public fact commit atomically.	Consumer duplicate/failure/restart matrix.
TXN-DSP-001	Dispatcher never holds the database transaction across broker publish and preserves the same fact through gaps.	Lease/publish/confirm/crash schedule linked to FLOW evidence.
TXN-SAVE-001	Normal mutation path uses one provider-supported SaveChanges transaction.	Generated SQL/transaction trace and all-or-none state comparison.
TXN-CON-001	Stale-token conflicts reload and reapply policy; no last-writer merge.	Two-writer schedules and invariant/final-state results.
TXN-UNQ-001	Only exact provider/constraint identities enter duplicate/first-winner handling.	Provider error/constraint classifier and negative matrix.
TXN-ISO-001	Isolation/locking protects declared predicates without remote or long-lived work.	Provider lock/MVCC/write-skew/deadlock report.
TXN-RETRY-001	Retries replay the whole semantic unit with stable identities and a bound.	Transient/deadlock/serialization attempt graph and state equivalence.
TXN-AMB-001	Commit disconnect/timeout becomes unknown until a fresh durable verification resolves it.	Commit-boundary fault injection and intent reconciliation report.
TXN-CAN-001	Cancellation phase cannot turn a known commit into rollback or hide ambiguity.	Before/during/after cancellation matrix.
TXN-SP-001	Manual multi-save/savepoint/cross-context paths require separate provider proof.	Accepted exception ADR, settings and failure schedule; otherwise absence scan.
TXN-REST-001	Restore/replay preserves commit, receipt, pending fact and mapping identities.	Restore-replay-reconcile rehearsal linked to MIG-REST-001.
TXN-OBS-001	Every attempt/outcome is observable without leaking private IDs or tokens.	Redacted logs/metrics/traces/alerts and runbook exercise.
TXN-TEST-001	Exact net8/provider paths execute all adversarial schedules.	Machine results bound to build, migration, provider and fault identities.
TXN-REVIEW-001	Owners, data, architecture, reliability, security and operations accept residual risk.	Signed evidence index, exceptions and next review trigger.

ADR-016 - One semantic operation per local commit

ADR field	ADR-016 record
Status	Accepted for the NEXUS-1 reference design; provider implementation and execution evidence remain pending.
Context	Service extraction replaces one process transaction with independent owner commits and unavoidable delivery/response gaps.
Decision	Each command/consumer commits one owner-local semantic operation in one short transaction, normally one SaveChanges. Receipt/outcome, owned mutation and pending public intent are atomic. Concurrency triggers fresh semantic replay; exact unique constraints choose first winners; retries preserve identities; ambiguous commits reconcile through durable evidence. Dispatcher publication remains outside the database transaction.
Consequences	The design accepts duplicate delivery, asynchronous publication, explicit unknown outcomes, provider-pinned classifiers, retry budgets and reconciliation/runbook work. Local invariants become testable without DTC.
Rejected	Cross-service/database/broker transaction, broker call inside DB transaction, acknowledge-before-commit, sent-before-publish, last-writer wins, catch-all Duplicate, blind SaveChanges retry, timeout-means-rollback, new fact ID per attempt and unbounded high-isolation transactions.
Review trigger	Provider/driver/version, isolation/token/constraint, retry strategy, commit-intent design, SaveChanges count, broker confirm, acknowledgement, dispatcher lease, restore or public identity semantics change.

Risk register for local commits and concurrency

Risk	Early signal	Blocking control
Partial local effect	Receipt/case/verdict/pending fact counts disagree.	TXN-CMD/CNS/SAVE atomic failure matrix.
Acknowledged before commit	Broker offset advances without durable inbox outcome.	Adapter ordering test and redelivery exercise.
Broker inside transaction	Long lock/transaction duration follows broker latency.	TXN-DSP-001 boundary scan and outage schedule.
Blind retry after commit loss	Duplicate rows/facts or incompatible unique failure.	TXN-AMB-001 intent verification and stable identities.
Last-writer wins	Stale handler overwrites newer case/policy state.	TXN-CON-001 fresh replay and negative merge tests.
Broad duplicate catch	Constraint/model defects return success.	TXN-UNQ-001 exact provider/constraint classifier.
Retry mints meaning	Same operation appears under multiple public/decision IDs.	TXN-RETRY/ID-FACT stable identity assertions.
Long/high-isolation transaction	Lock waits, deadlocks, pool starvation and latency spikes.	TXN-ISO-001 short-unit budgets and provider schedules.
Cancellation lies	Client sees cancelled while durable mutation exists and cannot resolve it.	TXN-CAN/TXN-AMB phase and reconciliation contract.
Dispatcher says sent too early	Local status is terminal before broker evidence.	Chapter 27 state machine constrained by TXN-DSP-001.
Restore creates second history	Replayed receipt/pending fact receives new identity.	TXN-REST/ID-REST/MIG-REST rehearsal.
Trace becomes transaction key	Sampling/loss changes dedup or reconciliation.	Business/commit identities durable; TXN-OBS/ID-TRACE tests.

Official-source validation

The EF Core mechanics were checked against current Microsoft documentation on 19 July 2026. The stable points used here are that a provider-supported single `SaveChanges` is transactional; manual transactions interact with retrying execution strategies; `SaveChanges` inside an existing transaction may use savepoints; SQL Server MARS disables those savepoints; optimistic tokens add the original token to update/delete predicates; zero rows cause `DbUpdateConcurrencyException`; insert uniqueness is provider-specific; and a connection loss during commit can leave outcome unknown.

Official source	Stable point used here	NEXUS-1 boundary
Microsoft Learn - EF Core transactions	One <code>SaveChanges</code> uses a transaction when supported; manual transactions/savepoints/provider behavior have explicit limits.	Normal command/consumer path saves once; multi-save/cross-context paths require separate provider evidence.
Microsoft Learn - EF Core concurrency	Concurrency tokens protect update/delete predicates; zero rows raise <code>DbUpdateConcurrencyException</code> ; insert uniqueness is provider-specific.	Tokens trigger semantic replay; exact constraint classifiers handle concurrent inserts.
Microsoft Learn - EF Core connection resiliency	Retry explicit transactions requires replaying the unit; commit connection loss can be unknown; verification/idempotency is required.	Stable identities and <code>CommitIntentId</code> reconcile whole-unit retries without minting new meaning.

Sources: <https://learn.microsoft.com/en-us/ef/core/saving/transactions> ; <https://learn.microsoft.com/en-us/ef/core/saving/concurrency> ; <https://learn.microsoft.com/en-us/ef/core/miscellaneous/connection-resiliency>

Technical material still required

The owner boundaries, three commit lanes, one-`SaveChanges` default, consumer atomic set, dispatcher gap, token replay, unique classification, isolation, retry, commit ambiguity, cancellation, restore, evidence IDs and ADR are specified. No provider has executed the reference paths; no failure has been injected during commit; no constraint classifier, retry strategy, commit-intent verifier, acknowledgement or dispatcher path has run.

TECHNICAL MATERIAL STATUS - [LOCAL TRANSACTION/CONCURRENCY/AMBIGUOUS-COMMIT PROVIDER EXECUTION EVIDENCE REQUIRED]

Implement the net8.0 command, consumer and dispatcher unit-of-work surfaces in AlarmManagementDb and RootCauseDb; pin compatible EF Core/provider/driver/engine versions, concurrency tokens, exact constraints, isolation and retry settings; persist stable commit, receipt, decision and public-fact identities; execute all-or-none, stale-writer, concurrent insert, changed-payload, deadlock/serialization, cancellation, commit-disconnect, response/ack loss, publish-crash, restart and restore schedules; publish TXN-OWN-001 through TXN-REVIEW-001 linked to ALMDB-TXN-001, RCDB-TXN-001, ID, FLOW and MIG evidence. RootCause remains G0 accepted and G1 specified; runtime evidence has not advanced.

Deliverable	Minimum technical content	Blocking evidence
Command unit	Authorization, aggregate decision, stable IDs, pending fact and one commit.	TXN-CMD-001 / TXN-SAVE-001 / ALMDB/RCDB-TXN-001
Consumer unit	Receipt/fingerprint/outcome, owned effect and pending fact in one commit.	TXN-CNS-001 / ID-IDEM-001
Dispatcher boundary	Short local lease/status commits around external publish; immutable stored fact.	TXN-DSP-001 / FLOW evidence
Concurrency/uniqueness	Provider token plus exact constraint classifier and semantic replay.	TXN-CON-001 / TXN-UNQ-001
Isolation/retry	Pinned provider behavior, short transaction and whole-unit bounded retry.	TXN-ISO-001 / TXN-RETRY-001
Ambiguity/cancellation	Commit intent, fresh verification and phase-aware outcome contract.	TXN-AMB-001 / TXN-CAN-001
Recovery/operations	Restore/replay reconciliation, metrics, alerts and operator path.	TXN-REST-001 / TXN-OBS-001
Acceptance	Machine results bound to build/provider/schema/fault identities and review.	TXN-TEST-001 / TXN-REVIEW-001

Implementation sequence

1. Accept ADR-016 and publish the owner/operation/atomic-row/identity catalogue for command, consumer, dispatcher and projection commits.
2. Pin the net8.0-compatible EF Core runtime/design/tool, provider/driver and engine versions plus connection, MARS, isolation and retry settings.
3. Implement one scoped DbContext and one SaveChanges normal path for AlarmManagement and RootCause command/consumer units.
4. Add stable CommitIntentId, request/source/decision/public-fact identities and durable outcome lookup without exposing private keys.
5. Configure provider-supported concurrency tokens and exact unique constraints from Chapters 14, 15 and 17.
6. Implement a narrow provider error/constraint classifier; unknown errors remain failures.
7. Implement bounded fresh-context semantic replay for stale tokens and concurrent first inserts.
8. Implement whole-unit execution-strategy retries and commit-ambiguity verification with stable identities.
9. Keep broker acknowledgement after consumer commit and dispatcher publish outside every database transaction; preserve crash gaps for Chapters 26-29.
10. Execute the adversarial matrix on the real provider, then migration/restore/replay and controller-observed fault paths; publish TXN evidence without advancing unproved gates.

CHAPTER 18 - CODE AUDIT

Every commit names one owner and one semantic operation

The executable harness makes each atomic set enumerable before code runs. Command, consumer and dispatcher lanes have different rows and external gaps, but every lane preserves stable identities and one owner's durable meaning.

EVIDENCE STATUS

These assets are compile-oriented reference material. No EF provider, engine, migration, broker, fault injector or executed component suite is present here. TXN, ALMDB-TXN and RCDB-TXN evidence remains PENDING.

EXECUTABLE ASSET 18-A - COMMIT-BOUNDARY CATALOGUE

```
public enum CommitLane { Command, Consumer, Dispatcher, Projection }

public sealed record CommitBoundary(
    string Owner,
    string Database,
    string Operation,
    CommitLane Lane,
    IReadOnlySet<string> AtomicTables,
    IReadOnlySet<string> StableIdentities,
    int MaximumSaveChangesCalls,
    TimeSpan MaximumDuration);

public sealed class CommitBoundaryCatalogue
{
    private readonly IReadOnlyDictionary<string, CommitBoundary> _operations;

    public CommitBoundaryCatalogue(IEnumerable<CommitBoundary> boundaries)
    {
        var items = boundaries.ToArray();
        if (items.GroupBy(x => x.Operation, StringComparer.Ordinal)
            .Any(x => x.Count() != 1))
            throw new DuplicateCommitOperation();
        if (items.Any(x => x.AtomicTables.Count == 0
            || x.StableIdentities.Count == 0
            || x.MaximumSaveChangesCalls != 1
            || x.MaximumDuration <= TimeSpan.Zero))
            throw new InvalidCommitBoundary();
        _operations = items.ToDictionary(
            x => x.Operation, StringComparer.Ordinal);
    }

    public CommitBoundary Demand(
        string operation, string owner, string database)
    {
        if (!_operations.TryGetValue(operation, out var boundary))
            || !StringComparer.Ordinal.Equals(boundary.Owner, owner)
            || !StringComparer.Ordinal.Equals(boundary.Database, database))
            throw new ForeignCommitBoundary(operation);
        return boundary;
    }
}
```

EXECUTABLE ASSET 18-B - OWNERSHIP AND LANE GATES

```
[Fact]
public void TXN_OWN_001_RootCause_consumer_names_its_atomic_set()
{
    var boundary = fixture.Catalogue.Demand(
        "rootcause.accept-alarm-flood",
        "RootCause", "RootCauseDb");

    boundary.Lane.Should().Be(CommitLane.Consumer);
    boundary.AtomicTables.Should().BeEquivalentTo(
        "InboundPublicFacts", "RootCauseCases",
        "Evidence", "ConsumerOutcomes", "PendingPublicFacts",
        "CommitIntents");
    boundary.MaximumSaveChangesCalls.Should().Be(1);
}

[Fact]
public void RootCause_operation_cannot_include_AlarmManagementDb()
{
    var act = () => fixture.Catalogue.Demand(
        "rootcause.accept-alarm-flood",
        "RootCause", "AlarmManagementDb");
    act.Should().Throw<ForeignCommitBoundary>();
}
```

COMMAND ATOMICITY

Decision, intent and public fact succeed or fail together

EXECUTABLE ASSET 18-C - ONE SAVECHANGES COMMAND PATH

```
public sealed class EvaluateAlarmFloodHandler(
    AlarmManagementDbContext db,
    IAlarmFloodPolicy policy,
    IAlarmFloodFactWriter facts,
    ICommitFaultProbe faults)
{
    public async Task<AlarmDecisionOutcome> HandleAsync(
        EvaluateAlarmFlood command,
        CancellationToken cancellationToken)
    {
        var prior = await db.CommitIntents
            .Where(x => x.CommitIntentId == command.CommitIntentId)
            .Select(x => x.Outcome)
            .SingleOrDefaultAsync(cancellationToken);
        if (prior is not null) return prior;

        var decision = policy.Evaluate(command.Observations);
        var pending = facts.Create(
            decision, command.PublicFactIdentity, command.Causality);
        var outcome = AlarmDecisionOutcome.Accepted(
            decision.Identity, pending.Identity);

        db.AlarmFloodDecisions.Add(decision);
        db.PendingPublicFacts.Add(pending);
        db.CommitIntents.Add(CommitIntentRow.Accepted(
            command.CommitIntentId, outcome));

        faults.Reach(FaultPoint.BeforeSaveChanges);
        await db.SaveChangesAsync(cancellationToken);
        return outcome;
    }
}
```

EXECUTABLE ASSET 18-D - ALL-OR-NONE COMMAND TESTS

```
[Theory]
[InlineData(FaultPoint.BeforeSaveChanges)]
[InlineData(FaultPoint.DuringFirstProviderCommand)]
[InlineData(FaultPoint.DuringLastProviderCommand)]
public async Task TXN_CMD_001_failure_leaves_no_partial_atomic_set(
    FaultPoint point)
{
    fixture.Faults.ThrowAt(point);
    var act = () => fixture.ExecuteCommandAsync();
    await act.Should().ThrowAsync<InjectedCommitFailure>();

    var state = await fixture.ReadCommandAtomicSetAsync();
    state.Decisions.Should().BeEmpty();
    state.PendingFacts.Should().BeEmpty();
    state.CommitIntents.Should().BeEmpty();
}

[Fact]
public async Task TXN_SAVE_001_normal_command_saves_once()
{
    await fixture.ExecuteCommandAsync();
    fixture.SaveChangesCalls.Should().Be(1);
    (await fixture.ReadCommandAtomicSetAsync()).Should()
        .BeEquivalentTo(fixture.ExpectedCommittedSet);
}

[Fact]
public async Task response_loss_retry_returns_same_outcome()
{
    var first = await fixture.CommitThenLoseHttpResponseAsync();
    var retry = await fixture.RetrySameCommandAsync();
    retry.Should().Be(first.DurableOutcome);
    fixture.PublicFactIdAllocations.Should().Be(1);
}
```

SUCCESS FOLLOWS THE DATABASE

The handler returns success only after the single SaveChanges completes. A lost HTTP response is resolved by CommitIntentId, not by repeating identity allocation.

CONSUMER ATOMICITY

Receipt, owned effect and pending fact precede acknowledgement

EXECUTABLE ASSET 18-E - ROOTCAUSE CONSUMER UNIT

```
public sealed class AcceptAlarmFloodConsumer(
    RootCauseDbContext db,
    IRootCausePolicy policy,
    IRootCauseFactWriter facts)
{
    public async Task<ConsumerOutcome> HandleAsync(
        AcceptedPublicFact<AlarmFloodDetectedV1> input,
        CancellationToken cancellationToken)
    {
        var prior = await db.InboundPublicFacts
            .Where(x => x.Producer == input.Fact.Producer.Value
                && x.PublicFactId == input.Fact.FactId
                && x.ContractType == input.Fact.ContractType
                && x.ContractVersion == input.Fact.ContractVersion)
            .Select(x => x.StableOutcome)
            .SingleOrDefaultAsync(cancellationToken);
        if (prior is not null)
            return ConsumerOutcome.Duplicate(prior);

        var result = policy.Apply(input);
        db.InboundPublicFacts.Add(
            InboundPublicFactRow.Accepted(input, result));
        db.RootCauseCases.Update(result.Case);
        db.ConsumerOutcomes.Add(result.StableOutcome);
        foreach (var pending in facts.CreateEligible(result))
            db.PendingPublicFacts.Add(pending);
        db.CommitIntents.Add(CommitIntentRow.Accepted(
            result.CommitIntentId, result.StableOutcome));

        await db.SaveChangesAsync(cancellationToken);
        return ConsumerOutcome.Committed(result.StableOutcome);
    }

    public async Task ConsumeAsync(
        BrokerDelivery delivery,
        CancellationToken cancellationToken)
    {
        await handler.HandleAsync(delivery.Fact, cancellationToken);
        await delivery.AcknowledgeAsync(cancellationToken);
    }
}
```

EXECUTABLE ASSET 18-F - COMMIT/ACK GAP TESTS

```
[Fact]
public async Task TXN_CNS_001_commit_failure_never_acknowledges()
{
    fixture.Faults.ThrowAt(FaultPoint.DuringLastProviderCommand);
    var act = () => fixture.ConsumeAsync();
    await act.Should().ThrowAsync<InjectedCommitFailure>();
    fixture.BrokerAcknowledgements.Should().Be(0);
    (await fixture.ReadConsumerAtomicSetAsync()).Should().BeEmpty();
}

[Fact]
public async Task commit_then_ack_loss_redelivers_one_effect()
{
    await fixture.ConsumeCommitThenLoseAckAsync();
    var retry = await fixture.RedeliverSameFactAsync();

    retry.Kind.Should().Be(ConsumerOutcomeKind.Duplicate);
    fixture.RootCauseEffects.Should().ContainSingle();
    fixture.PendingFacts.Should().ContainSingle();
    fixture.BrokerAcknowledgements.Should().Be(1);
}
```

ACKNOWLEDGEMENT IS OUTSIDE THE COMMIT

A local commit cannot include the broker. The durable receipt makes lost acknowledgement safe by turning redelivery into the same stored outcome.

OPTIMISTIC CONCURRENCY

A stale writer reloads truth and replays policy in a fresh scope

EXECUTABLE ASSET 18-G - BOUNDED SEMANTIC REPLAY

```
public sealed class OptimisticSemanticReplayer(
    IServiceScopeFactory scopes,
    ConcurrencyReplayOptions options)
{
    public async Task<CommandOutcome> ExecuteAsync(
        RootCauseCommand command,
        CancellationToken cancellationToken)
    {
        for (var attempt = 1; attempt <= options.MaximumAttempts; attempt++)
        {
            await using var scope = scopes.CreateAsyncScope();
            var handler = scope.ServiceProvider
                .GetRequiredService<RootCauseCommandHandler>();
            try
            {
                return await handler.HandleAsync(
                    command, cancellationToken);
            }
            catch (DbUpdateConcurrencyException)
                when (attempt < options.MaximumAttempts)
            {
                // The failed scope is disposed before the next iteration.
                // The stable command identities and accepted inputs are reused.
            }
        }
        return CommandOutcome.Conflict(
            command.CommitIntentId, "concurrency-replay-exhausted");
    }
}
```

EXECUTABLE ASSET 18-H - TWO-WRITER SCHEDULES

```
[Fact]
public async Task TXN_CON_001_stale_writer_never_overwrites_winner()
{
    var barrier = new Barrier(2);
    var outcomes = await Task.WhenAll(
        fixture.CloseCaseAsync(
            "evidence-set-A", barrier, fixture.IntentA),
        fixture.AddEvidenceAsync(
            "evidence-E9", barrier, fixture.IntentB));

    var durable = await fixture.ReadCaseInFreshContextAsync();
    durable.Should().Satisfy(caseState =>
        fixture.Policy.Accepts(
            caseState, fixture.AcceptedOperationOrder));
    fixture.LastWriterWinsMerges.Should().Be(0);
    outcomes.Should().OnlyContain(x =>
        x.Kind is CommandOutcomeKind.Committed
        or CommandOutcomeKind.RejectedByCurrentState
        or CommandOutcomeKind.Conflict);
}

[Fact]
public async Task replay_uses_same_commit_and_public_fact_identities()
{
    await fixture.ForceOneConcurrencyReplayAsync();
    fixture.CommitIntentIds.Distinct().Should().ContainSingle();
    fixture.PublicFactIds.Distinct().Should().ContainSingle();
}
```

RELOAD IS NOT MERGE

The loser discards tracked state, reloads the whole required graph and asks Domain policy again. Property-by-property last-writer merging is not a concurrency policy.

EXACT UNIQUE CONFLICTS

Only a named provider constraint may select a first winner

EXECUTABLE ASSET 18-I - PROVIDER-PINNED CLASSIFIER

```
public enum UniqueConflictKind
{
    InboundPublicFact,
    DecisionIdentity,
    ExternalIdentityLink
}

public sealed record ProviderErrorIdentity(
    int ProviderCode,
    string Constraint);

public interface IProviderErrorIdentityReader
{
    bool TryRead(
        DbUpdateException exception,
        out ProviderErrorIdentity identity);
}

public sealed class ExactUniqueConflictClassifier(
    IProviderErrorIdentityReader reader)
{
    private static readonly IReadOnlyDictionary<string, UniqueConflictKind>
        Accepted = new Dictionary<string, UniqueConflictKind>{
            StringComparer.Ordinal
        }
        {
            ["UX_InboundPublicFacts_Producer_Fact_Contract"] =
                UniqueConflictKind.InboundPublicFact,
            ["UX_AlarmFloodDecisions_DecisionIdentity"] =
                UniqueConflictKind.DecisionIdentity,
            ["UX_ExternalIdentityLinks_ScopedValue"] =
                UniqueConflictKind.ExternalIdentityLink
        };

    public bool TryClassify(
        DbUpdateException exception,
        out UniqueConflictKind kind)
    {
        kind = default;
        return reader.TryRead(exception, out var identity)
            && identity.ProviderCode is 2601 or 2627
            && Accepted.TryGetValue(identity.Constraint, out kind);
    }
}
```

EXECUTABLE ASSET 18-J - FRESH-SCOPE RESOLUTION AND NEGATIVE MATRIX

```
public async Task<ConsumerOutcome> ExecuteConsumerAsync(
    AcceptedPublicFact<AlarmFloodDetectedV1> input,
    CancellationToken cancellationToken)
{
    {
        await using var attempt = scopes.CreateAsyncScope();
        var handler = attempt.ServiceProvider
            .GetRequiredService<AcceptAlarmFloodConsumer>();
        try
        {
            return await handler.HandleAsync(
                input, cancellationToken);
        }
        catch (DbUpdateException exception)
        {
            when (conflicts.TryClassify(exception, out var kind)
                && kind == UniqueConflictKind.InboundPublicFact)
            {
                // Leaving this block disposes the failed DbContext.
            }
        }
    }

    await using var reload = scopes.CreateAsyncScope();
    var resolver = reload.ServiceProvider
        .GetRequiredService<InboundFactOutcomeResolver>();
    return await resolver.ResolveAsync(
        input, CancellationToken.None);
}

[Theory]
[InlineData(2601, "UX_InboundPublicFacts_Producer_Fact_Contract", true)]
[InlineData(2627, "UX_Users_Email", false)]
[InlineData(547, "FK_RootCauseCases_Tenant", false)]
public void TXN_UNQ_001_only_exact_constraints_are_semantic(
    int code, string constraint, bool expected)
{
    var error = fixture.DbUpdateException(code, constraint);
    fixture.Classifier.TryClassify(error, out _)
        .Should().Be(expected);
}

[Fact]
public async Task changed_payload_under_inbox_constraint_blocks()
{
    await fixture.AcceptFactAsync("payload-A");
    var act = () => fixture.AcceptSameIdentityAsync("payload-B");
    await act.Should().ThrowAsync<PublicFactIdentityConflict>();
    fixture.BusinessEffects.Should().ContainSingle();
}
```

DBUPDATEEXCEPTION IS NOT DUPLICATE

The classifier is bound to one provider and exact migrated constraint names. Foreign keys, unrelated unique indexes, deadlocks and unknown codes remain failures.

BOUNDED WHOLE-UNIT RETRY

Only a known rolled-back attempt may replay automatically

EXECUTABLE ASSET 18-K - CERTAINTY-AWARE RETRY RUNNER

```
public enum AttemptCertainty { RolledBack, CommitUnknown }

public sealed record StableOperationSeed(
    CommitIntentId CommitIntent,
    DecisionIdentity Decision,
    PublicFactIdentity PublicFact,
    byte[] AcceptedInput);

public sealed class SemanticTransactionRunner(
    IServiceScopeFactory scopes,
    ITransientFailureClassifier failures,
    TransactionRetryOptions options)
{
    public async Task<TransactionOutcome> ExecuteAsync(
        StableOperationSeed seed,
        CancellationToken cancellationToken)
    {
        for (var attempt = 1; attempt <= options.MaximumAttempts; attempt++)
        {
            await using var scope = scopes.CreateAsyncScope();
            var operation = scope.ServiceProvider
                .GetRequiredService<IStableSemanticOperation>();
            try
            {
                return await operation.ExecuteAsync(
                    seed, cancellationToken);
            }
            catch (Exception exception)
            {
                when (failures.TryClassify(
                    exception, out var certainty)
                    && certainty == AttemptCertainty.RolledBack
                    && attempt < options.MaximumAttempts)
                {
                    await options.Backoff.DelayAsync(
                        attempt, cancellationToken);
                }
            }
            catch (Exception exception)
            {
                when (failures.TryClassify(
                    exception, out var certainty)
                    && certainty == AttemptCertainty.CommitUnknown)
                {
                    return TransactionOutcome.Unknown(
                        seed.CommitIntent, exception);
                }
            }
            return TransactionOutcome.RetryBudgetExhausted(
                seed.CommitIntent);
        }
    }
}
```

EXECUTABLE ASSET 18-L - RETRY EQUIVALENCE GATES

```
[Theory]
[InlineData(TransientFault.DeadlockBeforeCommit, 2)]
[InlineData(TransientFault.SerializationAbort, 3)]
public async Task TXN_RETRY_001_replays_whole_unit_with_stable_ids(
    TransientFault fault, int expectedAttempts)
{
    fixture.FailRolledBackAttempts(fault, expectedAttempts - 1);
    var outcome = await fixture.RunStableOperationAsync();

    outcome.Kind.Should().Be(TransactionOutcomeKind.Committed);
    fixture.Attempts.Should().HaveCount(expectedAttempts);
    fixture.Attempts.Select(x => x.CommitIntent).Distinct()
        .Should().ContainSingle();
    fixture.Attempts.Select(x => x.PublicFact).Distinct()
        .Should().ContainSingle();
    (await fixture.ReadAtomicSetAsync()).Should()
        .BeEquivalentTo(fixture.SequentialOracle);
}

[Fact]
public async Task unknown_commit_is_not_retried_blindly()
{
    fixture.FailWithCommitResponseLoss();
    var outcome = await fixture.RunStableOperationAsync();
    outcome.Kind.Should().Be(TransactionOutcomeKind.Unknown);
    fixture.Attempts.Should().ContainSingle();
}
```

TRANSIENT IS NOT ENOUGH

Automatic replay requires proof that the failed attempt rolled back. A disconnect during commit is routed to durable reconciliation, even when the error is retryable.

AMBIGUOUS COMMIT AND CANCELLATION

A fresh primary read decides what the client token cannot

EXECUTABLE ASSET 18-M - DURABLE COMMIT RECONCILIATION

```
public enum CommitResolution { Committed, RolledBack, Unknown }

public sealed class CommitIntentReconciler(
    IDbContextFactory<RootCauseDbContext> contexts,
    ReconciliationOptions options)
{
    public async Task<CommitResolution> ResolveAsync(
        CommitIntentId intent)
    {
        using var budget = new CancellationTokenSource(
            options.MaximumDuration);
        try
        {
            await using var db = await contexts.CreateDbContextAsync(
                budget.Token);
            var observed = await db.CommitIntents
                .AsNoTracking()
                .SingleOrDefaultAsync(
                    x => x.CommitIntentId == intent,
                    budget.Token);
            return observed is null
                ? CommitResolution.RolledBack
                : CommitResolution.Committed;
        }
        catch (Exception exception)
            when (options.IsObservationUnavailable(exception))
        {
            return CommitResolution.Unknown;
        }
    }
}

public async Task<TransactionOutcome> ReconcileAsync(
    TransactionOutcome outcome)
{
    if (outcome.Kind != TransactionOutcomeKind.Unknown)
        return outcome;
    return await reconciler.ResolveAsync(outcome.CommitIntent) switch
    {
        CommitResolution.Committed =>
            await outcomes.DemandAsync(outcome.CommitIntent),
        CommitResolution.RolledBack =>
            TransactionOutcome.SafeToRetry(outcome.CommitIntent),
        _ => outcome
    };
}
```

EXECUTABLE ASSET 18-N - COMMIT-DISCONNECT AND CANCELLATION MATRIX

```
[Theory]
[InlineData(FaultPoint.BeforeProviderCommit, CommitResolution.RolledBack)]
[InlineData(FaultPoint.AfterProviderCommit, CommitResolution.Committed)]
[InlineData(FaultPoint.PrimaryUnavailable, CommitResolution.Unknown)]
public async Task TXN_AMB_001_fresh_state_resolves_commit_gap(
    FaultPoint fault, CommitResolution expected)
{
    await fixture.ExecuteUntilConnectionLossAsync(fault);
    var actual = await fixture.ReconcileInFreshContextAsync();
    actual.Should().Be(expected);
}

[Fact]
public async Task TXN_CAN_001_cancellation_after_commit_does_not_lie()
{
    var result = await fixture.CommitThenCancelCallerAsync();
    result.ClientToken.IsCancellationRequested.Should().BeTrue();
    result.Reconciled.Kind.Should()
        .Be(TransactionOutcomeKind.Committed);
    result.DurableOutcome.Should().NotBeNull();
}

[Fact]
public async Task unreadable_primary_keeps_outcome_unknown()
{
    await fixture.MakeCommitAmbiguousAndPrimaryUnavailableAsync();
    (await fixture.ReconcileInFreshContextAsync())
        .Should().Be(CommitResolution.Unknown);
    fixture.AutomaticRetries.Should().Be(0);
}
```

CANCELLATION IS NOT DATABASE EVIDENCE

Caller cancellation may stop waiting, but reconciliation uses its own bounded budget. Committed, rolled back and still unknown remain distinct public outcomes.

DISPATCHER GAPS, RESTORE AND ACCEPTANCE

Publication stays outside the transaction and retains one fact

EXECUTABLE ASSET 18-O - SHORT COMMITS AROUND EXTERNAL PUBLISH

```
public async Task<DispatchOutcome> DispatchOneAsync(
    CancellationToken cancellationToken)
{
    var claim = await claims.TryClaimAsync(cancellationToken);
    if (claim is null) return DispatchOutcome.None();

    BrokerPublishReceipt receipt;
    try
    {
        receipt = await broker.PublishAsync(
            claim.StoredEnvelopeBytes,
            claim.PublicFactIdentity,
            cancellationToken); // No DbContext transaction is open.
    }
    catch (Exception exception)
    {
        await claims.RecordUnknownAsync(
            claim.AttemptId, exception, CancellationToken.None);
        throw;
    }

    await claims.RecordBrokerEvidenceAsync(
        claim.AttemptId, receipt, CancellationToken.None);
    return DispatchOutcome.Published(claim.PublicFactIdentity);
}

[Fact]
public async Task TXN_DSP_001_accept_then_crash_reuses_same_fact()
{
    await fixture.PublishThenCrashBeforeStatusCommitAsync();
    await fixture.RestartAndReconcileOrRepublishAsync();

    fixture.PublishedFactIds.Distinct().Should().ContainSingle();
    fixture.ConsumerBusinessEffects.Should().ContainSingle();
    fixture.DispatcherDatabaseTransactionsDuringPublish.Should().Be(0);
}
```

EXECUTABLE ASSET 18-P - RESTORE AND ACCEPTANCE EVIDENCE

```
[Fact]
public async Task TXN_REST_001_restore_replay_preserves_atomic_sets()
{
    var before = await fixture.RootCauseCommitBaselineAsync();
    var alarmBefore = await fixture.AlarmManagementFingerprintAsync();

    await fixture.RestoreReplayAndReconcileRootCauseAsync();

    (await fixture.RootCauseCommitBaselineAsync())
        .Should().BeEquivalentTo(before);
    (await fixture.AlarmManagementFingerprintAsync())
        .Should().Be(alarmBefore);
    fixture.ReplayDuplicateEffects.Should().Be(0);
}

{
    "evidenceId": "TXN-REVIEW-001", "chapter": 18,
    "artifactCommit": null, "sdk": "net8.0",
    "provider": null, "driver": null, "engineVersion": null,
    "isolation": null, "retryProfile": null, "broker": null,
    "tests": {
        "TXN-OWN-001": "PENDING", "TXN-CMD-001": "PENDING",
        "TXN-CNS-001": "PENDING", "TXN-DSP-001": "PENDING",
        "TXN-SAVE-001": "PENDING", "TXN-CON-001": "PENDING",
        "TXN-UNQ-001": "PENDING", "TXN-ISO-001": "PENDING",
        "TXN-RETRY-001": "PENDING", "TXN-AMB-001": "PENDING",
        "TXN-CAN-001": "PENDING", "TXN-REST-001": "PENDING"
    },
    "faultSchedulesExecuted": false, "accepted": false
}
```

Chapter 18 code-audit result

The chapter now exposes one executable path from an enumerated owner-local atomic set to command and consumer commits, semantic concurrency replay, exact conflict classification, honest ambiguity and publication/restore recovery.

CURRENT VERDICT

The transaction model is implementable in shape, but no provider, engine, broker or fault schedule has executed. All transaction and recovery evidence remains explicitly PENDING.

What this chapter has proved

Chapter 18 has defined the exact local atomicity contract: command state plus pending intent, consumer receipt/outcome/effect plus pending intent, and dispatcher lease/status around an external publish. It has separated optimistic token conflicts from provider-specific unique insert conflicts; required fresh semantic replay, short provider-pinned isolation, whole-unit retries, stable identities, explicit commit ambiguity, phase-aware cancellation, restore reconciliation and evidence gates.

It has not compiled a DbContext, selected the provider token/error codes, generated migrations, executed SaveChanges, observed SQL/locks, injected a commit disconnect, retried a transaction, acknowledged a broker delivery, published a fact or restored a database. Transaction semantics are specified, not implemented or evidenced. RootCause remains G0 accepted and G1 specified.

LOCAL CERTAINTY, DISTRIBUTED HONESTY

NEXUS-1 can prove that one owner committed one meaning. It cannot truthfully claim that another service observed or committed merely because the first local transaction succeeded.

Close the commit design with one operation record

Catalogue field	Accepted record	Why it must be explicit
Owner / operation	Service, database, use case/consumer/dispatcher lane and semantic operation identity.	Prevents a workflow label from hiding multiple owners/commits.
Atomic set	Every table/row role, stable identity and invariant that commits together.	Makes partial-state and migration tests enumerable.
Concurrency / uniqueness	Token, exact constraints, classifier, reload/replay and conflict outcomes.	Separates stale writes, duplicates, collisions and defects.
Isolation / retry	Provider settings, transaction duration, attempt budget and stable inputs.	Prevents accidental long locks and meaning-changing retries.
Ambiguity / cancellation	Commit intent, verify query, unknown outcome and operator path.	A timeout cannot be misreported as rollback or success.
External gap	Response, ack or publish action after commit plus duplicate/recovery rule.	Exposes the unavoidable non-atomic boundary.
Evidence / review	Fault schedules, machine results, metrics/runbook, owner/reviewer and expiry.	Binds the transaction claim to executed provider artifacts.

Chapter checkpoint

You are ready to implement local transactions when every operation names one owner and atomic set, one normal SaveChanges holds all required rows, stale and unique conflicts follow different semantic paths, retries preserve identities, commit ambiguity is reconciled from durable evidence, cancellation cannot lie about commit, and broker/remote work remains outside the database transaction.

Question	Expected answer
What does one local transaction protect?	One owner's invariant-bearing state, receipt/outcome and pending public intent for one semantic operation.
May it span another service, database or broker?	No. External effects use durable facts, idempotency, reconciliation and later compensation policy.
What is the normal EF commit shape?	One scoped DbContext and one provider-supported SaveChanges transaction.
When is a manual transaction admitted?	Only for a named local requirement with provider, retry, savepoint, lock and failure evidence.
What does DbUpdateConcurrencyException mean?	A protected update/delete used stale original token state; Application must reload and re-evaluate meaning.
Does a unique insert raise that exception?	Generally no; uniqueness is provider-specific and requires exact constraint/error classification.

Question	Expected answer
May the loser overwrite the winner?	No. It reloads current state and replays policy; merge is explicit domain behavior only.
What stays stable during retry?	Request/source/decision/public-fact/commit identities, accepted payload and policy inputs required by the operation.
What if the connection drops during commit?	Outcome is unknown until a fresh durable lookup proves committed, rolled back or still unresolved.
Does cancellation prove rollback?	No. Its meaning depends on phase; a known commit remains committed.
When may a consumer acknowledge?	After its local commit is known or recoverably reconciled; lost acknowledgement becomes duplicate redelivery.
Why may a dispatcher publish twice?	Publish and database status cannot commit atomically; both attempts retain the same PublicFactIdentity.
What survives restore?	Commit/receipt/outcome/public-fact identities, immutable payloads, mappings, causal lineage and reconciliation state.
What is the current status?	Transaction semantics are specified, not provider-executed; RootCause remains G0 accepted and G1 specified.
What does Chapter 19 add?	The exact separation and mapping of domain events, integration events and API contracts.

NEXT CHAPTER

Chapter 19, Domain Events, Integration Events, and API Contracts, will separate private aggregate facts from producer-owned public promises and request/response contracts. It will define mapping time, publication eligibility, payload ownership and versioning boundaries without letting EF entities, internal domain objects or transport envelopes become the shared language.

PART IV

Public Contracts and the Synchronous Edge

Private owner meaning crosses a boundary only through explicit, stable and versioned contracts whose failure semantics are observable.

CHAPTER 19**Public vocabularies**

owner terms -> shared meaning

CHAPTER 20**Event envelope**

identity + time + causation

CHAPTER 21**Request contracts**

validation + outcomes

CHAPTER 22**BFF boundary**

client shape without domain leakage

CHAPTER 23**Version + idempotency**

replay-safe synchronous edge

CHAPTER 24**Compatibility**

consumer proof before evolution

ENTRY RULE

The edge maps and protects. The owner decides. Consumers never import an aggregate, database row or private exception shape.

BEGIN WITH CHAPTER 19 / PUBLISH MEANING, NOT STORAGE

PART IV - PUBLIC CONTRACTS AND THE SYNCHRONOUS EDGE

19. Domain Events, Integration Events, and API Contracts

Distribution becomes manageable when each boundary speaks its own deliberate language. A private domain event explains what happened inside one owner; an integration event is a producer-owned public promise; an API contract defines one synchronous request, response or failure without exposing the aggregate that implements it.

CHAPTER ARGUMENT

Keep domain events, integration events and API contracts as separate types with explicit mappers. Map only committed or commit-eligible owner meaning into a public fact, map public input into consumer-owned application language, and version the public promise independently from CLR classes, database rows and deployment versions.

Chapter 18 established that AlarmManagement and RootCause commit independently. This chapter names what crosses the gap. AlarmManagement may raise a private AlarmFloodQualified domain event while deciding its own state, but other services never subscribe to that CLR type. The producer deliberately maps eligible meaning into AlarmFloodDetected.v1, stores the immutable pending public fact in TX-A, and later publishes it. RootCause validates that public contract and translates it into its own application input before touching private case state.

The same separation applies to HTTP. DetectAlarmFloodRequestV1 and DetectAlarmFloodAcceptedV1 describe the synchronous AlarmManagement edge; they are not commands inside the aggregate, broker payloads or RootCause projections. A successful AlarmManagement response proves only the accepted AlarmManagement outcome described by Chapter 18. It cannot promise that RootCause has received, processed or committed anything.

Chapter 19 specifies vocabulary, ownership, mapping, publication eligibility and evolution boundaries. Chapter 20 owns the canonical versioned message envelope and exact serialization; Chapter 21 implements command/query endpoints; Chapter 23 implements endpoint versioning, Problem Details and idempotency keys; Chapter 24 executes compatibility tests. The net8.0 listings here are reference candidates, not compiled runtime evidence. RootCause remains G0 accepted and G1 specified.

Three vocabularies prevent accidental coupling

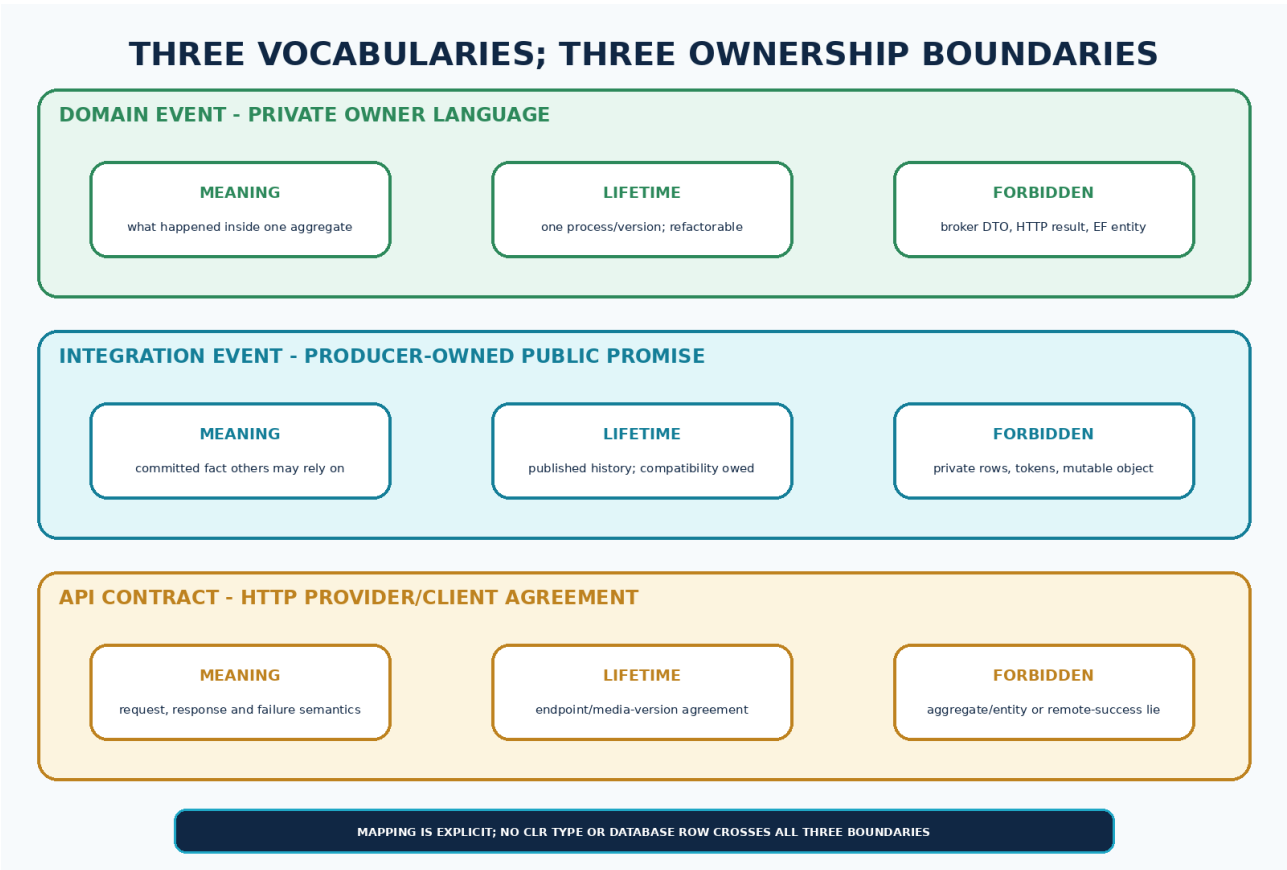


Figure 19.1 - Domain events, integration events and API contracts have different owners, lifetimes and forbidden dependencies.

Vocabulary	Owner and audience	Meaning	Durability/compatibility obligation
Domain event	One bounded context; Domain/Application handlers only.	A completed domain decision or state transition inside the owner language.	May be transient/in-process; local atomic effects still obey the owner transaction. Refactoring is private unless it changes a public mapping.
Integration event	Producer owns the promise; zero or more independent consumers rely on it.	A committed producer fact that consumers may use without importing producer internals.	Identity, accepted meaning and published history survive retry, replay, restore and supported-version windows.
API request	HTTP provider defines accepted input; authorized client supplies it.	A synchronous attempt to invoke one public capability under validation and idempotency rules.	Endpoint/media-version contract; not a persisted aggregate or event merely because it is a record type.
API response	HTTP provider defines what the completed/accepted response proves.	A representation of provider-local outcome, resource or operation status.	Status/body/header semantics remain compatible for supported clients; no remote-service success is implied.
API problem	HTTP provider defines a stable problem type and safe occurrence detail.	Machine-readable failure semantics at the HTTP interface.	Type/status/extensions evolve deliberately and never expose stack, schema, credential or private row detail.

TYPE SEPARATION IS SEMANTIC

Putting three records in one Contracts project is not separation if they share fields by inheritance, accept EF entities, reuse the same serializer shape or change together. Each boundary requires an explicit ownership decision and mapper.

One concept may have three deliberately different shapes

Concern	Private domain shape	Public integration shape	HTTP API shape
Identity	Typed DecisionIdentity and aggregate identity.	Stable public fact identity in Chapter 20 envelope plus producer-approved business identities.	Request/operation/resource identities required by client workflow; no private database key.
Time	Decision time and domain window in owner types.	Publicly defined occurrence/decision timestamps and units.	Request/response timestamps only where client semantics require them.
Evidence	Rich private objects, policy inputs and internal references.	Minimal sufficient public facts or stable approved references with availability semantics.	Client-supplied fields and safe result summary; never the internal evidence graph.
State	Full invariant-bearing aggregate state.	Past-tense fact; not a mutable aggregate snapshot unless that is the explicit contract.	Current provider representation or operation status with HTTP caching/version rules.
Failure	Exceptions or domain outcomes meaningful inside the owner.	Publication/consumption disposition belongs to messaging policy, not the payload business fact.	Status plus stable problem contract; infrastructure detail remains private.

Repetition across these shapes is intentional when the same concept is genuinely public. Duplication is cheaper than coupling the producer aggregate, consumer model and transport lifetime to one class. A mapper is the executable place where the team can review what meaning is copied, transformed, withheld or rejected.

A complete vertical slice: request to public fact to consumer meaning

Follow the same alarm-flood decision used in Chapter 18, but focus on language rather than commits. The HTTP edge accepts `DetectAlarmFloodRequestV1` and maps it to an `AlarmManagement` application command. Domain behavior produces private owner state and `AlarmFloodQualified`. A publication policy declares whether that domain meaning is eligible to become the producer promise `AlarmFloodDetected.v1`. The mapper creates only the reviewed public payload, which TX-A stores immutably. `RootCause` later validates that public contract, maps it into `AcceptAlarmFlood`, and raises its own private events while committing TX-B. None of those types is reused across all steps.

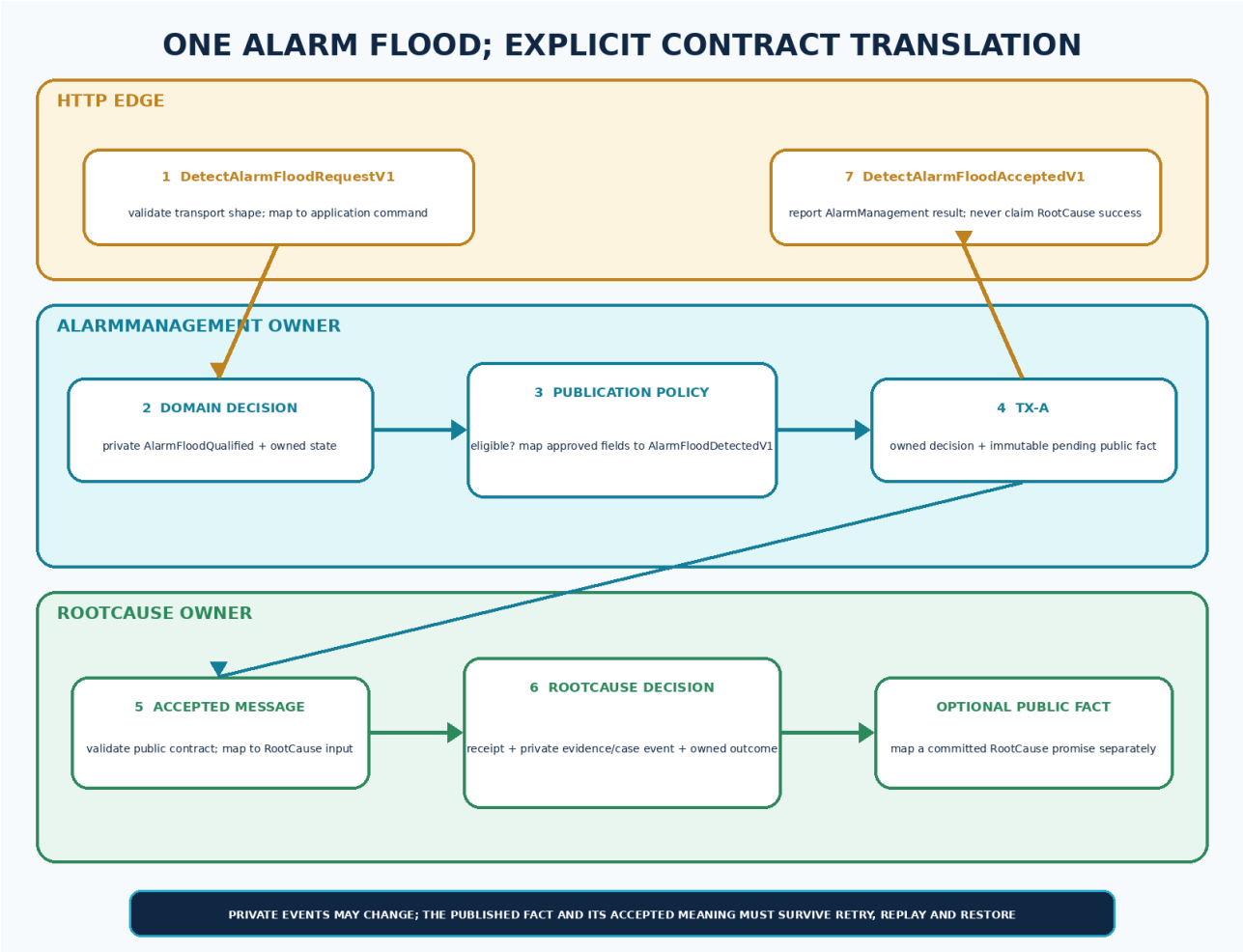


Figure 19.2 - One alarm-flood narrative passes through explicit HTTP, domain, integration and consumer mappings without sharing an internal model.

HOW TO READ THE WALKTHROUGH

The business occurrence is continuous, but each owner chooses its language. The public fact carries stable producer meaning; it does not carry AlarmManagement entities, RootCause commands or a promise that the consumer has succeeded.

The contract identity spine remains stable

Role	Readable example	Boundary rule	Reused by
HTTP request identity	request-am-20482	Identifies one accepted client operation; not the public fact identity.	AlarmManagement idempotency/outcome lookup and response.
AlarmManagement decision	decision-am-9184	Private owner identity may be exposed only when the public contract intentionally promises it.	Domain history, public causality and safe API outcome.
Domain event type	AlarmFloodQualified	Private CLR/domain vocabulary; no broker routing or external deserialization obligation.	AlarmManagement local policy/mapping before commit.
Public event type	nexus1.alarm-management.alarm-flood-detected.v1	Producer-owned semantic identifier; not derived from class/assembly names.	Envelope, publication registry, consumer acceptance and compatibility evidence.
Public fact identity	fact-am-7716	Allocated once for the accepted public promise and never changed by retry/version adapter.	Pending fact, dispatch, broker delivery, inbox and replay.
Consumer input	AcceptAlarmFlood	RootCause Application-owned type created after public validation.	RootCause policy and TX-B; never published as the producer contract.

Role	Readable example	Boundary rule	Reused by
RootCause case	case-rc-3107	Private consumer identity; absence from the producer event is expected.	RootCause state and its own later API/public facts.

Step 1 - HTTP accepts transport meaning, not an aggregate

The Edge validates media type, syntactic shape and public field constraints, authenticates the caller, and maps the accepted request into an application command. Application performs authorization and semantic validation. The request DTO has only client-visible concepts; it does not expose AlarmManagementRow, a concurrency token, a DbContext key or a domain event collection.

HTTP REQUEST TO APPLICATION COMMAND - [NEW][COMPILE + CONTRACT TEST REQUIRED]

```
public sealed record DetectAlarmFloodRequestV1(
    string RequestId,
    string SiteId,
    string LineId,
    DateTimeOffset WindowStartUtc,
    DateTimeOffset WindowEndUtc,
    IReadOnlyList<string> AlarmIds,
    string PolicyId);

internal static DetectAlarmFlood Map(
    DetectAlarmFloodRequestV1 request,
    AuthenticatedActor actor) => new(
    RequestIdentity.Parse(request.RequestId),
    CorrelationScope.Create(request.SiteId, request.LineId),
    AlarmWindow.Create(request.WindowStartUtc, request.WindowEndUtc),
    request.AlarmIds.Select(AlarmIdentity.Parse).ToArray(),
    PolicyIdentity.Parse(request.PolicyId),
    actor.Identity);
```

Request field	Why it is public	Validation owner	What is deliberately absent
requestId	Lets a client retry one operation under Chapter 23 policy.	Edge syntax; Application equivalence/idempotency.	CommitIntentId and database transaction identity.
siteId / lineId	Names the accepted correlation scope in client language.	Edge format; Application authorization/scope policy.	AlarmManagement internal partition/row keys.
windowStartUtc / windowEndUtc	Defines the requested evaluation interval.	Edge timestamp shape; Domain duration/order rules.	Server processing time or broker timestamp.
alarmIds	Names observations the caller is allowed to propose.	Edge collection limits; Application existence/authorization; Domain policy.	Raw payloads, credentials or private evidence rows.
policyId	Makes requested policy meaning explicit where the public capability permits selection.	Application authorization and supported-policy check.	Policy implementation class or assembly version.

Step 2 - Domain raises a private fact about its own decision

AlarmManagement evaluates the owner invariant and records AlarmFloodQualified as a private event. The event can be rich enough for local rules and audit assembly because no external consumer is promised its CLR shape. It describes a completed domain decision; it is not a command to RootCause and not yet proof that a public event is eligible.

PRIVATE DOMAIN EVENT - [NEW][COMPILE + DOMAIN TEST REQUIRED]

```
internal sealed record AlarmFloodQualified(
    DecisionIdentity Decision,
    CorrelationScope Scope,
    AlarmWindow Window,
    IReadOnlyList<AlarmIdentity> AcceptedAlarms,
    PolicyIdentity Policy,
    DateTimeOffset DecidedAtUtc) : IDomainEvent;

internal sealed class AlarmFloodDecision
{
    private readonly List<IDomainEvent> raised = [];

    public void Qualify(QualifiedFlood result)
    {
        Apply(result);
        raised.Add(new AlarmFloodQualified(
            result.Decision, result.Scope, result.Window,
            result.Alarms, result.Policy, result.DecidedAtUtc));
    }
}
```

PAST TENSE IS NECESSARY BUT NOT SUFFICIENT

A name such as AlarmFloodQualified says something happened. It becomes a valid domain event only when Domain behavior actually establishes that fact; it becomes a valid integration event only after a separate public eligibility and mapping decision.

Step 3 - Publication policy decides whether private meaning becomes public

Not every domain event deserves publication. Some exist only to coordinate local invariant work; some contain restricted evidence; some are implementation milestones with no stable public meaning. A publication policy accepts an explicit domain event and current owner context, returns Eligible, Suppressed or Rejected, and records the policy identity that explains the decision. Suppression is normal; rejection signals an unsafe or incomplete mapping.

Eligibility result	Meaning	Required action	Example
Eligible	A committed producer fact is useful and safe for independent consumers.	Allocate/reuse PublicFactIdentity; map reviewed contract; stage pending public fact in TX-A.	AlarmFloodQualified becomes AlarmFloodDetected.v1.
Suppressed	The domain fact is local-only or intentionally produces no public history.	Keep local event/effect evidence as required; do not create a public identity or broker message.	AlarmFloodEvaluationCacheRefreshed remains private.
Rejected	The proposed public contract leaks private data, lacks stable semantics or cannot support consumers independently.	Fail the mapping/review path; redesign before publication; preserve protected diagnostics.	Proposed payload includes access token, EF row or mutable rule object.

Step 4 - The producer maps one immutable public promise

The public payload is not the domain event serialized directly. A dedicated mapper copies only accepted semantics, converts private value objects into contract primitives with explicit units, orders collections canonically where order is not meaningful, and refuses values that cannot satisfy the public schema. The event type is a stable semantic name; it does not depend on namespace, record name or assembly version.

DOMAIN EVENT TO INTEGRATION PAYLOAD - [NEW][COMPILE + MAPPING TEST REQUIRED]

```
namespace AlarmManagement.Contracts.Integration.V1;

public sealed record AlarmFloodDetectedV1(
    string DecisionId,
    string SiteId,
    string LineId,
    DateTimeOffset WindowStartUtc,
    DateTimeOffset WindowEndUtc,
    IReadOnlyList<AlarmReferenceV1> Alarms,
    string PolicyId,
    DateTimeOffset DetectedAtUtc);

public sealed record AlarmReferenceV1(string AlarmId, int Position);

internal static AlarmFloodDetectedV1 MapPublic(AlarmFloodQualified source) => new(
    source.Decision.ToPublicString(),
    source.Scope.Site.ToPublicString(),
    source.Scope.Line.ToPublicString(),
    source.Window.StartUtc,
    source.Window.EndUtc,
    source.AcceptedAlarms
        .OrderBy(x => x.Position)
        .Select(x => new AlarmReferenceV1(x.ToPublicString(), x.Position))
        .ToArray(),
    source.Policy.ToPublicString(),
    source.DecidedAtUtc);
```

Chapter 20 will place this payload in the canonical envelope with message identity, event type, schema version, producer, occurrence time, correlation and causation. Chapter 19 fixes the payload meaning and mapper boundary only. TX-A must store the resulting immutable public representation or a deterministic, version-pinned form that Chapter 20 proves byte-compatible; a dispatcher never remaps current domain state during retry.

Step 5 - Delivery carries the producer promise, not the producer model

The dispatcher publishes fact-am-7716 under the accepted event type. A broker delivery may repeat, arrive after a deployment, or be replayed after restore. Consumers therefore accept the stable producer tuple and contract version, not a live producer object. The message does not contain a RootCause command, case identifier or desired verdict; those decisions belong to RootCause.

Producer promise	Consumer may rely on	Consumer must not infer
The producer qualified one alarm flood under the named scope/window/policy.	The defined public fields and units for the supported contract version.	RootCause must open a case, choose a hypothesis or issue a verdict.
The public fact identity and producer tuple are stable.	Duplicate detection, audit lineage, replay and reconciliation under that identity.	Exactly-once delivery or globally ordered delivery.
The payload was created from committed/commit-eligible producer meaning.	The producer will not mutate the historical payload on retry.	The consumer database committed merely because the producer did.
The event type/version has an accepted producer specification.	Compatibility obligations documented for its supported window.	CLR namespace/class compatibility or database-schema compatibility.

Step 6 - RootCause translates the public contract into its own language

RootCause Edge/Infrastructure authenticates the producer and validates the Chapter 20 envelope, accepted event type and payload shape. An anti-corruption mapper then creates AcceptAlarmFlood, an Application-owned input using RootCause value objects. Application resolves the inbox tuple, applies RootCause policy and raises private events such as FloodEvidenceAccepted or InvestigationOpened. It never passes the producer DTO into the aggregate.

PUBLIC CONTRACT TO ROOTCAUSE APPLICATION INPUT - [NEW][COMPILE + NEGATIVE TEST REQUIRED]

```

internal sealed record AcceptAlarmFlood(
    ProducerFactTuple Source,
    CanonicalFingerprint Fingerprint,
    ExternalAlarmFloodIdentity Flood,
    CorrelationScope Scope,
    AlarmWindow Window,
    IReadOnlyList<ExternalAlarmReference> Alarms,
    ExternalPolicyReference ProducerPolicy,
    DateTimeOffset ProducerDecidedAtUtc);

internal static AcceptAlarmFlood MapAccepted(
    AcceptedEnvelope envelope,
    AlarmFloodDetectedV1 payload)
{
    return new AcceptAlarmFlood(
        envelope.ProducerFact,
        envelope.CanonicalFingerprint,
        ExternalAlarmFloodIdentity.From(payload.DecisionId),
        CorrelationScope.Create(payload.SiteId, payload.LineId),
        AlarmWindow.Create(payload.WindowStartUtc, payload.WindowEndUtc),
        payload.Alarms.Select(ExternalAlarmReference.From).ToArray(),
        ExternalPolicyReference.From(payload.PolicyId),
        payload.DetectedAtUtc);
}

```

Validation layer	Accepts/rejects	Must not decide
Transport/envelope	Authenticated producer, supported type/version, size, encoding and required envelope fields.	Whether the alarm flood is useful RootCause evidence.
Contract deserializer	Payload shape, required values, basic ranges and representation rules.	Whether a RootCause case should be opened.
Anti-corruption mapper	Conversion into RootCause value objects; rejects unknown units/unsafe identifiers.	Producer domain invariants or RootCause final verdict.
Application	Inbox identity/fingerprint, authorization policy for producer, lifecycle and orchestration.	Private aggregate invariants without Domain behavior.
Domain	Evidence acceptance, case/hypothesis/verdict invariants and private events.	Broker acknowledgement, HTTP status or producer state.

Step 7 - HTTP response reports only synchronous owner truth

AlarmManagement returns a provider-local outcome after TX-A is known or reconciled. The response may say Accepted/Committed and provide a status resource for the AlarmManagement operation. It must not include case-rc-3107 unless a later BFF/query contract explicitly composes that independently authorized and freshness-labeled view. The asynchronous event and RootCause work continue outside the HTTP transaction.

OWNER-LOCAL HTTP SUCCESS CONTRACT - [NEW][COMPILE + HTTP TEST REQUIRED]

```

public sealed record DetectAlarmFloodAcceptedV1(
    string RequestId,
    string DecisionId,
    string PublicFactId,
    string Status,
    Uri StatusUri);

internal static DetectAlarmFloodAcceptedV1 MapResponse(
    CommittedFloodDecision outcome,
    IOperationLinks links) => new(
    outcome.Request.ToPublicString(),
    outcome.Decision.ToPublicString(),
    outcome.PublicFact.ToPublicString(),
    "accepted",
    links.AlarmFloodDecision(outcome.Decision));

```

A 2XX RESPONSE HAS A BOUNDED CLAIM

It can prove the synchronous provider outcome described by the endpoint. It cannot prove broker publication, delivery, RootCause processing or downstream projection freshness unless a separate contract obtain:

Failure points remain explicit across the walkthrough

Interruption or defect	Truth retained	Safe result	Forbidden shortcut
Request DTO is valid JSON but violates a domain window rule	No accepted AlarmManagement decision exists.	Application/domain rejection mapped to the accepted HTTP problem type.	Put domain rules in serializer attributes and bypass Domain.
Domain decision exists but publication policy suppresses it	Private owner state may commit without a public fact if that is the accepted operation contract.	Return owner-local outcome; record suppression evidence where required.	Publish every domain event automatically.
Public mapper cannot represent a private value safely	No valid public payload exists yet.	Reject commit/publication path according to atomic policy; alert and preserve diagnostics.	Serialize the domain event or EF entity as fallback.
HTTP response is lost after TX-A	Decision and fact-am-7716 remain committed.	Retry by request identity returns the same API outcome/public fact identity.	Mint a new event because the client retried.
RootCause receives unsupported event version	Producer fact may be valid; this consumer cannot accept it.	Do not invoke Domain; apply retry/quarantine/dead-letter policy with evidence.	Deserialize into the nearest CLR type and hope fields align.
Same producer tuple carries changed payload	Identity/content conflict exists.	Reject/quarantine; retain both protected fingerprints and diagnostics.	Treat as Duplicate or overwrite prior receipt.
RootCause maps successfully but rejects evidence semantically	Public fact remains true; RootCause outcome is owned and independent.	Commit stable Abstained/Rejected consumer outcome as designed.	Call the producer fact invalid merely because consumer policy declined it.

What the walkthrough proves - and what it deliberately does not

The walkthrough makes concrete	It does not claim
Distinct HTTP, Domain, public integration and consumer application types.	That any listed type currently compiles in a NEXUS-1 repository.
Publication eligibility and explicit public mapping before durable intent.	That every domain event must become an integration event.
A producer promise that is useful without exposing producer internals.	That consumers share producer behavior or state ownership.
Consumer anti-corruption and private RootCause decision making.	That a public event commands RootCause or predicts its outcome.
Stable identities across response loss, dispatch, duplicate delivery and replay.	Exactly-once delivery, one global model or a global transaction.

Domain events remain private owner history

A domain event captures an invariant-relevant occurrence in the ubiquitous language of one bounded context. It may coordinate multiple aggregates only within an explicitly accepted local transaction/consistency policy; it is not a license for hidden remote calls. Its handlers belong to Domain/Application composition, and every durable side effect still fits the Chapter 18 commit boundary.

Domain-event moment	Allowed behavior	Durability rule	Common failure
Raised by aggregate behavior	Record immutable owner meaning and required private values.	Collection in memory is not durability; owner state determines eventual commit.	Event says something happened before invariant validation completes.
Handled before SaveChanges	Apply local policies, stage owner rows and public intents required atomically.	All durable local effects commit or roll back in the same owner unit.	Handler calls broker/remote service inside the transaction.
Observed after commit	Trigger only work whose source is durable and independently recoverable.	Use persisted fact/outbox/checkpoint; an in-memory callback is not recovery proof.	Post-commit in-process event is treated as guaranteed delivery.
Refactored	Rename/split private events when local behavior remains correct.	Re-prove public mappers and retained replay paths affected by the change.	Private rename silently changes public event type or payload.

A synchronous or asynchronous C# handler signature does not define the architectural guarantee. The guarantee comes from the transaction and recovery design. An async method can still run before

SaveChanges; a synchronous callback can still be lost after process failure. NEXUS-1 records dispatch phase, owner, atomic effects and replay source explicitly.

Integration events are facts, not remote commands in disguise

Candidate name	Classification	Reason	Preferred correction
AlarmFloodDetected.v1	Integration event candidate.	Past producer fact with stable public meaning; consumers choose reactions.	Keep payload minimal/sufficient and producer-owned.
OpenRootCauseCaseNow	Remote command disguised as event.	Names a consumer action and assumes RootCause workflow/state ownership.	Publish the alarm-flood fact or define an explicit command protocol with responsibility/timeout semantics.
AlarmManagementRowChanged	Persistence leak.	Exposes storage mechanics and unstable row shape rather than business meaning.	Name the business occurrence and map approved fields.
AlarmFloodUpdated	Ambiguous fact.	Does not say which meaning changed, why consumers care or whether snapshots replace history.	Use a precise new fact or explicitly defined snapshot contract.
RootCauseVerdictIssued.v1	Integration event candidate.	Producer-owned RootCause fact; Audit/Compliance choose independent effects.	Keep private VerdictIssued domain event separate from the public contract.

CONSUMER AUTONOMY

A producer may publish what it knows happened. It must not encode the consumer's aggregate transition, database key, retry command or desired acknowledgement into the event merely to preserve the old monolith workflow.

Public payloads balance sufficiency, stability and minimization

Payload choice	Benefit	Risk	Acceptance rule
Minimal fact values	Low coupling and exposure.	Consumer may lack enough context and call back synchronously.	Include every stable value required for intended independent consumer decisions.
Rich immutable fact	Supports independent processing and replay.	Larger compatibility/privacy surface.	Each field has public meaning, classification, unit, retention and owner.
Stable public reference	Avoids copying large or restricted data.	Reference may be unavailable, stale or authorization-dependent.	Specify authority, freshness, availability, authorization and failure behavior; never imply a foreign key.
Aggregate snapshot	Simple initial consumer projection.	Publishes private shape and blurs event versus state transfer.	Use only as an explicitly named/versioned snapshot contract with replacement semantics.
Opaque blob	Preserves source bytes.	Consumers cannot validate meaning; may leak secrets or bind to internal format.	Reject unless the blob itself is the reviewed public artifact with media/schema/security contract.

API contracts describe the synchronous edge

An API contract includes more than a C# request record. It names method/path or operation, media type/version, authentication expectation, request constraints, idempotency semantics, response status/body/headers, problem types, authorization visibility and caching/freshness. Chapters 21 and 23 implement these mechanics; Chapter 19 ensures the models remain boundary-owned.

API surface	Owns	Must not expose	Later implementation chapter
Command request	Client-visible intent and representation constraints.	Aggregate method parameters, EF entities, concurrency token unless explicitly public.	Chapter 21 command boundary; Chapter 23 idempotency/versioning.
Command response	What accepted/completed provider-local outcome means and how to locate it.	Unproved downstream success or internal transaction diagnostics.	Chapters 21 and 23.
Query representation	Authorized, freshness-labeled provider/BFF view	Live cross-database join or private	

API surface	Owns	Must not expose	Later implementation chapter
Problem detail	Stable failure type, HTTP status and safe occurrence-specific extensions.	Stack trace, SQL/constraint text, credentials or existence leaks.	Chapter 23.
OpenAPI description	Published operation/schema documentation for supported clients.	A claim that implementation behavior is compatible without tests.	Chapters 21, 23 and 24.

RFC 9457 defines Problem Details as a machine-readable HTTP error format and warns against exposing implementation internals. NEXUS-1 will use stable problem-type semantics rather than serializing domain exceptions. The exact ASP.NET Core registration, status mapping, extensions and idempotency interaction remain Chapter 23 work.

Mapping is a reviewed architectural component

Mapping ID	Source	Target	Owner	When executed
MAP-API-AM-001	DetectAlarmFloodRequestV1	DetectAlarmFlood	AlarmManagement Edge/Application	After transport validation; before authorization/domain behavior.
MAP-PUB-AM-001	AlarmFloodQualified + accepted owner context	AlarmFloodDetectedV1 payload	AlarmManagement Application	After domain decision; before TX-A stages immutable public intent.
MAP-CNS-RC-001	Accepted AlarmFloodDetectedV1 envelope/payload	AcceptAlarmFlood	RootCause Edge/Application	After producer/schema validation; before inbox/domain work.
MAP-API-AM-002	CommittedFloodDecision	DetectAlarmFloodAcceptedV1	AlarmManagement Edge	After commit outcome is known/reconciled.
MAP-PUB-RC-001	Private VerdictIssued	RootCauseVerdictIssuedV1 payload	RootCause Application	After verdict policy; before RootCause TX stages public intent.

Mapper proof	Required assertion
Completeness	Every required target field comes from an accepted source or explicit deterministic default.
No leakage	No private key, entity, token, credential, stack/SQL detail or unreviewed personal/operational data crosses.
Unit/time clarity	Every transformed quantity, timestamp and ordering rule has one public interpretation.
Determinism	Same accepted source plus mapping policy produces equivalent public meaning and canonical representation inputs.
Failure	Unknown enum/unit/identity or unrepresentable value fails explicitly; it is never silently coerced.
Lineage	Public fact retains source decision/correlation/causation identities required for audit without exposing private implementation IDs.
Compatibility	Old/new target versions and supported consumers follow the Chapter 24 matrix before publication.

Public contracts version independently

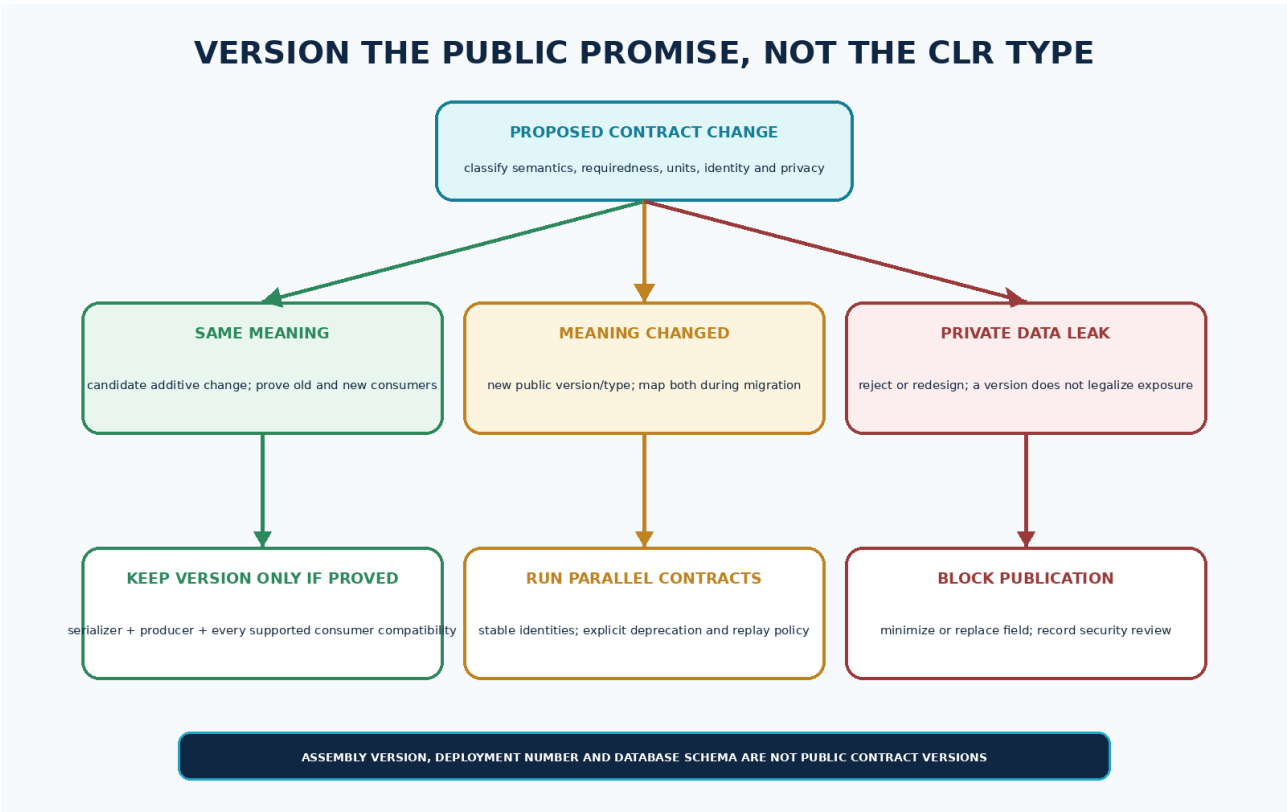


Figure 19.3 - Contract changes are classified by public meaning and privacy impact, not by CLR or deployment version.

A private domain event may change without a public version when all public mappings remain equivalent. An API contract may version without changing the integration event. An integration event may add a new version while the producer database remains unchanged. Conversely, a database migration never grants permission to change a public payload. These lifecycles are intentionally independent.

Proposed change	Default classification	Version action	Required evidence
Add optional field with stable independent meaning	Potentially backward compatible, not automatically safe.	Keep version only if every supported serializer/consumer tolerates absence/presence and validation remains valid.	Old producer/new consumer and new producer/old consumer tests; replay samples.
Make optional field required	Breaking for older payloads/producers.	New version or migration protocol with explicit default semantics.	Historical replay and missing-field negative tests.
Rename/remove field	Breaking representation and often semantic change.	New version; parallel mapper/consumer window.	Dual-publication/consumption, deprecation and restore/replay proof.
Change unit, scale, timezone or rounding	Semantic breaking change even if JSON type is unchanged.	New version/type with explicit conversion.	Boundary values, historical samples and consumer decision comparison.
Add enum value	Compatibility depends on consumer unknown-value policy.	Keep only if open-enum behavior was contracted/tested; otherwise new version.	Unknown-value tests for every supported consumer.
Change identity meaning or reuse ID	Breaking integrity change.	New contract and identity policy; often reject migration.	Dedup, correlation, restore and audit lineage proof.
Expose previously private field	Security/privacy review change, not just schema evolution.	Reject until classification, minimization, authorization and retention are accepted.	Threat/privacy review plus producer/consumer handling evidence.

ADDITIVE IS A HYPOTHESIS

Adding a JSON property is compatible only when the accepted serializers, validators, generated clients and every supported consumer behave correctly with both old and new payloads. Chapter 24 must prove the matrix.

Time, order and identity are separate contract concepts

Concept	Question answered	Owner	Misuse to prevent
Occurred/decision time	When did the business fact become true under producer semantics?	Producer public contract.	Using broker enqueue or consumer receipt time as business occurrence.
Recorded time	When did the producer durably record the fact?	Producer/outbox evidence; Chapter 20 envelope decision.	Assuming it equals occurrence or global time order.
Published time	When was a publish attempt/confirmation observed?	Dispatcher/broker evidence.	Mutating payload or public meaning on republish.
Received/processed time	When did this consumer observe/commit its reaction?	Consumer private state/metrics.	Publishing it as producer truth or ordering independent consumers by it.
Public fact identity	Which immutable producer fact is this?	Producer, then carried unchanged.	Replacing with trace ID, broker delivery tag or consumer key.
Correlation/causation	Which business conversation/source fact relates to this fact?	Declared contract/envelope policy.	Treating correlation as uniqueness, authorization or transaction identity.

Security and privacy are contract design inputs

- Classify every public field by business purpose, sensitivity, producer authority, allowed consumers, retention and observability/redaction rule.
- Never publish access tokens, connection strings, stack traces, raw SQL/provider messages, private concurrency tokens or reusable authorization evidence.
- Do not expose a private identifier merely because it is technically harmless; document how consumers may use it and whether it survives restore/merge/split.
- Authenticate producer/envelope independently from trusting payload business meaning; a valid signature does not make an unsupported schema or impossible value acceptable.
- Keep human-readable error detail safe and non-authoritative; machines branch on stable problem/event types and reviewed fields.
- Treat replay and long retention as first-class exposure: data acceptable for a transient request may be unsafe in a durable integration event.

Contract tests start at the mapping boundary

Test family	Minimum scenario	Required result	Evidence retained
Domain event	Invariant accepts/rejects boundary values.	Private event raised exactly when the owner fact becomes true; values match decision.	Domain test input, event and owner state.
Publication eligibility	Eligible, suppressed and rejected examples.	Only eligible meaning allocates/stages a public fact; unsafe mapping blocks.	Policy identity and outcome.
Public mapping	Golden and boundary values including ordering/units/time.	Exact approved payload meaning; no private fields; deterministic equivalence.	Source snapshot, mapping policy and target document.
Serializer contract	Old/new payload samples and canonical representation.	Chapter 20 envelope/payload round-trip and fingerprint rules hold.	Schema/version/sample corpus.
Consumer anti-corruption	Unknown enum/unit/identifier, unsupported version and valid payload.	Reject before Domain or map one accepted input; no partial state.	Adapter result and RootCause input.
HTTP request/response	Valid, invalid, forbidden, duplicate and lost-response retry.	Stable public status/body/problem semantics without entity or remote-success leak.	OpenAPI/fixtures/HTTP transcript.
Compatibility	Old producer/new consumer; new producer/old consumer; replay retained history.	Accepted changes pass; breaking change is rejected or versioned.	Chapter 24 matrix and machine results.
Security/privacy	Restricted fields, overlong payload, malicious detail and observability output.	No protected leakage; bounded rejection and redacted diagnostics.	Threat/privacy test and log capture.

Contract evidence gates

Evidence ID	Blocking rule	Required artifact
CTR-OWN-001	Every domain event, integration event and API contract names one owner, audience, semantic version policy and reviewer.	Contract ownership catalogue and architecture review.
EVT-DOM-001	Private domain events arise only from accepted owner behavior and do not cross transport boundaries.	Domain event catalogue, invariant tests and dependency scan.
EVT-PUB-001	Each public event is a past producer fact with stable meaning and independent consumer autonomy.	Published event specification and producer/consumer responsibility review.
EVT-ELIG-001	Publication eligibility is explicit; suppressed/rejected facts do not silently publish.	Eligibility policy matrix and machine tests.
EVT-MAP-001	Public mapping is deterministic, complete, unit-explicit and free of private implementation leakage.	Mapper source, golden fixtures, negative and privacy tests.
EVT-VERS-001	Semantic/requiredness/unit/identity changes follow accepted version and migration rules.	Change-classification record and supported-version policy.
EVT-TIME-001	Occurrence, record, publish and consume times retain distinct names and semantics.	Timestamp glossary and boundary/replay tests.
API-REQ-001	Requests expose public intent only and map through authorization/application boundaries.	Request schema, mapping and HTTP/application tests.
API-RESP-001	Responses claim only provider-local known truth and label asynchronous status/freshness.	Response/status table and lost-response/idempotency fixture.
API-ERR-001	Stable problem types map failures without exposing implementation/security detail.	RFC 9457-aligned catalogue, negative/security tests.
CNS-ACL-001	Consumers validate the public contract then translate into owner application/domain language.	Anti-corruption mapper and unsupported/invalid-input matrix.
SEC-DATA-001	Every public field has purpose, classification, allowed audience, retention and redaction rule.	Field-level data catalogue and threat/privacy review.
CTR-TEST-001	Supported producer/consumer/API versions execute the compatibility and replay matrix.	Machine results bound to build, schema/version and fixture corpus.
CTR-REVIEW-001	Producer, consumers, API clients, security, data, operations and architecture accept obligations/residual risk.	Signed evidence index, expiry and review triggers.

ADR-017 - Separate private events, public events and API contracts

ADR field	ADR-017 record
Status	Accepted for the NEXUS-1 reference design; implementation, serialization and compatibility execution evidence remain pending.
Context	Service extraction creates private owner models, asynchronous public history and synchronous HTTP interfaces with different audiences and evolution lifetimes.
Decision	Use distinct Domain, integration and API types. Domain events remain owner-private. Publication policy and explicit mappers create producer-owned past-tense public facts only from eligible meaning before durable public intent. Consumers validate and translate through anti-corruption mappers. APIs expose provider-local request/response/problem semantics. Public versions follow meaning, requiredness, unit, identity and privacy changes rather than CLR/database/deployment versions.
Consequences	Mapping code, field catalogues, version windows, fixtures and cross-team review become first-class artifacts. Some data is intentionally duplicated across shapes; domain refactors remain easier while public compatibility obligations become explicit.
Rejected	Serialize domain event or EF entity directly; share one Contracts model across Domain/API/broker; event names that command consumers; regenerate payload on dispatch; assembly-version event types; silent field coercion; API success that claims remote completion; catch-all exception serialization.
Review trigger	Public meaning, field classification, units/time, identity, mapper, serializer/envelope, supported consumer, endpoint representation, retention, replay or ownership changes.

Risk register for public contracts

Risk	Early signal	Blocking control
Domain type becomes wire contract	Broker payload contains namespace/class discriminators or private value objects.	EVT-DOM/EVT-MAP dependency and payload scan.
EF entity leak	Contract fields follow columns/navigation changes or include row/concurrency keys.	Contract-only model projects and private-field negative tests.
Event commands consumer	Payload names RootCause action/case/verdict or producer expects one reaction.	EVT-PUB responsibility review and consumer-autonomy scenarios.
Dispatch remaps history	Same PublicFactIdentity produces different bytes/fingerprint after deployment.	Immutable stored payload and replay comparison linked to Chapter 20/26.
API overclaims workflow	2xx response says case opened before RootCause evidence exists.	API-RESP truth table and asynchronous status tests.
Additive change breaks consumer	Unknown property/enum or stricter validation fails old consumer.	EVT-VERS/CTR-TEST full supported-version matrix.
Private data becomes durable public history	Payload/log/dead-letter contains restricted evidence or credentials.	SEC-DATA catalogue, minimization, redaction and retention tests.
Timestamp becomes ordering proof	Consumers sort independent facts by producer/broker clock and overwrite state.	EVT-TIME glossary and ordering policy tests.
Correlation becomes uniqueness	Unrelated facts collapse under shared workflow/trace ID.	PublicFactIdentity/producer tuple remain dedup keys.
Unsupported version reaches Domain	Deserializer/mapper defaults unknown values and aggregate acts.	CNS-ACL reject-before-Domain matrix.
Problem detail leaks internals	SQL/provider/stack or resource-existence detail appears in response.	API-ERR security fixtures and output scan.
Private refactor breaks replay	Historical event cannot map/consume after domain rename/deletion.	Retained version adapters/fixtures and restore-replay rehearsal.

Official-source validation

The terminology and boundary rules were checked against current primary sources on 19 July 2026. Microsoft distinguishes in-process domain events within one domain from asynchronous integration events used to propagate committed updates outside a microservice. Microsoft also describes integration events as data-holding contracts decoupled from other services. RFC 9457 defines machine-readable Problem Details for HTTP APIs, makes the type URI the primary problem identifier, and warns that problem details are not a debugging channel and must not expose implementation or sensitive information. The exact NEXUS-1 mappings, names, payloads and evidence gates remain project decisions.

Official source	Stable point used here	NEXUS-1 boundary
Microsoft Learn - Domain events: design and implementation	Domain events make local domain side effects explicit; integration events propagate successfully persisted updates asynchronously outside the microservice.	Private events never become wire contracts automatically; eligible public facts are mapped and staged under the owner commit.
Microsoft Learn - Integration event-based microservice communications	Integration events communicate state across independent microservices and are data-holding application-level contracts.	Producer owns public meaning; consumers translate into their own Application/Domain language.
RFC 9457 - Problem Details for HTTP APIs	Problem type is the primary machine identifier; details/extensions require security care and must not expose implementation internals.	API failures use stable safe public semantics; Chapter 23 implements exact ASP.NET Core behavior.

Sources: <https://learn.microsoft.com/en-us/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/domain-events-design-implementation> ;
<https://learn.microsoft.com/en-us/dotnet/architecture/microservices/multi-container-microservice-net-applications/integration-event-based-microservice-communications> ;
<https://www.rfc-editor.org/rfc/rfc9457.html>

Technical material still required

The three vocabularies, AlarmManagement-to-RootCause mapping path, publication eligibility, public payload, anti-corruption input, API request/response boundary, version classification, field security rules, evidence IDs and ADR are specified. No NEXUS-1 contract assembly has compiled; no mapper, serializer, envelope, HTTP endpoint, OpenAPI document, broker adapter, schema fixture or old/new producer-consumer pair has executed.

TECHNICAL MATERIAL STATUS - [DOMAIN/INTEGRATION/API CONTRACT IMPLEMENTATION AND COMPATIBILITY EVIDENCE REQUIRED]

Implement the net8.0 Domain, Application.Contracts, Integration.Contracts and Edge contract surfaces for AlarmManagement and RootCause without reference cycles; compile the private AlarmFloodQualified and VerdictIssued events, public AlarmFloodDetected.v1 and RootCauseVerdictIssued.v1 payloads, HTTP request/response/problem models and every explicit mapper; bind immutable public payloads to Chapter 18 TX-A/TX-B and Chapter 20 envelope identities; execute eligible/suppressed/rejected publication, mapping boundary, unsupported version, changed payload, unknown enum/unit, lost response, duplicate/replay/restore, privacy/redaction and old/new producer-consumer/API matrices; publish CTR-OWN-001 through CTR-REVIEW-001 linked to TXN, ID, FLOW and later ENV/API/COMP evidence. RootCause remains G0 accepted and G1 specified; runtime evidence has not advanced.

Deliverable	Minimum technical content	Blocking evidence
Private events	Owner-only immutable event types, raise/dispatch phase and local effect catalogue.	EVT-DOM-001 / dependency scan / domain tests
Public events	Stable semantic type/version, approved payload and producer responsibility.	EVT-PUB-001 / CTR-OWN-001
Eligibility/mapping	Eligible/suppressed/rejected policy and deterministic no-leak mappers.	EVT-ELIG-001 / EVT-MAP-001 / SEC-DATA-001
Consumer boundary	Envelope/payload validation and RootCause anti-corruption mapping before Domain.	CNS-ACL-001 / ID-IDEM-001
API contracts	Request/response/problem/status semantics with no entity or remote-success leak.	API-REQ/RESP/ERR-001
Evolution	Meaning-based event/API versions, migration/deprecation and retained replay fixtures.	EVT-VERS-001 / Chapter 24 COMP evidence
Time/security	Distinct timestamp roles and field classification/retention/redaction rules.	EVT-TIME-001 / SEC-DATA-001
Acceptance	Machine results across supported versions bound to build/schema/fixture identities and review.	CTR-TEST-001 / CTR-REVIEW-001

Implementation sequence

1. Accept ADR-017 and publish the contract ownership/audience/lifetime catalogue for Domain, integration and API vocabularies.
2. Create reference project boundaries that prevent Domain from depending on Integration.Contracts, Edge or Infrastructure and prevent public contracts from referencing EF/domain types.
3. Implement and test private AlarmFloodQualified and VerdictIssued domain events with explicit raise/dispatch phases under Chapter 18 transactions.
4. Define publication eligibility policies and field-level data classifications for AlarmFloodDetected.v1 and RootCauseVerdictIssued.v1.
5. Implement deterministic public mappers and stage immutable payload intent with stable PublicFactIdentity values in the owner commit.
6. Implement RootCause anti-corruption mapping from the accepted AlarmFloodDetected.v1 contract into AcceptAlarmFlood before Domain behavior.

CHAPTER 19 - CODE AUDIT

Three vocabularies compile as three different boundaries

The executable design prevents a private domain occurrence, a producer-owned public fact and an HTTP request or response from becoming the same CLR type. References flow inward through explicit mappers; ownership never flows outward.

EVIDENCE STATUS

These listings are compile-oriented reference material. No contract package, serializer, endpoint host, OpenAPI document, broker path or compatibility suite has executed here. CTR, EVT, CNS and API evidence remains PENDING.

EXECUTABLE ASSET 19-A - REFERENCE PROJECT TOPOLOGY

```
// Nexus1.AlarmManagement.Domain
public interface IDomainEvent { }
public sealed class AlarmManagementDomainAssembly { }

// Nexus1.AlarmManagement.Application
public sealed class AlarmManagementApplicationAssembly { }

// Nexus1.AlarmManagement.Integration.Contracts
public sealed class AlarmManagementIntegrationContractsAssembly { }

// Nexus1.AlarmManagement.Edge.Contracts
public sealed class AlarmManagementEdgeContractsAssembly { }

// Nexus1.RootCause.Application
public sealed class RootCauseApplicationAssembly { }

public static class ContractProjectRules
{
    public static readonly string[] PublicContractForbidden =
    [
        "Microsoft.EntityFrameworkCore",
        "Nexus1.AlarmManagement.Domain",
        "Nexus1.AlarmManagement.Infrastructure",
        "Nexus1.RootCause.Domain"
    ];
}
```

EXECUTABLE ASSET 19-B - ARCHITECTURE GATES

```
[Fact]
public void CTR_OWN_001_Domain_does_not_reference_public_or_edge_contracts()
{
    var assembly = typeof(AlarmManagementDomainAssembly).Assembly;
    foreach (var forbidden in new[]
    {
        "Integration.Contracts",
        "Edge.Contracts",
        "Infrastructure"
    })
    {
        Types.InAssembly(assembly).Should()
            .NotHaveDependencyOn(forbidden).GetResult()
            .IsSuccessful.Should().BeTrue();
    }
}

[Theory]
[MemberData(nameof(PublicContractAssemblies))]
public void public_contracts_reference_no_domain_or_EF_assembly(
    Assembly assembly)
{
    foreach (var forbidden in ContractProjectRules.PublicContractForbidden)
    {
        Types.InAssembly(assembly).Should()
            .NotHaveDependencyOn(forbidden).GetResult()
            .IsSuccessful.Should().BeTrue();
    }
}
```

PRIVATE DOMAIN OCCURRENCES

The aggregate raises owner-local meaning, not a wire promise

EXECUTABLE ASSET 19-C - PRIVATE EVENT AND AGGREGATE BUFFER

```
internal sealed record AlarmFloodQualified(
    AlarmFloodDecisionIdentity Decision,
    AlarmFloodIdentity Flood,
    SiteIdentity Site,
    TimeWindow Window,
    AlarmCount QualifiedAlarmCount,
    AlarmSeverity Severity,
    PolicyIdentity QualificationPolicy,
    DateTimeOffset OccurredAtUtc) : IDomainEvent;

public sealed class AlarmFloodDecision
{
    private readonly List<IDomainEvent> _uncommitted = [];

    public AlarmFloodDecisionOutcome Qualify(
        AlarmFloodCandidate candidate,
        IAlarmFloodPolicy policy,
        DateTimeOffset occurredAtUtc)
    {
        var outcome = policy.Evaluate(candidate);
        if (!outcome.IsQualified) return outcome;

        ApplyQualifiedState(outcome);
        _uncommitted.Add(new AlarmFloodQualified(
            Identity,
            outcome.Flood,
            candidate.Site,
            candidate.Window,
            outcome.QualifiedAlarmCount,
            outcome.Severity,
            policy.Identity,
            occurredAtUtc));
        return outcome;
    }

    internal IReadOnlyList<IDomainEvent> DequeueUncommittedEvents()
    {
        var events = _uncommitted.ToArray();
        _uncommitted.Clear();
        return events;
    }
}
```

EXECUTABLE ASSET 19-D - EXPLICIT IN-TRANSACTION DISPATCH PHASE

```
public async Task<AlarmDecisionOutcome> HandleAsync(
    EvaluateAlarmFlood command,
    CancellationToken cancellationToken)
{
    var decision = await repository.DemandAsync(
        command.DecisionIdentity, cancellationToken);
    var outcome = decision.Qualify(
        command.Candidate, policy, clock.GetUtcNow());
    var domainEvents = decision.DequeueUncommittedEvents();

    await localDomainEvents.DispatchAsync(
        domainEvents, cancellationToken);
    foreach (var domainEvent in domainEvents)
    {
        var draft = publication.TryCreateDraft(
            domainEvent, command.PublicFactIdentity, command.Causality);
        if (draft is not null) db.PendingPublicFactDrafts.Add(draft);
    }

    await db.SaveChangesAsync(cancellationToken);
    return AlarmDecisionOutcome.From(outcome);
}

[Fact]
public async Task EVT_DOM_001_suppressed_decision_has_no_public_draft()
{
    await fixture.ExecuteBelowThresholdDecisionAsync();
    fixture.PrivateEvents.Should().BeEmpty();
    fixture.PendingPublicFactDrafts.Should().BeEmpty();
    fixture.BrokerCalls.Should().Be(0);
}
```

PRIVATE DOES NOT MEAN UNSTRUCTURED

The event is immutable and testable, but it remains owner-local. Only an explicit eligibility decision and mapper may create a separate public contract draft.

PUBLICATION ELIGIBILITY AND MAPPING

A producer publishes only approved past-tense meaning

EXECUTABLE ASSET 19-E - ELIGIBILITY IS A CLOSED OUTCOME

```
public enum PublicationDecisionKind
{
    Eligible,
    Suppressed,
    Rejected
}

public sealed record PublicationDecision(
    PublicationDecisionKind Kind,
    string PolicyIdentity,
    string ReasonCode);

public sealed class AlarmFloodPublicationPolicy(
    PublicFieldCatalogue fields)
{
    public PublicationDecision Evaluate(AlarmFloodQualified source)
    {
        if (!fields.IsApproved("siteId")
            || !fields.IsApproved("qualifiedAlarmCount")
            || !fields.IsApproved("severity")
            || !fields.IsApproved("qualificationPolicy"))
        {
            return new(
                PublicationDecisionKind.Rejected,
                "alarm-flood-publication/1",
                "field-catalogue-not-approved");
        }

        return source.Severity is
            AlarmSeverity.Warning or AlarmSeverity.Critical
            ? new(PublicationDecisionKind.Eligible,
                "alarm-flood-publication/1", "qualified")
            : new(PublicationDecisionKind.Suppressed,
                "alarm-flood-publication/1", "below-public-severity");
    }
}
```

EXECUTABLE ASSET 19-F - DETERMINISTIC PUBLIC MAPPER

```
public enum AlarmSeverityV1 { Warning, Critical }

public sealed record AlarmFloodDetectedV1(
    string AlarmFloodId,
    string SiteId,
    DateTimeOffset WindowStartedAtUtc,
    DateTimeOffset WindowEndedAtUtc,
    int QualifiedAlarmCount,
    AlarmSeverityV1 Severity,
    string QualificationPolicy);

public sealed class AlarmFloodDetectedV1Mapper(
    AlarmFloodPublicationPolicy eligibility)
{
    public PublicContractMapping<AlarmFloodDetectedV1> Map(
        AlarmFloodQualified source)
    {
        var decision = eligibility.Evaluate(source);
        if (decision.Kind != PublicationDecisionKind.Eligible)
            return PublicContractMapping<AlarmFloodDetectedV1>
                .WithoutPayload(decision);

        var severity = source.Severity switch
        {
            AlarmSeverity.Warning => AlarmSeverityV1.Warning,
            AlarmSeverity.Critical => AlarmSeverityV1.Critical,
            _ => throw new UnrepresentablePublicSeverity(source.Severity)
        };
        return PublicContractMapping<AlarmFloodDetectedV1>.Eligible(
            new(
                source.Flood.Value,
                source.Site.PublicValue,
                source.Window.StartUtc,
                source.Window.EndUtc,
                source.QualifiedAlarmCount.Value,
                severity,
                source.QualificationPolicy.Value),
            decision);
    }
}
```

MAPPING IS A POLICY BOUNDARY

The mapper copies approved semantics, converts typed values deliberately and throws for an unrepresentable public value. It never serializes a domain event or EF row.

OWNER COMMIT AND IMMUTABLE DRAFT

The public payload is created once before the owner commit

EXECUTABLE ASSET 19-G - CONTRACT DRAFT STAGING

```
public readonly record struct IntegrationContractIdentity(
    string EventType,
    int SchemaVersion);

public sealed record PendingPublicFactDraft<TPayload>(
    PublicFactIdentity Fact,
    IntegrationContractIdentity Contract,
    DateTimeOffset OccurredAtUtc,
    CausalityContext Causality,
    TPayload Payload,
    string MappingPolicyIdentity);

public sealed class AlarmFloodPublicationMapper(
    AlarmFloodDetectedV1Mapper mapper)
{
    public PendingPublicFactDraft<AlarmFloodDetectedV1>? TryCreateDraft(
        IDomainEvent domainEvent,
        PublicFactIdentity fact,
        CausalityContext causality)
    {
        if (domainEvent is not AlarmFloodQualified qualified)
            return null;
        var mapped = mapper.Map(qualified);
        if (mapped.Decision.Kind == PublicationDecisionKind.Suppressed)
            return null;
        if (mapped.Decision.Kind == PublicationDecisionKind.Rejected)
            throw new PublicContractMappingRejected(
                mapped.Decision.ReasonCode);

        return new(
            fact,
            new(
                "nexus1.alarm-management.alarm-flood-detected.v1",
                1),
            qualified.OccurredAtUtc,
            causality,
            mapped.DemandPayload(),
            mapped.Decision.PolicyIdentity);
    }
}
```

EXECUTABLE ASSET 19-H - MAP-ONCE AND ATOMICITY GATES

```
[Fact]
public void EVT_MAP_001_same_private_event_maps_identically()
{
    var source = fixture.AcceptedAlarmFloodQualified;
    var first = fixture.Mapper.Map(source);
    var second = fixture.Mapper.Map(source);
    second.Should().BeEquivalentTo(first);
}

[Fact]
public async Task eligible_payload_and_decision_commit_together()
{
    await fixture.ExecuteEligibleDecisionAsync();
    var state = await fixture.ReadAlarmAtomicSetAsync();
    state.Decisions.Should().ContainSingle();
    state.PendingPublicFactDrafts.Should().ContainSingle();
    state.SaveChangesCalls.Should().Be(1);
}

[Fact]
public async Task dispatcher_never_invokes_domain_to_public_mapper()
{
    var stored = await fixture.CommitEligibleDraftAsync();
    await fixture.DispatchStoredDraftTwiceAsync(stored);
    fixture.DomainToPublicMapperCallsAfterCommit.Should().Be(0);
    fixture.DispatchedPublicFactIds.Distinct()
        .Should().ContainSingle().Which.Should().Be(stored.Fact);
}
```

CHAPTER 19 FIXES MEANING; CHAPTER 20 FIXES BYTES

The owner commit retains the typed immutable public draft and its identity. The next chapter defines canonical envelope serialization without remapping history.

CONSUMER ANTI-CORRUPTION BOUNDARY

RootCause validates public meaning before constructing its command

EXECUTABLE ASSET 19-I - ACCEPTED CONTRACT TO APPLICATION INPUT

```
public sealed record AcceptAlarmFlood(
    PublicFactIdentity SourceFact,
    PublicSubjectReference FloodReference,
    PublicSiteReference Site,
    ObservationWindow Window,
    PositiveAlarmCount QualifiedAlarmCount,
    RootCauseSeverity Severity,
    PolicyIdentity ProducerPolicy,
    CorrelationIdentity Correlation,
    CommitIntentId CommitIntent);

public sealed class AlarmFloodDetectedV1AntiCorruptionMapper
{
    public AcceptAlarmFlood Map(
        AcceptedPublicPayload<AlarmFloodDetectedV1> accepted)
    {
        var value = accepted.Payload;
        if (value.WindowStartedAtUtc >= value.WindowEndedAtUtc)
            throw new InvalidPublicObservationWindow();

        var severity = value.Severity switch
        {
            AlarmSeverityV1.Warning => RootCauseSeverity.Investigate,
            AlarmSeverityV1.Critical => RootCauseSeverity.Urgent,
            _ => throw new UnsupportedPublicSeverity(value.Severity)
        };

        return new(
            accepted.Fact,
            PublicReferences.FloodDecision(
                accepted.Fact.Producer, value.AlarmFloodId),
            PublicSiteReference.Parse(value.SiteId),
            ObservationWindow.Create(
                value.WindowStartedAtUtc, value.WindowEndedAtUtc),
            PositiveAlarmCount.Create(value.QualifiedAlarmCount),
            severity,
            PolicyIdentity.Parse(value.QualificationPolicy),
            accepted.Correlation,
            CommitIntentId.From(accepted.Fact));
    }
}
```

EXECUTABLE ASSET 19-J - REJECT-BEFORE-DOMAIN MATRIX

```
[Theory]
[InlineData(0, AlarmSeverityV1.Warning, "count")]
[InlineData(42, (AlarmSeverityV1)999, "severity")]
public void CNS_ACL_001_invalid_public_value_never_reaches_Domain(
    int count, AlarmSeverityV1 severity, string expectedField)
{
    var accepted = fixture.AcceptedV1 with
    {
        Payload = fixture.AcceptedV1.Payload with
        {
            QualifiedAlarmCount = count,
            Severity = severity
        }
    };
    var act = () => fixture.AntiCorruption.Map(accepted);
    act.Should().Throw<PublicContractRejected>()
        .Where(x => x.Field == expectedField);
    fixture.RootCauseDomainCalls.Should().Be(0);
}

[Fact]
public void consumer_command_contains_no_producer_private_key()
{
    var command = fixture.AntiCorruption.Map(fixture.AcceptedV1);
    command.Should().NotBeAssignableTo<AlarmFloodDetectedV1>();
    command.FloodReference.Authority.Should()
        .Be(fixture.AlarmManagementAuthority);
}
```

CONSUMER AUTONOMY BEGINS AT MAPPING

RootCause owns the command, severity interpretation and local policy. Unsupported versions or values are rejected before any aggregate is loaded or changed.

SYNCHRONOUS API TRUTH

A 202 response describes only the accepted owner-local outcome

EXECUTABLE ASSET 19-K - REQUEST, RESPONSE AND SAFE PROBLEM CONTRACTS

```
public sealed record DetectAlarmFloodRequestV1(
    string RequestId,
    string SiteId,
    DateTimeOffset WindowStartedAtUtc,
    DateTimeOffset WindowEndedAtUtc);

public sealed record DetectAlarmFloodAcceptedV1(
    string RequestId,
    string DecisionId,
    string PublicFactId,
    string StatusUri);

public sealed record PublicProblemV1(
    string Type,
    string Title,
    int Status,
    string Code,
    string TraceReference);

public static class AlarmFloodHttpMapper
{
    public static IResult ToHttp(AlarmDecisionOutcome outcome) =>
        outcome switch
        {
            AlarmDecisionOutcome.Accepted accepted =>
                Results.Accepted(
                    $"/v1/alarm-flood-requests/{accepted.RequestId}",
                    new DetectAlarmFloodAcceptedV1(
                        accepted.RequestId.Value,
                        accepted.Decision.Value,
                        accepted.PublicFact.FactId.ToString("D"),
                        $"/v1/alarm-flood-requests/{accepted.RequestId}")),
            AlarmDecisionOutcome.Invalid invalid =>
                Results.Problem(
                    type: "https://nexus1.example/problems/invalid-alarm-window",
                    title: "Alarm window is invalid",
                    statusCode: StatusCodes.Status422UnprocessableEntity,
                    extensions: new Dictionary<string, object?>
                    {
                        ["code"] = invalid.PublicCode
                    },
                    _ => Results.StatusCode(StatusCodes.Status503ServiceUnavailable)
        );
}
```

EXECUTABLE ASSET 19-L - RESPONSE TRUTH AND LEAK TESTS

```
[Fact]
public async Task API_RESP_001_accepted_response_claims_no_downstream_success()
{
    var response = await fixture.PostAcceptedAlarmFloodAsync();
    response.StatusCode.Should().Be(HttpStatusCode.Accepted);
    var json = await response.Content.ReadAsStringAsync();
    json.Should().NotContain("rootCause");
    json.Should().NotContain("caseId");
    json.Should().NotContain("consumerCompleted");
}

[Fact]
public async Task lost_response_retry_returns_same_public_contract()
{
    var first = await fixture.CommitThenLoseResponseAsync();
    var retry = await fixture.PostSameRequestAsync();
    retry.Body.Should().BeEquivalentTo(first.DurableResponse);
    fixture.AlarmDecisionCount.Should().Be(1);
}

[Fact]
public async Task API_ERR_001_problem_contains_no_private_diagnostics()
{
    var response = await fixture.PostInvalidRequestAsync();
    var json = await response.Content.ReadAsStringAsync();
    json.Should().NotContain("SqlException");
    json.Should().NotContain("stackTrace");
    json.Should().NotContain(fixture.PrivateIdentityCanary);
}
```

HTTP SUCCESS IS INTENTIONALLY NARROW

Accepted means AlarmManagement accepted and durably recorded its own operation. It does not promise broker publication, RootCause receipt or case creation.

COMPATIBILITY BY MEANING

A version matrix executes every supported producer-consumer pair

EXECUTABLE ASSET 19-M - CHANGE CLASSIFICATION

```
public enum ContractChangeKind
{
    CompatibleCandidate,
    NewVersionRequired,
    Forbidden
}

public sealed record ContractFieldChange(
    string Field,
    bool WasRequired,
    bool IsRequired,
    string OldMeaning,
    string NewMeaning,
    string OldUnit,
    string NewUnit,
    bool PrivacyExpanded);

public static class ContractChangeClassifier
{
    public static ContractChangeKind Classify(
        ContractFieldChange change)
    {
        if (change.PrivacyExpanded)
            return ContractChangeKind.Forbidden;
        if (change.WasRequired != change.IsRequired
            || !StringComparer.Ordinal.Equals(
                change.OldMeaning, change.NewMeaning)
            || !StringComparer.Ordinal.Equals(
                change.OldUnit, change.NewUnit))
            return ContractChangeKind.NewVersionRequired;
        return ContractChangeKind.CompatibleCandidate;
    }
}
```

EXECUTABLE ASSET 19-N - SUPPORTED CONTRACT CORPUS

```
public sealed record CompatibilityCase(
    string ProducerVersion,
    string ConsumerVersion,
    string Fixture,
    ExpectedCompatibility Expected);

public static IEnumerable<object[]> SupportedPairs()
{
    yield return
    [
        new CompatibilityCase("alarm-v1", "rootcause-v1",
            "alarm-flood-v1-minimal.json", ExpectedCompatibility.Accept)
    ];
    yield return
    [
        new CompatibilityCase("alarm-v1", "rootcause-v2",
            "alarm-flood-v1-minimal.json", ExpectedCompatibility.Accept)
    ];
    yield return
    [
        new CompatibilityCase("alarm-v2", "rootcause-v1",
            "alarm-flood-v2-required-unit-change.json",
            ExpectedCompatibility.RejectUnsupportedVersion)
    ];
}

[Theory]
[MemberData(nameof(SupportedPairs))]
public async Task EVT_VERS_001_supported_matrix_is_explicit(
    CompatibilityCase test)
{
    var result = await fixture.ExecutePairAsync(test);
    result.Compatibility.Should().Be(test.Expected);
    result.RootCauseDomainCalls.Should().Be(
        test.Expected == ExpectedCompatibility.Accept ? 1 : 0);
}

[Fact]
public void additive_field_is_only_a_candidate_until_clients_execute()
{
    var change = fixture.OptionalAdditiveField();
    ContractChangeClassifier.Classify(change).Should()
        .Be(ContractChangeKind.CompatibleCandidate);
    fixture.CompatibilityEvidence.IsAccepted(change).Should().BeFalse();
}
```

COMPATIBLE IS AN EXECUTED CLAIM

Static classification selects the required evidence. Only serializer, validator, consumer and generated-client results can accept a supported compatibility window.

REPLAY, RESTORE AND ACCEPTANCE

Historical public meaning survives private refactoring

EXECUTABLE ASSET 19-O - RETAINED MAPPER AND REPLAY GATE

```
public sealed class RetainedPublicContractRegistry(
    IReadOnlyDictionary<IntegrationContractIdentity,
    IRetainedPublicContractAdapter> adapters)
{
    public IRetainedPublicContractAdapter Demand(
        IntegrationContractIdentity contract) =>
        adapters.TryGetValue(contract, out var adapter)
        ? adapter
        : throw new UnsupportedPublicContract(contract);
}

[Fact]
public async Task CTR_TEST_001_v1_replay_survives_domain_refactor()
{
    var historical = fixture.Corporus.Read(
        "alarm-flood-v1-minimal.json");
    fixture.ReplacePrivateAlarmFloodAggregateImplementation();

    var outcome = await fixture.ReplayThroughRetainedV1AdapterAsync(
        historical);
    outcome.ApplicationCommand.Should()
        .BeEquivalentTo(fixture.ExpectedAcceptAlarmFlood);
    outcome.RootCauseEffects.Should().ContainSingle();
}

[Fact]
public async Task restore_reuses_contract_identity_and_public_fact()
{
    var before = await fixture.ContractHistoryBaselineAsync();
    await fixture.RestoreReplayAndReconcileAsync();
    var after = await fixture.ContractHistoryBaselineAsync();
    after.Should().BeEquivalentTo(before);
    fixture.NewPublicFactIdsDuringReplay.Should().BeEmpty();
}
```

EXECUTABLE ASSET 19-P - ACCEPTANCE EVIDENCE RECORD

```
{
  "evidenceId": "CTR-REVIEW-001", "chapter": 19,
  "artifactCommit": null, "sdk": "net8.0",
  "contractPackage": null, "serializer": null,
  "openApiDigest": null, "broker": null,
  "tests": {
    "EVT-DOM-001": "PENDING", "EVT-PUB-001": "PENDING",
    "EVT-ELIG-001": "PENDING", "EVT-MAP-001": "PENDING",
    "CNS-ACL-001": "PENDING", "API-REQ-001": "PENDING",
    "API-RESP-001": "PENDING", "API-ERR-001": "PENDING",
    "EVT-VERS-001": "PENDING", "EVT-TIME-001": "PENDING",
    "SEC-DATA-001": "PENDING", "CTR-TEST-001": "PENDING"
  },
  "supportedPairsExecuted": false,
  "restoreRehearsed": false, "accepted": false
}
```

Chapter 19 code-audit result

The chapter now exposes one executable path from a private owner occurrence to an eligible producer contract, owner-commit draft, consumer-owned command, truthful HTTP response and retained compatibility/replay corpus.

CURRENT VERDICT

The contract separation is implementable in shape, but no package, serializer, endpoint, OpenAPI document, broker path or version matrix has executed. All contract evidence remains explicitly PENDING.

- 7. Define DetectAlarmFlood request/response and stable safe problem semantics without exposing entities, tokens or downstream-success claims.
- 8. Classify every proposed change by meaning, requiredness, units, enums, identity and privacy; define supported parallel-version/deprecation windows.
- 9. Bind Chapter 20 envelope/serialization and Chapters 21/23 HTTP implementations to the accepted contracts without changing their meaning.
- 10. Execute Chapter 24 compatibility, replay, restore and security matrices; publish CTR evidence without advancing unproved gates.

What this chapter has proved

Chapter 19 has separated private owner occurrences, producer-owned public facts and synchronous HTTP contracts. It has made publication eligibility and mapping explicit, shown one complete AlarmManagement-to-RootCause translation, preserved consumer autonomy, bounded API success claims, classified public evolution by meaning rather than implementation version, and defined field/time/security/evidence obligations.

It has not compiled a contract project, raised or dispatched a domain event, serialized an envelope, staged or published a payload, accepted a broker message, mapped a RootCause input, served an HTTP endpoint, generated OpenAPI, returned Problem Details or executed a compatibility matrix. Contract semantics are specified, not implemented or evidenced. RootCause remains G0 accepted and G1 specified.

SEPARATE LANGUAGE, STABLE PROMISE

Private models may evolve at owner speed. Public facts and APIs evolve only through their accepted compatibility obligations. Explicit mapping is the boundary that makes both statements true.

Close each public contract with one record

Catalogue field	Accepted record	Why it must be explicit
Owner / audience	Producer/provider, intended consumers/clients and named reviewers.	Prevents a shared model with no accountable semantic owner.
Meaning / non-meaning	What fact/request/result is promised and which inferences are forbidden.	Stops event-as-command and API remote-success lies.
Identity / time	Public fact/request/resource identity plus occurrence/record/publication semantics.	Protects dedup, replay, audit and ordering from transport IDs/clocks.
Fields / classification	Name, type, requiredness, units, source, privacy, retention and redaction for every field.	Makes mapping, security and change impact enumerable.
Mapping / eligibility	Source/target, policy identity, deterministic transforms and failure behavior.	Prevents direct entity serialization and silent coercion.
Version / lifecycle	Compatibility policy, supported versions, deprecation, replay/restore and migration path.	Separates public history from CLR/database/deployment changes.
Evidence / review	Fixtures, matrices, machine results, owner/reviewer, expiry and triggers.	Binds the promise to executed producer/consumer/API behavior.

Chapter checkpoint

You are ready to implement contracts when every type belongs to exactly one vocabulary, every public fact has a producer-owned past-tense meaning, every API status has a bounded owner-local claim, publication eligibility and mapping are explicit, consumers translate before Domain behavior, public fields carry units/privacy/lifecycle rules, and version changes are classified and tested against supported history.

Question	Expected answer
Who consumes a domain event?	Handlers inside the owning bounded context under an explicit local transaction/recovery phase; never an external broker consumer by default.
Is a domain event automatically an integration event?	No. Publication eligibility and an explicit reviewed mapper create a separate public promise.
Who owns an integration event?	The producer owns its public meaning and compatibility obligation; consumers own their independent reactions.
May a public event tell RootCause to open a case?	Not as a fact contract. Publish what AlarmManagement knows happened; use an explicit command protocol only if that responsibility is intentionally designed.
May the mapper serialize an EF entity or private event directly?	No. It constructs a distinct contract from approved semantics and rejects unsafe/unrepresentable values.
When is the public payload created?	After the producer domain decision and eligibility check, before immutable pending public intent commits; dispatch never remaps history.
What does RootCause consume?	A validated public envelope/payload translated into RootCause Application-owned input before Domain behavior.
What may an AlarmManagement 2xx response prove?	Only the accepted provider-local operation outcome; not broker, RootCause or downstream completion.
Does an added JSON field guarantee compatibility?	No. It is a hypothesis that must pass supported serializer, validator, generated-client and consumer tests.
What forces a new public version?	A breaking change in meaning, requiredness, units, enums, identity, representation or supported privacy/lifecycle behavior.
Is an assembly or database schema version a contract version?	No. Public contract versions follow accepted public meaning and compatibility obligations.
How are API failures represented?	By stable safe HTTP problem semantics; domain exceptions, stack traces, SQL/provider details and secrets remain private.
What is the current status?	Contracts and mappings are specified, not compiled/serialized/executed; RootCause remains G0 accepted and G1 specified.
What does Chapter 20 add?	The canonical versioned message envelope and exact serialization/compatibility rules for public messages.

NEXT CHAPTER

Chapter 20, The Versioned Message Envelope, will implement the canonical envelope around each public payload. It will define and test message identity, event type, schema version, occurrence time, correlation, causation, producer, serialization, canonical fingerprint and compatibility behavior without allowing broker metadata or CLR type names to become business identity.

PART IV - PUBLIC CONTRACTS AND THE SYNCHRONOUS EDGE

20. The Versioned Message Envelope

A public event is durable history only when every delivery attempt carries the same identity and accepted representation. The envelope makes that history inspectable: it names the fact, its public type and version, its producer, occurrence time and causal context without turning broker metadata, CLR names or tracing spans into business truth.

CHAPTER ARGUMENT

Build the envelope once from a Chapter 19 public fact, serialize and fingerprint it under explicit deterministic rules, persist the exact bytes in the producer transaction, and validate identity, integrity, version and payload before any consumer behavior. Retry, redelivery, replay and restore reuse the fact; they never manufacture a replacement.

Chapter 19 fixed the meaning of AlarmFloodDetected.v1. Chapter 20 now fixes how that public fact is carried. The canonical NEXUS-1 envelope contains stable public fact identity, semantic event type, schema version, occurrence time, producer, correlation, causation, content type, a content fingerprint and the producer-approved payload. Broker exchange, routing key, delivery tag, redelivery flag, queue, trace span and client library headers remain delivery or diagnostic concerns outside the fact contract.

The complete walkthrough preserves the identities already introduced: request-am-20482 identifies the initiating HTTP request and correlation context; decision-am-9184 identifies the immediate AlarmManagement decision; fact-am-7716 identifies the one public fact; case-rc-3107 is created or resolved only inside RootCause. A retry may receive a new AMQP delivery tag and tracing span, but it keeps fact-am-7716, the accepted fingerprint and the same serialized bytes.

This is a net8.0 reference specification, not an implemented runtime. The listings require a real canonicalizer, source-generated serializer contexts, broker adapters, storage mappings and executed fixtures. Chapter 25 owns RabbitMQ topology; Chapters 26 and 27 own outbox storage/dispatch; Chapter 28 owns the production inbox consumer. RootCause remains G0 accepted and G1 specified.

Separate the fact from its delivery mechanism

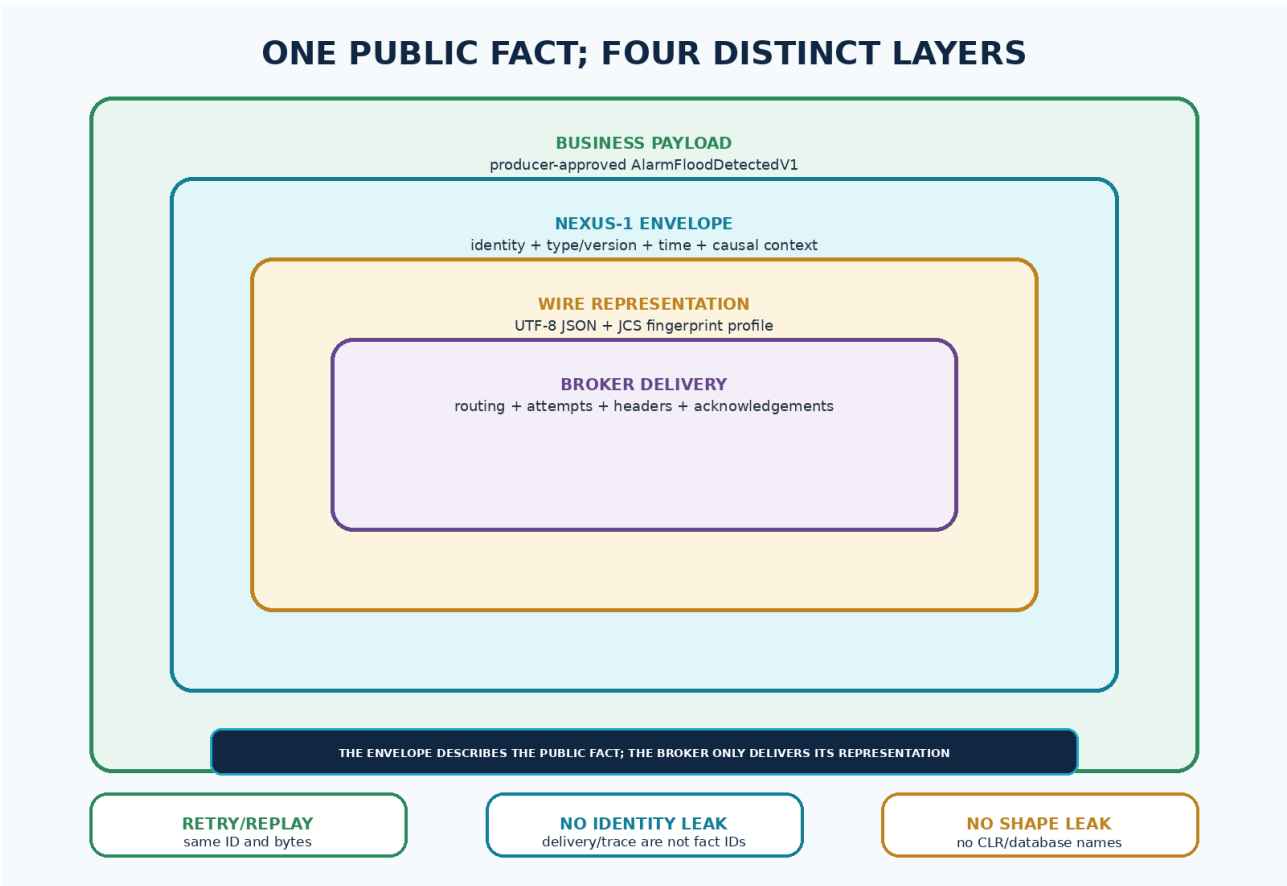


Figure 20.1 - Payload, NEXUS-1 envelope, wire representation and broker delivery are separate layers with separate owners.

Layer	Owner	Stable content	Must not become its identity
Business payload	Producing bounded context under Chapter 19 contract ownership.	AlarmFloodDetectedV1 meaning, approved business identities, units and occurrence semantics.	EF/domain types, internal row keys, private evidence or consumer commands.
NEXUS-1 envelope	Cross-service contract governance; producer instantiates it.	Message identity, public type/version, producer, time, correlation, causation, content type and fingerprint.	Assembly version, CLR namespace, deployment number, queue or transport delivery tag.
Wire representation	Versioned codec owned by the envelope contract.	UTF-8 JSON shape, number/string/date rules, canonicalization and media type.	Pretty-print whitespace, property insertion order or serializer defaults.
Broker delivery	Message-backbone implementation in Part V.	Routing, durability, headers, acknowledgements, delivery attempts and observability metadata.	Public fact identity, business order, authenticity or consumer completion.

BROKER-NEUTRAL BY DESIGN

The same envelope bytes can be stored in an outbox, published through RabbitMQ, copied to an audit corpus or replayed through a test harness. A protocol binding may duplicate selected envelope fields into broker properties for routing and diagnostics; the envelope remains authoritative.

The canonical envelope field set

JSON member	Example	Required meaning and validation
envelopeVersion	1	Positive integer selecting the outer metadata/serialization contract. It does not version the business payload.
messageId	fact-am-7716	Producer-stable public fact identity. Retry, replay and restore reuse it; consumer dedup begins with {producer,messageId}.

JSON member	Example	Required meaning and validation
eventType	nexus1.alarm-management.alarm-flood-detected.v1	Lowercase semantic public name ending in .v{schemaVersion}; never a CLR or assembly-qualified type.
schemaVersion	1	Positive payload-contract version. It must agree with eventType and registry entry before payload deserialization.
occurredAtUtc	2026-07-19T07:43:11.482Z	Producer-defined business occurrence/decision time, normalized to UTC with explicit precision; not publication or broker time.
producer	alarm-management	Stable authenticated logical producer name from an allow-list; not a pod, host, assembly or deployment instance.
correlationId	request-am-20482	Opaque workflow grouping token. Useful for search; not uniqueness, order or causality proof.
causationId	decision-am-9184	Identity of the immediate accepted cause known to the producer. It may name a request, message or decision under a typed policy.
contentType	application/json	Payload representation understood by the registered codec. Charset is UTF-8 by envelope v1 rule.
fingerprintAlgorithm	sha-256+jcs	Algorithm/profile identifier. Envelope v1 permits only the reviewed profile; no silent algorithm negotiation.
contentFingerprint	sha256:8d8b...d71f	Lowercase digest over the RFC 8785-canonical envelope material with this member omitted. Integrity check, not authentication.
payload	{ ... }	One JSON object matching the registered eventType/schemaVersion contract and Chapter 19 field policy.

Envelope v1 deliberately has no open-ended top-level extension bag. Unknown envelope members are rejected because they can change security and processing semantics. Additive evolution belongs inside a governed payload version when compatibility is proved, or in a new envelopeVersion when outer semantics change. Trace context is propagated as diagnostic metadata under the W3C format; it is never hashed into business identity or required for replay correctness.

A complete AlarmFloodDetected envelope

IMPLEMENTATION SHAPE

```
{
  "envelopeVersion": 1,
  "messageId": "fact-am-7716",
  "eventType": "nexus1.alarm-management.alarm-flood-detected.v1",
  "schemaVersion": 1,
  "occurredAtUtc": "2026-07-19T07:43:11.482Z",
  "producer": "alarm-management",
  "correlationId": "request-am-20482",
  "causationId": "decision-am-9184",
  "contentType": "application/json",
  "fingerprintAlgorithm": "sha-256+jcs",
  "contentFingerprint": "sha256:<64 lowercase hexadecimal digits>",
  "payload": {
    "alarmFloodId": "flood-am-6031",
    "siteId": "site-ath-17",
    "windowStartedAtUtc": "2026-07-19T07:38:00.000Z",
    "windowEndedAtUtc": "2026-07-19T07:43:00.000Z",
    "qualifiedAlarmCount": 42,
    "severity": "critical",
    "qualificationPolicy": "alarm-flood-policy/3"
  }
}
```

The displayed fingerprint is intentionally a placeholder because this chapter has not executed a canonicalization implementation. A release fixture must contain the exact computed digest and byte sequence. Publishing a decorative or hand-calculated digest would be false evidence.

Reference types keep envelope and payload versions independent

IMPLEMENTATION SHAPE

```
using System.Text.Json;
using System.Text.Json.Serialization;

public sealed record MessageEnvelopeV1<TPayload>(
    [property: JsonPropertyOrder(0)] int EnvelopeVersion,
    [property: JsonPropertyOrder(1)] string MessageId,
    [property: JsonPropertyOrder(2)] string EventType,
    [property: JsonPropertyOrder(3)] int SchemaVersion,
    [property: JsonPropertyOrder(4)] DateTimeOffset OccurredAtUtc,
    [property: JsonPropertyOrder(5)] string Producer,
    [property: JsonPropertyOrder(6)] string CorrelationId,
    [property: JsonPropertyOrder(7)] string CausationId,
    [property: JsonPropertyOrder(8)] string ContentType,
    [property: JsonPropertyOrder(9)] string FingerprintAlgorithm,
    [property: JsonPropertyOrder(10)] string ContentFingerprint,
    [property: JsonPropertyOrder(11)] TPayload Payload);

public sealed record AlarmFloodDetectedV1(
    string AlarmFloodId,
    string SiteId,
    DateTimeOffset WindowStartedAtUtc,
    DateTimeOffset WindowEndedAtUtc,
    int QualifiedAlarmCount,
    string Severity,
    string QualificationPolicy);
```

PROPERTY ORDER IS NOT CANONICALIZATION

JsonPropertyOrder can make ordinary output predictable for people and fixtures, but it does not implement RFC 8785. Canonical JSON requires defined string escaping, number serialization and lexicographic property ordering throughout nested payloads. Use a reviewed canonicalizer and prove it with official and project vectors.

One build; one immutable representation

1. Accept the Chapter 19 AlarmFloodDetectedV1 payload only after the AlarmManagement decision and publication-eligibility policy have succeeded.
2. Allocate fact-am-7716 once inside the producer application flow. Supply occurrence, producer, correlation and causation values from typed, validated sources; do not read broker or current trace state later during dispatch.
3. Construct an envelope with an empty fingerprint value, validate the field registry and canonicalize all remaining envelope members including the payload under the sha-256+jcs profile.
4. Hash the canonical UTF-8 bytes with SHA-256, encode lowercase hexadecimal with the sha256: prefix and create the final immutable envelope.
5. Serialize the final envelope using the version-pinned codec and persist those exact UTF-8 bytes, fingerprint, message identity, event type and schema version in TX-A beside the owner decision.
6. On dispatch, load and publish the stored bytes. Duplicate selected fields into broker properties only after comparing them with the authoritative envelope; never reconstruct from current domain state.

IMPLEMENTATION SHAPE

```
public interface IJsonCanonicalizer
{
    byte[] Canonicalize(JsonElement value);
}

public interface IMessageEnvelopeCodec
{
    byte[] Encode<TPayload>(MessageEnvelopeV1<TPayload> value);
    ValidatedEnvelope DecodeAndValidate(ReadOnlySpan<byte> utf8Json);
}

public sealed class EnvelopeFactory(IJsonCanonicalizer canonicalizer)
{
    public MessageEnvelopeV1<TPayload> Create<TPayload>(
        EnvelopeInputs inputs, TPayload payload)
    {
        var unsigned = inputs.ToEnvelope(payload, contentFingerprint: "");
        EnvelopeRules.Validate(unsigned);

        JsonElement material = FingerprintMaterial.WithoutFingerprint(unsigned);
        byte[] canonical = canonicalizer.Canonicalize(material);
        string digest = "sha256:" + Convert.ToHexString(
            SHA256.HashData(canonical)).ToLowerInvariant();

        return unsigned with { ContentFingerprint = digest };
    }
}
```

The code is deliberately an interface-level reference. `FingerprintMaterial.WithoutFingerprint` must remove only `contentFingerprint`, preserve every other member, reject duplicate member names and use the same source-generated contract as the final codec. A generic serialize-parse-reserialize shortcut is not accepted until its numeric, Unicode, date and nested-object behavior passes the canonical vector corpus.

Canonicalization and fingerprint scope

Concern	Envelope v1 decision	Failure if unspecified
Input grammar	UTF-8 RFC 8259 JSON; reject byte-order mark, invalid sequences, comments, trailing commas and duplicate names at every object depth.	Different parsers accept different messages or disagree on signed/hashed meaning.
Canonical profile	RFC 8785 JCS over the full envelope except <code>contentFingerprint</code> ; payload included; output encoded as UTF-8.	Whitespace/property order/number formatting produce unstable digests.
Numbers	Payload contract restricts numeric ranges and forbids NaN/Infinity; JCS number rules apply; high-precision identifiers use strings.	Rounding or exponent normalization changes business values and fingerprints.
Strings	Unicode strings are validated; contract validators bound length and allowed code points where identifiers/enums require them.	Visually similar or ill-formed strings bypass allow-lists or compare differently.
Dates	RFC 3339-compatible UTC text produced from <code>DateTimeOffset</code> , exactly three fractional digits for NEXUS-1 fixtures.	Equivalent instants acquire several representations or local offsets.
Fingerprint comparison	Require sha256: plus 64 lowercase hex digits; compare decoded bytes in constant time after recomputation.	Case/format ambiguity or timing-sensitive comparison weakens validation.
Persistence	Store final exact envelope bytes and digest in TX-A; dispatch publishes them unchanged.	Retry after deployment regenerates different historical content.
Security meaning	Digest detects content mismatch only. Producer authenticity/authorization comes from secured transport and producer policy; signatures require a separate canonical signed profile.	Consumers treat an attacker-computed digest as proof of origin.

The complete producer-to-consumer path

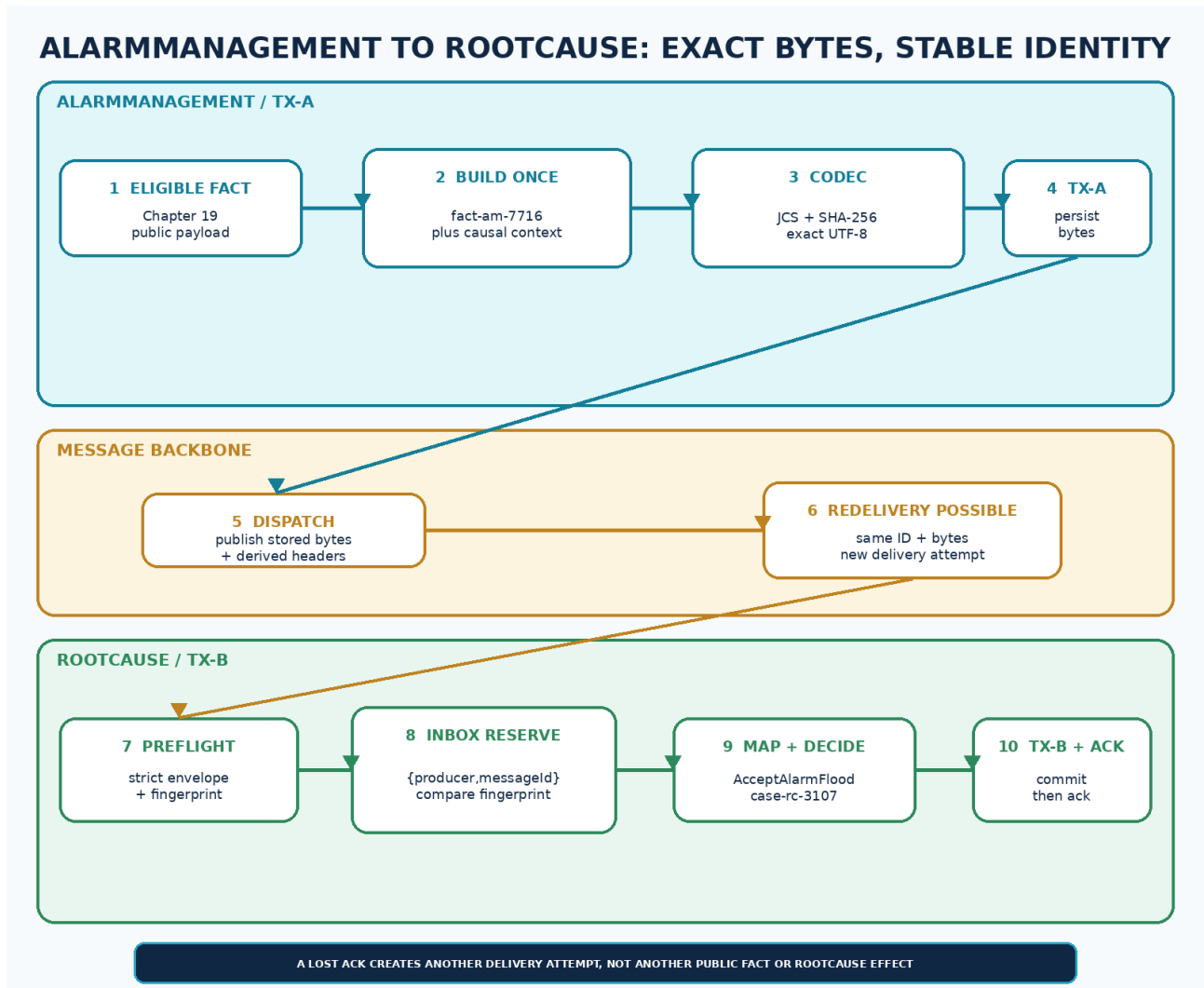


Figure 20.2 - AlarmManagement stages one immutable envelope; RootCause validates, deduplicates and commits one owner-local effect.

Step	Accepted state	Identity/evidence that must survive failure
1. Eligible payload	AlarmManagement owns a committed/commit-eligible AlarmFloodDetectedV1 meaning from Chapter 19.	decision-am-9184; mapper and eligibility policy versions.
2. Envelope creation	One envelope binds fact-am-7716 to event type v1, schema 1 and producer causal context.	messageId, occurredAtUtc, producer, correlationId and causationId.
3. Codec	Canonical material and SHA-256 digest are computed; final UTF-8 JSON is produced.	codec/canonicalizer build, vectors, fingerprint and exact byte length.
4. TX-A	Owner state and pending exact bytes commit atomically; no broker success is claimed.	outbox record, row/concurrency token, messageId and byte hash.
5. Dispatch	Stored bytes are published; broker properties are derived and cross-checked.	outbox attempt and publish confirmation; fact identity unchanged.
6. Redelivery	Lost confirm/ack may repeat delivery with different transport metadata.	same envelope bytes/messageId; delivery tag is attempt-only.
7. Preflight	RootCause rejects malformed, unknown, unsupported, tampered or unsafe input before Domain.	validation result and bounded safe diagnostic code.
8. Inbox	{alarm-management,fact-am-7716} is reserved; same fingerprint is duplicate success, mismatch is collision.	inbox unique constraint, stored event type/version/fingerprint.
9. Map/decide	Validated payload becomes AcceptAlarmFlood; RootCause may create or update case-rc-3107.	anti-corruption mapper and RootCause owner decision.
10. TX-B/ack	RootCause effect and inbox complete together; broker ack happens after commit.	case/inbox transaction evidence and acknowledgement attempt.

Producer codec and storage boundary

IMPLEMENTATION SHAPE

```
[JsonSourceGenerationOptions(  
    PropertyNamingPolicy = JsonKnownNamingPolicy.CamelCase,  
    WriteIndented = false,  
    DefaultIgnoreCondition = JsonIgnoreCondition.Never,  
    NumberHandling = JsonNumberHandling.Strict)]  
[JsonSerializable(typeof(MessageEnvelopeV1<AlarmFloodDetectedV1>))]  
internal partial class AlarmManagementMessageJsonContext : JsonSerializerContext;  
  
public async Task StageAsync(  
    MessageEnvelopeV1<AlarmFloodDetectedV1> envelope,  
    CancellationToken cancellationToken)  
{  
    byte[] exactBytes = codec.Encode(envelope);  
    codec.DecodeAndValidate(exactBytes); // producer self-check, same rules  
  
    await outbox.AddAsync(new PendingPublicMessage(  
        producer: envelope.Producer,  
        messageId: envelope.MessageId,  
        eventType: envelope.EventType,  
        schemaVersion: envelope.SchemaVersion,  
        contentFingerprint: envelope.ContentFingerprint,  
        bodyUtf8: exactBytes), cancellationToken);  
}
```

Source generation reduces reflection ambiguity but does not by itself freeze a wire contract. The project must pin the target framework, serializer options, generated context, payload attributes and canonicalizer version in evidence. Any runtime/library upgrade that changes emitted bytes is a review trigger even when deserialization still succeeds.

Validate before RootCause behavior

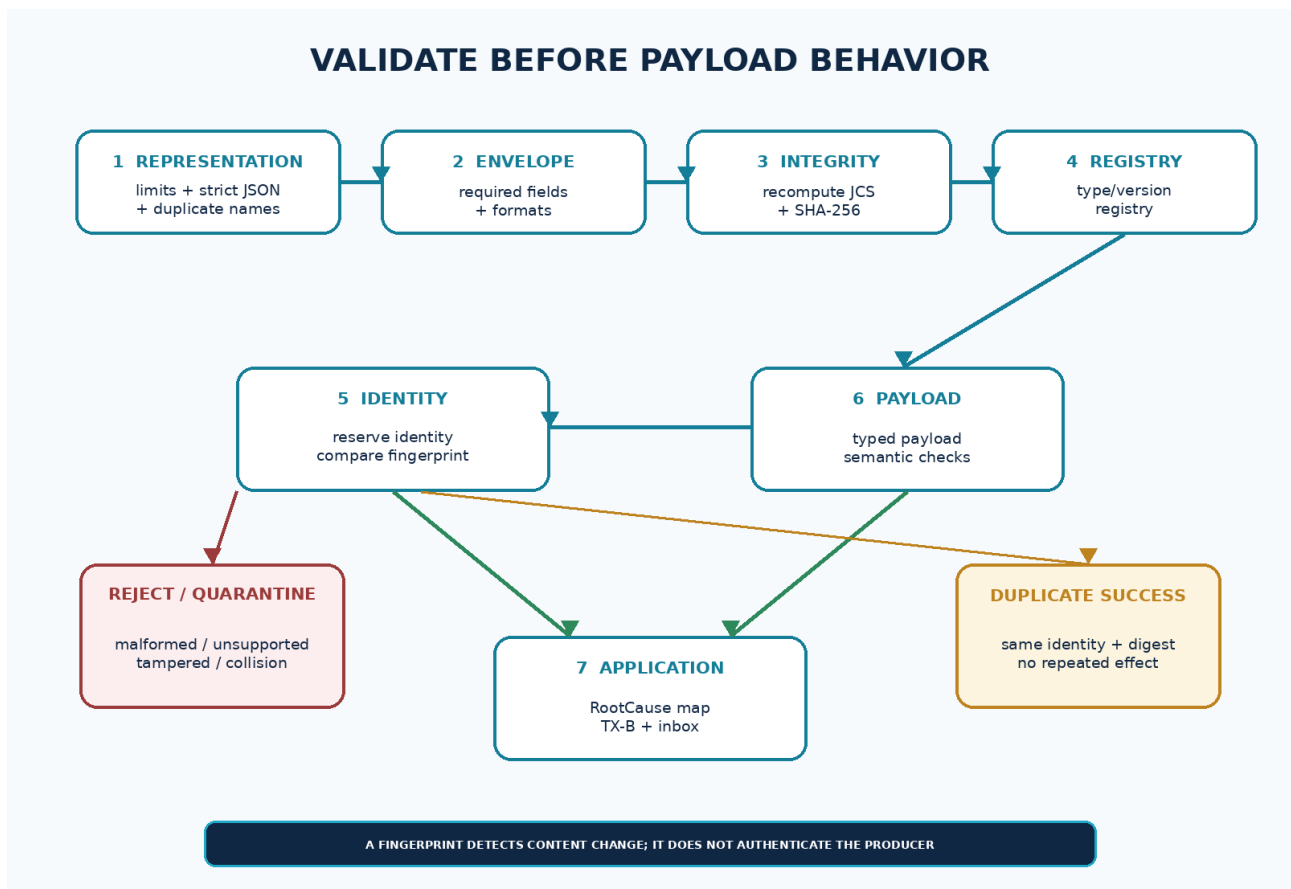


Figure 20.3 - Representation, envelope, integrity, registry and identity checks precede payload mapping and RootCause behavior.

1. Enforce transport/body limits before allocation: maximum body bytes, maximum depth, maximum string/property/array sizes and bounded diagnostics

2. Parse strict UTF-8 JSON and reject comments, trailing commas, duplicate property names, invalid numbers, invalid timestamps and unexpected top-level envelope members.
3. Validate envelopeVersion, eventType/schema agreement, producer allow-list, message/correlation/causation syntax, UTC occurrence time, content type and fingerprint profile.
4. Remove only contentFingerprint, canonicalize the accepted material, recompute SHA-256 and compare digest bytes in constant time.
5. Resolve the exact {eventType,schemaVersion,contentType} registry entry. Unsupported versions never reach a catch-all dynamic serializer or Domain layer.
6. Reserve {producer,messageId}. If a completed inbox row has the same event type, schema and fingerprint, return duplicate success. If any differ, quarantine an identity collision.
7. Deserialize the payload with its registered source-generated metadata, then run structural and semantic validation before the RootCause anti-corruption mapper.
8. Commit the RootCause owner effect and inbox completion in TX-B; acknowledge only afterwards. A lost acknowledgement repeats validation and yields duplicate success.

IMPLEMENTATION SHAPE

```
public ConsumptionDecision Classify(
    ValidatedEnvelope incoming,
    InboxRecord? existing)
{
    if (existing is null)
        return ConsumptionDecision.ReserveAndProcess;

    bool sameFact = existing.EventType == incoming.EventType
        && existing.SchemaVersion == incoming.SchemaVersion
        && CryptographicOperations.FixedTimeEquals(
            existing.FingerprintBytes, incoming.FingerprintBytes);

    return sameFact
        ? ConsumptionDecision.AlreadyCommitted
        : ConsumptionDecision.QuarantineIdentityCollision;
}
```

Failure classification is part of the contract

Condition	Classification/action	What must never happen
Oversize, invalid UTF-8/JSON, duplicate member, excessive depth	Permanent malformed input; reject or bounded quarantine according to Part V policy.	Unbounded parse, retry storm, payload logging or Domain call.
Unknown envelopeVersion	Unsupported outer contract; quarantine with safe code env_version_unsupported.	Best-effort guessing or treating it as v1.
Unknown eventType/schema/contentType	Unsupported public contract; route to compatibility/quarantine policy.	Deserialize into nearest CLR type or silently drop fields.
eventType and schemaVersion disagree	Malformed/ambiguous contract; reject before payload parse.	Choose one discriminator opportunistically.
Fingerprint format invalid or digest mismatch	Integrity failure; quarantine, alert and preserve bounded forensic metadata.	Recompute and replace the sender value or process anyway.
Producer unauthorized for event type	Security violation; reject and alert.	Trust the producer string merely because it is inside JSON.
Same {producer,messageId}, same fingerprint	Idempotent duplicate; acknowledge after confirming prior commit.	Repeat RootCause effects or create a second case.
Same {producer,messageId}, different type/version/fingerprint	Identity collision/tampering; quarantine and page owner.	Overwrite inbox, accept latest or assign a consumer-generated ID.
Payload structurally valid but semantic rule fails	Permanent contract violation with stable safe validation code.	Default enum/unit/time or send invalid values into Domain.
RootCause transient infrastructure failure before TX-B commit	Retryable consumption attempt; no inbox completion.	Acknowledge or record completion without owner effect.
TX-B committed but ack lost	Redelivery becomes duplicate success.	Rollback the committed owner fact or rerun behavior.

Correlation, causation and tracing are not interchangeable

Identifier	Question answered	NEXUS-1 rule
messageId	Which one public fact is this?	Producer-unique within producer scope; immutable; dedup/audit/replay identity.
correlationId	Which larger workflow or operator request groups these facts?	Opaque search/grouping value; may span services; not unique and not an ordering key.
causationId	Which immediately accepted occurrence caused this fact?	Typed policy permits request, message or decision identity; never inferred from timing.
trace-id / span-id	Which sampled diagnostic execution path observed this attempt?	W3C Trace Context at transport/diagnostic edge; a retry/replay may create new spans and a trace may be absent.
broker delivery tag	Which delivery attempt on this channel is being acknowledged?	Transport-local and short-lived; never persisted as message identity.
case-rc-3107	Which RootCause owner aggregate/resource records the reaction?	Consumer-owned business identity created/resolved after validation; never placed retroactively into the producer fact.

NO GLOBAL ORDER FROM TIMESTAMPS

occurredAtUtc explains producer occurrence semantics. It does not prove a total order across producers, broker partitions, retries or restored histories. Consumer decisions use owner invariants, message identity, explicit causal references and domain versions - not wall-clock sorting alone.

Version the correct layer

Change	Version action	Migration/compatibility obligation
Payload gains optional field with unchanged meaning	May remain schemaVersion 1 only if every supported consumer ignores/handles it safely and fixture matrices pass.	Old/new producer-consumer tests; canonical fingerprints naturally differ per message content.
Payload field meaning, units, requiredness, enum behavior or identity changes	New eventType suffix and schemaVersion; for example .v2 / 2.	Parallel registry entries, explicit producer mapping, supported migration/deprecation and historical replay fixtures.
Envelope adds or changes required metadata semantics	New envelopeVersion.	New strict parser/codec profile; dual-read/controlled-write window; old envelope bytes remain immutable.
Serializer implementation changes but accepted bytes remain identical	No public version change after byte-vector proof.	Golden corpus across old/new runtime and canonicalizer builds.
Serializer output changes but canonical meaning/digest remains accepted	Potential codec/evidence change; release only after exact policy review.	Stored historical bytes remain publishable; consumers accept both representations only if envelope v1 rules allow them.
Broker exchange/routing binding changes	No envelope or payload version by itself.	Topology migration and header/body cross-check; public fact identity/bytes unchanged.
Fingerprint algorithm/profile changes	New envelopeVersion unless a predeclared negotiation profile was accepted; v1 has none.	Parallel validation, security review, replay corpus and downgrade protection.
Assembly, deployment or database schema changes	No public version solely for that reason.	Regression evidence proves contract output and historical dispatch/consumption remain accepted.

Envelope strictness and payload extensibility are separate decisions. RootCause uses a strict v1 envelope parser so unrecognized outer semantics cannot slip through. The registered AlarmFloodDetectedV1 payload codec may skip unknown optional members to support a proved additive change, while still requiring known properties, valid enums/units/ranges and semantic constraints. System.Text.Json can skip or disallow unmapped members in .NET 8; NEXUS-1 selects the mode per registered contract rather than relying on a global default.

Replay and restore preserve history

Operation	Required behavior	Forbidden regeneration
Dispatcher retry	Read the same outbox bytes and publish them again under fact-am-7716.	New messageId, current time, remapped payload or new correlation.
Broker redelivery	Consumer revalidates and resolves duplicate through inbox identity/fingerprint.	Second RootCause effect because delivery tag changed.
Operational replay	Select an explicit historical corpus and republish original envelope bytes with replay audit metadata outside the envelope.	Changing occurredAtUtc, payload version or public identity to look current.
Database restore	Restored outbox/inbox rows retain envelope bytes, fingerprint and uniqueness constraints.	Rebuilding envelope from current aggregate/table shape.
Payload migration	New producer fact/version is created only when the business protocol explicitly defines a new fact; historical v1 remains v1.	Silent upcast that rewrites audited history.
Consumer rebuild	Process original facts into a new projection using versioned adapters and a separate rebuild checkpoint.	Mutating producer history or marking operational inbox rows ambiguously.

Compatibility and negative test matrix

Fixture	Expected result	Evidence assertion
Golden v1 AlarmFloodDetected envelope	Encode, canonicalize, fingerprint, decode and revalidate successfully.	Exact UTF-8 bytes, canonical bytes and digest match checked-in fixture.
Same JSON meaning with reordered properties/whitespace	Strict parse succeeds; canonical material and digest equal the golden semantic value.	JCS implementation independence from source layout.
One payload value changed	Recomputed digest differs; unchanged claimed fingerprint fails validation.	Integrity mismatch detected before payload behavior.
Duplicate member at envelope or payload depth	Rejected before typed deserialization.	No first/last-wins ambiguity.
Unknown envelope member	Rejected under envelopeVersion 1.	No accidental extension semantics.
Unknown optional v1 payload member	Accepted only in the explicit additive-compatibility fixture.	Old RootCause codec safely skips it and semantic validation still passes.
Missing required/unknown enum/wrong units or time order	Permanent payload contract rejection.	No default/coercion/Domain call.
Unsupported eventType/schema/contentType	Quarantine with stable safe reason.	No dynamic or nearest-type fallback.
Same identity and fingerprint delivered 1, 2 and 20 times	One RootCause effect; later attempts duplicate success.	Inbox uniqueness and case-rc-3107 count/state stable.
Same identity with different fingerprint	Identity collision quarantine.	No inbox overwrite or business effect.
Crash before TX-A / after TX-A / after publish / before TX-B / after TX-B-before-ack	Each point yields no lost committed fact and at-most-one owner effect.	Transaction, outbox, inbox and acknowledgement evidence linked.
Runtime/canonicalizer upgrade	Golden corpus identical or release blocked by reviewed version decision.	Old/new build matrix and tool versions recorded.

Evidence catalogue

Evidence ID	Acceptance statement	Minimum artifact
ENV-SPEC-001	Envelope v1 fields, grammar, requiredness, ownership and non-meaning are accepted.	Versioned JSON schema/contract catalogue and signed ADR-018.
ENV-ID-001	messageId/correlation/causation/producer scopes and collision behavior are executable.	Identity grammar, uniqueness constraints and retry/replay tests.
ENV-TIME-001	Occurrence precision/timezone rules and non-ordering semantics are enforced.	Clock/time vectors and invalid/local/offset cases.
ENV-CODEC-001	net8.0 source-generated codec emits and reads the accepted exact v1 representation.	Build, serializer options, generated context and golden byte corpus.
ENV-CAN-001	Canonicalizer conforms to RFC 8785 and project restrictions.	Official JCS vectors plus nested NEXUS-1 vectors across supported builds.
ENV-FP-001	Fingerprint scope, SHA-256 format and constant-time comparison detect mutation.	Golden digests, mutation matrix and mismatch diagnostics.

Evidence ID	Acceptance statement	Minimum artifact
ENV-REG-001	Only registered eventType/schema/contentType combinations reach typed payload codecs.	Registry snapshot and unsupported/mismatch tests.
ENV-IN-001	RootCause validates then deduplicates one fact into one TX-B effect.	Preflight/inbox/collision/crash-point machine results.
ENV-SEC-001	Producer authorization, limits, safe diagnostics and no-payload logging controls hold.	Threat model, fuzz/limit tests and output/log scans.
ENV-COMP-001	Supported envelope/payload versions survive producer-consumer and runtime upgrades.	Old/new matrix linked to Chapter 24 compatibility evidence.
ENV-REVIEW-001	Producer, consumer, security, operations and architecture accept residual risks.	Evidence index with owners, versions, expiry and review triggers.

ADR-018 - Canonical versioned public-message envelope

ADR field	ADR-018 record
Status	Accepted for the NEXUS-1 reference design; implementation, canonicalization, serializer, broker, inbox and machine evidence remain pending.
Context	Public facts must survive independent deployment, outbox retry, broker redelivery, operational replay and restore without confusing payload, serialization, transport and diagnostic identities.
Decision	Envelope v1 uses strict UTF-8 JSON with the listed fields; message identity is producer-stable; eventType ends in the schema version; payload and envelope versions are independent; occurrence, correlation and causation have separate semantics; the sha-256+jcs fingerprint covers RFC 8785 canonical envelope material excluding only contentFingerprint; exact final bytes are stored in TX-A and validated before RootCause payload behavior; {producer,messageId} deduplicates, with fingerprint/type/version mismatch treated as collision.
Consequences	Contract registry, canonicalizer, codec, immutable byte storage, golden corpus, input limits, collision quarantine and cross-version tests become release-critical. Broker bindings can change without rewriting public history.
Rejected	Serialize domain/EF types; use CLR type names; regenerate at dispatch; use broker delivery tag/trace ID as message ID; rely on property order; hash only payload; trust digest as authentication; accept duplicate JSON names; dynamically deserialize unknown versions; overwrite an inbox collision; timestamp-sort as global order.
Review trigger	Envelope/payload field or meaning change, codec/canonicalizer/runtime upgrade, digest/security profile, producer identity, time precision, registry behavior, limits, broker binding, replay/restore process or supported-version window.

Risk register

Risk	Early signal	Blocking control
Envelope regenerated on retry	Same messageId has different bytes or occurredAtUtc after deployment.	Immutable TX-A bytes and ENV-CODEC/FP replay comparison.
Property order mistaken for canonical form	Digest changes between serializers/runtimes for equivalent JSON.	RFC 8785 vectors and independent canonicalizer tests.
Digest trusted as producer proof	Unauthorized sender supplies a self-consistent message and hash.	Authenticated transport, producer-event allow-list and explicit signature decision.
Identity collision treated as update	Inbox row is overwritten or latest payload wins.	Unique {producer,messageId}; mismatch quarantine and alert.
Unknown version reaches Domain	Catch-all JsonElement/dynamic path maps partially.	Strict envelope plus registry-before-payload validation.
Additive payload change breaks old consumer	Unknown member/enum or stricter validator rejects production data.	ENV-COMP/Chapter 24 supported-pair fixtures.
Correlation or trace becomes dedup key	Distinct facts collapse or replay loses identity when trace absent.	ENV-ID semantics and inbox key inspection.
Timestamp becomes total order	Late/redelivered fact overwrites newer owner state by clock comparison.	Owner invariants, causal policy and ordering negative tests.
Oversize/hostile JSON exhausts consumer	High allocation/CPU, deep nesting or payload logs during rejection.	Pre-parse byte/depth/string limits, fuzz tests and bounded diagnostics.
Historical bytes fail after upgrade	Restored outbox cannot publish or new consumer rejects golden v1.	Golden corpus, old/new runtime matrix and restore rehearsal.
Broker metadata contradicts body	AMQP message id does not differ from envelope body.	Publisher and consumer cross-check; envelope

Risk	Early signal	Blocking control
Sensitive payload duplicated into logs/dead letters	Validation diagnostics contain full body, identifiers or evidence.	Field classification, safe codes, redaction and retention scans.

Official-source validation

The envelope design was checked against current primary sources on 19 July 2026. CloudEvents 1.0.2 provides a protocol-neutral event metadata model with identity, source, type, spec version, optional subject/time and data; NEXUS-1 uses its own narrower field names and validation profile rather than claiming CloudEvents conformance. RFC 8785 defines a JSON Canonicalization Scheme suitable for invariant hashing/signing material. W3C Trace Context defines interoperable trace context for distributed tracing; NEXUS-1 therefore keeps trace identifiers diagnostic and separate from public fact identity. Microsoft documents that .NET 8 System.Text.Json can explicitly skip or disallow unmapped members, supporting per-contract compatibility policy rather than accidental defaults.

Primary source	Stable point used here	NEXUS-1 decision
CloudEvents specification v1.0.2 and JSON Event Format	Protocol-neutral event context is separate from event data and has explicit JSON representation rules.	Adopt the separation principle; retain a project-specific strict envelope and do not claim full CloudEvents conformance.
RFC 8785 - JSON Canonicalization Scheme	Deterministic JSON serialization supports repeatable hashes/signatures across property ordering and representation differences.	Use JCS for fingerprint material, with project restrictions and official/project vectors.
W3C Trace Context Recommendation	traceparent/tracestate propagate distributed tracing context and carry security/privacy considerations.	Keep trace context in diagnostics/transport metadata, not messageld, correlation or fingerprint-required business history.
Microsoft Learn - System.Text.Json unmapped members	.NET 8 can configure unknown members to Skip or Disallow.	Strict envelope parser; payload behavior selected per registered version and proved in compatibility tests.

Sources: <https://github.com/cloudevents/spec/tree/ce@v1.0.2> ;
<https://www.rfc-editor.org/rfc/rfc8785.html> ; <https://www.w3.org/TR/trace-context/> ;
<https://learn.microsoft.com/en-us/dotnet/standard/serialization/system-text-json/missing-members>

Technical material still required

The envelope field set, example, validation order, canonicalization/fingerprint scope, identity/time/correlation semantics, retry/replay behavior, failure classification, compatibility matrix, evidence catalogue and ADR are specified. No NEXUS-1 envelope package has compiled; no RFC 8785 implementation, exact digest, source-generated serializer, JSON schema, outbox byte column, RabbitMQ binding, inbox registry, quarantine route, fuzz fixture or old/new runtime pair has executed.

TECHNICAL MATERIAL STATUS - [VERSIONED ENVELOPE IMPLEMENTATION, CANONICALIZATION, SERIALIZATION AND RUNTIME EVIDENCE REQUIRED]

Implement the net8.0 envelope contracts, source-generated System.Text.Json codecs, strict preflight parser, duplicate-member detector, RFC 8785 canonicalizer integration, SHA-256 fingerprint material/verification, producer-event registry, exact-byte TX-A storage, broker-property cross-check, RootCause inbox collision policy and safe quarantine diagnostics; compile and execute official JCS vectors plus NEXUS-1 golden, mutation, malformed, oversize, unsupported-version, additive-payload, duplicate/redelivery, crash-point, replay, restore and runtime-upgrade matrices; publish ENV-SPEC-001 through ENV-REVIEW-001 linked to Chapter 18 TXN/ID evidence, Chapter 19 CTR evidence and Chapters 24-29 COMP/backbone evidence. RootCause remains G0 accepted and G1 specified; runtime evidence has not advanced.

Deliverable	Minimum technical content	Blocking evidence
Envelope package	Immutable v1 types, field/grammar validators and producer/type registry without Domain/EF dependencies.	ENV-SPEC/ID/TIME/REG-001

CHAPTER 20 - CODE AUDIT

The envelope is a strict document, not a dynamic property bag

The executable path fixes one dependency-free transport document, validates its grammar before typed decoding and requires an exact registry match before payload bytes can become consumer-owned Application input.

EVIDENCE STATUS

These listings are compile-oriented reference material. No selected RFC 8785 implementation, real digest, broker, outbound/inbox migration or executed corpus exists here. ENV, TXN, ID and compatibility evidence remains PENDING.

EXECUTABLE ASSET 20-A - STRICT ENVELOPE DOCUMENT

```
[JsonUnmappedMemberHandling(JsonUnmappedMemberHandling.Disallow)]
public sealed record EnvelopeDocumentV1(
    int EnvelopeVersion,
    string MessageId,
    string EventType,
    int SchemaVersion,
    DateTimeOffset OccurredAtUtc,
    string Producer,
    string CorrelationId,
    string? CausationId,
    string ContentType,
    string FingerprintAlgorithm,
    string ContentFingerprint,
    JsonElement Payload);

public readonly record struct EnvelopeContractKey(
    string Producer,
    string EventType,
    int SchemaVersion,
    string ContentType);

public sealed record EnvelopeLimits(
    int MaximumBytes,
    int MaximumDepth,
    int MaximumStringBytes,
    int MaximumPropertiesPerObject);

[JsonSourceGenerationOptions(
    PropertyNamingPolicy = JsonKnownNamingPolicy.CamelCase,
    GenerationMode = JsonSourceGenerationMode.Metadata)]
[JsonSerializable(typeof(EnvelopeDocumentV1))]
internal sealed partial class EnvelopeJsonContext
    : JsonSerializerContext;
```

EXECUTABLE ASSET 20-B - DEPENDENCY AND FIELD GATES

```
[Fact]
public void ENV_SPEC_001_envelope_package_has_no_runtime_adapter_dependency()
{
    var assembly = typeof(EnvelopeDocumentV1).Assembly;
    foreach (var forbidden in new[]
    {
        "Microsoft.EntityFrameworkCore",
        "RabbitMQ.Client",
        "Microsoft.AspNetCore",
        "Nexus1.AlarmManagement.Domain"
    })
    {
        Types.InAssembly(assembly).Should()
            .NotHaveDependencyOn(forbidden).GetResult()
            .IsSuccessful.Should().BeTrue();
    }
}

[Fact]
public void envelope_has_exact_reviewed_top_level_members()
{
    typeof(EnvelopeDocumentV1).GetProperties()
        .Select(x => x.Name)
        .Should().BeEquivalentTo(
            "EnvelopeVersion", "MessageId", "EventType",
            "SchemaVersion", "OccurredAtUtc", "Producer",
            "CorrelationId", "CausationId", "ContentType",
            "FingerprintAlgorithm", "ContentFingerprint", "Payload");
}
```

STRICT RECURSIVE PREFLIGHT

Duplicate names and hostile shape are rejected before allocation grows

EXECUTABLE ASSET 20-C - BOUNDED DUPLICATE-MEMBER SCANNER

```
public sealed class StrictJsonPreflight(EnvelopeLimits limits)
{
    public void Demand(ReadOnlySpan<byte> utf8)
    {
        if (utf8.Length == 0 || utf8.Length > limits.MaximumBytes)
            throw new EnvelopeSizeRejected(utf8.Length);

        var options = new JsonReaderOptions
        {
            AllowTrailingCommas = false,
            CommentHandling = JsonCommentHandling.Disallow,
            MaxDepth = limits.MaximumDepth
        };
        var reader = new Utf8JsonReader(utf8, options);
        var objectMembers = new Stack<HashSet<string>>();

        while (reader.Read())
        {
            switch (reader.TokenType)
            {
                case JsonTokenType.StartObject:
                    objectMembers.Push(new(StringComparer.Ordinal));
                    break;
                case JsonTokenType.EndObject:
                    objectMembers.Pop();
                    break;
                case JsonTokenType.PropertyName:
                    var name = reader.GetString();
                    ?? throw new EnvelopeGrammarRejected();
                    if (Encoding.UTF8.GetByteCount(name)
                        > limits.MaximumStringBytes
                        || objectMembers.Count == 0
                        || !objectMembers.Peek().Add(name))
                        throw new DuplicateOrOversizeJsonMember(name);
                    if (objectMembers.Peek().Count
                        > limits.MaximumPropertiesPerObject)
                        throw new EnvelopePropertyLimitRejected();
                    break;
                case JsonTokenType.String:
                    if (reader.HasValueSequence
                        ? reader.ValueSequence.Length > limits.MaximumStringBytes
                        : reader.ValueSpan.Length > limits.MaximumStringBytes)
                        throw new EnvelopeStringLimitRejected();
                    break;
            }
        }
        if (objectMembers.Count != 0)
            throw new EnvelopeGrammarRejected();
    }
}
```

EXECUTABLE ASSET 20-D - PREFLIGHT MUTATION MATRIX

```
[Theory]
[MemberData(nameof(MalformedEnvelopeCorpus))]
public void ENV_CODEC_001_malformed_input_is_rejected_before_decode(
    byte[] bytes, Type expected)
{
    var act = () => fixture.Preflight.Demand(bytes);
    act.Should().Throw<Exception>()
        .Which.Should().BeOfType(expected);
    fixture.TypedDeserializerCalls.Should().Be(0);
    fixture.RootCauseDomainCalls.Should().Be(0);
}

public static IEnumerable<object[]> MalformedEnvelopeCorpus()
{
    yield return
        [EnvelopeCorpus.OversizeBytes(), typeof(EnvelopeSizeRejected)];
    yield return [EnvelopeCorpus.InvalidUtf8(), typeof(JsonException)];
    yield return [EnvelopeCorpus.DepthBomb(), typeof(JsonException)];
    yield return
    [
        EnvelopeCorpus.DuplicateTopLevel("messageId"),
        typeof(DuplicateOrOversizeJsonMember)
    ];
    yield return
    [
        EnvelopeCorpus.DuplicateNestedPayload("severity"),
        typeof(DuplicateOrOversizeJsonMember)
    ];
}
```

DUPLICATE DETECTION IS RECURSIVE

The scanner tracks names separately for every JSON object, including payload objects. Strict envelope decoding begins only after byte, depth and member limits pass.

CANONICALIZATION AND FINGERPRINT SCOPE

The digest covers JCS material with one field removed

EXECUTABLE ASSET 20-E - REVIEWED RFC 8785 BOUNDARY

```
public interface IRfc8785Canonicalizer
{
    byte[] Canonicalize(ReadOnlySpan<byte> utf8Json);
}

public readonly record struct ContentFingerprint(string Value)
{
    public static ContentFingerprint FromDigest(ReadOnlySpan<byte> digest) =>
        new("sha256:" + Convert.ToHexString(digest).ToLowerInvariant());
}

public sealed class EnvelopeFingerprint(
    IRfc8785Canonicalizer canonicalizer)
{
    public ContentFingerprint Compute(ReadOnlySpan<byte> envelopeUtf8)
    {
        using var document = JsonDocument.Parse(envelopeUtf8.ToArray());
        if (document.RootElement.ValueKind != JsonValueKind.Object)
            throw new EnvelopeGrammarRejected();

        var buffer = new ArrayBufferWriter<byte>();
        using (var writer = new Utf8JsonWriter(buffer))
        {
            writer.WriteStartObject();
            var excluded = 0;
            foreach (var property in document.RootElement.EnumerateObject())
            {
                if (property.NameEquals("contentFingerprint"))
                {
                    excluded++;
                    continue;
                }
                writer.WritePropertyName(property.Name);
                property.Value.WriteTo(writer);
            }
            writer.WriteEndObject();
            writer.Flush();
            if (excluded != 1)
                throw new FingerprintFieldCountRejected(excluded);
        }

        var material = canonicalizer.Canonicalize(buffer.WrittenSpan);
        return ContentFingerprint.FromDigest(SHA256.HashData(material));
    }
}
```

EXECUTABLE ASSET 20-F - OFFICIAL AND PROJECT VECTORS

```
[Theory]
[MemberData(nameof(Rfc8785OfficialVectors))]
public void ENV_CAN_001_official_JCS_vectors_match(
    byte[] input, byte[] expected)
{
    fixture.Canonicalizer.Canonicalize(input)
        .Should().Equal(expected);
}

[Fact]
public void ENV_FP_001_property_order_does_not_change_fingerprint()
{
    var first = fixture.EnvelopeBytes(propertyOrder: "reviewed");
    var reordered = fixture.EnvelopeBytes(propertyOrder: "reversed");

    fixture.Fingerprints.Compute(first).Should()
        .Be(fixture.Fingerprints.Compute(reordered));
}

[Fact]
public void payload_mutation_changes_fingerprint()
{
    var original = fixture.EnvelopeBytes(severity: "critical");
    var changed = fixture.EnvelopeBytes(severity: "warning");
    fixture.Fingerprints.Compute(original).Should()
        .NotBe(fixture.Fingerprints.Compute(changed));
}
```

JCS IS SELECTED, NOT REIMPLEMENTED CASUALLY

The production canonicalizer is a reviewed component behind this boundary. Acceptance requires the official RFC 8785 corpus plus NEXUS-1 cross-runtime vectors.

PRODUCER CODEC AND EXACT-BYTE STORAGE

The owner commits final bytes that dispatch never rebuilds

EXECUTABLE ASSET 20-G - FINAL ENVELOPE CODEC

```
public sealed record EncodedEnvelope(
    EnvelopeDocumentV1 Document,
    ContentFingerprint Fingerprint,
    byte[] FinalBytes);

public sealed class EnvelopeV1Codec(
    IRfc8785Canonicalizer canonicalizer,
    EnvelopeFingerprint fingerprints)
{
    public EncodedEnvelope Encode(EnvelopeDocumentV1 unsigned)
    {
        var provisional = JsonSerializer.SerializeToUtf8Bytes(
            unsigned with
            {
                ContentFingerprint =
                    "sha256:" + new string('0', 64)
            },
            EnvelopeJsonContext.Default.EnvelopeDocumentV1);

        var fingerprint = fingerprints.Compute(provisional);
        var finalDocument = unsigned with
        {
            ContentFingerprint = fingerprint.Value
        };
        var serialized = JsonSerializer.SerializeToUtf8Bytes(
            finalDocument,
            EnvelopeJsonContext.Default.EnvelopeDocumentV1);
        var finalBytes = canonicalizer.Canonicalize(serialized);

        if (fingerprints.Compute(finalBytes) != fingerprint)
            throw new EnvelopeSelfVerificationFailed();
        return new(finalDocument, fingerprint, finalBytes);
    }
}
```

EXECUTABLE ASSET 20-H - TX-A STORES AND REPUBLISHES EXACT BYTES

```
public PendingPublicFactRow Stage(
    PendingPublicFactDraft<AlarmFloodDetectedV1> draft)
{
    var payload = payloads.Encode(
        draft.Contract, draft.Payload);
    var document = factory.CreateDocument(draft, payload);
    var encoded = codec.Encode(document);
    return PendingPublicFactRow.Create(
        draft.Fact,
        draft.Contract,
        encoded.Fingerprint,
        encoded.FinalBytes);
}

[Fact]
public async Task TXN_BND_001_state_and_final_bytes_commit_together()
{
    await fixture.ExecuteAlarmDecisionAsync();
    var row = await fixture.ReadPendingFactAsync();
    row.FinalEnvelopeBytes.Should().Equal(fixture.GoldenV1Bytes);
    row.Fingerprint.Should().Be(fixture.GoldenV1Fingerprint);
    fixture.SaveChangesCalls.Should().Be(1);
}

[Fact]
public async Task dispatcher_retry_publishes_byte_identical_body()
{
    await fixture.DispatchWithTwoRetriesAsync();
    fixture.PublishedBodies.Should().HaveCount(3);
    fixture.PublishedBodies.Distinct(ByteArrayComparer.Instance)
        .Should().ContainSingle();
    fixture.EnvelopeCodecCallsDuringDispatch.Should().Be(0);
}
```

STORED HISTORY IS AUTHORITATIVE

TX-A stores the exact final representation and its fingerprint. Dispatch may bind derived broker properties, but it cannot remap or reserialize the public fact.

REGISTRY AND CONSUMER PREFLIGHT

Registry selection and integrity pass before payload behavior

EXECUTABLE ASSET 20-I - EXACT CONTRACT REGISTRY

```
public interface IRegisteredPayloadCodec
{
    object Decode(JsonElement payload);
    Type PayloadType { get; }
}

public sealed class EnvelopeContractRegistry(
    IReadOnlyDictionary<EnvelopeContractKey, IRegisteredPayloadCodec> codecs)
{
    public IRegisteredPayloadCodec Demand(EnvelopeContractKey key) =>
        codecs.TryGetValue(key, out var codec)
            ? codec
            : throw new UnsupportedEnvelopeContract(key);
}

public sealed record AcceptedEnvelope(
    EnvelopeDocumentV1 Document,
    EnvelopeContractKey Contract,
    object TypedPayload,
    byte[] ReceivedBytes);
```

EXECUTABLE ASSET 20-J - ORDERED CONSUMER VALIDATION

```
public AcceptedEnvelope Accept(
    ReadOnlySpan<byte> body,
    AuthenticatedProducer producer,
    BrokerProperties broker)
{
    preflight.Demand(body);
    var document = JsonSerializer.Deserialize(
        body,
        EnvelopeJsonContext.Default.EnvelopeDocumentV1)
        ?? throw new EnvelopeGrammarRejected();
    grammar.Demand(document);

    var key = new EnvelopeContractKey(
        document.Producer,
        document.EventType,
        document.SchemaVersion,
        document.ContentType);
    producer.DemandAuthorized(key);
    broker.DemandConsistentWith(document);
    var payloadCodec = registry.Demand(key);

    var observed = fingerprints.Compute(body);
    fingerprintParser.DemandEqual(
        document.ContentFingerprint, observed);
    var payload = payloadCodec.Decode(document.Payload);

    return new(document, key, payload, body.ToArray());
}

[Fact]
public void ENV_REG_001_unknown_contract_never_decodes_payload()
{
    var body = fixture.ValidEnvelope(eventType: "unknown.v99");
    var act = () => fixture.Consumer.Accept(
        body, fixture.AuthenticatedAlarm, fixture.BrokerProperties);
    act.Should().Throw<UnsupportedEnvelopeContract>();
    fixture.PayloadCodecCalls.Should().Be(0);
    fixture.RootCauseDomainCalls.Should().Be(0);
}
```

VALIDATION ORDER IS A SECURITY BOUNDARY

Limits, grammar, producer authorization, broker cross-check, registry and fingerprint all pass before a version-specific payload codec or Domain mapper can run.

INBOX COLLISION AND QUARANTINE

Same identity with changed content is never a duplicate success

EXECUTABLE ASSET 20-K - INBOX RESOLUTION IN A FRESH SCOPE

```
public async Task<EnvelopeConsumeOutcome> ConsumeAsync(
    AcceptedEnvelope accepted,
    CancellationToken cancellationToken)
{
    {
        await using var attempt = scopes.CreateAsyncScope();
        var handler = attempt.ServiceProvider
            .GetRequiredService<RootCauseEnvelopeHandler>();
        try
        {
            return await handler.HandleAsync(
                accepted, cancellationToken);
        }
        catch (DbUpdateException exception)
            when (conflicts.IsInboundEnvelopeIdentity(exception))
        {
            // Failed context is disposed before durable reload.
        }
    }

    await using var reload = scopes.CreateAsyncScope();
    var inbox = reload.ServiceProvider
        .GetRequiredService<IEnvelopeInboxReader>();
    var stored = await inbox.DemandAsync(
        accepted.Document.Producer,
        accepted.Document.MessageId,
        cancellationToken.None);

    if (!CryptographicOperations.FixedTimeEquals(
        stored.FingerprintBytes,
        fingerprintParser.ReadDigest(
            accepted.Document.ContentFingerprint)))
        throw new EnvelopeIdentityCollision(
            accepted.Document.Producer,
            accepted.Document.MessageId);
    return EnvelopeConsumeOutcome.Duplicate(stored.StableOutcome);
}
```

EXECUTABLE ASSET 20-L - SAFE QUARANTINE AND BROKER CONTRADICTION TESTS

```
public sealed record SafeEnvelopeQuarantineRecord(
    string ReasonCode,
    string ProducerAlias,
    string MessageIdentityDigest,
    string ReceivedBodyDigest,
    DateTimeOffset RecordedAtUtc);

[Fact]
public async Task ENV_IN_001_same_id_changed_body_is_quarantined()
{
    await fixture.ConsumeAcceptedEnvelopeAsync(severity: "critical");
    var act = () => fixture.ConsumeSameIdentityAsync(severity: "warning");
    await act.Should().ThrowAsync<EnvelopeIdentityCollision>();
    fixture.RootCauseEffects.Should().ContainSingle();
    fixture.Quarantine.Should().ContainSingle(x =>
        x.ReasonCode == "envelope-identity-collision");
    fixture.Logs.Should().NotContain(
        x => x.Contains(fixture.RawPayloadCanary, StringComparison.Ordinal));
}

[Fact]
public void broker_message_id_contradiction_blocks_before_inbox()
{
    var body = fixture.ValidEnvelope(messageId: "fact-am-7716");
    var broker = fixture.BrokerProperties with
    {
        MessageId = "fact-am-9999"
    };
    var act = () => fixture.Consumer.Accept(
        body, fixture.AuthenticatedAlarm, broker);
    act.Should().Throw<BrokerEnvelopeContradiction>();
    fixture.InboxWrites.Should().Be(0);
}
```

QUARANTINE RETAINS EVIDENCE, NOT THE SECRET

Collision diagnostics use safe reason codes and protected digests. Raw payload, tenant identifiers and private evidence do not enter logs by default.

COMPATIBILITY AND HOSTILE-INPUT CORPUS

Supported bytes execute across contract and runtime versions

EXECUTABLE ASSET 20-M - ENVELOPE COMPATIBILITY CASE

```
public sealed record EnvelopeCompatibilityCase(
    string ProducerRuntime,
    string ConsumerRuntime,
    int EnvelopeVersion,
    string EventType,
    int SchemaVersion,
    string Fixture,
    ExpectedEnvelopeOutcome Expected);

public static IEnumerable<object[]> SupportedEnvelopePairs()
{
    yield return
    [
        new EnvelopeCompatibilityCase(
            "net8-build-A", "net8-build-B", 1,
            "nexus1.alarm-management.alarm-flood-detected.v1",
            1, "alarm-flood-envelope-v1.json",
            ExpectedEnvelopeOutcome.Accept),
    ];
    yield return
    [
        new EnvelopeCompatibilityCase(
            "net8-build-B", "net8-build-A", 2,
            "nexus1.alarm-management.alarm-flood-detected.v1",
            1, "unknown-envelope-v2.json",
            ExpectedEnvelopeOutcome.RejectEnvelopeVersion),
    ];
}

[Theory]
[MemberData(nameof(SupportedEnvelopePairs))]
public async Task ENV_COMP_001_supported_matrix_is_explicit(
    EnvelopeCompatibilityCase test)
{
    var result = await fixture.ExecutePairAsync(test);
    result.Outcome.Should().Be(test.Expected);
    result.DomainCalls.Should().Be(
        test.Expected == ExpectedEnvelopeOutcome.Accept ? 1 : 0);
}
```

EXECUTABLE ASSET 20-N - GOLDEN AND HOSTILE CORPUS GATES

```
[Theory]
[InlineData("duplicate-envelope-member.json")]
[InlineData("duplicate-payload-member.json")]
[InlineData("invalid-utf8.json")]
[InlineData("depth-limit.json")]
[InlineData("oversize-string.json")]
[InlineData("fingerprint-mismatch.json")]
[InlineData("unauthorized-producer.json")]
public async Task ENV_SEC_001_hostile_corpus_has_no_Domain_effect(
    string fixtureName)
{
    var result = await fixture.ExecuteRejectedFixtureAsync(fixtureName);
    result.IsRejected.Should().BeTrue();
    result.RootCauseDomainCalls.Should().Be(0);
    result.LogOutput.Should().NotContain(fixture.RawPayloadCanary);
}

[Fact]
public async Task golden_v1_bytes_survive_runtime_upgrade()
{
    var bytes = fixture.Corpus.ReadBytes(
        "alarm-flood-envelope-v1.json");
    var oldResult = await fixture.Net8A.AcceptAsync(bytes);
    var newResult = await fixture.Net8B.AcceptAsync(bytes);
    newResult.ApplicationCommand.Should()
        .BeEquivalentTo(oldResult.ApplicationCommand);
}
```

A GOLDEN FILE CONTAINS EXACT BYTES

The corpus retains byte sequences, expected fingerprint, accepted command or safe rejection and the runtime/contract identities that produced the result.

REPLAY, RESTORE AND ACCEPTANCE

Restored history republishes the same accepted representation

EXECUTABLE ASSET 20-O - EXACT-BYTE RECOVERY GATE

```
[Fact]
public async Task ENV_REST_001_restore_reuses_bytes_and_identity()
{
    var before = await fixture.PendingEnvelopeBaselineAsync();
    var alarmState = await fixture.AlarmManagementFingerprintAsync();

    await fixture.RestoreReplayAndReconcileRootCauseAsync();
    await fixture.RepublishRestoredPendingEnvelopeAsync();

    var after = await fixture.PendingEnvelopeBaselineAsync();
    after.Should().BeEquivalentTo(before);
    fixture.RepublishedBodies.Should()
        .OnlyContain(x => x.SequenceEqual(before.FinalBytes));
    fixture.RepublishedFactIds.Should()
        .OnlyContain(x => x == before.PublicFactIdentity);
    fixture.RootCauseDuplicateEffects.Should().Be(0);
    (await fixture.AlarmManagementFingerprintAsync())
        .Should().Be(alarmState);
}
```

EXECUTABLE ASSET 20-P - ACCEPTANCE EVIDENCE RECORD

```
{
  "evidenceId": "ENV-REVIEW-001", "chapter": 20,
  "artifactCommit": null, "sdk": "net8.0",
  "envelopePackage": null, "serializer": null,
  "jcsImplementation": null, "jcsVersion": null,
  "broker": null, "provider": null,
  "tests": {
    "ENV-SPEC-001": "PENDING", "ENV-CODEC-001": "PENDING",
    "ENV-CAN-001": "PENDING", "ENV-FP-001": "PENDING",
    "ENV-ID-001": "PENDING", "ENV-TIME-001": "PENDING",
    "ENV-REG-001": "PENDING", "ENV-IN-001": "PENDING",
    "ENV-SEC-001": "PENDING", "ENV-COMP-001": "PENDING",
    "ENV-REST-001": "PENDING", "TXN-BND-001": "PENDING"
  },
  "officialJcsVectorsAccepted": false,
  "goldenCorpusAccepted": false, "accepted": false
}
```

Chapter 20 code-audit result

The chapter now exposes one executable path from a strict public document to bounded preflight, registry selection, JCS fingerprinting, exact-byte owner storage, consumer collision handling and cross-runtime restore/replay evidence.

CURRENT VERDICT

The envelope pipeline is implementable in shape, but no JCS component, real digest, broker, provider or compatibility corpus has executed. All envelope evidence remains explicitly PENDING.

Deliverable	Minimum technical content	Blocking evidence
Codec	Pinned source-generated System.Text.Json contexts and strict UTF-8 parser with duplicate/depth/size controls.	ENV-CODEC-001
Canonicalization	Reviewed RFC 8785 implementation and fingerprint material excluding only contentFingerprint.	ENV-CAN/FP-001 official and project vectors
Producer storage	Final exact bytes/fingerprint staged atomically in TX-A and published without remapping.	TXN-BND-001 / ENV-CODEC-001 / Chapter 26
Consumer boundary	Envelope/integrity/registry validation, inbox identity collision rules and RootCause mapping before Domain.	ENV-IN/SEC-001 / Chapter 28
Compatibility	Envelope/payload/runtime matrices, preserved historical bytes and controlled version windows.	ENV-COMP-001 / Chapter 24
Operations	Safe diagnostics, quarantine policy, replay/restore manifests and broker property/body cross-check.	ENV-SEC/REVIEW-001 / Chapters 25-29

Implementation sequence

1. Accept ADR-018 and freeze the envelope v1 field catalogue, grammar, identity/time semantics, limits and fingerprint profile.
2. Create a dependency-free net8.0 Integration.Envelope package plus a producer/type/schema/content-type registry; no Domain, EF, RabbitMQ or ASP.NET Core types.
3. Implement source-generated codecs and strict recursive JSON preflight including duplicate-member, depth, size, timestamp and discriminator validation.
4. Integrate and independently test an RFC 8785 canonicalizer; define FingerprintMaterial that excludes only contentFingerprint and compile official/project vectors.
5. Implement EnvelopeFactory, SHA-256 encoding/comparison and producer self-validation; generate the exact AlarmFloodDetected v1 fixture and real digest.
6. Extend AlarmManagement TX-A/outbox storage to persist exact final bytes and metadata; prove dispatcher retry never regenerates the envelope.
7. Bind RabbitMQ properties as a derived protocol mapping and reject property/body contradiction without treating delivery metadata as identity.
8. Implement RootCause preflight, registry, inbox {producer,messageId} uniqueness, same-fingerprint duplicate success and mismatch collision quarantine before mapping.
9. Execute malformed, mutation, size/fuzz, duplicate/redelivery, crash, replay, restore and old/new runtime/contract matrices.
10. Publish ENV evidence and residual-risk review without advancing RootCause beyond G0/G1 until compiled runtime behavior exists.

What this chapter has proved

Chapter 20 has specified a broker-neutral, versioned envelope around the Chapter 19 public payload. It has separated message, workflow, cause, trace and delivery identities; defined strict envelope fields and time semantics; fixed canonicalization and fingerprint scope; required exact-byte TX-A persistence; shown the full AlarmManagement-to-RootCause validation/dedup/TX-B path; classified failures, retries, replay, restore and version changes; and made security and compatibility evidence explicit.

It has not compiled a net8.0 contract package, produced a real digest, executed RFC 8785 vectors, serialized or stored an envelope, published through RabbitMQ, validated a consumer message, inserted an inbox record, created case-rc-3107, quarantined a collision, replayed history or survived a runtime upgrade. The design is specified, not implemented or evidenced. RootCause remains G0 accepted and G1 specified.

ONE FACT, ONE IDENTITY, ONE ACCEPTED REPRESENTATION

Delivery attempts may multiply. The public fact and its owner-local effects must not. Stable identity, immutable bytes, strict validation and inbox completion make that statement testable.

Chapter checkpoint

Question	Expected answer
What does messageId identify?	One producer-owned public fact. fact-am-7716 survives retry, redelivery, replay and restore.
Is a broker delivery tag the message ID?	No. It identifies one transport attempt/channel acknowledgement and may change on redelivery.
What is schemaVersion?	The payload contract version registered for eventType; it is independent from envelopeVersion.
Why does eventType also end in .v1?	It names the accepted semantic public type; envelope v1 requires the suffix and numeric schemaVersion to agree.
What does occurredAtUtc mean?	The producer-defined business occurrence/decision instant, normalized to UTC; not publication time or global order.
Are correlationId and causationId dedup keys?	No. They group a workflow and identify an immediate cause; {producer,messageId} begins dedup.
Is traceparent part of business identity?	No. It is diagnostic context; retries/replays may create new spans or have no trace.
What is fingerprinted?	RFC 8785 canonical envelope material including payload and excluding only contentFingerprint.
Does the SHA-256 fingerprint authenticate AlarmManagement?	No. It detects content mismatch; secured transport and producer authorization establish origin unless a separate signature protocol is designed.
May dispatch rebuild the envelope?	No. TX-A stores the final exact bytes and dispatch publishes them unchanged.
What happens on same identity and same fingerprint?	After prior TX-B completion, it is duplicate success and no RootCause effect repeats.
What happens on same identity and different fingerprint?	Identity collision or tampering: quarantine and alert; never overwrite or process.
Are unknown envelope and payload members treated alike?	No. Envelope v1 is strict; payload unknown-member policy is version-specific and compatibility-tested.
What is the current evidence status?	Envelope behavior is specified only; no codec/canonicalizer/broker/inbox runtime evidence exists; RootCause remains G0/G1.
What does Chapter 21 add?	Versioned command/query endpoints, request validation, authorization, status semantics and failure contracts at the synchronous service boundary.

NEXT CHAPTER

Chapter 21, Commands and Queries at a Service Boundary, will implement the synchronous edge around the same owner boundaries. It will define versioned request/response models, validation, authorization, status codes and failure contracts without passing HTTP DTOs into aggregates or claiming that AlarmManagement success proves RootCause completion.

PART IV - PUBLIC CONTRACTS AND THE SYNCHRONOUS EDGE

21. Commands and Queries at a Service Boundary

A service boundary is not a thin serialization wrapper around an aggregate. It is a deliberate translation and policy surface: HTTP supplies transport semantics, public DTOs describe client-visible intent and representations, authorization constrains who may attempt the use case, Application coordinates owner work, and Domain protects invariants without learning about routes, status codes or bearer tokens.

CHAPTER ARGUMENT

Expose commands and queries as versioned provider-owned contracts. Validate each concern at the boundary that owns it, authorize before revealing protected meaning, map public DTOs into Application inputs, commit only owner-local truth, and return HTTP responses that state exactly what this provider knows - never a leaked entity, exception or downstream completion claim.

Chapters 19 and 20 defined the public fact and its envelope. This chapter implements the synchronous edge that may create that fact. The reference command accepts `DetectAlarmFloodRequestV1`, authenticates and authorizes the actor, maps to `AlarmManagement` Application input, executes one owner-local transaction and returns `DetectAlarmFloodAcceptedV1`. The response identifies `request-am-20482`, `decision-am-9184` and `fact-am-7716`, but it cannot include `case-rc-3107` because `RootCause` has not yet validated or committed anything.

The selected response is 202 Accepted with a Location URI for an `AlarmManagement`-owned `decision-status` resource. It proves the provider accepted and committed its local decision/public intent; it does not promise broker delivery, `RootCause` processing or final investigation success. A separate authorized `RootCause` query - or the Chapter 22 BFF - can expose consumer-owned state when it exists. The browser never chains service commands to manufacture a distributed transaction.

This remains a net8.0 reference specification. No route, authentication scheme, authorization handler, validator, mapper, endpoint filter, Problem Details writer, OpenAPI document, HTTP fixture or deployed service has executed. Chapter 23 owns exact endpoint-version negotiation, deprecation and idempotency-key behavior; Chapter 24 owns compatibility execution. `RootCause` remains G0 accepted and G1 specified.

The synchronous edge owns translation and HTTP truth

THE SYNCHRONOUS EDGE TRANSLATES; IT DOES NOT OWN THE DOMAIN

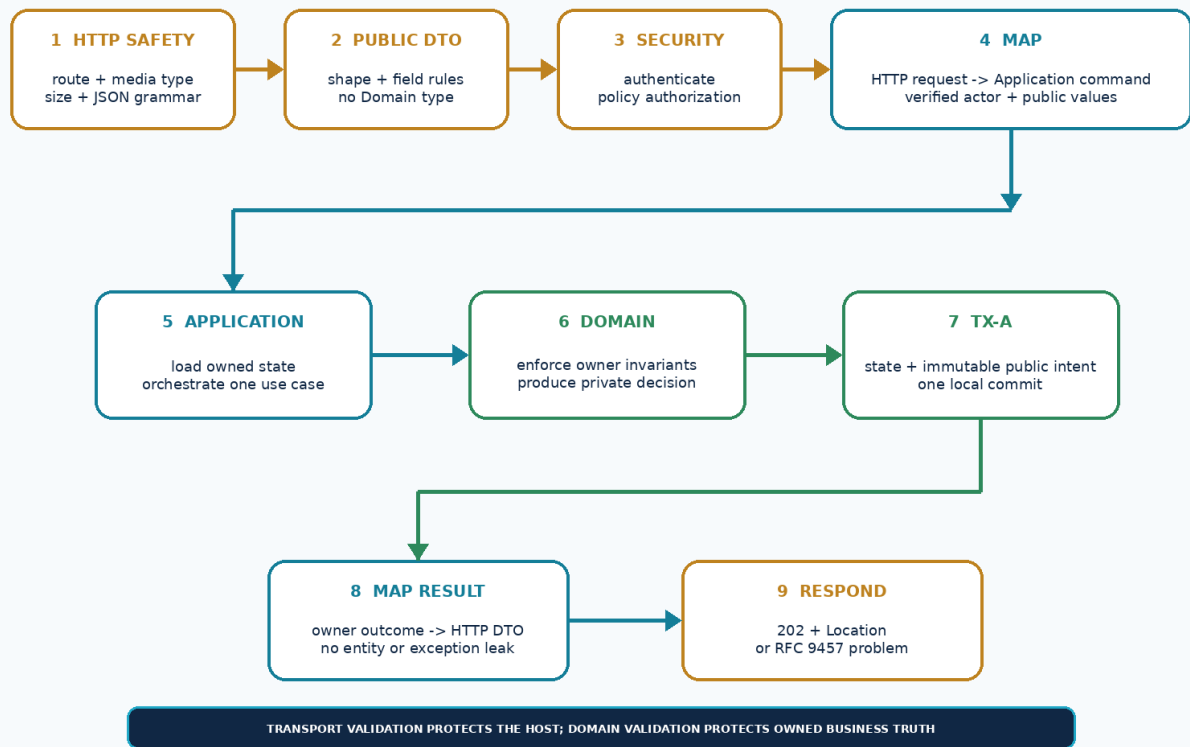


Figure 21.1 - HTTP safety, public contracts, security, Application, Domain, transaction and response mapping are distinct stages.

Stage	Owner	Accepted responsibility	Forbidden shortcut
Routing/transport	ASP.NET Core Edge host	Method/path/media type/body limits/JSON grammar and protocol headers.	Treat malformed input as a Domain exception or allocate unbounded bodies.
Public request	API contract owner	Stable versioned field names, types, requiredness, units and cross-field public rules.	Bind an aggregate, EF entity, internal command or shared broker DTO.
Authentication	Security/host configuration	Validate credential scheme, issuer, audience, signature, lifetime and identity context.	Trust caller-supplied actor/tenant fields in the body.
Authorization	Named policy + resource-aware handler	Decide whether the verified actor may attempt the operation/read this local resource.	Role checks scattered in controllers or Domain, or existence leaks.
Application mapping	Owning service Application layer	Create owner-specific command/query from verified actor and public values; orchestrate one use case.	Pass HttpContext/DTO/token into Domain or call another service inside TX-A.
Domain	Owning aggregate/policy	Enforce invariants and decide private owner outcome.	Return HTTP status/ProblemDetails or inspect claims/routes.
Persistence	Owning Infrastructure	One local commit, concurrency and immutable public intent under Chapters 18-20.	Distributed transaction or response before commit.
Response mapping	API Edge contract	Map owner outcome to provider-local representation/status/problem semantics.	Serialize entity/exception or claim broker/RootCause success.

Commands, queries and public events remain different protocols

Protocol	Meaning	Primary success proof	NEXUS-1 example
HTTP command	An authorized request to attempt one provider capability; may change owner state.	Provider-local commit/accepted operation semantics defined by static	POST /api/v1/alarm-flood-detections

Protocol	Meaning	Primary success proof	NEXUS-1 example
HTTP query	An authorized request for a provider-owned representation; no business side effect.	A representation, not-modified result or bounded absence/access outcome.	GET /api/v1/alarm-flood-decisions/decision-am-9184.
Integration event	A producer-owned past-tense fact published asynchronously for independent consumers.	Immutable message identity/contract; no synchronous consumer result.	AlarmFloodDetected.v1 inside the Chapter 20 envelope.
BFF composition	Edge-owned view aggregation for the operator; never a browser-orchestrated transaction.	A UI-specific representation with source/freshness semantics.	Chapter 22 combines AlarmManagement and RootCause read models.

QUERIES DO NOT HIDE COMMANDS

A GET must not create a case, mark an alarm acknowledged, trigger dispatch or repair state. If observing a representation requires a side effect, name and authorize that operation as a command; keep diagnostics/metrics outside business semantics.

Public request and response models

IMPLEMENTATION SHAPE

```
public sealed record DetectAlarmFloodRequestV1(  
    string SiteId,  
    DateTimeOffset WindowStartedAtUtc,  
    DateTimeOffset WindowEndedAtUtc,  
    IReadOnlyList<string> AlarmIds,  
    string RequestedSeverity,  
    string ClientRequestId);  
  
public sealed record DetectAlarmFloodAcceptedV1(  
    string RequestId,  
    string DecisionId,  
    string AlarmFloodId,  
    string PublicFactId,  
    string Status,  
    DateTimeOffset AcceptedAtUtc,  
    string StatusUrl);  
  
public sealed record AlarmFloodDecisionViewV1(  
    string DecisionId,  
    string AlarmFloodId,  
    string DecisionStatus,  
    DateTimeOffset AcceptedAtUtc,  
    string? PublicFactId);
```

The records are API types, not Domain types. SiteId and AlarmIds are public strings until the mapper validates syntax and creates owner value objects. RequestedSeverity is a public enum token whose unknown values are rejected; the Domain determines whether the evidence qualifies under policy. ClientRequestId is a client correlation value, not the Chapter 23 idempotency key and never the owner decision identity.

Field	Public rule	Mapping/response rule
SiteId	Required canonical public identifier; bounded length/character set; actor must be authorized for the site.	Map to AlarmManagement SiteIdentity only after resource authorization; never expose database key.
WindowStartedAtUtc / WindowEndedAtUtc	UTC, millisecond precision, start < end and bounded maximum duration.	Map to owner time window; occurrence is not server receipt time.
AlarmIds	1..500 unique canonical identifiers; bounded aggregate byte size; stable ordering is not inferred.	Resolve owner-local evidence references in one bounded query; missing/unauthorized cases follow policy.
RequestedSeverity	Accepted lowercase public vocabulary; unknown tokens rejected rather than defaulted.	Map explicitly to owner value; it is input intent, not proof of Domain severity.
ClientRequestId	Opaque bounded client correlation ID; no authorization or identity authority.	Echo only if safe; decision-am-9184/request-am-20482 remain provider-owned identities.
StatusUrl	Relative/absolute provider-owned URI documented by the API contract.	Location header and body agree; never point directly to queue. outbox row or RootCause private resource.

Complete command walkthrough

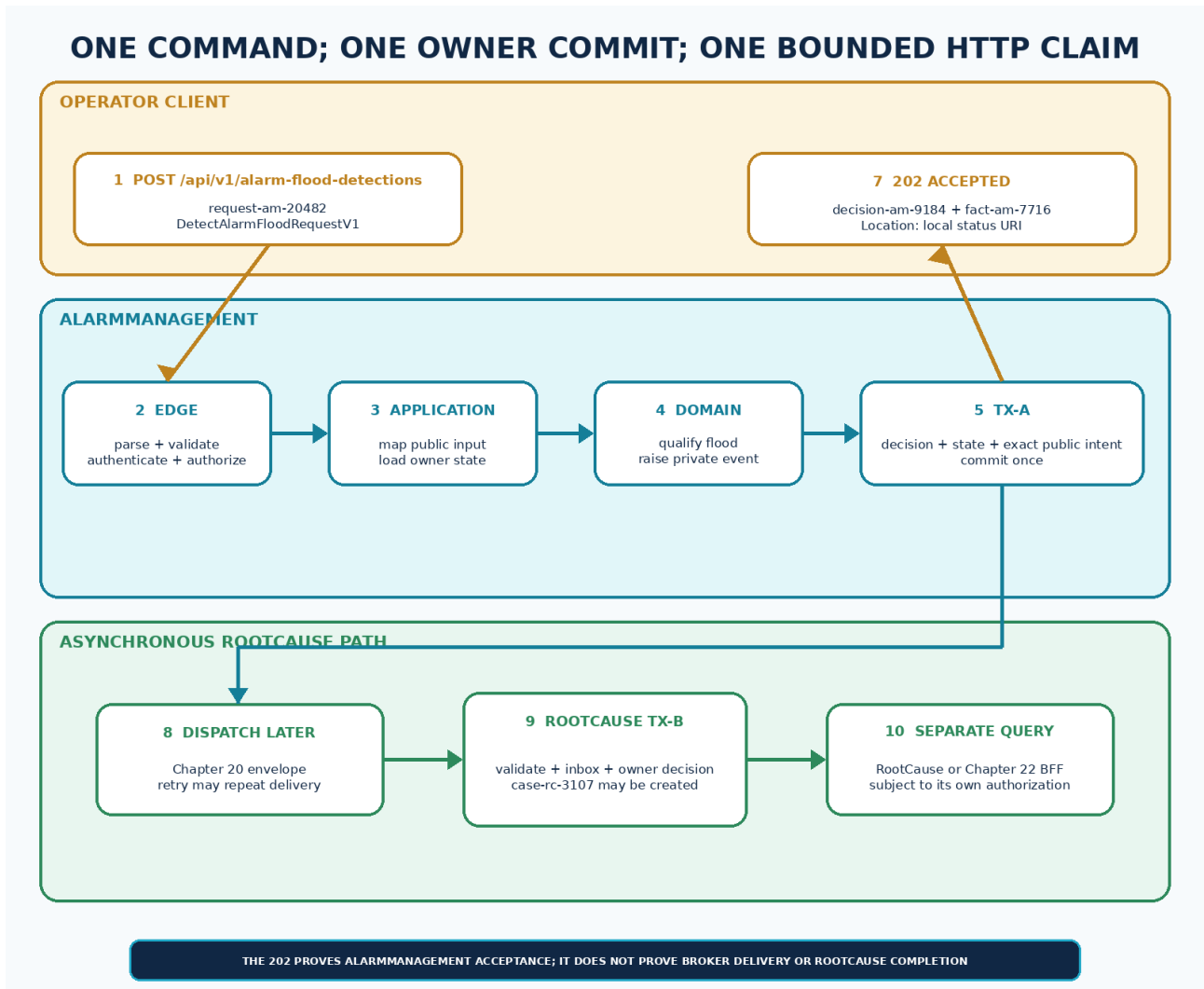


Figure 21.2 - The command commits one AlarmManagement result and returns a bounded 202 while RootCause proceeds asynchronously under a separate boundary.

Step	Accepted behavior	Evidence/identity
1. Request	Authorized client POSTs DetectAlarmFloodRequestV1 as application/json.	request-am-20482, authenticated principal, received media/byte metadata.
2. Edge	Route/content limits/JSON/DTO rules pass; authentication and named authorization policy succeed.	API-VAL/AUTHN/AUTHZ results; safe occurrence ID.
3. Mapping	DTO plus verified actor become AlarmManagement Application command; no HttpContext enters Domain.	API-MAP-001 mapper version and input snapshot classification.
4. Decision	AlarmManagement qualifies the flood and produces decision-am-9184 under owner invariants.	Domain/private event and concurrency evidence.
5. TX-A	State, fact-am-7716 and Chapter 20 exact envelope bytes commit once.	TXN-BND/ENV-CODEC evidence; no broker claim.
6. Result mapping	Owner outcome becomes DetectAlarmFloodAcceptedV1; response contains no entity or RootCause state.	API-RESP-001 mapping fixture.
7. 202	Location identifies AlarmManagement decision status; body says accepted provider-local truth.	HTTP status/header/body agreement and lost-response fixture.
8. Dispatch	Outbox publishes later; retry keeps fact identity/bytes.	Chapter 20 and Chapters 26-27 evidence.
9. RootCause	Consumer validates, deduplicates and commits TX-B; case-rc-3107 may then exist.	RootCause-owned inbox/case evidence, absent from 202.
10. Read	Client queries RootCause or the BFF under separate authentication/authorization and freshness semantics.	Independent query contract; no browser-orchestrated write chain.

A bounded HTTP exchange

IMPLEMENTATION SHAPE

```
POST /api/v1/alarm-flood-detections HTTP/1.1
Host: alarm-management.nexus-1.example
Authorization: Bearer <redacted>
Content-Type: application/json
Accept: application/json, application/problem+json

{
  "siteId": "site-ath-17",
  "windowStartedAtUtc": "2026-07-19T07:38:00.000Z",
  "windowEndedAtUtc": "2026-07-19T07:43:00.000Z",
  "alarmIds": ["alarm-1001", "alarm-1002", "alarm-1003"],
  "requestedSeverity": "critical",
  "clientRequestId": "request-am-20482"
}

HTTP/1.1 202 Accepted
Location: /api/v1/alarm-flood-decisions/decision-am-9184
Content-Type: application/json
Cache-Control: no-store

{
  "requestId": "request-am-20482",
  "decisionId": "decision-am-9184",
  "alarmFloodId": "flood-am-6031",
  "publicFactId": "fact-am-7716",
  "status": "accepted",
  "acceptedAtUtc": "2026-07-19T07:43:11.482Z",
  "statusUrl": "/api/v1/alarm-flood-decisions/decision-am-9184"
}
```

WHAT 202 DOES NOT SAY

It does not say delivered, consumed, investigation-opened, verdict-ready or succeeded. It contains no RootCause case ID. RFC 9110 describes 202 as intentionally noncommittal for processing that is not complete; the NEXUS-1 body narrows the known provider-local acceptance without inventing a later distributed result.

Validation is layered, ordered and observable

Layer	Examples	Failure surface
Protocol safety	Method/path, TLS policy, Content-Length/body cap, decompression limit, media type/charset, JSON depth/grammar.	400/405/413/415 or connection handling; bounded detail; no Domain call.
Public shape	Required/null/type/range/string/array limits and unknown-member policy for v1.	400 for malformed representation; validation problem with safe JSON pointers.
Authentication	Bearer scheme, signature, issuer, audience, expiry/not-before and token replay policy.	401 plus applicable WWW-Authenticate; never report resource existence.
Authorization	DetectAlarmFloodAtSite policy, tenant/site scope and actor capability.	403 or policy-safe 404; stable problem semantics without revealing denied resource data.
Public semantic validation	start < end, bounded window, unique AlarmIds, known severity and consistent units.	422 with stable problem type and field/cross-field pointers.
Application preconditions	Owner reference exists, supported lifecycle state, expected version/If-Match when contract requires it.	404/409/412 according to accepted public meaning.
Domain invariants	Flood qualification, duplicate business decision, policy version and concurrency-sensitive owner rules.	409/422 only through explicit outcome mapping; no exception serialization.
Infrastructure	Database timeout/unavailability, optimistic concurrency after bounded retry, cancellation.	503/409/500 by stable classification; no SQL/provider/stack detail.

Only the minimum safe parsing needed to protect the host happens before authentication. Detailed validation that could reveal resource policy or data occurs after the caller is authenticated and, where necessary, after coarse authorization. The sequence is a security design, not merely a performance optimization. Validation errors are deterministic and field-addressable: authorization failures are

Authentication and authorization are separate decisions

IMPLEMENTATION SHAPE

```
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("alarm-management.detect-alarm-flood", policy =>
    {
        policy.RequireAuthenticatedUser();
        policy.RequireClaim("scope", "alarm-flood.write");
        policy.AddRequirements(new SiteAccessRequirement());
    });

    options.AddPolicy("alarm-management.read-alarm-flood", policy =>
    {
        policy.RequireAuthenticatedUser();
        policy.RequireClaim("scope", "alarm-flood.read");
        policy.AddRequirements(new SiteAccessRequirement());
    });
});
```

The snippet is a reference candidate. Token validation belongs to the authentication scheme; claims are accepted only from the configured issuer/audience. The SiteAccessRequirement handler evaluates the verified principal against an authorized public resource context. It returns a decision, not a Domain value or HTTP body, and its logs use bounded identifiers. The endpoint never trusts Siteld/tenant/actor claims supplied inside the request JSON.

Question	Authentication answer	Authorization answer
Who is calling?	Verified principal from accepted credential scheme.	Not answered; identity alone grants nothing.
May this actor invoke the capability?	Not answered.	Named policy requirements/handlers evaluate capability and resource context.
What if credentials are absent/invalid?	401 and WWW-Authenticate where applicable.	No resource-dependent evaluation or detailed leak.
What if identity is valid but permission fails?	Authentication remains successful.	403 or policy-safe 404 under documented existence-disclosure policy.
May Domain inspect claims/roles?	No.	No. Application receives a verified ActorContext and Domain receives only owner business inputs.
May authorization call RootCause?	No.	Not for this owner command. AlarmManagement authorization uses owned/local policy evidence; remote dependency would couple availability and leak boundaries.

Endpoint mapping remains explicit

IMPLEMENTATION SHAPE

```
group.MapPost("/alarm-flood-detections", async (
    DetectAlarmFloodRequestV1 request,
    ClaimsPrincipal principal,
    IAuthorizationService authorization,
    DetectAlarmFloodHandler handler,
    CancellationToken ct) =>
{
    PublicValidationResult valid = DetectAlarmFloodV1Rules.Validate(request);
    if (!valid.IsValid)
        return ApiProblems.Validation(valid);

    var resource = new SiteAccessResource(request.SiteId);
    AuthorizationResult allowed = await authorization.AuthorizeAsync(
        principal, resource, "alarm-management.detect-alarm-flood");
    if (!allowed.Succeeded)
        return ApiProblems.ForbiddenOrHidden(resource);

    DetectAlarmFlood command = DetectAlarmFloodMapper.Map(
        request, ActorContext.FromVerifiedPrincipal(principal));
    DetectAlarmFloodOutcome outcome = await handler.Handle(command, ct);

    return DetectAlarmFloodHttpMapper.Map(outcome);
})
.RequireAuthorization()
.Accepts<DetectAlarmFloodRequestV1>("application/json")
.Produces<DetectAlarmFloodAcceptedV1>(StatusCodes.Status202Accepted)
.ProducesProblem(StatusCodes.Status400BadRequest)
.ProducesProblem(StatusCodes.Status401Unauthorized)
.ProducesProblem(StatusCodes.Status403Forbidden)
.ProducesProblem(StatusCodes.Status409Conflict)
.ProducesProblem(StatusCodes.Status422UnprocessableEntity);
```

The route listing is intentionally not claimed as compiled. Production wiring must ensure authentication precedes authorization, transport limits run before binding, cancellation does not turn a committed TX-A into an error response, and every listed result belongs to one closed response union. A global exception handler maps only known safe categories; unexpected failures become an opaque 500 occurrence, never an exception dump.

Queries expose owned representations, not aggregates

Query concern	Decision-status endpoint	Boundary rule
Route	GET /api/v1/alarm-flood-decisions/{decisionId}	Provider-owned resource name; no table name, aggregate CLR type or outbox route.
Authorization	alarm-management.read-alarm-flood plus resource/site policy	Authorize before returning representation; existence-disclosure policy is consistent.
Source	Dedicated read projection from AlarmManagement-owned state	No cross-database join and no synchronous RootCause call hidden in repository.
Representation	decisionId, alarmFloodId, acceptedAtUtc, decisionStatus, optional publicFactId	No EF navigation, rowversion bytes, private event list, token or stack trace.
Downstream state	Not represented	A missing RootCause field is not false/failed; query RootCause/BFF separately with freshness semantics.
Caching	Private/no-store by default; ETag only after data-classification review	Authorized personalized responses never enter a shared cache accidentally; 304 preserves authorization.
Pagination/filtering	Not required for single-resource lookup	Collection queries require bounded page/filter/sort vocabularies and stable continuation semantics.
Consistency	Provider-local committed view with documented projection lag if any	No strong distributed snapshot claim across AlarmManagement and RootCause.

Status and failure semantics

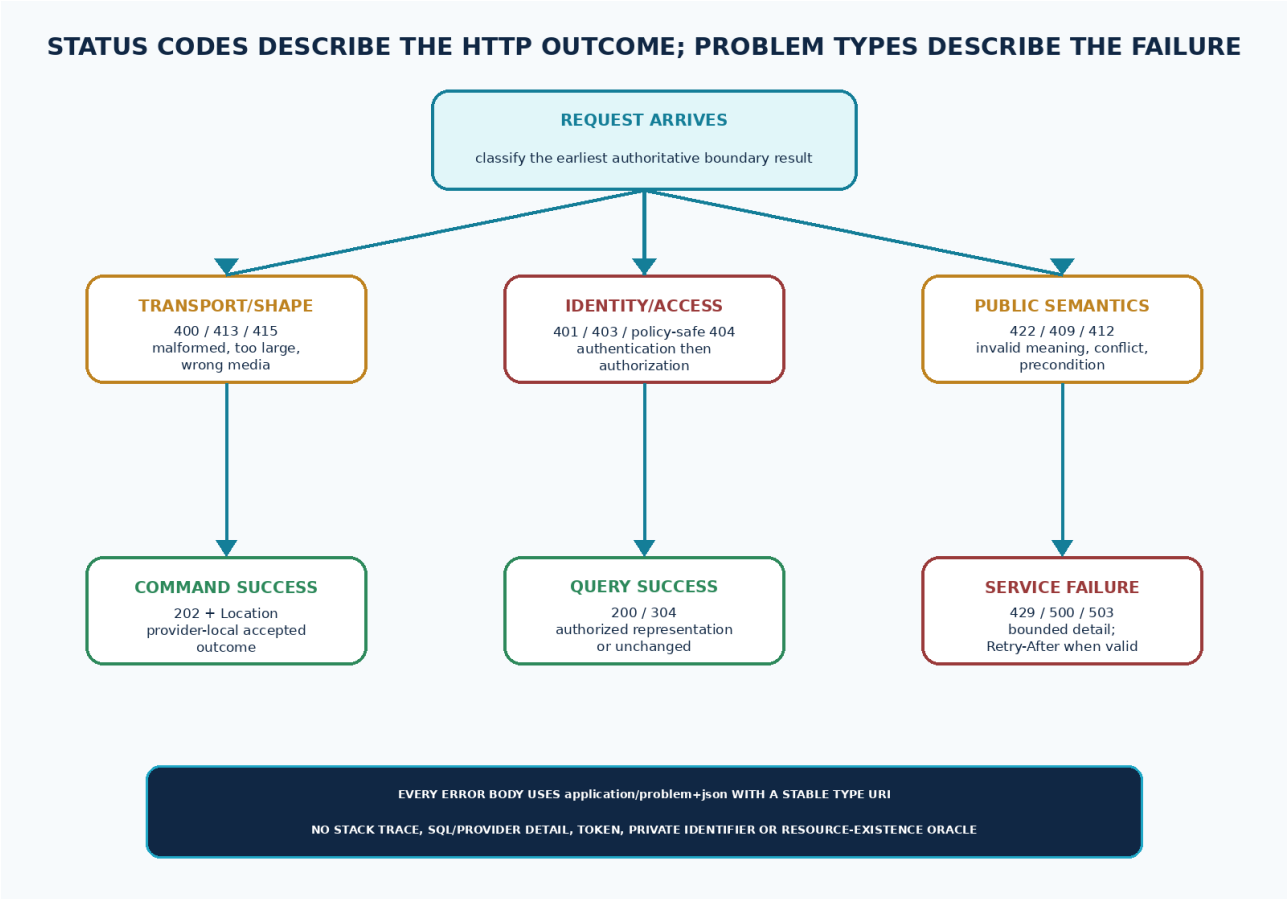


Figure 21.3 - HTTP status codes classify the boundary outcome while stable Problem Details types carry safe machine-readable failure meaning.

Status	NEXUS-1 meaning	Required response behavior
200 OK	Authorized query returns current provider-owned representation.	Document media/version/cache semantics; never imply a cross-service snapshot.
202 Accepted	AlarmManagement accepted and committed the local command/public intent; wider asynchronous workflow is incomplete.	Location plus DetectAlarmFloodAcceptedV1; no downstream result; Cache-Control: no-store.
304 Not Modified	Authorized GET representation matches conditional validator.	No body; authorization and cache policy still evaluated; ETag semantics documented.
400 Bad Request	Malformed JSON/public representation or invalid request syntax.	Stable validation Problem Details; bounded pointers; no internal parser detail.
401 Unauthorized	Accepted authentication credentials are absent or invalid.	Applicable WWW-Authenticate; no resource existence or authorization detail.
403 Forbidden	Caller is authenticated but policy denies the operation.	Stable safe problem; policy may use 404 to hide protected resource existence.
404 Not Found	Provider route/resource is absent or deliberately hidden by accepted disclosure policy.	Identical safe semantics for hidden/absent cases where required; no oracle.
409 Conflict	Request conflicts with current owner state or a completed incompatible business decision.	Stable conflict type and safe corrective hint; no raw concurrency/row detail.
412 Precondition Failed	Documented If-Match/If-Unmodified-Since condition is false.	Current safe validator if allowed; Chapter 23 defines exact mutation use.
413 Content Too Large	Body exceeds pre-parse/decompressed contract limit.	Reject before large allocation; do not echo body.
415 Unsupported Media Type	Request representation/content coding is unsupported.	Document accepted media types; no best-effort parser fallback.
422 Unprocessable Content	Well-formed request violates public semantic/cross-field rules.	Stable problem type and error pointers; Domain exception is not required.
429 Too Many Requests	Caller/tenant/site quota is exceeded.	Retry-After when meaningful; do not expose other tenants or limiter internals.
500 Internal Server Error	Unexpected provider failure after safe classification.	Opaque occurrence ID; no stack, SQL/provider details,

Status	NEXUS-1 meaning	Required response behavior
503 Service Unavailable	Provider cannot currently execute required owner work.	Retry-After when known; never return 202 unless TX-A actually committed.

One stable Problem Details shape

IMPLEMENTATION SHAPE

```
HTTP/1.1 422 Unprocessable Content
Content-Type: application/problem+json

{
  "type": "https://nexus-1.example/problems/alarm-flood-window-invalid",
  "title": "The alarm-flood window is invalid.",
  "status": 422,
  "detail": "The supplied time window does not satisfy the public contract.",
  "instance": "urn:nexus1:problem:problem-am-5508",
  "errorCode": "alarm_flood_window_invalid",
  "errors": [
    {
      "pointer": "/windowEndedAtUtc",
      "code": "must_be_after_window_start"
    }
  ]
}
```

The HTTP status and status member must agree. The type URI is the primary machine identifier; title/detail are safe human-readable text and may evolve without changing program logic. instance names one problem occurrence, not the request, decision, trace, alarm, case or database row. Unknown extension members are ignored by clients as required by RFC 9457; the API still version-controls errorCode and errors semantics.

Problem member	Rule	Never expose
type	Stable documentation URI controlled by NEXUS-1; primary programmatic identifier.	CLR exception name, internal wiki path or ephemeral host URL.
title	Short summary of the problem type; stable enough for operators, not machine branching.	Sensitive resource/user data.
status	Same integer as the actual HTTP status.	A contradictory advisory value.
detail	Occurrence-specific but safe and actionable at the public boundary.	Stack, SQL/provider, policy internals, credentials, private evidence or existence detail.
instance	Opaque problem-occurrence URI/URN usable by authorized support correlation.	Trace/span ID presented as business identity or guessable row key.
errorCode	Stable NEXUS-1 machine code subordinate to type URI.	Localized sentence or raw exception message.
errors	Bounded list of public JSON pointers/codes for same validation problem type.	Private property names, full rejected values or mixed unrelated failure types.

Lost responses, cancellation and timeouts

Timing point	Server truth	Client-visible rule
Disconnect before validation	No Application command and no TX-A.	No result can be assumed; retry follows Chapter 23 idempotency policy.
Cancellation before commit	Handler stops if safe; no owner change/public intent.	No false success; server records bounded cancellation outcome.
TX-A commits, response write fails	decision-am-9184 and fact-am-7716 exist despite lost 202.	Client cannot infer failure. Chapter 23 idempotency/status lookup resolves ambiguity.
Response succeeds, dispatch pending	AlarmManagement acceptance exists; broker/RootCause status unknown/not complete.	202 remains correct; do not block response on broker publish.
Broker/RootCause later fails	TX-A remains committed; recovery/dead-letter policies own later attempts.	Original 202 is not rewritten. Separate query/notification reports later owner-visible state.
Request timeout after Domain work but before commit result known	Corrupted state; no TX-A; no response	Client must not assume success; must follow Chapter 23 idempotency policy.

HTTP and authorization test matrix

Fixture	Expected result	Blocking assertion
Golden authorized POST	202, Location/body agreement and exact provider-local identities.	TX-A committed once; no RootCause field/claim.
Malformed JSON / duplicate member / wrong type	400 before Application/Domain.	Stable problem and no input echo/private parser detail.
Oversize/decompression bomb / excessive array	413 or bounded 400 by documented layer.	Allocation/CPU limits and no body logging.
Unsupported media/content coding	415 with accepted representation guidance.	No best-effort deserialization.
Missing/invalid/expired/wrong-audience token	401 plus correct challenge.	No resource lookup/existence detail.
Authenticated without scope/site permission	403 or policy-safe 404.	Same response timing/body class for protected absence policy.
Unknown severity, invalid window, duplicate AlarmIds	422 with stable pointers/codes.	No defaulting, Domain call or leaked rejected values.
Owner state conflict/concurrency	409 or documented 412.	No exception/rowversion bytes; one accepted outcome mapping.
DB unavailable before TX-A	503/opaque safe problem; Retry-After if meaningful.	No 202 and no pending public fact.
TX-A committed, response lost	One decision/public fact remains; retry ambiguity is observable.	Chapter 23 status/idempotency fixture; no duplicate effect.
Authorized decision GET	200 provider-owned projection; conditional 304 if enabled.	No cross-service call/entity leak; cache headers safe.
Unauthorized/hidden decision GET	403 or policy-safe 404 consistently.	No existence oracle in body/timing/logs.
Unexpected exception	500 application/problem+json with opaque instance.	Stack, SQL/provider, token, headers and private IDs absent.
OpenAPI/client fixture	Declared request/response/problem/status union matches runtime.	No undocumented status/body and no internal schemas.

Evidence catalogue

Evidence ID	Acceptance statement	Minimum artifact
API-BOUND-001	HTTP, API DTO, Application, Domain and Infrastructure dependencies point inward without type leaks.	Project/reference scan and endpoint dependency test.
API-CMD-001	DetectAlarmFlood command exposes one versioned public capability and bounded 202 semantics.	Route/DTO/status/Location contract and golden POST fixture.
API-QRY-001	Decision query returns an authorized provider-owned representation without side effects or remote joins.	Read model, mapping, cache and absence/access tests.
API-VAL-001	Protocol, shape and public semantic rules execute at defined stages with bounded diagnostics.	Validator catalogue, malformed/limit/cross-field machine results.
API-AUTHN-001	Only accepted credential schemes/issuers/audiences create verified actor context.	Authentication configuration and negative token matrix.
API-AUTHZ-001	Named policies/resource handlers enforce capability/site access without existence leaks.	Policy catalogue, handler tests and 401/403/404 oracle tests.
API-MAP-001	Explicit DTO/Application/outcome/response mappers carry approved fields only.	Mapping tests and entity/HttpContext/exception dependency scan.
API-STATUS-001	Every branch has accepted status/header/body semantics and 202 never overclaims downstream completion.	Closed response matrix and HTTP integration results.
API-PROB-001	RFC 9457 problem types are stable, safe and consistent with actual status.	Problem catalogue, snapshot/client/security scans.
API-LOSS-001	Disconnect/cancellation/commit/response-loss points preserve honest outcome semantics.	Fault-injection matrix linked to TX-A and Chapter 23.
API-COMP-001	OpenAPI/generated clients and old/new request/response/problem versions remain compatible.	Chapter 24 producer/client matrix and schema corpus.
API-REVIEW-001	API owner, security, consumers, operations and architecture accept obligations/residual risks.	Evidence index with build/config/owners/expiry/triggers.

ADR-019 - Explicit command/query service boundaries

ADR field	ADR-019 record
Status	Accepted for the NEXUS-1 reference design; endpoint, security, HTTP, Problem Details and runtime evidence remain pending.
Context	The synchronous edge must expose provider capabilities and representations without importing aggregate/EF shapes, leaking security/domain internals or promising distributed completion.
Decision	Use distinct versioned HTTP request/response/problem contracts and explicit mappers. Commands authenticate, authorize, validate and execute one owner-local Application use case; queries return owner projections without business side effects or hidden cross-service joins. POST alarm-flood-detections returns 202 plus an AlarmManagement Location only after TX-A commit, never RootCause completion. Errors use RFC 9457 with stable safe types. Status semantics follow RFC 9110. Exact endpoint versioning/idempotency is deferred to Chapter 23 and compatibility execution to Chapter 24.
Consequences	Route/DTO/policy/validator/mapper/status/problem catalogues and HTTP fault/security fixtures become release artifacts. Some contract duplication is intentional. The BFF owns UI aggregation; services retain independent authorization and data ownership.
Rejected	Bind aggregate/EF entity; share broker/API DTO; pass HttpContext/token into Domain; role checks scattered in handlers; validate protected resource detail before authorization; query with side effects; synchronous RootCause call in TX-A; return 200/202 before commit; expose case ID or downstream success; serialize exception/stack/SQL; undocumented status/body; browser-orchestrated workflow.
Review trigger	Route/method/media/version, request/response field, status/problem type, authentication issuer/scheme, authorization policy, validation order, mapper, cache/precondition, timeout/loss behavior, downstream claim or supported client changes.

Risk register

Risk	Early signal	Blocking control
Entity becomes API contract	Navigation/rowversion/private fields appear in JSON or OpenAPI follows migration changes.	API-BOUND/MAP scan and contract-only DTOs.
202 overclaims workflow	Response contains case ID, delivered/processed/succeeded or waits for RootCause.	API-STATUS golden fixture and downstream-field negative scan.
Authorization after data disclosure	Detailed 404/validation/timing differs for denied resources.	API-AUTHZ existence-oracle matrix and validation-order review.
Body identity trusted	Actor/tenant/site scope comes from unverified JSON fields.	Verified principal + resource policy; mapper input provenance tests.
Validation becomes Domain exception	Malformed/unknown public fields enter aggregates or raw messages return.	API-VAL layer tests and stable outcome mapping.
Query causes side effect	GET changes acknowledgement, dispatch, metrics-backed business state or case status.	Read-only dependency/transaction tests and audit.
Hidden cross-service join	AlarmManagement GET depends on RootCause availability or shared DB query.	API-QRY owner-only read model; Chapter 22 BFF composition.
Lost response creates duplicates	Client retry produces second decision/fact after TX-A committed.	API-LOSS plus Chapter 23 idempotency/status evidence.
Problem body leaks internals	Stack, SQL/provider, token, headers, private IDs or rejected body in output/logs.	API-PROB/SEC output and log scans.
Status/body drift	OpenAPI declares one union while runtime returns undocumented 200/500/plain text.	Closed response tests and generated-client matrix.
Shared cache leaks authorized representation	Decision view cached across actors/tenants.	Private/no-store default, Vary/ETag review and cache tests.
Security dependency becomes availability coupling	Authorization calls RootCause or another remote owner per request.	Local owned policy evidence and explicit fail behavior.

Official-source validation

The HTTP and ASP.NET Core rules were checked against current primary sources on 19 July 2026. RFC 9110 defines HTTP method/status semantics and describes 202 Accepted as intentionally noncommittal when processing is not complete. RFC 9457 defines application/problem+json, makes the type URI the primary problem identifier, requires the status member to agree with the HTTP status, permits extension members and warns against

policy-based authorization as requirements evaluated by handlers and ASP.NET Core's Problem Details service/error-handling middleware. The NEXUS-1 routes, fields, policies, problem catalogue and disclosure choices remain project decisions.

Primary source	Stable point used here	NEXUS-1 decision
RFC 9110 - HTTP Semantics	Methods, status codes, conditional requests and 202 Accepted have defined protocol meaning.	Status codes classify the real boundary outcome; 202 never becomes a generic successful POST.
RFC 9457 - Problem Details for HTTP APIs	type is the primary problem identifier; status agrees with HTTP; extensions evolve; sensitive implementation detail requires care.	Stable safe problem types/codes/pointers; no exception, SQL, token or private evidence exposure.
Microsoft Learn - Policy-based authorization in ASP.NET Core	Policies contain requirements; handlers evaluate requirements against authorization context/resource.	Named capability/site policies separate verified identity from access decisions and remain independently testable.
Microsoft Learn - Handle errors in ASP.NET Core APIs	ASP.NET Core provides IProblemDetailsService/AddProblemDetails and middleware integration for API failures.	One configured writer maps accepted categories to the NEXUS-1 RFC 9457 catalogue; development detail never reaches production.

Sources: <https://www.rfc-editor.org/rfc/rfc9110.html> ; <https://www.rfc-editor.org/rfc/rfc9457.html> ; <https://learn.microsoft.com/en-us/aspnet/core/security/authorization/policies> ; <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/error-handling-api>

Technical material still required

The command/query boundary, DTOs, mappings, 202/Location exchange, validation/security order, policy names, query projection, status/problem catalogue, lost-response behavior, tests, evidence IDs and ADR are specified. No NEXUS-1 Edge project has compiled; no authentication scheme, authorization handler, validator, endpoint, Problem Details writer, OpenAPI schema, generated client, HTTP server, TX-A fault injection or negative security fixture has executed.

TECHNICAL MATERIAL STATUS - [HTTP COMMAND/QUERY ENDPOINT IMPLEMENTATION, AUTHORIZATION, FAILURE CONTRACT AND RUNTIME EVIDENCE REQUIRED]

Implement the net8.0 AlarmManagement Edge contracts and routes, strict transport/public validators, authentication configuration, named policy/resource authorization handlers, DTO/Application/outcome/response mappers, owner-only decision projection, closed typed result unions, RFC 9457 writer/catalogue, safe exception/status mapping, body/decompression limits, cache/conditional behavior and OpenAPI generation; compile and execute golden 202, query, malformed/oversize/media, token issuer/audience/expiry, permission/existence-oracle, semantic/conflict/precondition, DB/concurrency, disconnect/cancellation/TX-A-response-loss, 5xx output/log scan and old/new generated-client matrices; publish API-BOUND-001 through API-REVIEW-001 linked to TXN/CTR/ENV evidence and Chapters 23-24. RootCause remains G0 accepted and G1 specified; runtime evidence has not advanced.

Deliverable	Minimum technical content	Blocking evidence
Command edge	Versioned DTOs, POST route, strict validators, 202/Location and explicit mappings.	API-CMD/VAL/MAP/STATUS-001
Security	Accepted authentication scheme plus named capability/resource policies and safe 401/403/404 behavior.	API-AUTHN/AUTHZ-001
Query edge	Owner projection, read-only handler, access/caching/absence semantics and no remote join.	API-QRY-001
Failure contract	RFC 9457 catalogue/writer, closed status union and safe exception/infrastructure categories.	API-PROB/STATUS-001
Ambiguous outcomes	Disconnect/cancellation/commit/response-loss fault injection tied to TX-A and Chapter 23.	API-LOSS-001
Compatibility	OpenAPI/schema/generated-client old/new matrices for requests, responses and problems.	API-COMP-001 / Chapter 24
Acceptance	Machine results bound to build/config/fixtures with security/consumer/operations review.	API-REVIEW-001

CHAPTER 21 - CODE AUDIT

The public contract compiles without importing owner internals

The Edge package fixes client-visible names and JSON behavior. It carries no aggregate, EF entity, token object, broker envelope or RootCause type; every transition into Application is explicit and reviewable.

EVIDENCE STATUS

These listings are compile-oriented reference material. No Edge project, token issuer, endpoint host, database provider, OpenAPI document or HTTP corpus has executed here. API evidence remains PENDING.

EXECUTABLE ASSET 21-A - VERSIONED EDGE CONTRACTS

```
[JsonUnmappedMemberHandling(JsonUnmappedMemberHandling.Disallow)]
public sealed record DetectAlarmFloodRequestV1(
    string SiteId,
    DateTimeOffset WindowStartedAtUtc,
    DateTimeOffset WindowEndedAtUtc,
    IReadOnlyList<string> AlarmIds,
    string RequestedSeverity,
    string ClientRequestId);

public sealed record DetectAlarmFloodAcceptedV1(
    string RequestId,
    string DecisionId,
    string AlarmFloodId,
    string PublicFactId,
    string Status,
    DateTimeOffset AcceptedAtUtc,
    string StatusUrl);

public sealed record AlarmFloodDecisionViewV1(
    string DecisionId,
    string AlarmFloodId,
    string DecisionStatus,
    DateTimeOffset AcceptedAtUtc,
    string? PublicFactId);

[JsonSourceGenerationOptions(
    PropertyNamingPolicy = JsonKnownNamingPolicy.CamelCase,
    GenerationMode = JsonSourceGenerationMode.Metadata)]
[JsonSerializable(typeof(DetectAlarmFloodRequestV1))]
[JsonSerializable(typeof(DetectAlarmFloodAcceptedV1))]
[JsonSerializable(typeof(AlarmFloodDecisionViewV1))]
internal sealed partial class AlarmEdgeJsonContext
    : JsonSerializerContext;
```

EXECUTABLE ASSET 21-B - CONTRACT DEPENDENCY GATE

```
[Fact]
public void API_BOUND_001_Edge_contracts_have_no_owner_runtime_dependency()
{
    var assembly = typeof(DetectAlarmFloodRequestV1).Assembly;
    foreach (var forbidden in new[]
    {
        "Microsoft.EntityFrameworkCore",
        "Microsoft.AspNetCore.Http",
        "Nexus1.AlarmManagement.Domain",
        "Nexus1.AlarmManagement.Infrastructure",
        "Nexus1.RootCause",
        "RabbitMQ.Client"
    })
    {
        Types.InAssembly(assembly).Should()
            .NotHaveDependencyOn(forbidden).GetResult()
            .IsSuccessful.Should().BeTrue();
    }
}
```


DETERMINISTIC PUBLIC VALIDATION

Public shape is rejected before owner values are created

EXECUTABLE ASSET 21-C - STABLE VALIDATION RESULT

```
public sealed record PublicViolation(string Pointer, string Code);

public sealed record PublicValidationResult(
    IReadOnlyList<PublicViolation> Violations)
{
    public bool IsValid => Violations.Count == 0;
}

public sealed class DetectAlarmFloodRequestValidator
{
    private static readonly HashSet<string> Severities =
        new(StringComparer.Ordinal)
        {
            "warning", "critical"
        };

    public PublicValidationResult Validate(
        DetectAlarmFloodRequestV1 request)
    {
        var failures = new List<PublicViolation>();
        if (string.IsNullOrEmpty(request.SiteId)
            || request.SiteId.Length > 80)
            failures.Add(new("/siteId", "site-id-invalid"));
        if (request.WindowStartedAtUtc >= request.WindowEndedAtUtc)
            failures.Add(new("/windowEndedAtUtc", "window-order-invalid"));
        if (request.WindowEndedAtUtc - request.WindowStartedAtUtc
            > TimeSpan.FromMinutes(15))
            failures.Add(new("/windowEndedAtUtc", "window-too-large"));
        if (request.AlarmIds is null
            || request.AlarmIds.Count is < 1 or > 500)
            failures.Add(new("/alarmIds", "alarm-count-invalid"));
        else if (request.AlarmIds.Any(string.IsNullOrEmpty)
            || request.AlarmIds.Distinct(StringComparer.Ordinal).Count()
            != request.AlarmIds.Count)
            failures.Add(new("/alarmIds", "alarm-id-set-invalid"));
        if (!Severities.Contains(request.RequestedSeverity))
            failures.Add(new("/requestedSeverity", "severity-unknown"));
        if (string.IsNullOrEmpty(request.ClientRequestId)
            || request.ClientRequestId.Length > 100)
            failures.Add(new("/clientRequestId", "request-id-invalid"));
        return new(failures);
    }
}
```

EXECUTABLE ASSET 21-D - EXPLICIT PROVENANCE-AWARE MAPPER

```
public sealed record VerifiedActorContext(
    string Subject,
    string Tenant);

public sealed class DetectAlarmFloodMapper
{
    public DetectAlarmFlood ToApplication(
        DetectAlarmFloodRequestV1 request,
        VerifiedActorContext actor) =>
        new(
            SiteIdentity.Parse(request.SiteId),
            AlarmWindow.Create(
                request.WindowStartedAtUtc,
                request.WindowEndedAtUtc),
            request.AlarmIds.Select(AlarmIdentity.Parse).ToArray(),
            AlarmSeverity.Parse(request.RequestedSeverity),
            ClientCorrelationId.Parse(request.ClientRequestId),
            ActorIdentity.Parse(actor.Subject),
            TenantIdentity.Parse(actor.Tenant));
}

[Fact]
public void API_MAP_001_body_cannot_supply_actor_or_owner_identity()
{
    var command = fixture.Map(
        fixture.ValidRequest,
        fixture.VerifiedActor("subject-17", "tenant-a"));
    command.Actor.Should().Be(ActorIdentity.Parse("subject-17"));
    command.Tenant.Should().Be(TenantIdentity.Parse("tenant-a"));
    command.DecisionIdentity.Value.Should().NotBe(
        fixture.ValidRequest.ClientRequestId);
}
```

VALIDATION HAS ONE PUBLIC VOCABULARY

Pointers and codes are stable API data. Domain exceptions, localized messages, database keys and authorization facts never become validation output.

AUTHENTICATION AND RESOURCE AUTHORIZATION

Verified claims grant no capability until a named policy succeeds

EXECUTABLE ASSET 21-E - PINNED BEARER VALIDATION AND POLICIES

```
services
  .AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
  .AddJwtBearer(options =>
  {
    options.Authority = configuration["Auth:Authority"]
    ?? throw new InvalidOperationException(
      "Auth authority is required.");
    options.Audience = "nexus1.alarm-management";
    options.RequireHttpsMetadata = true;
    options.MapInboundClaims = false;
    options.TokenValidationParameters = new()
    {
      ValidateIssuer = true,
      ValidIssuer = configuration["Auth:Issuer"],
      ValidateAudience = true,
      ValidAudience = "nexus1.alarm-management",
      ValidateLifetime = true,
      ValidateIssuerSigningKey = true,
      ClockSkew = TimeSpan.FromSeconds(30),
      NameClaimType = "sub"
    };
  });

services.AddAuthorization(options =>
{
  options.AddPolicy(PolicyNames.AlarmFloodWriteCoarse, policy =>
  {
    policy.RequireAuthenticatedUser();
    policy.RequireClaim("scope", "alarm-flood.write");
  });
  options.AddPolicy(PolicyNames.DetectAlarmFlood, policy =>
  {
    policy.RequireAuthenticatedUser();
    policy.RequireClaim("scope", "alarm-flood.write");
    policy.AddRequirements(new SiteAccessRequirement());
  });
  options.AddPolicy(PolicyNames.ReadAlarmFlood, policy =>
  {
    policy.RequireAuthenticatedUser();
    policy.RequireClaim("scope", "alarm-flood.read");
    policy.AddRequirements(new SiteAccessRequirement());
  });
});
```

EXECUTABLE ASSET 21-F - RESOURCE HANDLER AND ACTOR FACTORY

```
public sealed record AlarmSiteResource(string CanonicalSiteId);
public sealed class SiteAccessRequirement : IAuthorizationRequirement;

public sealed class SiteAccessHandler :
  AuthorizationHandler<SiteAccessRequirement, AlarmSiteResource>
{
  protected override Task HandleRequirementAsync(
    AuthorizationHandlerContext context,
    SiteAccessRequirement requirement,
    AlarmSiteResource resource)
  {
    {
      var allowed = context.User.FindAll("site")
        .Any(x => StringComparer.Ordinal.Equals(
          x.Value, resource.CanonicalSiteId));
      if (allowed) context.Succeed(requirement);
      return Task.CompletedTask;
    }
  }
}

public sealed class VerifiedActorFactory
{
  public VerifiedActorContext Create(ClaimsPrincipal principal)
  {
    {
      var subject = principal.FindFirstValue("sub")
        ?? throw new UnusableAuthenticatedPrincipal();
      var tenant = principal.FindFirstValue("tenant")
        ?? throw new UnusableAuthenticatedPrincipal();
      return new(subject, tenant);
    }
  }
}

[Theory]
[InlineData("wrong-issuer")]
[InlineData("wrong-audience")]
[InlineData("expired")]
[InlineData("missing-signature")]
public async Task API_AUTHN_001_invalid_token_has_no_owner_effect(
  string tokenCase)
{
  var response = await fixture.PostWithTokenAsync(tokenCase);
  response.StatusCode.Should().Be(HttpStatusCode.Unauthorized);
  fixture.AlarmManagementEffects.Should().BeEmpty();
}
```

THE BODY IS NOT AN AUTHORITY SOURCE

Site access is evaluated from the verified principal against a canonical resource. A JSON actor, tenant or site claim cannot authenticate or authorize itself.

COMMAND EDGE AND BOUNDED 202

The endpoint returns only after TX-A proves owner-local acceptance

EXECUTABLE ASSET 21-G - PRODUCTION-PATH POST HANDLER

```
public static async Task<IResult> DetectAlarmFloodAsync(
    DetectAlarmFloodRequestV1 request,
    ClaimsPrincipal principal,
    DetectAlarmFloodRequestValidator validator,
    IAuthorizationService authorization,
    VerifiedActorFactory actors,
    DetectAlarmFloodMapper mapper,
    IDetectAlarmFloodUseCase useCase,
    DetectAlarmFloodResponseMapper responses,
    CancellationToken cancellationToken)
{
    var validation = validator.Validate(request);
    if (!validation.IsValid)
        return responses.InvalidPublicRequest(validation);

    var resource = new AlarmSiteResource(request.SiteId);
    var access = await authorization.AuthorizeAsync(
        principal, resource, PolicyNames.DetectAlarmFlood);
    if (!access.Succeeded)
        return TypedResults.NotFound();

    var actor = actors.Create(principal);
    var command = mapper.ToApplication(request, actor);
    var outcome = await useCase.HandleAsync(
        command, cancellationToken);
    return responses.Map(outcome);
}

routes.MapPost(
    "/api/v1/alarm-flood-detections",
    DetectAlarmFloodAsync)
    .WithName("DetectAlarmFloodV1")
    .Accepts<DetectAlarmFloodRequestV1>("application/json")
    .Produces<DetectAlarmFloodAcceptedV1>(
        StatusCodes.Status202Accepted)
    .ProducesProblem(StatusCodes.Status422UnprocessableEntity)
    .RequireAuthorization(PolicyNames.AlarmFloodWriteCoarse);
```

EXECUTABLE ASSET 21-H - CLOSED OWNER OUTCOME MAPPING

```
public sealed class DetectAlarmFloodResponseMapper(ApiProblems problems)
{
    public IResult Map(DetectAlarmFloodOutcome outcome) =>
    {
        outcome switch
        {
            DetectAlarmFloodOutcome.Accepted x => Accepted(x),
            DetectAlarmFloodOutcome.SemanticallyInvalid x =>
                problems.Semantic(x.Code, x.Pointer),
            DetectAlarmFloodOutcome.Conflict x =>
                problems.Conflict(x.Code),
            DetectAlarmFloodOutcome.PreconditionFailed x =>
                problems.Precondition(x.Code),
            _ => throw new UnmappedDetectAlarmFloodOutcome(
                outcome.GetType())
        };

        private static IResult Accepted(
            DetectAlarmFloodOutcome.Accepted outcome)
        {
            var location =
                $"/api/v1/alarm-flood-decisions/{outcome.DecisionId}";
            var body = new DetectAlarmFloodAcceptedV1(
                outcome.ClientRequestId,
                outcome.DecisionId,
                outcome.AlarmFloodId,
                outcome.PublicFactId,
                "accepted",
                outcome.AcceptedAtUtc,
                location);
            return TypedResults.Accepted(location, body);
        }

        public IResult InvalidPublicRequest(PublicValidationResult value) =>
            problems.Validation(value.Violations);
    }

    [Fact]
    public void API_STATUS_001_accepted_body_contains_no_consumer_claim()
    {
        var json = fixture.MapAcceptedOutcomeToJson();
        json.Should().NotContain("caseId");
        json.Should().NotContain("delivered");
        json.Should().NotContain("processed");
        json.Should().NotContain("rootCause");
    }
}
```

202 IS NOT A WORKFLOW VERDICT

The use case returns only after the owner-local transaction commits. The response names the AlarmManagement decision and public fact, never a RootCause result.

OWNER-ONLY QUERY EDGE

The GET reads one projection without effects or remote joins

EXECUTABLE ASSET 21-I - READ-ONLY PROVIDER PROJECTION

```
internal sealed record DecisionAccessProjection(
    string DecisionId,
    string SiteId,
    string AlarmFloodId,
    string Status,
    DateTimeOffset AcceptedAtUtc,
    string? PublicFactId,
    long Version);

internal sealed class SqlAlarmFloodDecisionQueries(
    AlarmManagementDbContext db) : IAlarmFloodDecisionQueries
{
    public Task<DecisionAccessProjection?> FindAsync(
        string decisionId,
        CancellationToken cancellationToken) =>
        db.AlarmFloodDecisions
            .AsNoTracking()
            .Where(x => x.DecisionId == decisionId)
            .Select(x => new DecisionAccessProjection(
                x.DecisionId,
                x.SiteId,
                x.AlarmFloodId,
                x.Status,
                x.AcceptedAtUtc,
                x.PublicFactId,
                x.Version))
            .SingleOrDefaultAsync(cancellationToken);
}
```

EXECUTABLE ASSET 21-J - AUTHORIZED CONDITIONAL GET

```
public static async Task<IResult> GetAlarmFloodDecisionAsync(
    string decisionId,
    HttpContext http,
    IAlarmFloodDecisionQueries queries,
    IAuthorizationService authorization,
    CancellationToken cancellationToken)
{
    var row = await queries.FindAsync(decisionId, cancellationToken);
    if (row is null) return TypedResults.NotFound();

    var access = await authorization.AuthorizeAsync(
        http.User,
        new AlarmSiteResource(row.SiteId),
        PolicyNames.ReadAlarmFlood);
    if (!access.Succeeded) return TypedResults.NotFound();

    var etag = $"{row.Version:x16}";
    http.Response.Headers.ETag = etag;
    http.Response.Headers.CacheControl = "private, no-store";
    if (http.Request.Headers.IfNoneMatch.Any(
        x => StringComparer.Ordinal.Equals(x, etag)))
        return TypedResults.StatusCode(
            StatusCodes.Status304NotModified);

    return TypedResults.Ok(new AlarmFloodDecisionViewV1(
        row.DecisionId,
        row.AlarmFloodId,
        row.Status,
        row.AcceptedAtUtc,
        row.PublicFactId));
}

[Fact]
public async Task API_QRY_001_GET_has_no_business_or_remote_effect()
{
    await fixture.GetDecisionAsync("decision-am-9184");
    fixture.SaveChangesCalls.Should().Be(0);
    fixture.OutboxWrites.Should().Be(0);
    fixture.RootCauseClientCalls.Should().Be(0);
    fixture.Db.ChangeTracker.Entries().Should().BeEmpty();
}
```

ABSENCE AND DENIAL SHARE THE ACCEPTED DISCLOSURE POLICY

The Edge may read the owner projection needed for resource authorization, but it returns the same safe 404 representation for missing and denied identities.

RFC 9457 FAILURE CONTRACT

Every failure branch has a stable type and bounded detail

EXECUTABLE ASSET 21-K - SAFE PROBLEM CATALOGUE

```
public sealed record ApiProblemDescriptor(
    int Status,
    string Type,
    string Title,
    string Code);

public static class AlarmApiProblemCatalogue
{
    public static readonly ApiProblemDescriptor Invalid =
        new(422, "/problems/invalid-alarm-flood",
            "Request semantics are invalid", "alarm-flood-invalid");
    public static readonly ApiProblemDescriptor Conflict =
        new(409, "/problems/alarm-flood-conflict",
            "The owner state conflicts", "alarm-flood-conflict");
    public static readonly ApiProblemDescriptor Precondition =
        new(412, "/problems/precondition-failed",
            "The HTTP precondition failed", "precondition-failed");
    public static readonly ApiProblemDescriptor Unavailable =
        new(503, "/problems/service-unavailable",
            "The service is temporarily unavailable",
            "service-unavailable");
    public static readonly ApiProblemDescriptor Unexpected =
        new(500, "/problems/internal-error",
            "The request could not be completed", "internal-error");
}

public sealed class SafeExceptionHandler(
    IProblemDetailsService writer,
    ApiProblemFactory problems) : IExceptionHandler
{
    public async ValueTask<bool> TryHandleAsync(
        HttpContext http,
        Exception exception,
        CancellationToken cancellationToken)
    {
        var descriptor = exception switch
        {
            OwnerStoreUnavailable =>
                AlarmApiProblemCatalogue.Unavailable,
            _ => AlarmApiProblemCatalogue.Unexpected
        };
        await writer.WriteAsync(new ProblemDetailsContext
        {
            HttpContext = http,
            ProblemDetails = problems.Create(
                http, descriptor, extensions: null)
        });
        return true;
    }
}
```

EXECUTABLE ASSET 21-L - STATUS AND LEAK GATES

```
[Theory]
[MemberData(nameof(ClosedFailureMatrix))]
public async Task API_PROB_001_status_type_and_code_agree(
    FailureFixture test)
{
    var response = await fixture.ExecuteAsync(test);
    var problem = await response.Content
        .ReadFromJsonAsync<ProblemDetails>();
    ((int)response.StatusCode).Should().Be(test.Status);
    problem!.Status.Should().Be(test.Status);
    problem.Type.Should().Be(test.Type);
    problem.Extensions["code"].ToString().Should().Be(test.Code);
}

[Fact]
public async Task unexpected_failure_exposes_no_internal_canary()
{
    var response = await fixture.TriggerUnexpectedFailureAsync(
        "Server=secret;Password=canary; stack=canary");
    var body = await response.Content.ReadAsStringAsync();
    body.Should().NotContain("Password=");
    body.Should().NotContain("stack=canary");
    fixture.PublicLogs.Should().NotContain(
        x => x.Contains("Password=", StringComparison.Ordinal));
}

[Fact]
public async Task missing_and_denied_have_same_public_oracle()
{
    var missing = await fixture.GetMissingDecisionAsync();
    var denied = await fixture.GetDeniedDecisionAsync();
    denied.StatusCode.Should().Be(missing.StatusCode);
    (await denied.Content.ReadAsStringAsync()).Should()
        .Be(await missing.Content.ReadAsStringAsync());
}
```

EXCEPTION TEXT IS NOT AN API CONTRACT

The catalogue selects status, type, title and code. Provider messages, SQL, token material, private identifiers, request bodies and stack traces remain internal.

PRODUCTION-PATH AND AMBIGUOUS-OUTCOME TESTS

The real host proves 202 truth and a lost response stays ambiguous

EXECUTABLE ASSET 21-M - GOLDEN HOSTED EXCHANGE

```
public sealed class AlarmApiFactory :
    WebApplicationFactory<Program>
{
    protected override void ConfigureWebHost(
        IWebHostBuilder builder) =>
    {
        builder.UseEnvironment("ContractTest")
            .ConfigureTestServices(services =>
            {
                services.ReplaceOwnedSqlProvider();
                services.ReplaceIssuerWithTestIssuer();
            });
    }
}

[Fact]
public async Task API_CMD_001_real_host_returns_bounded_202()
{
    using var request = fixture.ValidDetectAlarmFloodRequest();
    using var response = await fixture.Client.SendAsync(request);
    response.StatusCode.Should().Be(HttpStatusCode.Accepted);

    var body = await response.Content
        .ReadFromJsonAsync<DetectAlarmFloodAcceptedV1>();
    response.Headers.Location!.OriginalString.Should()
        .Be(body!.StatusUrl);
    body.Status.Should().Be("accepted");
    body.PublicFactId.Should().Be("fact-am-7716");

    var raw = await response.Content.ReadAsStringAsync();
    raw.Should().NotContain("case-rc-");
    fixture.CommittedDecisions.Should().ContainSingle();
    fixture.PendingPublicFacts.Should().ContainSingle();
}
```

EXECUTABLE ASSET 21-N - COMMIT THEN DISCONNECT

```
[Fact]
public async Task API_LOSS_001_disconnect_after_TX_A_does_not_undo_truth()
{
    fixture.ResponseGate.BlockAfterOwnerCommit();
    using var cancellation = new CancellationTokenSource();
    var send = fixture.Client.SendAsync(
        fixture.ValidDetectAlarmFloodRequest(),
        cancellation.Token);

    await fixture.ResponseGate.WaitForOwnerCommitAsync();
    cancellation.Cancel();
    var observe = async () => await send;
    await observe.Should()
        .ThrowAsync<OperationCanceledException>();

    await fixture.Persistence.WaitUntilVisibleAsync();
    fixture.CommittedDecisions.Should().ContainSingle(
        x => x.DecisionId == "decision-am-9184");
    fixture.PendingPublicFacts.Should().ContainSingle(
        x => x.PublicFactId == "fact-am-7716");
    fixture.RootCauseEffects.Should().BeEmpty();
}

[Theory]
[InlineData("application/xml", HttpStatusCode.UnsupportedMediaType)]
[InlineData("oversize-json", HttpStatusCode.RequestEntityTooLarge)]
[InlineData("malformed-json", HttpStatusCode.BadRequest)]
public async Task unsafe_transport_never_enters_Application(
    string requestCase, HttpStatusCode expected)
{
    var response = await fixture.SendCaseAsync(requestCase);
    response.StatusCode.Should().Be(expected);
    fixture.ApplicationCalls.Should().Be(0);
}
```

CANCELLATION IS NOT ROLLBACK EVIDENCE

Once TX-A commits, client disconnect cannot make owner truth disappear. The client must treat the result as unknown; Chapter 23 owns safe replay resolution.

OPENAPI, CLIENT COMPATIBILITY AND ACCEPTANCE

The published surface and runtime surface must remain the same protocol

EXECUTABLE ASSET 21-O - SCHEMA AND OLD/NEW CLIENT GATES

```
public sealed record ApiCompatibilityCase(
    string ClientBuild,
    string ServerBuild,
    string Fixture,
    HttpStatusCode ExpectedStatus);

[Fact]
public async Task API_COMP_001_OpenAPI_declares_closed_response_set()
{
    var operation = (await fixture.GenerateOpenApiAsync())
        .Paths["/api/v1/alarm-flood-detections"].Operations[
        OperationType.Post];
    operation.Responses.Keys.Should().BeEquivalentTo(
        "202", "400", "401", "403", "404", "409",
        "412", "413", "415", "422", "500", "503");
    operation.RequestBody.Content.Keys.Should()
        .BeEquivalentTo("application/json");
}

[Theory]
[MemberData(nameof(SupportedClientServerPairs))]
public async Task old_and_new_generated_clients_observe_same_contract(
    ApiCompatibilityCase test)
{
    var result = await fixture.ExecutePairAsync(test);
    result.Status.Should().Be(test.ExpectedStatus);
    result.UndocumentedResponse.Should().BeFalse();
    result.DeserializationFailure.Should().BeFalse();
}
```

EXECUTABLE ASSET 21-P - ACCEPTANCE EVIDENCE RECORD

```
{
  "evidenceId": "API-REVIEW-001", "chapter": 21,
  "artifactCommit": null, "sdk": "net8.0",
  "edgePackage": null, "hostBuild": null,
  "issuer": null, "provider": null, "openApiDigest": null,
  "tests": {
    "API-BOUND-001": "PENDING", "API-CMD-001": "PENDING",
    "API-QRY-001": "PENDING", "API-VAL-001": "PENDING",
    "API-AUTHN-001": "PENDING", "API-AUTHZ-001": "PENDING",
    "API-MAP-001": "PENDING", "API-STATUS-001": "PENDING",
    "API-PROB-001": "PENDING", "API-LOSS-001": "PENDING",
    "API-COMP-001": "PENDING"
  },
  "hostedCorpusAccepted": false,
  "securityReviewAccepted": false,
  "accepted": false
}
```

Chapter 21 code-audit result

The chapter now exposes one executable path from strict Edge contracts and verified identity to resource authorization, owner-local command/query work, bounded HTTP results, safe problems and hosted compatibility evidence.

CURRENT VERDICT

The synchronous boundary is implementable in shape, but no host, token, provider, OpenAPI document or fault/security corpus has executed. All API evidence remains explicitly PENDING.

Implementation sequence

1. Accept ADR-019 and publish the route/method/media/request/response/status/problem ownership catalogue for AlarmManagement commands and queries.
2. Create Edge.Contracts v1 DTOs and explicit JSON/field/cross-field validators without Domain, EF, broker or RootCause dependencies.
3. Configure accepted authentication scheme/issuer/audience/lifetime and produce a minimal verified ActorContext; execute the negative token matrix.
4. Implement named detect/read policies and resource-aware site authorization with documented 401/403/policy-safe-404 disclosure behavior.
5. Implement DetectAlarmFlood DTO-to-Application and outcome-to-HTTP mappers; prevent HttpContext/claims/API DTOs from entering Domain.
6. Implement the POST endpoint and prove 202/Location/body only after TX-A commit, with no RootCause case/status fields.
7. Implement the owner-only decision query projection/handler plus access, absence, cache and conditional semantics without remote joins or side effects.
8. Configure one RFC 9457 Problem Details catalogue/writer and safe known/unexpected failure mapping across routing, validation and exception middleware.
9. Execute HTTP, security, limit, concurrency, disconnect, cancellation, lost-response, output/log-scan and OpenAPI/client matrices.
10. Publish API evidence without advancing RootCause beyond G0/G1; hand idempotency/version negotiation to Chapter 23 and compatibility execution to Chapter 24.

What this chapter has proved

Chapter 21 has specified commands and queries as versioned HTTP protocols at an explicit service boundary. It has separated transport validation, public DTOs, authentication, authorization, Application mapping, Domain decisions, TX-A and response mapping; shown a complete DetectAlarmFlood flow; bounded 202 to AlarmManagement truth; defined owner-only queries, status codes, RFC 9457 problems, failure/security/loss behavior and acceptance evidence; and preserved RootCause as an asynchronous independent owner.

It has not hosted ASP.NET Core, validated a token, evaluated a policy, bound a request, compiled a mapper, committed through an endpoint, returned a real 202/Problem Details response, generated OpenAPI, served a query, injected a lost response or executed a client/security matrix. The API boundary is specified, not implemented or evidenced. RootCause remains G0 accepted and G1 specified.

ONE PROVIDER, ONE BOUNDED CLAIM

A response is trustworthy when it states only what the responding owner has proved. AlarmManagement may prove its local acceptance and public intent; RootCause proves its own case state later through its own authorized contract.

Chapter checkpoint

Question	Expected answer
May an API request be the aggregate command type?	No. The versioned public DTO maps explicitly to an Application input; Domain remains HTTP-agnostic.
What happens before authentication?	Only bounded transport/parsing safety needed to protect the host; protected semantic detail is not disclosed.
Are authentication and authorization the same?	No. Authentication verifies caller identity; named policies decide whether that identity may perform the capability/read the resource.
May SiteId or actor identity in JSON grant access?	No. Authorization uses verified identity and policy-controlled resource context.

Question	Expected answer
What does the 202 prove?	AlarmManagement accepted and committed its local decision/public intent; wider asynchronous processing is incomplete.
May the 202 contain case-rc-3107?	No. RootCause owns that identity and may not have processed the event.
Where does the client learn RootCause state?	Through an authorized RootCause query or Chapter 22 BFF composition with explicit freshness/source semantics.
May a GET acknowledge an alarm or trigger dispatch?	No. Queries return owned representations without business side effects.
When are 400 and 422 used?	400 for malformed/public representation syntax; 422 for well-formed input that violates documented public semantic rules.
When are 409 and 412 used?	409 for current owner-state conflict; 412 for a documented HTTP precondition that evaluates false.
How are API failures represented?	RFC 9457 application/problem+json with stable type/status and safe bounded extensions; no implementation details.
What if TX-A commits but the response is lost?	The owner decision/fact remains committed; clients cannot infer failure and Chapter 23 resolves replay ambiguity.
May AlarmManagement query RootCause synchronously inside its repository?	No. That hides cross-service coupling; the BFF composes authorized reads at the edge.
What is the current evidence status?	Routes/security/mappers/problems are specified only; no HTTP runtime evidence exists; RootCause remains G0/G1.
What does Chapter 22 add?	A thin Operator Console BFF that composes service-owned read models and routes commands without browser orchestration.

NEXT CHAPTER

Chapter 22, The Operator Console BFF, will implement the UI-facing composition boundary. It will combine authorized AlarmManagement and RootCause read models with explicit source/freshness/partial-failure semantics, route commands to the responsible service and keep the browser from coordinating the distributed workflow.

PART IV - PUBLIC CONTRACTS AND THE SYNCHRONOUS EDGE

22. The Operator Console BFF

The Operator Console needs one screen-shaped view of a story whose facts belong to different services at different times. The BFF may compose that view, but it may not become a third business owner, a shared database, a hidden workflow engine or a browser-accessible substitute for service boundaries.

CHAPTER ARGUMENT

Put client-specific composition at the UI edge. Authenticate and authorize the operator, query each service through its public contract, preserve source identity and time, classify partial failure explicitly, route every command to the responsible owner and return a representation that is convenient for the console without pretending that distributed state is simultaneous or centrally owned.

Chapter 21 ended with decision-am-9184 committed by AlarmManagement while RootCause could still be pending. This chapter gives the operator one route for that evolving story. The first read returns the AlarmManagement slice and states that RootCause has not yet exposed an authorized case. A later read may include case-rc-3107 after RootCause has consumed the public fact, applied its own policy and committed TX-B. The BFF never inserts the case ID into the earlier response and never treats the later composition as one distributed snapshot.

The chosen endpoint is GET /ui-api/v1/alarm-flood-investigations/{decisionId}. It is owned by the Operator Console experience, not by AlarmManagement or RootCause. The BFF obtains the AlarmManagement decision view and a RootCause view addressed by the published alarm-flood/public-fact identity. It then maps both into AlarmFloodInvestigationViewV1. A browser refresh repeats only this BFF query; JavaScript does not fan out to service APIs or correlate raw responses.

This remains a net8.0 reference specification. No OperatorConsole.Bff project has compiled; no delegated credential flow, downstream client, resilience pipeline, cache policy, trace, fault fixture or deployed route has executed. RootCause remains G0 accepted and G1 specified. Chapter 23 owns exact version negotiation, BFF Problem Details evolution and idempotency keys; Chapter 24 owns generated-client and compatibility execution.

A BFF is an experience boundary, not a business boundary

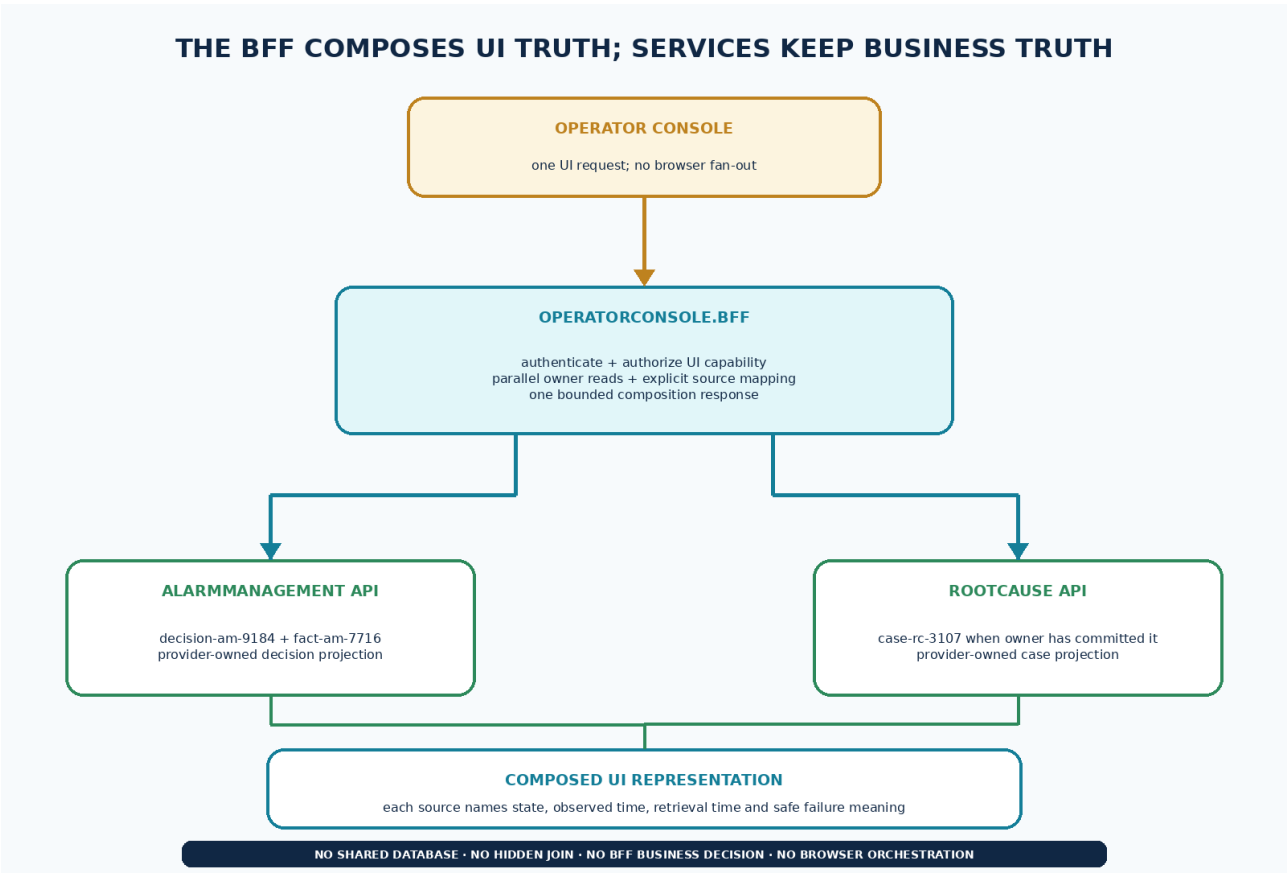


Figure 22.1 - The Operator Console BFF composes authorized service-owned views while AlarmManagement and RootCause retain business truth.

Concern	BFF responsibility	Service responsibility	Forbidden result
UI contract	Expose screen-shaped versioned routes, fields, links and partial-state vocabulary.	Expose provider-owned command/query contracts independent of one UI.	Browser binds directly to aggregate/entity or joins service DTOs.
Authentication	Validate the browser-facing credential/session and create a bounded operator context.	Validate the accepted downstream credential for its own audience.	Forward an arbitrary browser token or trust body-supplied identity.
Authorization	Authorize the console capability and minimize visible UI actions.	Re-authorize every owner read/command against provider-local policy/resource.	Elevated BFF token becomes an authorization bypass or confused deputy.
Queries	Fan out only to documented public reads; map results into source slices.	Return authorized owner projections with local observed/validator semantics.	Cross-database join, repository call or hidden internal endpoint.
Commands	Map a UI intent and route it once to exactly one owner; preserve owner outcome.	Validate, decide and commit owner state; return provider-local result.	BFF writes business state, calls two commands or invents success.
Partial failure	Classify required versus optional source results and state completeness explicitly.	Return its own safe status/problem meaning.	Silent nulls, stale-as-current data or generic 200 masking missing anchor.
Time	Carry composition/retrieval timestamps while preserving source-observed timestamps.	Own business and projection observation times.	One composedAtUtc presented as a distributed business timestamp.
Persistence	Prefer no durable business store; bounded ephemeral technical state only if accepted.	Own durable domain/read-model data and lifecycle.	BFF database becomes shadow system of record or workflow ledger.

OWNERSHIP TEST

If a rule changes a business decision, identifier lifecycle, case state or public fact, it belongs to an owning service. If it changes how one operator interface requests, combines, labels, paginates or displays authorized owner data, it may belong to the BFF.

The BFF contract names every source slice

Contract type	Purpose	Key fields	Boundary rule
AlarmFloodInvestigationViewV1	One operator-screen representation.	compositionId, decisionId, completeness, composedAtUtc, sources, alarmManagement, rootCause, availableActions	No aggregate, EF row, broker envelope or raw downstream DTO.
SourceSnapshotV1<T>	Generic envelope for one requested provider slice.	source, state, retrievedAtUtc, sourceObservedAtUtc, ageMs, value, safeProblem	state controls whether value/problem may appear; timestamps retain provenance.
AlarmManagementDecisionSliceV1	Provider-local accepted decision view.	decisionId, alarmFloodId, publicFactId, acceptedAtUtc, decisionStatus	Does not contain RootCause case/status or dispatch-success claim.
RootCauseCaseSliceV1	Consumer-owned case projection when authorized and present.	caseId, alarmFloodId, caseStatus, openedAtUtc, lastObservedAtUtc	Never synthesized from AlarmManagement; caseId belongs to RootCause.
AvailableActionV1	UI affordance derived from BFF policy and current representation.	rel, method, href, owner, requiresIdempotencyKey	A hint, not an authorization grant; owner authorizes again.
BffProblemV1	BFF-level RFC 9457 problem for unusable composite requests.	type, title, status, detail, instance, errorCode	No downstream body dump, token, stack, endpoint topology or private identifier.

IMPLEMENTATION SHAPE

```
public enum SourceStateV1
{
    Available,
    NotYetObserved,
    NotDisclosed,
    TemporarilyUnavailable,
    Stale
}

public sealed record SourceSnapshotV1<T>(
    string Source,
    SourceStateV1 State,
    DateTimeOffset RetrievedAtUtc,
    DateTimeOffset? SourceObservedAtUtc,
    long? AgeMs,
    T? Value,
    SafeSourceProblemV1? SafeProblem);

public sealed record AlarmFloodInvestigationViewV1(
    string CompositionId,
    string DecisionId,
    string Completeness,
    DateTimeOffset ComposedAtUtc,
    SourceSnapshotV1<AlarmManagementDecisionSliceV1> AlarmManagement,
    SourceSnapshotV1<RootCauseCaseSliceV1> RootCause,
    IReadOnlyList<AvailableActionV1> AvailableActions);
```

Null is never the source-state vocabulary. When state is available, value is required and safeProblem is absent. When state is notYetObserved or notDisclosed, value is absent and no detailed provider failure is exposed. temporarilyUnavailable may carry only a stable safe code and optional retry hint. stale requires the last known value plus its source observation and age, and is forbidden unless a reviewed stale-data policy exists.

Source state	Meaning	Allowed value/problem	Operator interpretation
available	Authorized provider projection returned within the request budget.	value required; sourceObservedAtUtc required when provider supplies it; no problem.	This slice may be displayed as current for the stated source time.
notYetObserved	Authorized provider query proves no matching projection is currently exposed.	No value; no hidden resource detail; stable state only.	Processing may still be pending; absence is not failure or finality.
notDisclosed	The BFF cannot safely disclose the source slice under the accepted policy.	No value and no existence-sensitive reason.	UI hides protected detail and does not distinguish absent from denied.
temporarilyUnavailable	Timeout, circuit open or safe 5xx prevented this optional read.	No value; bounded safe problem/retry hint only.	Screen is partial; refresh may recover.
stale	Reviewed fallback returned a prior authorized value after current retrieval failed.	Value plus original observed/retrieved	UI must visibly label age; never render

Complete walkthrough: first partial, then complete

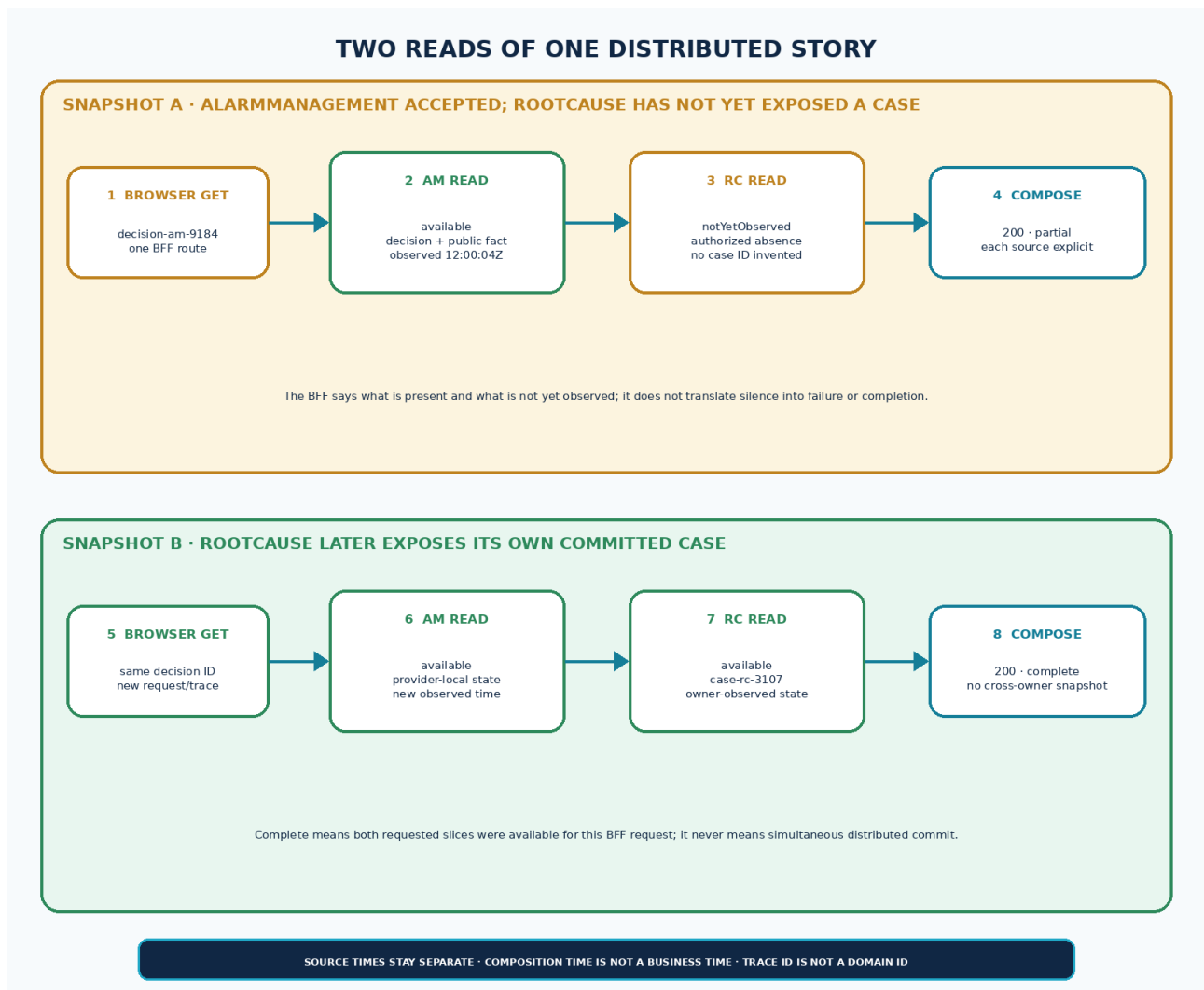


Figure 22.2 - Two BFF requests show the same distributed story before and after RootCause exposes its own committed case.

The two reads are deliberately separate HTTP requests with separate trace contexts. Snapshot A can happen immediately after the Chapter 21 POST returns 202. AlarmManagement already exposes decision-am-9184 and fact-am-7716; the authorized RootCause query by alarmFloodId/publicFactId returns provider-safe absence. The BFF returns 200 because the required AlarmManagement anchor is available, marks completeness as partial and represents RootCause as notYetObserved.

Snapshot B occurs later. RootCause has accepted the public fact through its own inbox, evaluated its policy and committed case-rc-3107. The BFF repeats both reads, receives two available slices and returns completeness complete. Complete is a statement about requested source availability during this one BFF request. It does not assert that acceptedAtUtc, openedAtUtc or sourceObservedAtUtc are equal, and it does not create a distributed transaction or global version.

SNAPSHOT A · PARTIAL COMPOSITION

```
GET /ui-api/v1/alarm-flood-investigations/decision-am-9184 HTTP/1.1
Accept: application/vnd.nexus1.operator-console.investigation-v1+json
Authorization: Bearer <operator-console credential>
traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01

HTTP/1.1 200 OK
Content-Type: application/vnd.nexus1.operator-console.investigation-v1+json
Cache-Control: private, no-store

{
  "compositionId": "comp-bff-4402",
  "decisionId": "decision-am-9184",
  "completeness": "partial",
  "composedAtUtc": "2026-07-19T12:00:05.220Z",
  "alarmManagement": {
    "source": "AlarmManagement",
    "state": "available",
    "retrievedAtUtc": "2026-07-19T12:00:05.118Z",
    "sourceObservedAtUtc": "2026-07-19T12:00:04.991Z",
    "ageMs": 229,
    "value": {
      "decisionId": "decision-am-9184",
      "alarmFloodId": "flood-am-6031",
      "publicFactId": "fact-am-7716",
      "decisionStatus": "accepted"
    }
  },
  "rootCause": {
    "source": "RootCause",
    "state": "notYetObserved",
    "retrievedAtUtc": "2026-07-19T12:00:05.204Z",
    "sourceObservedAtUtc": null,
    "ageMs": null,
    "value": null,
    "safeProblem": null
  },
  "availableActions": [
    { "rel": "refresh", "method": "GET",
      "href": "/ui-api/v1/alarm-flood-investigations/decision-am-9184",
      "owner": "OperatorConsole.Bff", "requiresIdempotencyKey": false }
  ]
}
```

The response carries no `caseld`, no assumption about delivery, and no raw 404 from `RootCause`. The BFF may use a provider-specific absence result only after the downstream request was authorized and the service contract documents that absence does not disclose protected existence. Otherwise the safe state is `notDisclosed` or the whole BFF view is denied under the accepted policy.

SNAPSHOT B · BOTH REQUESTED SLICES AVAILABLE

```

HTTP/1.1 200 OK
Content-Type: application/vnd.nexus1.operator-console.investigation-v1+json
Cache-Control: private, no-store

{
  "compositionId": "comp-bff-4479",
  "decisionId": "decision-am-9184",
  "completeness": "complete",
  "composedAtUtc": "2026-07-19T12:00:09.410Z",
  "alarmManagement": {
    "source": "AlarmManagement", "state": "available",
    "retrievedAtUtc": "2026-07-19T12:00:09.301Z",
    "sourceObservedAtUtc": "2026-07-19T12:00:08.990Z", "ageMs": 420,
    "value": { "decisionId": "decision-am-9184",
      "alarmFloodId": "flood-am-6031", "publicFactId": "fact-am-7716",
      "decisionStatus": "accepted" }
  },
  "rootCause": {
    "source": "RootCause", "state": "available",
    "retrievedAtUtc": "2026-07-19T12:00:09.392Z",
    "sourceObservedAtUtc": "2026-07-19T12:00:09.180Z", "ageMs": 230,
    "value": { "caseId": "case-rc-3107", "alarmFloodId": "flood-am-6031",
      "caseStatus": "open", "openedAtUtc": "2026-07-19T12:00:08.602Z" }
  },
  "availableActions": [
    { "rel": "refresh", "method": "GET",
      "href": "/ui-api/v1/alarm-flood-investigations/decision-am-9184",
      "owner": "OperatorConsole.Bff", "requiresIdempotencyKey": false }
  ]
}

```

Composition executes bounded parallel reads

The BFF starts downstream reads concurrently only after the browser request passes protocol safety, authentication and BFF-level authorization. Each typed client receives a service-specific address, accepted media type, credential plan, timeout budget and response union. The clients return closed outcomes rather than throwing provider detail into the endpoint. The composer applies source rules after both tasks settle or the overall deadline expires.

IMPLEMENTATION SHAPE

```

public sealed class GetAlarmFloodInvestigationHandler(
    AlarmManagementQueryClient alarmManagement,
    RootCauseQueryClient rootCause,
    IOperatorConsoleAuthorization authorization,
    TimeProvider clock)
{
    public async Task<BffOutcome> Handle(
        GetAlarmFloodInvestigationQuery query,
        VerifiedOperatorContext actor,
        CancellationToken requestAborted)
    {
        AuthorizationDecision allowed = await authorization.CanReadAsync(
            actor, query.DecisionId, requestAborted);
        if (!allowed.Succeeded)
            return BffOutcome.NotDisclosed();

        using var deadline = CancellationTokenSource.CreateLinkedTokenSource(requestAborted);
        deadline.CancelAfter(TimeSpan.FromMilliseconds(1200)); // candidate; prove with SLO evidence

        Task<AlarmManagementReadOutcome> alarmTask =
            alarmManagement.GetDecisionAsync(query.DecisionId, actor, deadline.Token);
        Task<RootCauseReadOutcome> rootTask =
            rootCause.FindCaseByPublicIdentityAsync(query.PublicIdentity, actor, deadline.Token);

        await Task.WhenAll(alarmTask, rootTask).ConfigureAwait(false);
        return AlarmFloodInvestigationComposer.Compose(
            await alarmTask, await rootTask, clock.GetUtcNow());
    }
}

```

The listing is a reference candidate, not compiled evidence. A production implementation must ensure the optional RootCause call cannot extend the required AlarmManagement deadline, and must observe every task even when the client discor

architectural constant; it is a proposed SLO budget that requires measured latency, queueing, retry and cancellation evidence.

Budget element	Candidate allocation	Required control	Failure meaning
BFF total	1200 ms wall-clock	One linked deadline covering authentication-complete composition and mapping.	If required anchor is unresolved, return safe 504/503; never wait indefinitely.
AlarmManagement required read	900 ms maximum	Typed client; provider-specific circuit/bulkhead; no hidden nested retry budget.	Timeout/unavailable makes the composite unusable.
RootCause optional read	700 ms maximum	Parallel call with shorter budget; safe outcome union.	Timeout may become temporarilyUnavailable while anchor remains displayable.
Retry	At most one safe GET retry candidate	Only within remaining budget and only for accepted transient categories.	Attempt count/elapsed time recorded; no retry storm.
Hedging	Disabled in reference	Enable only with duplicate-load and consistency evidence.	Never multiply load invisibly or hedge unsafe commands.
Cancellation	Propagate browser abort and total deadline	Clients and composer settle/observe tasks; no abandoned work leak.	Query has no business side effect; telemetry records bounded cancellation.

Authentication, delegation and authorization remain separate

The browser-facing credential proves who is using the Operator Console. It does not automatically satisfy either service audience. The BFF must have an accepted downstream credential strategy: token exchange/on-behalf-of, a constrained service identity with verified actor context, or another documented mechanism. Whichever mechanism is selected, the service validates its audience and re-evaluates provider-local authorization. The BFF never forwards unvalidated claims and never upgrades a weak browser identity into a privileged service identity.

Decision	OperatorConsole.Bff	Downstream service	Evidence required
Who is calling the UI?	Validate accepted browser scheme/issuer/audience/lifetime; create VerifiedOperatorContext.	Not answered by the BFF result alone.	Negative token/session matrix and identity provenance tests.
May the operator open this screen?	Named policy operator-console.read-alarm-flood plus public resource context.	Still independently authorizes its requested projection.	BFF policy tests plus downstream 401/403/policy-safe-404 fixtures.
Which downstream credential?	Obtain service-specific credential through an approved delegated/constrained flow.	Accept only configured audience/scope and verified actor/tenant binding.	Issuer/audience/scope/actor propagation and confused-deputy tests.
May the UI show a command?	Derive a non-authoritative availableAction from UI policy/current view.	Authorizes again when command arrives.	Action visibility and owner-denial tests; no optimistic privilege.
What if RootCause is denied?	Return notDisclosed or deny the whole view according to documented policy.	Does not reveal resource existence or private reason.	Existence-oracle equivalence across body/status/timing/logs.
May Domain see claims?	No; map verified identity into bounded Application/client inputs.	No; owner Domain receives business actor values only where required.	Dependency scan and mapper provenance tests.

CONFUSED-DEPUTY RULE

A BFF that can call every service as an all-powerful technical principal is not authorization; it is a lateral-movement accelerator. Downstream capability, tenant/site scope and operator provenance must stay bounded and independently enforceable.

Partial failure is explicit contract data

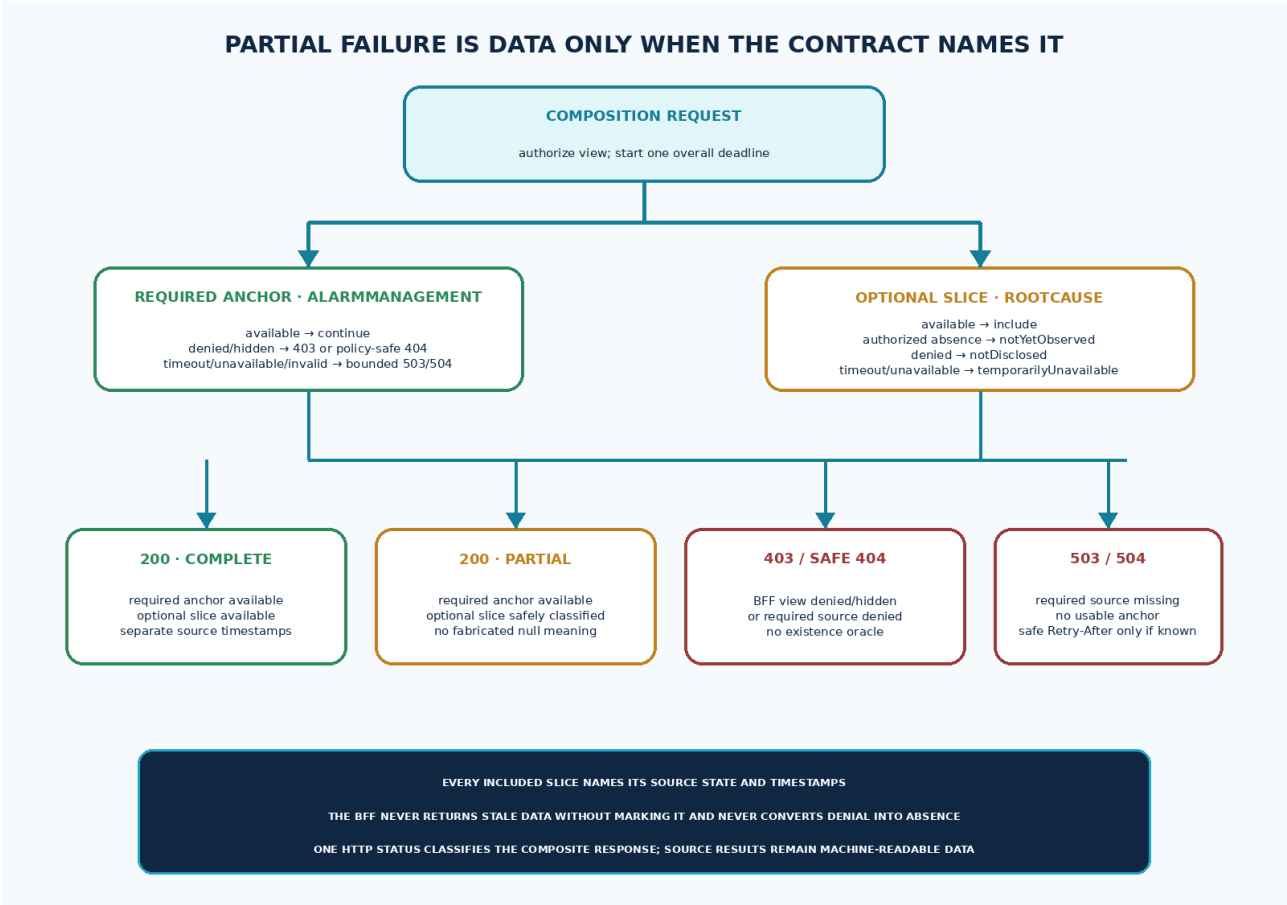


Figure 22.3 - The required AlarmManagement anchor controls whether the composite is usable while RootCause outcomes remain explicit source data.

AlarmManagement result	RootCause result	BFF HTTP outcome	Composition rule
available	available	200 complete	Include both mapped values with separate source timestamps.
available	authorized absence	200 partial	RootCause state notYetObserved; no case ID and no failure claim.
available	denied/hidden	200 partial or whole-view 403/safe 404	Use only the accepted disclosure policy; never distinguish hidden existence.
available	timeout/503/circuit open	200 partial	RootCause temporarilyUnavailable with bounded safe code; no downstream body dump.
available	invalid response/schema	200 partial plus internal alert, or fail closed by policy	Do not map malformed provider data; evidence must define whether screen remains safe.
denied/hidden	any	403 or policy-safe 404	Required anchor cannot be disclosed; do not call or expose RootCause unnecessarily.
not found	any	404	Decision anchor is absent under an authorized provider query; no orphan composite.
timeout/503/circuit open	any	503 or 504	No usable anchor; safe Problem Details and Retry-After only when known.
invalid response/schema	any	502/503 project decision	Provider contract breach is not a user validation error; alert with opaque occurrence ID.

HTTP 206 Partial Content is not used for this semantic. It describes byte-range transfer semantics, not an application composition with optional sources. The BFF returns a normal 200 representation when the required anchor is usable and names completeness/source states inside its contract. When no meaningful screen can be produced, the HTTP status expresses that BFF-level failure.

OPTIONAL SOURCE UNAVAILABLE · COMPOSITE STILL USABLE

```
HTTP/1.1 200 OK
Content-Type: application/vnd.nexus1.operator-console.investigation-v1+json
Cache-Control: private, no-store

{"rootCause": {
  "source": "RootCause",
  "state": "temporarilyUnavailable",
  "retrievedAtUtc": "2026-07-19T12:00:12.700Z",
  "sourceObservedAtUtc": null,
  "ageMs": null,
  "value": null,
  "safeProblem": {
    "code": "rootcause_temporarily_unavailable",
    "retryAfterSeconds": 5
  }
}
```

retryAfterSeconds appears only when the BFF has a bounded, trustworthy retry hint. It is not copied blindly from an arbitrary downstream header, and it never promises recovery. A source-level hint does not require an overall Retry-After header on a successful 200 composite; the UI refresh policy consumes the source field according to the BFF contract.

Commands pass through one owner without orchestration

The browser may post a UI-shaped request to the BFF, but the BFF maps it to exactly one service command. For DetectAlarmFlood, it validates only public UI shape and BFF capability, obtains the AlarmManagement credential, sends one owner request and maps the owner result. It does not call RootCause, wait for a case or start a compensating workflow. The owner remains the only component that can prove the local commit behind 202.

IMPLEMENTATION SHAPE

```
group.MapPost("/alarm-flood-detections", async (
    DetectAlarmFloodUiRequestV1 request,
    VerifiedOperatorContext actor,
    AlarmManagementCommandClient client,
    CancellationToken ct) =>
{
    DetectAlarmFloodRequestV1 downstream = DetectAlarmFloodUiMapper.Map(request);
    AlarmManagementCommandOutcome outcome =
        await client.DetectAlarmFloodOnceAsync(downstream, actor, ct);

    return OperatorConsoleCommandHttpMapper.Map(outcome);
})
.RequireAuthorization("operator-console.detect-alarm-flood")
.Accepts<DetectAlarmFloodUiRequestV1>("application/json")
.Produces<DetectAlarmFloodUiAcceptedV1>(StatusCodes.Status202Accepted)
.ProducesProblem(StatusCodes.Status400BadRequest)
.ProducesProblem(StatusCodes.Status401Unauthorized)
.ProducesProblem(StatusCodes.Status403Forbidden)
.ProducesProblem(StatusCodes.Status409Conflict)
.ProducesProblem(StatusCodes.Status422UnprocessableEntity)
.ProducesProblem(StatusCodes.Status503ServiceUnavailable);
```

Command concern	BFF behavior	Owner truth	Blocking prohibition
Mapping	Explicit UI DTO to provider DTO/application contract mapping.	AlarmManagement validates and maps into its own Application command.	Pass through arbitrary JSON, internal DTO or actor fields.
Authorization	BFF capability check controls UI entry point.	AlarmManagement re-authorizes provider capability/resource.	Treat availableAction or BFF success as owner authorization.
Send count	One deliberate outbound attempt in the reference path.	Owner idempotency/status contract resolves ambiguous commits in Chapter 23.	Automatic retry/hedging of unsafe POST without an accepted idempotency key.
202	Map only a real owner 202 received after owner commit; expose UI-safe Location.	Proves AlarmManagement local acceptance/public intent only.	BFF-generated 202 before response, cached success or RootCause claim.
Problem Details	Translate only documented safe categories into BFF problem catalogue.	Owens provider status/type and private diagnostics.	Copy downstream detail/instance/headers blindly or return stack/SQL/token.
Lost response	Do not infer failure and do not retry blindly.	Committed owner state may exist despite BFF timeout/disconnect.	Second effect caused by BFF retry; Chapter 23 owns key/status semantics.

UNSAFE RETRY RULE

Queries may use narrowly proven retry within their deadline. POST, PUT, PATCH and DELETE are not automatically retried by the BFF. An idempotency key and owner replay contract must exist before Chapter 23 can authorize a different policy.

Caching preserves authorization and source age

The reference composite is Cache-Control: private, no-store because it contains operator-specific authorized data and the project has not proved a safe cache key or invalidation model. RFC 9111 distinguishes private from shared caches and defines freshness, validation and no-store semantics. A future cache may be accepted only after the key includes every disclosure dimension and the representation exposes source age without presenting a cached composition as a fresh distributed read.

Caching question	Reference answer	Evidence before change
May a shared proxy cache the composite?	No. Authorization and tenant/site/operator disclosure make accidental reuse unsafe.	Identity/authorization cache-key proof, Vary analysis, poisoning/timing tests and data-classification approval.
May the browser retain it?	private, no-store is the conservative baseline; UI may retain in memory only under explicit product/security rules.	Threat model, logout/session-boundary behavior and browser-history tests.
Can one ETag represent two owners?	Not yet. A BFF validator would be a composition validator, not a distributed business version.	Stable combined-validator algorithm, per-source validator provenance and Chapter 23 contract.
Can stale-if-error be used?	Not in the baseline. If accepted, stale must be a source state with observed time and age.	Maximum age, source eligibility, disclosure, outage and operator-label fixtures.
What follows a command?	The BFF cannot assume all intermediaries invalidated every related composite URI.	Explicit UI refresh/status flow, targeted invalidation design and lost-response tests.

Observability follows the fan-out without becoming identity

One BFF request creates one server span and one child client span per downstream attempt. W3C traceparent/tracestate propagate technical context across HTTP boundaries. compositionId identifies the returned BFF representation for support, while request-am-20482, decision-am-9184, fact-am-7716, flood-am-6031 and case-rc-3107 retain their established business meanings. Trace/span IDs are never serialized as business identifiers or used as database keys.

Telemetry	Required dimensions	Never record	Acceptance signal
BFF request span	route template, outcome class, completeness, total duration, caller class.	Raw token, request body, user PII, private case detail.	One bounded span per request; trace continues from accepted context.
Downstream client span	logical service, route template, safe status class, attempt, duration, timeout/circuit outcome.	Full URL with private IDs, response body, credentials, provider stack.	Exactly one span per actual attempt; parent/child relation intact.
Composition metrics	complete/partial/unusable count, source-state count, source latency, deadline utilization.	Unbounded decision/case IDs as metric labels.	Low-cardinality dashboards reconcile with HTTP fixtures.
Logs	compositionId, safe occurrence ID, bounded owner/public IDs where approved, policy/outcome code.	Existence-sensitive denial reason, SQL, token, raw headers or DTO dump.	Output/log scans prove forbidden fields absent.
Trace propagation	valid traceparent and accepted tracestate mutation rules.	Trace ID as idempotency key, public fact ID or audit identity.	Cross-service trace fixture and malformed-context negative tests.

Test the composition, not only the happy response

Fixture	Expected result	Blocking assertion
Golden partial snapshot	200 partial; AlarmManagement available; RootCause notYetObserved.	No case ID, no downstream 404 body and separate source times.
Golden complete snapshot	200 complete with case-rc-3107 after independent RootCause commit.	No equal-time/global-version claim and mappings contain approved fields only.

Fixture	Expected result	Blocking assertion
RootCause timeout/circuit open	200 partial; temporarilyUnavailable with bounded safe code.	Total latency within BFF budget; no raw exception or overall false 503.
AlarmManagement timeout/503	503/504 safe BFF Problem Details.	No orphan RootCause-only screen and no unobserved task.
RootCause denied/hidden	notDisclosed or whole-view denial per policy.	Absent/denied equivalence across status/body/timing/logs.
Malformed downstream JSON/schema	No unsafe mapping; partial or fail-closed outcome per accepted contract.	Contract alert; no partially deserialized private data.
Browser disconnect	Linked cancellation reaches reads; no business state change.	All tasks settle/are observed; cancellation classified, not 5xx noise.
Safe GET transient retry	Only accepted category retried within remaining budget.	Attempt count bounded; no retry storm or deadline overrun.
Unsafe POST transient/lost response	No automatic retry; ambiguous outcome preserved for Chapter 23.	Exactly one outbound attempt and no duplicate owner effect.
Authorization chain	BFF and each service independently accept/deny configured identity/resource.	No elevated bypass, actor/tenant substitution or existence oracle.
Cache/browser history	private/no-store headers on every personalized composite/problem path.	No cross-actor reuse; command followed by explicit refresh behavior.
Trace/log/metric	One server span, actual client attempts, bounded labels and safe logs.	No IDs as metric labels, no token/body/stack/private detail.
Old/new client contract	BFF request/response/source-state/problem union remains compatible.	Chapter 24 fixtures cover unknown fields/states and media versions.

Evidence catalogue

Evidence ID	Acceptance statement	Minimum artifact
BFF-BOUND-001	OperatorConsole.Bff depends only on public service clients/contracts and contains no Domain/repository/shared-DB dependency.	Project/reference scan and architecture dependency test.
BFF-CONTRACT-001	AlarmFloodInvestigationViewV1 and source-state union are versioned, closed and screen-specific.	Schema/OpenAPI snapshots and generated-client compile fixture.
BFF-COMP-001	Parallel owner reads preserve provenance, timestamps and required/optional semantics.	Golden partial/complete composition fixtures.
BFF-SOURCE-001	Every slice obeys state/value/problem/timestamp invariants.	Property/state-machine tests and invalid-combination rejection.
BFF-AUTHN-001	Only accepted browser credentials create verified operator context.	Issuer/audience/lifetime/signature negative matrix.
BFF-AUTHZ-001	BFF and services independently enforce capability/resource policies without existence leaks.	Policy chain and 401/403/safe-404 oracle tests.
BFF-DELEG-001	Downstream credentials preserve bounded capability and actor/tenant provenance.	Token/credential exchange fixtures and confused-deputy negative tests.
BFF-QUERY-001	Typed clients use bounded parallel GETs, closed outcomes and one overall deadline.	Latency/retry/circuit/cancellation fault matrix.
BFF-CMD-001	Each UI command maps to exactly one owner request and preserves owner outcome.	Outbound-count, 202/Location/problem and lost-response fixtures.
BFF-FAIL-001	Partial and unusable compositions follow the closed HTTP/source decision table.	Provider outcome matrix and safe output scans.
BFF-CACHE-001	Personalized responses cannot leak through caches; source age is never hidden.	Header, cache-key, history, stale-policy and cross-actor tests.
BFF-OBS-001	Trace fan-out, metrics and logs are bounded, correlated and free of forbidden data.	Trace graph, low-cardinality metric and log-redaction fixtures.
BFF-COMPAT-001	Old/new clients tolerate accepted additive fields and source-state/problem evolution.	Chapter 24 producer/consumer matrix and schema corpus.
BFF-REVIEW-001	UI, service, security, operations and architecture owners accept obligations and residual risks.	Signed evidence index with build/config/fixture owners and review triggers.

ADR-020 - Operator-specific composition without business ownership

ADR field	ADR-020 record
Status	Accepted for the NEXUS-1 reference design; BFF host, delegated identity, typed clients, partial-state contract and runtime evidence remain pending.
Context	The Operator Console needs one representation spanning AlarmManagement and RootCause, but browser fan-out would duplicate security, correlation, failure and compatibility logic while service-side joins would violate ownership.
Decision	Create a thin OperatorConsole.Bff. It authenticates/authorizes the UI request, uses service-specific credentials and public typed clients, executes bounded parallel reads, treats AlarmManagement as the required anchor, maps RootCause into an explicit optional source slice, preserves source timestamps, defaults personalized responses to private/no-store, propagates trace context and routes every command once to exactly one owner. It owns no business decisions, case/decision IDs, durable workflow or shared database.
Consequences	The UI gets one stable screen contract and explicit partial behavior. An extra network hop, client-specific deployment, duplicated mapping and new SLO/security/compatibility evidence are accepted costs. Provider services remain independently reusable and authoritative.
Rejected	Browser fan-out/correlation; shared database join; BFF aggregate/repository; generic enterprise API for every client; raw service DTO passthrough; all-powerful downstream identity; sequential fan-out; silent nulls; 206 misuse; stale-as-current response; BFF business workflow; unsafe command retry; trace ID as business identity.
Review trigger	New frontend, source/requiredness, source-state meaning, downstream identity, owner contract, deadline/retry/circuit policy, cache/stale policy, command route, persistent state, streaming/subscription, data classification or SLO change.

Risk register

Risk	Early signal	Blocking control
BFF becomes business owner	Domain rules, workflow state, outbox or case transitions appear in BFF.	BFF-BOUND scan, ADR review and move decisions to owning service.
Hidden shared model	Raw service DTO/entity fields flow directly into UI schema.	Explicit mappers and BFF-CONTRACT field allowlist/snapshot tests.
Confused deputy	BFF technical identity can read/command across actor/tenant/site boundaries.	BFF-DELEG/AUTHZ credential provenance and negative lateral-movement matrix.
Partial means silent failure	Null fields or 200 response omit source state/problem/timestamps.	BFF-SOURCE/FAIL closed state union and golden response fixtures.
Distributed snapshot overclaim	composedAtUtc/global ETag presented as one business version.	Separate sourceObservedAtUtc/age; Chapter 23 validator review.
Optional source controls latency	RootCause timeout exhausts total budget or blocks anchor response.	Parallel shorter budget, bulkhead/circuit and deadline-utilization evidence.
Retry amplification	Nested retries/hedging multiply calls during provider incident.	One owned resilience pipeline, bounded attempts and fault-load tests.
Unsafe command duplicate	BFF retries POST after timeout/lost response.	Disable unsafe retry until Chapter 23 idempotency contract/evidence.
Authorization leak	Denied and absent RootCause states differ in body/timing/logs.	Existence-oracle equivalence and notDisclosed policy.
Stale data appears current	Cached slice lacks original observed time/age or UI label.	No-store baseline; accepted stale state/maximum-age/display fixtures.
High-cardinality telemetry	decision/case/actor IDs become metric labels or raw bodies enter logs.	BFF-OBS label allowlist, redaction and output scans.
BFF becomes generic gateway	Mobile, partner and automation clients add conflicting fields/policies.	One experience boundary; create/justify separate interface-specific contract.

Official-source validation

The BFF, resilience, caching and tracing rules were checked against current primary sources on 19 July 2026. Microsoft Azure Architecture Center describes a BFF as an interface-specific layer between a frontend and backend services and identifies the extra hop, lifecycle/operational cost, client-specific scope and authorization/monitoring/routing considerations. Microsoft .NET resilience guidance documents HttpClient resilience handlers and explicitly warns that retries for unsafe methods such as

and no-store semantics. W3C Trace Context defines the traceparent and tracestate propagation formats. NEXUS-1 source-state names, required/optional policy, budgets, credentials and UI contracts remain project decisions.

Primary source	Stable point used here	NEXUS-1 decision
Microsoft Azure Architecture Center - Backends for Frontends pattern	An interface-specific service sits between a client and backend services; client tailoring adds an extra hop and operational/security considerations.	OperatorConsole.Bff is specific to the operator experience and remains thin, independently deployable and non-authoritative for business truth.
Microsoft Learn - Build resilient HTTP apps	HttpClient resilience can combine timeout/retry/circuit/rate-limit strategies; unsafe-method retries can cause duplicate state changes.	Queries use one bounded policy candidate; unsafe command retries stay disabled until Chapter 23 evidence.
RFC 9111 - HTTP Caching	Freshness, validation, cache keys, private/shared behavior and no-store have defined HTTP meaning.	Personalized composites default to private/no-store; future caching must preserve authorization dimensions and source age.
W3C Trace Context	traceparent and tracestate propagate technical request context across services.	One BFF server span fans out to actual client attempts; trace identifiers never become business identifiers.

Sources: <https://learn.microsoft.com/en-us/azure/architecture/patterns/backends-for-frontends> ;
<https://learn.microsoft.com/en-us/dotnet/core/resilience/http-resilience> ;
<https://www.rfc-editor.org/rfc/rfc9111.html> ; <https://www.w3.org/TR/trace-context/>

Technical material still required

The BFF boundary, versioned view/source contracts, two-snapshot walkthrough, required/optional source rules, partial-failure matrix, command routing, authorization/delegation constraints, caching baseline, trace model, tests, evidence IDs and ADR are specified. No OperatorConsole.Bff host, downstream credential exchange, typed client, resilience handler, policy, mapper, endpoint, cache, trace exporter, fault injection, OpenAPI document or generated client has executed.

TECHNICAL MATERIAL STATUS - [OPERATOR CONSOLE BFF IMPLEMENTATION, DELEGATED AUTHORIZATION, PARTIAL-FAILURE CONTRACT AND RUNTIME EVIDENCE REQUIRED]

Implement the net8.0 OperatorConsole.Bff host, versioned UI DTOs and RFC 9457 catalogue, strict public validators, browser authentication, operator-console policies, accepted downstream credential/delegation flow, AlarmManagement and RootCause typed clients, explicit provider-to-source mappers, closed source-state/composition outcome unions, bounded parallel deadlines, provider-specific timeout/retry/circuit/bulkhead policies, private/no-store response behavior, trace propagation and low-cardinality telemetry; compile and execute golden partial/complete, authorized absence, not-disclosed, optional timeout/circuit, required-source failure, invalid-schema, cancellation, lost-response, unsafe-command, cache/history, confused-deputy, existence-oracle, output/log and old/new client matrices; publish BFF-BOUND-001 through BFF-REVIEW-001 linked to API/TXN/CTR/ENV evidence and Chapters 23-24. RootCause remains G0 accepted and G1 specified; runtime evidence has not advanced.

Deliverable	Minimum technical content	Blocking evidence
BFF host/boundary	net8.0 project, dependency rules, routes, health/config and no business persistence.	BFF-BOUND-001
UI contract	AlarmFloodInvestigationViewV1, SourceSnapshotV1 and BFF problem/media schemas.	BFF-CONTRACT/SOURCE-001
Security/delegation	Browser scheme, BFF policies, service audiences and bounded actor/tenant credential flow.	BFF-AUTHN/AUTHZ/DELEG-001
Query composition	Typed clients, parallel deadlines, closed outcomes, mappings and partial/unusable matrix.	BFF-COMP/QUERY/FAIL-001
Command routing	One owner request, exact 202/Location/problem mapping and no unsafe retry.	BFF-CMD-001 / Chapter 23
Cache/observability	private/no-store baseline, source age, trace fan-out, metrics/log allowlists.	BFF-CACHE/OBS-001
Compatibility/review	OpenAPI/generated clients, old/new corpus, SLO/fault/security/operations acceptance.	BFF-COMPAT/REVIEW-001 / Chapter 24

CHAPTER 22 - CODE AUDIT

The BFF owns a screen contract, never another business model

The executable boundary contains UI contracts, typed public clients and composition policy only. Domain assemblies, EF repositories, broker adapters and shared database access are rejected before the host can ship.

EVIDENCE STATUS

These listings are compile-oriented reference material. No BFF host, delegated credential flow, downstream service, resilience pipeline, OpenAPI document or fault corpus has executed here. BFF evidence remains PENDING.

EXECUTABLE ASSET 22-A - STRICT SCREEN CONTRACT

```
public enum SourceStateV1
{
    Available,
    NotYetObserved,
    NotDisclosed,
    TemporarilyUnavailable
}

public sealed record SafeSourceProblemV1(
    string Code,
    int? RetryAfterSeconds);

public sealed record SourceSnapshotV1<T>(
    string Source,
    SourceStateV1 State,
    DateTimeOffset RetrievedAtUtc,
    DateTimeOffset? SourceObservedAtUtc,
    long? AgeMs,
    T? Value,
    SafeSourceProblemV1? SafeProblem);

[JsonUnmappedMemberHandling(JsonUnmappedMemberHandling.Disallow)]
public sealed record AlarmFloodInvestigationViewV1(
    string CompositionId,
    string DecisionId,
    string Completeness,
    DateTimeOffset ComposedAtUtc,
    SourceSnapshotV1<AlarmManagementDecisionSliceV1>
        AlarmManagement,
    SourceSnapshotV1<RootCauseCaseSliceV1> RootCause,
    IReadOnlyList<AvailableActionV1> AvailableActions);
```

EXECUTABLE ASSET 22-B - NO BUSINESS-OWNER DEPENDENCY

```
[Fact]
public void BFF_BOUND_001_project_contains_no_owner_runtime_reference()
{
    var assembly = typeof(AlarmFloodInvestigationViewV1).Assembly;
    foreach (var forbidden in new[]
    {
        "Microsoft.EntityFrameworkCore",
        "Nexus1.AlarmManagement.Domain",
        "Nexus1.AlarmManagement.Infrastructure",
        "Nexus1.RootCause.Domain",
        "Nexus1.RootCause.Infrastructure",
        "RabbitMQ.Client"
    })
    {
        Types.InAssembly(assembly).Should()
            .NotHaveDependencyOn(forbidden).GetResult()
            .IsSuccessful.Should().BeTrue();
    }
}

[Fact]
public void BFF_has_no_business_store_or_background_workflow()
{
    fixture.RegisteredDbContexts.Should().BeEmpty();
    fixture.RegisteredOutboxes.Should().BeEmpty();
    fixture.RegisteredBusinessWorkers.Should().BeEmpty();
}
```

DELEGATED IDENTITY AND AUDIENCE SEPARATION

Each downstream request receives a bounded service credential

EXECUTABLE ASSET 22-C - VERIFIED OPERATOR AND CREDENTIAL PORT

```
public sealed record VerifiedOperatorContext(
    string Subject,
    string Tenant,
    IReadOnlySet<string> SiteScope);

public enum DownstreamAudience
{
    AlarmManagement,
    RootCause
}

public sealed record DownstreamAccessToken(
    string Value,
    DownstreamAudience Audience,
    DateTimeOffset ExpiresAtUtc);

public interface IDownstreamCredentialProvider
{
    ValueTask<DownstreamAccessToken> AcquireAsync(
        DownstreamAudience audience,
        VerifiedOperatorContext actor,
        CancellationToken cancellationToken);
}

public static class BffRequestOptions
{
    public static readonly HttpRequestOptionsKey<VerifiedOperatorContext>
        VerifiedOperator = new("nexus1.verified-operator");
}
```

EXECUTABLE ASSET 22-D - AUDIENCE-SPECIFIC DELEGATING HANDLER

```
public sealed class DownstreamCredentialHandler(
    DownstreamAudience audience,
    IDownstreamCredentialProvider credentials)
    : DelegatingHandler
{
    protected override async Task<HttpResponseMessage> SendAsync(
        HttpRequestMessage request,
        CancellationToken cancellationToken)
    {
        if (!request.Options.TryGetValue(
            BffRequestOptions.VerifiedOperator, out var actor))
            throw new MissingVerifiedOperatorContext();

        var token = await credentials.AcquireAsync(
            audience, actor, cancellationToken);
        if (token.Audience != audience)
            throw new DownstreamAudienceMismatch();

        request.Headers.Authorization =
            new AuthenticationHeaderValue("Bearer", token.Value);
        return await base.SendAsync(request, cancellationToken);
    }
}

[Fact]
public async Task BFF_DELEG_001_browser_token_is_never_forwarded()
{
    await fixture.ComposeAsync(
        browserToken: "browser-token-canary");
    fixture.DownstreamAuthorizationHeaders.Should()
        .OnlyContain(x => x != "Bearer browser-token-canary");
    fixture.AcquiredAudiences.Should().BeEquivalentTo(
        DownstreamAudience.AlarmManagement,
        DownstreamAudience.RootCause);
}

[Fact]
public async Task cross_tenant_credential_request_fails_closed()
{
    var act = () => fixture.AcquireRootCauseTokenAsync(
        actorTenant: "tenant-a", requestedTenant: "tenant-b");
    await act.Should().ThrowAsync<DelegationScopeRejected>();
    fixture.DownstreamCalls.Should().Be(0);
}
```

THE BFF IS NOT A PRIVILEGED TUNNEL

The browser credential authenticates the UI request. A separate accepted flow produces an audience-, capability-, tenant- and actor-bound service credential.

TYPED PUBLIC CLIENTS AND CLOSED OUTCOMES

Provider status becomes safe composition data, not leaked JSON

EXECUTABLE ASSET 22-E - CLOSED PROVIDER READ UNION

```
public abstract record ProviderReadOutcome<T>
{
    public sealed record Available(
        T Value,
        DateTimeOffset SourceObservedAtUtc,
        DateTimeOffset RetrievedAtUtc)
        : ProviderReadOutcome<T>;
    public sealed record NotObserved(
        DateTimeOffset RetrievedAtUtc) : ProviderReadOutcome<T>;
    public sealed record NotDisclosed(
        DateTimeOffset RetrievedAtUtc) : ProviderReadOutcome<T>;
    public sealed record TemporarilyUnavailable(
        string SafeCode,
        TimeSpan? RetryAfter,
        DateTimeOffset RetrievedAtUtc)
        : ProviderReadOutcome<T>;
    public sealed record ContractViolation(
        string SafeCode,
        DateTimeOffset RetrievedAtUtc)
        : ProviderReadOutcome<T>;
}

public interface IAlarmManagementPublicClient
{
    Task<ProviderReadOutcome<AlarmDecisionPublicV1>> GetDecisionAsync(
        string decisionId,
        VerifiedOperatorContext actor,
        CancellationToken cancellationToken);
}

public interface IRootCausePublicClient
{
    Task<ProviderReadOutcome<RootCauseCasePublicV1>>
        FindByPublicFactAsync(
            string publicFactId,
            VerifiedOperatorContext actor,
            CancellationToken cancellationToken);
}
```

EXECUTABLE ASSET 22-F - STRICT ALARMMANAGEMENT CLIENT

```
public async Task<ProviderReadOutcome<AlarmDecisionPublicV1>>
    GetDecisionAsync(
        string decisionId,
        VerifiedOperatorContext actor,
        CancellationToken cancellationToken)
{
    using var request = new HttpRequestMessage(
        HttpMethod.Get,
        $"api/v1/alarm-flood-decisions/{Uri.EscapeDataString(decisionId)}");
    request.Headers.Accept.ParseAdd(
        "application/vnd.nexus1.alarm-decision-v1+json");
    request.Options.Set(BffRequestOptions.VerifiedOperator, actor);

    using var response = await http.SendAsync(
        request,
        HttpCompletionOption.ResponseHeadersRead,
        cancellationToken);
    return response.StatusCode switch
    {
        HttpStatusCode.OK => await DecodeAvailableAsync(
            response, cancellationToken),
        HttpStatusCode.NotFound =>
            new ProviderReadOutcome<AlarmDecisionPublicV1>.NotObserved(
                clock.GetUtcNow()),
        HttpStatusCode.Forbidden =>
            new ProviderReadOutcome<AlarmDecisionPublicV1>.NotDisclosed(
                clock.GetUtcNow()),
        HttpStatusCode.ServiceUnavailable =>
            SafeUnavailable(response),
        _ => new ProviderReadOutcome<AlarmDecisionPublicV1>
            .ContractViolation(
                "alarm-provider-response-unsupported",
                clock.GetUtcNow())
    };
}

[Fact]
public async Task malformed_provider_body_never_reaches_composer()
{
    var outcome = await fixture.ReadMalformedAlarmResponseAsync();
    outcome.Should().BeOfType<
        ProviderReadOutcome<AlarmDecisionPublicV1>.ContractViolation>();
    fixture.ComposerCalls.Should().Be(0);
    fixture.Logs.Should().NotContain(
        x => x.Contains(fixture.ProviderBodyCanary));
}
```

ONLY REVIEWED FIELDS CROSS THE CLIENT SEAM

Exact media type, bounded body, strict JSON and source-specific mapping must pass. Raw Problem Details, private identifiers and partially decoded bodies stop here.

CAUSAL LOOKUP AND DEADLINE OWNERSHIP

The required anchor supplies the optional lookup identity

EXECUTABLE ASSET 22-G - MONOTONIC REQUEST BUDGET

```
public sealed class RequestBudget(
    TimeProvider clock,
    TimeSpan total)
{
    private readonly long started = clock.GetTimestamp();

    public TimeSpan Remaining
    {
        get
        {
            var value = total - clock.GetElapsedTime(started);
            return value > TimeSpan.Zero ? value : TimeSpan.Zero;
        }
    }

    public BudgetSlice CreateSlice(
        TimeSpan maximum,
        CancellationToken requestAborted)
    {
        var duration = Remaining < maximum ? Remaining : maximum;
        if (duration <= TimeSpan.Zero)
            throw new CompositionDeadlineExpired();
        var timeout = new CancellationTokenSource(duration, clock);
        var linked = CancellationTokenSource.CreateLinkedTokenSource(
            requestAborted, timeout.Token);
        return new BudgetSlice(timeout, linked);
    }
}

public sealed class BudgetSlice(
    CancellationTokenSource timeout,
    CancellationTokenSource linked) : IDisposable
{
    public CancellationToken Token => linked.Token;

    public void Dispose()
    {
        linked.Dispose();
        timeout.Dispose();
    }
}
```

EXECUTABLE ASSET 22-H - REQUIRED-ANCHOR-FIRST QUERY

```
public async Task<BffCompositionOutcome> HandleAsync(
    GetAlarmFloodInvestigation query,
    VerifiedOperatorContext actor,
    CancellationToken requestAborted)
{
    var budget = new RequestBudget(clock, options.TotalBudget);
    using var anchorToken = budget.CreateSlice(
        options.AlarmMaximum, requestAborted);
    var alarm = await alarmClient.GetDecisionAsync(
        query.DecisionId, actor, anchorToken.Token);

    if (alarm is not
        ProviderReadOutcome<AlarmDecisionPublicV1>.Available anchor)
        return failures.RequiredAnchor(alarm);

    ProviderReadOutcome<RootCauseCasePublicV1> rootCause;
    try
    {
        using var optionalToken = budget.CreateSlice(
            options.RootCauseMaximum, requestAborted);
        rootCause = await rootClient.FindByPublicFactAsync(
            anchor.Value.PublicFactId,
            actor,
            optionalToken.Token);
    }
    catch (OperationCanceledException)
    {
        when (!requestAborted.IsCancellationRequested)
        {
            rootCause = new ProviderReadOutcome<RootCauseCasePublicV1>
                .TemporarilyUnavailable(
                    "root-cause-timeout", RetryAfter: null,
                    RetrievedAtUtc: clock.GetUtcNow());
        }
    }
    catch (CompositionDeadlineExpired)
    {
        rootCause = new ProviderReadOutcome<RootCauseCasePublicV1>
            .TemporarilyUnavailable(
                "root-cause-budget-exhausted", RetryAfter: null,
                RetrievedAtUtc: clock.GetUtcNow());
    }

    return composer.Compose(
        query.DecisionId, anchor, rootCause, clock.GetUtcNow());
}
```

PARALLELIZE ONLY INDEPENDENT READS

The route supplies a decision ID, not a trusted public-fact ID. RootCause lookup therefore starts only after the authorized AlarmManagement anchor supplies it. Speculative fan-out would use an unproven identity.

SOURCE-STATE INVARIANTS AND COMPOSITION

Every slice carries one legal value, problem and time combination

EXECUTABLE ASSET 22-I - SOURCE SNAPSHOT FACTORY

```
public static class SourceSnapshot
{
    public static SourceSnapshotV1<T> Available<T>(
        string source,
        T value,
        DateTimeOffset observed,
        DateTimeOffset retrieved,
        DateTimeOffset composed)
    {
        var age = Math.Max(
            0, (long)(composed - observed).TotalMilliseconds);
        return new(
            source, SourceStateV1.Available, retrieved,
            observed, age, value, SafeProblem: null);
    }

    public static SourceSnapshotV1<T> NotYetObserved<T>(
        string source,
        DateTimeOffset retrieved) =>
        new(source, SourceStateV1.NotYetObserved, retrieved,
            null, null, default, null);

    public static SourceSnapshotV1<T> NotDisclosed<T>(
        string source,
        DateTimeOffset retrieved) =>
        new(source, SourceStateV1.NotDisclosed, retrieved,
            null, null, default, null);

    public static SourceSnapshotV1<T> Unavailable<T>(
        string source,
        DateTimeOffset retrieved,
        string safeCode,
        int? retryAfterSeconds) =>
        new(source, SourceStateV1.TemporarilyUnavailable,
            retrieved, null, null, default,
            new(safeCode, retryAfterSeconds));
}
```

EXECUTABLE ASSET 22-J - CLOSED COMPOSITION MAPPING

```
public AlarmFloodInvestigationViewV1 Compose(
    string decisionId,
    ProviderReadOutcome<AlarmDecisionPublicV1>.Available alarm,
    ProviderReadOutcome<RootCauseCasePublicV1> root,
    DateTimeOffset composedAtUtc)
{
    var alarmSlice = SourceSnapshot.Available(
        "AlarmManagement",
        alarm.Mapper.Map(alarm.Value),
        alarm.SourceObservedAtUtc,
        alarm.RetrievedAtUtc,
        composedAtUtc);

    var rootSlice = root switch
    {
        ProviderReadOutcome<RootCauseCasePublicV1>.Available x =>
            SourceSnapshot.Available(
                "RootCause", root.Mapper.Map(x.Value),
                x.SourceObservedAtUtc, x.RetrievedAtUtc, composedAtUtc),
        ProviderReadOutcome<RootCauseCasePublicV1>.NotObserved x =>
            SourceSnapshot.NotYetObserved<RootCauseCaseSliceV1>(
                "RootCause", x.RetrievedAtUtc),
        ProviderReadOutcome<RootCauseCasePublicV1>.NotDisclosed x =>
            SourceSnapshot.NotDisclosed<RootCauseCaseSliceV1>(
                "RootCause", x.RetrievedAtUtc),
        ProviderReadOutcome<RootCauseCasePublicV1>
            .TemporarilyUnavailable x =>
            SourceSnapshot.Unavailable<RootCauseCaseSliceV1>(
                "RootCause", x.RetrievedAtUtc, x.SafeCode,
                x.RetryAfter is null
                ? null : (int)x.RetryAfter.Value.TotalSeconds),
        _ => throw new UnsafeProviderOutcome();
    };

    return views.Create(
        decisionId,
        rootSlice.State == SourceStateV1.Available
            ? "complete" : "partial",
        composedAtUtc,
        alarmSlice,
        rootSlice);
}
```

COMPOSED TIME IS NOT A GLOBAL BUSINESS TIME

Each available value preserves its provider observation time and calculated age. The composition timestamp describes only when this screen representation was built.

COMMAND ROUTING AND RESILIENCE

A UI command is sent once to exactly one owning service

EXECUTABLE ASSET 22-K - PROVIDER-SPECIFIC CLIENT POLICIES

```
services.AddHttpClient<IAAlarmManagementPublicClient,
    AlarmManagementPublicClient>()
    .AddHttpMessageHandler(sp =>
        new DownstreamCredentialHandler(
            DownstreamAudience.AlarmManagement,
            sp.GetRequiredService<IDownstreamCredentialProvider>()))
    .AddStandardResilienceHandler(options =>
    {
        options.TotalRequestTimeout.Timeout =
            TimeSpan.FromMilliseconds(900);
        options.AttemptTimeout.Timeout =
            TimeSpan.FromMilliseconds(700);
        options.Retry.MaxRetryAttempts = 1;
        options.Retry.DisableForUnsafeHttpMethods();
    });

services.AddHttpClient<IRootCausePublicClient,
    RootCausePublicClient>()
    .AddHttpMessageHandler(sp =>
        new DownstreamCredentialHandler(
            DownstreamAudience.RootCause,
            sp.GetRequiredService<IDownstreamCredentialProvider>()))
    .AddStandardResilienceHandler(options =>
    {
        options.TotalRequestTimeout.Timeout =
            TimeSpan.FromMilliseconds(700);
        options.Retry.MaxRetryAttempts = 1;
        options.Retry.DisableForUnsafeHttpMethods();
    });
```

EXECUTABLE ASSET 22-L - SINGLE-OWNER COMMAND ROUTE

```
public async Task<IResult> RouteDetectAlarmFloodAsync(
    DetectAlarmFloodUiRequestV1 request,
    VerifiedOperatorContext actor,
    CancellationToken cancellationToken)
{
    var ownerRequest = commandMapper.ToAlarmManagement(request);
    var outcome = await alarmCommands.DetectAlarmFloodAsync(
        ownerRequest, actor, cancellationToken);
    return commandResponses.MapOwnerOutcome(outcome);
}

[Fact]
public async Task BFF_CMD_001_command_targets_one_owner_once()
{
    var response = await fixture.PostDetectAlarmFloodAsync();
    response.StatusCode.Should().Be(HttpStatusCode.Accepted);
    fixture.AlarmManagementPostAttempts.Should().Be(1);
    fixture.RootCauseCommandAttempts.Should().Be(0);
    fixture.BffBusinessWrites.Should().Be(0);
    response.Headers.Location.Should()
        .Be(fixture.AlarmManagementLocation);
}

[Fact]
public async Task lost_POST_response_is_not_retried_by_BFF()
{
    var act = () => fixture.PostWithLostOwnerResponseAsync();
    await act.Should().ThrowAsync<AmbiguousOwnerOutcome>();
    fixture.AlarmManagementPostAttempts.Should().Be(1);
    fixture.RootCauseCommandAttempts.Should().Be(0);
}
```

QUERY RETRY POLICY CANNOT LEAK INTO COMMANDS

Safe GET retry remains bounded by the request budget. POST automatic retry is disabled; Chapter 23 must first prove an idempotency and replay contract.

HOSTED FAULT, CACHE AND OBSERVABILITY TESTS

Partial success remains fast, private and visibly sourced

EXECUTABLE ASSET 22-M - GOLDEN PARTIAL AND COMPLETE HOST FIXTURES

```
[Fact]
public async Task BFF_COMP_001_optional_absence_returns_honest_partial()
{
    fixture.AlarmManagement.ReturnsAvailable(
        publicFactId: "fact-am-7716");
    fixture.RootCause.ReturnsNotObserved();

    var response = await fixture.GetInvestigationAsync(
        "decision-am-9184");
    response.StatusCode.Should().Be(HttpStatusCode.OK);
    var view = await response.Content
        .ReadFromJsonAsync<AlarmFloodInvestigationViewV1>();
    view!.Completeness.Should().Be("partial");
    view.AlarmManagement.State.Should().Be(SourceStateV1.Available);
    view.RootCause.State.Should().Be(
        SourceStateV1.NotYetObserved);
    view.RootCause.Value.Should().BeNull();
    (await response.Content.ReadAsStringAsync())
        .Should().NotContain("case-rc-");
}

[Fact]
public async Task later_independent_read_returns_complete()
{
    fixture.AlarmManagement.ReturnsAvailable(
        publicFactId: "fact-am-7716");
    fixture.RootCause.ReturnsAvailable(caseId: "case-rc-3107");
    var view = await fixture.GetViewAsync("decision-am-9184");
    view.Completeness.Should().Be("complete");
    view.RootCause.Value!.CaseId.Should().Be("case-rc-3107");
    view.AlarmManagement.SourceObservedAtUtc.Should()
        .NotBe(view.RootCause.SourceObservedAtUtc);
}
```

EXECUTABLE ASSET 22-N - DEADLINE, CACHE AND TRACE GATES

```
[Fact]
public async Task BFF_FAIL_001_optional_timeout_does_not_hide_anchor()
{
    fixture.AlarmManagement.ReturnsAvailable(
        publicFactId: "fact-am-7716");
    fixture.RootCause.TimesOut();
    var measured = await fixture.MeasureGetAsync();
    measured.Response.StatusCode.Should().Be(HttpStatusCode.OK);
    measured.View.RootCause.State.Should().Be(
        SourceStateV1.TemporarilyUnavailable);
    measured.Elapsed.Should().BeLessThan(fixture.TotalBudget);
}

[Fact]
public async Task personalized_response_is_never_shared_or_stored()
{
    var response = await fixture.GetInvestigationAsync(
        "decision-am-9184");
    response.Headers.CacheControl!.Private.Should().BeTrue();
    response.Headers.CacheControl.NoStore.Should().BeTrue();
    response.Headers.Vary.Should().NotContain("");
}

[Fact]
public async Task BFF_OBS_001_trace_matches_actual_attempts()
{
    await fixture.GetInvestigationAsync("decision-am-9184");
    fixture.TraceGraph.ServerSpans.Should().ContainSingle();
    fixture.TraceGraph.DownstreamSpans.Should().HaveCount(
        fixture.DownstreamAttempts);
    fixture.MetricLabelNames.Should().NotContain(
        x => x.Contains("decisionId") || x.Contains("caseId"));
    fixture.Logs.Should().NotContain(
        x => x.Contains(fixture.BrowserTokenCanary));
}
```

THE REQUIRED ANCHOR CONTROLS USABILITY

Optional RootCause failure becomes explicit source state. Required AlarmManagement failure produces a safe unusable-composite problem, never a RootCause-only screen.

COMPATIBILITY AND ACCEPTANCE

Old and new consoles must preserve source-state meaning

EXECUTABLE ASSET 22-O - COMPATIBILITY AND INVARIANT CORPUS

```
[Theory]
[MemberData(nameof(SourceStateCases))]
public void BFF_SOURCE_001_only_legal_snapshot_shapes_are_accepted(
    SourceSnapshotCase test)
{
    var act = () => fixture.SourceSchema.Validate(test.Json);
    if (test.IsLegal) act.Should().NotThrow();
    else act.Should().Throw<SourceSnapshotContractRejected>();
}

[Theory]
[MemberData(nameof(SupportedBffPairs))]
public async Task BFF_COMPAT_001_old_new_pairs_preserve_meaning(
    BffCompatibilityCase test)
{
    var result = await fixture.ExecutePairAsync(test);
    result.Status.Should().Be(test.ExpectedStatus);
    result.Completeness.Should().Be(test.ExpectedCompleteness);
    result.UnknownStateWasSilentlyDefaulted.Should().BeFalse();
    result.UndocumentedProblem.Should().BeFalse();
}

[Fact]
public async Task denied_and_absent_follow_one_disclosure_oracle()
{
    var denied = await fixture.GetDeniedRootCauseAsync();
    var absent = await fixture.GetHiddenAbsentRootCauseAsync();
    denied.PublicSnapshot.Should().BeEquivalentTo(
        absent.PublicSnapshot);
    denied.Logs.Should().NotContain("denied-resource-exists");
}
```

EXECUTABLE ASSET 22-P - ACCEPTANCE EVIDENCE RECORD

```
{
  "evidenceId": "BFF-REVIEW-001", "chapter": 22,
  "artifactCommit": null, "sdk": "net8.0",
  "hostBuild": null, "credentialFlow": null,
  "alarmClient": null, "rootCauseClient": null,
  "resilienceConfigDigest": null, "openApiDigest": null,
  "tests": {
    "BFF-BOUND-001": "PENDING", "BFF-CONTRACT-001": "PENDING",
    "BFF-SOURCE-001": "PENDING", "BFF-AUTHN-001": "PENDING",
    "BFF-AUTHZ-001": "PENDING", "BFF-DELEG-001": "PENDING",
    "BFF-QUERY-001": "PENDING", "BFF-COMP-001": "PENDING",
    "BFF-CMD-001": "PENDING", "BFF-FAIL-001": "PENDING",
    "BFF-CACHE-001": "PENDING", "BFF-OBS-001": "PENDING",
    "BFF-COMPAT-001": "PENDING"
  },
  "faultCorpusAccepted": false,
  "securityReviewAccepted": false,
  "accepted": false
}
```

Chapter 22 code-audit result

The chapter now exposes one executable route from verified operator identity to service-specific credentials, strict typed reads, anchor-first causal lookup, honest source states, one-owner commands and bounded runtime evidence.

CURRENT VERDICT

The Operator Console boundary is implementable in shape, but no identity flow, host, provider client, fault corpus or compatibility matrix has executed. All BFF evidence remains explicitly PENDING.

Implementation sequence

1. Accept ADR-020 and publish the Operator Console route, source-state, requiredness, disclosure and ownership catalogue.
2. Create the net8.0 OperatorConsole.Bff project with strict dependency tests and no Domain/repository/shared-database reference.
3. Define versioned UI request/response/problem DTOs and state/value/problem/timestamp invariants; generate the Chapter 24 schema corpus.
4. Configure browser authentication and operator-console policies; select and prove the downstream credential/delegation model.
5. Implement typed AlarmManagement and RootCause clients with explicit media types, closed outcomes and provider-specific authorization handling.
6. Implement explicit provider-to-source mappers and the required-anchor/optional-slice composition state machine.
7. Add one bounded parallel query deadline with measured per-source budgets, cancellation, bulkhead/circuit and narrowly proven safe GET retry.
8. Implement UI command mapping to one owner, disable unsafe automatic retry and defer idempotency/replay semantics to Chapter 23.
9. Apply private/no-store baseline, trace-context propagation, bounded logs and low-cardinality completeness/source metrics.
10. Execute fault, latency, security, cache, cancellation, lost-response, output/log and old/new client matrices; publish BFF evidence without advancing RootCause beyond G0/G1.

What this chapter has proved

Chapter 22 has specified the Operator Console BFF as an interface-specific composition boundary. It has shown a full first partial and later complete read for decision-am-9184, fact-am-7716, flood-am-6031 and case-rc-3107; preserved separate owner truth and source timestamps; defined explicit source-state, partial-failure, authorization/delegation, timeout, cache and telemetry semantics; and routed commands to one provider without browser orchestration or BFF business ownership.

It has not hosted ASP.NET Core, exchanged a downstream credential, called a service, enforced a real policy, measured a budget, opened a circuit, cancelled a fan-out, served cached data, propagated an actual trace, forwarded a command or produced runtime evidence. OperatorConsole.Bff is specified, not implemented or evidenced. RootCause remains G0 accepted and G1 specified.

ONE SCREEN, MULTIPLE OWNERS, HONEST STATE

The BFF may make a distributed story easier to read. It may not make the story falsely atomic. Every value keeps its owner, every time keeps its meaning, every partial result keeps its reason, and every command returns to the service that can actually decide and commit it.

Chapter checkpoint

Question	Expected answer
Does the BFF own AlarmManagement or RootCause state?	No. It owns only the Operator Console contract, composition policy and UI-specific mapping.
May the browser call both services and merge JSON?	No. The BFF centralizes authenticated/authorized composition, provenance, deadlines and partial semantics.
What is required for a usable investigation view?	An authorized AlarmManagement decision anchor; RootCause is an explicit optional slice in this reference.
What does notYetObserved mean?	An authorized RootCause query exposes no matching projection now; it does not mean failure, final absence or downstream completion.

Question	Expected answer
What does complete mean?	Both requested slices were available for this BFF request; it is not a simultaneous distributed snapshot.
May a RootCause timeout become null?	No. It becomes temporarilyUnavailable with bounded safe meaning, or the whole response fails if policy makes it required.
Why is 206 not used?	206 is for HTTP partial-content range transfer, not application-level multi-source completeness.
May the BFF use one powerful service credential?	Only if an accepted constrained design preserves capability and actor/tenant scope; no authorization bypass is allowed.
May the BFF retry POST after a timeout?	Not automatically. Chapter 23 must define and prove an idempotency/replay contract first.
Can one ETag prove a global version?	No. A future BFF validator would identify a composition representation, not one distributed business version.
What cache policy applies now?	Personalized composites use private, no-store until cache key, freshness, invalidation and disclosure evidence exists.
What does Chapter 23 add?	Exact version/media negotiation, stable Problem Details evolution and idempotency-key/replay semantics for ambiguous commands.

NEXT CHAPTER

Chapter 23, Versioning, Problem Details, and Idempotency Keys, will define how public and BFF contracts evolve, how clients negotiate supported representations, how stable failure types remain machine-readable and how an idempotency key resolves the ambiguous command outcomes that Chapters 18, 21 and 22 deliberately refused to guess.

PART IV - PUBLIC CONTRACTS AND THE SYNCHRONOUS EDGE

23. Versioning, Problem Details, and Idempotency Keys

A public contract survives change only when the client can identify the representation it asked for, classify failure without parsing prose and retry an ambiguous command without causing a second business effect. Versioning, Problem Details and idempotency belong to one boundary discipline: every repeated exchange must preserve meaning while the system evolves.

CHAPTER ARGUMENT

Use explicit major routes and media contracts, evolve compatible representations additively, publish stable RFC 9457 problem types and require an owner-scoped idempotency key for retrievable non-idempotent commands. Store the key claim, request fingerprint and replay response in the same owner-local atomic boundary as the accepted decision. A retry may recover the original outcome; it may never invent a new one.

Chapters 18, 21 and 22 stopped at the same ambiguity. AlarmManagement may commit decision-am-9184 and public fact-am-7716 in TX-A, yet the 202 response can disappear before the client receives it. The BFF cannot call that failure and cannot safely send a second POST. This chapter closes that gap while preserving the earlier truth boundary: the initial and replayed 202 describe AlarmManagement acceptance only. RootCause case-rc-3107 can exist later after its independent TX-B; it is never inserted into the owner replay record.

The reference route is POST /api/v1/alarm-flood-detections. The caller sends Idempotency-Key: idem-am-7f9c2a and explicit v1 request/response media types. AlarmManagement resolves a caller/site/capability scope, computes a canonical semantic fingerprint, claims the key and executes the command once. If the response is lost, the same key and fingerprint return the recorded 202, Location and body without executing the command again.

This remains a net8.0 reference specification. No negotiator, Problem Details catalogue, idempotency store, transaction integration, cleanup job, concurrent fixture or compatibility client has compiled or executed. RootCause remains G0 accepted and G1 specified. Chapter 24 owns executable compatibility tests. The expired IETF Idempotency-Key Internet-Draft informs terminology; it is not an RFC and does not turn NEXUS-1 project choices into standards.

Version is not one number

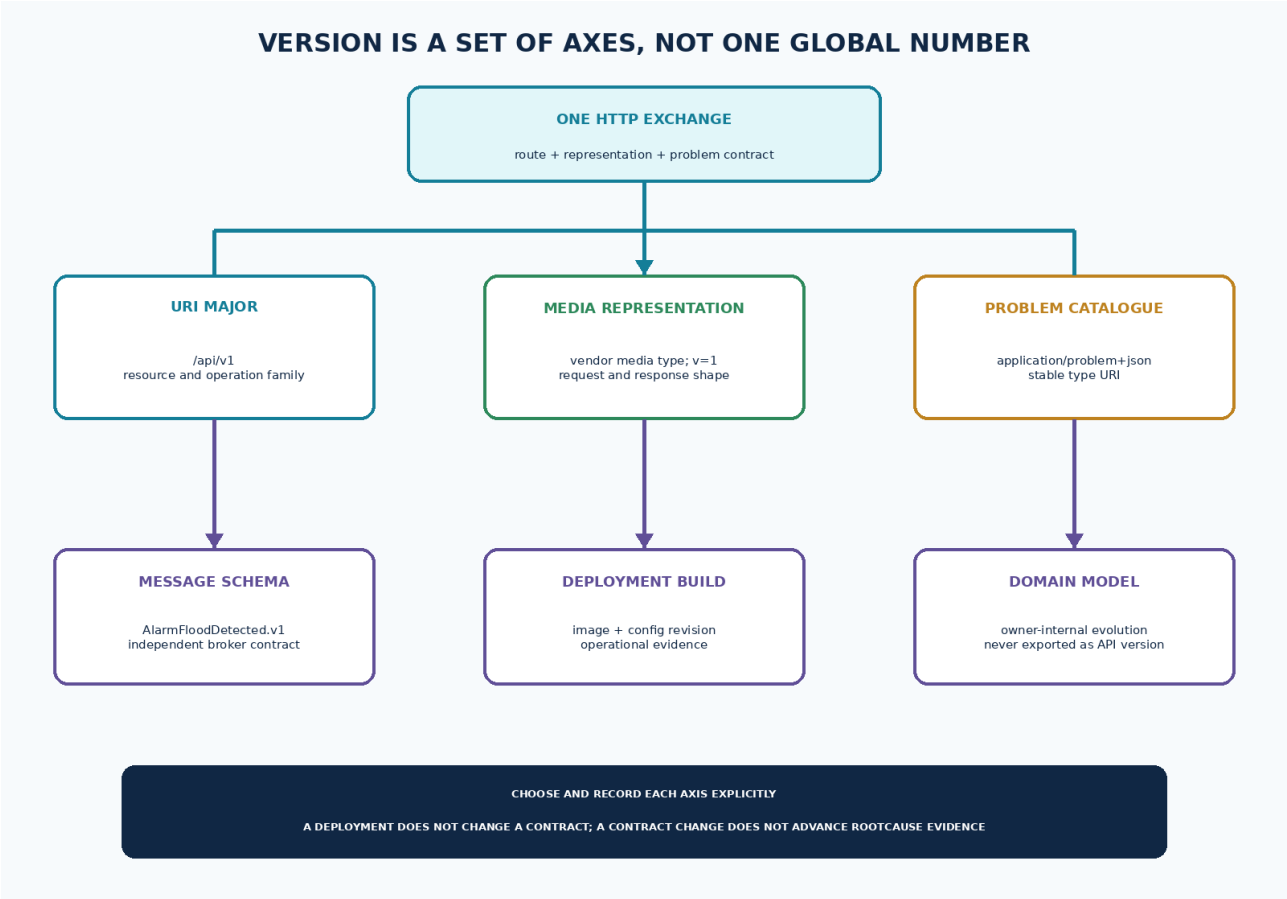


Figure 23.1 - NEXUS-1 separates API, representation, problem, message, deployment and domain evolution instead of using one global version.

Version axis	Identifies	Changes when	Must not be confused with
HTTP protocol	Wire protocol such as HTTP/1.1, HTTP/2 or HTTP/3.	Transport capability changes.	NEXUS-1 API version.
API route major	Resource/operation family at /api/v1 or /ui-api/v1.	Breaking public route or required-input change is accepted.	Deployment or database schema.
Media representation	Request/response field vocabulary and serialization.	A separately negotiated representation changes.	CLR class or serializer build.
Problem catalogue	Stable type URI and safe extension semantics.	A machine-distinct failure type is introduced.	Human title/detail or exception class.
Integration message	Public broker fact such as AlarmFloodDetected.v1.	Consumer-facing event compatibility changes.	Synchronous HTTP response.
Deployment/build	Image, package, commit and configuration running.	Software/configuration is deployed.	A public contract promise.
Domain model	Owner-internal aggregates and invariants.	Owner model evolves.	An exported DTO.

NO GLOBAL VERSION RULE

decision-am-9184, fact-am-7716, case-rc-3107 and a BFF composition can have different owner, schema, projection and observation versions. A deployment label or BFF ETag cannot collapse them into one distributed business version.

The NEXUS-1 versioning profile

The reference profile uses a URI major for discoverability/routing and explicit vendor media types for representation identity. Content-Type identifies the request representation sent. Accept identifies response representations the caller can consume application/problem+json remains the standardized

Decision	Reference rule	Failure behavior
Route major	Required in /api/v1 and /ui-api/v1 templates.	Unknown/hidden route follows policy-safe 404; no silent rewrite.
Request media	Content-Type: application/vnd.nexus1.<contract>+json;v=1.	Unsupported type/version returns 415 Problem Details.
Response media	Accept lists a supported representation; q-values follow HTTP.	No acceptable representation returns 406.
Missing Accept	Treat as */* and return the operation's documented default v1.	Never switch silently to a newer default.
Problem media	Errors use application/problem+json and a stable type URI.	Failure identity is independent of success media.
Minor evolution	Compatible optional response fields may appear in v1.	Breaking semantics require a new major/representation.

IMPLEMENTATION SHAPE

```

POST /api/v1/alarm-flood-detections HTTP/1.1
Authorization: Bearer <credential>
Idempotency-Key: idem-am-7f9c2a
Content-Type: application/vnd.nexus1.alarm-flood-detection-request+json;v=1
Accept: application/vnd.nexus1.alarm-flood-accepted+json;v=1

{
  "siteId": "site-ath-01",
  "alarmIds": ["alarm-701", "alarm-702", "alarm-703"],
  "requestedSeverity": "critical",
  "clientRequestId": "request-am-20482"
}

```

clientRequestId remains correlation data. It is not the idempotency key, decision ID or trace ID. The server accepts the key only in one canonical header location and never uses its raw value as a metric label or business identifier.

Negotiation is deterministic

1. Match the explicit route major and operation; never redirect a command to a newer major.
2. Authenticate and apply coarse capability authorization before returning resource-specific detail.
3. Validate request Content-Type; reject unsupported request media/version with 415.
4. Select a response representation from Accept; absent Accept uses the documented default, while no match returns 406.
5. Validate exactly one Idempotency-Key before command execution.
6. Bind/validate the selected schema, resolve scope and compute the versioned semantic fingerprint.
7. Execute or replay through the owner state machine; emit the selected Content-Type or application/problem+json.

Condition	Status	Problem suffix	Client action
Request media unsupported	415	unsupported-media-type	Send a documented request representation.
No acceptable response media	406	representation-not-acceptable	Send a supported Accept value.
Malformed selected request	400	request-validation-failed	Correct bounded public errors.
Authenticated but forbidden	403 or safe 404	operation-forbidden/hidden	Do not infer protected existence.
Key missing or invalid	400	idempotency-key-required/invalid	Create one valid high-entropy key.

Compatible evolution and retirement

Change	Within v1?	Provider obligation
Add optional response member	Usually yes	Document absence/default; never repurpose an old field.
Add optional request member	Only with an unambiguous omitted default	Old omission retains old behavior

Change	Within v1?	Provider obligation
Add enum/state value	Only if clients model unknown safely	Otherwise use a new major/representation.
Make optional field required	No	Stage capability or create new contract.
Remove/rename field	No	Retain through support window or create new major.
Change status/unit/null/time meaning	No	Wire compatibility cannot hide a semantic break.
Add optional problem extension	Yes	Never change an existing extension type/meaning.
Change type URI meaning	No	Mint a new problem type URI.

Deprecation does not silently change behavior. RFC 9745 Deprecation and the deprecation link relation can signal a migration window. RFC 8594 Sunset can signal expected future unresponsiveness. Sunset must not precede Deprecation. These fields are lifecycle signals, not cache freshness, authorization or guarantees of availability.

IMPLEMENTATION SHAPE

```
HTTP/1.1 200 OK
Content-Type: application/vnd.nexus1.alarm-flood-investigation+json;v=1
Deprecation: @1798675200
Sunset: Thu, 31 Dec 2026 23:59:59 GMT
Link: <https://docs.nexus1.example/migrations/alarm-flood-v1>;
      rel="deprecation"; type="text/html"
Vary: Accept
Cache-Control: private, no-store
```

RETIREMENT EVIDENCE

A date is not permission to remove a route. Removal needs client inventory, migration guide, usage telemetry, compatibility results, support window, rollback path and explicit owner/consumer/security/operations approval.

Problem Details is a catalogue, not an exception dump

RFC 9457 defines the standard problem detail representation. type is the stable machine identifier. title and detail are human-readable and may be localized; clients never parse them. The actual HTTP status carries protocol semantics and agrees with the status member. instance is an opaque safe occurrence identity only when disclosure policy permits it.

Member	NEXUS-1 rule	Forbidden use
type	Stable HTTPS URI with immutable documented semantics.	One generic URI or a URI whose meaning changes.
title	Short human summary; localizable.	Client branch condition or raw exception.
status	Same as HTTP status when present.	Body 400 inside actual HTTP 500.
detail	Occurrence-specific safe explanation.	Stack, SQL, token, path or protected existence.
instance	Opaque safe occurrence URI/URN.	Private decision/case/user identifier.
errorCode	Stable bounded code when documented.	CLR/database/vendor exception code.
errors	Bounded JSON Pointer entries to public input.	Private model/query/full body echo.
retryAfterSeconds	Hint only when bounded and known.	Promise of completion.

The stable problem catalogue

Problem suffix	Status	Stable meaning	Retry/disclosure rule
request-validation-failed	400	Selected public request violates constraints.	Correct bounded public pointers.
authentication-required	401	Authentication proof is missing/invalid.	No protected resource detail.

Problem suffix	Status	Stable meaning	Retry/disclosure rule
representation-not-acceptable	406	No response representation matches Accept.	Choose documented media.
unsupported-media-type	415	Request media/version unsupported.	Choose documented media.
state-conflict	409	Request conflicts with owner state.	Refresh; no blind retry.
idempotency-key-required	400	Command requires one key header.	No command ran.
idempotency-key-invalid	400	Key profile is invalid.	Never echo the raw key.
idempotency-key-reused	409	Scoped key binds a different fingerprint.	No execution or prior-response disclosure.
idempotency-request-in-progress	409	Matching key is owned by unfinished attempt.	Optional bounded Retry-After.
idempotency-record-expired	409	Prior use known; replay payload unavailable.	Reconcile; never silently execute new.
dependency-unavailable	503	Required dependency prevents usable result.	Retry-After only if known.
unexpected-failure	500	Unclassified server failure.	Opaque occurrence; diagnostics private.

The exact 409 choices for mismatch, in-progress and expired-known records are NEXUS-1 reference decisions, not claims that an RFC mandates them. The route contract and Chapter 24 fixtures must make every branch explicit.

IMPLEMENTATION SHAPE

```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json
Cache-Control: no-store

{
  "type": "https://problems.nexus1.example/idempotency/key-reused",
  "title": "The idempotency key was already used.",
  "status": 409,
  "detail": "Use the original request or choose a new key.",
  "instance": "urn:nexus1:problem:occ-7a91",
  "errorCode": "IDEMPOTENCY_KEY_REUSED"
}
```

NO KEY ORACLE

Mismatch confirms only that the current authenticated scope cannot reuse this key for this request. It never returns the stored fingerprint, actor, request body, decision, Location or prior response.

The idempotency contract

Element	Reference specification	Invariant
Header	Exactly one value; 16-128 visible ASCII characters; at least 128 bits of randomness.	Reject empty, duplicate, control and oversized values before execution.
Scope	provider + route major + capability + verified actor/tenant + site.	No replay across owners, actors, tenants, sites or commands.
Fingerprint	SHA-256 over version, method, target, media and canonical semantic fields.	Formatting differences match; meaningful changes do not.
Record	Unique (scopeHash,keyHash); raw key is not a business ID.	Concurrent claims elect one execution owner.
States	inProgress, completed, expiredKnown plus bounded terminal policy.	Only completed + matching fingerprint authorizes success replay.
Atomicity	TX-A commits decision, public fact/outbox and completed replay together.	No completed replay without effect; no committed effect lacking recovery identity.
Replay	Status, response media, Location, safe headers and immutable body.	No mutable remapping or RootCause state.
Retention	Reference target 24 hours; policy/evidence required.	Published retry window never exceeds proven storage/backup/cleanup.
Expiry	Tombstone/status identity refuses unsafe silent reuse.	Payload purge cannot create a fresh command while ambiguity remains.

REFERENCE VALUES, NOT CAPACITY PROOF

The 24-hour target, key bounds, SHA-256 and entropy minimum are explicit design candidates. Threat, volume, latency, backup, cleanup and offline-window evidence are still required.

Canonical request fingerprint

The fingerprint asks whether this retry carries the same contracted command meaning within the same scope. It is not a raw-message checksum. The server binds v1, validates it, normalizes identifiers/timestamps according to the contract and serializes a stable semantic tuple. Verified identity/site/capability belong in server-resolved scope, never in client-editable body fields.

IMPLEMENTATION SHAPE

```
fingerprintInput = {
  "contract": "alarm-flood-detection-request.v1",
  "method": "POST",
  "target": "/api/v1/alarm-flood-detections",
  "siteId": "site-ath-01",
  "alarmIds": ["alarm-701", "alarm-702", "alarm-703"],
  "requestedSeverity": "critical",
  "clientRequestId": "request-am-20482"
}
fingerprint = "sha-256:" + Base64Url(SHA256(CanonicalUtf8(fingerprintInput)))
```

Concern	Canonical rule	Fixture
Property order/whitespace	Parse schema and hash semantic fields, not raw bytes.	Reformatted equal JSON matches.
Identifiers/Unicode	Use documented exact normalization; reject ambiguity.	Look-alikes never collide.
Array order	Preserve when meaningful; sort only for documented sets.	Reorder follows schema meaning.
Timestamps	Convert accepted values to UTC invariant precision.	Equivalent offsets match.
Unknown fields	Reject or ignore only as the selected schema states.	Security field cannot evade hash.
Media version	Include selected request representation.	v1 and future v2 never collide.

Fingerprint algorithm evolution and collision handling

Condition	Required behavior	Why
Same scope/key/profile/digest	Match only after validation and current authorization.	Digest alone does not authorize disclosure.
Same scope/key, different digest	409 key-reused; no execution/disclosure.	A changed command cannot inherit an old effect.
Unknown stored profile	Fail closed and reconcile operationally.	Guessing creates false match or duplicate.
Algorithm migration	Dual-read old accepted records; write only new profile after cutover.	Replay promises survive deployment.
Suspected collision/corruption	Quarantine, preserve evidence and refuse execution.	Degraded availability is safer than a second effect.

Chapter 24 must freeze the canonical schema and golden input/output corpus so a serializer, framework or algorithm upgrade cannot silently change replay identity.

Complete walkthrough: the 202 is lost and replayed

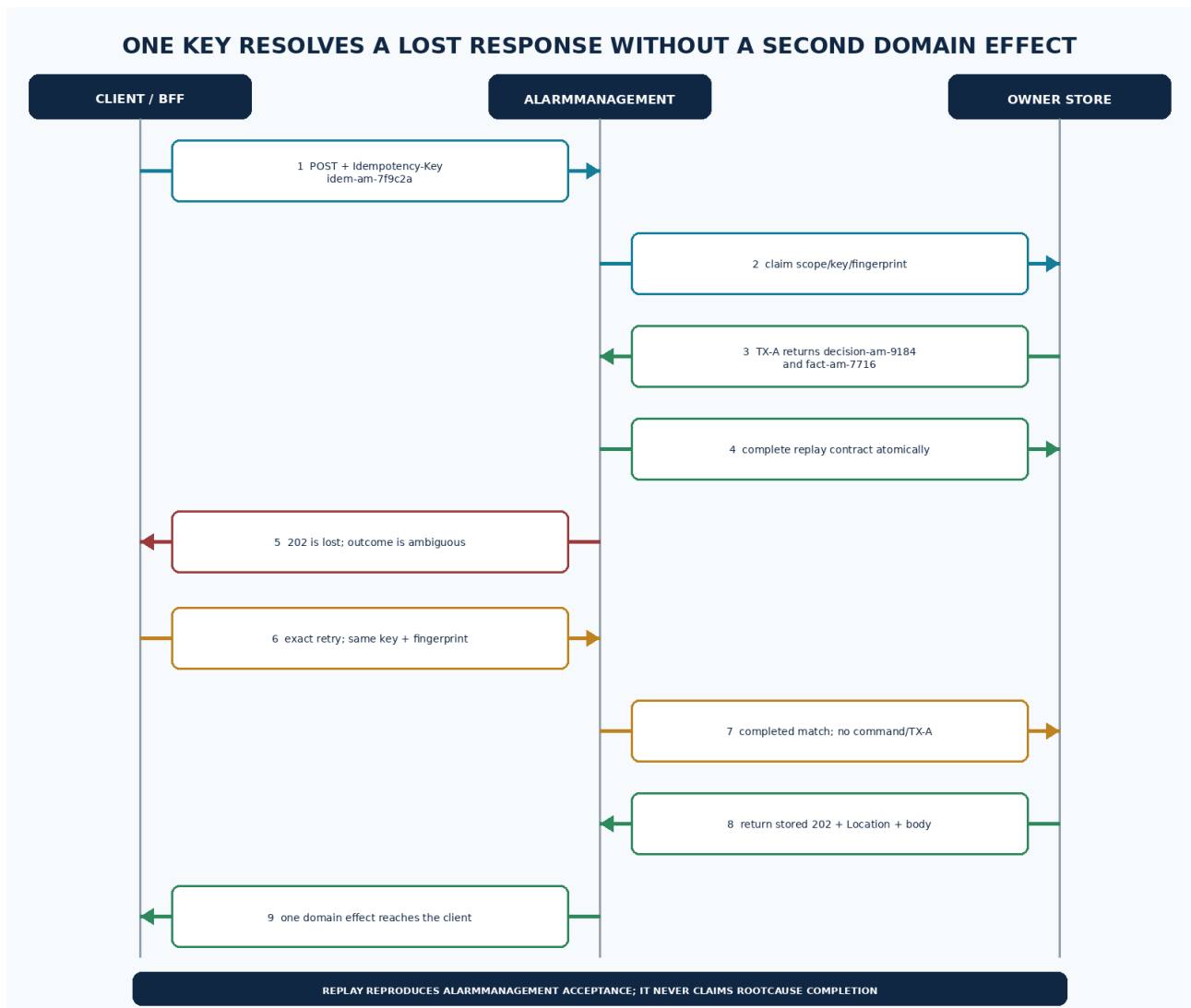


Figure 23.2 - AlarmManagement replays the original accepted response after a lost 202 without executing a second domain decision.

1. The authenticated BFF sends the v1 POST with key idem-am-7f9c2a and request-am-20482. Credential, route, capability and site resolve the server-side scope.
2. AlarmManagement validates route/media/key/schema, computes the canonical v1 fingerprint and claims the unique (scopeHash,keyHash) record as inProgress.
3. The absent-record claimant creates one Application command. TX-A commits decision-am-9184, flood-am-6031, fact-am-7716 and the public outbox intent.
4. The same TX-A completes replay metadata with matching fingerprint, 202, v1 media, Location and immutable response body.
5. The HTTP writer loses the response after TX-A. The caller knows only that the outcome is ambiguous; committed owner truth remains.
6. The client retries the exact request/key. AlarmManagement re-authenticates, re-authorizes, validates and recomputes scope/fingerprint.
7. The completed match bypasses Application, Domain, decision repository and outbox append. It returns the recorded owner contract.
8. The client receives decision-am-9184 and fact-am-7716. If RootCause later commits case-rc-3107, a new GET/BFF composition may expose it; the replayed POST never does.

IMPLEMENTATION SHAPE

```

HTTP/1.1 202 Accepted
Location: /api/v1/alarm-flood-detections/decision-am-9184
Content-Type: application/vnd.nexus1.alarm-flood-accepted+json;v=1
Cache-Control: no-store
Idempotency-Replayed: true

{
  "requestId": "request-am-20482",
  "decisionId": "decision-am-9184",
  "alarmFloodId": "flood-am-6031",
  "publicFactId": "fact-am-7716",
  "decisionStatus": "accepted",
  "acceptedAtUtc": "2026-07-19T12:00:04Z"
}

```

REPLAY TRUTH BOUNDARY

The replay is the exact accepted owner response for TX-A. It says nothing about broker dispatch, consumer receipt, RootCause evaluation or case-rc-3107. Rebuilding from today's multi-service state would turn replay into a time-varying query.

Replay under changed authorization and contract state

Idempotency proves what the owner accepted; it does not grant permanent access to stored data. Every retry is a new HTTP request and passes current authentication, policy and scope resolution before replay material is disclosed. Revocation can deny the body without erasing or repeating the historical effect.

Change	Replay decision	Rule
Same scope and still authorized	Replay matching completed record.	Original contract plus documented current transport headers.
Credential renewed	Evaluate the new credential/current policy.	Never accept an old credential by association with a key.
Capability/site revoked	Keep record; deny stored response.	Policy-safe 401/403/404; no key/decision oracle.
v1 deprecated but served	Replay original v1 contract.	Current Deprecation/Link/Sunset may accompany it.
v1 retired before retry window ends	Use accepted migration/reconciliation route.	Never reinterpret old body as v2 or execute new v2 command.
Stronger data classification	Fail closed unless an accepted redacted/status alternative exists.	Historical storage never overrides current disclosure.

The replay response envelope

Response element	Stored or recomputed?	Replay rule
HTTP status	Stored	Return the original accepted status; never upgrade 202 to 200 because later work completed.
Success Content-Type/version	Stored	Return the representation selected for the original accepted response.
Location	Stored	Preserve the AlarmManagement owner location for decision-am-9184.
Response body	Stored immutable bytes or proven deterministic owner snapshot	Never rebuild from current multi-service state or insert case-rc-3107.
Cache-Control	Current allowlisted policy constrained by original contract	Keep command/replay data no-store; a weaker historical header is never revived.
Deprecation/Link/Sunset	Current lifecycle metadata when its scope is documented	May accompany a still-served v1 replay without changing the stored result.
Date, trace, Server and transport fields	Recomputed or omitted	They describe this HTTP attempt, not the original domain acceptance.
Authentication/cookies/downstream headers	Never stored for replay	Credentials, sessions and raw provider headers cannot cross attempts or callers.

Concurrent duplicates and state transitions

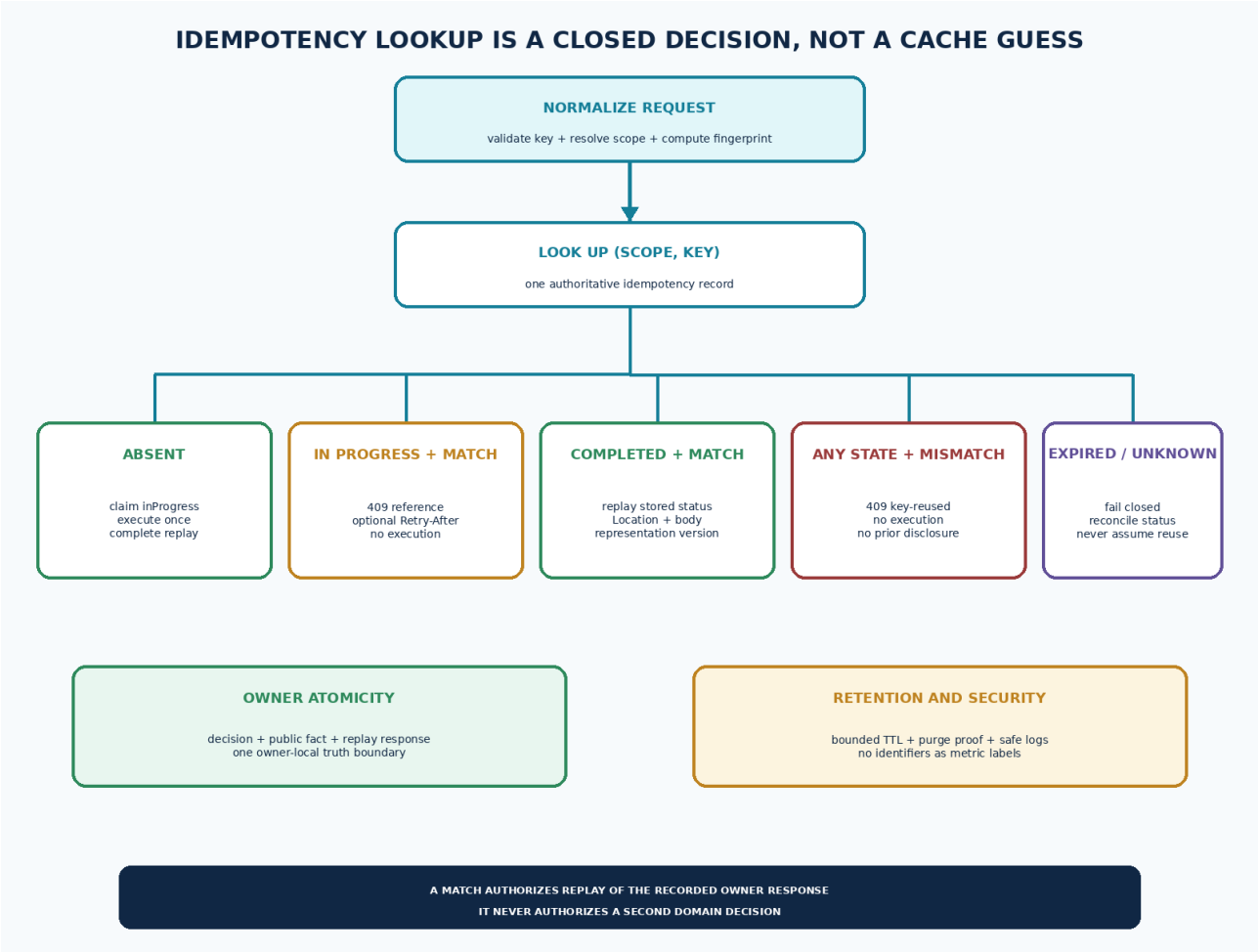


Figure 23.3 - Scope, key, fingerprint and state produce a closed execute, wait, replay, mismatch or expiry decision.

Record	Fingerprint	Server action	Domain execution
Absent	Current	Atomically claim inProgress; this request owns execution.	Exactly one claimant.
inProgress	Match	Return project-defined in-progress problem; optional Retry-After.	Zero for duplicate.
completed	Match	Re-authorize and replay stored contract.	Zero for duplicate.
terminal safe failure	Match	Replay only if policy declares immutable/replayable.	Zero for duplicate.
any live state	Mismatch	409 key-reused without prior disclosure.	Zero.
expiredKnown/tombstone	Any	Refuse execution; direct to status/reconciliation.	Zero until resolved.

The server never holds an unbounded duplicate open, starts a second command, guesses that the claimant died or returns the eventual body before completion. A lease alone is insufficient: recovery must consult owner transaction/status evidence before reopening an abandoned claim. Uncertain records go to reconciliation rather than silent execution.

Owner-local storage and transaction shape

IMPLEMENTATION SHAPE

```
public sealed record IdempotencyRecord(
    string ScopeHash, string KeyHash, string Fingerprint,
    string FingerprintProfile, IdempotencyState State,
    DateTimeOffset CreatedAtUtc, DateTimeOffset ExpiresAtUtc,
    int? ResponseStatus, string? ResponseMediaType,
    string? Location, byte[]? ResponseBody,
    string? OwnerDecisionId, long Revision);

public enum IdempotencyState { InProgress, Completed, ExpiredKnown }
```

ScopeHash and KeyHash require a server-controlled keyed hash or equivalent threat-reviewed construction. Response headers are allowlisted: Date, Server, trace IDs, cookies, credentials and hop-specific fields are not replay payload. The record is a reference persistence candidate, not compiled code or permission to store secrets.

IMPLEMENTATION SHAPE

```
await ownerTransaction.ExecuteAsync(async ct =>
{
    var claim = await store.ClaimAsync(scopeHash, keyHash, fingerprint, ct);
    if (claim is not IdempotencyClaim.Owner owner)
        return MapDuplicate(claim);

    var accepted = await useCase.ExecuteAsync(command, ct);
    var replay = AcceptedReplayContractV1.From(accepted);
    await outbox.AppendAsync(accepted.PublicFact, ct);
    await store.CompleteAsync(owner, replay, ct);
    return replay;
});
```

The pseudocode states one invariant, not implementation proof: owner effect, public intent/outbox and completed replay share one owner-local recoverable boundary. A physically separate store needs equivalent failure/recovery evidence; a best-effort second write is insufficient.

Failure, retention and cleanup

Failure point	Truth	Recovery rule	Forbidden inference
Reject before claim	No record or owner command.	Stable auth/media/validation problem.	Assume an effect.
Claim; command not started	InProgress claim; no effect.	Prove no TX-A before reopen/expiry.	Delete on timeout and run duplicate.
Domain rejects before TX-A	No accepted decision/fact.	Replay immutable rejection only if policy says so.	Persist transient exception forever.
TX-A rolls back	No decision/fact/completed replay.	Rollback claim or reconcile.	Return stored 202.
TX-A commits; response lost	Decision/fact/outbox/replay completed.	Matching retry replays exact 202.	Run command again.
Cleanup removes payload	Tombstone/status identity remains.	Refuse reuse or reconcile.	Treat key as new.
Restore loses recent record	Possible gap against owner data.	Reconcile before traffic.	Rely on key uniqueness alone.

RETENTION IS CORRECTNESS

TTL, partitioning, backup, restore, purge and tombstone policy define how long a client can safely retry and whether a restored service can distinguish a lost response from a new command.

BFF propagation and retry ownership

Concern	BFF rule	AlarmManagement rule	Invariant
Key origin	Accept/create once according to UI route policy.	Validate provider profile and scope.	One UI intent reuses one provider key.

Concern	BFF rule	AlarmManagement rule	Invariant
Propagation	Forward one allowlisted typed header.	Never trust body identity/scope fields.	No raw browser-header passthrough.
Automatic retry	Disabled until owner evidence accepts it.	Matching completion can replay without execution.	Resilience never creates second effect.
Lost response	Preserve ambiguity; exact retry only under policy.	Replay or explicit in-progress/mismatch/expiry.	BFF never invents 202.
Problem mapping	Translate documented provider types only.	Own provider catalogue/diagnostics.	No blind detail/instance/header copy.
RootCause	Not called in command retry path.	Publishes AlarmManagement acceptance only.	case-rc-3107 never enters replay.

Security, privacy, caching and observability

Concern	Control	Evidence
Cross-actor replay	Verified actor/tenant/site/capability scope; re-authorize before disclosure.	Lateral replay matrix.
Low-entropy keys	Entropy/length limits, rate limits and quotas.	Abuse/load fixtures.
Key/fingerprint leakage	Hash/pseudonymize; no raw values or metric labels.	Database/log/output scans.
Stored response disclosure	Protect by classification; minimize retention; current authorization.	Revocation fixtures.
Cache confusion	Success and idempotency problems use no-store.	Proxy/history tests.
Existence/timing oracle	Bounded safe bodies and reviewed timing.	Status/size/latency/log comparisons.
DoS via claims	Per-scope quota, max inProgress, size/TTL/backpressure.	Saturation tests.
High-cardinality telemetry	Labels only route/version/outcome/state.	Cardinality budget.

Metrics may count claim, replay, mismatch, in-progress and cleanup outcomes by bounded route/version labels. decisionId, key, fingerprint, actor, site and alarm IDs are never metric labels. Traces may link original and replay attempts without pretending they are one span or one HTTP request.

Reference test matrix

Fixture	Expected result	Blocking assertion
First valid v1 command	202 with decision/fact/Location.	One use-case execution; TX-A includes replay completion.
Same key + identical/equal JSON	Replay original 202.	Zero new command/outbox/domain effect.
Same key + changed command	409 key-reused.	No prior disclosure or execution.
Two simultaneous matches	One owner; in-progress/replay duplicate.	Race proves one effect.
TX-A committed; socket fails	Retry replays committed 202.	Fault injection leaves one effect.
Crash after claim	Reconcile before reopen.	No lease-expiry duplicate.
Unsupported request/response media	415 or 406 stable problem.	No claim/domain execution.
Missing/invalid key	400 required/invalid.	No raw key echo or execution.
Cross-actor/tenant/site key	Independent/denied safe scope.	No disclosure across scope.
Expired payload+tombstone	409 reconciliation.	No silent new execution.
Deprecated v1	Same behavior plus correct lifecycle headers.	Syntax/date/link verified.
Old/new clients	Accepted additive changes usable.	Chapter 24 matrix.
Safe-output scan	No stack, SQL, token, raw key/body/private IDs.	Response/log/trace/metric allowlists.

Evidence catalogue

Evidence ID	Acceptance statement	Minimum artifact
VER-NEG-001	Routes deterministically select/reject request/response versions.	Accept/Content-Type/default/q-value matrix.
VER-MEDIA-001	Success/problem media, Content-Type and Vary match contract.	Wire/OpenAPI/cache snapshots.
VER-DEPR-001	Deprecation/Link/Sunset lifecycle is correct.	Migration, consumer inventory and rollback.
PROB-CAT-001	Every expected failure maps to stable type/status/code.	Catalogue/schema/exhaustive mapping.
PROB-SAFE-001	Problems expose no forbidden data.	Output/oracle scans.
IDEM-KEY-001	Key profile rejects invalid headers pre-execution.	Parser/endpoint negatives.
IDEM-SCOPE-001	No replay across provider/route/actor/tenant/site.	Lateral authorization matrix.
IDEM-FP-001	Canonicalization matches equals and rejects differences.	Golden/property/fuzz corpus.
IDEM-TXN-001	Effect/outbox/replay share atomic recoverable boundary.	Transaction fault matrix.
IDEM-REPLAY-001	Matching retry returns original contract with zero new effect.	Lost-response fixture/counts.
IDEM-CONC-001	Concurrent duplicates elect one execution owner.	Barrier/multi-instance stress.
IDEM-EXP-001	TTL/cleanup/tombstone/restore preserve safe window.	Clock/restore/reconciliation.
IDEM-OBS-001	Telemetry is bounded and free of leakage.	Cardinality/redaction/trace graph.
VER-COMPAT-001	Old/new producers and clients preserve additive compatibility.	Chapter 24 matrix.
VER-REVIEW-001	Owners accept lifecycle and residual risks.	Signed evidence index.

ADR-021 - Explicit contract evolution and owner-scoped command replay

ADR field	ADR-021 record
Status	Accepted for the reference design; negotiation, catalogue, persistence, transaction, cleanup and runtime evidence remain pending.
Context	Clients must survive additive evolution, classify failures and recover when a POST commits but its response is lost. Blind retry can duplicate the effect.
Decision	Use URI major routes, negotiated versioned media, stable RFC 9457 types and an owner-scoped Idempotency-Key. Bind a canonical fingerprint to one scope/key record and atomically commit decision, public intent/outbox and replay response. Matching completion replays; concurrent, mismatched and expired-known requests never execute.
Consequences	Deterministic negotiation and recovery require persistent technical records, canonicalization, transaction coupling, cleanup/backup, security and compatibility evidence.
Rejected	Deployment as API version; silent latest default; prose errors; raw downstream problems; key in body; key as business ID; raw-byte fingerprint; in-memory dedupe; shared cross-service store; lease expiry authorizing duplicate; BFF inventing acceptance; RootCause in replay.
Review trigger	New major/media/problem, default/state change, unsafe operation, scope/fingerprint/retention/storage change, replay disclosure, retirement or a future standards-track Idempotency-Key publication.

Risk register

Risk	Early signal	Blocking control
Version means deployment	Clients branch on build/default.	VER-NEG/MEDIA wire fixtures.
Semantic break hidden as additive	Old client misreads state/unit/null/time.	Classifier + Chapter 24 corpus.
Problem instability/leakage	Clients parse prose or output contains secrets.	PROB-CAT/SAFE documentation and scans.
Duplicate effect	Two decisions/facts under timeout/race.	IDEM-TXN/REPLAY/CONC gates.
Fingerprint false match/mismatch	Changed request replays or equal retry rejects.	Versioned golden/property/fuzz corpus.
Cross-scope disclosure	Same key reveals another response.	Server scope + current authorization.
Lease expiry duplicates commit	Stale claim reopens without reconciliation.	Owner transaction/status recovery.
Cleanup destroys correctness	Retry window exceeds payload/tombstone/backup.	IDEM-EXP capacity/restore gates.

Risk	Early signal	Blocking control
RootCause overclaim	Replay contains case-rc-3107/completion.	Golden body limited to TX-A truth.

Official-source validation

The HTTP, problem, deprecation, sunset and idempotency terminology was checked against current primary sources on 19 July 2026. RFC 9110 defines HTTP representation selection, media/method/status semantics. RFC 9457 defines application/problem+json, type/title/status/detail/instance, extensions and security considerations. RFC 9745 standardizes Deprecation and the deprecation link relation. RFC 8594 defines Sunset as an informational future-unresponsiveness signal.

The IETF HTTPAPI document The Idempotency-Key HTTP Header Field, revision 07, expired on 18 April 2026 and is archived as an inactive Internet-Draft. It discusses a request header, uniqueness, fingerprints, expiry and duplicate scenarios, but is not an RFC. Exact NEXUS-1 header profile, 409 branches, scope, fingerprint, storage, retention, replay and recovery remain project decisions.

Primary source	Stable point	NEXUS-1 decision
RFC 9110	Negotiation, media, idempotent method and statuses.	URI major + vendor media + explicit 406/415.
RFC 9457	Machine-readable problem model and security review.	Stable safe type catalogue.
RFC 9745	Deprecation informs without changing behavior.	Lifecycle headers only under migration plan.
RFC 8594	Sunset signals expected future unresponsiveness.	Sunset follows deprecation approval/date.
Expired IETF draft-ietf-httpapi-idempotency-key-header-07	Proposed key/fingerprint/expiry/duplicate terminology.	Design input only; project behavior explicitly tested.

Sources: <https://www.rfc-editor.org/rfc/rfc9110.html> ; <https://www.rfc-editor.org/rfc/rfc9457.html> ; <https://www.rfc-editor.org/rfc/rfc9745.html> ; <https://www.rfc-editor.org/rfc/rfc8594.html> ; <https://datatracker.ietf.org/doc/draft-ietf-httpapi-idempotency-key-header/>

Technical material still required

The version axes, route/media profile, negotiation algorithm, compatibility rules, lifecycle, Problem Details catalogue, key/scope/fingerprint/state/retention contract, lost-response walkthrough, owner transaction, concurrency/expiry/recovery, BFF propagation, security/observability, tests, evidence IDs and ADR are specified. No NEXUS-1 Edge/BFF/provider project has compiled or executed them.

TECHNICAL MATERIAL STATUS - [VERSION NEGOTIATION, PROBLEM DETAILS CATALOGUE, IDEMPOTENCY STORE/REPLAY IMPLEMENTATION AND RUNTIME EVIDENCE REQUIRED]

Implement the net8.0 negotiator, OpenAPI media declarations, RFC 9457 catalogue/writer, lifecycle headers, key validator, versioned canonical fingerprint library, authoritative idempotency store, unique claims, TX-A coupling to decision/public fact/outbox/replay, immutable replay writer, in-progress/mismatch/expiry/tombstone/reconciliation state machine, cleanup/backup/restore, BFF propagation and retry ownership, authorization guard, no-store/redaction and bounded telemetry; execute media/problem, canonicalization, simultaneous duplicate, response-loss, crash/clock, rollback/commit, recovery, cleanup/restore, cross-scope/oracle, capacity, safe-output and old/new client matrices; publish VER-NEG-001 through VER-REVIEW-001 linked to prior evidence and Chapter 24. RootCause remains G0 accepted and G1 specified; runtime evidence has not advanced.

Deliverable	Minimum content	Evidence
Negotiation/lifecycle	Selectors, defaults, 406/415, Vary, OpenAPI, Deprecation/Link/Sunset.	VER-NEG/MEDIA/DEPR-001
Problem catalogue	Type registry, schemas, writer/mapping, safe extensions/output.	PROB-CAT/SAFE-001
Key/scope/fingerprint	Validator, entropy/rate limits, server scope and canonicalizer.	IDEM-KEY/SCOPE/FP-001

CHAPTER 23 - CODE AUDIT

Request and response representations are selected independently

The route major identifies the operation family. Content-Type selects the request schema, Accept selects the success representation, and Problem Details keeps a separate failure media contract.

EVIDENCE STATUS

These listings are compile-oriented reference material. No negotiator, OpenAPI document, provider store, transaction race, cleanup job or compatibility corpus has executed here. VER and IDEM evidence remains PENDING.

EXECUTABLE ASSET 23-A - PARSED MEDIA BOUNDARY AND REGISTRY

```
public sealed record ParsedMediaRange(
    string Type,
    int? Version,
    double Quality);

public interface IStrictMediaHeaderParser
{
    ParsedMediaRange ParseContentType(string value);
    IReadOnlyList<ParsedMediaRange> ParseAccept(
        IReadOnlyList<string> values);
}

public sealed record RepresentationSelection(
    string RequestMedia,
    string ResponseMedia,
    int ContractVersion);

public static class AlarmFloodMediaV1
{
    public const string Request =
        "application/vnd.nexus1.alarm-flood-detection-request+json";
    public const string Response =
        "application/vnd.nexus1.alarm-flood-accepted+json";
    public const int Version = 1;
}
```

EXECUTABLE ASSET 23-B - DETERMINISTIC NEGOTIATION

```
public sealed class AlarmFloodRepresentationNegotiator(
    IStrictMediaHeaderParser parser)
{
    public RepresentationSelection Select(
        string contentType,
        IReadOnlyList<string> acceptValues)
    {
        var request = parser.ParseContentType(contentType);
        if (!StringComparer.OrdinalIgnoreCase.Equals(
            request.Type, AlarmFloodMediaV1.Request)
            || request.Version != AlarmFloodMediaV1.Version)
            throw new UnsupportedRequestRepresentation();

        var accepted = parser.ParseAccept(acceptValues);
        if (accepted.Count == 0)
            return SelectedV1();

        var match = accepted
            .Where(MatchesResponseV1)
            .OrderByDescending(Specificity)
            .ThenByDescending(x => x.Quality)
            .FirstOrDefault();
        return match is null || match.Quality <= 0
            ? throw new NoAcceptableRepresentation()
            : SelectedV1();
    }

    private static bool MatchesResponseV1(ParsedMediaRange value) =>
        value.Type == "*/*"
        || StringComparer.OrdinalIgnoreCase.Equals(
            value.Type, "application/*")
        || (StringComparer.OrdinalIgnoreCase.Equals(
            value.Type, AlarmFloodMediaV1.Response)
            && value.Version == AlarmFloodMediaV1.Version);

    private static int Specificity(ParsedMediaRange value) =>
        value.Type == "*/*" ? 0
        : value.Type.EndsWith("/*", StringComparison.Ordinal) ? 1
        : 2;

    private static RepresentationSelection SelectedV1() =>
        new(AlarmFloodMediaV1.Request,
            AlarmFloodMediaV1.Response,
            AlarmFloodMediaV1.Version);
}

[Theory]
[MemberData(nameof(NegotiationMatrix))]
public void VER_NEG_001_matrix_is_closed(NegotiationCase test) =>
    fixture.Negotiate(test).Should().Be(test.Expected);
```

STABLE RFC 9457 PROBLEM CATALOGUE

Clients branch on immutable type and code, never exception prose

EXECUTABLE ASSET 23-C - CLOSED PROBLEM REGISTRY

```
public sealed record ProblemDescriptor(
    string Type,
    int Status,
    string Title,
    string Code);

public static class PublicProblemCatalogue
{
    public static readonly ProblemDescriptor UnsupportedMedia =
        new("/problems/unsupported-media-type", 415,
            "The request representation is unsupported",
            "UNSUPPORTED_MEDIA_TYPE");
    public static readonly ProblemDescriptor NotAcceptable =
        new("/problems/representation-not-acceptable", 406,
            "No acceptable response representation exists",
            "REPRESENTATION_NOT_ACCEPTABLE");
    public static readonly ProblemDescriptor KeyRequired =
        new("/problems/idempotency/key-required", 400,
            "An idempotency key is required",
            "IDEMPOTENCY_KEY_REQUIRED");
    public static readonly ProblemDescriptor KeyInvalid =
        new("/problems/idempotency/key-invalid", 400,
            "The idempotency key is invalid",
            "IDEMPOTENCY_KEY_INVALID");
    public static readonly ProblemDescriptor KeyReused =
        new("/problems/idempotency/key-reused", 409,
            "The idempotency key cannot be reused",
            "IDEMPOTENCY_KEY_REUSED");
    public static readonly ProblemDescriptor InProgress =
        new("/problems/idempotency/request-in-progress", 409,
            "The original request is still unresolved",
            "IDEMPOTENCY_REQUEST_IN_PROGRESS");
    public static readonly ProblemDescriptor Expired =
        new("/problems/idempotency/record-expired", 409,
            "The original replay window has expired",
            "IDEMPOTENCY_RECORD_EXPIRED");
}
```

EXECUTABLE ASSET 23-D - ONE SAFE WRITER AND EXHAUSTIVE MAPPING

```
public sealed class PublicProblemWriter(
    IProblemDetailsService service)
{
    public async ValueTask WriteAsync(
        HttpContext http,
        ProblemDescriptor descriptor,
        IReadOnlyDictionary<string, object?> safeExtensions)
    {
        http.Response.StatusCode = descriptor.Status;
        http.Response.Headers.CacheControl = "no-store";
        var problem = new ProblemDetails
        {
            Type = "https://problems.nexus1.example" + descriptor.Type,
            Title = descriptor.Title,
            Status = descriptor.Status,
            Instance = http.TraceIdentifier is { Length: > 0 } id
                ? "urn:nexus1:problem:" + occurrence.Hash(id)
                : null
        };
        problem.Extensions["errorCode"] = descriptor.Code;
        foreach (var extension in safeExtensions)
            problem.Extensions.Add(extension.Key, extension.Value);
        await service.WriteAsync(new ProblemDetailsContext
        {
            HttpContext = http,
            ProblemDetails = problem
        });
    }
}

[Theory]
[MemberData(nameof(AllPublicFailures))]
public async Task PROB_CAT_001_mapping_is_exhaustive(FailureCase test)
{
    var response = await fixture.ExecuteAsync(test);
    var problem = await response.ReadProblemAsync();
    problem.Status.Should().Be((int)response.StatusCode);
    problem.Type.Should().Be(test.Type);
    problem.Extensions["errorCode"].Should().Be(test.Code);
    response.RawBody.Should().NotContain(fixture.SecretCanary);
}
```

PROBLEM IDENTITY IS A CATALOGUE DECISION

Status, type and errorCode stay stable. Titles may be localized; exception text, raw keys, fingerprints, stored responses and protected owner IDs never appear.

PRODUCTION KEY PROFILE AND SERVER SCOPE

A key proves encoded capacity, while the client owns randomness

EXECUTABLE ASSET 23-E - EXACTLY ONE 128-BIT KEY VALUE

```
public readonly record struct IdempotencyKey(string Value)
{
    public static IdempotencyKey Parse(IHeaderDictionary headers)
    {
        var values = headers["Idempotency-Key"];
        if (values.Count == 0)
            throw new IdempotencyKeyRequired();
        if (values.Count != 1)
            throw new IdempotencyKeyInvalid();

        var value = values[0]!;
        if (!value.StartsWith("idem-", StringComparison.Ordinal)
            || value.Length != 27)
            throw new IdempotencyKeyInvalid();
        try
        {
            var decoded = WebEncoders.Base64UrlDecode(value[5..]);
            if (decoded.Length != 16)
                throw new IdempotencyKeyInvalid();
        }
        catch (FormatException)
        {
            throw new IdempotencyKeyInvalid();
        }
        return new(value);
    }
}

[Theory]
[InlineData("")]
[InlineData("idem-short")]
[InlineData("idem-AAECAwQFBgcICQoLDA00Dw==")]
[InlineData("idem-AAECAwQFBgcICQoLDA00D!")]
public void IDEM_KEY_001_invalid_profiles_stop_before_claim(string key)
{
    var act = () => fixture.ParseKey(key);
    act.Should().Throw<IdempotencyKeyException>();
    fixture.StoreCalls.Should().Be(0);
}
```

EXECUTABLE ASSET 23-F - SERVER-DERIVED SCOPE AND KEYED IDENTIFIERS

```
public sealed record IdempotencyScope(
    string Provider,
    string RouteMajor,
    string Capability,
    string Subject,
    string Tenant,
    string Site);

public sealed record ProtectedIdempotencyIdentity(
    byte[] ScopeHash,
    byte[] KeyHash);

public sealed class IdempotencyIdentityProtector(
    IKeyedHashKeyProvider keys,
    ILengthPrefixedUtf8 codec)
{
    public ProtectedIdempotencyIdentity Protect(
        IdempotencyScope scope,
        IdempotencyKey key)
    {
        var scopeBytes = codec.Encode(
            scope.Provider, scope.RouteMajor, scope.Capability,
            scope.Subject, scope.Tenant, scope.Site);
        var keyBytes = Encoding.ASCII.GetBytes(key.Value);
        using var hmac = new HMACSHA256(
            keys.CurrentIdempotencyIndexKey());
        return new(
            hmac.ComputeHash(scopeBytes),
            hmac.ComputeHash(keyBytes));
    }
}

[Fact]
public void IDEM_SCOPE_001_body_fields_cannot_change_scope()
{
    var first = fixture.ResolveIdentity(bodyTenant: "tenant-a");
    var hostile = fixture.ResolveIdentity(bodyTenant: "tenant-b");
    hostile.Should().BeEquivalentTo(first);
    fixture.Scope.Subject.Should().Be(fixture.VerifiedSubject);
}
```

STRUCTURE CANNOT PROVE RANDOM GENERATION

The server proves that the suffix encodes 128 bits and enforces abuse controls. Clients must generate those bits with a cryptographically secure random source. Short manuscript keys are illustrative aliases, not production values.

VERSIONED SEMANTIC FINGERPRINT

Length-prefixed typed fields remove JSON formatting ambiguity

EXECUTABLE ASSET 23-G - CANONICAL V1 MATERIAL

```
public sealed record AlarmFloodFingerprintMaterialV1(
    string Contract,
    string Method,
    string Target,
    string SiteId,
    DateTimeOffset WindowStartedAtUtc,
    DateTimeOffset WindowEndedAtUtc,
    IReadOnlyList<string> AlarmIds,
    string RequestedSeverity,
    string ClientRequestId);

public sealed class AlarmFloodFingerprintV1
{
    public const string Profile =
        "alarm-flood-detection-request.v1/len32-utf8/sha256";

    public byte[] Compute(ValidatedDetectAlarmFloodV1 request)
    {
        using var writer = new LengthPrefixedHashWriter(
            HashAlgorithmName.SHA256);
        writer.Add("alarm-flood-detection-request.v1");
        writer.Add("POST");
        writer.Add("/api/v1/alarm-flood-detections");
        writer.Add(request.SiteId.Canonical);
        writer.Add(request.Window.StartUtc.ToUniversalTime()
            .ToString("0", CultureInfo.InvariantCulture));
        writer.Add(request.Window.EndUtc.ToUniversalTime()
            .ToString("0", CultureInfo.InvariantCulture));
        foreach (var alarm in request.AlarmIds
            .Select(x => x.Canonical)
            .OrderBy(x => x, StringComparer.Ordinal))
            writer.Add(alarm);
        writer.Add(request.Severity.Canonical);
        writer.Add(request.ClientRequestId.Value);
        return writer.Finish();
    }
}
```

EXECUTABLE ASSET 23-H - UNAMBIGUOUS HASH WRITER AND GOLDEN GATES

```
public sealed class LengthPrefixedHashWriter(
    HashAlgorithmName algorithm) : IDisposable
{
    private readonly IncrementalHash hash =
        IncrementalHash.CreateHash(algorithm);

    public void Add(string value)
    {
        var bytes = Encoding.UTF8.GetBytes(value);
        Span<byte> length = stackalloc byte[4];
        BinaryPrimitives.WriteInt32BigEndian(length, bytes.Length);
        hash.AppendData(length);
        hash.AppendData(bytes);
    }

    public byte[] Finish() => hash.GetHashAndReset();
    public void Dispose() => hash.Dispose();
}

[Fact]
public void IDEM_FP_001_equivalent_v1_requests_match_golden_digest()
{
    var first = fixture.ValidatedRequest(
        propertyOrder: "normal",
        timestampOffset: TimeSpan.Zero,
        alarmOrder: "forward");
    var equivalent = fixture.ValidatedRequest(
        propertyOrder: "reversed",
        timestampOffset: TimeSpan.FromHours(2),
        alarmOrder: "reverse");
    fixture.Fingerprint.Compute(first).Should()
        .Equal(fixture.Fingerprint.Compute(equivalent));
    fixture.Fingerprint.Compute(first).Should()
        .Equal(fixture.GoldenFingerprintV1);
}

[Fact]
public void meaningful_change_never_matches()
{
    fixture.Fingerprint.Compute(fixture.ValidatedRequest("critical"))
        .Should().NotEqual(
            fixture.Fingerprint.Compute(
                fixture.ValidatedRequest("warning")));
}
```

CANONICALIZATION FOLLOWS CONTRACT MEANING

Alarm IDs are sorted only because v1 declares them a set. Times use accepted UTC precision. The profile name freezes every normalization and hash decision.

AUTHORITATIVE OWNER-LOCAL STORE

The unique scope/key row retains exact replay bytes

EXECUTABLE ASSET 23-I - PERSISTENCE RECORD AND STATE

```
public enum IdempotencyState
{
    InProgress,
    Completed,
    ExpiredKnown
}

public sealed class IdempotencyRecordRow
{
    public long Id { get; private set; }
    public required byte[] ScopeHash { get; init; }
    public required byte[] KeyHash { get; init; }
    public required string FingerprintProfile { get; init; }
    public required byte[] Fingerprint { get; init; }
    public IdempotencyState State { get; private set; }
    public DateTimeOffset CreatedAtUtc { get; init; }
    public DateTimeOffset ExpiresAtUtc { get; private set; }
    public int? ResponseStatus { get; private set; }
    public string? ResponseMediaType { get; private set; }
    public string? Location { get; private set; }
    public byte[]? ResponseBody { get; private set; }
    public string? OwnerDecisionId { get; private set; }
    public byte[] Version { get; private set; } = [];

    public void Complete(
        AcceptedReplayContractV1 replay,
        DateTimeOffset expiresAtUtc)
    {
        if (State != IdempotencyState.InProgress)
            throw new InvalidIdempotencyTransition();
        State = IdempotencyState.Completed;
        ResponseStatus = replay.Status;
        ResponseMediaType = replay.MediaType;
        Location = replay.Location;
        ResponseBody = replay.ExactBodyBytes.ToArray();
        OwnerDecisionId = replay.DecisionId;
        ExpiresAtUtc = expiresAtUtc;
    }
}
```

EXECUTABLE ASSET 23-J - EF CORE MAPPING AND INVARIANTS

```
public sealed class IdempotencyRecordConfiguration :
    IEntityTypeConfiguration<IdempotencyRecordRow>
{
    public void Configure(
        EntityTypeBuilder<IdempotencyRecordRow> builder)
    {
        builder.ToTable("IdempotencyRecords", "AlarmManagement");
        builder.HasKey(x => x.Id);
        builder.Property(x => x.ScopeHash).HasMaxLength(32)
            .IsRequired();
        builder.Property(x => x.KeyHash).HasMaxLength(32)
            .IsRequired();
        builder.HasIndex(x => new { x.ScopeHash, x.KeyHash })
            .IsUnique();
        builder.Property(x => x.Fingerprint).HasMaxLength(32)
            .IsRequired();
        builder.Property(x => x.FingerprintProfile).HasMaxLength(100)
            .IsRequired();
        builder.Property(x => x.ResponseMediaType).HasMaxLength(160);
        builder.Property(x => x.Location).HasMaxLength(500);
        builder.Property(x => x.ResponseBody)
            .HasMaxLength(32 * 1024);
        builder.Property(x => x.Version).IsRowVersion();
    }
}

[Fact]
public async Task completed_row_requires_effect_fact_and_body()
{
    await fixture.ExecuteFirstRequestAsync();
    var row = await fixture.ReadIdempotencyRowAsync();
    row.State.Should().Be(IdempotencyState.Completed);
    row.ResponseBody.Should().Equal(fixture.AcceptedBodyBytes);
    fixture.Decisions.Should().ContainSingle();
    fixture.PendingPublicFacts.Should().ContainSingle();
}
```

THE RECORD IS TECHNICAL AUTHORITY, NOT A BUSINESS AGGREGATE

Raw keys and scopes are not stored. The unique protected identity elects one owner, while exact response bytes remain bounded, classified and access-controlled.

TX-A EXECUTION AND DUPLICATE RESOLUTION

Claim, owner effect, public fact and replay commit together

EXECUTABLE ASSET 23-K - ONE DATABASE TRANSACTION

```
public async Task<IdempotentExecutionResult> ExecuteOwnerAsync(
    PreparedIdempotentCommand prepared,
    CancellationToken cancellationToken)
{
    await using var transaction = await db.Database
        .BeginTransactionAsync(cancellationToken);

    var row = IdempotencyRecordRow.Begin(
        prepared.Identity,
        prepared.FingerprintProfile,
        prepared.Fingerprint,
        clock.UtcNow());
    db.IdempotencyRecords.Add(row);
    await db.SaveChangesAsync(cancellationToken);

    var accepted = await useCase.ExecuteAsync(
        prepared.Command, cancellationToken);
    db.PendingPublicFacts.Add(
        publicFacts.From(accepted.PublicFact));
    var replay = replayCodec.Encode(accepted);
    row.Complete(
        replay,
        clock.UtcNow() + options.ReplayWindow);
    await db.SaveChangesAsync(cancellationToken);
    await transaction.CommitAsync(cancellationToken);
    return IdempotentExecutionResult.Executed(replay);
}
```

EXECUTABLE ASSET 23-L - FRESH-SCOPE COLLISION PATH

```
public async Task<IdempotentExecutionResult> ExecuteAsync(
    PreparedIdempotentCommand prepared,
    CancellationToken cancellationToken)
{
    try
    {
        await using var attempt = scopes.CreateAsyncScope();
        var owner = attempt.ServiceProvider
            .GetRequiredService<IdempotentOwnerExecution>();
        return await owner.ExecuteOwnerAsync(
            prepared, cancellationToken);
    }
    catch (DbUpdateException exception)
        when (conflicts.IsScopeKeyUniqueViolation(exception))
    {
        // The failed DbContext and transaction are already disposed.
    }

    await using var reload = scopes.CreateAsyncScope();
    var duplicates = reload.ServiceProvider
        .GetRequiredService<IdempotencyDuplicateResolver>();
    return await duplicates.ResolveAsync(
        prepared, cancellationToken);
}

public IdempotentExecutionResult Resolve(
    PreparedIdempotentCommand request,
    IdempotencyRecordRow stored)
{
    if (!CryptographicOperations.FixedTimeEquals(
        request.Fingerprint, stored.Fingerprint)
        || request.FingerprintProfile != stored.FingerprintProfile)
        return IdempotentExecutionResult.KeyReused();
    return stored.State switch
    {
        IdempotencyState.Completed =>
            IdempotentExecutionResult.Replay(stored),
        IdempotencyState.InProgress =>
            IdempotentExecutionResult.InProgress(),
        IdempotencyState.ExpiredKnown =>
            IdempotentExecutionResult.Expired(),
        _ => throw new InvalidIdempotencyState()
    };
}
```

A TIMEOUT DOES NOT REOPEN OWNERSHIP

The database unique key serializes competing claims. A failed context is never reused; the duplicate reloads durable state in a fresh scope and never guesses that the first owner died.

EXACT REPLAY AND CURRENT AUTHORIZATION

A matching retry receives recorded TX-A truth without new effects

EXECUTABLE ASSET 23-M - REPLAY WRITER

```
public sealed record AcceptedReplayContractV1(
    int Status,
    string MediaType,
    string Location,
    string DecisionId,
    byte[] ExactBodyBytes);

public static async Task WriteReplayAsync(
    HttpContext http,
    AcceptedReplayContractV1 replay,
    CancellationToken cancellationToken)
{
    http.Response.StatusCode = replay.Status;
    http.Response.ContentType = replay.MediaType;
    http.Response.Headers.Location = replay.Location;
    http.Response.Headers.CacheControl = "no-store";
    http.Response.Headers["Idempotency-Replayed"] = "true";
    http.Response.ContentLength = replay.ExactBodyBytes.Length;
    await http.Response.Body.WriteAsync(
        replay.ExactBodyBytes, cancellationToken);
}

public async Task<IResult> ReplayAuthorizedAsync(
    ClaimsPrincipal principal,
    IdempotencyRecordRow stored,
    CancellationToken cancellationToken)
{
    var authorized = await authorization.AuthorizeAsync(
        principal,
        new AlarmDecisionResource(stored.OwnerDecisionId!),
        PolicyNames.DetectAlarmFlood);
    if (!authorized.Succeeded)
        return TypedResults.NotFound();
    return new ExactReplayResult(replayCodec.Read(stored));
}
```

EXECUTABLE ASSET 23-N - LOST-RESPONSE RECOVERY GATE

```
[Fact]
public async Task IDEM_REPLAY_001_lost_202_replays_exact_owner_contract()
{
    fixture.ResponseGate.FailAfterTransactionCommit();
    var first = () => fixture.PostAsync(
        fixture.ProductionProfileKey,
        fixture.ValidRequestBytes);
    await first.Should().ThrowAsync<HttpRequestException>();

    var replay = await fixture.PostAsync(
        fixture.ProductionProfileKey,
        fixture.EquivalentRequestBytes);
    replay.StatusCode.Should().Be(HttpStatusCode.Accepted);
    replay.Headers.GetValues("Idempotency-Replayed")
        .Should().ContainSingle("true");
    replay.Headers.Location.Should().Be(fixture.OriginalLocation);
    (await replay.Content.ReadAsByteArrayAsync())
        .Should().Equal(fixture.OriginalAcceptedBodyBytes);
    fixture.DomainExecutions.Should().Be(1);
    fixture.Decisions.Should().ContainSingle();
    fixture.PendingPublicFacts.Should().ContainSingle();
}

[Fact]
public async Task changed_request_returns_oracle_safe_409()
{
    await fixture.PostFirstAsync();
    var response = await fixture.PostChangedRequestAsync();
    response.StatusCode.Should().Be(HttpStatusCode.Conflict);
    response.RawBody.Should().NotContain(fixture.StoredBodyCanary);
    response.RawBody.Should().NotContain(fixture.FingerprintCanary);
    fixture.DomainExecutions.Should().Be(1);
}
```

REPLAY IS HISTORICAL RECOVERY, NOT A FRESH QUERY

Current authorization guards disclosure, but the body, media and Location come from the completed record. RootCause state and present-day remapping are excluded.

CONCURRENCY, EXPIRY, RESTORE AND ACCEPTANCE

Retention keeps the key identity after replay bytes are purged

EXECUTABLE ASSET 23-O - RACE AND EXPIRY GATES

```
[Fact]
public async Task IDEM_CONC_001_simultaneous_matches_create_one_effect()
{
    var results = await fixture.ReleaseConcurrentPostsAsync(
        count: 32,
        key: fixture.ProductionProfileKey,
        body: fixture.ValidRequestBytes);
    results.Count(x => x.Executed).Should().Be(1);
    results.Count(x => x.Replayed || x.InProgress).Should().Be(31);
    fixture.DomainExecutions.Should().Be(1);
    fixture.PendingPublicFacts.Should().ContainSingle();
}

[Fact]
public async Task IDEM_EXP_001_cleanup_keeps_expired_known_identity()
{
    await fixture.ExecuteFirstRequestAsync();
    fixture.Clock.Advance(fixture.ReplayWindow);
    await fixture.CleanupReplayPayloadAsync();

    var row = await fixture.ReadIdempotencyRowAsync();
    row.State.Should().Be(IdempotencyState.ExpiredKnown);
    row.ResponseBody.Should().BeNull();
    var retry = await fixture.PostSameRequestAsync();
    retry.StatusCode.Should().Be(HttpStatusCode.Conflict);
    retry.ProblemType.Should().EndWith(
        "/idempotency/record-expired");
    fixture.DomainExecutions.Should().Be(1);
}

[Fact]
public async Task restore_reconciles_owner_truth_before_traffic()
{
    await fixture.RestoreWithMissingRecentReplayRowAsync();
    await fixture.StartReconciliationAsync();
    fixture.Readiness.Should().BeFalse();
    await fixture.RebuildExpiredKnownIdentityFromOwnerEvidenceAsync();
    fixture.Readiness.Should().BeTrue();
    fixture.DomainExecutions.Should().Be(1);
}
```

EXECUTABLE ASSET 23-P - ACCEPTANCE EVIDENCE RECORD

```
{
  "evidenceId": "VER-REVIEW-001", "chapter": 23,
  "artifactCommit": null, "sdk": "net8.0",
  "negotiatorBuild": null, "problemCatalogueDigest": null,
  "provider": null, "migration": null,
  "fingerprintProfile": "alarm-flood-detection-request.v1/len32-utf8/sha256",
  "replayWindow": null, "backupRestoreRun": null,
  "tests": {
    "VER-NEG-001": "PENDING", "VER-MEDIA-001": "PENDING",
    "VER-DEPR-001": "PENDING", "PROB-CAT-001": "PENDING",
    "PROB-SAFE-001": "PENDING", "IDEM-KEY-001": "PENDING",
    "IDEM-SCOPE-001": "PENDING", "IDEM-FP-001": "PENDING",
    "IDEM-TXN-001": "PENDING", "IDEM-REPLAY-001": "PENDING",
    "IDEM-CONC-001": "PENDING", "IDEM-EXP-001": "PENDING",
    "IDEM-OBS-001": "PENDING", "VER-COMPAT-001": "PENDING"
  },
  "raceCorpusAccepted": false,
  "restoreEvidenceAccepted": false,
  "accepted": false
}
```

Chapter 23 code-audit result

The chapter now exposes one executable route from deterministic representation selection and stable problems to a protected key identity, versioned semantic fingerprint, atomic owner execution, exact replay and safe expiry.

CURRENT VERDICT

The replay boundary is implementable in shape, but no provider race, real key traffic, compatibility matrix, cleanup capacity or restore reconciliation has executed. All VER/IDEM evidence remains explicitly PENDING.

Deliverable	Minimum content	Evidence
Store/transaction	Unique claims, state/revision and TX-A effect/outbox/replay coupling.	IDEM-TXN/CONC-001
Replay/expiry	Immutable response, authorization, mismatch/in-progress/tombstone/restore.	IDEM-REPLAY/EXP-001
Operations/review	Capacity, abuse, latency, telemetry, runbooks and signed review.	IDEM-OBS/VER-REVIEW-001
Compatibility	Old/new server/generated-client requests/responses/problems/replay.	VER-COMPAT-001 / Chapter 24

Implementation sequence

1. Accept ADR-021 and publish route/media/problem/lifecycle catalogues with owners and review triggers.
2. Implement the net8.0 negotiator/OpenAPI declarations and freeze 406/415/default/Vary wire fixtures.
3. Implement one RFC 9457 writer and exhaustive safe mapping for expected/unexpected failures.
4. Implement the key validator and versioned canonical fingerprint library; execute golden/property/fuzz corpora.
5. Create the AlarmManagement store with uniqueness, revision, hashed identifiers, payload protection, quotas and indexes.
6. Integrate claim/completion with TX-A decision/public-fact/outbox; prove rollback/commit/response-loss points.
7. Implement replay, in-progress, mismatch, terminal and expired-known/tombstone outcomes with current authorization.
8. Implement cleanup, backup/restore reconciliation, capacity alarms and crash/clock recovery without lease duplicates.
9. Enable BFF propagation and bounded unsafe retry only after provider replay evidence; keep RootCause outside the path.
10. Execute security/load/output/old-new matrices and publish evidence for Chapter 24.

What this chapter has proved

Chapter 23 has specified how NEXUS-1 identifies/evolves public and BFF contracts, distinguishes request/response media failures, retires versions without silent behavior change and preserves RFC 9457 machine-readable failure meaning. It has defined the complete owner-scoped idempotency contract from validation/fingerprint through unique claim, TX-A completion, lost-response replay, concurrency, mismatch, retention, tombstone, recovery, BFF propagation, security and observability.

It has not compiled a negotiator, served a vendor media type, emitted a real problem, computed a production fingerprint, created a store, won a database race, committed replay with TX-A, lost a socket, replayed a response, purged a tombstone, restored a backup, measured capacity or executed an old/new client. The design is specified, not implemented or evidenced. RootCause remains G0 accepted and G1 specified.

RETRY THE EXCHANGE, NOT THE DECISION

A repeated request is safe only when the owner proves that the same authorized scope, key and semantic command already produced one recorded response. Replay recovers that response; it never repeats the Domain decision, reconstructs newer distributed state or advances downstream evidence.

Chapter checkpoint

Question	Expected answer
Is deployment the API version?	No. Route, media, problem, message, deployment and domain are distinct axes.
What selects request/response representation?	Content-Type selects request; Accept selects response; unsupported outcomes use 415/406.
What is problem machine identity?	The RFC 9457 type URI; clients do not parse title/detail.
May a problem expose exception/request?	No. Bounded safe fields and opaque occurrence identity only.
What does the key identify?	One retrievable command within server-resolved provider/route/capability/actor/tenant/site scope.
Is clientRequestId the key?	No. It is correlation data and cannot authorize replay.
When may command execute?	Only the unique absent-record claimant; duplicates never execute.
Same key + different fingerprint?	Stable 409 key-reused with no prior disclosure/execution.
What is replayed after lost 202?	Original AlarmManagement 202, media, Location, safe headers and immutable body.
Does replay include case-rc-3107?	No. RootCause is a later independent owner; replay records TX-A only.
May expiry create new command?	No. Tombstone/status reconciliation first eliminates ambiguity.
Is this Idempotency-Key behavior an RFC?	No. The latest IETF document is an expired Internet-Draft; exact behavior is project policy.
What does Chapter 24 add?	Executable old/new schema, server, generated-client, problem and replay compatibility tests.

NEXT CHAPTER

Chapter 24, Contract Compatibility Tests, will turn the version, media, Problem Details and idempotency promises into executable producer/consumer evidence. It will define an old/new schema corpus, generated-client build matrix, wire snapshots and negative fixtures that distinguish a compatible addition from a silent semantic break.

PART IV - PUBLIC CONTRACTS AND THE SYNCHRONOUS EDGE

24. Contract Compatibility Tests

A contract is compatible only when a concrete old or new consumer can send a real HTTP exchange to a concrete old or new provider and preserve the meaning it was built to understand. A schema diff is an input to that decision, not the decision itself. NEXUS-1 therefore freezes wire artifacts, generates clients, executes both directions of the matrix and publishes evidence tied to immutable builds.

CHAPTER ARGUMENT

Treat compatibility as executable evidence over producer/consumer pairs. Prove additive evolution with old-client/new-provider and new-client/old-provider fixtures; freeze status, headers, media, JSON shape and stable problem identity; then run the complete AlarmManagement lost-response replay without allowing RootCause state to leak into the owner response.

Chapter 23 specified route/media versioning, RFC 9457 Problem Details and owner-scoped idempotency. This chapter turns VER-COMPAT-001 into a release gate. The reference target remains a net8.0 design: the test projects, generated clients, provider host, database fixtures and evidence publisher described here have not compiled or executed. RootCause remains G0 accepted and G1 specified.

Compatibility is observed behavior, not textual similarity

Two OpenAPI descriptions can differ while every supported consumer still succeeds; they can also look similar while a changed default, enum meaning, unit, precision or retry rule silently breaks behavior. NEXUS-1 separates structural compatibility, serialization compatibility, protocol compatibility and semantic compatibility. A release passes only when every applicable layer has evidence.

Layer	Question	Decisive evidence
Shape	Can the intended instance validate and deserialize?	JSON Schema corpus plus generated-client compile/run.
Wire	Are names, casing, nulls, numbers, dates and unknown fields encoded as promised?	Byte/semantic JSON snapshots with explicit normalization policy.
HTTP	Are route, method, media, status and headers stable?	TestServer exchange fixtures for success and every public failure.
Meaning	Does the old client make the same safe decision?	Consumer assertions over values, defaults, enums and state transitions.
Temporal	Do retry, deprecation and concurrency preserve outcome?	Lost-response, replay, in-progress, mismatch and expiry scenarios.
Ownership	Does each service report only truth it owns?	Golden responses and event-driven RootCause observation.

	Provider v1	Provider v1+	Provider v2
Client v1	PASS	PASS	BLOCK
Client v1+	PASS	PASS	BLOCK
Client v2	N/A	NEGOTIATE	PASS

Compatibility is a property of a producer/consumer pair, not of one schema in isolation.

Figure 24.1 - Minimum old/new compatibility matrix.

The four pair matrix

Every proposed provider change is evaluated against at least four executable pairs. “Old” means the last supported released artifact, not an ad hoc DTO copied into the test project. “New” means the candidate produced by the current build. Each client is generated or restored from an immutable contract digest so the matrix cannot accidentally compile both sides from the same source.

Pair	Purpose	Expected disposition
C-old -> P-old	Control: prove fixtures and archived client still work.	Must pass; failure invalidates the corpus.
C-old -> P-new	Backward compatibility: protect deployed consumers.	Must pass for additive/minor candidate.
C-new -> P-old	Forward tolerance and rollout order.	Pass only for features declared optional/fallback-capable.
C-new -> P-new	Candidate correctness.	Must pass before any compatibility claim.
C-unsupported -> P-new	Version/media rejection.	Must fail deterministically with 406/415 or route miss.
Raw oracle -> P-new	Detect generator blind spots.	Must match frozen bytes/semantics independently of SDK.

NO DIAGONAL-ONLY TESTING

C-new -> P-new proves that today’s producer and today’s client agree with each other. It does not prove that either remains compatible with anything already deployed.

Artifact topology and identity

The compatibility suite consumes immutable artifacts rather than project references that drift under it. Every result binds the provider image/build, OpenAPI digest, standalone schema digest, generated client package digest, fixture corpus digest, database migration and test-run identifier. An evidence record without those identities cannot be reproduced and is not a gate result.

Artifact	Canonical location	Freeze rule
OpenAPI entry document	contracts/http/alarm-management/v1/openapi.yaml	Bundled and hashed after reference resolution.
JSON Schemas	contracts/http/.../schemas/*.schema.json	Explicit Draft 2020-12/OAS dialect; no implicit validator mode.
Problem catalogue	contracts/problems/problem-types.yaml	Stable type URI, status, code, safe extensions.
Wire corpus	contracts/fixtures/http/v1/...	Raw request/response headers and bodies plus semantic form.
Generated clients	artifacts/clients/{digest}/{language}	Package and generator version retained together.
Provider build	artifacts/providers/{buildDigest}	Same deployable or byte-identical test target.
Evidence index	evidence/compat/{runId}/index.json	Signed/immutable result with links to logs and diffs.



A diff proposes risk; executable old/new behavior decides release eligibility.

Figure 24.2 - Evidence-producing compatibility pipeline.

Compatibility policy: additive does not mean harmless

Candidate change	Default classification	Required proof
Add optional response property	Potentially compatible	Old generated client ignores it; raw oracle confirms unchanged known fields.
Make optional request property required	Breaking	New major route/media; old requests must not be reinterpreted

Candidate change	Default classification	Required proof
Add enum value	Potential semantic break	Old client has explicit unknown branch and safe behavior.
Change number to string or unit	Breaking	New representation with migration and dual-read plan.
Tighten min/max/pattern	Request breaking	Archived valid corpus remains accepted or major version.
Relax request validation	Usually compatible	Security/domain invariant review still required.
Change null to absent/default	Potential semantic break	Golden client behavior for all three states.
Change status or Location	Protocol breaking	Frozen exchange must remain stable.
Change problem title/detail only	Compatible for machines	Clients assert type/code/status, never prose.
Change problem type URI/code	Breaking	New stable identity and migration plan.
Change idempotency scope/fingerprint/TTL	Behavioral breaking	Chapter 23 replay corpus and explicit policy version.
Reorder JSON object properties	Compatible	Semantic comparator ignores member order; raw signature policy reviewed.

Schema diff and instance corpus must agree

The static diff classifier catches obvious surface changes quickly, but every classification is challenged by instances. Positive fixtures include minimum, typical, maximum and extension-bearing messages. Negative fixtures isolate one violated invariant each. Metamorphic fixtures transform a valid instance in a way that should preserve or reject meaning. Property and fuzz corpora search combinations the curated set did not anticipate.

Draft 2020-12 fixture manifest (illustrative)

```

schema: alarm-flood-detection-response-v1
dialect: https://json-schema.org/draft/2020-12/schema
cases:
  - id: RESP-V1-MIN-001
    instance: response/minimum.json
    expected: valid
  - id: RESP-V1-UNKNOWN-001
    transform: add /diagnostics/vendorHint
    oldClient: ignore-and-preserve-known-meaning
  - id: RESP-V1-ENUM-001
    transform: replace /severity with "EXTREME"
    oldClient: explicit-unknown-safe-branch
  - id: RESP-V1-NULL-001
    variants: [absent, null, explicit-default]
    expected: distinct-policy-assertions

```

Corpus class	What it detects	Minimum ownership
Golden valid	Accidental rejection and serialization drift.	Provider contract owner.
Golden invalid	Silent relaxation or wrong public problem.	Provider + security/domain owner.
Historical production-shaped	Real combinations omitted by exi	Sanitized/approved telemetry owner.

Corpus class	What it detects	Minimum ownership
Generated boundary	min/max/length/precision/date edge.	Schema tooling owner.
Metamorphic	Unknown fields, reordering, optionality, defaults.	Compatibility owner.
Fuzz/resource	Parser ambiguity, regex/size exhaustion.	Security/performance owner.

Wire snapshots are protocol evidence

A schema validator cannot prove Content-Type, Accept negotiation, Vary, Location, Deprecation, Link, Sunset, cache controls or the exact status branch. The wire harness records the complete exchange and compares it under a declared normalization policy. It may normalize volatile correlation identifiers and dates, but it may not normalize away contract-significant values.

Canonical success fixture (illustrative)

```
POST /api/v1/alarm-flood-detections HTTP/1.1
Accept: application/vnd.nexus.alarm-flood-detection.v1+json
Content-Type: application/vnd.nexus.alarm-flood-command.v1+json
Idempotency-Key: idem-am-7f9c2a

HTTP/1.1 202 Accepted
Content-Type: application/vnd.nexus.alarm-flood-detection.v1+json
Location: /api/v1/alarm-flood-detections/decision-am-9184
Vary: Accept

{"decisionId":"decision-am-9184","status":"accepted","factId":"fact-am-7716"}
```

Field	Comparator	Never normalize
Method/path/query	Exact after documented URI canonicalization.	Major route or parameter presence.
Media types	Case-aware token/parameter parser.	Version vendor subtype.
Status	Exact.	202 vs 200/201; 406 vs 415; 409 branches.
Headers	Per-header semantics.	Location, Vary, Deprecation, Link, Sunset, Retry-After.
JSON	Parsed semantic comparison plus selected raw checks.	Known property values/types/null semantics.
Problem	Stable type/status/code; safe extension allowlist.	Machine identity or leakage checks.

Generated clients are executable consumers

The suite compiles archived and candidate clients in isolated test projects. A generated type is not treated as proof until it sends a request and makes the same decision a real application makes. Compile failure is useful evidence, but successful compilation alone can hide runtime enum fallbacks, null coercion, default insertion or unknown-field rejection.

Old-client behavioral assertion (illustrative C#)

```
[Fact]
public async Task OldClient_reads_candidate_addition_without_meaning_change()
{
    using var host = CandidateAlarmManagementFactory.Create();
    var old = ArchivedClients.V1.Create(host.CreateClient());
    var result = await old.CreateFloodDetectionAsync(CommandV1.Minimum,
        idempotencyKey: "idem-am-7f9c2a");

    Assert.Equal("decision-am-9184", result.DecisionId);
    Assert.Equal(AcceptanceStatus.Accepted, result.Status);
    Assert.Equal("fact-am-7716", result.FactId);
    Assert.DoesNotContain("case-rc-3107", result.RawBody);
}
```

Client behavior	Required assertion	Failure meaning
Unknown response property	Known fields deserialize unchanged.	Deserializer/configuration is not forward tolerant.
Unknown enum value	Explicit safe Unknown result, never default member.	Silent semantic corruption.
Absent optional field	Documented fallback only.	Candidate changed required consumer assumption.
Explicit null	Distinct from absent where contract says so.	Null/default collapse.
Problem response	Typed branch by type/code/status.	Client parses prose or treats all errors alike.
202 replay	Same stored body and headers.	Client/provider duplicated or reconstructed outcome.

Provider host: realistic enough to decide

A fast lane uses WebApplicationFactory/TestServer to exercise routing, middleware, negotiation, serialization, Problem Details mapping and authorization without external network overhead. A database lane uses the production database engine and real migrations because uniqueness, transaction boundaries and concurrency cannot be proved with an in-memory substitute. A deployable lane runs the built image behind the same proxy/header policy used in staging. Evidence names the lane; no result is silently promoted across lanes.

Lane	Includes	Can prove	Cannot prove alone
In-process HTTP	Real ASP.NET pipeline; controlled dependencies.	Route/media/status/header/body mapping.	Database races, proxy behavior, deployment config.
Infrastructure integration	Production DB engine, migrations, outbox.	Unique claim, rollback/commit, replay persistence.	Real proxy/network loss unless injected.
Container/system	Deployable image, proxy, DB, broker.	Packaging, header forwarding, cross-process concurrency.	Production capacity/operations by itself.
Runtime observation	Canary traffic and bounded telemetry.	Actual distribution and operational behavior.	Exhaustive compatibility without controlled fixtures.

Problem Details compatibility

Every public failure has a catalogue entry and an exhaustive server mapping. The matrix tests success and failures independently because a compatible success schema does not protect error behavior. The old client must classify the candidate provider by stable type URI, HTTP status and project code. title and detail remain human-facing and may vary without changing machine meaning; output is scanned for secrets, stack traces, request bodies and internal identifiers.

Scenario	Expected public result	Compatibility assertion
Unsupported request media	415 problem	Stable unsupported-media type/code; no command execution.
Unsupported response media	406 problem	Stable not-acceptable type/code; Vary behavior correct.
Invalid command	400/422 project policy	One catalogue identity; bounded field violations.
Same key, different fingerprint	409 key-reused problem	No prior body disclosure and no execution.
Matching key in progress	409/425 project policy	Retry guidance stable and bounded.
Known expired/tombstoned key	Stable project branch	Never treated as absent before reconciliation.
Unexpected exception	500 safe problem	Opaque occurrence identity; zero implementation leakage.

Problem compatibility oracle (illustrative)

```
assert response.content_type == "application/problem+json"
assert problem.type == "https://errors.nexus-1.example/idempotency/key-reused"
assert problem.status == 409
assert problem.code == "IDEM_KEY_REUSED"
assert "stackTrace" not in raw_body
assert "requestBody" not in raw_body
assert previous_response_body not in raw_body
```

Idempotency compatibility is a stateful protocol

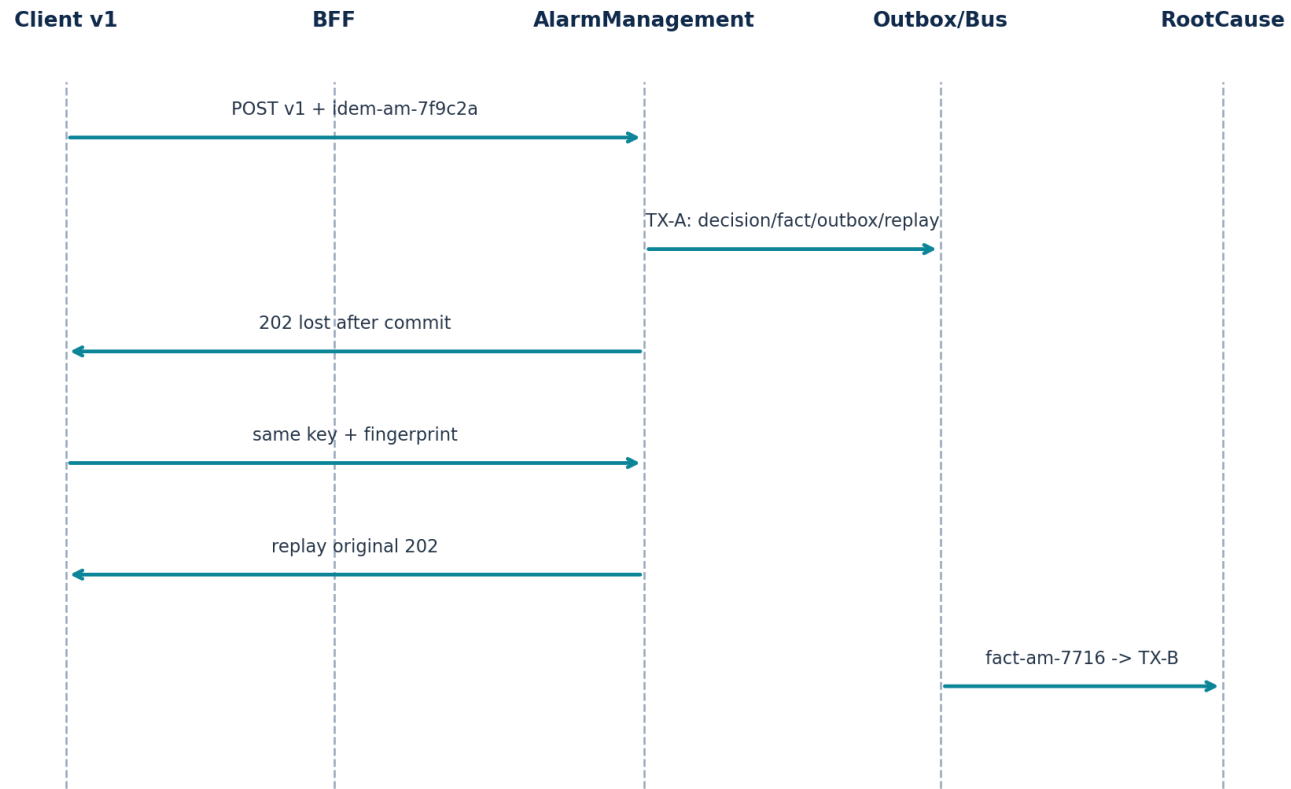
Replay tests cannot be reduced to two sequential POSTs against a mock. The harness controls commit, response delivery, process death, concurrency, time and storage visibility. It records the count of decisions, public facts, outbox records and replay records before and after every attempt. A test passes only when the owner executes once and the matching retry returns the original stored contract.

Scenario	Injected point	Required invariant
Duplicate after success	After response received.	Same response; execution count remains one.
Lost 202	After TX-A commit, before client receives bytes.	Matching retry replays original 202/Location/body.
Concurrent duplicate	Barrier before authoritative claim.	One winner; no two decisions/facts/outbox records.
Different fingerprint	Existing key in same scope.	Stable 409; no execution or prior response disclosure.
Crash before commit	Inside TX-A before durable commit.	Rollback; recovery reaches one unambiguous outcome.
Crash after commit	Aft	

Scenario	Injected point	Required invariant
Clock/expiry boundary	Before/at/after TTL and tombstone.	No expired record silently becomes a fresh command.
Backup/restore	Restore to controlled point.	Reconciliation prevents duplicate lease/commit.

Complete reference flow: lost 202 through independent RootCause

The following walkthrough is one coherent fixture. It intentionally spans HTTP, owner persistence, outbox publication and downstream observation so the reader can see exactly which compatibility assertions belong to which boundary. It does not collapse RootCause into the synchronous response.



Compatibility assertion: replay stays TX-A truth; case-rc-3107 is observed only through RootCause.

Figure 24.3 - Lost-response compatibility sequence.

Step	Action and fixture	Observable assertion
1	Client v1 sends POST /api/v1/alarm-flood-detections with idem-am-7f9c2a.	Candidate provider accepts v1 media and archived client serialization.
2	AlarmManagement resolves scope and fingerprint, wins unique claim.	One owner record; authorization/scope derived server-side.
3	TX-A commits decision-am-9184, fact-am-7716, outbox-am-6031 and stored 202 replay.	All owner effects and replay share atomic outcome.
4	Harness drops response after commit.	Client observes transport ambiguity; server state remains committed.
5	Client/BFF retries same key and semantically identical v1 request.	No second command execution; matching fingerprint found.
6	AlarmManagement returns stored 202, Loc	Bytes/semantics match original; no

Step	Action and fixture	Observable assertion
7	Outbox publishes fact-am-7716; RootCause independently commits TX-B case-rc-3107.	Event and consumer evidence are separate from HTTP replay.
8	RootCause query eventually exposes case-rc-3107.	G1 contract specified; runtime evidence still required.
9	Counts and evidence index are sealed.	decision=1, fact=1, outbox logical message=1, replay=1, case <=1 per consumer idempotency.

Scenario skeleton with explicit evidence IDs

Given provider build P-new and archived client C-v1
 And database state is empty for owner scope S
 When C-v1 sends request-am-20482 with key idem-am-7f9c2a
 And fault HTTP-DROP-001 drops the committed 202
 And C-v1 repeats the identical semantic command
 Then IDEM-REPLAY-001 returns decision-am-9184 and fact-am-7716
 And IDEM-TXN-001 proves one TX-A decision/fact/outbox/replay
 And VER-COMPAT-001 proves the C-v1 -> P-new wire contract
 And RC-COMPAT-001 observes case-rc-3107 only through RootCause

TRUTH BOUNDARY UNDER TEST

The replay response proves AlarmManagement acceptance in TX-A. The later RootCause case proves independent TX-B processing. Compatibility fails if the replay is enriched with case-rc-3107, even if that enrichment appears convenient.

Fault injection must identify the impossible-to-see interval

A “timeout test” is too vague. The fixture names the exact point, arms it deterministically and records whether the owner transaction committed. Network delay before the request reaches the provider is a different case from loss after commit. Only the latter proves replay of an ambiguous outcome. Random sleeps are not synchronization; barriers, hooks and externally observed durable state are.

Fault ID	Point	Pass oracle
HTTP-DROP-001	After response body stored and TX-A committed; before client reads.	Retry returns stored response; effect count one.
TXA-ROLLBACK-001	After decision insert, before outbox/replay commit.	No partial owner state visible.
PROC-KILL-001	After commit, before transport completion.	New process replays from durable store.
CLAIM-RACE-001	N requests released at unique-claim barrier.	One claimant executes; others follow declared state branches.
CLOCK-EDGE-001	Controlled instant around expiry/tombstone.	No lease-based duplicate or ambiguous fresh execution.
BUS-REDLV-001	Duplicate fact delivery to RootCause.	TX-B consumer idempotency; at most one logical case.

CI lanes and release gates

Lane	Trigger	Budget	Blocking evidence
Contract lint/diff	Every contract edit.	Minutes.	Dialect parse, catalogue references, classified diff.
Fast pair matrix	Every pull request.	Minutes.	C-old/P-new and C-new/P-new HTTP/schema/problem fixtures.
Generated-client matrix	Every pull request affecting public surface.	Minutes to tens.	Compile and behavioral results for supported SDKs.
Stateful replay	Every idempotency/transaction change; nightly otherwise.	Tens of minutes.	Lost-response, mismatch, crash and concurrency invariants.
Deployable system	Release candidate.	Controlled longer run.	Proxy/header/container/database/broker evidence.
Runtime canary	Approved deployment.	Time-boxed.	Compatibility telemetry within thresholds; rollback ready.

A static diff marked “compatible” cannot waive a failing executable pair. A flaky compatibility test is quarantined only with owner, incident, expiry and compensating gate; it does not become a passing result. Baseline regeneration requires explicit review of the semantic change and cannot occur in the same unreviewed step that detected the difference.

Evidence catalogue

Evidence ID	Acceptance statement	Minimum artifact
COMP-ART-001	Every result identifies immutable provider/client/schema/corpus digests.	Evidence index and retention proof.
COMP-DIFF-001	Every change is classified with rationale and reviewer.	Machine diff plus semantic decision record.
COMP-SCHEMA-001	Positive/negative/metamorphic instances behave under declared dialect.	Corpus and validator output.
COMP-WIRE-001	HTTP status, headers, media and bodies match normalization policy.	Raw and semantic snapshots.
COMP-CLIENT-001	Archived generated client compiles and preserves meaning against candidate.	Build log and behavioral assertions.
COMP-PROBLEM-001	Every failure maps to stable safe Problem Details identity.	Exhaustive catalogue matrix and leak scan.
COMP-IDEM-001	Lost response, replay, mismatch and concurrency preserve one owner effect.	Fault traces and database counts.
COMP-RC-001	RootCause remains independent and never enriches AlarmManagement replay.	Golden HTTP body plus event/query evidence.
COMP-SEC-001	Corpus and output resist abuse/leakage.	Fuzz/resource/redaction results.
COMP-REVIEW-001	Owners accept residual risk and release disposition.	Signed evidence index and waiver register.

ADR-022 - Executable old/new compatibility evidence

ADR field	ADR-022 record
Status	Accepted for the reference design; implementation and runtime evidence pending.
Context	Schema-only and same-version tests cannot prove deployed consumers survive additive changes or stateful retry semantics.
Decision	Freeze immutable contract/client/provider/corpus artifacts; execute old/new pair matrix, raw wire oracles, generated-client behavior, Problem Details and stateful idempotency scenarios; publish evidence IDs.
Consequences	Requires artifact retention, deterministic fault hooks, production-like database lane, client generation, evidence storage, ownership and CI capacity.
Rejected	Diagonal-only tests; mock-only contracts; snapshot auto-approval; schema diff as final verdict; prose-based errors; in-memory idempotency; RootCause enrichment in replay.
Review trigger	New supported client/language, schema dialect/generator, version axis, normalization rule, problem identity, retry state, storage/TTL, proxy or rollout policy.

Risk register

Risk	Early signal	Blocking control
False green from shared model	Old/new tests compile from same DTO/package.	Digest-pinned archived client in isolated project.
Golden-file blindness	Baseline update hides semantic break.	Independent behavioral assertions and approval.
Validator drift	Local and CI dialect/format behavior differ.	Pinned engine/version/config and conformance corpus.
Flaky race proof	Sleep-based concurrency sometimes passes.	Deterministic barrier plus durable count oracle.
Mock fidelity gap	Test double accepts behavior production rejects.	Real host/database/deployable lanes.
Sensitive fixtures	Production-shaped corpus leaks data.	Synthetic/sanitized fixtures and access/retention review.
Evidence without identity	Pass cannot be reproduced.	Immutable digest manifest.
RootCause overclaim	Replay contains case-rc-3107 or completion.	COMP-RC-001 golden body and separate TX-B evidence.

Official-source validation

The contract and test terminology was checked against current primary sources on 19 July 2026. OpenAPI defines a language-agnostic HTTP API description used by documentation, generators and testing tools; OAS 3.1 uses the JSON Schema Draft 2020-12 model with an explicit dialect. JSON Schema validation asserts structural constraints but format support can vary and semantic validation remains an application responsibility. Microsoft documents TestServer/WebApplicationFactory as a functional-test host and recommends keeping the test host close to production behavior. Part

the exact application versions and verified contracts. NEXUS-1's exact matrix, evidence IDs, replay oracle and release policy remain project decisions.

Primary source	Stable point	NEXUS-1 use
OpenAPI Specification 3.1.1	API description, schema dialect, examples and generators.	Frozen bundled OAD plus generated clients and wire tests.
JSON Schema Draft 2020-12	Instance validation vocabulary and explicit format behavior.	Pinned dialect/validator corpus; semantics tested separately.
Microsoft ASP.NET Core testing guidance	TestServer/WebApplicationFactory functional host; infrastructure tests.	Fast HTTP lane plus real DB/deployable lanes.
Pact versioning guidance	Consumer/provider application versions bind verification/deploy decisions.	Immutable old/new identities; optional broker is not required by design.

Sources: <https://spec.openapis.org/oas/v3.1.1.html> ; <https://json-schema.org/draft/2020-12> ; <https://learn.microsoft.com/aspnet/core/test/integration-tests> ; https://docs.pact.io/getting_started/versioning_in_the_pact_broker

Technical material still required

The compatibility model, pair matrix, artifact topology, change classifier, corpora, wire oracle, generated-client behavior, Problem Details matrix, provider lanes, full lost-202 walkthrough, fault catalogue, CI gates, evidence IDs and ADR are specified. No NEXUS-1 repository has generated the clients, compiled the test projects, started WebApplicationFactory, applied a migration, won a database race, injected a committed-response loss, replayed a durable 202 or published a signed evidence index.

TECHNICAL MATERIAL STATUS - [CONTRACT CORPUS, OLD/NEW GENERATED CLIENTS, PROVIDER TEST HOST, STATEFUL FAULT INJECTION AND EXECUTABLE EVIDENCE REQUIRED]

Implement the net8.0 compatibility solution, frozen OpenAPI/JSON Schema/problem/wire artifacts, generator lockfiles, archived client packages, WebApplicationFactory/TestServer provider, production-engine database fixtures, deterministic fault barriers, idempotency/outbox/RootCause count oracles, CI matrices, safe-output scans and signed evidence publication. Execute COMP-ART-001 through COMP-REVIEW-001. RootCause remains G0 accepted and G1 specified; runtime evidence has not advanced.

Deliverable	Minimum content	Evidence
Frozen contracts	Bundled OAS, explicit schemas, catalogue, raw/semantic fixtures.	COMP-ART/SCHEMA/WIRE-001
Old/new clients	Pinned generator and isolated archived/current packages.	COMP-CLIENT-001
Provider harness	HTTP host plus production DB and deployable lanes.	COMP-WIRE/PROBLEM-001
Fault harness	Drop/rollback/crash/race/clock/bus controls and counts.	COMP-IDEM/RC-001
CI and publication	Pair matrix, waivers, digests, signed evidence index.	COMP-REVIEW-001

CHAPTER 24 - CODE AUDIT

Immutable artifacts define compatibility identity

A closed allowlist is resolved beneath one artifact root, hashed as bytes, ordered canonically and bound to the result before any pair executes.

EVIDENCE STATUS

These listings are compile-oriented. No client, provider, migration, pair or fault scenario has executed here. COMP evidence remains PENDING.

EXECUTABLE ASSET 24-A - CLOSED ARTIFACT MANIFEST

```
public sealed record ArtifactFile(
    string RelativePath,
    long Length,
    string Sha256Hex);

public sealed record CompatibilityArtifactSet(
    string SetId,
    string ManifestProfile,
    IReadOnlyList<ArtifactFile> Files,
    string ManifestSha256Hex);

public static class ArtifactManifestBuilder
{
    public const string Profile =
        "nexus.compat-artifacts.v1/len32-utf8+sha256";

    public static CompatibilityArtifactSet Build(
        string setId,
        DirectoryInfo root,
        IReadOnlyCollection<string> allowList)
    {
        var rootPath = Path.GetFullPath(root.FullName);
        var files = allowList
            .Select(path => Resolve(rootPath, path))
            .DistinctBy(x => x.RelativePath,
                StringComparer.Ordinal)
            .OrderBy(x => x.RelativePath,
                StringComparer.Ordinal)
            .Select(HashFile)
            .ToArray();

        if (files.Length != allowList.Count)
            throw new DuplicateArtifactPath();

        return new(setId, Profile, files,
            HashManifest(files));
    }

    private static (string RelativePath, string FullPath)
        Resolve(string root, string requested)
    {
        var full = Path.GetFullPath(Path.Combine(root, requested));
        var relative = Path.GetRelativePath(root, full);
        if (Path.IsPathRooted(relative)
            || relative == "."
            || relative.StartsWith(
                $"{Path.DirectorySeparatorChar}",
                StringComparison.Ordinal))
            throw new ArtifactEscapesRoot(requested);

        var info = new FileInfo(full);
        if (!info.Exists || info.LinkTarget is not null)
            throw new InvalidArtifactFile(requested);
        for (var dir = info.Directory;
            dir is not null
            && !StringComparer.Ordinal.Equals(
                dir.FullName, root);
            dir = dir.Parent)
            if (dir.LinkTarget is not null)
                throw new InvalidArtifactFile(requested);
        return (relative.Replace('\\', '/'), full);
    }

    private static ArtifactFile HashFile(
        (string RelativePath, string FullPath) value)
    {
        using var stream = File.OpenRead(value.FullPath);
        var digest = SHA256.HashData(stream);
        return new(value.RelativePath, stream.Length,
            Convert.ToHexString(digest).ToLowerInvariant());
    }
}
```

EXECUTABLE ASSET 24-B - FORMATTING-FREE MANIFEST DIGEST

```
private static string HashManifest(
    IReadOnlyList<ArtifactFile> files)
{
    using var hash = IncrementalHash.CreateHash(
        HashAlgorithmName.SHA256);
    foreach (var file in files)
    {
        AddUtf8(hash, file.RelativePath);
        AddInt64(hash, file.Length);
        hash.AppendData(Convert.FromHexString(
            file.Sha256Hex));
    }
    return Convert.ToHexString(hash.GetHashAndReset())
        .ToLowerInvariant();
}

private static void AddUtf8(
    IncrementalHash hash, string value)
{
    var bytes = Encoding.UTF8.GetBytes(value);
    Span<byte> length = stackalloc byte[4];
    BinaryPrimitives.WriteInt32BigEndian(
        length, bytes.Length);
    hash.AppendData(length);
    hash.AppendData(bytes);
}

private static void AddInt64(
    IncrementalHash hash, long value)
{
    Span<byte> bytes = stackalloc byte[8];
    BinaryPrimitives.WriteInt64BigEndian(bytes, value);
    hash.AppendData(bytes);
}

public sealed class ArtifactManifestTests(
    ArtifactFixture fixture)
{
    [Fact]
    public void COMP_ART_001_mutation_changes_identity()
    {
        var before = fixture.BuildManifest();
        fixture.Mutate("contracts/wire/accepted.json");
        var after = fixture.BuildManifest();
        after.ManifestSha256Hex.Should()
            .NotBe(before.ManifestSha256Hex);
    }
}
```

PAIR TOPOLOGY AND CLOSURE

Every verdict names both executable sides

Old and new are immutable build identities, not aliases to the current source tree. The matrix validator rejects missing controls, duplicate rows and any result that cannot name its exact consumer and provider artifacts.

EXECUTABLE ASSET 24-C - TYPED OLD/NEW PAIR MATRIX

```
public enum BuildAge { Old, New, Unsupported, RawOracle }
public enum PairDisposition
{
    MustPass,
    ConditionalForwardTolerance,
    MustReject
}

public sealed record ConsumerArtifact(
    string Id,
    BuildAge Age,
    string ExecutableSha256,
    string ContractSha256,
    string GeneratorIdentity);

public sealed record ProviderArtifact(
    string Id,
    BuildAge Age,
    string ImageSha256,
    string ContractSha256);

public sealed record CompatibilityPair(
    string PairId,
    ConsumerArtifact Consumer,
    ProviderArtifact Provider,
    PairDisposition Disposition);

public static class AlarmCompatibilityMatrix
{
    public static IReadOnlyList<CompatibilityPair> Create(
        ConsumerArtifact oldClient,
        ConsumerArtifact newClient,
        ConsumerArtifact unsupportedClient,
        ConsumerArtifact rawOracle,
        ProviderArtifact oldProvider,
        ProviderArtifact newProvider) =>
    [
        new("C-old__P-old", oldClient, oldProvider,
            PairDisposition.MustPass),
        new("C-old__P-new", oldClient, newProvider,
            PairDisposition.MustPass),
        new("C-new__P-old", newClient, oldProvider,
            PairDisposition.ConditionalForwardTolerance),
        new("C-new__P-new", newClient, newProvider,
            PairDisposition.MustPass),
        new("C-unsupported__P-new", unsupportedClient,
            newProvider, PairDisposition.MustReject),
        new("raw-oracle__P-new", rawOracle, newProvider,
            PairDisposition.MustPass)
    ];
}
```

EXECUTABLE ASSET 24-D - FAIL-CLOSED MATRIX GUARD

```
public static class PairMatrixGuard
{
    private static readonly string[] Required =
    [
        "C-old__P-old", "C-old__P-new",
        "C-new__P-old", "C-new__P-new",
        "C-unsupported__P-new", "raw-oracle__P-new"
    ];

    public static void AssertClosed(
        IReadOnlyCollection<CompatibilityPair> pairs)
    {
        var ids = pairs.Select(x => x.PairId).ToArray();
        if (ids.Length != ids.Distinct(
            StringComparer.Ordinal).Count())
            throw new DuplicateCompatibilityPair();

        var missing = Required.Except(
            ids, StringComparer.Ordinal).ToArray();
        if (missing.Length != 0)
            throw new MissingCompatibilityPairs(missing);
        var unknown = ids.Except(
            Required, StringComparer.Ordinal).ToArray();
        if (unknown.Length != 0)
            throw new UnknownCompatibilityPairs(unknown);

        if (pairs.Any(x =>
            String.IsNullOrEmpty(
                x.Consumer.ExecutableSha256)
                || String.IsNullOrEmpty(
                    x.Provider.ImageSha256)))
            throw new UnidentifiedPairArtifact();
    }
}
```

CONTROL BEFORE CANDIDATE

C-old -> P-old must pass first. If the archived control fails, the corpus or client archive is invalid and no candidate compatibility verdict is publishable.

ISOLATED GENERATED CONSUMERS

Client runners share a protocol, never provider DTOs

Each client is built in its own project from its frozen contract and launched as a separate process. The harness accepts only a small observation document, so an accidental reference to the candidate provider model cannot create a false green.

EXECUTABLE ASSET 24-E - PROCESS-ISOLATED CLIENT RUNNER

```
public sealed record ClientRunnerArtifact(
    string Id,
    string ExecutablePath,
    string ExecutableSha256,
    string ContractSha256,
    string GeneratorIdentity,
    string DependencyManifestSha256);

public sealed record ClientObservation(
    int Status,
    string? DecisionId,
    string? AcceptanceStatus,
    string? FactId,
    string? ProblemType,
    string? ProblemCode,
    string RawBodySha256);

public sealed class IsolatedClientRunner
{
    public async Task<ClientObservation> ExecuteAsync(
        ClientRunnerArtifact artifact,
        Uri provider,
        string scenarioId,
        CancellationToken cancellationToken)
    {
        VerifyExecutableDigest(artifact);
        var start = new ProcessStartInfo
        {
            FileName = artifact.ExecutablePath,
            RedirectStandardOutput = true,
            RedirectStandardError = true,
            UseShellExecute = false
        };
        start.ArgumentList.Add("--provider");
        start.ArgumentList.Add(provider.AbsoluteUri);
        start.ArgumentList.Add("--scenario");
        start.ArgumentList.Add(scenarioId);

        using var process = Process.Start(start)
            ?? throw new ClientRunnerDidNotStart();
        var outputTask = process.StandardOutput
            .ReadToEndAsync(cancellationToken);
        var errorTask = process.StandardError
            .ReadToEndAsync(cancellationToken);
        await process.WaitForExitAsync(cancellationToken);
        var output = await outputTask;
        var error = await errorTask;

        if (process.ExitCode != 0)
            throw new ClientRunnerFailed(
                artifact.Id, process.ExitCode, Redact(error));
        return JsonSerializer.Deserialize<ClientObservation>(
            output, StrictObservationJson.Options)
            ?? throw new MissingClientObservation();
    }
}
```

EXECUTABLE ASSET 24-F - OLD-CLIENT MEANING ASSERTION

```
public sealed class ClientCompatibilityTests(
    CompatibilityFixture fixture,
    IsolatedClientRunner runners,
    ClientArtifactCatalogue artifacts)
{
    [Fact]
    public async Task COMP_CLIENT_001_old_client_keeps_meaning()
    {
        await using var provider =
            await fixture.StartCandidateProviderAsync();

        var old = await runners.ExecuteAsync(
            artifacts.ArchivedV1,
            provider.BaseAddress,
            "accepted-with-new-optional-diagnostics",
            fixture.CancellationToken);

        old.Status.Should().Be(202);
        old.DecisionId.Should().Be("decision-am-9184");
        old.AcceptanceStatus.Should().Be("accepted");
        old.FactId.Should().Be("fact-am-7716");
        fixture.WireCapture.LastResponseBody.Should()
            .Contain("diagnostics");
        old.ProblemType.Should().BeNull();
    }

    [Fact]
    public void COMP_CLIENT_002_unknown_enum_is_not_defaulted()
    {
        ArchivedV1SemanticPolicy.ParseStatus("EXTREME")
            .Should().Be(ClientAcceptance.Unknown);
        ArchivedV1SemanticPolicy.ParseStatus("EXTREME")
            .Should().NotBe(ClientAcceptance.Accepted);
    }
}
```

GENERATOR IDENTITY IS EVIDENCE

The runner records generator name/version and dependency manifest digest. Successful compilation is not enough; the archived executable must perform the exchange and preserve its documented decision.

PRODUCTION-PATH PROVIDER HARNESS

The fast host changes dependencies, not contract behavior

WebApplicationFactory runs the candidate routing, middleware, authorization, negotiation and serializers. The database boundary is replaced only with a leased instance of the production engine and real migrations.

EXECUTABLE ASSET 24-G - CANDIDATE PROVIDER FACTORY

```
public sealed record ProviderTestDependencies(
    string SqlServerConnectionString,
    TimeProvider Clock,
    IAfterCommitResponseGate ResponseGate,
    ITestTokenIssuer Tokens);

public sealed class CandidateAlarmProviderFactory(
    ProviderTestDependencies dependencies)
    : WebApplicationFactory<Program>
{
    protected override void ConfigureWebHost(
        IWebHostBuilder builder)
    {
        builder.UseEnvironment("CompatibilityTests");
        builder.ConfigureServices(services =>
        {
            services.RemoveAll<
                DbContextOptions<AlarmManagementDb>>();
            services.AddDbContext<AlarmManagementDb>(options =>
                options.UseSqlServer(
                    dependencies.SqlServerConnectionString,
                    sql => sql.MigrationsAssembly(
                        typeof(AlarmManagementDb).Assembly
                            .GetName().Name));
            services.RemoveAll<TimeProvider>();
            services.AddSingleton(dependencies.Clock);
            services.RemoveAll<IAfterCommitResponseGate>();
            services.AddSingleton(
                dependencies.ResponseGate);
            services.AddSingleton(dependencies.Tokens);
        });
    }

    public async Task MigrateAsync(
        CancellationToken cancellationToken)
    {
        await using var scope =
            Services.CreateAsyncScope();
        var db = scope.ServiceProvider
            .GetRequiredService<AlarmManagementDb>();
        await db.Database.MigrateAsync(cancellationToken);
    }

    public HttpClient CreateObservedClient(string actor)
    {
        var client = CreateClient(
            new WebApplicationFactoryClientOptions
            {
                AllowAutoRedirect = false,
                HandleCookies = false,
                BaseAddress = new("https://compat.nexus.test")
            });
        client.DefaultRequestHeaders.Authorization =
            new("Bearer", dependencies.Tokens.Issue(actor));
        return client;
    }
}
```

EXECUTABLE ASSET 24-H - PROVIDER LEASE LIFECYCLE

```
public sealed class CandidateProviderLease : IAsyncDisposable
{
    private readonly CandidateAlarmProviderFactory _factory;
    private readonly ISqlServerLease _database;
    public Uri BaseAddress { get; }

    private CandidateProviderLease(
        CandidateAlarmProviderFactory factory,
        ISqlServerLease database,
        Uri baseAddress)
    {
        _factory = factory;
        _database = database;
        BaseAddress = baseAddress;
    }

    public static async Task<CandidateProviderLease> StartAsync(
        ISqlServerLease database,
        ProviderTestDependencies dependencies,
        CancellationToken cancellationToken)
    {
        await database.StartAsync(cancellationToken);
        var factory =
            new CandidateAlarmProviderFactory(dependencies);
        await factory.MigrateAsync(cancellationToken);
        using var client =
            factory.CreateObservedClient("operator-17");
        return new(factory, database, client.BaseAddress!);
    }

    public async ValueTask DisposeAsync()
    {
        await _factory.DisposeAsync();
        await _database.DisposeAsync();
    }
}
```

NO SILENT FIDELITY PROMOTION

This lane can decide HTTP pipeline behavior and real database semantics. It does not prove proxy packaging or network behavior; those results require the named deployable lane.

EXACT WIRE ORACLE

The exchange is captured before client interpretation

A raw oracle observes status, media type, Location, Vary and body bytes. The semantic comparator ignores JSON member order but never normalizes away version, null/default meaning, machine problem identity or owner-visible fields.

EXECUTABLE ASSET 24-I - RECORDED HTTP EXCHANGE

```
public sealed record RecordedExchange(
    int Status,
    string? MediaType,
    Uri? Location,
    IReadOnlyList<string> Vary,
    byte[] Body)
{
    public static async Task<RecordedExchange> CaptureAsync(
        HttpResponseMessage response,
        CancellationToken cancellationToken)
    {
        var body = await response.Content
            .ReadAsByteArrayAsync(cancellationToken);
        return new(
            (int)response.StatusCode,
            response.Content.Headers.ContentType?.MediaType,
            response.Headers.Location,
            response.Headers.Vary.ToArray(),
            body);
    }
}

public sealed class RawHttpOracle(HttpClient client)
{
    public async Task<RecordedExchange> SendAsync(
        HttpRequestMessage request,
        CancellationToken cancellationToken)
    {
        using var response = await client.SendAsync(
            request,
            HttpCompletionOption.ResponseHeadersRead,
            cancellationToken);
        return await RecordedExchange.CaptureAsync(
            response, cancellationToken);
    }
}
```

EXECUTABLE ASSET 24-J - ACCEPTED-V1 PROTOCOL AND OWNERSHIP ORACLE

```
public static class AcceptedV1Oracle
{
    private const string Media =
        "application/vnd.nexus1.alarm-flood-accepted+json";

    public static void AssertContract(
        RecordedExchange exchange)
    {
        Assert.Equal(202, exchange.Status);
        Assert.Equal(Media, exchange.MediaType);
        Assert.Equal(
            "/api/v1/alarm-flood-detections/decision-am-9184",
            exchange.Location?.PathAndQuery);
        Assert.Contains("Accept", exchange.Vary,
            StringComparer.Ordinal.IgnoreCase);

        using var json = JsonDocument.Parse(exchange.Body);
        var root = json.RootElement;
        Assert.Equal("decision-am-9184",
            root.GetProperty("decisionId").GetString());
        Assert.Equal("accepted",
            root.GetProperty("status").GetString());
        Assert.Equal("fact-am-7716",
            root.GetProperty("factId").GetString());
        Assert.False(root.TryGetProperty(
            "rootCauseCaseId", out _));
        Assert.DoesNotContain("case-rc-3107",
            Encoding.UTF8.GetString(exchange.Body),
            StringComparison.Ordinal);
    }
}

public sealed class WireCompatibilityTests(
    CompatibilityFixture fixture)
{
    [Fact]
    public async Task COMP_WIRE_001_raw_oracle_is_exact()
    {
        var exchange = await fixture.RawOracle
            .SendAsync(Fixtures.AcceptedV1Request(),
                TestContext.Current.CancellationToken);
        AcceptedV1Oracle.AssertContract(exchange);
    }
}
```

NORMALIZATION IS NAMED POLICY

Correlation IDs and dates may be normalized only by an explicit profile. Status, vendor media type, Location, Vary, known JSON values and stable problem identity remain contract-significant.

PROBLEM AND SEMANTIC COMPATIBILITY

Problems are exhaustive; prose is not contractual

The failure corpus enumerates every public branch and rejects any type, status or code drift. Client policy maps unknown enum values explicitly and preserves the difference between an absent member and a present null.

EXECUTABLE ASSET 24-K - STABLE PROBLEM DETAILS SNAPSHOT

```
public sealed record PublicProblemSnapshot(
    string Type,
    int Status,
    string Code,
    IReadOnlySet<string> PropertyNames);

public static class ProblemOracle
{
    private static readonly HashSet<string> Allowed =
        new(StringComparer.Ordinal)
        {
            "type", "title", "status", "detail",
            "instance", "code", "traceId"
        };

    public static PublicProblemSnapshot Parse(
        RecordedExchange exchange)
    {
        Assert.Equal("application/problem+json",
            exchange.MediaType);
        using var json = JsonDocument.Parse(exchange.Body);
        var root = json.RootElement;
        var names = root.EnumerateObject()
            .Select(x => x.Name)
            .ToHashSet(StringComparer.Ordinal);
        Assert.Empty(names.Except(Allowed));

        var raw = Encoding.UTF8.GetString(exchange.Body);
        Assert.DoesNotContain("stackTrace", raw,
            StringComparer.OrdinalIgnoreCase);
        Assert.DoesNotContain("connectionString", raw,
            StringComparer.OrdinalIgnoreCase);

        var status = root.GetProperty("status").GetInt32();
        Assert.Equal(exchange.Status, status);
        return new(
            root.GetProperty("type").GetString()!,
            status,
            root.GetProperty("code").GetString()!,
            names);
    }
}
```

EXECUTABLE ASSET 24-L - EXHAUSTIVE FAILURE AND MEANING MATRIX

```
public sealed record ProblemCase(
    string Scenario,
    int Status,
    string Type,
    string Code);

public enum ClientAcceptance { Accepted, Rejected, Unknown }
public enum OptionalFieldState { Absent, Null, Value }

public sealed record SemanticObservation(
    ClientAcceptance Status,
    OptionalFieldState Note);

public static class SemanticReader
{
    public static SemanticObservation Read(string payload)
    {
        using var json = JsonDocument.Parse(payload);
        var root = json.RootElement;
        var status = root.GetProperty("status").GetString() switch
        {
            "accepted" => ClientAcceptance.Accepted,
            "rejected" => ClientAcceptance.Rejected,
            _ => ClientAcceptance.Unknown
        };
        var note = !root.TryGetProperty("note", out var value)
            ? OptionalFieldState.Absent
            : value.ValueKind == JsonValueKind.Null
            ? OptionalFieldState.Null
            : OptionalFieldState.Value;
        return new(status, note);
    }
}

public sealed class ProblemCompatibilityTests(
    CompatibilityFixture fixture)
{
    public static TheoryData<ProblemCase> PublicFailures => new()
    {
        new("unsupported-content-type", 415,
            "/problems/unsupported-media-type",
            "UNSUPPORTED_MEDIA_TYPE"),
        new("unacceptable-response", 406,
            "/problems/representation-not-acceptable",
            "REPRESENTATION_NOT_ACCEPTABLE"),
        new("same-key-changed-command", 409,
            "/problems/idempotency-key-reused",
            "IDEMPOTENCY_KEY_REUSED"),
        new("validation-failed", 422,
            "/problems/validation-failed",
            "VALIDATION_FAILED")
    };

    [Theory]
    [MemberData(nameof(PublicFailures))]
    public async Task COMP_PROBLEM_001_catalogue_is_stable(
        ProblemCase expected)
    {
        var exchange = await fixture.SendAsync(expected.Scenario);
        var actual = ProblemOracle.Parse(exchange);
        (actual.Status, actual.Type, actual.Code).Should().Be(
            (expected.Status, expected.Type, expected.Code));
    }

    [Fact]
    public void COMP_SEM_001_absent_null_and_unknown_are_distinct()
    {
        SemanticReader.Read(
            @"{"status":"","EXTREME"}")
            .Should().Be(new(ClientAcceptance.Unknown,
                OptionalFieldState.Absent));
        SemanticReader.Read(
            @"{"status":"","accepted":"","note":null}")
            .Note.Should().Be(OptionalFieldState.Null);
    }
}
```

STATEFUL TEMPORAL COMPATIBILITY

The old client survives a committed but lost 202

A deterministic gate is armed before the request and invoked only after TX-A commits. The retry uses the same archived client and idempotency key; durable counts and stored replay bytes decide whether meaning was preserved.

EXECUTABLE ASSET 24-M - ONE-SHOT POST-COMMIT RESPONSE LOSS

```
public interface IAfterCommitResponseGate
{
    ValueTask BeforeResponseAsync(
        HttpContext context,
        CancellationToken cancellationToken);
}

public sealed class NoResponseLossGate
    : IAfterCommitResponseGate
{
    public ValueTask BeforeResponseAsync(
        HttpContext context,
        CancellationToken cancellationToken) =>
        ValueTask.CompletedTask;
}

public sealed class OneShotResponseLossGate
    : IAfterCommitResponseGate
{
    private int _armed;
    private readonly TaskCompletionSource<bool> _observed =
        new(TaskCreationOptions.RunContinuationsAsynchronously);

    public void Arm() =>
        Interlocked.Exchange(ref _armed, 1);

    public Task CommitObserved => _observed.Task;

    public ValueTask BeforeResponseAsync(
        HttpContext context,
        CancellationToken cancellationToken)
    {
        _observed.TrySetResult(true);
        if (Interlocked.Exchange(ref _armed, 0) == 1)
        {
            context.Abort();
            throw new InjectedCommittedResponseLoss();
        }
        return ValueTask.CompletedTask;
    }
}

// The endpoint calls the gate after application TX-A returns,
// and before the response writer emits status, headers or body.
```

EXECUTABLE ASSET 24-N - FULL LOST-202 REPLAY ORACLE

```
public sealed class StatefulCompatibilityTests(
    CompatibilityFixture fixture)
{
    [Fact]
    public async Task COMP_IDEM_001_old_client_replays_one_effect()
    {
        var key = fixture.Keys.New128BitKey();
        fixture.ResponseLoss.Arm();

        await Assert.ThrowsAnyAsync<Exception>(() =>
            fixture.OldClient.CreateAsync(
                Fixtures.CommandV1, key));
        await fixture.ResponseLoss.CommitObserved;

        var stored = await fixture.Database
            .LoadCompletedReplayAsync(key);
        var replay = await fixture.OldClient.CreateAsync(
            Fixtures.CommandV1, key);

        replay.Status.Should().Be(202);
        replay.BodyBytes.Should().Equal(stored.BodyBytes);
        replay.Location.Should().Be(stored.Location);
        replay.MediaType.Should().Be(stored.MediaType);

        var counts = await fixture.Database
            .ReadOwnerCountsAsync(key);
        counts.Should().Be(new OwnerCounts(
            IdempotencyRows: 1,
            Decisions: 1,
            PendingPublicFacts: 1));
        replay.RawBody.Should()
            .NotContain("case-rc-3107");
        (await fixture.RootCause.CountCasesAsync())
            .Should().Be(0);
    }

    [Fact]
    public async Task COMP_IDEM_002_race_keeps_one_owner_effect()
    {
        var key = fixture.Keys.New128BitKey();
        var results = await fixture.ReleaseSimultaneouslyAsync(
            consumers: 32, Fixtures.CommandV1, key);
        results.Select(x => x.DecisionId)
            .Distinct().Should().ContainSingle();
        (await fixture.Database.ReadOwnerCountsAsync(
            key))
            .Decisions.Should().Be(1);
    }
}
```

ROOTCAUSE REMAINS INDEPENDENT

The replay proves only the AlarmManagement owner outcome. RootCause may later observe fact-am-7716 in TX-B, but case-rc-3107 never appears in the stored 202.

ACCEPTANCE AND EVIDENCE PUBLICATION

A zero-test or unidentified run cannot become PASS

EXECUTABLE ASSET 24-O - FAIL-CLOSED COMPATIBILITY EVIDENCE GATE

```
public enum EvidenceStatus { Pending, Passed, Failed }

public sealed record CompatibilityEvidence(
    string EvidenceId,
    EvidenceStatus Status,
    int DiscoveredTests,
    string ConsumerSha256,
    string ProviderSha256,
    string ContractSha256,
    string CorpusSha256,
    string EnvironmentSha256,
    string ResultSha256);

public static class CompatibilityEvidenceGate
{
    private static readonly string[] Required =
    [
        "COMP-ART-001", "COMP-SCHEMA-001",
        "COMP-WIRE-001", "COMP-CLIENT-001",
        "COMP-PROBLEM-001", "COMP-IDEM-001",
        "COMP-RC-001", "COMP-SEC-001",
        "COMP-REVIEW-001"
    ];

    public static void Accept(
        IReadOnlyCollection<CompatibilityEvidence> evidence)
    {
        var byId = evidence.ToDictionary(
            x => x.EvidenceId, StringComparer.Ordinal);
        var missing = Required.Where(x => !byId.ContainsKey(x))
            .ToArray();
        if (missing.Length != 0)
            throw new MissingCompatibilityEvidence(missing);

        foreach (var item in Required.Select(id => byId[id]))
        {
            if (item.Status != EvidenceStatus.Passed)
                throw new EvidenceNotPassed(item.EvidenceId);
            if (item.DiscoveredTests <= 0)
                throw new ZeroDiscoveryEvidence(item.EvidenceId);
            if (new[]
            {
                item.ConsumerSha256, item.ProviderSha256,
                item.ContractSha256, item.CorpusSha256,
                item.EnvironmentSha256, item.ResultSha256
            }.Any(String.IsNullOrEmpty))
                throw new UnidentifiedEvidence(item.EvidenceId);
        }
    }
}
```

EXECUTABLE ASSET 24-P - ACCEPTANCE EVIDENCE RECORD

```
{
  "evidenceId": "COMP-REVIEW-001",
  "chapter": 24,
  "artifactProfile":
    "nexus.compat-artifacts.v1/len32-utf8+sha256",
  "pairs": {
    "C-old_P-old": "PENDING",
    "C-old_P-new": "PENDING",
    "C-new_P-old": "PENDING",
    "C-new_P-new": "PENDING",
    "C-unsupported_P-new": "PENDING",
    "raw-oracle_P-new": "PENDING"
  },
  "tests": {
    "COMP-ART-001": "PENDING",
    "COMP-SCHEMA-001": "PENDING",
    "COMP-WIRE-001": "PENDING",
    "COMP-CLIENT-001": "PENDING",
    "COMP-PROBLEM-001": "PENDING",
    "COMP-IDEM-001": "PENDING",
    "COMP-RC-001": "PENDING",
    "COMP-SEC-001": "PENDING"
  },
  "discoveredTests": null,
  "accepted": false
}
```

Chapter 24 code-audit result

The chapter now exposes one executable path from frozen artifact identity and closed pair topology through isolated old/new consumers, production-path HTTP, exact wire/problem assertions, deterministic lost-response replay and a fail-closed evidence verdict.

CURRENT VERDICT

The compatibility boundary is implementable in shape, but no client package, provider build, database race, proxy lane, fault scenario or signed evidence index has executed. Every COMP result remains explicitly PENDING.

Implementation sequence

1. Accept ADR-022 and name owners for public contracts, client archives, provider fixtures and evidence retention.
2. Bundle and hash the v1 OpenAPI description, Draft 2020-12 schemas, problem catalogue and wire corpus.
3. Pin generators and build isolated archived/current clients; prove the control C-old -> P-old pair.
4. Implement the candidate WebApplicationFactory host and execute route/media/status/header/body/problem fixtures.
5. Add static diff classification and instance corpora without allowing either to waive behavioral failure.
6. Provision the production database engine, real migrations, authoritative idempotency claim and TX-A count oracle.
7. Add deterministic response-drop, rollback, process-kill, concurrency, clock and restore hooks.
8. Execute the complete request-am-20482 lost-202 scenario and keep case-rc-3107 outside replay.
9. Run the full old/new matrix, security/resource corpus and deployable proxy/container lane.
10. Publish COMP-ART-001 through COMP-REVIEW-001 with immutable identities and explicit residual risk.

What this chapter has proved

Chapter 24 has specified how NEXUS-1 will decide contract compatibility through executable producer/consumer evidence rather than a schema diff alone. It has defined immutable artifacts, four old/new pairs, structural/wire/protocol/semantic/temporal layers, change classification, schema and wire corpora, archived generated clients, provider lanes, Problem Details compatibility, stateful idempotency fault injection, CI gates, evidence IDs and ADR-022.

It has also traced one complete lost-response scenario from archived Client v1 through AlarmManagement TX-A, stored 202 replay and later independent RootCause TX-B. That walkthrough reduces conceptual complexity without merging ownership boundaries or claiming execution. No runtime evidence has been produced; RootCause remains G0 accepted and G1 specified.

A CONTRACT PASSES ONLY AS A PAIR

A provider is not “backward compatible” in isolation. The claim is meaningful only for a named consumer artifact, provider artifact, exchange corpus and expected behavior, all bound to one reproducible evidence record.

Chapter checkpoint

Question	Expected answer
Is a schema diff the compatibility verdict?	No. It classifies risk; executable producer/consumer behavior decides.
Which pair protects deployed consumers?	Archived client against candidate provider: C-old -> P-new.
Why test C-new -> P-old?	To prove forward tolerance and permitted rollout order/fallback.
What does a wire fixture add?	Status, headers, media, serialization and stable problem identity.
May unknown enum map to the first/default member?	No. It needs an explicit safe unknown branch.
Can TestServer prove database uniqueness?	No. Use the production database engine and real transactions.
What makes lost-response evidence valid?	TX-A commit is known, response is dropped, retry replays stored outcome, counts remain one.
Does replay contain case-rc-3107?	No. RootCause observes fact-am-7716 later in independent TX-B.
What must every result identify?	Provider, client, schema, corpus, environment and run digests/versions.
Has this suite executed?	No. The design is specified; implementation and runtime evidence are required.

NEXT CHAPTER

Chapter 25, RabbitMQ Topology for NEXUS-1, will turn the conceptual message flow into explicit exchanges, queues, routing keys, durability, acknowledgement rules, dead-letter routes, a topology diagram, and executable declarations.

PART V

PART V

The Message Backbone

*A committed local fact must survive broker absence.
A delivered message may repeat. Every guarantee must name its boundary.*

Part V converts the conceptual outbox/inbox path into a real RabbitMQ backbone. It begins with topology, then adds producer-local persistence, confirmed dispatch, consumer-local idempotency, bounded failure handling, and explicit treatment of ordering, duplication, and contract evolution.

PART V CONTRACT

The target is not exactly-once delivery. The target is durable intent, at-least-once publication where proved, idempotent effects, visible partial progress, bounded failure, and evidence that states precisely which boundary was exercised.

CHAPTERS 25-30

- RabbitMQ Topology for NEXUS-1
- The Transactional Outbox in Each Producer
- Dispatch, Publisher Confirms, and Recovery
- Inbox and Idempotent Consumers
- Retry, Backoff, Dead Letters, and Poison Messages
- Ordering, Duplication, and Contract Evolution